GnssProcessor: Mathematical Reference

Revision date: October 30, 2026

Contents

1 Introduction **7**

I Foundations **9**

2 Notation and conventions **9**

3 State vector layout **11**
3.1 Schematic layout . . . 12
3.2 Component-by-component description . . . 12
3.3 Declared state families . . . 15
3.4 Dynamic indexing . . . 15
3.5 Network station parameters . . . 17

4 Observation equations **19**
4.1 Code (pseudorange) observation . . . 19
4.2 Carrier-phase observation . . . 19
4.3 Geometric range and linearisation . . . 20
4.4 One equation for every parameter set . . . 21
4.5 Stochastic model . . . 21

5 Differencing transformations **23**
5.1 Between-satellite single difference (BSSD) . . . 23
5.2 Between-receiver single difference (BRSD) . . . 26
5.3 Double difference (DD) . . . 26
5.4 S-transform structure . . . 26

II Reference systems and measurement models **27**

6 Coordinate Frames and Time Systems **27**
6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames . . . 28
6.2 Station reference points: marker, ARP, and phase centre . . . 29
6.3 GNSS time scales and the GPS reference epoch . . . 30
6.4 Leap seconds and per-system dispatch . . . 31
6.5 GLONASS time-of-day to full GPS week . . . 33
6.6 BeiDou time: the 14 s offset and independent week base . . . 33

6.7 Legacy ECI→ECEF: the equinox-based chain . . . 34
6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006) . . . 35
6.9 Earth-orientation parameters: parse and interpolation . . . 35
6.10 The Sun and Moon series these transforms position . . . 36

7 Terrestrial Reference Frames, Epochs, and Datum Transformation 37
7.1 Frame versus epoch . . . 37
7.2 Transforming a velocity . . . 38
7.3 The rigid-plate approximation . . . 38
7.4 Rotation-sign conventions . . . 39
7.5 Published parameter sets and their composition . . . 39
7.6 NATRF2022 . . . 40
7.7 SIF occupations and the datasheet convention . . . 40
7.8 Monument versus ARP . . . 41
7.9 Station coordinates in the engine . . . 42
7.10 Worked example: CORV . . . 43
7.11 Deformation models and validation . . . 44

8 Orbit and Clock Product Evaluation 46
8.1 Precise orbit: derotated Neville interpolation . . . 46
8.2 Precise orbit: sidereal-trigonometric LSQ fit . . . 48
8.3 Precise clock interpolation . . . 48
8.4 Broadcast Keplerian evaluation . . . 50
8.5 Broadcast GLONASS: PZ-90 RK4 integration . . . 51
8.6 Broadcast satellite clock . . . 51
8.7 Ephemeris selection and IOD bookkeeping . . . 52

9 Atmospheric Delay Modelling: Troposphere and Ionosphere 53
9.1 The slant tropospheric delay . . . 54
9.1.1 Saastamoinen zenith hydrostatic delay . . . 54
9.1.2 The Marini mapping form . . . 54
9.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients . . . 55
9.1.4 The gradient mapping and a degenerate fallback . . . 56
9.2 The slant ionospheric delay . . . 56
9.2.1 The $1/f^2$ scaling and its sign . . . 57
9.2.2 IONEX / GIM thin-shell slant delay . . . 57
9.2.3 The GIM iono-weighting pseudo-observation . . . 58
9.2.4 The co-estimated spherical-harmonic VTEC field . . . 58
9.2.5 Klobuchar broadcast seed . . . 61
9.3 Higher-order ionospheric corrections . . . 62
9.3.1 The geomagnetic dipole field . . . 62

10 Site Displacement and Geophysical Corrections 63
10.1 Sign convention and projection into the range . . . 64
10.2 Solid Earth tide (IERS 2010, Steps 1 + 2) . . . 64
10.2.1 Step 1: in-phase degree 2 and 3 . . . 64
10.2.2 Step 1: out-of-phase anelasticity and the L^1 term . . . 65
10.2.3 Step 2: frequency dependence (Doodson tables) . . . 65

10.3 Ocean tidal loading (OTL) . . . 66
10.3.1 Astronomical argument and the Doodson–Warburg phase bias . . . 66
10.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid . . . 67
10.3.3 Nodal corrections and the UTC Greenwich argument . . . 68
10.3.4 The net translation of the diurnal loading . . . 68
10.4 Pole tide (rotational deformation, IERS 7.1.4) . . . 69
10.5 Ocean pole tide (Desai 2002, IERS 7.1.5) . . . 69
10.6 Atmospheric pressure loading (S1/S2 thermal tide) . . . 70
10.7 Integration into the measurement model . . . 71

11 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections 71
11.1 Satellite attitude: the body frame . . . 72
11.2 Antenna phase-centre corrections . . . 74
11.3 Carrier-phase wind-up . . . 76
11.4 Periodic relativistic clock correction . . . 78
11.5 Shapiro (gravitational-delay) correction . . . 78
11.6 Sagnac (Earth-rotation) correction . . . 78
11.7 Group delays and inter-frequency biases . . . 79
11.8 Provenance and assembly . . . 80

III Estimation engine 81

12 Estimation engine: the Kalman filter and RTS smoother 81
12.1 Time update . . . 81
12.2 Measurement update . . . 83
12.3 Iterated relinearisation (IEKF) . . . 84
12.4 Innovation and residual gating . . . 85
12.5 Backward RTS smoother . . . 87

13 Stochastic Model and Quality Control 89
13.1 The stochastic-model dispatch . . . 89
13.2 A-posteriori variance factor . . . 91
13.3 Redundancy numbers, the Baarda w-test, and the MDB . . . 92
13.4 Data-snooping gate override . . . 94
13.5 Correlation-aware full-$\boldsymbol{R}$ kernel . . . 95
13.6 Block Baarda test on a line of sight . . . 95
13.7 Variance components of the carried priors . . . 96
13.8 Measure-only instruments of the network . . . 97
13.9 Network RAIM: within-epoch adaptation, withhold and post-fit tiers . . . 98
13.10 Pooled variance factors and the per-constellation carve-out . . . 100
13.11 Emission-time gates . . . 100

14 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management 101
14.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed . . . 102
14.2 First-epoch state seeding by residual differencing . . . 103

14.3 BSSD pivot management . . . 106
14.3.1 Lexicographic candidate ranking . . . 106
14.3.2 Ephemeris-freshness guardrail . . . 107
14.3.3 Pivot disturbance detection and handover . . . 107

IV Cycle-slip detection and repair **107**

15 Cycle-slip detection and repair **108**
15.1 Detector registry and composition . . . 108
15.2 Test-statistic combinations . . . 109
15.2.1 Melbourne–Wübbena (wide-lane) . . . 109
15.2.2 Extra-wide-lane (EWL) . . . 111
15.2.3 GRAPHIC (single-frequency half-sum) . . . 111
15.2.4 Geometry-free phase (staged) . . . 112
15.2.5 Ionosphere-free phase (staged on the user path; armed on the network path) 113
15.3 Network slip QC: raw evidence and refloat over repair . . . 115
15.3.1 The slip funnel . . . 118
15.3.2 Arc-scoped geometry-free jump test . . . 118
15.4 TurboEdit dual-frequency resolution . . . 119
15.4.1 The 2×2 system . . . 119
15.4.2 Integer rounding and validation . . . 120
15.4.3 MW-only / code-event null line . . . 121
15.4.4 Gap-spanning GF bridge . . . 121
15.4.5 Atomic reconciliation . . . 122
15.4.6 Soft-LLI hold . . . 122
15.4.7 Non-causal two-sided bridge (staged) . . . 123
15.5 Response on the user path . . . 123

V Integer ambiguity resolution **124**

16 Integer ambiguity resolution **124**
16.1 The integer least-squares problem . . . 124
16.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform . . . 125
16.3 Wide-lane / narrow-lane decomposition (PPP path) . . . 126
16.4 Partial / subset ambiguity resolution . . . 128
16.5 Validation: ratio test and fixed-failure-rate gate . . . 129
16.6 Feedback: constrained pseudo-observation update . . . 130
16.7 Network integer machinery (all default off) . . . 131
16.7.1 The observation graph and its bases . . . 132
16.7.2 Ledger of fixed cycles . . . 133
16.7.3 Fixed cycles folded into a detached held solve . . . 134
16.8 Configuration summary . . . 136

VI Inertial aiding **137**

17 Inertial aiding **137**
17.1 Strapdown mechanization . . . 138
17.2 INS error-state dynamics . . . 139
17.3 Covariance splice and attitude feedback . . . 140
17.4 Coupling modes and the GNSS update . . . 140

VII Real-time SSR corrections **141**

18 Real-time SSR corrections **141**
18.1 Orbit correction: refines ρ_r^i . . . 142
18.2 Clock correction: refines $-c\,\mathrm{d}t^i$ . . . 142
18.3 IOD gate at correction-set boundaries . . . 143
18.4 Satellite biases: refine $-B_s^i$ and $-b_s^i$ . . . 144
18.5 Satellite-bias state gate and SSR code biases . . . 145
18.6 Phase-bias steps and the user's ambiguities . . . 146
18.7 GLONASS . . . 147
18.8 Ionosphere: $\mu_s I_{r,1}^i$ off the stream, via the slant channel . . . 148
18.9 The user of the network's corrections . . . 149
18.10 Provenance and code pointers . . . 149

VIII Real-time engine **150**

19 Real-time engine **150**
19.1 Streaming control flow . . . 151
19.2 NTRIP and TCP transport . . . 151
19.3 RTCM 3 framing and CRC . . . 152
19.4 MSM observation decode . . . 153
19.5 Broadcast nav, SSR and station coordinates . . . 154
19.6 Time-system handling . . . 155
19.7 Observation/SSR synchronisation . . . 156
19.8 Ephemeris management . . . 156
19.9 A-priori seeding from RTCM 1005/1006 . . . 157
19.10 The per-epoch real-time update . . . 158
19.11 Pre-update innovation screen . . . 160
19.12 Ambiguity-level check . . . 162
19.13 Broadcast-baseline jump response . . . 162
19.14 Filter-state snapshot and warm resume . . . 163
19.15 Assessing a real-time run . . . 164

IX Network mode **165**

20 Network mode **165**
20.1 Architecture: the decomposed epoch loop . . . 166
20.2 Joint multi-station observation model . . . 170
20.3 The explicit satellite phase-bias state . . . 172
20.4 Rank deficiency and the minimum-constraint S-basis . . . 173

20.5 The clock datum and the pivot election . . . 174
20.6 The phase datum . . . 176
20.7 The satellite code-bias datum . . . 176
20.8 The rank gate and the S-basis completion . . . 178
20.9 Per-station Schur reduction (arrowhead normals) . . . 178
20.10 Constellation scope . . . 180
20.11 The emitted product and the test station . . . 182

21 Worked example: estimability in a two-receiver network 183
21.1 Code only: clocks and one gauge freedom . . . 184
21.2 Adding carrier phase: the satellite phase bias needs the network . . . 185

22 Between-epoch carry: arrowhead admissibility and the DD cycle basis 187
22.1 The carried families and how they are folded . . . 188
22.2 Three doors into an arrowhead system . . . 189
22.3 The observation graph and its cycle space . . . 191
22.4 Folding the carried double differences . . . 193
22.5 Worked example: the smallest network with a cycle . . . 195
22.6 Where each object lives . . . 199

23 The float ambiguity vehicle 199
23.1 The armed posture . . . 200
23.2 The carried local column . . . 200
23.3 The joint local block . . . 202
23.4 Birth prior and the bounded drop . . . 203
23.5 The held S-basis . . . 204
23.6 The coupling time update . . . 205
23.7 Status . . . 207

24 Event responses and the emitted product 208
24.1 The event doctrine . . . 208
24.2 One slip frees an arc once: the MW re-flag skip . . . 209
24.3 Gap re-entry, keyed per signal . . . 209
24.4 Re-admission after a long absence . . . 211
24.5 Carrier-phase wind-up: accumulation and its restart . . . 212
24.6 The gross line-of-sight screen . . . 213
24.7 The emitted clock . . . 214
24.8 The emitted phase bias . . . 216
24.9 The product contract . . . 217
24.10 The user's answer . . . 218
24.11 Status . . . 219

X Reference 220

25 Pipeline and architecture diagrams 220

A Generated reference tables **233**
A.1 Processing modes . . . 233
A.2 Execution mode and data source . . . 233
A.3 Differencing regime . . . 234
A.4 Config-to-differencing mapping . . . 234
A.5 Feature toggles . . . 234
A.6 Model-correction switches . . . 246
A.7 Logging and trace channels . . . 250
A.8 Cycle-slip detector registry . . . 251
A.9 State-vector key schema . . . 252

B Supplement: linear algebra in GNSS **253**
B.1 The model used throughout . . . 253
B.2 Norms . . . 254
B.3 Hermitian matrices . . . 258
B.4 The singular value decomposition . . . 259
B.5 The pseudoinverse . . . 261
B.6 Exercises . . . 263
B.7 Reproducing the numbers and further reading . . . 264

1 Introduction

This document is the mathematical companion to `GnssProcessor`, a C++17 post-processing and real-time GNSS engine implementing PPP, PPP-RTK (single-receiver and network), and double-differenced baseline modes. Its scope is the estimation mathematics and design rationale: the material that ages slowly, because it is settled theory or algebra. This revision describes the source tree of 30 October 2026 (commit `5127429`).

Two configuration files fix what the network runs with, and the text calls them the *reference configurations*: `config/golden/golden_field_refound.json` (real data, 58 stations) and `config/golden/golden_sim_refound.json` (a simulated network of 60 stations). Both use GPS L1/L2 and Galileo E1/E5a. A setting described as default off is off in the loader; the verdict on every network setting is recorded in `docs/CONFIG_promotion_ledger.json`.

The reference is organised as a set of *component documents*, each of which also compiles to a standalone PDF. This combined volume gathers them in roughly the order the engine applies them:

- **Foundations** (Sections 2 to 5): notation; the variable-dimension state vector, its declared state families and the network's station parameter set; the *uncombined* pseudorange / carrier-phase observation model, written once and shared by the user filter, the network estimator and the state seeds; and the between-satellite single-difference (BSSD) and double-difference (DD) transformations. Every other component builds on this.

- **Reference systems and measurement models** (Sections 6 to 11): the deterministic substrate and physics behind the observation model the Foundations document writes abstractly: the coordinate frames and time systems every other chapter assumes, together with the terrestrial reference frames, datum transformations, and station-coordinate epoch propagation that relate them to national frames and published coordinates; the baseline satellite orbit/clock product evaluation that SSR refines; tropospheric and ionospheric delay modelling; the geophysical site

displacements (solid Earth, ocean, and pole tides) that correct the receiver coordinate; and the antenna, attitude, relativity, and range corrections that map the raw measurement to the modelled observable.

- **Estimation engine** (Sections 12 to 14): the variable-dimension Kalman filter (Joseph-form update, prior-anchored iterated relinearisation, robust gating) and the backward RTS smoother; the measurement stochastic model and the a-posteriori quality-control layer (variance factor, Baarda data-snooping, minimal detectable bias, the block test on a line of sight, variance components of the carried priors, and the network's RAIM tiers); and the supporting infrastructure that bootstraps the filter: the weighted least-squares seed, the first-epoch residual-difference state initialisation, and the pivot selection / disturbance logic.

- **Cycle-slip detection and repair** (Section 15): the linear-combination test statistics (Melbourne–Wübbena, geometry-free, GRAPHIC, extra-wide-lane, ionosphere-free time difference), the network's slip layer and its slip funnel, and the TurboEdit 2×2 integer repair.

- **Integer ambiguity resolution** (Section 16): the MLAMBDA integer least-squares search, wide-lane / narrow-lane decomposition, validation, constrained feedback and fix-and-hold, and the network's integer machinery (the observation graph and its bases, the block search, the fixed-cycle fold), all default off.

- **Inertial aiding** (Section 17): the strapdown mechanisation and 15-state INS error model of the tightly-coupled IMU path.

- **Real-time SSR corrections** (Section 18): how State Space Representation orbit, clock, and bias corrections refine the undifferenced observation model, and how the user's filter answers a step in the network's phase biases.

- **Real-time engine** (Section 19): the streaming architecture: NTRIP and TCP transport, RTCM/MSM decoding, observation/SSR time synchronisation, the per-epoch real-time update path, the pre-update innovation screen, the broadcast-baseline jump response and the filter-state snapshot.

- **Network mode** (Section 20): the multi-station PPP-RTK network estimator that *generates* the SSR-style corrections consumed by the single-receiver modes: the architecture of the per-epoch loop, the network's parameter set and joint observation model, the S-basis datum and its clock, phase and satellite code-bias parts, the rank gate, the per-station Schur reduction with exact sparse station normals, and the constellation scope.

- **Between-epoch carry** (Section 22): how information reaches that arrowhead system from outside the current epoch: the carried families and how each is folded, the three folds an arrowhead admits and where each must be applied, the double-difference cycle basis (the spanning tree's *chords*) that is the only gauge-free quantity worth time-linking, the Woodbury algebra that folds it, and a fully numerical minimum-parameter example.

- **The float ambiguity vehicle** (Section 23): the carry the reference configurations use: the carried local column and its joint local block, the floored birth prior and the bounded drop, the held phase-datum basis, and the exact conditional time update of the coupling rows with the temporal clock anchor.

- **Event responses and the emitted product** (Section 24): how the network answers slips (one slip frees a carried arc once), gap re-entry, re-admission, wind-up restarts and gross line-of-sight faults on its carried state, and what it emits: the clock with its step test, the phase-bias generation hold and its phase-clock mode, the product contract, and the user's answer to a held bias's presence edges.
- **Pipeline and architecture diagrams** (Section 25): the per-mode data-flow figures (user PPP, real time, the network's run, epoch, solve stages, emission and harvest) that place every chapter above in its loop.
- **Generated reference tables** (Section A, appendix): processing modes, feature toggles, the cycle-slip detector registry, and the state-vector key schema, produced directly from the source tree by `scripts/gen_report_tables.py` rather than maintained by hand.
- **Linear algebra supplement** (Section B, appendix): study material that ties norms, Hermitian matrices, the singular value decomposition and the pseudoinverse to computations in the engine through thirteen numerical examples, every number printed by `scripts/linalg_supplement_examples.py`.

Implementation details (function signatures, class responsibilities, config field reference) live in the headers under `include/` and are rendered via Doxygen. When this document references a specific algorithmic component, it cites the responsible *symbol* (a function, class, or JSON config key) together with its source file (e.g. `KalmanFilter::update` in `src/core/filter/KalmanFilter.cpp`, or `network.cold_clock_seed`). Line numbers are deliberately never cited: they rot within weeks, symbol names do not.

The notation follows the IGS conventions adopted by Subirana, Zornoza, and Hernández-Pajares [34] and Teunissen and Montenbruck [39]; see Section 2 for the symbol table this document depends on.

Part I
Foundations

2 Notation and conventions

Indices and sets. Indices are lower-case Latin letters; sets are calligraphic upper-case:

- $r \in \mathcal{R}$ is the receiver. Single-receiver modes (PPP, PPP-RTK) have $|\mathcal{R}| = 1$. Baseline modes have a fixed reference and one or more rovers. The network estimator (`PPP_RTK_Network`) processes a peer set of $|\mathcal{R}| \geq 2$ stations (50–305 in practice) with no reference/rover asymmetry.
- $g \in \mathcal{G} = \{\text{GPS}, \text{GAL}, \text{BDS}, \text{GLO}, \text{QZS}, \dots\}$ is the constellation.
- $i \in \mathcal{S}_g$ is a satellite of constellation g. p_g denotes the pivot satellite of constellation g under the BSSD parameterisation (Section 5).
- $s \in \mathcal{B}$ is the signal/observation type, identified by its RINEX-3 three-character code (e.g. `C1C`, `L2W`, `L5X`).
- $k \in \mathbb{N}$ is the discrete epoch index.

Vectors and matrices. Vectors are bold lower-case ($\boldsymbol{x}$, $\boldsymbol{r}$); matrices are bold upper-case ($\boldsymbol{H}$, $\boldsymbol{P}$). $\boldsymbol{a}^{\mathrm{T}}$ is the transpose. A hat denotes a state estimate and a tilde a one-step prediction: $\hat{\boldsymbol{x}}_{k|k}$ and $\tilde{\boldsymbol{x}}_{k|k-1}$.

Observables. Each receiver/satellite/signal triple supplies two real-valued observables per epoch, both in metres:

- $P^i_{r,s}$ is the pseudorange (code), in metres.
- $\Phi^i_{r,s}$ is the carrier phase, scaled to metres by λ_s. In addition to the range, clock and atmosphere terms it shares with the code, it carries an ambiguous (non-range) part: $\lambda_s N^i_{r,s}$ plus the fractional phase biases $b_{r,s} - b^i_s$ (Equation (5)).

Where the distinction matters, s also denotes the band ($L_1, L_2, L_5, \ldots$) and the tracking attribute (e.g. `C`, `W`, `X`). The leading observable-type letter (`C`/`L`) selects code or phase. The frequency-only quantities f_s, λ_s and $N^i_{r,s}$ depend only on the (band, attribute) part, so a code signal and its sibling phase share them. The state keys follow the same split: ambiguity keys carry the full phase code (`"G01_L1C"`), receiver code-bias keys only the band digit (`"GPS_IFB2"`).

Frequencies and wavelengths. f_s is the carrier frequency (Hz) and $\lambda_s = c/f_s$ the wavelength (m). The frequency-squared factor that maps the L1-equivalent slant ionospheric delay onto signal s is

$$\mu_s \;:=\; \frac{f_1^2}{f_s^2}. \tag{1}$$

Here f_1 is the GPS L1 frequency (1575.42 MHz), the common reference for all constellations; it is not the band-1 frequency of the satellite's own system. The estimated state $I^i_{r,1}$ is therefore GPS-L1-equivalent for BDS, GLO and GAL satellites as well (the slant-ionosphere term of `measurement/ObservationEquation.cpp`, `addIonosphere`: `f_ref = Constants::GPS_L1_FREQ`).

Differencing operators. Three differencing operators recur throughout:

$$\Delta q^{i,p_g}_{r,s} := q^i_{r,s} - q^{p_g}_{r,s}, \quad \nabla q^i_{r_1 r_2,s} := q^i_{r_1,s} - q^i_{r_2,s}, \quad \nabla\Delta q^{i,p_g}_{r_1 r_2,s} := \Delta\big(\nabla q^i_{r_1 r_2,s}\big).$$

Δq is the between-satellite single difference of q relative to the pivot p_g, ∇q the between-receiver single difference, and $\nabla\Delta q$ the double difference.

Sign conventions. The slant ionospheric delay $I^i_{r,s}$ enters the code observable with a positive sign and the carrier phase observable with a negative sign; this sign difference produces code–phase divergence. All other range-additive terms (geometry, clocks, troposphere, biases) carry the same sign on both observables. Equations (4) and (5) give the explicit equations. In the implementation the satellite bias parameters (code and phase) enter each row with the partial $+1$ and the satellite clock correction with the partial -1, one constant each shared by every code that builds the row (Section 4.4); a bias parameter therefore carries the sign of the term as it appears in the observation, $-B^i_s$ or $-b^i_s$ of Equations (4) and (5).

Symbol table.

Symbol	Units	Meaning
$\boldsymbol{r}_r, \boldsymbol{r}^i$	m	ECEF position of receiver/satellite
ρ_r^i	m	geometric range $\|\boldsymbol{r}^i - \boldsymbol{r}_r\|$
$\boldsymbol{e}_r^i$	none	unit LOS from r to i in ECEF
$c\,\mathrm{d}t_r$	m	receiver clock offset (scaled by c)
$c\,\mathrm{d}t^i$	m	satellite clock offset (CLK product in user modes; the estimated correction `"<sat>|SC"` in `PPP_RTK_Network`)
$c\,\delta t_r^{(g)}$	m	receiver inter-system time offset for constellation $g \neq g_{\mathrm{ref}}$
T_r^i	m	total tropospheric path delay
$\mathrm{ZHD}_r^i, \mathrm{ZWD}_r^i$	m	zenith hydrostatic / wet components
$I_{r,1}^i$	m	L1-equivalent slant ionospheric delay
$N_{r,s}^i$	cyc	integer carrier-phase ambiguity
$b_{r,s}, b_s^i$	m	receiver / satellite phase biases
$\bar{b}_s^i$	m	estimable satellite phase bias (network mode; Section 3.5)
$B_{r,s}, B_s^i$	m	receiver / satellite code biases (OSB)
$B_{r,k}^{(g,b)}$	m	per-FCN-slot GLONASS receiver IFB, band b; here $k \in [-7, +6]$ is the signed FDMA channel number (a context-local reuse, not the epoch index)
$\varepsilon^P, \varepsilon^\Phi$	m	residual (multipath + thermal)
$\boldsymbol{v}_r$	m/s	receiver velocity (optional / IMU state)
$\lambda_{\mathrm{WL}},\ \lambda_{\mathrm{NL}},\ \lambda_{\mathrm{EWL}}$	m	wide- / narrow- / extra-wide-lane wavelength $c/(f_a \mp f_b)$ for the lane's band pair
λ_{IF}	m	effective ionosphere-free wavelength $\|c_1\lambda_1 - c_2\lambda_2\|$ with the IF coefficients $c_1 - c_2 = 1$; not an integer wavelength (Section 15)
$\boldsymbol{C}_b^e$	none	body-to-ECEF attitude (direction-cosine) matrix

3 State vector layout

The Kalman filter operates on a state vector $\boldsymbol{x}_k \in \mathbb{R}^{n_k}$ whose dimension n_k varies across epochs. A per-satellite or per-(satellite, signal) state is appended when its satellite or signal is first required, and removed when it is absent from the epoch's measurement set (for example, after the satellite sets below the elevation mask). A cycle slip does not remove the ambiguity state; it re-seeds it (see the ambiguity paragraph below). `StateManager` holds all bookkeeping: the key→index maps, the appending of rows, and the compaction of $\boldsymbol{x}$ and $\boldsymbol{P}$ on removal, which shifts every later index down by one (there is no free list). `KalmanFilter` reads the state through that interface and never indexes it by hard-coded offsets. The dynamic part of the vector is a set of families, each with one key→index map, declared once in a table (Section 3.3).

This section describes the single-receiver layout (PPP, PPP-RTK and the baseline modes). The network estimator keeps no `StateManager`: each of its stations owns a parameter set of its own, described in Section 3.5.

3.1 Schematic layout

In schematic form, suppressing the epoch index k:

$$\begin{aligned}\boldsymbol{x} \;=\; \big[\; & \underbrace{\boldsymbol{r}_r^{\mathrm{T}}}_{3},\ \underbrace{\boldsymbol{v}_r^{\mathrm{T}}}_{0\text{ or }3},\ \underbrace{c\,\mathrm{d}t_r}_{1},\ \underbrace{(c\,\delta t_r^{(g)})^{\mathrm{T}}_{g\neq g_{\mathrm{ref}}}}_{|\mathcal{G}|-1},\\ & \underbrace{\mathrm{ZWD}_r,\ (g_N, g_E)}_{1+0\text{ or }1+2},\ \underbrace{(c\,\mathrm{d}t_r^{\phi})}_{0\text{ or }1},\\ & \underbrace{(I^i_{r,1})^{\mathrm{T}}_{i\in\mathcal{V}_k}}_{|\mathcal{V}_k|},\ \underbrace{(N^i_{r,s})^{\mathrm{T}}_{(i,s)\in\mathcal{A}_k}}_{|\mathcal{A}_k|},\\ & \underbrace{(B_{r,s})^{\mathrm{T}}_{s\in\mathcal{B}}}_{|\mathcal{B}|},\ \underbrace{(B^i_s)^{\mathrm{T}}_{(i,s)\in\mathcal{D}_k}}_{|\mathcal{D}_k|}\;\big]^{\mathrm{T}}. \end{aligned} \tag{2}$$

where

- $\mathcal{V}_k$ is the set of satellites visible at epoch k (above the elevation mask, with valid orbit and clock products and at least one tracked signal);
- $\mathcal{A}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the set of phase-tracked (satellite, signal) pairs;
- $\mathcal{D}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the subset of (satellite, signal) pairs that carry a satellite code-bias state: the pairs for which the BIA product lacks a satellite OSB (`estimation.estimate_sat_biases`).

The schematic omits several conditional refinements, detailed below: the rate states that accompany velocity (a receiver clock rate and one inter-system clock-rate offset per non-reference constellation, both absent under BSSD); the optional per-satellite slant-ionosphere rate under `enable_iono_rate`; the receiver inter-signal code biases; the nine-state inertial augmentation of Section 17; and the removal of the whole receiver-side clock and inter-system-offset block under the BSSD parameterisation (Section 5.1).

The schematic groups the states by physical role; it is not the storage order. `StateManager` places a fixed core first (`StateManagerLayout.cpp`), in the order position, receiver clock (absent under BSSD), ZWD, velocity, receiver clock rate, phase clock, the two troposphere gradients, the nine inertial states, the inter-system time offsets and the inter-system clock-rate offsets, each present only when enabled. Every dynamic state (ionosphere, ionosphere rate, ambiguity, receiver code bias, satellite code bias) is appended after the core in order of creation, so the categories interleave; each category is addressed only through its key map, as described under dynamic indexing below.

3.2 Component-by-component description

Position $\boldsymbol{r}_r \in \mathbb{R}^3$. ECEF Cartesian position of the antenna reference point. Always present. `Application::initializePosition` seeds it from the RINEX header approximate position, shifted from the marker to the antenna reference point by the header's antenna H/E/N offset (or from the Station Information File coordinate when that a-priori source is enabled). The seed must lie within the linearisation radius of the geometric range model (Section 4.3). The per-epoch Weighted Least-Squares (WLS) solution seeds the receiver clock (on the first epoch, after a clock jump and after a data gap); it re-seeds the position only in the kinematic gap-recovery branch of `Application::handleGap`.

Velocity $\boldsymbol{v}_r \in \mathbb{R}^3$. Optional. Enabled by `estimation.estimate_velocity = true`; tightly-coupled IMU integration forces it on. With velocity present, position and velocity form a coupled continuous-white-noise-acceleration (CWNA, integrated-random-walk) block driven by a single acceleration spectral density, `process_noise.acceleration_psd_q` (Equation (165)); the scalar `position_q` of the velocity-free case is then unused. When BSSD is off, velocity estimation also adds a receiver clock-rate state $c\,\dot{\mathrm{d}}t_r$ and one inter-system clock-rate offset (ISBR) per non-reference constellation, each CWNA-coupled to its bias counterpart (`clock_rate_q`, `system_time_rate_offset_q`). Under BSSD both rate states are omitted, because same-system single differencing cancels them.

Receiver clock $c\,\mathrm{d}t_r$. Single state, in metres. Its process noise is large (`clock_bias_q` $\sim 10^4\,\mathrm{m}^2$ per second), so the state is in effect re-estimated each epoch. This follows oscillator drift without imposing a stochastic model that may not match the receiver's oscillator. Under the BSSD (between-satellite single-difference) parameterisation (Section 5.1) the receiver clock is unobservable and the state is not allocated; neither are the inter-system offsets below, so PPP-RTK SD mode has no receiver-side clock block.

Inter-system time offsets $c\,\delta t_r^{(g)}$. One per active constellation other than GPS, which is the fixed reference in `StateManager`. These absorb the per-constellation clock biases that arise from mixing analysis-centre clock products across systems ("GAL − GPS time offset," etc.).

Tropospheric ZWD and gradients. ZWD_r is the zenith wet delay. The zenith hydrostatic delay ZHD_r is deterministic, from the Saastamoinen model [30]. The total slant tropospheric delay is

$$T_r^i \;=\; m_h(\theta_r^i)\,\mathrm{ZHD}_r \;+\; m_w(\theta_r^i)\,\mathrm{ZWD}_r \;+\; m_g(\theta_r^i)\big[g_N\cos\alpha_r^i + g_E\sin\alpha_r^i\big], \tag{3}$$

with elevation θ_r^i, azimuth α_r^i, the mapping functions m_h, m_w (the Niell tables by default, labelled `Gmf` in the configuration, or VMF3 with GPT3 coefficients when configured) and the Chen–Herring gradient mapping $m_g(\theta) = 1/(\sin\theta\tan\theta + 0.0032)$ [5] (Section 9). The gradients (g_N, g_E) are optional.

Per-satellite slant ionosphere $I_{r,1}^i$. Uncombined processing in every mode except the baseline modes (`processing.uncombined_enabled`, forced on for the PPP-RTK family; `GnssConfig::carriesSlantIonosphere`). One state per visible satellite, expressed at the L1 frequency; signal-specific delays follow Equation (1). The seed is formed from the geometry-free combination of the satellite's BIA-covered code residuals (`seedIonosphere`, Section 14.2). With fewer than two covered codes on different bands, the seed is zero and the initial standard deviation is inflated tenfold. The initial variance, like the process noise, is scaled by the obliquity factor. The process noise is a random walk whose zenith spectral density `ionosphere_q` is projected onto the slant by the squared obliquity factor $1/\sin^2\theta_r^i$ (Equation (164)), so low-elevation states vary faster. It is applied on every run that carries the states, in PPP as in PPP-RTK; without it the state would be a constant that cannot follow a slant delay moving by metres per pass. These explicit per-satellite states let PPP-RTK keep integer ambiguity recovery without forming the ionosphere-free combination, which amplifies code noise by a factor of ~ 3. Under `estimation.enable_iono_rate` (default off), each visible satellite also carries a slant-ionosphere rate, so that the pair forms a constant-velocity block $[\,I_{r,1}^i,\ \dot{I}_{r,1}^i\,]$. The rate $\dot{I}_{r,1}^i$ has no direct observation ($H = 0$); only the F-coupling into the level state drives it. It has its own spectral density `process_noise.iono_rate_psd` and its own slip response (`reinflateIonosphereRateVariance`), and lets the filter track a linear slant ramp rather than a

pure random walk. The rate state is a satellite-specific dynamic row, not a core state: it is appended with its level state, pruned with it and treated with the other per-satellite states in gap recovery, but after prunes and re-adds it need not sit next to the level state, so it is addressed only through its own map (`iono_rate_indices_`).

Carrier-phase ambiguities $N^i_{r,s}$. One state per phase-tracked (i, s) pair, in metres (the state value is λN; the `nl_wl` basis below is the exception, carrying integers in cycles). The ambiguities are float through the prediction and update cycle; when AR is enabled, the LAMBDA-family algorithms in `filter/AmbiguityResolver` [37] resolve them to integers afterwards. A cycle slip calls `StateManager::resetAmbiguity`, which decorrelates the state rather than repairing it: the ambiguity's covariance row and column are zeroed, its diagonal is reset to σ_m^2, and the key is marked pending re-seed. The state value is left unchanged by the reset; `ensureRequiredStates` (in `MeasurementModel`) then overwrites it with a geometry-based seed (`seedAmbiguity`, Section 14.2). This is the project's reset-over-repair convention: a slipped ambiguity has no trustworthy correlation with the rest of the state, so its cross-covariances are cleared. A reset for a key that is not in the state (for example, a signal excluded by the band filter) is refused and creates no state.

Receiver code biases $B_{r,s}$. These absorb the receiver-side inter-frequency biases that the OSB convention does not reference to zero. They are not one per signal. The receiver-bias rule (`ReceiverBiasResolver::resolveReceiverBias`, `measurement/ReceiverBias.hpp`) keys them per (system, frequency band): both bands of the system's clock-reference signal pair map to one per-system differential code bias (`"<system>_DCB"`), each band with its own coefficient, and every other band gets its own inter-frequency bias (`"<system>_IFB<band>"`). Like the receiver clock they are receiver-common, so under BSSD they receive no process noise and the SD transform zeroes their columns. The same rule serves the network estimator, whose stations additionally give a second code of a band, one that is not the station's latched canonical code on that band, a receiver inter-signal code bias (`"<system>_ISC_<signal>"`, unit coefficient, GLONASS excluded; `network.second_code_channel` or `processing.channel_mode:all`). The network treatment, with its carry, is in Section 3.5.

Per-slot GLONASS inter-frequency bias $B^{(g,b)}_{r,k}$. Optional, off by default. GLONASS FDMA assigns each slot $k \in [-7, +6]$ a distinct carrier frequency $f = f_{\text{base}} + k\,\Delta f$, so the receiver group delay differs per slot and a single per-band IFB state cannot represent it. With `estimation.glonass_ifb.enable = true` and its prerequisite `estimation.estimate_ifb` (without which the loader warns and the flag is inert), the receiver-bias rule instead creates one state per (FDMA band $b \in \{\text{G1}, \text{G2}\}$, slot k) with unit measurement coefficient, keyed `"GLONASS_IFB<band>_k<±slot>"`. The central channel $k = 0$ is the held datum and has no state; the GLONASS inter-system time offset absorbs its constant. The CDMA bands (G3/G4/G6) have a slot-independent frequency; with the flag on, the GLONASS branch of the rule returns no receiver bias key for them (it handles every GLONASS signal and does not fall through to the per-band path). With the flag off, GLONASS carries no per-band IFB state on a non-reference band. Unlike the receiver-common biases, these columns survive between-satellite single differencing, because two GLONASS satellites on different slots do not cancel. `BssdTransform` and `StateManager` therefore exempt them from the receiver-block zeroing and from the process-noise suppression (via `isGlonassPerSlotIfbKey`); only the inter-slot differences are estimable in SD.

Satellite code biases B_s^i. Per (satellite, signal) pair, but only for a pair whose OSB the BIA file does not supply, typically a third-frequency signal in a triple-frequency setup (`estimation.estimate_sat_biases`). The network estimator estimates its own satellite code biases as parameters instead, covering every raw code and not using the external `.BIA` (Section 3.5). These states are weakly stochastic (`sat_bias_q` $\sim 10^{-10}$), because the underlying physical biases are nearly stationary over a day.

Inertial augmentation. With IMU integration enabled, the core state gains nine entries: a three-axis attitude error and the three-axis accelerometer and gyroscope biases. The strapdown mechanisation propagates them, rather than the random-walk process model. Section 17 treats them in full.

Baseline differential troposphere. In baseline mode the core is the position alone; double differencing cancels the receiver clocks and inter-system offsets, and on a short baseline the ionosphere and troposphere, so none of those states exists. An optional single differential zenith-wet-delay state $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}}r_{\mathrm{ref}}}$ follows the position (`baseline.estimate_differential_tropo`, Section 5.3). This is the only mode in which `StateManager` reads its `core_state_size` constructor argument; the PPP and PPP-RTK paths ignore it and derive the core layout from the configuration.

3.3 Declared state families

The dynamic families are declared once, in `include/filter/StateFamily.hpp` (`kStateFamilies`). Each row gives the family's label prefix and snapshot tag, the variance re-inflation it answers to, its gap response, the rule that names the satellite of a key (a state whose satellite is a frozen pivot gets no process noise), its process-noise rule and spectral-density key, the configuration that enables its birth and the source of its initial variance. Every place that visits all families (the state size, the index shift on removal, the state labels, the real-time snapshot) reads the table. Table 1 lists the rows. At run start `StateManager::logFamiliesAsArmed` prints one line per family: whether the configuration births it, the label and tag, the initial standard deviation with its source, the process-noise rule with its value and the events wired to it (re-inflation, reset, prune, gap decoupling, the frozen-pivot skip, the snapshot). A family that is born and has no process noise reads `Q none` in that line.

The rows with a network birth condition name parameters that a network run keeps in the station's own parameter set, not in `StateManager` (Section 3.5). The core block (position, clock, ZWD, velocity, gradients, inertial states, inter-system offsets) is index-stable by construction and is not a family; its process noise is built by `StateManagerProcessNoise.cpp`.

3.4 Dynamic indexing

`StateManager` addresses the variable-dimension blocks through key→index maps. `include/filter/StateKeySchema.hpp` defines every key format once, each with a `make` builder and, for the categories that are read back, a `parse` helper (the ionosphere and inter-system time-offset keys have no parser, since the key is the bare satellite or system name). Some call sites still build keys by string concatenation rather than through the builders (for example the double-difference keys in `MeasurementModel`, `DdPivotManager` and `KalmanFilter`); the strings they produce match the schema formats. The one function that renders a typed parameter identity as a user-side key is `userKey` (`measurement/UserParamView.hpp`). The formats are:

- iono key: `"<sys><prn>"`, e.g. `"G05"` (delimiter-free);

Table 1: The dynamic state families of `StateManager` (`kStateFamilies`). The configuration keys of the process noise are under `filter.process_noise`, those of the initial standard deviation under `filter.initial_std`.

Family	Key example	Born under	Process noise	Initial std
slant ionosphere	`G05`	`processing.uncombined_enabled`, not a baseline mode	obliquity random walk, `ionosphere_q`	`ionosphere`
ionosphere rate	`G05`	and `estimation.enable_iono_rate`	CWNA with the level, `iono_rate_psd`	`ionosphere_rate`
ambiguity	`G05_L1C`	every processed phase row	random walk, `ambiguity_q`	1000 m, no key
receiver code bias	`GPS_IFB5`	uncombined and `estimation.estimate_ifb`	random walk, `code_bias_q`	`code_bias`
satellite code bias	`G05|C1W`	`estimation.estimate_sat_biases`	random walk, `sat_bias_q`	`sat_bias`
satellite phase bias	`G05|L1C|PB`	network, `estimation.estimate_sat_phase_biases`	random walk, `sat_phase_bias_q`	`sat_phase_bias`
receiver phase bias	`GPS_RPB_L1C`	network, `network.phase_datum_eq34`	none	`sat_phase_bias`
phase colour	`G05`	`network.phase_colour_state`	none	set by the creating code
satellite clock	`G05|SC`	network, `estimation.estimate_sat_clocks`	random walk, `sat_clock_q`	`sat_clock`

- ambiguity key: `"<sys><prn>_<signal>"`, e.g. `"G05_L1C"`, split on the *first* underscore, so an iono-free combination `"G05_L1C_L5Q"` retains its compound signal;
- satellite signal-bias key (code): `"<sys><prn>|<signal>"`, e.g. `"G05|C1W"`, with a pipe delimiter and its own map, disjoint from the underscore ambiguity namespace;
- satellite phase-bias key (network): `"<sys><prn>|<signal>|PB"`, e.g. `"G05|L1C|PB"`, where the trailing `|PB` keeps it disjoint from the code-bias namespace;
- receiver phase-bias key (network): `"<system>_RPB_<signal>"`, e.g. `"Galileo_RPB_L5Q"`, where the `_RPB_` infix keeps it disjoint from the receiver code-bias namespace (which matches on `IFB`/`DCB`);
- satellite clock key (network): `"<sat>|SC"`, e.g. `"G05|SC"`, per satellite rather than per signal; the trailing `|SC` discriminator gives the clock its own namespace;
- receiver code-bias key: `"<system>_IFB<band>"` or `"<system>_DCB"`; the per-slot GLONASS variant appends an FCN marker, `"GLONASS_IFB<band>_k<±slot>"` (e.g. `"GLONASS_IFB1_k-4"`);
- receiver inter-signal code-bias key (network): `"<system>_ISC_<signal>"`, e.g. `"GPS_ISC_C1W"`, stored in the receiver code-bias map; the `_ISC_` infix matches neither the IFB nor the DCB parser;
- inter-system time-offset key: the bare system name (`"Galileo"`, ...; GPS is the reference and never appears);
- double-difference ambiguity key (baseline): `"DD|<cons>|<sat>|<code>|<pivot>"`.

Addressing states by key rather than by position removes the index bookkeeping errors of hand-managed dynamic state vectors, and lets the BSSD and DD S-transforms (Section 5) operate as block operations on the relevant key sets.

Two further categories in the schema header are absent from the list above, because they are not rows of $\boldsymbol{x}$. The first is the BSSD pivot absolute-seed lookup key (`"<cons>|<band>"`). Its builder has no production caller and it has no parser: `MeasurementModel` builds the `pivot_abs_seeds` keys by concatenation, with the full phase signal code in place of the band (`"GPS|L1C"`), plus narrow-lane and wide-lane variants under the `nl_wl` basis. The second is the spherical-harmonic VTEC coefficient key (`"SH|<n>|<m>|<C|S>"`, with the second shell on the distinct `SH2` prefix), which names a column of the network's shared border, not a `StateManager` state, and has a `make`/`parse` pair. It belongs to the optional co-estimated map that the reference configurations do not use. Both appear in Table 18, which is derived from the schema header by a script and is therefore the completeness reference for the list above.

Ambiguity basis. By default the ambiguity block carries one per-band float ambiguity per tracked (satellite, signal), in metres. Setting `estimation.state_ambiguity_basis = "nl_wl"` switches the whole run to a narrow-lane / wide-lane parameterisation. Each satellite has a reference band, the lowest-numbered band it is observed on that epoch, and carries one narrow-lane state tied to that band (key `"<sat>_NL_<band>"`) and one wide-lane state for each other observed band, paired with the reference band (`"<sat>_WL_<a>_<b>"`, with the band digits in ascending order, semantically $N_a - N_b$). A change of reference band produces new keys, and the old states are pruned. Both kinds are in cycles rather than metres, so the wide-lane and LAMBDA arithmetic stays native to integers; the Jacobian carries the wavelength scaling (Table 2). The two bases never coexist in a single state vector. The network estimator does not use this basis.

3.5 Network station parameters

The network estimator runs no `KalmanFilter` and keeps no `StateManager`. Each station owns a parameter set (`StationParams`, `network/state/StationParams.hpp`): its columns in column order, each with a typed identity (kind, form, satellite, signal), the linearisation point the epoch's rows are evaluated at, and the disposition the station split gave it. The set is the station's own state and the network its only writer. A station's layout holds the receiver position (held at the roster coordinate and never estimated), the receiver clock, the zenith wet delay and one inter-system bias per active non-reference constellation. The families born during the run are the per-satellite slant ionosphere, the ambiguities (always one per band), the receiver code biases and, when armed, the phase colour. The set also names the shared satellite parameters the station's rows reference: the satellite clock correction and the satellite phase and code biases. The optional columns of the single-receiver filter (velocity, clock rate, phase clock, gradients, inertial states, NL/WL ambiguities, between-satellite differencing) are not part of it, and a network run that enables one is warned at start. The rows of a station are built from the set by the shared observation equation (Section 4.4); the births of the local families and their first-epoch seeds are in `network/assembly/LocalBirth.cpp` (Section 14.2).

Parameters as armed. The network declares each kind of parameter once (`network/assembly/ParamRegistry.hpp`) and prints a table at run start, `network/assembly/ParamInventory.hpp`, that counts every class of parameter (kind, constellation, signal or band) by what the network did with each instance on the epoch: estimated in the shared border, estimated in the station's local block, held at its seed outside the solve, absorbed (a shared parameter that a gate withheld from the station, another column taking it up), pinned (the station coordinates), dropped, or unobserved. A declared rule whose premise does not hold on the run is printed as a warning under the table.

Time linking. The estimator solves the normal equations of an epoch. It links epochs with explicit carried priors in information form, of effective variance $\mathrm{var}_{\mathrm{eff}} = \mathrm{var} + q\,\Delta t$, not through a $\boldsymbol{Q}(\Delta t)$ (Section 20). The satellite-side densities `sat_clock_q`, `sat_phase_bias_q` and `sat_code_bias_q` are read as those carry rates. They can be set through the legacy `filter.process_noise.*` keys or through the network-scoped `network.process_noise` class table, whose per-class fallbacks equal the legacy defaults.

Satellite clocks $c\,\Delta\delta t^i$. Gated by `estimation.estimate_sat_clocks`, which the network-estimator loader forces on together with the phase-bias flag (`GnssConfigLoadNetwork.cpp`, the line that logs *forcing estimate_sat_clocks + estimate_sat_phase_biases*) and which `GnssConfig::applyNetworkConsumerCouplings` turns off again for the closed-loop user, who takes the same quantity as a known correction. The network estimates an explicit per-satellite clock correction $\Delta\delta t^i$ (in metres, relative to the a-priori precise or real-time product clock already removed from the modelled value), so the emitted clocks stay consistent with the chosen S-system datum. There is one parameter per satellite, shared by all its signals. The satellite orbit stays fixed from the product; only the clock is estimated. It enters each row with the partial -1 (Section 4.4); its identity is spelled `"<sat>|SC"`.

Satellite phase biases $\bar{b}_s^i$. Gated by `estimation.estimate_sat_phase_biases`; the JSON default is off and the network-estimator loader forces it on, as above. The user side imports the satellite phase bias as a known SSR correction (Section 18) or absorbs it into the float ambiguity. The network instead carries an explicit per-(satellite, signal) phase-bias parameter on the undifferenced phase rows: the estimable bias of Odijk [24], which lumps the network pivot-receiver ambiguity. On a single station's row it is collinear with the per-receiver float ambiguity; only the joint network estimator separates it, across stations (Section 20.3). Its identity is spelled `"<sat>|<signal>|PB"`, disjoint from the code-bias namespace. It is near-stationary between epochs, with the carry rate `sat_phase_bias_q`.

Satellite code biases. In the network's satellite-OSB mode (`network.estimate_sat_code_bias`, armed in the reference configurations) the parameter set covers every raw code (i, s), not only the pairs the BIA file leaves uncovered, and the external `.BIA` is not used, so the network's own estimate is the single code-bias source. The datum that makes it estimable is derived in Section 20. The carry rate is `sat_code_bias_q`.

Receiver code biases and inter-signal biases. The receiver code-bias rule is the one of the single-receiver filter (above). On the network, a code that is not the station's latched canonical code on its band gets the separate receiver inter-signal code bias. These columns have no Kalman time update, because the network runs no `KalmanFilter`; they are re-estimated each epoch unless `network.carry_rx_code_bias` is armed (loader default off; armed in the reference configurations). That setting adds a diagonal random-walk carry ($\mathrm{var} + q\,\Delta t$) held in the network run state. Two guards protect it: at load, the loader forces the carry off unless the code datum is time-linked (`network.code_datum_time_linked`); at inject time, an innovation gate (`rxCbInjectGate`, `network/driver/RunTypes.hpp`) withholds the fold when the carried level and the station's own estimate disagree at receiver-reset scale. With the satellite code biases estimated, the receiver inter-frequency biases of the bands outside a constellation's clock-reference pair stay in the set at the value their first-epoch seed gave them and are left out of the station's local block, unless `network.estimate_rx_extra_band_code_delay` is armed (Section 20.10).

Receiver phase biases $\delta_{r,s}$. The receiver-side counterpart of the satellite phase bias, gated by `network.phase_datum_eq34` (the single-receiver-chain phase datum; see Section 20). The setting is off in the reference configurations, which use the held phase-datum basis of the float vehicle (Section 23). When armed, each $\delta_{r,s}$ is an estimable parameter, so the receiver fractional phase bias no longer lumps into the emitted satellite phase bias. It is per (receiver, system, signal), spelled `"<system>_RPB_<signal>"`. Unlike every other bias parameter described here, these are local, each observed by a single station. At `network.phase_datum_eq34_rpb_q_m2s` $= 0$ each $\delta_{r,s}$ is re-estimated every epoch; at the non-zero default ($3 \times 10^{-6}\,\mathrm{m^2\,s^{-1}}$) a local time-linking prior toward the previous estimate, of variance $\mathrm{var} + q\,\Delta t$, suppresses the per-epoch bounce that a weakly observable local parameter otherwise injects, anti-correlated, into every individual float ambiguity. The prior is erased for any (i, s) whose chain frame stepped at that epoch's election (`network/solve/ChainDatum.hpp`).

Phase colour. With `network.phase_colour_state` a per-satellite nuisance state with the partial $+1$ on every phase row of the satellite is born at value zero; it is off in the reference configurations.

4 Observation equations

This section gives the *undifferenced* observation model about which the filter linearises at each epoch. The PPP-RTK parameterisation is uncombined: the filter consumes per-signal pseudorange and carrier phase directly and forms no ionosphere-free or wide-lane combination as a preprocessing step. The combinations (IF, MW, GF) are computed for diagnostics and slip detection only (Section 15; `slip/MwSlipDetector`, `slip/GraphicSlipDetector`) and do not enter the measurement update. The loader (`GnssConfig::loadProcessingSection_`) forces `uncombined_enabled=true` for the whole PPP-RTK family (the single-receiver PPP-RTK modes and the network estimator) and for the baseline double-difference modes, which process each signal independently (Section 5.3). Plain PPP mode keeps the flag at its default `false` and builds ionosphere-free combined observables (keys such as `C1W_C5X`); these enter the same observation equation (Section 4.4) and the update. The rest of this section describes the uncombined case.

4.1 Code (pseudorange) observation

For receiver r tracking satellite i on signal s, the pseudorange is

$$\boxed{P^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;+\; \mu_s\, I^i_{r,1} \;+\; B_{r,s} - B^i_s \;+\; \varepsilon^P_{r,i,s}.} \tag{4}$$

4.2 Carrier-phase observation

The carrier-phase observation, scaled to metres by λ_s, is

$$\boxed{\Phi^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;-\; \mu_s\, I^i_{r,1} \;+\; \lambda_s\, N^i_{r,s} \;+\; b_{r,s} - b^i_s \;+\; \varepsilon^\Phi_{r,i,s}.} \tag{5}$$

The two equations differ structurally in the sign of the ionospheric term: $+\mu_s I$ on code, $-\mu_s I$ on phase. Further remarks:

1. The phase ambiguity $N^i_{r,s}$ is an integer in principle, but the receiver and satellite phase biases $b_{r,s} - b^i_s$ contaminate it. A *float* solution absorbs these biases into $N^i_{r,s}$, which leaves a non-integer state with no structural advantage over the code observation. Integer-valued ambiguities therefore

require either external Fractional Cycle Bias (FCB) products or differencing (Section 5), which cancels the common-mode bias terms.

2. The clock product $c\,\mathrm{d}t^i$ comes from the analysis-centre CLK file (post-processing) or from the broadcast clock refined by a streamed SSR correction (real time, Section 18). The filter treats it as deterministic. Its error enters the observation: the part common to all satellites is absorbed by the receiver clock state, and the satellite-specific part remains in the residuals. The network estimator adds a per-satellite clock correction to the product clock as a parameter of its own (Section 4.4).
3. The system time offset $c\,\delta t_r^{(g(i))}$ is zero by construction when $g(i)$ is the reference constellation, which is GPS (the state manager keys no offset for GPS). With the receiver clock $c\,\mathrm{d}t_r$, a multi-constellation run therefore carries $|\mathcal{G}|$ clock-like states in total.
4. The slant tropospheric delay is the hydrostatic zenith delay mapped by the dry mapping function plus the wet zenith delay mapped by the wet mapping function (`PppObservation` fields `mf_d` and `mf_w`).
5. The equations omit the deterministic model corrections applied to the computed range: satellite and receiver antenna phase-centre offsets and variations, the relativistic terms, site displacement, and, on phase rows only, the carrier-phase wind-up. The wind-up follows the `nominal-wu93` sign convention.

4.3 Geometric range and linearisation

The geometric range $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|_2$ enters Equations (4) and (5) non-linearly through the receiver position $\boldsymbol{r}_r$. The filter linearises about the current estimate $\hat{\boldsymbol{r}}_r$:

$$\rho_r^i \;=\; \hat{\rho}_r^i \;-\; (\boldsymbol{e}_{\hat{r}}^i)^{\mathrm{T}}(\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r) \;+\; \mathcal{O}\big(\|\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r\|^2\big)\,, \tag{6}$$

with the unit line-of-sight

$$\boldsymbol{e}_{\hat{r}}^i \;:=\; \frac{\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r}{\|\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r\|_2}. \tag{7}$$

The position columns of the Jacobian row $\boldsymbol{H}_{(i,s)}$ are therefore $-(\boldsymbol{e}_{\hat{r}}^i)^{\mathrm{T}}$. One linearisation per epoch suffices for sub-metre residual targets. For large initial position offsets the engine can relinearise within the epoch (`processing.relinearize_geometry`, Section 12.3). The flag defaults to off in code; the PPP, PPP-RTK, baseline and real-time configurations in `config/` enable it, and the network reference configurations (`config/golden/golden_field_refound.json`, `config/golden/golden_sim_refound.json`) leave it off.

The satellite position $\boldsymbol{r}^i$ is a function of the transmission time. `Geometry::computeEmissionState` obtains the emission time in one step from the *pseudorange* rather than by iterating against the geometric range, which depends on $\boldsymbol{r}_r$. The time of flight $\tau = P/c$ and the satellite clock are subtracted from the receiver epoch. The receiver clock bias $\mathrm{d}t_r$ enters the receiver epoch as $+\mathrm{d}t_r$ and the pseudorange as $+c\,\mathrm{d}t_r$, so it cancels exactly. The remaining error $(I + T + \varepsilon)/c \approx 30$–$60$ ns displaces the satellite by less than a millimetre, below any PPP threshold. The seed pseudorange is the ionosphere-free combination of the clock-reference signals when available (`Geometry::extractPseudorange`), so the ionospheric part of that error applies only to the single-frequency fallback. A range-based fixed point is avoided on purpose: iterating against ρ_r^i would reintroduce an uncancelled $\mathrm{d}t_r$ and drive a positive-feedback loop through the receiver-clock state.

The one second pass is the `model_corrections.refine_emission_time` block in `Geometry::computeSatelliteGeometry` (default on). It re-runs `computeEmissionState` on a cleaned pseudorange: the seed minus the slant hydrostatic delay and the IF-combined ATX projection, plus

the periodic relativistic clock term. This refines the pseudorange; it is not a range-versus-position iteration.

4.4 One equation for every parameter set

Equations (4) and (5) are written once in code, as the list of terms that `observationEquation` (`measurement/ObservationEquation.hpp`) returns for a row. Three callers build a row through it: the single-receiver filter (`MeasurementModel`), the network estimator's station rows (`networkStationRows`, `network/assembly/StationRows.cpp`) and the first-epoch seeds (`measurement/Seeds.cpp`, Section 14.2). The user's rows and the network's rows are therefore one equation evaluated over two different parameter sets. The predicted observable of a row is

$$\hat{z}^{i}_{r,s} \;=\; z^{i,\mathrm{mod}}_{r,s} \;+\; \sum_{e} \; a_e \sum_{k} h_{e,k}\, x_{e,k}, \tag{8}$$

where z^{mod} is the a-priori model value of the signal (the range at the linearisation position, the a-priori troposphere, the product clock and the deterministic corrections of the last remark above), e runs over the terms of Table 2 in the order listed, a_e is the term's own factor (one unless stated), $h_{e,k}$ the partial of the k-th parameter of the term and $x_{e,k}$ its value at the linearisation point. The innovation of the row is $\boldsymbol{\nu} = z - \hat{z}$ and the Jacobian row holds $a_e h_{e,k}$ in the column the parameter occupies. A term whose parameter does not exist in the parameter set is absent, which is the same as adding zero; a phase row whose ambiguity does not exist has no equation.

Parameter views. The caller owns the parameters. It passes a lookup that answers, for one typed identity (kind, form, satellite, signal), the column and the linearisation value, or no column when the parameter does not exist. The single-receiver filter answers from its state vector through `UserParamView` (`measurement/UserParamView.hpp`), which maps an identity to the key formats of Section 3.4 and to the fixed core columns. The network answers from the station's own parameter set through `NetView` (Section 3.5). A shared satellite parameter (clock, phase bias, code bias) answers with its border column and the value zero, because it is absolute in the joint solve and is never written back to a station. A seed asks for fewer terms: it subtracts only those known when its state is born, selected by a mask of terms (Section 14.2). A network station always carries one ambiguity per band, whatever `estimation.state_ambiguity_basis` says.

Signs shared by the network and the user. The network estimates the satellite biases and clocks as parameters; a user of the network's corrections takes the same quantities as known terms. The two agree only if the partial of the estimated parameter and the multiplier of the known term are the same number, so one constant each serves both sides: $+1$ for a satellite bias, code or phase (`kSatBiasSign`), and -1 for the satellite clock correction (`kSatClockPartial`). The clock enters with -1 because the observable carries $-c\,\mathrm{d}t^{i}$ and the a-priori product clock is already inside the modelled value. The imported bias sign is treated with the SSR mapping (Section 18). Doppler rows are not part of this equation: velocity, clock rate and the inertial states are columns only a single-receiver configuration has.

4.5 Stochastic model

The observation noise covariance of an epoch is diagonal, with entries

$$\sigma^{2}_{r,i,s} \;=\; \sigma^{2}_{0,\mathrm{type}(s)} \;\cdot\; w(\theta^{i}_{r})^{-1} \;\cdot\; \xi(\mathrm{C/N_0 \text{ at } } (r,i,s)), \tag{9}$$

Table 2: Terms of the predicted observable, in the order of the sum (`ObsEffect` in `measurement/ObservationEquation.hpp`). μ_s is Equation (1).

Term	Parameter	Partial	Present on
range correction	receiver position $\boldsymbol{r}_r$ (three columns)	$-\boldsymbol{e}^i$, applied to $\boldsymbol{r}_r - \boldsymbol{r}_0$ (Equation (6))	every row
receiver clock	$c\,\mathrm{d}t_r$	$+1$	every row
phase clock	$c\,\mathrm{d}t_r^\phi$	$+1$	phase rows
troposphere	ZWD_r	$m_w(\theta_r^i)$	every row
gradients	(g_N, g_E)	$m_g(\theta)\cos\alpha$, $m_g(\theta)\sin\alpha$ (Equation (3))	every row
inter-system bias	$c\,\delta t_r^{(g)}$, g not the time reference	$+1$	every row
slant ionosphere	$I_{r,1}^i$	$+\mu_s$ on code, $-\mu_s$ on phase	uncombined rows
receiver code bias	$B_{r,\cdot}$ (Section 3)	coefficient of the receiver-bias rule (1, or the geometry-free coefficient of Equation (217))	code rows, uncombined, with `estimation.estimate_ifb`
satellite code bias	B_s^i	$+1$	code rows, with `estimation.estimate_sat_biases`
satellite phase bias	$\bar{b}_s^i$ and, with the receiver phase bias, $\delta_{r,s}$	$+1$ each	uncombined phase rows, with `estimation.estimate_sat_phase_biases`
satellite clock	$\Delta\delta t^i$	-1	uncombined rows, with `estimation.estimate_sat_clocks`
SSR bias	none (a known value)	the decoded bias times $+1$	every row; none on the network's own rows
ambiguity	λN in metres; on the NL/WL basis N_{NL} and N_{WL} in cycles	$+1$; on the NL/WL basis λ_s and $\mp\lambda_s$	phase rows
phase colour	per-satellite nuisance	$+1$	phase rows, with `network.phase_colour_state`

The three factors are:

- a per-type baseline ($\sigma_{0,\text{code}}$, $\sigma_{0,\text{phase}}$ from `filter.measurement_noise`);
- an elevation down-weighting factor $w(\theta_r^i)^{-1} = (\sin\theta_r^i)^{-2p_{\text{type}}}$ with a continuous, type-dependent exponent. Code and Doppler rows use `filter.code_elevation_power` and carrier-phase rows `filter.phase_elevation_power`; both default to 1.0, which scales the variance by the squared cosecant of the elevation. When the sine of the elevation is at most `1e-6`, a guard returns σ_0^2 unamplified. The derivation is in Equation (181);
- a carrier-to-noise-ratio term ξ, parameterised by `snr_model_params`.

`filter.stochastic_model` selects the weighting strategy: `constant`, `elevation` (the default), `snr` or `satellite_block`. Only the `snr` strategy uses the carrier-to-noise term, and it does not multiply the baseline variance by a factor: for code and phase rows it replaces the baseline variance by the variance-additive Hadas–Bosy form built from the observed carrier-to-noise ratio (coefficients `code_a`, `code_b`, `phase_a`, `phase_b`; the ratio is clamped to 20–55 dB-Hz by default; code rows take a per-band scale), then applies the elevation factor (`StochasticModelWeight::computeSnr`). Cross-correlations between observations are zero in the undifferenced parameterisation; differencing introduces the structured correlations treated in Section 5.

The variance of a row is Equation (9) on the network estimator's own rows (`rowVariance` in `measurement/MeasurementModelVariance.cpp`). A user of the network's corrections adds the uncertainty of the corrections it applies to the same variance: the satellite accuracy index with its clock-staleness growth, the phase-bias sigma on phase rows, the combined-correction sigma where it replaces those two, and a down-weighting factor on code rows without a bias-product entry (Section 18).

5 Differencing transformations

PPP-RTK with the single-difference transform and the baseline modes do not consume the undifferenced observations of Equations (4) and (5) directly. Each forms a linear combination across satellites (BSSD), receivers (BRSD), or both (DD). The combination cancels common-mode terms and leaves a residual structure in which the ambiguities can be integer. This section gives the algebraic cancellations and the matrix-form S-transforms that map undifferenced parameters to differenced ones.

5.1 Between-satellite single difference (BSSD)

Each constellation g has a pivot satellite p_g, elected by `PivotManager::selectPivotCandidates` under a sticky retention policy (the ordering is given in the *Pivot selection* paragraph below). The same `PivotManager` pivot serves both the combined (ionosphere-free) and the uncombined parameterisation. For any non-pivot satellite $i \in \mathcal{S}_g \setminus \{p_g\}$ and signal s, define

$$\begin{aligned} \Delta P_{r,s}^{i,p_g} &:= P_{r,s}^{i} - P_{r,s}^{p_g}, \\ \Delta \Phi_{r,s}^{i,p_g} &:= \Phi_{r,s}^{i} - \Phi_{r,s}^{p_g}. \end{aligned} \tag{10}$$

Substituting Equations (4) and (5) and cancelling the receiver-side common terms ($c\,\mathrm{d}t_r$, $c\,\delta t_r^{(g)}$, $B_{r,s}$, $b_{r,s}$) yields

$$\Delta P_{r,s}^{i,p_g} = \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} + \mu_s\,\Delta I_{r,1}^{i,p_g} - \Delta B_s^{i,p_g} + \Delta\varepsilon^{P}, \tag{11}$$

$$\Delta \Phi_{r,s}^{i,p_g} = \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} - \mu_s\,\Delta I_{r,1}^{i,p_g} + \lambda_s\,\Delta N_{r,s}^{i,p_g} - \Delta b_s^{i,p_g} + \Delta\varepsilon^{\Phi}. \tag{12}$$

The receiver-side cancellations are exact only when both observations enter the filter with the same modelling (tropospheric mapping, antenna PCV correction and so on). The satellite-dependent terms do not cancel: ΔT, for example, is the difference of two slant delays at different elevations and is small but not zero.

`BssdTransform::apply` realises the cancellation by zeroing the design-matrix columns of the receiver-shared states after differencing: receiver clock, phase clock, ISB, clock rate and ISB rate (when allocated), and the receiver code-bias (DCB/IFB) states. In both parameterisations it also zeroes the pivot's ambiguity columns (under the NL/WL basis, the pivot's NL column and, on a non-reference band, its WL column).

The cancellation of $B_{r,s}$ in Equation (11) holds for CDMA constellations, whose group delay is a single per-signal receiver term. GLONASS is the exception. Its FDMA inter-frequency biases are per-FCN-slot states, so two satellites on different slots carry different receiver delays and a residual $B_{r,s}^{(k(i))} - B_{r,s}^{(k(p_g))}$ survives the single difference. `BssdTransform` therefore exempts the per-slot GLONASS IFB states (identified by `StateKeys::isGlonassPerSlotIfbKey`) from the column zeroing. `PivotManager::buildEpochMaps` drops GLONASS from the BSSD candidate maps unless the flag is set, and `BssdTransform::apply` drops any row whose group has no pivot, so with the flag off GLONASS contributes no single-differenced row. GLONASS BSSD is off by default (`estimation.between_satellite_sd.enable_glonass`); when enabled, Equation (11) acquires the surviving inter-slot IFB difference as an additional estimated column.

Pivot selection. The incumbent pivot is kept while it remains at or above `estimation.between_satellite_sd.pivot_min_elevation` (default 45 degrees), is not in the exclusion set, and passes the ephemeris-freshness bar described below. Otherwise a new pivot is elected. The primary key is a hard lexicographic tier on phase-signal coverage: a full-coverage candidate outranks any partial-coverage candidate regardless of score, because the BSSD frame requires the pivot to track every signal another satellite tracks. Within a coverage tier the additive score $-\min\bigl(1, \sqrt{\sigma^2_{\hat{N},\max}}\bigr) + \mathrm{el}/90 - \frac{1}{2}\mathbb{1}[\mathrm{el} < 20^\circ]$ ranks the candidates. Float-ambiguity convergence $\sqrt{\sigma^2_{\hat{N},\max}}$ and elevation enter as co-equal terms, with a low-elevation penalty below 20°; a candidate with no ambiguity state yet takes the saturated convergence penalty of 1. The ephemeris-freshness bar excludes from pivot election any candidate whose broadcast $|\mathrm{obs} - \mathrm{toe}|$ exceeds that of its constellation's freshest qualified peer by more than `kPivotToeSpreadMaxS` (1200 s); the satellite remains usable as a non-pivot. The bar prevents the extrapolated orbit and clock error of a stale-ephemeris pivot from entering every SD row it anchors. Candidates below the minimum elevation are excluded; only when no satellite of the constellation passes the filters does the election fall back to the best-ranked or highest satellite without them. The baseline DD path scores differently: `DdPivotManager` keeps the incumbent while it remains in the candidate set of satellites shared by both receivers, and otherwise ranks on $\mathrm{avg_snr} + 0.01 \cdot \mathrm{num_sigs}$, without elevation.

Pivot transitions. A handover has three kinds of trigger. The first is selection: when the incumbent fails the retention test above (it leaves the visible set, drops below the minimum elevation, is excluded, or fails the freshness bar), p_g is re-elected from the remaining satellites. The second is a disturbed state: `isPivotDisturbed` reports the pivot disturbed when any of its phase ambiguity states has left $(0, 0)$ (absolute value above 1×10^{-6} or variance above 1×10^{-12}), for example after a cycle-slip reset. The third is an event: the Kalman filter's post-fit residual analyser (the pivot-offset detectors of Section 12.4) calls `PivotManager::requestForcedHandover` when the pivot's observation has slipped while its filter state is intact, a case `isPivotDisturbed` cannot

detect. The forced request carries an exclusion set, so the handover cannot land on another offset satellite. The `retired_pivots_this_epoch_` set bars a pivot retired in the current epoch from re-election within it, which terminates handover cascades, and the same-epoch re-entry gate (`hasPendingForcedHandover`) applies the handover before the rebuilt measurement update. The request also carries a re-inflation set: non-pivot states that absorbed a coherent pivot-side code bias have their covariance re-inflated after the re-reference (when slip-state re-inflation is enabled), because the information-conserving transform below cannot remove that offset. The baseline DD path has the counterpart `DdPivotManager::requestForcedRerank`, which drops the incumbent and excludes the flagged satellites from the next re-rank. The event response is required because the pivot's states are frozen at zero variance (below): a stiff, time-linked zero-q quantity requires a wired jump response.

On a handover the float ambiguities $\{\Delta N_{r,s}^{i,p_g^{\mathrm{old}}}\}_{i,s}$ are re-expressed relative to p_g^{new} by the linear S-transformation $\boldsymbol{x}^{\mathrm{new}} = \boldsymbol{S}\,\boldsymbol{x}^{\mathrm{old}}$, with $\boldsymbol{S}$ chosen so that $\Delta N_{r,s}^{p_g^{\mathrm{new}},p_g^{\mathrm{old}}}$ becomes the new reference (zero). The covariance transforms as $\boldsymbol{P}^{\mathrm{new}} = \boldsymbol{S}\,\boldsymbol{P}^{\mathrm{old}}\,\boldsymbol{S}^{\mathrm{T}}$. The transform conserves the filter's information and so preserves the integer information accumulated in the ambiguities. In the uncombined PPP-RTK parameterisation `PivotManager` applies the same $\boldsymbol{S}\,\boldsymbol{P}\,\boldsymbol{S}^{\mathrm{T}}$ re-reference to three state groups (`reReferenceAmbiguities`, `reReferenceIonosphere`, which also moves the ionosphere-rate states when they exist, and `reReferenceSatBiases`). This is why `BssdTransform` zeroes the pivot's ionosphere and sat-bias $\boldsymbol{H}$-columns after differencing, and why $\Delta I_{r,1}^{i,p_g}$ in Equations (11) and (12) is a pivot-referenced SD quantity rather than an absolute slant delay.

Where the transform cannot carry a state onto the new pivot, a re-seed follows or replaces it. In PPP-RTK three cases arise. An ambiguity on a phase signal the new pivot does not track stays referenced to the old pivot, so it is reset and marked for re-seeding against the new pivot. When the old pivot was disturbed, its re-referenced ambiguities are re-seeded from the single-differenced observations. When a disturbed pivot has no eligible replacement (every other satellite of the constellation is excluded or absent), `PivotManager::handleDisturbedPivot` re-zeroes the pivot's phase ambiguity states in $\boldsymbol{x}$ and the corresponding rows and columns of $\boldsymbol{P}$ in place, and no handover takes place. In the baseline DD path `DdPivotManager` applies the transform per (cons, code) group; a group in which the new pivot has no existing ambiguity is skipped (logged as "reset-and-reseed"), and its stale rows are pruned and re-seeded on the new pivot.

The S-basis constraint is realised by freezing the pivot's ambiguity, ionosphere (and ionosphere-rate, when enabled) and sat-bias covariances to zero and suppressing their process noise (`freezePivotAmbiguityCovariance`, `freezePivotIonoAndBiasCovariance`). The frozen ambiguity state is the condition `isPivotDisturbed` tests.

`PivotManager::getCleanHandovers` reports the *clean* handovers of the epoch: those that were neither forced nor triggered by a disturbed pivot. A clean handover is a lossless, invertible frame change, so the RTS smoother inverse-references the affected states across it; at a forced or disturbed handover the old pivot's ambiguity is discontinuous and the smoother starts a new segment. The losslessness stated above holds only for the clean subset.

The two modes use distinct code paths that share the algebra above. In PPP-RTK the pivot election, the sticky retention policy and the re-reference live in `filter/PivotManager`. The baseline DD path performs the re-reference in `filter/DdPivotManager`, under the `baseline.enable_pivot_handover` flag (default on). There the S-transform is applied in place, as a row pass over the state vector and covariance followed by a column pass over the covariance, without forming the transform matrix; the result passes through the positive-semidefinite guard `CovReReference::enforcePsd` (whose only caller is `DdPivotManager`), and the ambiguity keys are renamed onto the new pivot. With the flag off, every pivot change, including a forced re-rank, erases the DD ambiguities and

re-seeds them.

5.2 Between-receiver single difference (BRSD)

Baseline modes process one or more rovers r_{rov} against a single fixed-coordinate reference r_{ref}; each rover has its own measurement model. The BRSD operator forms

$$\nabla P^{i}_{r_{\mathrm{rov}} r_{\mathrm{ref}},s} \;:=\; P^{i}_{r_{\mathrm{rov}},s} - P^{i}_{r_{\mathrm{ref}},s}, \tag{13}$$

and analogously for $\nabla\Phi$. The cancellations of $\nabla\cdot$ are dual to those of $\Delta\cdot$: BRSD cancels the satellite-side common terms ($c\,\mathrm{d}t^i$, B^i_s, b^i_s) and retains the difference of the two stations' receiver clocks and biases. Once ∇ is applied, the satellite biases need no state in the filter. `BrsdTransform::apply` carries only the rover-side line of sight in its output: in baseline mode only the rover position is estimated, so the reference-station line of sight multiplies no state.

5.3 Double difference (DD)

The composition $\nabla\Delta\cdot = \Delta\big(\nabla\,\cdot\big)$ is order-independent up to noise correlation: $\Delta\big(\nabla\,\cdot\big) = \nabla\big(\Delta\,\cdot\big)$. Applied to the carrier-phase model, with the common-mode terms cancelled, it gives

$$\nabla\Delta\Phi^{i,p_g}_{r_{\mathrm{rov}} r_{\mathrm{ref}},s} \;=\; \nabla\Delta\rho^{i,p_g}_{r_{\mathrm{rov}} r_{\mathrm{ref}}} + \nabla\Delta T^{i,p_g}_{r_{\mathrm{rov}} r_{\mathrm{ref}}} - \mu_s\,\nabla\Delta I^{i,p_g}_{r_{\mathrm{rov}} r_{\mathrm{ref}},1} + \lambda_s\,\nabla\Delta N^{i,p_g}_{r_{\mathrm{rov}} r_{\mathrm{ref}},s} + \nabla\Delta\varepsilon^{\Phi}. \tag{14}$$

Equation (14) retains the geometry term (linear in the rover position perturbation), the differential atmospheric delays and the integer DD ambiguity. This structure is the basis of carrier-phase RTK.

Short vs. long baselines. On short baselines ($\lesssim 10\,\mathrm{km}$) the differential atmospheric terms in Equation (14) are negligible at the millimetre level and are not modelled. The differential ionosphere $\nabla\Delta I$ grows linearly with separation and reaches typically 1 m at a few hundred kilometres, which exceeds the carrier wavelength. `GnssProcessor` offers two mitigations, both off by default:

- `baseline.use_iono_free` collapses the per-band DD rows into the dual-frequency ionosphere-free combination. It eliminates the first-order ionospheric term at the cost of a non-integer ambiguity (IF phase ambiguities stay float and are excluded from AR) and $\sim 3\times$ amplified code noise. Satellite pairs with fewer than two shared bands are dropped in this mode.
- `baseline.estimate_differential_tropo` adds a single $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}} r_{\mathrm{ref}}}$ random-walk state (`differential_tropo_init_std_m`, default 0.1 m; `differential_tropo_process_noise_m_per_sqrt_s`, default 1×10^{-4}). It absorbs the differential wet delay that survives the BRSD on long baselines or baselines with a large height difference.

5.4 S-transform structure

BSSD and DD both have a matrix form $\boldsymbol{x}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{x}_{\mathrm{undiff}}$, with $\boldsymbol{S}$ a block design transform. For BSSD restricted to the ambiguity block of one constellation g on one signal s, with $|\mathcal{S}_g|$ tracked satellites, $\boldsymbol{S}$ is a $(|\mathcal{S}_g|-1)\times|\mathcal{S}_g|$ matrix of the form

$$\boldsymbol{S}_{\mathrm{BSSD}} \;=\; \begin{bmatrix} 1 & 0 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & & & \\ 0 & 0 & \cdots & 0 & -1 & 0 & \cdots & 1 \end{bmatrix}, \tag{15}$$

where the column of -1's sits at the index of the pivot p_g. The DD operator is the composition of the two single-difference operators, $\boldsymbol{S}_{\mathrm{DD}} = \boldsymbol{S}_{\mathrm{BSSD}}\,\boldsymbol{S}_{\mathrm{BRSD}}$, realised in code as `BrsdTransform::apply` followed by `BssdTransform::differenceBrsd` (`include/measurement/BssdTransform.hpp`). The composition coincides with the Kronecker product $\mathbf{D}_{\mathrm{sat}} \otimes \mathbf{D}_{\mathrm{rx}}$ of the between-satellite and between-receiver differencing matrices acting on the vectorised observations only on a fully populated satellite $\times$ receiver observation grid. The Kronecker form is a model; real data give a non-rectangular grid, and the code differences whatever pairs exist. The transformed covariance $\boldsymbol{P}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{P}_{\mathrm{undiff}}\,\boldsymbol{S}^{\mathrm{T}}$ is in general dense even when $\boldsymbol{P}_{\mathrm{undiff}}$ is diagonal, because every differenced row shares the pivot column. A DD filter therefore cannot be implemented as independent per-pair scalar updates: the correlated joint update is what integer ambiguity recovery through the LAMBDA decorrelation requires. Teunissen [36] gives the general theory and a catalogue of admissible S-transforms; Jonge [14] treats the rank defect.

Implementation notes. `GnssProcessor` does not materialise the chosen S-system as an explicit minimum-constraint $\boldsymbol{S}$-basis matrix. It is realised implicitly by the pivot-differencing operator (`measurement/BssdTransform`), an S-basis-invariant realisation of the uncombined parameterisation of Odijk [24]. A `DifferencingMode` strategy in `measurement/MeasurementBuilder` dispatches the three regimes (undifferenced, BSSD and DD) to separate builder classes.

Integer recovery on a DD row requires all four contributing observations to share the same RINEX signal code. `BssdTransform::differenceBrsd` marks a row `integer_capable` only when three conjuncts hold: `!brsd_aliased` on both the i_i and the pivot BRSD entries (rover and reference agree on each, i.e. no cross-attribute alias), and the rover uses the same code on i_i as on the pivot ($\texttt{rov_code}_i = \texttt{rov_code}_{\mathrm{piv}}$). The third conjunct is independent of the `brsd_aliased` flags: a row can be un-aliased on both entries and still carry a non-integer DCB/IFB residual if the rover mixes codes across i_i and the pivot. Rows that fail any conjunct are carried as float and excluded from ambiguity resolution.

Part II
Reference systems and measurement models

6 Coordinate Frames and Time Systems

Every model in this reference (the tropospheric and ionospheric mappings of Section 9, the solid-Earth, ocean, and pole tides of Section 10, the antenna, attitude, and phase-wind-up terms of Section 11, and the orbit evaluation of Section 8) assumes a fixed set of coordinate frames and time scales. This document specifies them in three parts. The first is the WGS84 ellipsoid and the three coordinate representations the engine uses: Cartesian ECEF, geodetic, and local ENU. The second is the GNSS time scales and the leap-second handling that ties them to UTC. The third is the Earth-orientation chain that rotates the celestial frame, in which the Sun, Moon, and satellite ephemerides are expressed, into the terrestrial frame of the receiver. None of these conversions is an estimator state. The one estimated frame quantity is the inter-system time offset (ISB), which absorbs the unmodelled bias between constellation time scales and belongs to Section 3. This document covers only the *deterministic* conversions that the ISB is defined relative to. Code references are to `src/core/math/Coordinates.cpp`, `src/core/math/TimeUtils.cpp`, `src/core/model/Planetary.cpp`, `src/core/products/LiveEphemerisStore.cpp`, `src/core/products/ProductParser.cpp`, and `src/core/rinex/RinexParser.cpp`.

6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames

The terrestrial reference frame is the WGS84 Earth-Centred, Earth-Fixed (ECEF) Cartesian system: origin at the geocentre, z along the conventional pole, x toward the IERS reference meridian, y completing a right-handed triad. The defining ellipsoid has semi-major axis a and flattening f, from which the first eccentricity squared follows (`Coordinates::ecefToLlh` and `Coordinates::llhToEcef` each derive it from f):

$$e^2 \;=\; f\,(2-f). \tag{16}$$

The radius of curvature in the prime vertical at geodetic latitude φ is

$$N(\varphi) \;=\; \frac{a}{\sqrt{1-e^2\,\sin^2\varphi}}, \tag{17}$$

which scales the entire forward map from geodetic coordinates (φ, λ, h) to ECEF (`Coordinates::llhToEcef`):

$$\boxed{\begin{aligned} x &= \big(N+h\big)\cos\varphi\,\cos\lambda, \\ y &= \big(N+h\big)\cos\varphi\,\sin\lambda, \\ z &= \big((1-e^2)\,N+h\big)\sin\varphi. \end{aligned}} \tag{18}$$

The factor $(1-e^2)$ on the z component is what makes the surface an ellipsoid. On a sphere every axis would carry N unchanged. The factor is also why the geodetic normal, which is perpendicular to the ellipsoid, differs from the geocentric radius vector.

Inverse map (Bowring fixed point). The inverse problem, recovering (φ, λ, h) from (x, y, z), has no closed form for the latitude because N in (18) itself depends on φ. The longitude is immediate, $\lambda = \operatorname{atan2}(y, x)$. For the latitude the engine runs a Bowring fixed-point iteration on the equatorial projection $p = \sqrt{x^2+y^2}$ (`Coordinates::ecefToLlh`). The seed neglects height,

$$\varphi^{(0)} = \operatorname{atan2}\big(z,\; p\,(1-e^2)\big), \tag{19}$$

and each pass recomputes N and the height and re-evaluates the latitude with the height feedback folded back into the eccentricity term:

$$h^{(k)} = \frac{p}{\cos\varphi^{(k)}} - N^{(k)}, \qquad \varphi^{(k+1)} = \operatorname{atan2}\left(z,\; p\left[1 - e^2\,\frac{N^{(k)}}{N^{(k)}+h^{(k)}}\right]\right). \tag{20}$$

The iteration is capped at ten passes and exits early once $|\varphi^{(k+1)} - \varphi^{(k)}| < 10^{-13}$; for terrestrial heights it converges to machine precision in two or three steps.

Pole branch. When $p < 10^{-12}\,\mathrm{m}$ the point lies on (or within a picometre of) the z axis and both the longitude and the $p/\cos\varphi$ height formula degenerate. The code takes an explicit branch (`Coordinates::ecefToLlh`, the $p < 10^{-12}$ guard): the longitude retains the $\operatorname{atan2}(y, x)$ value computed before the guard, formally undefined on the axis and immaterial there, while the branch fixes $\varphi = \pm\pi/2$ by the sign of z and reads the height directly off the polar semi-minor axis,

$$h = |z| - N\,(1-e^2) = |z| - a\sqrt{1-e^2} \quad (\text{at the pole, } \sin^2\varphi = 1), \tag{21}$$

since $N(1-e^2)$ evaluated at the pole is exactly the polar radius $a\sqrt{1-e^2}$. This guard keeps the inverse map total over all of $\mathbb{R}^3$, which matters because the planetary and attitude code occasionally evaluates geometry for satellite positions that transit near the polar axis.

Local East–North–Up frame. Many of the downstream models (the tropospheric mapping function, the elevation mask, the antenna phase-centre projection) are most natural in the topocentric ENU frame anchored at the receiver's geodetic (φ, λ). The columns of the ENU→ECEF rotation are the three local unit vectors expressed in ECEF (`Coordinates::enuToEcefMatrix`):

$$\boxed{\boldsymbol{R}_{\mathrm{ENU}}(\varphi,\lambda) = \begin{bmatrix} \hat{\boldsymbol{e}}_{\mathrm{E}} & \hat{\boldsymbol{e}}_{\mathrm{N}} & \hat{\boldsymbol{e}}_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} -\sin\lambda & -\sin\varphi\cos\lambda & \cos\varphi\cos\lambda \\ \cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi\sin\lambda \\ 0 & \cos\varphi & \sin\varphi \end{bmatrix}.} \tag{22}$$

The Up column is the outward geodetic normal, identical to the direction along which h is measured in (18); East is its horizontal cross-product with the pole, and North completes the right-handed triad. Because the columns are orthonormal, $\boldsymbol{R}_{\mathrm{ENU}}$ is a proper rotation and its inverse is its transpose: an ECEF displacement is resolved into local components by $\boldsymbol{v}_{\mathrm{ENU}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\,\boldsymbol{v}_{\mathrm{ECEF}}$ (`Coordinates::ecefToEnu`, with `Coordinates::enuToEcef` the forward direction), avoiding any explicit matrix inversion.

Elevation and azimuth. With the receiver-to-satellite line-of-sight resolved into the local frame, $\boldsymbol{e}_{\mathrm{ENU}} = (e_{\mathrm{E}}, e_{\mathrm{N}}, e_{\mathrm{U}})^{\mathrm{T}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\boldsymbol{e}$, the elevation and azimuth that gate the measurement (the elevation mask of Section 4.3 and the mapping-function argument of Section 9) follow directly:

$$\mathrm{el} = \arcsin\!\big(e_{\mathrm{U}}\big), \qquad \mathrm{az} = \operatorname{atan2}\!\big(e_{\mathrm{E}},\, e_{\mathrm{N}}\big), \tag{23}$$

with azimuth measured clockwise from North. Equations (22)–(23) connect the ECEF state vector to every elevation-dependent weighting and antenna model in this reference.

6.2 Station reference points: marker, ARP, and phase centre

A station coordinate can refer to any of three points. The *marker* (monument) is the surveyed benchmark, and is what an IGS SINEX/SSC solution, an NGS Station Information File, and the RINEX `APPROX POSITION XYZ` header record all refer to. The *antenna reference point* (ARP) sits where the antenna meets its mount and is the origin against which the ANTEX phase-centre offsets of Section 11 are tabulated. The *antenna phase centre* is where the measurement is physically realised. The estimator's position state is the ARP, because the receiver PCO/PCV it applies are referenced to the ARP. The antenna model covers the step from ARP to phase centre, so only the step from marker to ARP belongs to this document. Treating a marker coordinate as an ARP coordinate, or the reverse, puts a constant bias equal to the eccentricity into the solution, mostly in height.

The leg itself is the RINEX `ANTENNA:DELTA H/E/N` eccentricity $\boldsymbol{e}_{\mathrm{HEN}} = (\delta H,\, \delta E,\, \delta N)^{\mathrm{T}}$, defined as the offset from the marker to the ARP in the topocentric frame *at the marker*. RINEX writes the triple as $(H, E, N) = (\mathrm{Up}, \mathrm{East}, \mathrm{North})$. The rotation of (22) expects (E, N, U), so the triple must be reordered before it is rotated (`Coordinates::markerToArp`, the single owner of this convention):

$$\boxed{\boldsymbol{r}_{\mathrm{ARP}} \;=\; \boldsymbol{r}_{\mathrm{mkr}} \;+\; \boldsymbol{R}_{\mathrm{ENU}}\big(\varphi_{\mathrm{mkr}},\, \lambda_{\mathrm{mkr}}\big) \begin{bmatrix} \delta E \\ \delta N \\ \delta H \end{bmatrix},} \tag{24}$$

with $(\varphi_{\mathrm{mkr}}, \lambda_{\mathrm{mkr}})$ recovered from $\boldsymbol{r}_{\mathrm{mkr}}$ by the inverse map of (20). Write $\boldsymbol{e}'_{\mathrm{HEN}}$ for the permuted $(\delta E, \delta N, \delta H)^{\mathrm{T}}$ that the rotation acts on.

The inverse is not a sign flip. Reducing an ARP estimate to the monument is not the negative of (24). The local frame is defined at the marker, and the marker is the unknown. Evaluating $\boldsymbol{R}_{\mathrm{ENU}}$ at the ARP instead leaves an error of order $\|\boldsymbol{e}_{\mathrm{HEN}}\|^2/R_\oplus$. The engine therefore takes one fixed-point step (`Coordinates::arpToMarker`):

$$\boldsymbol{r}^{(0)} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}_{\mathrm{ARP}})\big)\,\boldsymbol{e}'_{\mathrm{HEN}}, \qquad \boldsymbol{r}_{\mathrm{mkr}} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}^{(0)})\big)\,\boldsymbol{e}'_{\mathrm{HEN}}, \tag{25}$$

which closes the round trip with (24) to well below the millimetre level this reference grades at.

Where the reduction is applied, and where it is not. Every position derived from a RINEX *header* is reduced: the post-processing filter seed (`Application::initializePosition` and the `PppModel` state initialiser), the real-time seed (`RealTimeEngine::initialize`), the baseline rover a priori, and the closed-loop consumer truth all take the marker through (24), some calling `Coordinates::markerToArp` directly, others through the accessor `RinexParser::getArpPositionXyz` that pairs the two header fields. A coordinate resolved from a Station Information File is a monument coordinate too and is reduced through the same function (Section 7.7). A coordinate supplied *directly in the configuration* is not reduced: it is taken at face value, on the stated policy that the user owns its reference point. So the baseline *reference* coordinate is used as given while the rover a priori read from its header is reduced, an asymmetry inside one mode. The network estimator's station coordinates (`StationConfig::position_ecef`, optionally SIF-resolved under `network.use_sif_coordinates`) are monument coordinates held *fixed*, their three position columns dropped from the normal equations. With `network.station_marker_to_arp` (default off; on in both reference configurations, `config/golden/golden_field_refound.json` and `golden_sim_refound.json`) the network setup reduces each roster coordinate through (24) once, with the station's own RINEX eccentricity, before the coordinate reaches the state, the model or any geometry; the test station's truth coordinate takes the same function. A station whose header carries no eccentricity record is left unchanged and named in a warning. With the setting off, a monument-referenced coordinate is not absorbed by a position state; it survives into the residuals and hence into the estimated clocks, biases, and ionosphere. A zero or absent eccentricity makes (24) an exact, bitwise no-op, which is what keeps header-less RTCM/SBF streams and `0/0/0` stations untouched; the ordering, sign, no-op, and round-trip properties are pinned by the `AntennaEccentricityTest` suite.

6.3 GNSS time scales and the GPS reference epoch

Time inside the engine is carried as a `GnssTime` {week, tow} pair, an integer GPS week and a floating-point time-of-week in seconds, and is expanded to a single continuous count $T = \mathrm{week} \times 604800 + \mathrm{tow}$ wherever a monotone scalar is needed (e.g. ERP interpolation, Section 6.9). A week boundary is 604 800 s and a day 86 400 s (`src/core/math/TimeUtils.cpp`, header constants `SECONDS_IN_WEEK`, `SECONDS_IN_DAY`). The common origin is the GPS epoch, 1980-01-06 00:00:00 UTC, whose Julian Date is fixed in the code:

$$\mathrm{JD}_{\mathrm{GPS}} = 2444244.5 \qquad (\texttt{src/core/math/TimeUtils.cpp}, \texttt{GPS_EPOCH_JD}). \tag{26}$$

GPS time is a continuous atomic scale, since it never inserts a leap second, so it sits a fixed integer offset below International Atomic Time and a fixed fractional offset below Terrestrial Time (`src/core/math/TimeUtils.cpp`, header constants `TAI_GPS_OFFSET`, `TT_GPS_OFFSET`):

$$\boxed{\mathrm{TAI} - \mathrm{GPS} = 19\ \mathrm{s}, \qquad \mathrm{TT} - \mathrm{GPS} = 51.184\ \mathrm{s} = (32.184 + 19)\ \mathrm{s}.} \tag{27}$$

The 32.184 s in the TT relation is the constant TT−TAI offset; together these let any GPS time be promoted to the TT scale on which precession and nutation are evaluated. Terrestrial Time relative to the J2000.0 epoch is measured in Julian centuries,

$$t = \frac{\mathrm{JD_{TT}} - 2451545.0}{36525.0}, \tag{28}$$

the dimensionless argument of the astronomical series of Sections 6.7 and 6.10 (`TimeUtils::jdToJ2000Centuries`, constants `J2000_JD`, `DAYS_PER_JULIAN_CENTURY`).

Gregorian↔Julian Date. The calendar conversions follow the Meeus algorithm [21]. The forward map, with the month shifted so January and February count as months 13 and 14 of the prior year and a Gregorian-reform correction $b = 2 - A + \lfloor A/4 \rfloor$ for $A = \lfloor y/100 \rfloor$, is (`TimeUtils::gregorianToJulianDay`):

$$\mathrm{JD} = \lfloor 365.25\,(y + 4716) \rfloor + \lfloor 30.6001\,(m + 1) \rfloor + d + b - 1524.5 + \frac{(\text{time of day})}{86400}, \tag{29}$$

and the inverse (`TimeUtils::julianDayToGregorian`) reverses it through the integer auxiliaries $z = \lfloor \mathrm{JD}{+}0.5 \rfloor$ and the Gregorian-reform branch at $z \geq 2299161$. The code keeps two kinds of calendar date apart. A date already on the GPS scale needs no leap correction (`gregorianGpsToGpsTime`). A date on UTC must have the leap seconds at that epoch added to reach GPS time (`TimeUtils::gregorianToGpsTime`). Confusing the two produces whole-second timing errors.

Elapsed time and the start-offset comparison. Epoch times come from a calendar-to-GPS conversion that carries about 2×10^{-5} s of double-precision residue, and a continuous GPS second count near 1.4×10^{9} resolves only 2.4×10^{-7} s. Both are far below a millisecond and far below any epoch schedule, so elapsed time between two epochs is taken on a whole-millisecond grid (`TimeUtils::elapsedMs`):

$$\Delta_{\mathrm{ms}} \;=\; \mathrm{round}\big(10^{3}\,(T_{\mathrm{later}} - T_{\mathrm{earlier}})\big). \tag{30}$$

The start-offset seek (`network.start_offset_hours`) and the banding of the elapsed hour read it. An epoch has reached an offset o when $\Delta_{\mathrm{ms}} \geq \mathrm{round}(10^{3}\,o)$ (`TimeUtils::offsetReached`). An offset that lies on an epoch (a bare hour on a 30 s schedule) is therefore reached by that epoch, where a floating-point comparison reached it one epoch late whenever the residue was negative. An offset between two epochs (a half-epoch back-off) selects the same epoch under either comparison, because no epoch lies within a millisecond of it.

6.4 Leap seconds and per-system dispatch

UTC is tied to GPS by the historical table of leap-second insertions. The engine stores the table as $\big(\mathrm{JD}_k,\ \Delta_{\mathrm{GU}}(k)\big)$ pairs: the Julian Date of 0 h *UTC* on the insertion day and the GPS−UTC offset that holds from then on, the first row being the GPS epoch itself with offset zero (`src/core/math/TimeUtils.cpp`, `LEAP_TABLE`). The dates are UTC, so a lookup compares on the scale of its argument. A date already on UTC compares directly, and a reverse scan returns the offset of the latest boundary not after the epoch:

$$\Delta_{\mathrm{GU}}(\mathrm{JD_{UTC}}) = \Delta_{\mathrm{GU}}(k^{\star}), \qquad k^{\star} = \max\{\, k : \mathrm{JD}_k \leq \mathrm{JD_{UTC}} \,\}, \tag{31}$$

falling back to zero before the epoch (`TimeUtils::leapSecondsAtUtcJd`, used by the calendar UTC-to-GPS conversion `TimeUtils::gregorianToGpsTime`). An instant on the GPS scale is moved

onto UTC before the comparison. A new offset applies when UTC reaches 0 h of the insertion date, and that instant lies $\Delta_{\mathrm{GU}}(k)$ seconds later on the GPS clock, so

$$k^{\star} = \max\left\{ k : \mathrm{JD}_{\mathrm{GPS}} - \tfrac{\Delta_{\mathrm{GU}}(k)}{86400} \;\geq\; \mathrm{JD}_{k} \right\} \tag{32}$$

(`TimeUtils::leapSecondsGpsUtc`). Comparing a GPS-scale date with the UTC table directly would apply each offset early by its own value (18 s at the 2017-01-01 insertion). TAI−UTC is then $\Delta_{\mathrm{GU}} + 19$ s from (27) (`TimeUtils::leapSecondsTaiUtc`). The lookup must depend on the epoch. A leap second was inserted between 2005 and 2010, so the same UTC calendar instant maps to different GPS times in those two years.

The wall-clock leap constant and its runtime override. The boundary-aware table of (31) is the authority for every *epoch-referenced* conversion (a dated calendar timestamp, GLONASS day arithmetic, UTC formatting) because each of those carries the epoch at which the correct offset is known. The one conversion that does not is the promotion of the *system wall clock* to GPS time (`TimeUtils::nowContinuousGpsSeconds`, `nowGpsTime`), used only on the real-time decode and latency-stamping paths: POSIX `system_clock` is UTC and carries no epoch of its own, so the offset cannot be looked up and must instead be supplied as a fixed count. That count is the compiled constant `TimeUtils::LEAP_SECONDS` = 18 s (valid from 2017-01-01), read through an atomic accessor (`currentLeapSeconds`) so that a future insertion can be applied to a live binary without a rebuild: `setLeapSecondsOverride`, driven by the config key `realtime.leap_seconds` (`src/core/config/GnssConfigLoad.cpp`), replaces it at real-time startup. The override deliberately touches *only* the wall-clock path; the epoch-dependent (31) and (32) lookups are left alone, since they are correct at every historical epoch, the insertion instants included, and an override there would corrupt rather than update them.

Per-constellation Gregorian dispatch. The four supported time systems differ only in how a RINEX/broadcast calendar timestamp is referred to the GPS scale, dispatched by a system tag (`TimeUtils::GnssTimeFromDate`):

- **GPS / Galileo**: already on the GPS scale; the timestamp is converted with no leap correction (GST is steered to GPS time and shares the same week/second framing to within the few nanoseconds the ISB state of Section 3 absorbs).
- **BeiDou**: the timestamp is on BDT, which runs 14 s behind GPS (Section 6.6); the code converts on the GPS-scale assumption and then *adds* 14 s, carrying any week roll-over (`TimeUtils::GnssTimeFromDate`, BeiDou branch).
- **GLONASS**: the RINEX timestamp is UTC; the full leap correction of (31) is applied to bring it onto GPS time (`TimeUtils::GnssTimeFromDate`, GLONASS branch, through `TimeUtils::gregorianToGpsTime`).

GPS and Galileo therefore carry no leap correction, BeiDou a fixed 14 s, and GLONASS the full epoch-dependent leap count. Each matches the time reference that defines the constellation. There is a fourth branch: an unrecognised time-system tag falls through to the GPS-scale conversion with only a trace-level note, and a calendar date that fails validation returns the sentinel $\{0, 0\}$ rather than a fabricated epoch (`TimeUtils::GnssTimeFromDate`). The fallback logs nothing at default verbosity, so an unexpected tag produces a timing error of the full leap count with no warning.

6.5 GLONASS time-of-day to full GPS week

GLONASS broadcast and SSR records frequently carry only a time-of-day, seconds since the start of the GLONASS day, with no week and no absolute date. GLONASS time runs on UTC(SU) advanced by three hours (Moscow), so its day boundary is three hours ahead of the UTC midnight. Reconstructing a full GPS week from a bare time-of-day therefore needs an *anchor*: an approximate GPS-seconds estimate of the true epoch (typically the current obs epoch), against which the day and the ± 12 h wrap are resolved (`TimeUtils::glonassTodToGpsTime`, after RTKLIB's `adjday_glot`). The procedure, with day length $D = 86\,400$ s, half-day $D/2$, and Moscow offset $M = 3 \times 3600$ s:

1. Convert the anchor GPS seconds to UTC by subtracting the leap count at that epoch (the GPS-scale lookup (32)), $t_{\mathrm{UTC}} = t_{\mathrm{anchor}} - \Delta_{\mathrm{GU}}$, then shift into the Moscow-day frame $t_{\mathrm{M}} = t_{\mathrm{UTC}} + M$ and locate that day's midnight $t_0 = \lfloor t_{\mathrm{M}}/D \rfloor\, D$, giving the anchor's time-of-day $\tau_{\mathrm{a}} = t_{\mathrm{M}} - t_0$.

2. Place the reported time-of-day τ in the anchor's day, resolving the midnight wrap toward the nearest match:

$$\boxed{\tau \leftarrow \begin{cases} \tau + D, & \tau < \tau_{\mathrm{a}} - D/2, \\ \tau - D, & \tau > \tau_{\mathrm{a}} + D/2, \\ \tau, & \text{otherwise}, \end{cases} \qquad t_{\mathrm{GPS}} = \underbrace{(t_0 + \tau)}_{\text{Moscow}} - M + \Delta_{\mathrm{GU}}.} \tag{33}$$

The last expression removes the Moscow offset to recover UTC and then adds the same leap count Δ_{GU} to return to the GPS scale. The leap term enters once with each sign and cancels. It appears only because the Moscow day must be located on the UTC scale. If no usable anchor is supplied (a non-positive value) the routine returns the sentinel $\{0, 0\}$ rather than fabricate a week, so the caller can detect the failure and fall back; the continuous-seconds-to-week split that closes the routine carries the standard negative-TOW correction (`gpsSecondsToGnssTime`).

6.6 BeiDou time: the 14 s offset and independent week base

BeiDou time (BDT) differs from GPS time by a fixed number of seconds and by an independent week numbering. Different modules handle the two parts. The *seconds* relation,

$$\mathrm{BDT} = \mathrm{GPS} - 14\ \mathrm{s}, \tag{34}$$

reflects that BDT was synchronised to UTC at its 2006 zero epoch, by which time GPS already led UTC by 14 s; it is applied in `TimeUtils` when a BeiDou calendar timestamp is converted to GPS time (`TimeUtils::GnssTimeFromDate`, BeiDou branch; Section 6.4). The *week* relation is separate. BDT counts weeks from its own zero epoch (2006-01-01), which is GPS week 1356, and this base is applied at query time by the ephemeris store, not by `TimeUtils` (the file-local constant `kBdtToGpsOffsetSec`, consumed by `LiveEphemerisStore::computeSatellitePV`). When the store lifts a BeiDou ephemeris reference time onto the common GPS scale for a cross-system age check it adds the combined offset

$$\boxed{\Delta_{\mathrm{BDT}\to\mathrm{GPS}} = 1356 \times 604800\ \mathrm{s} + 14\ \mathrm{s},} \tag{35}$$

the week base plus the 14 s of (34) (`kBdtToGpsOffsetSec`, applied by the file-local `toeGpsContinuous`). Without the week base every BeiDou ephemeris would appear about 1356 weeks ($\sim$26 years) old, and the staleness guard in `LiveEphemerisStore::computeSatellitePV` would reject it without a message. The offset does not cancel, because the query time is on the GPS scale while the stored t_{oe} uses the BDT week.

6.7 Legacy ECI→ECEF: the equinox-based chain

The Sun, the Moon, and for some product streams the satellite positions are given in a quasi-inertial celestial frame. The tides, attitude, and phase wind-up that use them are computed in the terrestrial ECEF frame. The Earth-orientation matrix $\boldsymbol{U}$ connects the two: $\boldsymbol{r}_{\mathrm{ECEF}} = \boldsymbol{U}\,\boldsymbol{r}_{\mathrm{ECI}}$. The default (always-available) path is the classical equinox-based chain following the IAU-1976 precession and IAU-1980 nutation conventions [26], composed as (`Planetary::eci2ecef`):

$$\boxed{\boldsymbol{U} = \boldsymbol{W}_{\mathrm{pm}}\,\boldsymbol{R}_3(\mathrm{GAST})\,\boldsymbol{N}_{\mathrm{nut}}\,\boldsymbol{P}_{\mathrm{prec}}.} \tag{36}$$

Here $\boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3$ are the elementary right-handed axis rotations of `Planetary::Rxyz` and each factor reverses one piece of the celestial-to-terrestrial orientation: precession and nutation orient the true equator and equinox of date, the sidereal rotation $\boldsymbol{R}_3(\mathrm{GAST})$ spins the frame about the pole by the apparent sidereal time, and polar motion $\boldsymbol{W}_{\mathrm{pm}}$ ties the celestial pole to the conventional terrestrial pole.

Precession (IAU 1976). With t the TT centuries of (28) and arcsecond coefficients converted to radians, the three equatorial precession angles ζ_A, θ_A, z_A and the mean obliquity ε_0 are (the precession block of `Planetary::eci2ecef`):

$$\begin{aligned}
\zeta_A &= 2306.2181\,t + 0.30188\,t^2 + 0.017998\,t^3,\\
\theta_A &= 2004.3109\,t - 0.42665\,t^2 - 0.041833\,t^3,\\
z_A &= 2306.2181\,t + 1.09468\,t^2 + 0.018203\,t^3,\\
\varepsilon_0 &= 84381.448 - 46.8150\,t - 0.00059\,t^2 + 0.001813\,t^3 \quad (\mathrm{arcsec}),
\end{aligned} \tag{37}$$

assembled as $\boldsymbol{P}_{\mathrm{prec}} = \boldsymbol{R}_3(-z_A)\,\boldsymbol{R}_2(\theta_A)\,\boldsymbol{R}_3(-\zeta_A)$.

Nutation (IAU 1980, 106 terms). The nutation in longitude $\Delta\psi$ and in obliquity $\Delta\varepsilon$ are summed over the 106-term IAU-1980 series [26] in the five Delaunay fundamental arguments $f = (l, l', F, D, \Omega)$ (`Planetary::nutIau1980`; arguments from `Planetary::astArgs`):

$$\Delta\psi = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\psi} + \dot{A}_i^{\psi}\,t\right)\sin\left(\boldsymbol{n}_i\cdot f\right), \quad \Delta\varepsilon = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\varepsilon} + \dot{A}_i^{\varepsilon}\,t\right)\cos\left(\boldsymbol{n}_i\cdot f\right), \tag{38}$$

in arcseconds (the 10^{-4} scales the tabulated 0.1 mas units), then converted to radians. The nutation matrix rotates by the true obliquity $\varepsilon_0 + \Delta\varepsilon$: $\boldsymbol{N}_{\mathrm{nut}} = \boldsymbol{R}_1(-\varepsilon_0 - \Delta\varepsilon)\,\boldsymbol{R}_3(-\Delta\psi)\,\boldsymbol{R}_1(\varepsilon_0)$ (the nutation block of `Planetary::eci2ecef`).

Apparent sidereal time and the 0h-UT1 reference. Greenwich Apparent Sidereal Time adds the equation of the equinoxes, the nutation in longitude projected on the equator, plus the two leading complementary terms in the lunar node Ω, to the mean sidereal time (the GAST block of `Planetary::eci2ecef`):

$$\mathrm{GAST} = \mathrm{GMST} + \Delta\psi\cos\varepsilon_0 + \left(0.00264\sin\Omega + 0.000063\sin 2\Omega\right)\mathrm{arcsec}. \tag{39}$$

GMST itself is built from the IAU-1982 model evaluated at the *start* of the current UT1 day plus the Earth's rotation through the elapsed day fraction (`Planetary::utc2gmst`). The reference instant

must be 0h UT1 of the calendar day, and because a Julian Date is referenced to *noon*, that midnight is recovered as

$$\mathrm{JD}_{0\mathrm{h}} = \left\lfloor \mathrm{JD}_{\mathrm{UT1}} - 0.5 \right\rfloor + 0.5, \tag{40}$$

not $\lfloor \mathrm{JD} \rfloor$ (same routine). The bare floor gives the *previous* noon for any epoch after midnight. That half-day error is roughly 0.5° in GMST and, through (36), of order 10^6 km in the rotated Sun position. A comment in `Planetary::utc2gmst` records this failure.

Polar motion. The terrestrial-pole tie is $\boldsymbol{W}_{\mathrm{pm}} = \boldsymbol{R}_2(-x_p)\,\boldsymbol{R}_1(-y_p)$ for the interpolated pole coordinates (x_p, y_p) in radians (the polar-motion block of `Planetary::eci2ecef`), the small-angle two-axis rotation that aligns the celestial intermediate pole of date with the IERS conventional pole.

6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006)

An opt-in path (config key `estimation.enable_cio_frame`, getter `GnssConfig::enableCioFrame`; **default off**, so the equinox chain of (36) is the shipped path) replaces the equinox chain with the modern Celestial Intermediate Origin (CIO) based transform of the IAU 2000A/2006 conventions [26], evaluated through SOFA [12] and mirroring the M-PAGES `NgsFormC2T` construction (`Planetary::celestialToTerrestrial`). The celestial-to-terrestrial matrix is composed as

$$\boxed{\boldsymbol{U} = \mathtt{POM}(x_p, y_p, s')\;\boldsymbol{R}_3(\theta_{\mathrm{ERA}})\;\mathtt{C2I}(X, Y, s),} \tag{41}$$

i.e. the SOFA composition `iauC2tcio`. The Celestial Intermediate Pole coordinates (X, Y) and the CIO locator s come from the full IAU-2000A precession-nutation model via `iauXys00a`, after which the *observed* celestial-pole offsets are injected additively,

$$X \leftarrow X + \mathrm{d}X, \qquad Y \leftarrow Y + \mathrm{d}Y, \tag{42}$$

and the celestial-to-intermediate matrix is formed by `iauC2ixys`. The diurnal rotation is the Earth Rotation Angle θ_{ERA} (a strictly linear function of UT1, `iauEra00`) rather than the sidereal time of (39), and the terrestrial-pole tie `POM` uses `iauPom00` with the TIO locator s' from `iauSp00`. Two-part Julian Dates are passed with the $(2400000.5, \mathrm{MJD})$ split for numerical conditioning (same routine). The two paths should agree. With $\mathrm{d}X = \mathrm{d}Y = 0$ the CIO result is the bare IAU-2000A model. It differs from the legacy equinox chain only by the frame bias and the model version, which is below a milliarcsecond. The engine checks this at startup by logging the angular separation of the two Sun directions (the startup diagnostic in `Planetary::computeSunMoonPosition`).

6.9 Earth-orientation parameters: parse and interpolation

Both transforms above need the time-varying Earth-orientation parameters $(x_p, y_p, \mathrm{UT1} - \mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y)$. These arrive in a standard IGS `.ERP` file, parsed by `ProductParser::parseErp` (`src/core/products/ProductParser.cpp`). The parser reads the MJD time tag and the pole/UT1 columns and applies the IGS scale factors: the pole coordinates are tabulated in 10^{-6} arcseconds and UT1−UTC in 10^{-7} seconds (`ProductParser::parseErp`):

$$x_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad y_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad (\mathrm{UT1} - \mathrm{UTC}) = (\mathrm{col}) \times 10^{-7}\,\mathrm{s}. \tag{43}$$

The MJD tag is converted to a GPS week/TOW relative to the GPS-epoch MJD (same routine). The CIP offsets $\mathrm{d}X, \mathrm{d}Y$ are *not* carried by the standard `.ERP` format and default to zero; because the IAU-2000A model of (41) already includes the modelled precession-nutation, $\mathrm{d}X = \mathrm{d}Y = 0$ is

the correct nominal value, not a missing input (same routine). The parser does no interpolation. `Planetary::interpolateErp` binary-searches the chronologically sorted table on continuous GPS seconds and linearly interpolates every component between the bracketing entries:

$$p(t) = p_0 + \frac{t - t_0}{t_1 - t_0}\,(p_1 - p_0), \qquad p \in \{x_p, y_p, \mathrm{UT1{-}UTC}, \mathrm{d}X, \mathrm{d}Y\}, \tag{44}$$

clamping to the nearest endpoint outside the table span (same routine). The interpolated (x_p, y_p) feed the polar-motion factor of (36) / (41), while UT1−UTC corrects UTC to UT1 before the sidereal/ERA evaluation. The product-evaluation document (Section 8) cross-references this section rather than re-deriving the polar-motion application, since the same $\boldsymbol{W}_{\mathrm{pm}}$ rotates both the luni-solar vectors and any celestial-frame satellite product.

6.10 The Sun and Moon series these transforms position

The orientation chain exists to place the Sun and Moon in ECEF for the geophysical and observation-model corrections. The celestial-frame positions are computed first, from an analytic series in the J2000 centuries t and the Delaunay arguments f (`Planetary::sunMoonPosEci`), an enhanced RTKLIB/Meeus model [21, 22]. The Sun's ecliptic longitude is the mean longitude plus the equation of centre (three harmonics of the solar mean anomaly $M_\odot$) and small Venus, Jupiter, and lunar perturbation terms (same routine); the obliquity rotation

$$\varepsilon = 23.439291 - 0.0130042\,t \quad (\mathrm{deg}) \tag{45}$$

carries the resulting ecliptic $(\lambda_\odot, \beta_\odot)$ into equatorial ECI components:

$$\begin{aligned} r_x &= r\,\cos\beta_\odot \cos\lambda_\odot, \\ r_y &= r\,(\cos\varepsilon\,\cos\beta_\odot \sin\lambda_\odot - \sin\varepsilon\,\sin\beta_\odot), \\ r_z &= r\,(\sin\varepsilon\,\cos\beta_\odot \sin\lambda_\odot + \cos\varepsilon\,\sin\beta_\odot), \end{aligned} \tag{46}$$

with the solar distance r from the three-harmonic radius series. The Moon's longitude, latitude, and horizontal-parallax-derived distance follow the analogous Brown-series truncation in the same five fundamental arguments (same routine). The resulting ECI vectors are then rotated to ECEF by exactly the $\boldsymbol{U}$ of (36) or (41) according to the `estimation.enable_cio_frame` toggle (default off, Section 6.8), in `Planetary::computeSunMoonPosition`; the ECEF Sun and Moon are the inputs consumed by the solid-Earth and ocean tides of Section 10 and by the satellite-attitude and phase-wind-up models of Section 11. The two luni-solar series are ∼100 km accurate in position. Their output is in the mean-of-date frame, not J2000, and the ECI-to-ECEF rotation of the legacy chain applies precession again, so on the default path the Sun and Moon directions carry the precession accumulated since J2000 a second time (about 50 arcsec per year, roughly a third of a degree in 2024). `estimation.enable_sun_frame_correction` (default off, byte-identical when off) de-precesses the of-date vectors to J2000 before the rotation and removes that error (`Planetary::precessionIau1976`). The same setting makes the satellite attitude model form the orbit normal from the inertial velocity (Section 11.1). A non-finite guard substitutes a nominal 1 AU / mean-lunar-distance fallback rather than propagate a NaN into the correction stack (the output-validation tail of `Planetary::computeSunMoonPosition`).

7 Terrestrial Reference Frames, Epochs, and Datum Transformation

A terrestrial coordinate is three numbers, a frame, and an epoch. The engine works in ITRF at the epoch of observation, so the frame and the epoch are usually implicit. They become explicit when a result is compared with a national datum, a published datasheet, or a passive mark. Changing the frame and changing the epoch are different operations, and mixing them up produces decimetre-level offsets. The rule is:

$$\boxed{\textbf{Frames take a Helmert. Epochs take a velocity.}} \tag{47}$$

Sections 7.1 to 7.3 develop the rule and its one exception. Sections 7.4 and 7.5 give the sign conventions and the published parameter sets. Section 7.6 covers NATRF2022. Sections 7.7 and 7.8 cover the NGS Site Information File and the monument-to-ARP offset. Section 7.9 states where the engine reads station coordinates and how a weekly SINEX supplies roster coordinates. Section 7.10 is a numerical example at CORV, and Section 7.11 compares the NGS deformation model with station velocities.

7.1 Frame versus epoch

A Helmert similarity (seven parameters, or fourteen with their linear rates) maps frame A to frame B *at the same epoch*. It describes how two realisations differ in origin, orientation, and scale. It says nothing about how the ground moves.

The fourteen-parameter form is

$$p(t) \;=\; p_0 \;+\; \dot{p}\,(t - t_0), \qquad p \in \{\boldsymbol{T},\, s,\, \boldsymbol{\varepsilon}\}, \tag{48}$$

and the rates $\dot{p}$ are rates *of the parameters*: the drift of one frame relative to the other. They are not station velocities. Evaluating (48) at epoch t gives the transformation from A to B at that epoch. It does not move the coordinate through time.

Moving a coordinate between epochs requires information about the site:

$$\boldsymbol{x}(t) \;=\; \boldsymbol{x}(t_0) \;+\; \boldsymbol{v}\,(t - t_0) \;+\; \boldsymbol{d}_{\mathrm{per}}(t) \;+\; \big[\boldsymbol{d}_{\mathrm{PSD}}(t) - \boldsymbol{d}_{\mathrm{PSD}}(t_0)\big], \tag{49}$$

with $\boldsymbol{v}$ a station velocity, $\boldsymbol{d}_{\mathrm{per}}$ the periodic (annual + semiannual) terms of one block (Section 7.7), and $\boldsymbol{d}_{\mathrm{PSD}}$ the post-seismic deformation accumulated since the earthquakes. The reference epoch t_0 is that of the selected occupation record. The reference coordinate $\boldsymbol{x}(t_0)$ is the model position with $\boldsymbol{d}_{\mathrm{PSD}}(t_0)$ already in it (the SIF coordinates at 2020.0 exceed the ITRF2020 secular positions by that amount), so only the change since the reference epoch is added. The post-seismic term is the sum over earthquakes e of the logarithmic and exponential relaxations since the event epoch t_e, evaluated per local east, north and up component and zero for $t \le t_e$:

$$\boldsymbol{d}_{\mathrm{PSD}}(t) \;=\; \sum_{e} \Big[\sum_{j} A^{\log}_{e,j}\, \ln\!\Big(1 + \frac{t - t_e}{\tau^{\log}_{e,j}}\Big) \;+\; \sum_{j} A^{\exp}_{e,j} \Big(1 - e^{-(t-t_e)/\tau^{\exp}_{e,j}}\Big) \Big]. \tag{50}$$

The bracketed change in (49) is zero at t_0, and an event after t_0 enters in full. `resolveStationCoordinate` and `tools/sif_propagate` implement this form. The three site terms are observed or modelled. A transformation parameter set does not contain them, so a transformation that appears to handle an epoch change is using the rigid-plate approximation of Section 7.3.

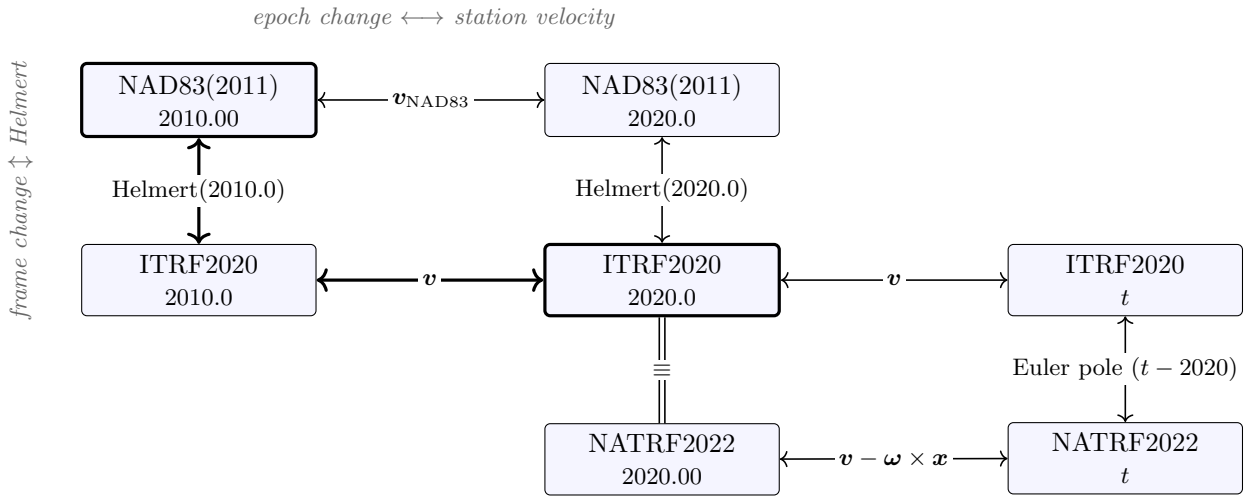


Figure 1: Frames (rows) and epochs (columns). Vertical moves change the frame at a fixed epoch; horizontal moves change the epoch within a frame and need a station velocity. Heavy lines mark the worked example of Section 7.10, from ITRF2020 at 2020.0 to NAD83(2011) at 2010.00. Because NAD83 rotates with the plate, $\boldsymbol{v}_{\mathrm{NAD83}}$ is small on the stable interior and larger at the plate margins (Section 7.3).

7.2 Transforming a velocity

Differentiating the linearised transformation with respect to time gives

$$\boldsymbol{v}_B \;=\; \boldsymbol{v}_A \;+\; \dot{\boldsymbol{T}} \;+\; \dot{s}\,\boldsymbol{x} \;+\; \dot{\boldsymbol{E}}\,\boldsymbol{x}. \tag{51}$$

Three properties of (51) matter in practice.

The rate field is rigid. The added terms $\dot{\boldsymbol{T}} + \dot{s}\boldsymbol{x} + \dot{\boldsymbol{E}}\boldsymbol{x}$ depend on the station's *position* only, not on its motion. This was checked by transforming fictitious stations at one position with different velocities: the added velocity was identical to the micron. The rates of a fourteen-parameter set are therefore a rigid velocity field, just as the seven parameters are a rigid displacement field.

Propagation and transformation commute, with one condition. Propagating in frame A and then transforming gives the same result as transforming and then propagating in frame B, to 0.000 mm, provided each propagation uses the velocity expressed in its own frame. That velocity comes from (51).

Using the wrong frame's velocity is a large error. Propagating a NAD83(2011) coordinate with an ITRF velocity costs about 20 mm/yr at CORV, or 203 mm per decade. No single-epoch check reveals it. The error exceeds every geophysical correction of Section 10 combined.

7.3 The rigid-plate approximation

The parameter rates can stand in for a station velocity in one case. A plate-fixed frame is built so that stations on the rigid plate do not move in it. Setting $\boldsymbol{v}_B = \boldsymbol{0}$ in (51) gives

$$\boldsymbol{v}_A \;=\; -\big(\dot{\boldsymbol{T}} + \dot{s}\,\boldsymbol{x} + \dot{\boldsymbol{E}}\,\boldsymbol{x}\big). \tag{52}$$

The right-hand side is then a *modelled* rigid-plate velocity, and (48) is being used as a one-term deformation model.

The approximation is as good as the assumption. On stable North America the horizontal velocity relative to the plate is 1–2 mm/yr, so (52) costs 1–3 cm per decade. At a plate margin it fails. CORV sits on the deforming Cascadia margin, and the fourteen-parameter-only reconstruction of Section 7.10 misses by 104 mm per decade.

7.4 Rotation-sign conventions

Two rotation conventions are in use, and the sources cited here use both. The NGS / EPSG:1056 *coordinate frame* form, linearised for small rotations, is

$$\boldsymbol{x}' \;=\; \boldsymbol{x} \;+\; \boldsymbol{T} \;+\; s\,\boldsymbol{x} \;+\; \boldsymbol{E}\,\boldsymbol{x}, \qquad \boldsymbol{E} \;=\; \begin{bmatrix} 0 & \varepsilon_z & -\varepsilon_y \\ -\varepsilon_z & 0 & \varepsilon_x \\ \varepsilon_y & -\varepsilon_x & 0 \end{bmatrix}. \tag{53}$$

The IERS / IGN *position vector* form is identical except that $\boldsymbol{E}$ carries the opposite sign throughout; equivalently, $\boldsymbol{\varepsilon} \to -\boldsymbol{\varepsilon}$.

Using one convention's parameters in the other convention's matrix applies the rotation backwards. For a rotation angle θ and a position of magnitude L the error is $2|\theta|L$, twice the size of omitting the rotation. The same holds for the rate field of Section 7.2. A rate field derived here with the wrong convention came out exactly negated, a $\sim$35 mm/yr error at CORV where the true field is $\sim$ 20 mm/yr. Check the sign of a rate field against a station of known velocity; a plausible magnitude is not evidence of a correct sign.

7.5 Published parameter sets and their composition

Table 3 lists the three sets the tooling uses. The first is in position-vector convention and the other two are in coordinate-frame convention. The IGN rotations are zero, so the composition below can add the sets without converting between the conventions of Section 7.4.

PROJ applies the pre-composed EPSG:10334 set for `EPSG:9988`→`EPSG:6317`, with `t_epoch`= 2010 and the `coordinate_frame` convention. It can be rebuilt from the other two sets in two steps:

1. **Run the IGN set to the NGS epoch.** Evaluate (48) at 2010.0 from its own $t_0 = 2015.0$: $\boldsymbol{T}(2010) = \boldsymbol{T}(2015) + \dot{\boldsymbol{T}} \cdot (-5) = (-1.4,\ -0.4,\ +0.4)$ mm.

2. **Add term by term** to the NGS set at its native 2010.0.

The result matches PROJ's set: the largest difference over the fourteen parameters is 4×10^{-15} in the respective units. Adding the sets is valid because the cross terms of the exact composition are below a micron at Earth radius. The rotation-times-translation term $\boldsymbol{\Omega T}$ is $\sim$0.3 µm and the rotation-squared term $\Omega_1\Omega_2 R$ is $\sim$0.1 µm. The two-step chain and the pre-composed set agree to 0.0000 mm at 2010.0, 2015.0, and 2020.0.

Two conditions apply. First, both sets must be in the same rotation convention before they are added. That step is unnecessary here only because the IGN rotations are zero. Second, step 1 is required: adding the IGN set at its own 2015.0 reference epoch introduces a 0.5 mm error in Y and 1.0 mm in Z.

The EPSG codes are ITRF2020 `9988` / `9990` / `9989` (geocentric XYZ, 2D, 3D), ITRF2014 `7789` / `9000` / `7912`, and NAD83(2011) `6317` / `6318` / `6319`.

Table 3: Published Helmert parameter sets. ITRF2020→ITRF2014 is IGN's, in position-vector convention; the two NAD83 sets are NGS/EPSG, in coordinate-frame convention. Rotations $\boldsymbol{\varepsilon}$ in milliarcseconds.

	ITRF2020→ITRF2014 IGN, position-vector $t_0 = 2015.0$	**ITRF2014→NAD83(2011)** NGS MYCS2, EPSG:8970 $t_0 = 2010.0$, coord-frame	**ITRF2020→NAD83(2011)** EPSG:10334 (pre-composed) $t_0 = 2010.0$, coord-frame
T_x	−1.4 mm	1.00530 m	1.00390 m
T_y	−0.9 mm	−1.90921 m	−1.90961 m
T_z	+1.4 mm	−0.54157 m	−0.54117 m
s	−0.42 ppb	0.36891 ppb	−0.05109 ppb
ε_x	0	26.78138 mas	26.78138 mas
ε_y	0	−0.42027 mas	−0.42027 mas
ε_z	0	10.93206 mas	10.93206 mas
$\dot{T}_x$	0.0 mm/yr	0.00079 m/yr	0.00079 m/yr
$\dot{T}_y$	−0.1 mm/yr	−0.00060 m/yr	−0.00070 m/yr
$\dot{T}_z$	+0.2 mm/yr	−0.00144 m/yr	−0.00124 m/yr
$\dot{s}$	0	−0.07201 ppb/yr	−0.07201 ppb/yr
$\dot{\varepsilon}_x$	0	0.06667 mas/yr	0.06667 mas/yr
$\dot{\varepsilon}_y$	0	−0.75744 mas/yr	−0.75744 mas/yr
$\dot{\varepsilon}_z$	0	−0.05133 mas/yr	−0.05133 mas/yr

7.6 NATRF2022

NATRF2022, the North American frame of NSRS2022, is ITRF2020 rotated about the North America Euler pole. It has no translation and no scale. The two frames are identical at epoch 2020.00 and separate as the plate moves. With $\boldsymbol{\omega}$ the Euler vector, positive counter-clockwise under the right-hand rule,

$$\boldsymbol{x}_{\mathrm{NATRF}}(t) = \boldsymbol{x}_{\mathrm{ITRF2020}}(t) \;-\; (\boldsymbol{\omega} \times \boldsymbol{x})\,(t - 2020.0), \tag{54}$$

$$\boldsymbol{v}_{\mathrm{NATRF}} = \boldsymbol{v}_{\mathrm{ITRF2020}} - \boldsymbol{\omega} \times \boldsymbol{x}. \tag{55}$$

The NGS BETA EPP2022 Euler poles are, in mas/yr, $\boldsymbol{\omega}_{\mathrm{NA}} = (0.046, -0.704, -0.047)$, with the companion plates $\boldsymbol{\omega}_{\mathrm{PA}} = (-0.409, 1.063, -2.188)$, $\boldsymbol{\omega}_{\mathrm{CA}} = (-0.056, -0.957, 0.589)$, and $\boldsymbol{\omega}_{\mathrm{MA}} = (-8.089, 5.937, 2.159)$ for PATRF2022, CATRF2022, and MATRF2022 respectively. These are BETA values from `https://beta.ngs.noaa.gov/NATRF2022/`. Fetch the current values before production use.

The sign of (55) was checked on stable-interior stations, whose plate-fixed velocity must be near zero. The NATRF2022 velocities, ENU in mm/yr, are AMC4 (−0.49, 1.01, −1.17), GODN (−0.12, 0.47, −1.49), TXAU (−0.34, 1.73, −0.52), WES2 (0.07, −0.03, −0.12), and MIA3 (0.28, 0.90, 0.56). A sign error would double the plate rate, giving 20–30 mm/yr here.

PROJ 9.5.1 has no NATRF2022 CRS, so the caller must apply (54) directly.

7.7 SIF occupations and the datasheet convention

The NGS Site Information File (SIF) gives ITRF2020 coordinates at epoch 2020.0 from the MYCS3 solution. A station can have several **occupation** windows, each with its own reference coordinate, velocity and reference epoch (2020.0 in the bundled files, except four flagged site-log records at 2010.0; `tools/sif_propagate/README.md`). An occupation boundary is an equipment change or a known offset, so the coordinate series is discontinuous there. Two consequences follow.

The datasheet convention. NGS produces a datasheet's NAD83(2011) epoch-2010.00 coordinate as follows. Take the *latest* occupation. Propagate its reference coordinate back ten years with its linear velocity and **no** periodic term. Apply the fourteen-parameter transformation of Section 7.5 at 2010.0. This recipe reproduces the datasheet to 0.4 mm at CORV (PID AH2488) and 0.6 mm at CHZZ (PID AJ6961). In `tools/sif_propagate` it is the `--datasheet` option.

The engine convention. `resolveStationCoordinate`, and `tools/sif_propagate` by default, select the occupation that *covers the requested epoch*; when no usable window covers it, the window whose midpoint is nearest is extrapolated and the result says how far and whether an antenna or receiver change lies in the gap. A coordinate for an observation session must describe the equipment installed at that time. At CORV a request for 2010.0 uses the 1998–2019 occupation and differs from the datasheet by 32.8 mm. Of that, $\delta U = -32.4$ mm is the jump at the 2019-07-24 antenna change (ASH700936E_C → TRM59800.99 SCIS). Both coordinates are correct for their purpose. One rule follows:

Remark 1. The difference of two resolved coordinates across an occupation break is *not* a velocity. It includes the equipment offset, which is often the larger part.

The seasonal term is one block, chosen by the epoch. A station carries periodic blocks, each with a window, a reference epoch t_b and sinusoids of periods P_f (annual and semiannual) in ECEF x, y and z. The seasonal displacement is that of a single block, chosen by the epoch alone and not by the occupation:

$$\boldsymbol{d}_{\mathrm{per}}(t) \;=\; \sum_{f} \Big[\boldsymbol{A}_f^{\cos} \cos\big(\omega_f (t - t_b)\big) + \boldsymbol{A}_f^{\sin} \sin\big(\omega_f (t - t_b)\big) \Big], \qquad \omega_f = \frac{2\pi}{P_f}. \tag{56}$$

The blocks are taken in start order. The first block whose window covers the epoch applies; before the first window, the first block; at or after the end of the last window, the last block; inside a gap, the block that ended last. This is the rule of NGS M-PAGES v1.2 (`PeriodicMotion::computeDisplacement`), implemented by `selectPeriodicBlock`. Overlapping windows are not summed, and the file order of the blocks is irrelevant. Outside its window the block continues on the phase of its own reference epoch, so the seasonal term does not fall to zero where the linear term goes on. The rule replaced a sum over the covering blocks, under which a station whose last segment had closed lost its seasonal term at once, a step of up to several millimetres. The result carries which case applied (`SifPeriodicReach`: covered, before first, after last, in gap), the signed distance outside the block, and its window.

Periodic terms are a constant offset at whole-year epochs. The SIF annual and semiannual terms are referenced to the reference epoch of the block. A whole number of years from that epoch, cos = 1 and sin = 0, so the terms add a fixed vector: 0.65 mm at CORV, 1.73 mm at NEWP, 3.18 mm at CHZZ. The offset cancels in a difference between two such epochs but not in an absolute comparison. Published coordinates exclude it, so a propagated coordinate that includes it differs from the datasheet by these amounts.

7.8 Monument versus ARP

A datasheet comparison must also use the same reference point. SIF and datasheet coordinates refer to the **monument** (ground reference point, GRP). The engine's position state and the NGS CORS coordinate files refer to the **ARP**. The offset between them is the RINEX `ANTENNA:DELTA`

H/E/N eccentricity, which the SIF publishes as ARP_MINUS_GRP_ENU. The reduction is developed in Section 6.2.

At CORV, $\delta H = 1.521$ m, and both comparisons close:

- SIF monument + 1.521 m Up versus the published ITRF2020 ARP (−2498424.007, −3802820.797, 4454737.751): 0.3 mm.
- Datasheet + 1.521 m Up versus the published NAD83(2011) ARP (−2498423.096, −3802822.020, 4454737.741): 0.5 mm.

Omitting this step at CORV gives an obvious 1.5 m height error. The harder case is a site with an eccentricity of a few centimetres, where the error is small enough to go unnoticed.

7.9 Station coordinates in the engine

The SIF registry and the single-receiver a priori. The SIF subsystem is off by default (processing.enable_station_info). The registry is read from files.station_info_path or files.station_info_paths, or, when neither is set, from every JSON file under static/sif/ of the data root. A station is found by its nine-character identifier, its DOMES number or its four-character prefix. A prefix match is a candidate only: it is accepted when the receiver's approximate position lies within 10 km of the candidate, or, without a position, for a bare four-character key (StationInfoFile::find, SIF_MAX_PREFIX_DISTANCE_M). With the subsystem on and the marker resolved, the single-receiver a priori position is the SIF coordinate at the first observation epoch, with the periodic and post-seismic terms, reduced from the monument to the ARP by reduceSifCoordinateToArp: through the RINEX ANTENNA:DELTA H/E/N record when the header carries one, and otherwise through the ARP_MINUS_GRP_ENU of the SIF antenna window covering the epoch. A disagreement of the two eccentricities above 1 mm is reported and the RINEX value is applied. A configured reference position takes precedence over the SIF seed. The reduction is the one of Section 6.2.

The network roster. With network.use_sif_coordinates, a roster station that has no position_ecef takes the SIF coordinate at the run date (data_download.year and doy), through resolveRosterCoordinate. An explicit position_ecef always wins. The declared coordinate epoch defaults to the propagation date. An extrapolated occupation and an extrapolated periodic block are each reported per station, so a coordinate that is held fixed for a run carries its own warning when it rests on a window that closed. The resolved roster coordinate is a monument coordinate and takes the reduction of network.station_marker_to_arp like any other.

Roster coordinates from an IGS weekly SINEX. No engine setting reads a SINEX. A weekly combined SINEX holds one marker position per station, at the middle of the week, and no velocity rows. scripts/roster_from_sinex.py reads it and writes, for every roster station the file holds, the marker coordinate of the solution that covers the epoch. A station the file lacks keeps its configured coordinate and is named, and a SINEX position more than 1 km from the configured one is refused as a reused station code. scripts/rapid_arm.py make --roster splices the result into network.stations[].position_ecef of a case configuration, so the engine sees ordinary configured coordinates. The SIF evaluator and the weekly SINEX agree within 3.1 mm per horizontal component at seven of the eight stations of the real-data roster with an active post-seismic term (tools/sif_propagate/README.md). Replacing the roster by its SINEX coordinates moved the emitted satellite clocks by 5 to 7 mm RMS and no test station by more than 2.4 mm (docs/

refound/SNXB_sinex_roster_arms_2026-10-19.md); nothing is armed, and a coordinate-source setting was not built.

7.10 Worked example: CORV

This example works every step above with numbers, at the monument, using the latest occupation (2019-08-04 to open). tools/sif_propagate/corv_worked_example.py reproduces it.

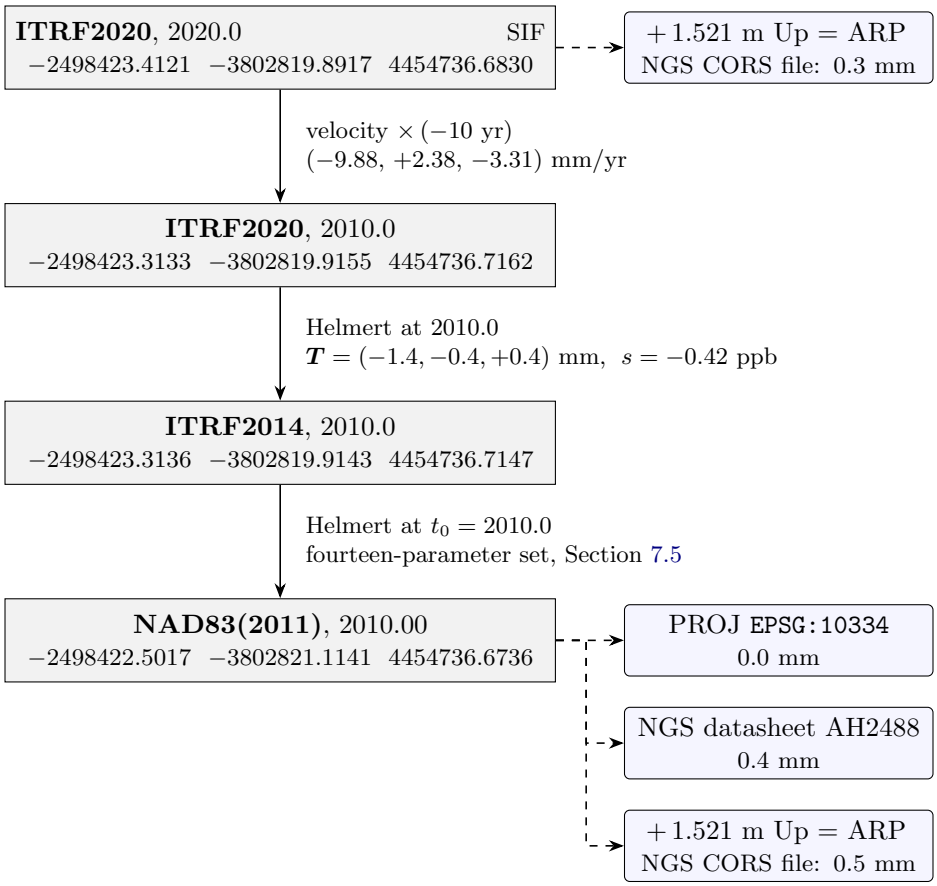


Figure 2: The CORV worked example, modern → legacy, at the monument (metres). Solid arrows are the computation; dashed arrows are independent checks, with the 3-D closure. The antenna reference point lies 1.521 m above the monument (RINEX DELTA H).

Starting point (SIF, ITRF2020 @ 2020.0). $\boldsymbol{x}$ = (−2498423.4121, −3802819.8917, 4454736.6830) m, h = 105.5881 m; velocity in ECEF (−9.8836, 2.3812, −3.3134) mm/yr, i.e. ENU (−9.57, −4.77, +0.12) mm/yr.

Step 2a: propagate to 2010.0 in ITRF2020. $\boldsymbol{x} - 10\boldsymbol{v}$ = (−2498423.3133, −3802819.9155, 4454736.7162) m; the station moved 106.9 mm.

Step 2b: ITRF2020 → ITRF2014 at 2010.0. With $\boldsymbol{T}$ = (−1.4, −0.4, +0.4) mm (the IGN set run to 2010.0) and s = −0.42 ppb: (−2498423.3136, −3802819.9143, 4454736.7147) m.

Step 2c: ITRF2014 → NAD83(2011) at 2010.0. Translation (1.00530, −1.90921, −0.54157) m; rotation term (−0.19247, 0.71082, 0.49885) m; scale term (−0.00092, −0.00140, 0.00164) m. Result (−2498422.5017, −3802821.1141, 4454736.6736) m. PROJ gives the same result (0.0 mm).

Closure against the datasheet. NGS datasheet AH2488 gives $\varphi = 44^\circ\,35'\,07.91300''$ N, $\lambda = 123^\circ\,18'\,16.51681''$ W, $h = 105.953$ m, i.e. (−2498422.5015, −3802821.1143, 4454736.6734) m. Hand computation minus datasheet is $\delta E = -0.3$, $\delta N = +0.2$, $\delta U = +0.1$ mm, which is 0.4 mm in three dimensions.

The reverse direction. Going from the datasheet to ITRF2020 at 2020.00, which is also NATRF2022 at that epoch (Section 7.6), requires ten years of station velocity. The miss against the SIF coordinate depends on the source of that velocity:

Table 4: CORV reverse reconstruction: miss against the SIF coordinate by velocity source. The last row is the rigid-plate approximation of Section 7.3, which does not hold at CORV.

Velocity source	**3D miss**	**Dominant component**
SIF velocity	0.4 mm	$\delta E = 0.3$
IFDM2022 grid (−8.68, −4.78, +0.04) mm/yr ENU	9.3 mm	$\delta E = 9.2$
BETA NCAT	9.6 mm	δE 6.9, δN − 5.4, δU 4.0
14-parameter only ($\boldsymbol{v}_{\mathrm{NAD83}} := \mathbf{0}$)	104.2 mm	δE − 54.6, δN − 88.2, δU 10.4

The same point in four frames. CORV velocity, ENU mm/yr: ITRF2020 (−9.57, −4.77, 0.12); plate motion $\boldsymbol{\omega} \times \boldsymbol{x}$ (−13.20, −13.12, −0.04); NATRF2022 (3.63, 8.35, 0.17); NAD83(2011) (5.49, 8.80, −1.05). The NATRF2022 velocity is what remains after removing rigid North America. At a deforming margin it is not small: a NATRF2022 coordinate at CORV moves (21.8, 50.1, 1.0) mm by 2026.0 and (36.3, 83.5, 1.7) mm by 2030.0 from its 2020.00 value. A plate-fixed coordinate is not a static one.

7.11 Deformation models and validation

A passive mark has no observed velocity, so a model must supply one. NGS supplies IFDM2022 with BETA NCAT. It is a global 0.1° ENU velocity grid with per-node standard deviations, in ITRF2020 (`epsg:9990`), plus 33 earthquake displacement grids. It was compared with SIF velocities at 21 stations in CONUS, Alaska, and British Columbia, using one occupation per station:

- $\|\boldsymbol{v}_{\mathrm{IFDM}} - \boldsymbol{v}_{\mathrm{SIF}}\|$: median 1.34 mm/yr, maximum 3.11 mm/yr.
- The grid's formal σ is optimistic by roughly a factor of two (median ratio 2.1, up to 5.9 in the stable interior). The comparison is *in-sample*, because the grid was built from these CORS. A passive mark should expect a larger error.
- In the stable interior the IFDM horizontal velocity equals the EPP2022 plate motion to 0.01 mm/yr. The model assigns zero intraplate velocity there.
- Velocity spread across a single grid cell is 0.03–0.05 mm/yr in the interior and up to 3.7 mm/yr in the Aleutians. Interpolation error matters only where the field is steep.

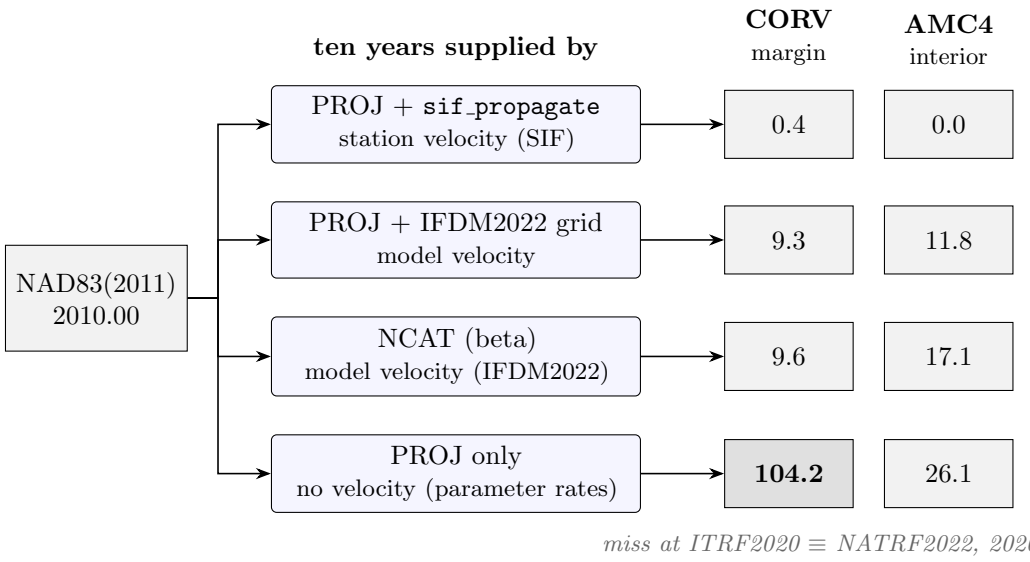


Figure 3: Legacy → modern, NAD83(2011) epoch 2010.00 to ITRF2020 ≡ NATRF2022 epoch 2020.00, by four routes. PROJ is the inverse of `EPSG:10334`, evaluated at 2010.0 (at 2020.0 in the last row, where it stands alone). The number is the 3-D miss against the SIF coordinate at 2020.0. CORV starts from NGS datasheet AH2488; AMC4 (Colorado Springs, single occupation) starts from its SIF-derived 2010.00 coordinate, so its first row is zero by construction.

NCAT's transformation from NAD83(2011) at 2010.00 to NATRF2022 at 2020.00 was compared with the SIF coordinate at 2020.0. The mean difference is (+4.9, −6.1, −1.3) mm and the median 3D difference is 18 mm, within NCAT's stated uncertainty of about 2 cm for most stations. The small mean confirms that NATRF2022 and ITRF2020 coincide at 2020.00 to a few millimetres, as Section 7.6 requires. Five stations are outliers: FAIR 115, SNI1 82, AB02 78, KYW1 61, and HOLP 43 mm. Four of them lie inside IFDM earthquake displacement grids. The SIF represents those events as occupation *breaks* rather than PSD terms, so the two sources model them differently. This attribution rests on grid coverage only. The displacement magnitudes were not evaluated, and KYW1 is unexplained.

Limitations of beta NCAT. It offers only fixed epochs: NAD83(2011) at 2010.00 and NATRF2022 at 2020.00. Its offline CLI performs coordinate *conversions* only, not datum transformations. Its XYZ input is rounded to 1 mm. The post-seismic terms of both tools are untested here, because no station in the comparison has a PSD term.

Mixing velocity sources. Within NSRS2022 the deformation model is the convention, so using IFDM (or HTDP) for a passive mark is correct by definition. The risk is mixing sources. A network that propagates its CORS with SIF velocities and its marks with grid velocities introduces a 1–2 mm/yr difference between the two groups, which distorts the network over a decade. The grid also gives only a diagonal covariance, and an optimistic one by the ratio above. SIF propagation gives the full

$$\boldsymbol{C}_{xx}(t) \;=\; \boldsymbol{C}_{xx} \;+\; \Delta t\,\big(\boldsymbol{C}_{xv} + \boldsymbol{C}_{xv}^{\mathrm{T}}\big) \;+\; \Delta t^{2}\,\boldsymbol{C}_{vv}, \tag{57}$$

including the position–velocity cross-covariance, which dominates (57) at short Δt and which the grid does not provide.

8 Orbit and Clock Product Evaluation

Every processing mode in the engine (post-processed PPP, PPP-RTK, the baseline modes, and the real-time engine) needs a position $\boldsymbol{r}^i$, velocity $\dot{\boldsymbol{r}}^i$, and clock bias δt^i for each satellite at each query epoch. These are evaluated *before* any state-space refinement. In post-processing they are final. In the real-time engine they are the broadcast values that the SSR overlay (Section 18) corrects with a RAC orbit delta and a scalar clock delta. This document derives the two precise interpolation paths, the two broadcast evaluation paths, and the ephemeris selection that feeds them. It stops where the SSR delta is added: the RAC frame, the IOD gate, and the freshness checks are in Section 18. The Sagnac (Earth-rotation) range correction, which makes $\boldsymbol{r}^i$ at transmit time consistent with the line of sight, is in Section 11. The ERP and polar-motion transforms between the terrestrial and celestial frames are in Section 6.

There is a *precise* path and a *broadcast* path. The precise path *interpolates* tabulated SP3 orbit samples and CLK clock samples (IGS Final / Rapid products). It is the post-processing default and is accurate to centimetres or better. The broadcast path evaluates a Keplerian ephemeris from the navigation message, or for GLONASS a numerically integrated PZ-90 ephemeris. It is accurate to metres and is what the live SSR stream corrects. By default the two paths do not mix for one satellite at one epoch: a loaded precise product that spans the query is used, and otherwise the engine falls back to the broadcast evaluation (`src/core/model/ModelGeometry.cpp`, `src/core/corrections/GnssCorrections.cpp`). The choice is made per quantity, though. The real-time RTB-4 gate `realtime.use_precise_orbits` selects the orbit and clock sources independently (`GnssConfig::realtimeUsesLiveOrbitSource` vs `realtimeUsesLiveOrbitClockSource`, `ProductParser::loadProducts(bool, bool)`). With that gate armed, an "ultra-rapid orbits + network clocks" configuration uses a precise interpolated orbit *and* a broadcast+SSR clock for the same satellite and epoch. The flag defaults to false. The orbit gate then returns the same value as the clock gate, the result is byte-identical, and the no-mixing statement holds for every non-real-time mode and for the default real-time path.

8.1 Precise orbit: derotated Neville interpolation

SP3 files tabulate the satellite position in the ECEF frame on a coarse grid (typically 900 s for IGS Final, 300 s for Rapid). The ECEF trajectory is *not* a smooth low-degree polynomial. The rotating frame adds an oscillation at the sidereal frequency, and a degree-10 fit cannot follow it to better than a few millimetres near the SP3 epoch boundaries. The engine therefore *derotates* each node into a common near-inertial frame anchored at the query epoch before interpolating (`src/core/model/ModelGeometry.cpp`, following the RTKLIB / cssrlib convention [35]). For a node sampled at t_i and a query at t_q, with the Earth-rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}$ rad/s (`Constants::EARTH_ANGULAR_VELOCITY`), the in-plane coordinates are rotated by the angle the frame swept between the two epochs:

$$\boldsymbol{r}^i_{\mathrm{I}}(t_i) = \boldsymbol{R}_z(\omega_\oplus\,(t_i - t_q))\;\boldsymbol{r}^i(t_i), \qquad \boldsymbol{R}_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{58}$$

Concretely, with $\alpha_i = \omega_\oplus\,(t_i - t_q)$ the derotated in-plane components written to the interpolation arrays are $\tilde{x}_i = \cos\alpha_i\, x_i - \sin\alpha_i\, y_i$ and $\tilde{y}_i = \sin\alpha_i\, x_i + \cos\alpha_i\, y_i$, with z unchanged (`ModelGeometry.cpp`). The rotation angle is zero at $t_i = t_q$, so the interpolated value at the query is already the ECEF position at t_q and needs no rotation back.

Bracket-centred window. The choice of node window determines whether the interpolated position is *continuous*. The engine selects an IGS_POINTS = 11-node window centred on the *bracket* $[t_a, t_{a+1}]$ that contains t_q, not the 11 nodes nearest to t_q (`ModelGeometry.cpp`). A nearest-N rule changes a node at the *midpoint* between SP3 epochs. There the two candidate polynomials disagree by a few millimetres, which produces the known 15-minute sawtooth in PPP position, synchronous across constellations. A bracket-centred rule changes the window when t_q crosses a node t_{a+1}. That node belongs to both windows, so the Neville value is the same on either side and the orbit is continuous. A gap guard re-picks a narrower MIN_POINTS = 9-node window and rejects the satellite if the residual span still exceeds the fit interval (thresholds in `ModelGeometry.cpp`; tuning values cited, not reproduced). The "fit interval" is not read from the product header. It is the *median* of the leading consecutive-node spacings (`ModelGeometry.cpp`). The reason is that a multi-file SP3 merge can be non-uniformly sampled. For example, a 5-minute ultra-rapid file concatenated with a 15-minute one keeps the finer file's off-grid nodes, because the 1 s dedup window correctly treats them as distinct (`ProductParser` warns on the mixed-rate concat). The median spacing gives a stable MAX_SPAN guard on such a series, where a nominal grid is not defined.

Neville value and derivative. Within the window the engine runs Neville's algorithm simultaneously for the value and its analytic derivative (`src/core/math/Interpolate.cpp`, `Interpolate::nevilleInterpDeriv`). The node abscissae are shifted to $s_i = t_i - t_q$ to avoid cancellation: raw continuous GPS seconds are $\sim 1.36 \times 10^8$, so an unshifted difference loses ~ 8 digits. The shift changes neither the polynomial value nor its derivative. With the tableau column P_i for the value and D_i for the derivative, the coupled recurrence over degree j (evaluating at the shifted query $s_q = 0$) is

$$P_{i,j} = \frac{(s_q - s_{i+j})\, P_{i,j-1} - (s_q - s_i)\, P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{59}$$

$$D_{i,j} = \frac{(s_q - s_{i+j})\, D_{i,j-1} + P_{i,j-1} - (s_q - s_i)\, D_{i+1,j-1} - P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{60}$$

seeded by $P_{i,0} = y_i$ and $D_{i,0} = 0$; the converged scalars are $P_{0,n-1}$ and $D_{0,n-1}$ (`Interpolate.cpp`). The derivative recurrence (60) is $\partial/\partial s_q$ of the value recurrence (59), so one pass gives both and no finite difference is needed.

Inertial-to-ECEF velocity. The Neville derivative is taken on the *derotated* channels, so it is the velocity in the inertial frame fixed at t_q. The Doppler line-of-sight rate, the orbit-normal attitude, and the relativistic $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ term need the ECEF velocity, which follows by removing the frame rotation $\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \dot{\boldsymbol{r}}^i_{\mathrm{I}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$. For $\boldsymbol{\omega}_\oplus = (0, 0, \omega_\oplus)$ this is the boxed transform applied in `ModelGeometry.cpp`:

$$\boxed{\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \begin{bmatrix} v_{\mathrm{I},x} + \omega_\oplus\, \boldsymbol{r}^i_y \\ v_{\mathrm{I},y} - \omega_\oplus\, \boldsymbol{r}^i_x \\ v_{\mathrm{I},z} \end{bmatrix}}. \tag{61}$$

The relativistic clock term is unaffected, since $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$. Only the Doppler and attitude computations see the difference (`ModelGeometry.cpp`).

8.2 Precise orbit: sidereal-trigonometric LSQ fit

A config-gated alternative (`config.enableTrigSp3Interp()`) replaces the Neville polynomial with an over-determined least-squares fit on a *sidereal-trigonometric basis*, run on the identical derotated channels and time base (`ModelGeometry.cpp`; `Interpolate::trigSvdInterp`). The basis is a constant plus four sine/cosine harmonic pairs of the fundamental sidereal angular rate, nine terms in all:

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\,\Omega_0\,\tau) + c_{2f}\cos(f\,\Omega_0\,\tau)\big], \qquad \tau = t - t_0, \tag{62}$$

where t_0 is the first node. The fitted curve does not depend on this choice, because the constant term absorbs the offset and the sine and cosine pairs absorb the phase. The fundamental rate is the angular frequency of the *mean sidereal day*. With the sidereal-day ratio $\kappa = 0.99726956634$ and a nominal solar day of 86400 s,

$$\boxed{\Omega_0 = \frac{2\pi}{\kappa \cdot 86400\ \mathrm{s}} \quad [\mathrm{rad/s}], \qquad \kappa = 0.99726956634,} \tag{63}$$

expressed *per second*, so τ stays in the caller's seconds and the analytic derivative is per second (`Interpolate.cpp`). M-PAGES uses rad/day and divides the velocity by 86400, which is numerically identical. The constant $\kappa = 0.99726956634$ is the IERS ratio of the mean sidereal day to the solar day. The model depends on it directly, so it is quoted here.

The fit is solved by Eigen's `JacobiSVD` with a relative singular-value threshold of 10^{-15}, which yields the minimum-norm least-squares solution, the Numerical-Recipes `svbksb` convention M-PAGES' `svdfit` uses [27]. Per-node weights derive from the SP3 `++` accuracy codes; because those are a single value per satellite, the weight set is uniform and the fit is effectively unweighted, matching M-PAGES. The analytic value and derivative at the query $\tau_q = t_q - t_0$ are

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau_q) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\Omega_0\tau_q) + c_{2f}\cos(f\Omega_0\tau_q)\big], \tag{64}$$

$$\dot{\boldsymbol{r}}^i{}_{\mathrm{c}}(\tau_q) = \sum_{f=1}^{4}(f\Omega_0)\big[c_{2f-1}\cos(f\Omega_0\tau_q) - c_{2f}\sin(f\Omega_0\tau_q)\big], \tag{65}$$

with the per-harmonic angular weight $w_f = f\,\Omega_0$ (`Interpolate.cpp`). The fit is an *over-determined smoother*. The window has up to 11 nodes against 9 basis terms, so for $N > 9$ the fitted curve does *not* pass through the nodes. This differs from the M-PAGES 9-point exact fit, and is kept so that the trig branch can reuse the derotated window unchanged. The same inertial-to-ECEF velocity conversion (61) is then applied to $\dot{\boldsymbol{r}}^i{}_{\mathrm{c}}$.

8.3 Precise clock interpolation

The CLK product tabulates the scalar satellite clock bias δt^i on its own grid (commonly 30 s). Clock noise behaves like a random walk in bias with occasional rate discontinuities, so the engine offers three interpolation modes, selected by `config.getSatelliteClockInterpolationMode()` (`src/core/corrections/GnssCorrections.cpp`). All three are bracket-centred for the same reason as the orbit window: the window must change at a shared node, never at a midpoint between epochs.

(a) Linear (MMSE for Brownian motion). The default is a two-node linear interpolation across the bracket $[t_0, t_1]$ containing t_q:

$$\delta t^i(t_q) = \delta t_0^i + \left(\delta t_1^i - \delta t_0^i\right) \frac{t_q - t_0}{t_1 - t_0}, \tag{66}$$

which is the minimum-mean-square-error estimator for a Wiener process sampled at the two bracketing epochs. If the gap exceeds a guard threshold the interpolation is rejected (`GnssCorrections.cpp`; gap value cited, not reproduced).

(b) Rate-anchored weighted LSQ. The `LsqSatClock` mode fits a local line $\delta t^i(t) \approx a + b\,(t - t_q)$ over an $\sim$11-node window by weighted least squares, with the centring shift $s_k = t_k - t_q$ making the fitted value at the query equal to the intercept a. The CLK product may also carry a clock *rate* m_j at the coarse 5-minute epochs. Each rate is added as a direct observation of the slope, $m_j = b + \mu_j$. A value equation $y_k = a + b\,s_k + \nu_k$ has $\partial/\partial a = 1$ and $\partial/\partial b = s_k$. A rate equation has $\partial/\partial a = 0$ and $\partial/\partial b = 1$, so the rate observations enter the slope row only. With value weights $w_k^{(y)} = 1/\sigma_{y,k}^2$ and rate weights $w_j^{(m)} = 1/\sigma_{m,j}^2$, the 2×2 weighted normal equations are

$$\boxed{\begin{bmatrix} \sum_k w_k^{(y)} & \sum_k w_k^{(y)} s_k \\ \sum_k w_k^{(y)} s_k & \sum_k w_k^{(y)} s_k^2 \;+\; \sum_j w_j^{(m)} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_k w_k^{(y)} y_k \\ \sum_k w_k^{(y)} s_k y_k \;+\; \sum_j w_j^{(m)} m_j \end{bmatrix}}, \tag{67}$$

solved in closed form for the intercept

$$a = \frac{\left(S_{wxx} + S_m\right) S_{wy} - S_{wx}\left(S_{wxy} + S_{my}\right)}{S_w\left(S_{wxx} + S_m\right) - S_{wx}^2}, \tag{68}$$

with the running sums named exactly as in `GnssCorrections.cpp` ($S_w = \sum w_k^{(y)}$, $S_{wx} = \sum w_k^{(y)} s_k$, $S_{wxx} = \sum w_k^{(y)} s_k^2$, $S_{wy} = \sum w_k^{(y)} y_k$, $S_{wxy} = \sum w_k^{(y)} s_k y_k$, $S_m = \sum w_j^{(m)}$, $S_{my} = \sum w_j^{(m)} m_j$). If the determinant in the denominator vanishes (a window of coincident times) the code falls back to linear interpolation. The estimate $\delta t^i(t_q) = a$ is returned. The fit averages $\sim N$ samples where linear interpolation uses two, so the observation-noise part of the estimate at the bracket midpoint falls by $\sim\sqrt{N/2}$ relative to (66). This matters over seconds to minutes, where CLK observation noise exceeds the per-sample process noise. The cost is that the fit no longer passes through the nodes, so millimetre-level discontinuities appear at the clock epochs when the window moves.

(c) Barycentric Lagrange. For high-rate (1 Hz) products a barycentric-form Lagrange interpolation over a LAGRANGE_POINTS = 9-node window is available (`GnssCorrections.cpp`; `Interpolate::barycentricWeights` / `barycentricEval`). The barycentric form

$$\delta t^i(t_q) = \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}\,\delta t_i^i \;\Bigg/\; \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}, \qquad w_i = \prod_{k \neq i} \frac{1}{t_i - t_k}, \tag{69}$$

is the numerically stable evaluation of the degree-$(n-1)$ Lagrange polynomial [3]. An extrapolation guard falls back to two-node linear interpolation when t_q lies outside the node span. This mode is *not* recommended for standard 30 s IGS Final clocks, because the polynomial oscillates between sparse samples. It is kept for high-rate input only.

8.4 Broadcast Keplerian evaluation

When no precise product spans the query, GPS, Galileo, BeiDou and QZSS are evaluated from their Keplerian broadcast ephemeris (`src/core/products/LiveEphemerisStore.cpp`, `keplerPosition`). The live SSR stream corrects this evaluation. Each system carries its own gravitational parameter μ and Earth-rotation rate $\omega_\oplus$ (`LiveEphemerisStore.cpp`): GPS / QZSS use $\mu = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$; Galileo / BeiDou use $\mu = 3.986004418 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, with BeiDou taking the slightly different $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$. BeiDou is evaluated on its native BeiDou Time (BDT) scale, where BDT week 0 is GPS week 1356 and BDT = GPS − 14 s. Both the query epoch and the stored t_{oe} stay on BDT, so the node-rotation term below uses BDT seconds-of-week (`LiveEphemerisStore.cpp`).

With the semi-major axis $a = (\sqrt{a})^2$ and the time-from-ephemeris $t_k = t - t_{oe}$ reduced into $[-302400, +302400]\,\mathrm{s}$ (week rollover), the corrected mean motion adds the broadcast δn to the Keplerian value, and the mean anomaly propagates linearly:

$$n = \sqrt{\frac{\mu}{a^3}} + \delta n, \qquad M = M_0 + n\,t_k. \tag{70}$$

Kepler's equation $M = E - e\sin E$ is solved by Newton iteration (up to 15 steps, convergence at $|\Delta E| < 10^{-13}$):

$$E_{k+1} = E_k + \frac{M - \big(E_k - e\sin E_k\big)}{1 - e\cos E_k}. \tag{71}$$

The true anomaly and the argument of latitude follow, then the broadcast second-harmonic corrections refine the argument of latitude u, the orbital radius r, and the inclination i:

$$\nu = \operatorname{atan2}\big(\sqrt{1-e^2}\,\sin E,\ \cos E - e\big), \qquad \phi = \nu + \omega, \tag{72}$$

$$\begin{aligned} u &= \phi + c_{us}\sin 2\phi + c_{uc}\cos 2\phi, \\ r &= a\big(1 - e\cos E\big) + c_{rs}\sin 2\phi + c_{rc}\cos 2\phi, \\ i &= i_0 + \dot{i}\,t_k + c_{is}\sin 2\phi + c_{ic}\cos 2\phi. \end{aligned} \tag{73}$$

The corrected longitude of the ascending node includes the node rate *relative to* the rotating Earth and the fixed node offset at the ephemeris epoch:

$$\boxed{\Omega = \Omega_0 + \big(\dot{\Omega} - \omega_\oplus\big)\,t_k - \omega_\oplus\,t_{oe},} \tag{74}$$

where the final $-\omega_\oplus\,t_{oe}$ uses the seconds-of-week of t_{oe} on the native scale (`LiveEphemerisStore.cpp`). With the in-plane position $(x_o, y_o) = (r\cos u,\ r\sin u)$ the ECEF position is the standard rotation through Ω and i:

$$\boldsymbol{r}^i = \begin{bmatrix} x_o\cos\Omega - y_o\cos i\,\sin\Omega \\ x_o\sin\Omega + y_o\cos i\,\cos\Omega \\ y_o\sin i \end{bmatrix}. \tag{75}$$

The additional fixed rotation for BeiDou GEO satellites is not applied, because GEO satellites are rarely used for PPP-RTK. Velocity is a central finite difference of (75), $\dot{\boldsymbol{r}}^i \approx [\boldsymbol{r}^i(t+\tfrac{1}{2}) - \boldsymbol{r}^i(t-\tfrac{1}{2})]/(2\,\Delta t)$ with $\Delta t = 0.5\,\mathrm{s}$, using the same position evaluator (`LiveEphemerisStore.cpp`).

8.5 Broadcast GLONASS: PZ-90 RK4 integration

GLONASS does not broadcast Keplerian elements. It broadcasts an ECEF state vector (position, velocity, and a luni-solar acceleration) valid at t_b, and the ICD prescribes numerical integration of the PZ-90 equations of motion to the query epoch [9]. The engine uses a fixed-step fourth-order Runge–Kutta integrator (`src/core/products/LiveEphemerisStore.cpp`, `glonassState`). The per-axis acceleration combines the central gravitational term, the J_2 oblateness perturbation, the centrifugal and Coriolis terms of the rotating frame, and the broadcast luni-solar acceleration held constant over the integration. Writing $r = \|\boldsymbol{r}^i\|$ and the oblateness coefficient

$$\kappa_{J_2} = \tfrac{3}{2}\, J_2\, \frac{\mu\, a_e^2}{r^5}, \qquad J_2 = 1.0826257 \times 10^{-3}, \tag{76}$$

(with $\mu = 3.9860044 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, $a_e = 6378136\,\mathrm{m}$, $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$, all PZ-90 constants at `LiveEphemerisStore.cpp`), the accelerations are

$$\boxed{\begin{aligned} \ddot{x} &= -\frac{\mu}{r^3}x + \kappa_{J_2}\, x\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 x + 2\omega_\oplus \dot{y} + a_x, \\ \ddot{y} &= -\frac{\mu}{r^3}y + \kappa_{J_2}\, y\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 y - 2\omega_\oplus \dot{x} + a_y, \\ \ddot{z} &= -\frac{\mu}{r^3}z + \kappa_{J_2}\, z\big(3 - 5\,z^2/r^2\big) + a_z, \end{aligned}} \tag{77}$$

where (a_x, a_y, a_z) is the broadcast luni-solar acceleration. The oblateness term on the z axis has the factor $3 - 5z^2/r^2$, against $1 - 5z^2/r^2$ on the in-plane axes. The centrifugal $\omega_\oplus^2$ and Coriolis $2\omega_\oplus$ terms appear only on the in-plane axes, as the PZ-90 rotating frame requires. The integrator uses a maximum step of $h = 30\,\mathrm{s}$, signed toward the query, and shortens the last step so that it ends on t_q. Each step is the classical RK4 update $y \leftarrow y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ (`LiveEphemerisStore.cpp`).

8.6 Broadcast satellite clock

The broadcast satellite clock for the Keplerian systems is the standard second-order polynomial in the time-from-clock-epoch $t - t_{oc}$ (on the native scale, with BeiDou on BDT as for the orbit node term):

$$\boxed{\delta t^i = a_{f0} + a_{f1}\,(t - t_{oc}) + a_{f2}\,(t - t_{oc})^2.} \tag{78}$$

GLONASS instead uses its two-term broadcast model with the 1020-message τ_n (DF123) and γ_n (DF120) about $t_b = t_{oe}$:

$$\delta t^i = -\tau_n + \gamma_n\,(t - t_b) \tag{79}$$

(`LiveEphemerisStore.cpp`). The shipped real-time replay configurations exclude GLONASS from `active_systems`, so they do not use this path. The default `active_systems` and the post-processing configurations enable GLONASS and do use it.

The precise-clock-baseline convention. The broadcast clock (78) *omits* both the periodic relativistic eccentricity term and the group delay (TGD/BGD). This is intentional. The IGS-RTS clock correction is generated against the bare polynomial, so the engine must *not* apply the periodic relativity or TGD before adding the SSR delta. Doing so would count them twice. The periodic relativistic term $-2\,\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i / c^2$ is added later, in the observation-physics path; see Section 11 for its derivation. With this convention the orbit, the bare clock, and the SSR overlay all refer to the same ephemeris.

8.7 Ephemeris selection and IOD bookkeeping

The broadcast paths assume that the correct ephemeris record has been selected. The live store holds a `current` and an optional `previous` record per satellite. The rules below decide which one is evaluated (`src/core/products/LiveEphemerisStore.cpp`).

Freshest-toe within a source. Within one source, the record with the newer reference epoch is kept. An incoming record with an older t_{oe} (a stale re-broadcast) is rejected. A record with the same epoch, or one differing by less than a second, overwrites the held record (`LiveEphemerisStore.cpp`). At query time the store chooses between `current` and `previous` by *closest* t_{oe} on a common GPS-continuous scale. BeiDou's BDT t_{oe} is first converted to GPS seconds (`toeGpsContinuous`). Without that, every BeiDou record would appear $\sim$1356 weeks old and be skipped.

Fit window. A held record is evaluated only within a per-system limit on $|t - t_{\mathrm{oe}}|$, about twice the broadcast fit interval of the system and longest for GPS and QZSS, shortest for GLONASS. Beyond it the satellite is skipped and the caller keeps its prior value, so that a stale record never produces a wrong position (`LiveEphemerisStore::withinFitWindow`).

Selection by transmission time. With `network.apriori_source = sp3_brdc_clock` (SP3 orbits, the broadcast clock, no CLK product; default `sp3`, not set by the reference configurations) the record of satellite s at epoch t is the latest healthy one whose transmission time is at or before t (`network.brdc_select = ttom`). The a priori clock is kept continuous across every cut-over by an offset table,

$$a_s(t) = \mathrm{brdc}_k(t) + O_{s,k}, \qquad O_{s,k} = O_{s,k-1} + \mathrm{brdc}_{k-1}(t_{\mathrm{sw}}) - \mathrm{brdc}_k(t_{\mathrm{sw}}), \tag{80}$$

with t_{sw} the first 30 s node at or after the transmission time of record k and brdc_k the polynomial (78), so the estimator sees no step. The emitted clock refers to the stamped record, $C_{\mathrm{prod}} = C_{\mathrm{est}} + O_{s,k}$ with `iode` $= k$, and a user holding record k reconstructs $\mathrm{brdc}_k(t) + C_{\mathrm{prod}} = a_s(t) + C_{\mathrm{est}}$ (`products/BrdcClockApriori.hpp`). The store is fed in transmission order (`LiveEphemerisStore::insertTransmitted`): the record replaces the current one whatever its t_{oe}, because the freshest-t_{oe} rule above rejects a re-upload whose t_{oe} is earlier than that of the held record.

IGS-source authority over receiver nav. When the engine ingests both an IGS broadcast stream and the receiver's own decoded navigation, the IGS record takes priority. The IGS-RTS SSR delta is computed against the IGS broadcast ephemeris, so the rover must pair the delta with the *same* ephemeris. Even at an identical IOD, the receiver's locally assembled orbit has slightly different fit parameters: a live Galileo I/NAV assembly was observed to differ by more than 100 m. Applying IGS-SSR to a receiver ephemeris therefore corrupts the single-difference observations. A receiver record never overwrites a current IGS record with an equal or older t_{oe}. It is accepted only when it is newer than the IGS record by more than one constellation refresh (1800 s, the GLONASS nominal step), which indicates a *stalled* IGS feed. A dead IGS stream then falls back to receiver navigation and the satellite stays usable (`LiveEphemerisStore.cpp`).

IODE-change demote and SSR re-referencing. When the IOD changes, the `current` record becomes `previous` and is not discarded, so the superseded orbit remains available across the cutover (`LiveEphemerisStore.cpp`). This is needed because the receiver can hold an ephemeris one version ahead of the SSR stream. When an SSR delta names an IOD, `pickForSsr` returns whichever held

record (`current` or `previous`) matches it. The orbit and clock are then evaluated on the ephemeris the correction was computed for, and the satellite is not dropped to broadcast accuracy because `current` and the SSR disagree. Keeping both records also supports a second use of the broadcast clock polynomial (78): `LiveEphemerisStore::ephCutoverShiftMetres` evaluates $c\,(\delta t^{i}_{\text{new}} - \delta t^{i}_{\text{old}})$ across an ephemeris version change (`computeSatelliteClock` run once on the freshest pick and once pinned to the old IODE), and returns nothing when the old version is no longer held. The network producer's `network.apriori_source = brdc_rts` eph-cutover re-level uses this value. That re-level is derived in Section 20.

The GNSS-IOD matching convention. The SSR "GNSS IOD" field (IGS-SSR / RTCM `4076`, IDF012) is an 8-bit quantity whose definition differs by constellation and differs from the raw broadcast IODE stored with the ephemeris. `ssrMatchIode` converts the held record's IODE to the SSR convention, in one place (`LiveEphemerisStore.cpp`):

$$\mathrm{IOD}_{\mathrm{SSR}}(i) = \begin{cases} \text{IODE} & \text{GPS, QZSS (8-bit LNAV, passthrough),} \\ \text{IODnav \& } \texttt{0xFF} & \text{Galileo (low 8 bits of the 10-bit I/NAV IODnav),} \\ \left\lfloor t_{oe}^{\mathrm{sow}}/720 \right\rfloor \bmod 240 & \text{BeiDou (toe-derived, per IGS-SSR v1.00),} \end{cases} \tag{81}$$

with t_{oe}^{sow} the BDT seconds-of-week. The BeiDou broadcast AODE that the decoder stores in the IODE field is a *different* quantity and never matches the SSR field, so the toe-derived value is used instead. The raw IODE is kept for the record bookkeeping above; only the value passed to the SSR gate is converted. Section 18.3 describes how that value gates the SSR orbit and clock delta: the delta is accepted only on an exact IOD match, and otherwise the satellite falls back to broadcast accuracy.

9 Atmospheric Delay Modelling: Troposphere and Ionosphere

The undifferenced code and phase equations, Equation (4) and Equation (5) of the Foundations document, each contain two propagation terms that are computed from *models* rather than estimated directly: the slant tropospheric delay T_r^i and the first-order slant ionospheric delay $\mu_s\, I_r^i$. The troposphere has an a-priori *zenith hydrostatic* part, which is computed and removed before the filter sees the measurement. The small *zenith wet* residual and, when `estimation.estimate_tropo_gradients` is set (default off), two horizontal gradients are estimated (Section 3, Equation (3)). In the uncombined parameterisation the ionosphere is one slant state per satellite (Section 3.4). The models in this chapter supply an optional pseudo-observation that ties the relative slants to a Global Ionosphere Map (GIM), and the small second- and third-order corrections that the $1/f^2$ scaling does not cover. The a priori seed of a slant state is the geometry-free difference of two code bands, not a model of this chapter (Section 14.2). The reference configurations of the network estimator use neither a GIM nor an ionosphere model: the slant states are free (`network.w6_iono_constraint = none`). A spherical-harmonic VTEC surface on one thin shell, or optionally two, co-estimated by the network is retained in the code, default off and refused by the loader beside that setting (Section 9.2.4). The column layout of that surface in the network normal equations is in Section 20. This document derives the mapping functions, the pressure reductions, the thin-shell geometry, the geomagnetic field, and the sign with which each term enters the modelled range.

9.1 The slant tropospheric delay

The neutral atmosphere is non-dispersive at GNSS frequencies. Its delay is the same on code and phase, and no ionosphere-free or geometry-free combination removes it. The standard decomposition splits the slant delay into a hydrostatic ("dry") and a wet zenith delay, each carried to the line of sight by its own elevation mapping function, plus an azimuthal gradient [34]:

$$\boxed{T_r^i = m_{\mathrm{h}}(\varepsilon)\,\mathrm{ZHD} + m_{\mathrm{w}}(\varepsilon)\,\mathrm{ZWD} + m_{\mathrm{g}}(\varepsilon)\big(G_{\mathrm{N}}\cos\alpha + G_{\mathrm{E}}\sin\alpha\big).} \tag{82}$$

ZHD is supplied a priori and removed from the prefit residual. ZWD, G_{N} and G_{E} are estimated states whose Jacobian columns are the mapping coefficients $m_{\mathrm{w}}(\varepsilon)$ and $m_{\mathrm{g}}(\varepsilon)\{\cos\alpha, \sin\alpha\}$ (`addTroposphere` and `addGradients` of `src/core/measurement/ObservationEquation.cpp`, the one term list that serves the network, the user and the state seeds; Equation (3)). The rest of this subsection derives each factor.

9.1.1 Saastamoinen zenith hydrostatic delay

The total surface pressure determines the zenith hydrostatic delay to better than a millimetre, through the Saastamoinen reduction [30] as coded in `src/core/atmosphere/Troposphere.cpp`:

$$\boxed{\mathrm{ZHD} = \frac{0.0022768\ P}{1 - 0.00266\cos 2\phi - 0.00028\,h_{\mathrm{km}}},} \tag{83}$$

with P the total pressure in millibar at the antenna, ϕ the geodetic latitude, and $h_{\mathrm{km}} = h/1000$ the ellipsoidal height in kilometres (`Troposphere::computeSaastamoinenZHD`, denominator `f_lat`). The coefficient $0.0022768\ \mathrm{m\,mbar^{-1}}$ combines the refractivity constant $k_1 = 77.604\ \mathrm{K\,mbar^{-1}}$ with the dry-gas constant and standard gravity. The denominator $f(\phi, h)$ accounts for the variation of gravity with latitude and height.

Unless the a priori source is set to GPT3 and the grid (below) loads, P is taken from a standard-atmosphere barometric reduction of a configured sea-level pressure P_0 and temperature T_0 (`receiver.zhd_pressure_mbar` $= 1013.25$ and `receiver.zhd_temp_kelvin` $= 288.15$ by default; `Troposphere.cpp`, `pressureFromHeightStdAtm`):

$$P(h) = P_0\left(1 - \frac{L\,h}{T_0}\right)^{g/(RL)}, \qquad L = 0.0065\ \mathrm{K\,m^{-1}}, \quad \frac{g}{RL} \approx 5.25588, \tag{84}$$

with $g = 9.80665\ \mathrm{m\,s^{-2}}$ and $R = 287.05\ \mathrm{J\,kg^{-1}\,K^{-1}}$ the specific gas constant of dry air. The base $(1 - Lh/T_0)$ is clamped to $[0.1, 2.0]$ and the result to $[100, 1100]$ mbar as a numerical guard.

9.1.2 The Marini mapping form

Both the hydrostatic and wet mappings use the normalised three-parameter continued fraction of Marini, normalised to unity at zenith (`Troposphere::mariniMapping`):

$$\boxed{m(\varepsilon;\,a,b,c) = \cfrac{1 + \cfrac{a}{1 + \cfrac{b}{1+c}}}{\sin\varepsilon + \cfrac{a}{\sin\varepsilon + \cfrac{b}{\sin\varepsilon + c}}}.} \tag{85}$$

The numerator is the denominator evaluated at $\varepsilon = 90^\circ$ ($\sin\varepsilon = 1$), so $m(90^\circ) = 1$. The hydrostatic and wet mappings, and the different models, differ only in the triple (a, b, c). Two families are implemented.

Niell / GMF coefficients. The default mapping interpolates the Niell tables [23] of (a, b, c) given at five reference latitudes $15^\circ, 30^\circ, 45^\circ, 60^\circ, 75^\circ$, each as a mean plus an annually varying amplitude (the coefficient tables inside `Troposphere::getTroposphereMapping`). The seasonal value is

$$v(\phi, t) = \bar{v}(\phi) - \delta v(\phi)\,\cos\frac{2\pi\,(\mathrm{doy} - \mathrm{doy}_0)}{365.25}, \qquad \mathrm{doy}_0 = \begin{cases} 28 & \phi \geq 0 \\ 28 + 182.625 & \phi < 0 \end{cases} \tag{86}$$

(`Troposphere::getTroposphereMapping`, over the day of year returned by `Troposphere::getDayOfYear`). Two details matter. The amplitude term is *subtracted*. The Southern Hemisphere is put in anti-phase by adding half a year to the phase reference, not by changing the sign. The latitude argument is the absolute latitude, linearly interpolated and clamped to $[15^\circ, 75^\circ]$. The wet coefficients have mean values only, with no annual term.

A height correction lifts the hydrostatic mapping from sea level to the station, identical in the Niell and VMF3 paths (`Troposphere::getTroposphereMapping`):

$$m_\mathrm{h}(\varepsilon, h) = m_\mathrm{h}(\varepsilon, 0) + \left[\frac{1}{\sin\varepsilon} - m(\varepsilon;\, a_\mathrm{ht}, b_\mathrm{ht}, c_\mathrm{ht})\right] h_\mathrm{km}, \qquad \begin{aligned} a_\mathrm{ht} &= 2.53 \times 10^{-5}, \\ b_\mathrm{ht} &= 5.49 \times 10^{-3}, \\ c_\mathrm{ht} &= 1.14 \times 10^{-3}, \end{aligned} \tag{87}$$

with the same Marini form (85) evaluated at the height triple. Only the hydrostatic mapping receives this correction.

VMF3 coefficients. The configuration chooses the mapping and the pressure source independently: `receiver.troposphere_mapping_function` is `gmf` (default; the Niell tables above, whatever the name) or `vmf3`, and `receiver.troposphere_apriori_source` is `standard_atmosphere` (default) or `gpt3`. When configured for VMF3 [18], the wet/hydrostatic a-coefficients come from the GPT3 grid (next paragraph) and the b, c coefficients are synthesised on the fly by a degree- and order-12 spherical-harmonic expansion of their seasonal columns (`src/core/atmosphere/Vmf3.cpp`). A Cartesian-form associated Legendre recursion builds the real and imaginary solid harmonics V_{nm}, W_{nm} at the station co-latitude and longitude, the $91 = \sum_{n=0}^{12}(n+1)$ harmonic terms are accumulated against the stored `Vmf3Coeffs` tables, and each of $b_\mathrm{h}, c_\mathrm{h}, b_\mathrm{w}, c_\mathrm{w}$ is then evaluated as a five-coefficient mean-plus-annual-plus-semiannual harmonic, with all three steps inside `Vmf3::computeMapping`:

$$v(\mathrm{doy}) = A_0 + A_1\cos\omega + B_1\sin\omega + A_2\cos 2\omega + B_2\sin 2\omega, \qquad \omega = \frac{2\pi\,\mathrm{doy}}{365.25}. \tag{88}$$

The synthesised (a, b, c) enter the same Marini fraction (85), followed by the height correction (87). If the grid query fails the code falls back to the Niell path without a message, so one bad sample does not stop the run.

9.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients

The GPT3 5° grid [18] supplies, per cell, the five harmonic coefficients of pressure, temperature, temperature lapse rate, geoid undulation, and the VMF3 hydrostatic and wet a-coefficients (`Gpt3::`

`load`, in `atmosphere/Gpt3`). The file stores the a-coefficients multiplied by 1000 (a_h near 1.2 for a physical 1.2×10^{-3}), and the load converts them to the physical, dimensionless values. A grid whose stored a_h or a_w mean lies outside the range of the shipped file is refused, so a file in other units cannot return mapping coefficients off by orders of magnitude, and any failed load leaves the instance empty and unloaded (`Gpt3::load`). The troposphere model then falls back, with a warning, to the Niell mapping and the standard-atmosphere pressure. Each quantity is the same annual-plus-semiannual harmonic as (88) evaluated per cell (`Gpt3::harmonic`):

$$V(\mathrm{doy}) = a_0 + A_1 \cos\omega + B_1 \sin\omega + A_2 \cos 2\omega + B_2 \sin 2\omega. \tag{89}$$

The grid value sits at the cell's orthometric reference height H_s; it is carried to the station orthometric height $h_\mathrm{ortho} = h - N$ (undulation N) by a barometric pressure reduction and a linear temperature lapse (`Gpt3::query`):

$$P_\mathrm{stn} = P_\mathrm{grid}\big(1 - K_\mathrm{P}\,\Delta h\big)^{n_\mathrm{P}}, \quad T_\mathrm{stn} = T_\mathrm{grid} + \frac{\mathrm{d}T}{\mathrm{d}z}\,\Delta h, \qquad \begin{aligned} K_\mathrm{P} &= 2.26 \times 10^{-5}\,\mathrm{m}^{-1}, \\ n_\mathrm{P} &= 5.225, \end{aligned} \tag{90}$$

with $\Delta h = h_\mathrm{ortho} - H_s$ and $\mathrm{d}T/\mathrm{d}z$ the grid lapse in $\mathrm{K\,km^{-1}}$ applied to Δh in kilometres. The exponent 5.225 is the same $g/(RL)$ ratio as (84), rounded for the grid. Four corners are read and combined by bilinear interpolation in latitude and (wrapped) longitude (also `Gpt3::query`); the resulting P_stn drives the Saastamoinen ZHD of (83) when the a priori source is GPT3 (the VMF3 mapping alone does not select it), and the interpolated $a_\mathrm{h}, a_\mathrm{w}$ drive the VMF3 mapping.

9.1.4 The gradient mapping and a degenerate fallback

The azimuthal gradient in (82) uses the Chen–Herring mapping [5], coded once in `addGradients` of `measurement/ObservationEquation.cpp` and shared by the Jacobian and the predicted observable (`MeasurementModel::buildJacobianRow`):

$$\boxed{m_\mathrm{g}(\varepsilon) = \frac{1}{\sin\varepsilon\tan\varepsilon + 0.0032}, \qquad T^\mathrm{grad} = m_\mathrm{g}(\varepsilon)\big(G_\mathrm{N}\cos\alpha + G_\mathrm{E}\sin\alpha\big).} \tag{91}$$

The partial derivatives $\partial\mathrm{T}/\partial G_\mathrm{N} = m_\mathrm{g}\cos\alpha$ and $\partial\mathrm{T}/\partial G_\mathrm{E} = m_\mathrm{g}\sin\alpha$ are the Jacobian columns of the north and east gradient states (Section 3).

A cold-start guard applies to the *elevation* (hydrostatic and wet) mapping, not to the gradient mapping. When the receiver position is not yet usable (norm below 10^3 m, or non-finite), the geodetic latitude and height needed by the Niell and VMF3 coefficients are unavailable. The hydrostatic and wet mappings then use the geometry-only Black–Eisner form $1/(\sin\varepsilon + a/(\tan\varepsilon + b))$ (the fallback branch of `Troposphere::getTroposphereMapping`), which keeps the mapping defined during cold start.

9.2 The slant ionospheric delay

The ionosphere is dispersive. To first order it advances the phase and delays the code by the same amount, $40.3\,\mathrm{STEC}/f^2$, where STEC is the slant total electron content in $\mathrm{el\,m^{-2}}$. The uncombined model carries one slant state I_r^i per satellite, referenced to GPS L1, and scales it to each signal by the dispersion ratio.

9.2.1 The $1/f^2$ scaling and its sign

For a signal of frequency f on a state referenced to $f_{\text{ref}} = f_{\text{L1}}^{\text{GPS}}$, the dispersion coefficient is

$$\boxed{\mu_s = \frac{f_{\text{ref}}^2}{f_s^2}, \qquad \frac{\partial P_s}{\partial I} = +\mu_s, \qquad \frac{\partial \Phi_s}{\partial I} = -\mu_s,} \tag{92}$$

so the slant-iono Jacobian column is $+\mu_s$ on code and $-\mu_s$ on phase (`addIonosphere` of `measurement/ObservationEquation.cpp`). This defines the μ_s of Equation (1). Every constellation's slant state is referenced to GPS L1, which lets the between-satellite and between-receiver differencing operators (Section 5) cancel the receiver iono datum. Because $f_{\text{ref}} = f_{\text{L1}}$, the L1 signal has $\mu = 1$ and the state is the L1 slant delay in metres.

9.2.2 IONEX / GIM thin-shell slant delay

A Global Ionosphere Map gives vertical TEC on a latitude/longitude grid at a single thin shell of height H above a base radius R_E. The slant delay along a ray at elevation ε follows from single-layer geometry (`computeIppGeometry` in `src/core/atmosphere/SphericalHarmonicIonosphere.cpp`, the shared helper that `GlobalIonosphereMap::slantIonoDelay` and the co-estimated spherical-harmonic VTEC model both call so the two never drift). The zenith angle z' at the ionospheric pierce point (IPP) obeys

$$\sin z' = \frac{R_E}{R_E + H}\cos\varepsilon, \qquad \text{mapping } M(\varepsilon) = \frac{1}{\cos z'}, \tag{93}$$

so that $\text{STEC} = M(\varepsilon)\,\text{VTEC}$ (`GlobalIonosphereMap::slantIonoDelay`). The earth-centred angle to the IPP is $\psi = \frac{\pi}{2} - \varepsilon - z'$, and the IPP latitude and longitude come from the spherical-triangle solution along the satellite azimuth α (`computeIppGeometry`). The helper treats the receiver's geodetic latitude ϕ_r as geocentric when crossing the thin shell. The approximation is worth well under a millimetre of L1 delay. The higher-order model (111) later uses the IPP in the same frame:

$$\phi_{\text{IPP}} = \arcsin\big(\sin\phi_r\cos\psi + \cos\phi_r\sin\psi\cos\alpha\big), \tag{94}$$

$$\lambda_{\text{IPP}} = \lambda_r + \arcsin\frac{\sin\psi\,\sin\alpha}{\cos\phi_{\text{IPP}}}. \tag{95}$$

The VTEC at the IPP is interpolated bilinearly in space and, in time, *sun-fixed*: each bracketing map is sampled at the IPP longitude rotated by the Earth's rotation between the map epoch and the query epoch, $\Delta\lambda = (t - t_{\text{map}}) \cdot 360^\circ/86400$ s, before the two are linearly blended (`GlobalIonosphereMap::interpolateVtec`, over `GlobalIonosphereMap::sampleMap`). The slant delay in metres at frequency f then uses the dispersion constant

$$\boxed{I^{\text{GIM}} = \text{STEC} \cdot \frac{40.3 \times 10^{16}}{f^2} \quad [\text{m}], \qquad \text{STEC} = M(\varepsilon)\,\text{VTEC [TECU]},} \tag{96}$$

the factor 10^{16} converting VTEC from TECU to $\text{el}\,\text{m}^{-2}$ and $40.3 = 40.3\ \text{m}\,\text{Hz}^2\,(\text{el}\,\text{m}^{-2})^{-1}$ the first-order refractivity constant (both the constant and the metre conversion live in `GlobalIonosphereMap::slantIonoDelay`).

9.2.3 The GIM iono-weighting pseudo-observation

The GIM is too coarse to constrain the *absolute* slant of one satellite, but it constrains the *relative* slants across a constellation well. The constraint is switched by `estimation.ionosphere_weighting.enable` (default off, off in both reference configurations). When on, it is applied as a Kalman pseudo-observation (`Application::applyGimIonoUpdate`), one row per satellite that owns a slant state above the elevation cutoff (`estimation.ionosphere_weighting.elevation_cutoff_deg`). The network estimator builds the same rows, with the same covariance (98), for its stations when a GIM is the ionosphere constraint (`networkAppendGimIonoRows`, `network/assembly/IonoConstraintRows.cpp`). The innovation is the GIM slant minus the current state,

$$z_i = I_i^{\mathrm{GIM}} - I_r^i, \qquad \boldsymbol{H} = \boldsymbol{I} \text{ (selection of the iono columns)}, \tag{97}$$

with the GIM evaluated at GPS L1 to match the state reference (`Application::applyGimIonoUpdate`). The measurement-noise matrix carries the structure. Per constellation g, with $\mathbf{1}$ the all-ones vector over that constellation's rows,

$$\boxed{\boldsymbol{R} = \bigoplus_{g\in\mathcal{G}} \Big(\mathrm{diag}(\sigma_i^2) + \sigma_c^2\, \mathbf{1}\mathbf{1}^{\mathrm{T}} \Big),} \tag{98}$$

(also `Application::applyGimIonoUpdate`). The rank-one $\sigma_c^2\,\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block gives a very large variance to the per-constellation *common mode* of the innovations. That common mode is the receiver iono datum, an arbitrary offset shared by all satellites of a constellation, which no model can supply. The pseudo-observation therefore leaves it free and constrains only the between-satellite differences, which the GIM does describe. The update uses the same Joseph-form pseudo-update as every other measurement (Section 12). The iono process noise is documented there (Equation (164)).

Why frozen pivots are excluded. In the BSSD parameterisation each constellation's pivot satellite holds its slant-iono state frozen at value 0 with variance 0 as the rank-fixing datum. Such a row is skipped entirely (the frozen-pivot guard in `Application::applyGimIonoUpdate`). A zero-variance state receives zero Kalman gain, so the GIM cannot improve it. It does share the common-mode $\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block of (98) with the other satellites of its constellation. If it were included, its innovation $I_{p_g}^{\mathrm{GIM}} - 0 \sim 5$–$30$ m would enter the gain of every other satellite through the off-diagonal of $\boldsymbol{S}^{-1}$ and shift the relative slants by the pivot's full absolute STEC. Excluding the pivot row is required for a correct result. The common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

9.2.4 The co-estimated spherical-harmonic VTEC field

The network estimator can estimate the vertical TEC surface itself instead of importing a map. The field is a truncated real spherical-harmonic expansion on the same kind of thin shell, solved jointly with the satellite code biases. The coefficients are *shared* columns of the network normal equations, registered through `NetworkEstimator::registerVtecCoeff` and reduced onto the shared border like any other network-wide family (Section 20). This section gives the model, its pseudo-observation, and its datum. The model is retired from the reference posture, in which the slant ionosphere is free per line of sight, and is kept in the code for a later user-side product (`docs/refound/AB1_sh_consumer_product_2026-09-20.md`).

The switch is `network.estimate_iono_sh`. It defaults to false and is false in both reference configurations, which set `network.w6_iono_constraint = none` instead; the loader refuses the

two together. Arming it forces `network.estimate_sat_code_bias` on, because the field is what makes the satellite code biases observable. It also suppresses external GIM loading, so the two ionosphere sources are never applied to the same run (`src/core/config/GnssConfigLoadNetwork.cpp`; the rows are built by `networkAppendShIonoRows` or `networkAppendGimIonoRows` in `network/assembly/IonoConstraintRows.cpp`).

Basis and normalisation. With β the geographic latitude of the pierce point and s its sun-fixed longitude, the field is

$$\boxed{\mathrm{VTEC}(\beta, s) = \sum_{n=0}^{N} \sum_{m=0}^{n} \bar{P}_{nm}(\sin\beta)\left[a_{nm}\cos(ms) + b_{nm}\sin(ms)\right] \quad [\mathrm{TECU}],} \tag{99}$$

truncated triangularly at degree N (`estimation.global_vtec_sh.max_degree`), which gives $(N{+}1)^2$ real coefficients: one cosine term per $(n, m \geq 0)$ and one sine term per $(n, m \geq 1)$, enumerated once in a fixed order that the shared-column layout mirrors (`SphericalHarmonicIonosphere::coeffs`). The normalisation is

$$\bar{P}_{nm} = N_{nm}\, P_{nm}, \qquad N_{nm} = \sqrt{(2 - \delta_{0m})\,(2n+1)\,\frac{(n-m)!}{(n+m)!}}, \tag{100}$$

chosen so the real surface harmonics $\{\bar{P}_{nm}\cos(ms),\ \bar{P}_{nm}\sin(ms)\}$ are orthonormal under $(1/4\pi)\int(\cdot)\,\mathrm{d}\Omega$ [34]. Because (99) is linear in the coefficients, the basis values are the partial derivatives. `SphericalHarmonicIonosphere::basis` returns $\partial\mathrm{VTEC}/\partial a_{nm}$ and $\partial\mathrm{VTEC}/\partial b_{nm}$ directly, and the Jacobian needs no numerical differentiation. One consequence is used repeatedly below: $\bar{P}_{00} \equiv 1$, so a_{00} is the global mean VTEC.

Frame and geometry. The expansion uses a sun-fixed frame, $s = \lambda_{\mathrm{IPP}} - \lambda_{\odot}$, with $\lambda_{\odot} = \alpha_{\odot} - \mathrm{GMST}$ the sub-solar geographic longitude (`meanSunLongitude`). In this frame the field changes slowly over a session; in a geographic frame it would move westward at the Earth's rotation rate. `estimation.global_vtec_sh.sun_fixed` selects geographic longitude instead. The pierce point and the thin-shell mapping $M(\varepsilon)$ that carries VTEC to STEC come from *the same* `computeIppGeometry` helper that `GlobalIonosphereMap::slantIonoDelay` calls, evaluated at `estimation.global_vtec_sh.layer_height_km` over base radius `earth_radius_km`. Equations (93), (94) and (95) are therefore also the geometry of the co-estimated model. The external map and the co-estimated field share one implementation so that they stay consistent.

The pin row. No measurement observes the field directly. One pseudo-observation per line of sight ties it to the estimated slant states (`networkAppendShIonoRows`). For every GPS or Galileo LOS above `ionosphere_weighting.elevation_cutoff_deg` whose slant state lives in that station's local block, the row asserts that the slant state equals the field's prediction:

$$\boxed{\begin{aligned} & g_r^i \equiv I_r^i \;-\; \kappa\, M(\varepsilon) \sum_k \bar{Y}_k(\mathrm{IPP})\, c_k \;=\; 0, \qquad \kappa = \frac{40.3 \times 10^{16}}{f_{\mathrm{L1}}^2}, \\ & \frac{\partial g_r^i}{\partial I_r^i} = +1, \qquad \frac{\partial g_r^i}{\partial c_k} = -\,\kappa\, M(\varepsilon)\, \bar{Y}_k(\mathrm{IPP}). \end{aligned}} \tag{101}$$

Here c_k runs over the field coefficients, $\bar{Y}_k$ over the corresponding basis functions of (99), and κ is the TECU-to-L1-metre conversion already introduced in (96). The $+1$ goes in the station's private block and the $-\kappa M \bar{Y}_k$ terms go in the *shared* border. The second write is what turns an ionosphere prior into a joint estimate. The right-hand side is $-I_r^i$ at the current state, and the shared coefficients enter the absolute solve at zero offset.

The measurement noise on these rows is *the same* common-mode-free per-constellation block as the GIM pseudo-observation, (98), built from the same `ionosphere_weighting.common_mode_sigma_m` over a per-LOS representativeness sigma

$$\sigma_i = \max\big(\texttt{floor_m},\ \sigma_{\mathrm{rep}}\, M(\varepsilon)\,\kappa\big), \qquad \sigma_{\mathrm{rep}} = \texttt{global_vtec_sh.representation_sigma_tecu}, \tag{102}$$

after which the block is whitened by the Cholesky factor of $\boldsymbol{R}$ into unit-information rows before it joins the normal equations. (98) is reused here for a modelling reason. The rank-one $\sigma_c^2\,\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block leaves the iono datum of each (receiver, constellation) pair free. The receiver DCB is part of that common mode, so the field constrains only the *between-satellite* slant structure, which is all a global VTEC surface can describe. As a result a_{00} is only weakly observable, through the spread of $M(\varepsilon)$ across the network, and the datum below has to handle that. GLONASS is excluded from the field (`networkAppendShIonoRows` skips it). Its FDMA per-slot inter-frequency bias does not cancel in the common-mode block and would leak into the global coefficients.

The Kaula–Tikhonov prior as the estimability datum. The coefficients are shared columns. A coefficient whose pierce-point support has moved over an ocean or polar gap is nearly unobservable in the reduced normal equations and would fail the all-or-nothing rank gate (`NetworkSbasis::assertNullSpaceRank`). A degree-dependent prior is therefore applied to *every* coefficient before the gate runs (`networkInjectShVtecPrior`):

$$\boxed{\begin{aligned} &\sigma_n = \frac{\texttt{kaula_scale_tecu}}{n} \quad (n \geq 1), \qquad \sigma_0 = \texttt{mean_vtec_prior_sigma_tecu}, \\ &\bar{c}_k = \begin{cases} \texttt{mean_vtec_prior_tecu}, & c_k = a_{00}, \\ 0, & \text{otherwise}, \end{cases} \end{aligned}} \tag{103}$$

combined in information space with an optional random-walk anchor toward the previous epoch's estimate, inflated by a per-degree spectral density $q_n = \texttt{q_per_degree}/n^2$ for $n \geq 1$. The prior has two separate roles. The Kaula term provides *estimability*. It keeps every coefficient positive definite, so the declared null-space dimension equals the number of constellations whatever the coverage, and it pulls unobserved coefficients toward climatology each epoch so that they do not random-walk. The a_{00} term is a deliberately *loose* climatology anchor. a_{00} trades against the absolute satellite-DCB level, whose only rigorous datum is the per-(constellation, signal) sum-to-zero constraint of Section 20. Fixing a_{00} to an external mean VTEC would bring back the GIM dependency that co-estimation removes, and would bias every emitted satellite code bias by the difference.

Extensions kept with the model: default off. Two extensions of this model are built, off by default, and retired with it (ledger rows PARAM, `docs/CONFIG_promotion_ledger.json`).

(i) A second thin shell (`estimation.global_vtec_sh.layer2_max_degree`, default 0 = single layer) registers a second coefficient set at its own height, and the pin row (101) then sums the shells, each evaluated at *its own* pierce geometry, $\sum_l \kappa\, M_l(\varepsilon)\, \bar{Y}_k(\mathrm{IPP}_l)\, c_{k,l}$ (`ShLayer`, assembled by `shLayers`, which every consumer of the field loops so a shell can never be evaluated at another

shell's geometry). The second shell's a_{00} anchor is deliberately *tight*, for the opposite reason to the first shell's. Ground-only geometry barely observes how the mean VTEC divides between the layers: only the difference of the two mapping profiles is sensitive to it. Prior information must therefore hold the split, while the first shell's loose anchor leaves the *total* mean to the data.

(ii) A per-(station, satellite) representativeness deviation (`estimation.global_vtec_sh.rep_error_tau_s`, default 0 = off) replaces the pin-to-zero of (101) with a pin to a tracked first-order Gauss–Markov deviation d, carried in vertical TECU so its level and steady state are elevation-stationary. Writing d^-, P^- for the aged prior, $\hat{u}, \hat{V}$ for the solved posterior mean and variance of the pin functional, and f^2 for the pin-row floor variance, the exact scalar posterior is

$$\hat{d} = d^- + \frac{P^-}{a}\big(\hat{u} - d^-\big), \qquad P^+ = P^- \frac{U + f^2}{U + a}, \qquad a = P^- + f^2, \quad U = \frac{\hat{V} a}{a - \hat{V}}, \tag{104}$$

with U the data-only information recovered from $\hat{V}$ (`shDevExactUpdate`). Two treatments against ratcheting are built on it, also default OFF.

`rep_error_demean` (`shDevDemean`) removes the information-weighted global and per-station means of the carried priors once per epoch. A deviation component that is common in vertical TECU has row signature $\kappa\, M(\varepsilon)$, the same as the a_{00} column. The pair is therefore exactly quasi-null, and the cycle of harvesting, ageing, and re-asserting the prior ratchets it. The de-mean enforces the state's own contract, which is that it carries per-LOS structure only.

`rep_error_d5_keep` (`shDevD5Retap`) makes the slip response depend on which detector fired. A slip confirmed by a raw detector still *erases* the deviation, following the rule that a stiff time-linked carry must have a wired slip response (Section 15). A re-tap triggered only by the innovation gate keeps the level and resets the variance to the Gauss–Markov steady state.

The warm-LOS counts that arm the de-mean are constants in the seam, not configuration. The loader warns that both treatments are inert unless `rep_error_tau_s` is positive, and that arming the erase-policy split without the de-mean allows the ratchet again.

9.2.5 Klobuchar broadcast seed

The broadcast Klobuchar half-cosine model [16] is a single-frequency a priori. It is built only when `model_corrections.enable_klobuchar_ionosphere` is set (default off, off in both reference configurations), and `PppModel::computeIonosphericDelay` returns zero whenever an ionosphere-free pair is available, so it is never a dual-frequency correction. It needs the eight `GPSA` and `GPSB` coefficients of the navigation header; with any other count the delay is 0 m (`Ionosphere::computeKlobucharDelay`). The latitude is the geodetic latitude of `Coordinates::ecefToLlh`, as IS-GPS-200 specifies. With elevation and azimuth in semicircles, the earth-centred angle, geomagnetic IPP latitude, local time and obliquity are

$$\psi = \frac{0.0137}{\varepsilon/180 + 0.11} - 0.022, \qquad \phi_i = \mathrm{clip}_{\pm 0.416}\big(\phi/180 + \psi\cos\alpha\big), \tag{105}$$

$$\phi_m = \phi_i + 0.064\cos\big((\lambda_i - 1.617)\pi\big), \qquad F = 1 + 16\,(0.53 - \varepsilon/180)^3, \tag{106}$$

with $\lambda_i = \lambda/180 + \psi\sin\alpha/\cos(\phi_i\pi)$ and local time $t_i = 43200\,\lambda_i + t \pmod{86400}$. Cubic polynomials in the geomagnetic latitude ϕ_m give the amplitude and period,

$$\mathrm{AMP} = \sum_{k=0}^{3} a_k\,\phi_m^k\ (\geq 0), \qquad \mathrm{PER} = \sum_{k=0}^{3} b_k\,\phi_m^k\ (\geq 72000\ \mathrm{s}), \tag{107}$$

and the zenith delay is the truncated half-cosine, mapped to slant by F and scaled to the requested frequency by the f_{L1}^2/f^2 dispersion ratio of (92) (the tail of `Ionosphere::computeKlobucharDelay`):

$$I^{\mathrm{Klob}} = c\,F\Big(5\times10^{-9} + \mathrm{AMP}\left[1 - \tfrac{X^2}{2} + \tfrac{X^4}{24}\right]\Big)\frac{f_{\mathrm{L1}}^2}{f^2}, \quad X = \frac{2\pi(t_i - 50400)}{\mathrm{PER}},\ |X| < 1.57, \tag{108}$$

and the constant 5×10^{-9} s night term alone when $|X| \geq 1.57$. The leading c converts the modelled signal delay from seconds to metres.

9.3 Higher-order ionospheric corrections

The $1/f^2$ scaling covers the first-order term. The second- and third-order delays scale as $1/f^3$ and $1/f^4$ and are applied as deterministic corrections to the modelled range (`HigherOrderIonosphere::slantHigherOrderComponents`), per signal, before an ionosphere-free pair is formed. The switch is `model_corrections.enable_higher_order_ionosphere` (default off, off in both reference configurations). The slant TEC and the pierce point come from the thin-shell GIM of the previous subsection, so the correction is zero when no IONEX map is loaded (`PppModel::computeHigherOrderIonoCorrection`). The second order is the magnetic-field–weighted delay

$$\boxed{I_2^P = 7527\,c\,\mathrm{STEC}\,\frac{\boldsymbol{B}\cdot\hat{\boldsymbol{k}}}{f^3}, \qquad I_2^{\Phi} = -\tfrac{1}{2}\,I_2^P,} \tag{109}$$

where STEC is in el m^{-2}, $\boldsymbol{B}$ is the geomagnetic flux density in tesla at the IPP, $\hat{\boldsymbol{k}}$ the unit propagation vector (satellite→receiver), and c the speed of light. The code term is a delay when the signal travels along the field and an advance when it travels against it: GPS is right-hand circularly polarized about $\hat{\boldsymbol{k}}$ (IEEE), the electrons gyrate right-handedly about $\boldsymbol{B}$, so a signal along $\boldsymbol{B}$ is the extraordinary (R) mode with the larger index deviation. The receiver→satellite vector inverts I_2; the propagation direction replaced it on 2026-10-01 (`docs/refound/I2S_second_order_iono_sign_2026-10-01.md`). The constant $7527\,c = e\,A/(2\pi m_e) \approx 2.2564\times10^{12}$ combines the electron charge, the first-order refractivity constant A, and the electron mass. The constants and the $-\frac{1}{2}$ phase factor are both in `HigherOrderIonosphere::slantHigherOrderComponents`. The third order depends on the path integral of N_e^2, which a shape factor η relates to the F2-peak density and STEC (also `HigherOrderIonosphere::slantHigherOrderComponents`):

$$\boxed{I_3^P = 2437\,\eta\,N_{\max}\,\frac{\mathrm{STEC}}{f^4}, \qquad N_{\max} = \kappa\,\mathrm{STEC}, \qquad I_3^{\Phi} = -\tfrac{1}{3}\,I_3^P,} \tag{110}$$

with $\eta = 0.66$ the Hoque–Jakowski shape factor [11], $2437 = 3A^2/8$, and $N_{\max} = \kappa\,\mathrm{STEC}$ the linear Hoque–Jakowski fit ($\kappa = (20-6)\times10^{12}/((4.55-1.38)\times10^{18})$ el m^{-3}/(el m^{-2})) of the F2-peak electron density to slant STEC. The third-order term is therefore *linear* in $N_{\max}\,\mathrm{STEC}$, not in STEC^2. Both corrections are below a millimetre at L1. They differ by satellite, so between-satellite differencing does not remove them, and each is applied with its own sign on code and on phase.

9.3.1 The geomagnetic dipole field

The field at the IPP is a centred, tilted magnetic dipole evaluated in ECEF (`MagneticField::fieldAtEcef`):

$$\boxed{\boldsymbol{B}(\boldsymbol{r}) = B_0\left(\frac{R_E}{r}\right)^3\big(\hat{\boldsymbol{m}} - 3(\hat{\boldsymbol{m}}\cdot\hat{\boldsymbol{r}})\,\hat{\boldsymbol{r}}\big),} \tag{111}$$

with $r = \|\boldsymbol{r}\|$, $\hat{\boldsymbol{r}} = \boldsymbol{r}/r$, $B_0 = 30 \times 10^{-6}$ T the equatorial surface field, $R_E = 6371.2$ km the IGRF reference radius, and $\hat{\boldsymbol{m}}$ the unit axis pointing to the geographic location of the geomagnetic north pole, IGRF-13 epoch 2020.0 at $(80.65°, -72.68°)$ (`MagneticField::dipoleAxisEcef`). The Arctic geomagnetic pole is the dipole's *south* pole. With $\hat{\boldsymbol{m}}$ pointing at it, (111) makes $\boldsymbol{B}$ point horizontally toward magnetic north at the magnetic equator, as a compass does. This convention sets the sign of $\boldsymbol{B} \cdot \hat{\boldsymbol{k}}$ and therefore of I_2 in (109). IONEX is a thin-shell model, so the IPP ECEF position is rebuilt from the geocentric IPP latitude and longitude at the IONEX base-plus-shell radius (`HigherOrderIonosphere::slantHigherOrderComponents`).

The a-priori ZHD, the estimated ZWD and gradient states, the uncombined slant-iono state, and these corrections account for every atmospheric term of the undifferenced observation equation. Section 5 describes the operators that then remove the receiver-common parts.

10 Site Displacement and Geophysical Corrections

The receiver coordinate that enters Equation (7) and Equation (6) of the Foundations document is not the instantaneous position of the crust. It is the conventional tide-free station position, to which the IGS/ITRF products are referenced, plus a sum of *a priori* geophysical corrections. Each correction is a modelled 3D displacement $\boldsymbol{d}(t)$ of the antenna reference point. The ARP is the position state, which the engine obtains from the RINEX monument and the header eccentricity (Section 6.2). None of the corrections is estimated. All are computed from the a priori position (good to centimetres), the Sun and Moon ephemerides, and the Earth-orientation parameters. They are then projected onto the line of sight and added to the modelled range. This document derives the models as the IERS 2010 conventions [26] intend them, so it can be used to settle questions of sign and phase in the implementation.

Write the corrected receiver position as

$$\boldsymbol{r} = \boldsymbol{r}_0 + \boldsymbol{d}_{\mathrm{SET}} + \boldsymbol{d}_{\mathrm{OTL}} + \boldsymbol{d}_{\mathrm{PT}} + \boldsymbol{d}_{\mathrm{OPT}} + \boldsymbol{d}_{\mathrm{APL}}, \tag{112}$$

with $\boldsymbol{r}_0$ the conventional a priori position and each $\boldsymbol{d}$ an ECEF vector of millimetres to decimetres. The five terms are:

- the solid Earth tide $\boldsymbol{d}_{\mathrm{SET}}$, the largest, up to $\sim$30 cm radially;
- ocean tidal loading $\boldsymbol{d}_{\mathrm{OTL}}$, centimetres near coasts;
- the solid (rotational-deformation) pole tide $\boldsymbol{d}_{\mathrm{PT}}$, below a centimetre to a centimetre, secular plus Chandler;
- the ocean pole tide $\boldsymbol{d}_{\mathrm{OPT}}$, below a millimetre to $\sim$2 mm;
- atmospheric pressure loading $\boldsymbol{d}_{\mathrm{APL}}$, 2–20 mm.

Only the sun-synchronous S_1/S_2 thermal-tide part of the atmospheric loading is modelled. The non-tidal (synoptic) part is a known gap (Section 10.6). All five are computed in `src/core/corrections/GnssCorrections.cpp` and `src/core/corrections/OceanTideModel.cpp`. The LOS projection and the summation into the modelled range are in `src/core/model/ModelGeometry.cpp` and `src/core/model/PppModel.cpp`. Each term has a config flag (`enable_solid_earth_tide`, `enable_ocean_tide_loading`, `enable_pole_tide`, `enable_ocean_pole_tide`, `enable_atmospheric_loading`, all under `model_corrections`). With a flag off, that term is a zero vector. The loader defaults are the solid Earth tide, ocean tidal loading and the pole tide on, and

the ocean pole tide and the atmospheric loading off; ocean tidal loading without a coefficient source is a zero vector (Section 10.3.2). The real-data reference configuration `config/golden/golden_field_refound.json` arms the solid tide, the pole tide and ocean tidal loading with its three extensions (Section 10.3); the simulated-network reference configuration `config/golden/golden_sim_refound.json` leaves ocean loading off, because the simulated data carry none.

10.1 Sign convention and projection into the range

Every displacement is projected onto the unit line of sight before it enters the modelled range. Let $\boldsymbol{e} = (\boldsymbol{r}^i - \boldsymbol{r})/\|\boldsymbol{r}^i - \boldsymbol{r}\|$ be the receiver-to-satellite unit vector (the convention of Equation (7); see `ModelGeometry.cpp` where `los = (sat_pos -rx_pos)/range`). A site displacement $\boldsymbol{d}$ changes the geometric range by its component along the line of sight. The range shortens when the antenna moves *toward* the satellite. The scalar correction added to the modelled range is

$$\boxed{\delta\rho \;=\; \boldsymbol{d}\cdot(-\boldsymbol{e}) \;=\; \boldsymbol{d}\cdot\frac{\boldsymbol{r}-\boldsymbol{r}^i}{\|\boldsymbol{r}-\boldsymbol{r}^i\|},} \tag{113}$$

so a displacement toward the satellite (a positive component along $\boldsymbol{e}$) gives $\delta\rho < 0$. The ocean-loading, atmospheric-loading, and pole-tide projections use this form (`Geometry::computeSatelliteGeometry`: `otl_corr = otl_ecef.dot(-los)`, `apl_corr = apl_ecef.dot(-los)`; `PppModel::computePoleTideProjection`: `pole_tide_ecef.dot(-los)`), and the modelled range adds them in (`PppModel::computeBaseModeledValue` return sums `+ otl_corr + apl_corr + solid_tide` and the pair processors add `+ pole_tide_corr`). The solid-Earth-tide helper does its own projection with the same vector $(\boldsymbol{r}-\boldsymbol{r}^i)/\|\boldsymbol{r}-\boldsymbol{r}^i\|$ (`GnssCorrections.cpp`, `computeSolidEarthTide`: `los = (receiver_pos -sat_pos)/rho_norm`, then `disp.dot(los)`), so all the tide terms use one sign convention. The rest of this document gives each model as a 3D ECEF (or local) displacement $\boldsymbol{d}$ and leaves Equation (113) implicit.

The Sun and Moon positions $\boldsymbol{r}^{\odot}, \boldsymbol{r}^{\mathrm{m}}$ on which the body tides depend, and the ECI↔ECEF rotation and Earth-orientation parameters they require, come from `src/core/model/Planetary.cpp` and are documented in Section 6. Here $\boldsymbol{r}^{\odot}, \boldsymbol{r}^{\mathrm{m}}$ are given geocentric ECEF vectors.

10.2 Solid Earth tide (IERS 2010, Steps 1 + 2)

The solid Earth tide is the elastic deformation of the whole Earth under the lunisolar tide-generating potential. The implementation is a port of the IERS Conventions `dehanttideinel` routine [26] and has two steps. Step 1 is evaluated in the time domain from the instantaneous Sun and Moon geometry. Step 2 is a frequency-domain correction for the few constituents whose Love and Shida numbers differ from the nominal degree-2 values. **The whole Step 1+2 block is validated against reference values**: it reproduces the official IERS `dehanttideinel` test vector and the in-tree POINT reference to below a millimetre, cross-checked by `scripts/solid_tide_reference.py` (`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement` and the `SolidEarthTideTest` suite). The permanent (zero-frequency) tide is *retained*, so the corrected coordinate is conventional tide-free, as in IGS/ITRF (see the routine's header comment).

10.2.1 Step 1: in-phase degree 2 and 3

For each tide-raising body $b \in \{\odot, \mathrm{m}\}$ at geocentric position $\boldsymbol{r}^b$ ($r_b = \|\boldsymbol{r}^b\|$) and the station at $\boldsymbol{r}_0$ ($r = \|\boldsymbol{r}_0\|$, geocentric latitude φ via direction cosines), the degree-2 in-phase displacement is built

from the cosine of the body zenith angle $\cos Z_b = \hat{\boldsymbol{r}}_0 \cdot \hat{\boldsymbol{r}}^b$ and a body scale factor

$$\mathrm{fac}_b \;=\; \frac{M_b}{M_\oplus}\, R_e \left(\frac{R_e}{r_b}\right)^3, \tag{114}$$

with $R_e = 6378136.55$ m and mass ratios $M_\odot/M_\oplus = 332945.943062$, $M_\mathrm{m}/M_\oplus = 0.012300034$ (`GnssCorrections.cpp` tide-helper constants `kTideRe`/`kTideMassRatioSun`/`kTideMassRatioMoon`). The radial and tangential weights are

$$p_2^b = 3\Big(\tfrac{h_2}{2} - l_2\Big)\cos^2 Z_b - \tfrac{h_2}{2}, \tag{115}$$

$$x_2^b = 3\, l_2 \cos Z_b, \tag{116}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`), where h_2, l_2 are the *latitude-dependent* degree-2 Love and Shida numbers of IERS Eq. 7.2,

$$h_2 = 0.6078 - 0.0006\,(1 - \tfrac{3}{2}\cos^2\varphi), \qquad l_2 = 0.0847 + 0.0002\,(1 - \tfrac{3}{2}\cos^2\varphi) \tag{117}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`). An analogous degree-3 term uses fixed $h_3 = 0.292$, $l_3 = 0.015$ (`kTideH3`/`kTideL3`) and $\mathrm{fac}_b^{(3)} = \mathrm{fac}_b\,(R_e/r_b)$ (same routine). The Step-1 in-phase displacement sums the radial weight along the station unit vector and the tangential weight along each body's unit vector:

$$\boxed{\boldsymbol{d}^{(1)}_{\mathrm{SET}} = \sum_{b\in\{\odot,\mathrm{m}\}} \mathrm{fac}_b\Big(x_2^b\,\hat{\boldsymbol{r}}^b + p_2^b\,\hat{\boldsymbol{r}}_0\Big) \;+\; (\text{degree-3 analogue}),} \tag{118}$$

matching the per-component accumulation in `computeSolidEarthTideDisplacement`.

10.2.2 Step 1: out-of-phase anelasticity and the L^1 term

Mantle anelasticity makes the diurnal and semidiurnal degree-2 Love numbers complex; the small imaginary parts produce out-of-phase displacements `ST1IDIU` (diurnal, $\delta h = -0.0025$, $\delta l = -0.0007$) and `ST1ISEM` (semidiurnal, $\delta h = -0.0022$, $\delta l = -0.0007$) (`GnssCorrections.cpp`, helpers `st1Diurnal`, `st1SemiD`). The latitude (L^1) dependence of the Love numbers contributes a horizontal-only correction `ST1L1` with $l^{(1)}_{\mathrm{diu}} = 0.0012$, $l^{(1)}_{\mathrm{sem}} = 0.0024$ (helper `st1L1`). These three helpers return local radial/north/east displacements rotated into ECEF and are added to Equation (118) inside `computeSolidEarthTideDisplacement`. Their trigonometric forms follow the IERS reference and are not reproduced here.

10.2.3 Step 2: frequency dependence (Doodson tables)

Step 2 corrects the constituents whose actual Love numbers differ from the nominal degree-2 value, summing harmonic terms keyed by the Doodson fundamental arguments. The astronomical arguments τ, s, h, p, N', p_s (lunar time, Moon/Sun mean longitudes, lunar perigee, negative node, solar perigee) are polynomials in Julian centuries T from J2000, with the 15°/hr Earth-rotation term entering τ through the fractional UT hour H:

$$s = 218.31664563^\circ + 481267.88194\,T - 0.0014663889\,T^2 + \ldots, \tag{119}$$

$$\tau = 15^\circ H + 280.4606184^\circ + 36000.7700536\,T + \ldots - s, \tag{120}$$

with the remaining h, p, N', p_s polynomials given to fourth order in `GnssCorrections.cpp` (`computeAngles`). For each table line the phase is $\theta = \tau + n_s s + n_h h + n_p p + n_{N'} N' + n_{p_s} p_s (+\,\lambda$ for diurnal) and the in-/out-of-phase radial and tangential amplitudes (millimetres) are applied with a latitude weight.

Diurnal band (Table 7.3a). Thirty-one lines, dominated by K_1 and P_1 (`GnssCorrections.cpp`, `st2Diurnal`). The displacement for one line is

$$\begin{aligned} \delta R &= (A_R^{\text{in}} \sin\theta + A_R^{\text{out}} \cos\theta)\, 2 \sin\varphi \cos\varphi, \\ \delta N &= (A_T^{\text{in}} \sin\theta + A_T^{\text{out}} \cos\theta)\, \cos 2\varphi, \\ \delta E &= (A_T^{\text{in}} \cos\theta + A_T^{\text{out}} \sin\theta)\, \sin\varphi, \end{aligned} \tag{121}$$

(`st2Diurnal`), summed over the table and rotated radial/north/east $\rightarrow$ ECEF. The reference amplitudes (mm) include $K_1 : (12.00, -0.80, -0.67, -0.03)$ and $P_1 : (-1.23, -0.07, 0.06, 0.01)$ in the order $(A_R^{\text{in}}, A_R^{\text{out}}, A_T^{\text{in}}, A_T^{\text{out}})$ (the `st2Diurnal` table `T`).

Long-period band (Table 7.3b). Five zonal lines, with no longitude or τ dependence and no east component (`GnssCorrections.cpp`, `st2Lon`). The radial weight is the zonal Legendre form $(3\sin^2\varphi - 1)/2$ and the north weight is $\sin 2\varphi$:

$$\delta R = (A_R^{\text{in}} \cos\theta + A_R^{\text{out}} \sin\theta)\, \frac{3\sin^2\varphi - 1}{2}, \qquad \delta N = (A_T^{\text{in}} \cos\theta + A_T^{\text{out}} \sin\theta)\, \sin 2\varphi, \tag{122}$$

(`st2Lon`). The full solid-tide displacement is $\boldsymbol{d}_{\text{SET}} = \boldsymbol{d}^{(1)}_{\text{SET}}$ + ST1IDIU + ST1ISEM + ST1L1 + STEP2DIU + STEP2LON (`computeSolidEarthTideDisplacement`).

10.3 Ocean tidal loading (OTL)

Ocean tidal loading is the elastic response of the crust to the moving mass of the ocean tides. It reaches centimetres near coasts and is computed as a harmonic sum over the eleven principal constituents M2, S2, N2, K2, K1, O1, P1, Q1, Mf, Mm, Ssa (`OceanTideModel.hpp`, `OTL_CONSTITUENT_NAMES`). No admittance interpolation to minor constituents is applied. Each constituent k contributes, on each local component $c \in \{\text{up}, \text{west}, \text{south}\}$,

$$\boxed{dr_c(t) \;=\; \sum_{k=1}^{11} f_k(t)\, A_{c,k}\, \cos\big(\chi_k(t) + u_k(t) - \phi_{c,k}\big),} \tag{123}$$

where $A_{c,k}$ is the station amplitude, $\phi_{c,k}$ the Greenwich phase lag, and $\chi_k(t)$ the astronomical argument (the eleven-constituent accumulation of `OceanTideModel::computeDisplacement`). The nodal factor f_k and nodal phase u_k are exactly 1 and 0 unless `model_corrections.otl_nodal_corrections` is set (Section 10.3.3), and the sum is then the original evaluation bit for bit. The internal accumulation is in the (up, west, south) frame; the conversion to ENU is $(E, N, U) = (-\text{west}, -\text{south}, \text{up})$, then ENU$\rightarrow$ECEF (same routine). A non-finite displacement, or one above 0.5 m, is replaced by zero.

10.3.1 Astronomical argument and the Doodson–Warburg phase bias

The argument χ_k is assembled from Doodson multipliers $[\,\tau, s, h, p, N', p_s\,]$ acting on the variables $[\,\text{GMST} + \pi,\ s,\ h,\ p,\ N',\ p_s\,]$, themselves derived from the Delaunay fundamental arguments $s = F + \Omega,\ h = s - D,\ p = s - l,\ N' = -\Omega$ (`OceanTideModel::computeTidalArguments`). The BLQ Greenwich phase lags and the EOT20/FES complex coefficients are both referenced to a tide argument that carries a *species-dependent Doodson–Warburg phase bias*. For the diurnal band the

bias is a quarter cycle ($\pm 90°$), because the Cartwright–Tayler–Edden tide-generating potential refers to a *sine* rather than a cosine when (degree $+ n_\tau$) is odd. The bias per constituent (cycles) is

M2	S2	N2	K2	K1	O1	P1	Q1	Mf	Mm	Ssa
0	0	0	0	+0.25	−0.25	−0.25	−0.25	0	0	0

(124)

applied as $\chi_k \mathrel{+}= 2\pi\,(\text{bias}_k)$ (the `CONSTITUENT_PHASE_BIAS_CYCLES` step of `OceanTideModel::computeTidalArguments`; the table itself in `OceanTideModel.hpp`). **Without this bias every diurnal constituent (K1/O1/P1/Q1) is wrong in phase by** 90°, and the whole diurnal OTL contribution is mismodelled. The bias must be present, with the signs of Equation (124). This settles the question of the diurnal OTL phase.

10.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid

The per-station amplitudes and phases come from the first available of four sources (`Geometry::loadData`, OTL coefficient block): a Station Information File (SIF), a station BLQ block, the global EOT20 grid, and finally zero.

The SIF is consulted when `processing.enable_station_info` or `model_corrections.otl_from_sif` is set. The second setting loads the registry for ocean loading alone (from `files.station_info_path(s)`, or `static/sif/` of the data root), without arming the a priori position and the ocean pole tide of the station-information subsystem. A prefix match of the marker is confirmed by the receiver's approximate position (Section 7.9). When the SIF has an OTL block for the marker, it supplies per-station three-component FES2014b coefficients, the best of the four sources. The block is generated from BLQ files, whose rows are radial, west-positive and south-positive, and stores them under the names UP, EAST and NORTH. The model accumulates in Up/West/South (Equation (123)). Two readings of the horizontal columns exist (`SifOtlHorizontal`, `otlCoefficientsFromSif`). Under `otl_from_sif` the EAST and NORTH columns are the BLQ west and south rows and are copied unchanged, as NGS M-PAGES passes them to its HARDISP routine. Two independent checks support that reading (`scripts/otl_compare/sif_horizontal_convention.py` and `sif_vs_blq_rows.py`): the horizontal vector of a station points up the gradient of the radial depression measured from its neighbours in 98 % of 568 stations for M2, and the Onsala semidiurnal columns match the CSR4.0 west and south rows without a shift. With `enable_station_info` alone, the columns are read as true east and north. Since East = −West and South = −North, the horizontal components then keep their amplitude and shift phase by +180°. That reading reverses the horizontal vector. It is kept so that runs that armed the subsystem stay unchanged. Up is copied in both readings. A marker with no OTL block in the SIF falls through to the BLQ block and then to the grid, with a warning.

Without a SIF entry the coefficients come from a station BLQ block. Without that, they are synthesised by bilinear interpolation of the global EOT20 complex load grid, which gives the radial component only. The grid is sampled at the RINEX header *marker* position (`RinexParser::getApproxPositionXyz`), not at the ARP position state. The difference is immaterial at this grid resolution, but the two points are distinct in the code. With no source the displacement is zero.

The EOT20 netCDF packs the real and imaginary load coefficients as `int16` with a `scale_factor/add_offset`. The netCDF C library does *not* apply these automatically, so the code unpacks them as $v = v_{\text{int16}} \cdot \text{scale} + \text{offset}$ (`OceanTideModel::parseEot20Load`: its `getScaleOffset` lambda and the manual unpack that follows). The scale is $\approx 5.34 \times 10^{-5}$ m/LSB, and omitting it inflates the amplitudes by ∼20000×. The complex load $Z = \text{Re} + i\,\text{Im}$ yields the real signal $h(t) = \text{Re}\{Z\,e^{i\chi}\} = A\cos(\chi - \phi)$ with

$$A = \sqrt{\text{Re}^2 + \text{Im}^2}, \qquad \phi = \text{atan2}(-\text{Im},\, \text{Re}) \tag{125}$$

(ϕ in BLQ format being the Greenwich phase lag in degrees; same routine). Because EOT20 orders its 17 constituents differently from the BLQ convention, the synthesis remaps the BLQ slot k to the EOT20 index via $\{4, 13, 8, 3, 2, 9, 10, 11, 6, 7, 15\}$ (the `EOT20_INDICES_FOR_BLQ` table in `OceanTideModel::parseEot20Load`).

10.3.3 Nodal corrections and the UTC Greenwich argument

Each constituent of (123) is one spectral line. The 18.6-year regression of the lunar node Ω splits a lunar line into a main line and side lines that differ from it only in the multiplier n_l of Ω, and the sum of the lines modulates the amplitude and advances the phase of the constituent. With `model_corrections.otl_nodal_corrections`, constituent k is scaled by f_k and advanced by u_k at the node longitude Ω of the epoch,

$$f_k\, e^{\,\mathrm{i}\,u_k} \;=\; 1 + \sum_{l \in \text{side lines of } k} r_{k,l}\, e^{-\mathrm{i}\,n_l\,\Omega}, \tag{126}$$

with $r_{k,l}$ the signed ratio of the equilibrium-tide amplitude of a side line to that of the main line (`OceanTideModel::nodalFactor`). The ratios are those of the Cartwright–Tayler–Edden development tabulated in the IERS HARDISP routine, and the loading admittance is taken as flat across a main line and its side lines. The series reproduces the usual closed forms (for M2, $f = 1 - 0.0373\cos\Omega$ and $u = -2.14^\circ \sin\Omega$) to 0.0005 in f and 0.1° in u for M2, N2, K1, O1, Q1 and Mf.

The same setting takes the Greenwich sidereal angle inside χ_k from the UTC Julian Date and not from the GPS-scale date, as HARDISP does. GPS time leads UTC by 18 s, which shifts a semidiurnal argument by 0.15°; UT1 − UTC (below 0.9 s) is neglected, as in HARDISP. Against HARDISP the evaluation agrees within 0.2 to 1.9 mm over six hours at four stations (`docs/refound/OTL_ocean_tide_loading_source_2026-10-16.md`).

10.3.4 The net translation of the diurnal loading

The SIF loading is computed relative to the centre of mass of the Earth system. Part of the diurnal loading moves every station by the same ECEF displacement, a translation of the origin: a uniform vector fitted to the 597 stations of the bundled IGS SIF carries 45 % (K1), 40 % (O1) and 45 % (P1) of the east, north and up variance, against 6 % for the semidiurnal species. Orbit and clock products whose solution absorbs a common network translation do not contain that part, so a user working against them does not see it in the data, and applying it moves the estimated position by minus the translation. `model_corrections.otl_remove_diurnal_translation` (with `otl_from_sif`) removes it from the coefficients of the four diurnal constituents K1, O1, P1 and Q1. Let $Z_{c,k} = A_{c,k}\, e^{-\mathrm{i}\phi_{c,k}}$ be the BLQ phasor of component c and $\boldsymbol{G}_k$ the fitted translation phasor of constituent k, one complex number per ECEF axis (`removeOtlDiurnalTranslation`; the table is generated by `scripts/otl_compare/net_translation.py`). The east, north and up components of $\boldsymbol{G}_k$ at the station are g_k^E, g_k^N, g_k^U, and

$$Z_{\mathrm{up},k} \leftarrow Z_{\mathrm{up},k} - g_k^U, \qquad Z_{\mathrm{west},k} \leftarrow Z_{\mathrm{west},k} + g_k^E, \qquad Z_{\mathrm{south},k} \leftarrow Z_{\mathrm{south},k} + g_k^N, \tag{127}$$

since east is minus west and north is minus south. The semidiurnal and long-period constituents are unchanged. The table holds for the SIF it was fitted to, and the function applies to the coefficients that `otlCoefficientsFromSif` returns, not to a BLQ file or the grid. On user replays of CODE-based network products the scale at which the north component of the translation appears in the data is −0.11 (95 % interval −0.75 to +0.27), against 1.19 for the rest of the model (same document).

Armed state. The real-data reference configuration sets `enable_ocean_tide_loading`, `otl_from_sif`, `otl_nodal_corrections` and `otl_remove_diurnal_translation`, and reads the registry from `static/sif/` of the data root. Over a whole tidal day the four settings gain 0.8 mm at the test station (`docs/refound/BOF_built_off_follow_through_2026-10-28.md`).

10.4 Pole tide (rotational deformation, IERS 7.1.4)

The solid pole tide is the elastic deformation caused by the centrifugal potential of polar motion. As the rotation pole moves relative to its secular mean, the bulge it raises moves with it. The wobble is the polar-motion offset from the IERS 2018 secular mean pole. In milliarcseconds, with t_y the decimal year,

$$\bar{x}_p = 55.0 + 1.677\,(t_y - 2000), \qquad \bar{y}_p = 320.5 + 3.460\,(t_y - 2000) \tag{128}$$

(`GnssCorrections.cpp`, `computePoleTideCorrection`). The post-2018 IGS products used by the engine are referenced to this linear model. The wobble components (arcseconds) are

$$m_1 = x_p - \bar{x}_p, \qquad m_2 = -\big(y_p - \bar{y}_p\big), \tag{129}$$

with x_p, y_p the polar-motion values linearly interpolated from the ERP table at the query epoch (`computePoleTideCorrection`; the $\bar{x}_p, \bar{y}_p$ millarcsecond means are scaled to arcseconds in the difference). **The wobble must be in arcseconds, not radians.** The displacement coefficients below are in metres per arcsecond, so a wobble in radians scales the whole pole tide by $\sim 4.85 \times 10^{-6}$. The wobble comment in `computePoleTideCorrection` records this as a bug that was once present.

IERS 2010 Eq. 7.24 gives the rotational-deformation displacement in colatitude $\theta = 90^\circ - \varphi$; substituting geodetic latitude ($\sin 2\theta = \sin 2\varphi$, $\cos 2\theta = -\cos 2\varphi$, $\cos\theta = \sin\varphi$, north $= -S_\theta$) yields the radial, north, and east components (metres):

$$\boxed{\begin{aligned} \boldsymbol{d}^R_{\mathrm{PT}} &= -0.033\,\sin 2\varphi\,\big(m_1\cos\lambda + m_2\sin\lambda\big), \\ \boldsymbol{d}^N_{\mathrm{PT}} &= -0.009\,\cos 2\varphi\,\big(m_1\cos\lambda + m_2\sin\lambda\big), \\ \boldsymbol{d}^E_{\mathrm{PT}} &= +0.009\,\sin\varphi\,\big(m_1\sin\lambda - m_2\cos\lambda\big), \end{aligned}} \tag{130}$$

(`computePoleTideCorrection`). **The east factor is $\sin\varphi$ $(= \cos\theta)$, not $\cos\varphi$.** A factor of $\cos\varphi$ is inconsistent with the radial and north terms, which use geodetic φ. It gives a scale error of $\sim 1.48\times$ at mid-latitude and the wrong latitude dependence, vanishing at the equator instead of the pole. This settles the question of the pole-tide east term. The local (R, N, E) vector is rotated into ECEF at the end of `computePoleTideCorrection`.

10.5 Ocean pole tide (Desai 2002, IERS 7.1.5)

Polar motion also redistributes ocean mass. The load of this ocean pole tide deforms the crust by a further amount, from below a millimetre to ∼2 mm. It is added to Equation (130) when the station's six Desai grid load coefficients are available (the `has_opt_coeffs_` branch of `computePoleTideCorrection`). Following Desai (2002) [6], the same wobble m_1, m_2 drives a complex pole-tide coefficient

$$c^R = m_1\,\gamma_2^R + m_2\,\gamma_2^I, \tag{131}$$

$$c^I = m_2\,\gamma_2^R - m_1\,\gamma_2^I, \tag{132}$$

with the IERS/Desai load Love combination $\gamma_2^R = 0.6870$, $\gamma_2^I = 0.0036$ (`kOptGammaReal`/`kOptGammaImag`). Each of the six per-station coefficients u_c^R, u_c^I (north, east, radial real and imaginary parts), bilinearly interpolated from the Desai grid, gives the local displacement

$$\boxed{\boldsymbol{d}^c_{\mathrm{OPT}} = \kappa \left(c^R \, u_c^R + c^I \, u_c^I \right), \qquad c \in \{E, N, U\},} \tag{133}$$

(`GnssCorrections.cpp`, `computeOceanPoleTideDisplacement`). The scale κ combines the IERS load-deformation constants with the arcsecond-to-radian conversion, so the wobble stays in arcseconds as in Equation (129):

$$\kappa = \frac{4\pi \, G \, a \, \rho_w \, H_p}{3 \, g_E} \, \frac{\pi}{648000}, \qquad H_p = \sqrt{\tfrac{8\pi}{15}} \, \frac{\Omega^2 a^4}{G M_\oplus}, \tag{134}$$

with $\rho_w = 1025$ kg m^{-3}, $a = 6378136.6$ m, $g_E = 9.7803278$ m s^{-2}, $\Omega = 7.292115 \times 10^{-5}$ rad s^{-1}, $G = 6.67428 \times 10^{-11}$, $GM_\oplus = 3.986004418 \times 10^{14}$ (the `kOpt*` constants; κ assembled in `oceanPoleTideKappa`). The whole ocean-pole-tide path reproduces the NGS M-PAGES implementation and the independent Python oracle `scripts/ocean_pole_tide_reference.py` to $\sim 10^{-12}$ (see the constants' header comment).

10.6 Atmospheric pressure loading (S1/S2 thermal tide)

Atmospheric pressure loading (APL) is the response of the crust to changes in surface pressure. It is ∼2–20 mm and mostly vertical. It has two parts. The S_1/S_2 tidal part is sun-synchronous and thermally driven (diurnal and semidiurnal), and is predictable from local solar time. The non-tidal (synoptic) part is driven by weather systems. The engine models the *tidal* part only; the non-tidal part is a gap (see the note below).

The thermal-tide displacement uses per-station diurnal (S_1) and semidiurnal (S_2) harmonic coefficients evaluated against the local-solar-time argument $\Theta_n = n\,(2\pi \, \mathrm{UT}_{\mathrm{frac}} + \lambda)$, $n \in \{1, 2\}$, with $\mathrm{UT}_{\mathrm{frac}}$ the UT fraction of day and λ the station longitude (`GnssCorrections.cpp`, `computeAtmosphericTideDisplacement`):

$$\boxed{\boldsymbol{d}^c_{\mathrm{APL}} = a^{\cos}_{c,S_1} \cos\Theta_1 + a^{\sin}_{c,S_1} \sin\Theta_1 + a^{\cos}_{c,S_2} \cos\Theta_2 + a^{\sin}_{c,S_2} \sin\Theta_2, \qquad c \in \{E, N, U\},} \tag{135}$$

formed in local ENU and rotated to ECEF, with a 1 cm sanity guard. The twelve coefficients per station are ordered $[\, a^{\cos}_E, a^{\sin}_E, a^{\cos}_N, a^{\sin}_N, a^{\cos}_U, a^{\sin}_U \,]$ for S_1 (indices 0–5) then the same six for S_2 (indices 6–11). They are bilinearly interpolated from a Ray–Ponte-style S_1/S_2 load grid (`loadAtmosphericTideFromGrid`) at the same RINEX header marker position as the OTL grid (Section 10.3.2). The per-epoch displacement is evaluated at the ARP position state. The term is gated by `model_corrections.enable_atmospheric_loading` (default `false`). It reads its grid from `files.atm_tide_grid_path`, which defaults to `<data>/static/apl/s1_s2_load_cm.dat`. That filename is PROVISIONAL and will be set to the supplied product. The coefficients are loaded in `Geometry::loadData` and projected to `apl_corr` alongside `otl_corr` in `computeSatelliteGeometry`. With the flag off, or with no usable grid, $\boldsymbol{d}_{\mathrm{APL}} = \mathbf{0}$ in Equation (112) and runs are byte-identical.

What is not modelled. Only the sun-synchronous S_1/S_2 tidal APL is modelled. The non-tidal (synoptic) pressure loading is larger and is driven by weather. Modelling it needs a surface-pressure field convolved with a Green's function, or a gridded non-tidal loading product. It is *not* modelled, and it is the leading candidate for the next site-displacement term.

10.7 Integration into the measurement model

Each epoch, the geometry builder evaluates every enabled displacement and projects it with Equation (113). The modelled-range assembly then adds the scalar corrections. In `Geometry::computeSatelliteGeometry` the OTL ECEF displacement is formed and projected to `otl_corr`, the S_1/S_2 atmospheric tide to `apl_corr`, and the solid tide is evaluated against the cached Sun/Moon positions and already returns its LOS-projected scalar. In `PppModel` the pole tide (and, when enabled, the ocean pole tide additively inside it) is projected by `computePoleTideProjection` and the base modelled value (`computeBaseModeledValue`) sums `+ otl_corr + apl_corr + solid_tide` while the pair processors add `+ pole_tide_corr`. The result is the $\rho(\boldsymbol{r})$ of Equation (6) with $\boldsymbol{r}$ replaced by Equation (112).

The differenced observables of Section 5.1 and Section 5.3 formulations contain whatever part of these corrections survives differencing. The solid Earth tide and the pole tide are highly correlated in space and largely cancel over short baselines. Ocean loading and atmospheric loading, the latter largely unmodelled in its non-tidal part, do not cancel. They are the dominant differential site-displacement error on long baselines.

11 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections

This document covers the *deterministic* corrections of the observation model, other than geometry, the atmosphere, and the streamed SSR overlay. Section 4 (Equation (4) and Equation (5)) introduce the satellite and receiver hardware biases B^i_s, b^i_s, $B_{r,s}$, $b_{r,s}$ only as symbols. Section 4.3 gives the geometric range ρ^i_r and its linearisation. This document supplies every other deterministic term between the raw measurement and the modelled range:

- the antenna phase-centre geometry, and the satellite attitude frame it depends on;
- the carrier-phase wind-up;
- the relativistic clock and propagation corrections;
- the Sagnac (Earth-rotation) frame correction;
- the bias terms, realised through broadcast group delays and the precise `.BIA` product.

The streamed SSR overlay corrects the same terms in real time and is described in Section 18. The two mechanisms differ in frames, sign conventions, and gating, and the differences are noted where they matter.

`PppModel::computeBaseModeledValue` in `model/PppModel.cpp` assembles every term below into one modelled observable. Both the iono-free-pair and the uncombined observation paths call it. It adds the terms to the geometric range from `model/ModelGeometry.cpp`. For the code observable the sum is

$$\hat{P}^i_{r,s} = \rho^i_r \;-\; c\,\mathrm{d}t^i \;-\; c\,\delta t_{\mathrm{rel}} \;+\; T^i_r \;+\; A^i_{r,s} \;+\; B_{\mathrm{grp}} \;+\; S^i_r \;+\; d_{\mathrm{atm}} \;+\; d_{\mathrm{tide}}, \tag{136}$$

where T^i_r is the a priori slant hydrostatic delay (the wet delay and the gradients are estimated; Section 9), $A^i_{r,s}$ is the total antenna correction (Section 11.2), δt_{rel} the periodic relativistic clock term (Section 11.4), S^i_r the Shapiro delay (Section 11.5), B_{grp} the deterministic code/phase bias (Section 11.7), and d_{atm}, d_{tide} the loading/site-displacement terms documented in Section 10. The

Sagnac correction (Section 11.6) is part of ρ_r^i and is not added separately; the reason is given below. The signs in (136) follow the return statement of `PppModel::computeBaseModeledValue` (`model/PppModel.cpp`): the satellite clock and relativistic terms are *subtracted*, and the antenna, Shapiro, loading and tide terms are *added*.

11.1 Satellite attitude: the body frame

The satellite phase-centre offset, the phase-centre variation, and the carrier wind-up are all defined in the *spacecraft body frame*, so that frame is constructed first. The engine uses the IGS/ANTEX convention: a right-handed orthonormal triad $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i, \hat{\boldsymbol{k}}_i)$ with $+Z$ toward the Earth's centre (nadir / antenna boresight), $+X$ in the Sun–satellite–Earth plane pointing toward the Sun, and $+Y$ completing the triad along the solar-panel (cross-track) axis (`model/SatAttitude.cpp`, `model/Antenna.cpp`).

Nominal yaw-steering. With $\boldsymbol{r}^i$ the satellite ECEF position and $\hat{\boldsymbol{e}}_\odot = (\boldsymbol{r}_\odot - \boldsymbol{r}^i)/\|\boldsymbol{r}_\odot - \boldsymbol{r}^i\|$ the unit direction to the Sun, the nadir axis and the cross-track axis are

$$\hat{\boldsymbol{k}}_i = -\frac{\boldsymbol{r}^i}{\|\boldsymbol{r}^i\|}, \qquad \hat{\boldsymbol{y}}_i = \frac{\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot}{\|\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot\|}, \qquad \hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i, \tag{137}$$

in the operand order of `SatAttitude::compute` (`model/SatAttitude.cpp`). One model serves the satellite PCO and PCV and the wind-up: the antenna path (`Antenna::getAntennaCorrection`) and the wind-up (`GnssCorrections::windUpGeometry`) both take the frame from `SatAttitude`, so the two cannot disagree on the mode selection or on the sign conventions. Forming $\hat{\boldsymbol{y}}_i$ first and then $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$ is the Gram–Schmidt orthogonalisation of the Sun direction against nadir. By the $\boldsymbol{a} \times (\boldsymbol{b} \times \boldsymbol{c})$ identity,

$$\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i = (\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot) \times \hat{\boldsymbol{k}}_i = \hat{\boldsymbol{e}}_\odot - \hat{\boldsymbol{k}}_i\,(\hat{\boldsymbol{k}}_i \cdot \hat{\boldsymbol{e}}_\odot), \tag{138}$$

so $\hat{\boldsymbol{x}}_i$ is the normalised component of $\hat{\boldsymbol{e}}_\odot$ orthogonal to nadir. The $+X$ axis therefore points toward the Sun as far as the nadir constraint allows, as the ANTEX convention requires. The body-frame matrix with these three columns,

$$\boldsymbol{F} = \big[\,\hat{\boldsymbol{x}}_i \;\; \hat{\boldsymbol{y}}_i \;\; \hat{\boldsymbol{k}}_i\,\big], \tag{139}$$

is orthonormal, so its inverse is its transpose. This is used below to rotate a body-frame offset into ECEF.

Sun elevation and the mode switch. Non-nominal attitude (eclipse seasons, the noon and midnight yaw turns) depends on the Sun's elevation above the orbital plane, β. The orbit plane is inertial, so the orbit angular-momentum (normal) unit vector $\hat{\boldsymbol{n}} = (\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i_{\mathrm{I}})/\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i_{\mathrm{I}}\|$ is formed from the inertial velocity $\dot{\boldsymbol{r}}^i_{\mathrm{I}} = \dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} + \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$ (`SatAttitude::inertialVelocity`, the inverse of Equation (61)),

$$\boxed{\beta = \arcsin\!\big(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{e}}_\odot\big).} \tag{140}$$

Forming $\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i_{\mathrm{ECEF}}$ instead removes $\boldsymbol{r}^i \times (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i)$ from the normal, a vector of 29 to 59 % of $\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i_{\mathrm{I}}\|$ at GPS and Galileo orbits, which tilts the normal by tens of degrees. The conversion applies when `estimation.enable_sun_frame_correction` is set (Section 6.10); with it off, the ECEF velocity is used as before, bit for bit. The noon-turn model below always converts. The angle is signed by the projection of the Sun direction onto the $+(\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)$ normal (`SatAttitude::`

computeBeta). With the conversion on, it agrees with an independent computation (SP3 positions rotated to the inertial frame and an almanac Sun, scripts/attitude_beta_check.py) to 0.01°, with no sign mismatch, on seven GPS satellites over a day. The mode selector (SatAttitude::determineMode) keeps GPS, GLONASS and Galileo in yaw-steering unconditionally, and switches BeiDou and QZSS to orbit-normal attitude below a constellation-specific β threshold (4.0° BeiDou, 20.0° QZSS, model/SatAttitude.cpp). The threshold compares the signed β, as NGS M-PAGES does. In orbit-normal mode the $+Y$ axis is pinned to the orbit normal, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{n}}$, and the triad is re-orthogonalised $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}_i$. The axes are those of the positive orbit normal in every orbit-normal case; the QZSS label OrbitNormalNegative serves traces only. The legacy default path also forces a hard-coded list of BeiDou-2 GEO PRNs (C01–C05, C59, C60) into orbit-normal mode whatever the value of β. The β-driven table is active only when processing.enable_orbit_normal_attitude is set (default off). With the flag off the result is bit-identical to the earlier inline frame (model/Antenna.cpp, model/SatAttitude.cpp). The wind-up takes the same frame but keeps only the BeiDou-2 GEO orbit-normal rule and yaw-steers every other satellite whatever that flag says; the flag moves the PCO frame alone. Without a Sun direction the frame is built from the velocity direction about the nadir axis (SatAttitude::velocityFrame), and a Sun collinear with nadir falls back to $\hat{\boldsymbol{y}}_i \propto \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{v}}$.

Nominal yaw steering for Galileo and the other systems. GPS, GLONASS and Galileo satellites carry the nominal frame (137) at every β. The model has no manoeuvre law for Galileo or GLONASS, no eclipse or midnight turn for any system, and no yaw bias. A Galileo satellite in the Earth's shadow is therefore excluded upstream when processing.exclude_eclipsing_satellites is set (default off, on in the reference configurations), and the nominal frame is used otherwise.

GPS noon turn. Near the orbit noon at small $|\beta|$ the nominal yaw would need a rate above what the attitude controller of a GPS Block IIF or IIR satellite delivers. The satellite then yaws at its maximum rate and rejoins the nominal yaw afterwards [17]. With μ the orbit angle from the orbit midnight (0 at midnight, π at noon) and $\dot{\mu}$ the mean orbit rate 0.00836°/s, the nominal yaw about the nadir axis, measured from the velocity direction, is

$$\psi_{\mathrm{nom}} = \operatorname{atan2}\big(-\tan\beta,\ \sin\mu\big), \tag{141}$$

and its rate peaks at noon at $\dot{\mu}/|\tan\beta|$. With $\dot{\psi}_{\max}$ the controller limit (0.11°/s for Block IIF, 0.20°/s for Block IIR, IIR-A, IIR-B and IIR-M, chosen from the block named in the ATX satellite type), no turn occurs when $|\beta| \geq \beta_0 = \arctan(\dot{\mu}/\dot{\psi}_{\max})$, that is 4.35° and 2.39°. Below β_0 the satellite follows ψ_{nom} until the nominal rate reaches $\dot{\psi}_{\max}$, at the orbit angle x_s before noon with $c = \cos x_s$ the positive root of

$$\dot{\psi}_{\max}\, c^2 + \dot{\mu}\,|\tan\beta|\, c - \dot{\psi}_{\max}\,(1 + \tan^2\beta) = 0, \tag{142}$$

then yaws at $\dot{\psi}_{\max}$ against the sign of β until the ramp meets ψ_{nom} again. Inside the turn the body X and Y axes are rotated about nadir by the difference $\Delta = \psi_{\mathrm{act}} - \psi_{\mathrm{nom}}$ (wrapped to $\pm\pi$):

$$\hat{\boldsymbol{x}}'_i = \cos\Delta\ \hat{\boldsymbol{x}}_i + \sin\Delta\,(\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}_i), \qquad \hat{\boldsymbol{y}}'_i = \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}'_i, \tag{143}$$

and outside the turn the frame is returned bit for bit (SatAttitude::noonTurnYaw, applyNoonTurn, noonTurnLaw). The same rotated frame feeds the satellite PCO and PCV (Antenna::getAntennaCorrection) and the wind-up dipole (149), so the two stay on one attitude. The model applies only to GPS Block IIF and IIR satellites and only when processing.enable_

`noon_turn_attitude` is set (default off; on in the real-data reference configuration, listed as a documented exception in the simulated-network one because the simulated data carry no noon-turn law). The model is described in `docs/refound/NTA_noon_turn_attitude_2026-10-16.md`.

Eclipse gating. The body frame above assumes a known yaw angle. That assumption fails during Earth-shadow passes. The actual yaw during eclipse and the recovery after it depend on the satellite block and are not modelled here, so the antenna PCO/PCV and the wind-up are unreliable for an eclipsing satellite. When the user opts in (`processing.exclude_eclipsing_satellites`), the satellite is dropped at the geometry stage by a *cylindrical umbra* test (`Planetary::isSatelliteEclipsing`, `model/Planetary.cpp`, called from `model/ModelGeometry.cpp`). With $\hat{\boldsymbol{u}}_\odot = \boldsymbol{r}_\odot / \|\boldsymbol{r}_\odot\|$ the Earth-to-Sun direction, the satellite's along-axis and perpendicular distances are $s_\parallel = \boldsymbol{r}^i \cdot \hat{\boldsymbol{u}}_\odot$ and $\boldsymbol{s}_\perp = \boldsymbol{r}^i - s_\parallel \, \hat{\boldsymbol{u}}_\odot$, and the predicate is

$$\text{eclipsed} \iff \big(s_\parallel < 0\big) \ \wedge \ \big(\|\boldsymbol{s}_\perp\| < \kappa \, R_\oplus\big), \tag{144}$$

i.e. the satellite is on the anti-Sun side *and* inside the shadow cylinder of radius $\kappa R_\oplus$, with $R_\oplus = 6\,371\,230\,\text{m}$ (`common/Constants.hpp`) and κ the user shadow scale (`processing.eclipse_shadow_scale`, default unity; cited rather than reproduced as a tuning knob). This is a conservative geometric umbra test. It does not model the penumbra.

11.2 Antenna phase-centre corrections

The range is measured between the satellite and receiver *antenna phase centres*. The estimated states refer to the satellite centre of mass and the receiver antenna reference point (ARP). The position state must be the ARP because the ANTEX receiver PCO/PCV are referenced to it. The antenna correction $A^i_{r,s}$ connects the two, as the sum of four geometric terms and the wind-up:

$$\boxed{A^i_{r,s} \ = \ -\,\delta^i_{\text{PCO}} \ + \ \delta^i_{\text{PCV}} \ + \ \delta^r_{\text{PCO}} \ + \ \delta^r_{\text{PCV}} \ + \ \varphi_{\text{wu}},} \tag{145}$$

as accumulated in `Antenna::getAntennaCorrection` (`model/Antenna.cpp`). The line of sight used in this section is the *satellite-to-receiver* unit vector, $\boldsymbol{e} = (\boldsymbol{r}_r - \boldsymbol{r}^i)/\|\boldsymbol{r}_r - \boldsymbol{r}^i\|$ (`Antenna.cpp`). It is the negative of the receiver-to-satellite $\boldsymbol{e}^i_{\hat{r}}$ of Section 4.3 (Equation (7)). The sign of each term below follows from this choice.

Which coordinate is the ARP. Every position the engine derives from a RINEX *header* is an ARP position. The marker coordinate is reduced to the ARP through the header eccentricity before it seeds the state (`Coordinates::markerToArp`, packaged as `RinexParser::getArpPositionXyz`; Section 6.2). A coordinate supplied directly in the configuration is *not* reduced. It is used as given, on the policy that the user owns its reference point. This matters in the network estimator. Its station coordinates come from the configuration (`StationConfig::position_ecef`) and are held fixed, with their position columns dropped from the normal equations. A monument coordinate there is not absorbed by a position state. The offset goes into the residuals, and from there into the estimated clocks, biases, and ionosphere. `network.station_marker_to_arp` (default off, on in both reference configurations) reduces each roster coordinate once at setup; a SIF coordinate is reduced through the same function (Section 6.2, Section 7.9).

Satellite PCO (subtracted). The satellite phase-centre offset $\boldsymbol{p}^i_{\text{body}}$ is published in the body frame (millimetres in ANTEX). It is rotated to ECEF by the body-frame matrix; because $\boldsymbol{F}$ collects the body axes as *columns*, $\boldsymbol{F}$ itself maps body $\rightarrow$ ECEF (and the orthonormal $\boldsymbol{F}^{\mathrm{T}} = \boldsymbol{F}^{-1}$ maps ECEF $\rightarrow$ body). It is then projected onto the LOS:

$$\delta^i_{\text{PCO}} = \big(\boldsymbol{F}\,\boldsymbol{p}^i_{\text{body}}\big) \cdot \boldsymbol{e}. \tag{146}$$

The antenna phase centre is *nearer the receiver* than the centre of mass, so the true range is shorter than the CoM range and the correction is *subtracted* in (145) (`correction -= sat_pco_corr`, `model/Antenna.cpp`). There is one real-time exception. When an applied SSR orbit correction is referenced to the antenna phase centre, the corrected orbit already points at the phase centre. The `skip_satellite_pco` path of `getAntennaCorrection` then drops the satellite-PCO term so that it is not counted twice. The path is gated by `realtime.honor_ssr_reference_point` and is off by default. The SSR reference-point convention is documented in Section 18.

Satellite PCV (added). The satellite phase-centre variation is a function of the *nadir angle*, the angle between the boresight $\hat{\boldsymbol{k}}_i$ and the line of sight, and of the azimuth in the body frame. With $\cos\theta_{\text{nad}} = \hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}$,

$$\theta_{\text{nad}} = \arccos\big(\hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}\big), \qquad \alpha_{\text{body}} = \operatorname{atan2}\big((\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{e})_y,\,(\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{e})_x\big), \tag{147}$$

and the PCV value is read by bilinear interpolation of the ANTEX grid over (zenith/nadir ring, azimuth sector), the look-up argument being $\theta_{\text{nad}} - \texttt{zen1}$ in grid units (`Antenna::interpolatePcv`, `model/Antenna.cpp`). The satellite PCV is *added* to the range correction.

Receiver PCO (added). The receiver PCO is published as a North/East/Up (NEU) triplet relative to the ARP. It is reordered to ENU, rotated to ECEF by the local ENU$\rightarrow$ECEF rotation $\boldsymbol{R}_{\text{enu}}(\phi,\lambda)$ at the receiver geodetic latitude/longitude (`Coordinates::enuToEcefMatrix`), and projected on the LOS:

$$\delta^r_{\text{PCO}} = \Big(\boldsymbol{R}_{\text{enu}}(\phi,\lambda)\,\big[\,p_E \;\; p_N \;\; p_U\,\big]^{\mathrm{T}}\Big) \cdot \boldsymbol{e}, \qquad \big[\,p_E\;p_N\;p_U\,\big] = \big[\,p_{\text{NEU},2} \;\; p_{\text{NEU},1} \;\; p_{\text{NEU},3}\,\big], \tag{148}$$

where the index swap $(N,E,U)\rightarrow(E,N,U)$ is explicit in `model/Antenna.cpp`. It is *added* to the correction.

Receiver PCV (added). The receiver PCV is indexed by *zenith angle* $z = 90^\circ - \text{el}$ (and, optionally, azimuth corrected for the antenna mounting orientation when `enableRxAntennaOrientation()` is set), with the same bilinear interpolation over $z - \texttt{zen1}$ (`model/Antenna.cpp`). It is *added* to the correction.

ATX frequency fallback. ANTEX entries often omit the newest carriers: L5/E5a/B2a on older spacecraft, and many receiver antennas published before triple-frequency tracking. Setting the antenna correction to zero would leave a vertical bias of several centimetres. The engine instead substitutes the *nearest carrier* when the requested ANTEX frequency key is absent (`model_corrections.enable_atx_frequency_fallback`, default on; `pickAtxFrequencyKey`, `model/Antenna.cpp`). The chains are ordered by carrier-frequency proximity, e.g. for GPS `G05` $\rightarrow$ {`G02`, `G01`} and for Galileo `E05` $\rightarrow$ {`E08`, `E07`, `E06`, `E01`}. The PCO of one physical antenna typically changes by less than a centimetre between frequencies. The PCO and PCV chains are searched *independently*, because an antenna may have a PCO but no PCV at a given key.

For the *satellite* antenna only the chain within the same system is allowed; another constellation's spacecraft entry would be meaningless. For the *receiver* antenna a *cross-system* chain to the nearest GPS band is allowed after the same-system chain is exhausted, and only when `model_corrections.enable_receiver_cross_system_atx_fallback` is set (default off; on in both reference configurations, where the Galileo E1 and E5a signals take the GPS L1 and L5 receiver entries; `pickReceiverAtxFrequencyKey`). This is justified because the receiver phase pattern depends on frequency, not on constellation. The important mappings are exact carrier matches (`E01` → `G01`, `E05` → `G05` at 1575.42 and 1176.45 MHz). The rest are approximations 14 to 51 MHz apart (`model/Antenna.cpp`). Each substitution sets a coverage flag and writes a log line. The same fallback policy is used for biases: substitute and report, and do not exclude the measurement.

11.3 Carrier-phase wind-up

A right-hand circularly polarised GNSS signal gains a phase shift equal to the relative rotation of the transmitting and receiving antenna dipoles about the line of sight [43]. This wind-up is the same in cycles on every signal, so as a range it is proportional to wavelength. It is computed in `GnssCorrections::computePhaseWindUp` (`corrections/GnssCorrections.cpp`). With $\boldsymbol{k}$ the unit line-of-sight, $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i)$ the satellite body $\pm X, +Y$ axes and $(\boldsymbol{x}_r, \boldsymbol{y}_r)$ the receiver local North/East axes, the *effective dipoles* are

$$\boldsymbol{D}_i = \hat{\boldsymbol{x}}_i - \boldsymbol{k}\,(\boldsymbol{k}\cdot\hat{\boldsymbol{x}}_i) - \boldsymbol{k}\times\hat{\boldsymbol{y}}_i, \qquad \boldsymbol{D}_r = \boldsymbol{x}_r - \boldsymbol{k}\,(\boldsymbol{k}\cdot\boldsymbol{x}_r) - \boldsymbol{k}\times\boldsymbol{y}_r, \tag{149}$$

which is the standard Wu construction. Each dipole is the antenna X axis projected into the plane orthogonal to $\boldsymbol{k}$, *minus* the $\boldsymbol{k}\times Y$ quadrature term (`corrections/GnssCorrections.cpp`).

The sign of the quadrature term is fixed by the antenna axes. It is $-\operatorname{sgn}\big((\boldsymbol{x}\times\boldsymbol{y})\cdot\boldsymbol{k}\big)$, which is negative when the antenna's third axis points along $+\boldsymbol{k}$. That is the case for both triads here: the satellite's $\boldsymbol{e}_z$ is the geocentric nadir, and the receiver's implied third axis is North × East = Down. The wrong sign gives $|\boldsymbol{D}| = 1 - \cos\varepsilon$, a dipole that *vanishes on boresight*. RTKLIB's $(-,+)$ pairing is stated with $\boldsymbol{y}_r = -$East and is the same construction; with the +East convention used here it becomes $(-,-)$. The construction with both signs negative [42] replaced a $(+,+)$ pairing on 2026-08-21 (`docs/SPEC_phase_windup_sign.md`); the revision is declared as `nominal-wu93` (see the paragraph on the shared convention below).

The receiver North and East axes are built from the geodetic latitude ϕ and longitude λ, $\boldsymbol{x}_r = (-\sin\phi\cos\lambda,\ -\sin\phi\sin\lambda,\ \cos\phi)$ and $\boldsymbol{y}_r = (-\sin\lambda,\ \cos\lambda,\ 0)$ (`GnssCorrections.cpp`). The signed wind-up angle is the atan2 of the cross and dot products of the unit dipoles. This covers the full range $(-\pi, \pi]$ and avoids the loss of precision of an arccos near 0 and π:

$$\boxed{\Delta\varphi = \operatorname{atan2}\Big(\boldsymbol{k}\cdot\big(\hat{\boldsymbol{D}}_i\times\hat{\boldsymbol{D}}_r\big),\ \hat{\boldsymbol{D}}_i\cdot\hat{\boldsymbol{D}}_r\Big), \qquad \varphi_{\mathrm{wu}} = \Delta\varphi\,\frac{\lambda_s}{2\pi},} \tag{150}$$

with $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{D}}_r$ the normalised dipoles and λ_s the signal wavelength (`corrections/GnssCorrections.cpp`); the $\boldsymbol{k}\cdot(\hat{\boldsymbol{D}}_i\times\hat{\boldsymbol{D}}_r)$ argument fixes the sign of the rotation about the line of sight. The result enters (145) as φ_{wu} on phase observables only (the code branch passes zero wavelength).

Fractional vs. accumulated modes. The atan2 returns the wind-up wrapped to one cycle. The *fractional* mode (`estimation.accumulate_phase_windup` false, the loader default) uses this wrapped angle directly. The float ambiguity $N^i_{r,s}$ absorbs any whole-cycle part, so the wrapping does no harm to a float solution, but each crossing of $\pm\pi$ steps the applied correction by one cycle on every signal of the satellite. Integer ambiguity resolution needs the ambiguity to stay a constant

integer across the arc. For that the engine offers an *accumulated* (unwrapped) mode, which unwraps each epoch relative to the previous accumulated angle A_{k-1} and restarts from the raw angle $\Delta\varphi_k$ otherwise:

$$A_k = \begin{cases} A_{k-1} + \mathrm{wrap}_{2\pi}\big(\Delta\varphi_k - A_{k-1}\big), & 0 < t_k - t_{k-1} < T_{\mathrm{wu}}, \\ \Delta\varphi_k \quad \text{(restart)}, & \text{otherwise}, \end{cases} \qquad \mathrm{wrap}_{2\pi}(x) = x - 2\pi\,\mathrm{round}\big(\tfrac{x}{2\pi}\big), \tag{151}$$

with T_{wu} the continuity horizon `estimation.phase_windup_max_gap_s` (default 300 s). The accumulator keeps one angle and one epoch per satellite; a repeated call within one epoch (once per signal and per rebuild pass) reuses the accumulated value and is not a restart. A larger gap, or a backward time step, restarts the accumulator (`corrections/GnssCorrections.cpp`, `computePhaseWindUp`). By (151), $A_k - \Delta\varphi_k \in 2\pi\mathbb{Z}$, so a restart steps the modelled phase of every signal of the satellite by the same whole number of cycles. On a single receiver the step cannot bias the solution, because an absent satellite's ambiguity is pruned and re-seeded on re-acquisition. Each restart is recorded as a (satellite, gap, epoch) entry, which the network's event response reads (Section 24.5). Both reference configurations accumulate.

Gap bridge. `estimation.phase_windup_gap_bridge` (default off; it needs the accumulated mode, and the loader switches it off with a warning otherwise) replaces the restart for a gap g with $T_{\mathrm{wu}} \le g < T_{\mathrm{br}}$, where T_{br} is `estimation.phase_windup_bridge_max_gap_s` (default 900 s, the horizon of the network's phase-leveling carry). The angle is walked across the gap through virtual epochs. Cut the gap into $n = \lceil g/\Delta \rceil$ equal steps, with $\Delta =$ `estimation.phase_windup_bridge_step_s` (default 30 s, accepted in $(0, 120]$). The raw angle $\theta(\cdot)$ at a virtual epoch comes from the same routine as the epoch's own angle, with the satellite state taken from the orbit products and the receiver where it is now (`Geometry::getLookBackState`). Each step is evaluated at its middle as well, and each half step is wrapped to half a cycle either way:

$$W = \sum_{i=1}^{n} \Big[\mathrm{wrap}_{2\pi}\big(\theta(t_{i-\frac{1}{2}}) - \theta(t_{i-1})\big) + \mathrm{wrap}_{2\pi}\big(\theta(t_i) - \theta(t_{i-\frac{1}{2}})\big) \Big], \tag{152}$$

$$N = \mathrm{round}\Big(\frac{A_{k-1} + W - \Delta\varphi_k}{2\pi}\Big), \tag{153}$$

with $t_i = t_{k-1} + i\,g/n$. The bridged angle is $A_k = \Delta\varphi_k + 2\pi N$: the fractional part is the epoch's own, and the walk supplies only the integer, so an error of the virtual geometry matters only if it moves the sum by a quarter of a cycle. The walk is refused, and the restart happens as without the bridge, when a virtual epoch has no satellite state or a degenerate geometry; when the satellite is in the Earth's umbra there (`Planetary::isSatelliteEclipsing`, with `processing.eclipse_shadow_scale`); when it is in a GPS noon-turn window, widened by one step of orbit motion (`SatAttitude::inNoonTurnWindow`); when a half step moves the angle by a quarter of a cycle or more; and when $|A_{k-1} + W - \Delta\varphi_k - 2\pi N|$ exceeds a quarter of a cycle (`WindUpBridge::walk`). Each attempt is counted by outcome (`WindUpBridge::Counters`), and the first attempts of a receiver are kept with their satellite, epoch and gap. The setting is UNVALIDATED in `docs/CONFIG_promotion_ledger.json`: in a six-hour real-data case the restarts fell from 139 to 91 and the refloated arcs from 116 to 23 with no change in the test station beyond 0.3 mm (`docs/refound/WGB_windup_gap_bridge_2026-10-17.md`). The simulator injects wind-up through the same function, so no simulated data can grade it.

One convention for the network, the user and the simulator. The wind-up angle depends on the satellite attitude model, the sign rule of the dipoles and the accumulation mode. A mismatch

between the network and a user is absorbed by the user's ambiguities and is invisible in the variance factor. The construction is declared by the compile-time constant `kWindUpYawModelId` = `"nominal-wu93"` (`network/solve/Sbasis.hpp`), written into the contract line of every emitted product, and a product that declares another value is refused by `NetworkProductReader::checkContract`. The bare token `"nominal"` names the earlier $(+,+)$ construction, whose products carry $2\varphi_{\mathrm{wu}}$ in their satellite clocks and phase biases. The suffix names a revision of the wind-up construction. The token does not record the accumulation mode or the noon-turn setting, so those are held by configuration. The network, the users and the simulator evaluate the same function: the simulator injects its wind-up through `GnssCorrections::computePhaseWindUp` in accumulated mode (`src/sim/EphemerisProvider.cpp`), so the simulated network, the network estimator and the user share one model.

11.4 Periodic relativistic clock correction

A clock on an eccentric orbit runs at a varying rate relative to a clock on the geoid. The broadcast or precise clock polynomial absorbs the constant and secular parts. The *periodic* part, caused by orbital eccentricity, must be applied separately. The standard expression [34], in `GnssCorrections::computeRelativisticCorrection` (`corrections/GnssCorrections.cpp`), is

$$\boxed{\delta t_{\mathrm{rel}} \;=\; -\,\frac{2\,(\boldsymbol{r}^i\cdot\dot{\boldsymbol{r}}^i)}{c^2}\quad[\mathrm{s}],\qquad \text{range term}\;=-\,c\,\delta t_{\mathrm{rel}},} \tag{154}$$

applied to the range by the $-c\,\delta t_{\mathrm{rel}}$ term of the assembly (136) (`model/PppModel.cpp`). Two points need stating.

First, the dot product $\boldsymbol{r}^i\cdot\dot{\boldsymbol{r}}^i$ is the same whether the position and velocity are expressed in the inertial frame or in the rotating ECEF frame. The ECEF velocity is $\dot{\boldsymbol{r}}^i_{\mathrm{ecef}} = \dot{\boldsymbol{r}}^i_{\mathrm{inertial}} - \boldsymbol{\omega}_\oplus\times\boldsymbol{r}^i$, and $\boldsymbol{r}^i\cdot(\boldsymbol{\omega}_\oplus\times\boldsymbol{r}^i)=0$ because the cross product is orthogonal to $\boldsymbol{r}^i$. The ECEF velocity from the broadcast ephemeris can therefore be used directly.

Second, the periodic term is computed here because the IGS precise CLK product, by convention, contains only the constant and drift parts of the clock. Applying (154) supplies the periodic part consistently for both precise and broadcast clock sources.

11.5 Shapiro (gravitational-delay) correction

The Earth's gravitational potential slows and bends the signal. The resulting Shapiro delay is a few centimetres [34]. With $r_i=\|\boldsymbol{r}^i\|$, $r_r=\|\boldsymbol{r}_r\|$ the geocentric radii and $r_{ir}=\|\boldsymbol{r}^i-\boldsymbol{r}_r\|$ the satellite–receiver distance, the delay (in metres) is

$$\boxed{S^i_r \;=\; \frac{2\,GM}{c^2}\,\ln\frac{r_i+r_r+r_{ir}}{r_i+r_r-r_{ir}},} \tag{155}$$

with $GM = 3.986005\times10^{14}\,\mathrm{m^3/s^2}$ (`Constants::GRAVITATIONAL_CONSTANT`, `common/Constants.hpp`) and the formula as coded in `GnssCorrections.cpp`. Since $r_{ir} < r_i + r_r$ the logarithm is positive, so $S^i_r>0$ and the term is *added* to the modelled range in (136) (`model/PppModel.cpp`). It is $\sim$1–2 cm for a typical MEO geometry.

11.6 Sagnac (Earth-rotation) correction

The pseudorange relates the satellite position at *signal emission* to the receiver position at *signal reception*, and both must be expressed in one ECEF frame. The ECEF frame rotates with the Earth

during the ∼70 ms transit, so the emission-time satellite coordinates are rotated into the reception-time frame. The engine applies the full rotation, not a scalar approximation. With the geometric time-of-flight $\tau = \|\boldsymbol{r}^i_{\mathrm{em}} - \boldsymbol{r}_r\|/c$ and the Earth rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$ (`common/Constants.hpp`), the emission-time position $\boldsymbol{r}^i_{\mathrm{em}}$ is rotated by $\theta = \omega_\oplus \tau$ about the Z axis,

$$\boxed{\boldsymbol{r}^i = \boldsymbol{R}_z(\omega_\oplus \tau)\, \boldsymbol{r}^i_{\mathrm{em}}, \qquad \boldsymbol{R}_z(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},} \tag{156}$$

after which $\rho^i_r = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line of sight are computed from the rotated $\boldsymbol{r}^i$ (`model/ModelGeometry.cpp`, the `rotateAndComputeTopo` lambda). The Sagnac correction is therefore *part of the geometric range*, which is why (136) has no separate Sagnac term. An equivalent scalar form, $(\omega_\oplus/c)\,(x_i y_r - y_i x_r)$, exists in the code as a *diagnostic* only (`GnssCorrections::computeSagnacCorrection`, `corrections/GnssCorrections.cpp`). It is the first-order expansion of the rotation and is not used for modelling. The Section 8 refers to this section for the Sagnac rotation. The same rotation $\boldsymbol{R}_z(\omega(t_{\mathrm{node}} - t_{\mathrm{query}}))$ is used at SP3 interpolation time to rotate ephemeris nodes into the query-epoch frame (`model/ModelGeometry.cpp`), for numerical conditioning, as described with the products.

11.7 Group delays and inter-frequency biases

Two products supply deterministic values for the bias terms of the Foundations observation equations: the satellite code and phase biases B^i_s, b^i_s and the receiver biases $B_{r,s}$, $b_{r,s}$. This section relates the products to those symbols.

Signals of the reference configurations. Both reference configurations (`config/golden/golden_field_refound.json` and `golden_sim_refound.json`) run GPS L1/L2 and Galileo E1/E5a in the dual-frequency mode (`processing.frequency_mode dual`), with one signal per band chosen by the priority rule. The Galileo priority list is `C1C`, `C1X`, `L1C`, `L1X`, `C5Q`, `C5X`, `L5Q`, `L5X`: each band is admitted in both tracking families, the pilot-only signal (`C1C`, `C5Q`) and the combined data-plus-pilot signal (`C1X`, `C5X`). The clock datum across the two families belongs to the network (Section 20.4).

Broadcast group delay. The broadcast navigation message carries a per-system inter-frequency group delay (GPS T_{GD}, Galileo BGD, BeiDou TGD). It refers each signal's code hardware delay to the ionosphere-free combination used to generate the broadcast clock. The observation model does not apply it as a correction: the broadcast clock of Section 8.6 omits it by convention, so that it matches the precise-clock baseline, and the ionosphere-free combination is free of it. The satellite code bias of an uncombined row is the precise-product value below or an estimated state. The delay is used by the network as the level of the satellite geometry-free code bias. For GPS the broadcast clock is the L1/L2 ionosphere-free clock, so $d_1 = c\,T_{\mathrm{GD}}$ and $d_2 = \gamma\, c\, T_{\mathrm{GD}}$ with $\gamma = f_1^2/f_2^2$, which makes the ionosphere-free combination of the two delays exactly zero and the geometry-free combination exactly $c\,T_{\mathrm{GD}}$:

$$\frac{f_1^2 d_1 - f_2^2 d_2}{f_1^2 - f_2^2} = 0, \qquad \frac{d_2 - d_1}{\mu_2 - \mu_1} = c\,T_{\mathrm{GD}}. \tag{157}$$

With `network.sat_gf_datum` = `broadcast_tgd` (both reference configurations) one pin row per satellite fixes that combination, over the pair of the ionosphere-free pin, at $c\,T_{\mathrm{GD}}$ for GPS and at $c\,\mathrm{BGD}(\mathrm{E1},\mathrm{E5a})$ for Galileo, and the pin is carried in time (`network.sat_gf_datum_time_linked`). The datum is developed in Section 20.4.

Precise OSB / DCB biases. When a precise bias product is loaded, the deterministic correction is the observable-specific bias (OSB) / differential code bias (DCB) entry for the exact (i,s) pair, time-windowed against its validity interval and returned by `GnssCorrections::getBiasCorrection` (`corrections/GnssCorrections.cpp`). This single scalar realises B_s^i (code) or b_s^i (phase) for that signal:

$$B_{\text{grp}} = \text{bias}(i, s, t), \qquad t_{\text{start}} \le t \le t_{\text{end}}, \tag{158}$$

with the application gated by `model_corrections.enable_code_phase_bias` (`model/PppModel.cpp`; default on, and off in both reference configurations, where the network estimates the satellite code biases itself). The bias index (`GnssCorrections::buildBiasIndex`) excludes two kinds of entry.

Zero-valued code entries, such as the MGEX placeholder `C2I/C6I` for BeiDou, are treated as *absent*. Otherwise the model would apply a null correction and also suppress the estimated bias state for that (i,s), leaving a real delay of tens of metres unmodelled.

GLONASS code entries are also treated as missing. The FDMA per-channel receiver delay cannot be represented by one value per (i,s), so those signals use the estimated-state path (`corrections/GnssCorrections.cpp`).

Estimated vs. deterministic biases. The estimator handles whatever the deterministic bias does not correct: mainly the receiver hardware delay $B_{r,s}$, $b_{r,s}$, and any satellite delay that no product covers. The receiver biases are common to all satellites on a signal. They are absorbed by the receiver clock and inter-system bias states, or cancel under between-satellite differencing (Section 5). The per-(i,s) satellite code bias may be *estimated* as a state when no product covers it (`estimation.estimate_sat_biases`, `addSatelliteCodeBias` of `measurement/ObservationEquation.cpp`). The network estimator estimates the satellite code biases of both reference configurations through its own columns (`network.estimate_sat_code_bias`). The state-allocation gate that prevents estimating a bias the products already supply, hasBIA(i, s) from the static product, OR'd with hasSsrCB(i, s) from the live stream, is documented with the SSR overlay (Equation (304)); the present document supplies only the *deterministic* half of that OR.

Relation to the SSR overlay. The streamed SSR code/phase OSB (Section 18) refines the *same* B_s^i, b_s^i terms in real time, as a known term of the observation equation (`addSsrBias` in `measurement/ObservationEquation.cpp`, with the sign constant `kSatBiasSign`) alongside the deterministic B_{grp} here. The network estimator takes no SSR bias term, because it estimates the biases as states and importing the same quantity would count it twice. The SSR bias is a per-(i,s) scalar like (158). The SSR *orbit* delta is different: it is expressed in a radial/along/cross (RAC) frame (Equation (297)), which is *not* the satellite body frame (139) of this document. The RAC frame is built from the broadcast position and velocity to express an orbit correction. The body frame is built from the Sun direction to orient the antenna. The two frames differ in construction and in purpose.

11.8 Provenance and assembly

All the deterministic corrections are combined in `model/PppModel.cpp`. `computeBaseModeledValue`, shared by the iono-free-pair and uncombined paths, returns the sum (136). It calls `model/ModelGeometry.cpp` for the Sagnac-rotated geometric range (156), `model/Antenna.cpp` for the total antenna correction (145) (which in turn calls `model/SatAttitude.cpp` for the body frame and `corrections/GnssCorrections.cpp` for the wind-up), and `corrections/GnssCorrections.cpp` for the relativistic (154), Shapiro (155) and bias (158) terms. The Sun and Moon ephemeris feeding

the attitude frame (137), the Sun-elevation (140), the eclipse predicate (144) and the wind-up (149) is produced by `model/Planetary.cpp` (an analytic RTKLIB/Meeus series rotated to ECEF through the IAU 1976/1980 precession-nutation chain, cached per epoch) and is documented in Section 6. Every term is a fixed physical model. The user-facing knobs are enable toggles and the eclipse-shadow scale; they are cited rather than reproduced because they set *whether* a correction is applied, never its functional form.

Part III
Estimation engine

12 Estimation engine: the Kalman filter and RTS smoother

This section documents the engine's recursive estimator. It is a variable-dimension extended Kalman filter (EKF) with an optional prior-anchored iterated (IEKF) relinearisation pass, robust gating of post-fit residuals, and a backward Rauch–Tung–Striebel (RTS) smoother for post-processing. The state vector $\boldsymbol{x}$ is the uncombined PPP / PPP-RTK parameterisation of Section 3: receiver position (and optionally velocity), receiver clock and per-system time offsets, zenith wet troposphere and gradients, per-satellite slant ionosphere, receiver and satellite code biases, and one float ambiguity per tracked carrier. The dimension changes from epoch to epoch as satellites rise and set and as ambiguities are reset. `filter/StateManager` owns the layout and the dynamic key→index maps. The filter operates on that state and assumes no fixed indices (`src/core/filter/KalmanFilter.cpp`; its sparse products are in `KalmanKernels.cpp`, the post-fit screen in `KalmanPostFitQc.cpp`, the statistical residual QC in `KalmanResidualQc.cpp` and the outlier responses in `KalmanOutlierResponse.cpp`). The network estimator does not use this filter (Section 20); it solves per-epoch normal equations and links epochs by carried priors.

The notation throughout is $\boldsymbol{F}$ for the state-transition matrix, $\boldsymbol{Q}$ for the discrete process-noise covariance, $\boldsymbol{R}$ for the measurement-noise covariance, $\boldsymbol{S}$ for the innovation covariance, $\boldsymbol{K}$ for the Kalman gain, and $\boldsymbol{\nu}$ for the innovation (measurement minus predicted measurement). Predicted (a-priori) and updated (a-posteriori) quantities use the tilde/hat and $k|k{-}1$ / $k|k$ conditioning convention fixed in Section 2.

12.1 Time update

State transition. The transition matrix $\boldsymbol{F}(\Delta t)$ for epoch interval $\Delta t = t_k - t_{k-1}$ is identity plus a small number of coupling blocks, assembled by `StateManager::getTransitionMatrix` (`src/core/filter/StateManagerProcessNoise.cpp`). The kinematic-integrator off-diagonal couplings are

$$\boldsymbol{F}[\,\boldsymbol{r},\boldsymbol{v}\,] = \Delta t\,\boldsymbol{I}_3, \qquad \boldsymbol{F}[\,c\mathrm{d}t,\ c\dot{\mathrm{d}}t\,] = \Delta t, \qquad \boldsymbol{F}[\,c\delta t^{(g)},\ c\dot{\delta} t^{(g)}\,] = \Delta t, \tag{159}$$

present only when the corresponding rate state exists (`hasVelocity()`, clock-rate, per-system rate). A fourth coupling appears when the constant-velocity slant-iono model is armed (`estimation.enable_iono_rate`, default off): a per-satellite level/rate integrator

$$\boldsymbol{F}[\,I^i,\ \dot{I}^i\,] = \Delta t \qquad \text{for every non-frozen-pivot satellite,} \tag{160}$$

which gives $I^i(k{+}1) = I^i(k) + \Delta t\,\dot{I}^i(k)$. Every other state has $\boldsymbol{F} = \boldsymbol{I}$ and is a random walk with constant mean: ambiguities, zenith wet delay, gradients, code and satellite biases, any inter-system offset without a rate, and (without `enable_iono_rate`) the per-satellite ionosphere.

Under the BSSD (single-difference) parameterisation some states are unobservable, because their $\boldsymbol{H}$ columns are zero: the receiver clock, the phase clock (`getPhaseClockIndex`), the inter-system offsets, and the frozen-pivot per-satellite ionosphere (`getFrozenPivotSatellites`). Their rows in Equation (159) are suppressed. After propagation their cross-covariances are set to zero and their diagonal variances are kept (the `freeze_state` block in `src/core/filter/KalmanFilter.cpp`).

The iono level/rate coupling of Equation (160) is *not* disabled under BSSD. The slant-iono level stays observable under single-differencing, since only the pivot column is zero, so every satellite other than the frozen pivot keeps its integrator. The frozen pivot is skipped. Its rate is held at $(0, 0)$ and its process noise is suppressed, so the coupling cannot move the pivot level away from the SD datum.

The propagation itself is the standard EKF time update

$$\boxed{\begin{aligned}\tilde{\boldsymbol{x}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\hat{\boldsymbol{x}}_{k-1|k-1}, &(161)\\ \tilde{\boldsymbol{P}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\boldsymbol{P}_{k-1|k-1}\,\boldsymbol{F}(\Delta t)^{\mathrm{T}} + \boldsymbol{Q}(\Delta t), &(162)\end{aligned}}$$

implemented in `KalmanFilter::predict`. A non-positive Δt (a duplicate or out-of-order epoch) does nothing: the routine returns before Equation (162).

Process-noise model. The discrete process noise $\boldsymbol{Q}(\Delta t)$ is built from continuous spectral densities by `StateManager::getProcessNoiseMatrix`, a short driver over one function per block of $\boldsymbol{Q}$ (`StateManagerProcessNoise.cpp`); the family blocks read their rule and spectral-density key from the family table (Section 3.3), which the run-start log prints. Two block structures appear. Pure random-walk states have a scalar discrete variance increment

$$\boldsymbol{Q}_{ii} = q_i\,\Delta t, \tag{163}$$

where q_i is the continuous variance rate (m^2/s) read from `filter.process_noise.*`. Representative entries are zenith wet troposphere (`troposphere_q`), the receiver code bias (`code_bias_q`), the satellite *code* bias (`sat_bias_q`, applied to `StateManager::sat_bias_indices_`), the satellite *phase* bias (`sat_phase_bias_q`, a separate density on the separate `sat_phase_bias_indices_` map), phase clock (`phase_clock_q`), and the float ambiguity (`ambiguity_q`). The three satellite blocks (code bias, phase bias and clock, one function over the family table) set $\boldsymbol{Q}$ to zero for a frozen-pivot satellite, as do the ambiguity and ionosphere blocks. The frozen state is held at (value, variance) $= (0, 0)$ as the datum, and any process noise would move it off that datum. The ambiguity density defaults to a numerical floor (10^{-12}), not a physical value. Per-key overrides are supported for reseeding individual arcs. The per-satellite slant ionosphere uses a random walk whose zenith density is mapped to the slant by the squared obliquity factor,

$$\boldsymbol{Q}_{ii} = \frac{q_{\mathrm{iono}}}{\sin^2\theta^i}\,\Delta t, \qquad \sin^2\theta^i \text{ floored at } 10^{-2}\ (\theta \approx 5.7^\circ), \tag{164}$$

with q_{iono} = `ionosphere_q` and optional per-satellite overrides driven by the adaptive-iono scheme (off by default). The rule applies on every run that carries slant-ionosphere states (`GnssConfig::carriesSlantIonosphere`: uncombined processing outside the baseline modes), PPP included. Under the constant-velocity model of Equation (160) this random walk is used only for satellites without a rate state. Otherwise the level/rate pair takes the CWNA block of Equation (165) below, with the acceleration PSD scaled by obliquity, $q = q_{\mathrm{iono}}/\sin^2\theta^i$, where q_{iono} = `process_noise.iono_rate_psd` (an acceleration PSD, $\mathrm{m}^2/\mathrm{s}^3$, default 10^{-10}). The adaptive-monitor override enters this branch as the *dimensionless* ratio $q_{\mathrm{ov}}/q_{\mathrm{iono}}$, which multiplies the rate PSD. The Sage–Husa storm response therefore acts on $\dot{I}^i$, not on the level.

Coupled position/velocity, clock/clock-rate, inter-system offset/rate, and, under `enable_iono_rate`, slant-iono level/rate pairs use the continuous white-noise-acceleration (CWNA, integrated-random-walk) discretisation. For a coupled pair $(\xi, \dot{\xi})$ driven by acceleration PSD q,

$$\boldsymbol{Q}_{[\xi,\dot{\xi}]} = \begin{bmatrix} \frac{1}{3}\,q\,\Delta t^3 & \frac{1}{2}\,q\,\Delta t^2 \\ \frac{1}{2}\,q\,\Delta t^2 & q\,\Delta t \end{bmatrix}, \tag{165}$$

with q given by `acceleration_psd_q` (position/velocity, the 3×3 block-diagonal extension of Equation (165) over the ECEF axes), `clock_rate_q`, `system_time_rate_offset_q`, and the obliquity-scaled `iono_rate_psd`$/\sin^2\theta^i$ respectively. When the velocity state is present the rate-free position density `position_q` is unused. Baseline (double-difference) mode reduces $\boldsymbol{Q}$ to the position block, a per-ambiguity random-walk term, and an optional differential-ZWD entry $\boldsymbol{Q}_{ii} = \sigma^2 \Delta t$ governed by `baseline.differential_tropo_process_noise_m_per_sqrt_s`.

Scope: the recursive estimator only. These densities apply to the *recursive* estimator. The network estimator does not run a `KalmanFilter`. `getTransitionMatrix` and `getProcessNoiseMatrix` are called only from `KalmanFilter::predict` / `predictAided` and from the smoother's `Application::capturePriorSnapshot`. The network solves per-epoch normal equations for corrections to each station's linearisation point. It links epochs with explicit information-form carry priors of effective variance $\mathrm{var}_{\mathrm{eff}} = \mathrm{var} + q\,\Delta t$, not through $\boldsymbol{Q}(\Delta t)$ (Section 20). There the satellite-side densities `sat_clock_q`, `sat_phase_bias_q` and `sat_code_bias_q` are read as those carry rates. They can also be set through the network-scoped `network.process_noise` class table, which is always active. Its per-class fallbacks equal the legacy `filter.process_noise.*` defaults, so an unmodified config gives the same values either way.

Aided time update. When inertial aiding is active, the aiding source (`aiding/ImuAidingSource`) mechanises the nominal position, velocity and attitude (strapdown) over the integrated interval and returns a predict contribution: a 15×15 INS error-state transition $\boldsymbol{\Phi}$ and noise $\boldsymbol{Q}_{\mathrm{ins}}$, with error order $[\delta\boldsymbol{p}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$, with the full-state indices it owns. `KalmanFilter::predictAided` writes each contribution over an identity $\boldsymbol{F}$ and over the baseline $\boldsymbol{Q}$ built for the integrated interval (`spliceAndPropagate`, `KalmanKernels.cpp`). The write is an assignment on the rows the contribution owns, so their baseline process noise is not counted twice, and the means of states no contribution owns are not propagated. The covariance recursion of Equation (162) then runs on the full state, followed by the symmetrisation $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$. With no valid contribution `predictAided` calls the GNSS-only `KalmanFilter::predict` and gives a byte-identical result.

12.2 Measurement update

The measurement update (`KalmanFilter::update`) is a *batch* update: all observation rows of the epoch are processed together, not one at a time. A sparse-$\boldsymbol{H}$ optimisation speeds up the four products that involve the design matrix ($\boldsymbol{HP}$, $\boldsymbol{HPH}^{\mathrm{T}}$, $\boldsymbol{KH}$ and $\boldsymbol{H}\,\delta\boldsymbol{x}$). It agrees with the dense computation: entries of $\boldsymbol{H}$ at or below 10^{-12} are dropped from the row lists (`src/core/filter/KalmanKernels.cpp`).

Given the design matrix $\boldsymbol{H}$ from the linearisation of the Equation (4) and Equation (5) of the Foundations document (position rows $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$, the Equation (6)), the innovation $\boldsymbol{\nu}$ (`measurement.innovations`), and the measurement-noise covariance $\boldsymbol{R}$ (a full matrix when supplied, otherwise

the diagonal $\boldsymbol{R} = \mathrm{diag}(\sigma^2_{r,i,s})$ of the Equation (9)), the innovation covariance and gain are

$$\boldsymbol{S} = \boldsymbol{H}\,\tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}, \tag{166}$$

$$\boldsymbol{K} = \tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}}\,\boldsymbol{S}^{-1}. \tag{167}$$

The gain is not formed by inverting $\boldsymbol{S}$. $\boldsymbol{S}$ is symmetric, so the engine factorises it once with an $\mathrm{LDL}^{\mathrm{T}}$ decomposition and obtains $\boldsymbol{K} = \left(\boldsymbol{S}^{-1}\boldsymbol{H}\tilde{\boldsymbol{P}}\right)^{\mathrm{T}}$ from one solve against $\boldsymbol{H}\tilde{\boldsymbol{P}}$ (`ldlt.solve(HP).transpose()`). If the factorisation fails it falls back to a complete-orthogonal (pseudo-inverse) solve. The state update is

$$\delta\boldsymbol{x} = \boldsymbol{K}\,\boldsymbol{\nu}, \qquad \hat{\boldsymbol{x}}_{k|k} = \tilde{\boldsymbol{x}}_{k|k-1} + \delta\boldsymbol{x}. \tag{168}$$

Joseph-form covariance update. The covariance update uses the *Joseph* form, not the short form $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\tilde{\boldsymbol{P}}$:

$$\boxed{\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\tilde{\boldsymbol{P}}_{k|k-1}\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}.} \tag{169}$$

The Joseph form keeps the covariance symmetric and positive semi-definite for any gain. That property is needed because the robust paths of Section 12.4 use a gain that is not the optimal one. The result is symmetrised again, $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$, to remove round-off. The factor $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})$ is built from the sparse $\boldsymbol{K}\boldsymbol{H}$ product by adding one on the diagonal. The same kernel, with the explicit gain $\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^{\mathrm{T}}\boldsymbol{S}^{-1}$, is the static routine `josephPseudoUpdate` (`KalmanKernels.cpp`). It is reused for the pseudo-observation constraints applied through `applyPseudoObservation` (the GIM ionosphere prior and the a-priori ZWD constraint) and for the IEKF pass of Section 12.3.

Post-fit residuals. After the update the post-fit residuals and their (diagonal) covariance are

$$\boldsymbol{r} = \boldsymbol{\nu} - \boldsymbol{H}\,\delta\boldsymbol{x}, \qquad S_{r,ii} = R_{ii} - \left(\boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}\right)_{ii}, \tag{170}$$

By default only the diagonal of S_r is formed. The full symmetrised $S_r = \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}$ is formed and stored when two conditions hold: the off-diagonal-R kernel is enabled (`enable_full_r_kernel`, default off), *and* a dense $\boldsymbol{R}$ was supplied (BSSD and baseline-DD epochs, whose $\boldsymbol{R}$ carries SD/DD cross-covariance). The redundancy and w-test QC of Section 12.4 then account for correlated observations through S_r^{-1}. The diagonal, and so Equation (171), is the same either way. The standardised residual used for gating (Section 12.4) is

$$s_i = \frac{r_i}{\sqrt{\max\{\, S_{r,ii},\, R_{ii},\, 10^{-12} \,\}}}. \tag{171}$$

The denominator is floored at the *working* R_{ii}, the inflated noise used in $\boldsymbol{R}$, not the formal noise. This is a deliberate compromise. When an ambiguity absorbs the measurement (double-differenced phase with an ambiguity gain near one), the rigorous denominator $S_{r,ii}$ shrinks toward $R\,\boldsymbol{S}^{-1}R$, and residuals near zero would register as outliers of many σ. The strictly correct standardisation uses $S_{r,ii}$ alone. The floor gives up that rigour for stability, and the source code says so.

12.3 Iterated relinearisation (IEKF)

When the initial position is far from the truth, one linearisation of the Equation (6) per epoch may not be enough. When `processing.relinearize_geometry` is enabled, and the run is not tightly-coupled IMU, the engine performs a prior-anchored iterated update (a Gauss–Newton, or iterated EKF, step) after the primary update.

Anchor. The iteration is anchored to the *stored pre-update prior* $(\boldsymbol{x}^-, \boldsymbol{P}^-)$ that the primary update Equations (162) and (169) used. It is captured only when relinearisation is armed (`getLastUpdatePriorX` / `getLastUpdatePriorP`). It is the prior *after* any in-epoch ambiguity augmentation and outlier reset, so its dimension matches the posterior. It is not a snapshot from the top of the epoch loop.

Gauss–Newton step. Let $\boldsymbol{x}_i$ be the current linearisation iterate (the committed state at which `measurement` was linearised, with innovations $\boldsymbol{z} - h(\boldsymbol{x}_i)$ and $\boldsymbol{H} = \partial h/\partial \boldsymbol{x}\big|_{\boldsymbol{x}_i}$). The prior-anchored iterate (`KalmanFilter::updateFromPrior`, via `josephPseudoUpdate`) is

$$\boldsymbol{\nu} = \big(\boldsymbol{z} - h(\boldsymbol{x}_i)\big) - \boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i), \tag{172}$$

$$\boldsymbol{K} = \boldsymbol{P}^-\boldsymbol{H}^{\mathrm{T}}\big(\boldsymbol{H}\boldsymbol{P}^-\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}\big)^{-1}, \qquad \boldsymbol{x}^+ = \boldsymbol{x}^- + \boldsymbol{K}\,\boldsymbol{\nu}, \tag{173}$$

$$\boldsymbol{P}^+ = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\boldsymbol{P}^-\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}. \tag{174}$$

The offset term $-\boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i)$ in Equation (172) expresses the innovation about the fixed prior, which makes this a MAP (Gauss–Newton) iteration. $\boldsymbol{K}$ and $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^-$ are always computed from the same fixed prior $(\boldsymbol{x}^-, \boldsymbol{P}^-)$, so the covariance does *not* shrink a second time and no information is counted twice.

Firing condition. The driver `Application::runGeometryRelinearization` (`src/core/app/PppEpochUpdate.cpp`) fires the iteration when all of the following hold:

1. `processing.relinearize_geometry` is on and a prior was stashed at the primary update;
2. the position change since the primary update exceeds a data-dependent threshold, `relinearize_dpos_threshold_phase_m` (default 1 m) when phase is present, or `relinearize_dpos_threshold_code_only_m` (default 100 m) for code-only epochs;
3. the per-run cap `relinearize_max_count` (-1 = unlimited) is not exhausted.

The inner loop recomputes the geometry and re-applies Equations (172) to (174) against the *fixed* exclusion set from the primary update. It runs at most five iterations and stops when the position step falls below 10^{-4} m. GNSS geometry is nearly linear at the metre level, so one pass usually converges, and the threshold of item 2 means the iteration rarely fires. The tightly-coupled IMU configuration takes a single legacy *naive* second pass, re-applying the measurement against the posterior, because error-state attitude and bias feedback conflicts with re-deriving the update from the prior.

12.4 Innovation and residual gating

The engine performs *no* χ^2 test on the innovation covariance. Outlier rejection (`KalmanFilter::validatePostFitResiduals`, a driver over named stages in `KalmanPostFitQc.cpp`) tests the standardised *post-fit* residuals s_i of Equation (171) against a fixed scalar threshold. Robust median-based detectors for pivot slips supplement it. The stages run in this order: the residual and statistical reports, the coherent pivot-offset detectors, the non-pivot multi-band slip detector, the Huber inflation pass, the gate (the σ threshold or the Baarda w-test), the response to the gated row and the complete-reset backstop. A detector that rejects a group ends the pass before the later stages. The two top-level thresholds are hardcoded `GnssConfig` defaults (`residual_threshold_` = 6.0 and `bad_meas_percent_` = 0.5 in `src/core/config/GnssConfig.cpp`). They are not exposed

to JSON, so there are no config keys of those names. The per-row rejection threshold is $s_{\max} = 6.0$ (read at runtime via `getResidualThreshold`). The fraction that triggers a complete reset is 0.5 (`getBadMeasPercent`). The Baarda data-snooping gate below, which is exposed to JSON, can replace both the dispatched row and the criterion.

Per-row rejection. If $\max_i |s_i| > 6.0$, the worst row is handled according to its type (`KalmanOutlierResponse.cpp`). A phase outlier triggers an ambiguity reset, with BSSD pivot handling. A real slip moves the observation by an integer number of wavelengths, so a raw residual below $\lambda/4$ (`estimation.pivot_slip_min_cycles`, default 0.25, with `estimation.enable_subcycle_slip_suppression`, default on) is sub-cycle bias: the row is excluded from the pass and its ambiguity state is kept. A code outlier is excluded from the epoch. Correlated phase rows of the same satellite on other bands are reset with it. Among phase outliers of one constellation within 5 % of the maximum $|s|$, the satellite with the largest Melbourne–Wübbena deviation score is chosen.

Baarda data-snooping gate (optional override). When `estimation.enable_data_snooping` is set, the fixed-σ gate triple (gate stat, gate row, gate crit) = ($\max_i |s_i|$, arg max, 6.0) is *replaced* in `validatePostFitResiduals` by the Baarda w-test. The test runs over *checkable* rows, those whose redundancy $r_i = S_{r,ii}/R_{ii}$ is at least 0.1. Rows with lower redundancy are skipped, because a blunder there goes into the estimate and not the residual. The row with the largest $|w_i|$ is selected, and the gate fires when $|w_i| >$ `estimation.data_snooping_critical` (default 3.29). Both the criterion *and* the dispatched row can therefore differ from the σ gate. The handling of the selected row by type is unchanged. The w-test statistic and redundancy are produced by the a-posteriori QC diagnostics layer (`computeResidualQc_` in `KalmanResidualQc.cpp`, run when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set): the variance factor $\hat{\sigma}_0^2 = \left(\sum_i v_i^2/R_{ii}\right)/\mathrm{dof}$, the per-row redundancy r_i, the w-test, and the minimal detectable bias (MDB). This layer accounts for a correlated R through S_r^{-1} when the full-R kernel is active (see Equation (170)).

Complete-reset backstop. If the number of rejected rows exceeds $0.5\times$ the original observation count (before any rows were removed), the filter covariance is re-inflated (`resetFilterCovariance`). This is a hard reset. In the smoother it marks a segment boundary (Section 12.5).

Robust pivot-slip detectors (BSSD). Before the per-row gate, single-difference rows are grouped by (pivot, signal) and screened for a disturbance common to the group, using median and median-absolute-deviation (MAD) statistics. Four detectors are the rows of one table (`kalman::kPivotDetectors`): phase per signal, phase pooled across bands, code per signal and code pooled across bands. A forced pivot handover fires when four conditions hold: the group is large enough (four rows in a per-signal group; three rows on two bands in a pooled one), at least half of its rows exceed the per-row threshold, the group median exceeds a floor (5 mm on phase, 0.30 m on code, above routine code multipath), and the MAD-to-median ratio is at most 0.4. A small ratio means a tight cluster, which indicates a common shift and not scattered noise. On phase per signal the median must also exceed the wavelength fraction $\lambda/4$ of `estimation.pivot_slip_min_cycles`, unless the cluster is tight (ratio at most 0.2), where only a common pivot disturbance explains several rows. A firing group requests a handover that excludes the satellites whose residuals disagree with the group median, and its rows are left out of the pass. Slips on non-pivot satellites across several bands are confirmed by integer-cycle consistency: $|r|/\lambda_{\mathrm{band}}$ must round to the same nonzero integer on each band.

Down-weighting (not gating). Two paths inflate $\boldsymbol{R}$ and do not reject. A persistent EWMA-bias term adds the squared mean bias to R_{ii} once an arc has warmed up (`estimation.persistent_bias_min_cycles`). A Huber post-fit scheme inflates R for sub-cycle rows whose $|s_i|$ exceeds the Huber knee (`estimation.phase_huber_sres_threshold`, default 6), iteratively on the post-fit residual (IRLS, the default) or once on the innovation, toggled by `enable_huber_r_inflation` (default on) and `enable_huber_innovation` (default off). Both change the working $\boldsymbol{R}$ used in Equations (166) and (171). The standardised residual is therefore scaled by the working noise, not the formal noise.

12.5 Backward RTS smoother

In post-processing modes, enabling `processing.run_smoother` runs a fixed-interval Rauch–Tung–Striebel recursion [28] over the stored forward trajectory (`include/filter/Smoother.hpp`, `src/core/filter/Smoother.cpp`). The flag is forced off under `AppMode::RealTime`, because smoothing uses future data.

Forward-pass capture. The backward pass does not reconstruct the a-priori from $\boldsymbol{F}$ and $\boldsymbol{Q}$. The forward pass stores, per epoch, the one-step prediction $\tilde{\boldsymbol{x}}_{k|k-1}, \tilde{\boldsymbol{P}}_{k|k-1}$, the posterior $\hat{\boldsymbol{x}}_{k|k}, \boldsymbol{P}_{k|k}$, the matrix $\boldsymbol{F}_k$, and all core and dynamic key→index maps (`capturePriorSnapshot` / `capturePosteriorSnapshot` in `src/core/app/PppEpochRecords.cpp`). The order of capture makes the stored prior equal to $\boldsymbol{F}_k \hat{\boldsymbol{x}}_{k-1|k-1}$. The a-priori is recorded immediately after `predict()`, before any cycle-slip reseeding or measurement update. The a-posteriori is recorded *before* integer ambiguity resolution, so the smoother uses *float* ambiguities only. One edit follows the capture: a covariance-preserving integer repair of an ambiguity (the TurboEdit repair of Section 15) moves the ambiguity by a known $\mathrm{d}N\,\lambda$ in the frame of the stored prior. It is folded into $\tilde{\boldsymbol{x}}_{k|k-1}$ as a deterministic control input (`Application::foldSlipRepairsIntoPrior_`), so the backward pass does not read the jump as an innovation; a satellite that was also reset in the epoch is segmented instead (below).

Recursion. Initialised at the terminal posterior $\hat{\boldsymbol{x}}_{N|N} = \hat{\boldsymbol{x}}_{N|N}$, $\boldsymbol{P}_{N|N} = \boldsymbol{P}_{N|N}$, the backward recursion is

$$\boxed{\begin{aligned} \boldsymbol{G}_k &= \boldsymbol{P}_{k|k}\,\boldsymbol{F}_{k+1}^{\mathrm{T}}\,\tilde{\boldsymbol{P}}_{k+1|k}^{-1}, && (175)\\ \hat{\boldsymbol{x}}_{k|N} &= \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{G}_k\big(\hat{\boldsymbol{x}}_{k+1|N} - \tilde{\boldsymbol{x}}_{k+1|k}\big), && (176)\\ \boldsymbol{P}_{k|N} &= \boldsymbol{P}_{k|k} + \boldsymbol{G}_k\big(\boldsymbol{P}_{k+1|N} - \tilde{\boldsymbol{P}}_{k+1|k}\big)\boldsymbol{G}_k^{\mathrm{T}}. && (177) \end{aligned}}$$

The smoother gain $\boldsymbol{G}_k$ is computed by an $\mathrm{LDL}^{\mathrm{T}}$ solve, not an explicit inverse. The predicted covariance is symmetrised, then $\boldsymbol{G}_k = \big(\tilde{\boldsymbol{P}}_{k+1|k}^{-1}(\boldsymbol{P}_{k|k}\boldsymbol{F}^{\mathrm{T}})^{\mathrm{T}}\big)^{\mathrm{T}}$ via `ldlt.solve(...).transpose()`. The smoothed covariance Equation (177) is symmetrised after the update. If the factorisation fails, the step is skipped and epoch k keeps its forward posterior.

Variable-dimension reconciliation. Because $\dim \boldsymbol{x}_k \neq \dim \boldsymbol{x}_{k+1}$ in general, Equations (175) to (177) are restricted at each step to the *common-key subspace*, the states present at both k and $k{+}1$, found by intersecting the dynamic key→index maps (`collectCommonIndices`, less any keys in the handover exclusion set of the segment-boundary paragraph below). Let $\boldsymbol{\Pi}_k$ and $\boldsymbol{\Pi}_{k+1}$ select the common rows in each epoch's state space; the implemented state update is then

$$\hat{\boldsymbol{x}}_{k|N} = \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{\Pi}_k^{\mathrm{T}}\,\boldsymbol{G}_{k,C}\big(\boldsymbol{\Pi}_{k+1}\hat{\boldsymbol{x}}_{k+1|N} - \boldsymbol{\Pi}_k\tilde{\boldsymbol{x}}_{k+1|k}\big), \tag{178}$$

with the analogous projection on Equation (177). States outside the subspace keep their forward a-posteriori value. The subspace holds the position and the ZWD; the receiver clock, velocity, phase clock and gradients when both epochs have them; and the ambiguity, ionosphere, ionosphere-rate, receiver code-bias, satellite code-bias and inter-system offset keys present at both epochs. The clock-rate, inter-system rate and inertial states are not in it and keep their forward value. The indexing needs care. $\tilde{\boldsymbol{x}}_{k+1|k}$, $\tilde{\boldsymbol{P}}_{k+1|k}$ and $\boldsymbol{F}_{k+1}$ are sized for epoch k's layout, because the predict step does not grow the state. They are addressed with epoch k's common indices. $\hat{\boldsymbol{x}}_{k+1|N}$ and $\boldsymbol{P}_{k+1|N}$ use epoch $k{+}1$'s layout.

Segment boundaries at resets and frame changes. Four mechanisms stop or restrict the backward flow of information at ambiguity resets, teardowns, and pivot handovers:

- A *complete* filter reset at $k{+}1$ (the covariance re-inflation of Section 12.4, recorded as `is_discontinuity`) is a hard segment boundary. It breaks the dynamic link $\tilde{\boldsymbol{x}}_{k+1|k} = \boldsymbol{F}\hat{\boldsymbol{x}}_{k|k}$, so $\boldsymbol{G}_k$ has no meaning. Backward flow stops and epoch k keeps its forward posterior.
- A *single-satellite* ambiguity reset is handled softly. That satellite's ambiguities are excluded from the common subspace for the $k \to k{+}1$ step. The re-seeded prior at $k{+}1$ has a large variance and carries no information backward, so smoothing it would only add noise. The exclusion matches by satellite prefix and is slightly broader than necessary. The extra states keep their forward values.
- A *dimension change* $\dim \tilde{\boldsymbol{x}}_{k+1|k} \neq \dim \hat{\boldsymbol{x}}_{k|k}$ is a hard boundary. Predict never grows the state, so a size mismatch means a long-gap teardown before predict (`Application::handleGap` prunes ambiguity, iono and sat-bias states before `predict()`). Across it, epoch k's indices would address the wrong states in the $k{+}1$ a-priori. Backward flow stops and epoch k keeps its forward posterior.
- A *BSSD pivot handover* between k and $k{+}1$ re-references one or more constellations in place. Each constellation is either reconciled or segmented, as described next.

BSSD pivot-handover reconciliation. When the pivot map changes between epochs ($\texttt{pivot_satellites}_k \neq \texttt{pivot_satellites}_{k+1}$) the smoothed posterior at $k{+}1$ is in the *new* pivot frame. The cached a-priori $\tilde{\boldsymbol{x}}_{k+1|k}$ was captured before the handover, in epoch k's layout, and is still in the *old* frame. The two cannot be compared directly in Equation (176). `Smoother::prepareHandoverFrame` makes them comparable, per constellation, on *copies* of the $k{+}1$ smoothed estimate. The reported $k{+}1$ result stays in its own frame. A handover that `PivotManager` recorded as *clean* (old and new pivots matching `clean_handovers`) is inverted back to the old pivot p_0, one signal group at a time, by the public static `Smoother::inverseReReferenceGroup`. This is the exact inverse of the forward re-reference. For a group with old-pivot datum row d,

$$x'(i) = x(i) - x(d), \quad x'(d) = 0, \qquad P'(i,\cdot) = P(i,\cdot) - P(d,\cdot), \quad P'(\cdot,j) = P(\cdot,j) - P(\cdot,d), \tag{179}$$

applied over *full* rows and columns, with the datum row and column set to zero last. Full rows are needed because the forward re-reference coupled the group to every other state. A signal group can be inverted only if the old pivot still has a live state on it *and* the forward step re-referenced that signal to the new pivot. Everything else is *segmented*: orphaned groups (the new pivot lacked the signal), disturbed or forced handovers, and the ionosphere and satellite-bias states of every handover constellation. Their keys are added to the `exclude_keys` set, which `collectCommonIndices` removes from the common subspace, and those states keep their forward posterior.

Static-position degeneracy. In static PPP the receiver position is nearly constant. The smoother gain for the position block then applies almost the same correction to every epoch, and the smoothed position is constant across epochs *by construction*. This is expected behaviour. The smoother improves the other states (troposphere, ionosphere, float ambiguities), and the position in kinematic runs, where it changes from epoch to epoch.

13 Stochastic Model and Quality Control

This document covers the *measurement* noise of the estimator. It describes the strategies that fill the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that tests an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ here has the block-diagonal structure of Equation (9), and this document gives the variance each strategy puts in it. The QC layer uses the post-fit residual statistics that the filter already computes for outlier rejection, and presents them in the classical geodetic detection theory of Baarda and Teunissen [39]. The network estimator has no Kalman residuals of this kind; its own quality control (a robust variance factor, the RAIM tiers, the block and carry statistics and the measure-only instruments) is treated in the later subsections, from Section 13.2 onward.

Two topics are outside this document and are only cross-referenced. The first is the reaction to a detected blunder: the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA inflation of $\boldsymbol{R}$. These are in Section 12.4. This document explains how the QC statistics drive an *alternative* gate but does not repeat the dispatch. The second is the process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias), which belongs to the dynamics and is described with the smoother (Equation (164)). Everything below is measurement noise.

13.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch is in `src/core/model/StochasticModel.cpp`. Each strategy returns a variance in m^2, which becomes one diagonal entry of $\boldsymbol{R}$. The three strategies that use geometry share a low-elevation guard. When $\sin\theta \leq 10^{-6}$ the elevation amplification is skipped and the unamplified variance is returned, so a satellite at the horizon does not receive a numerically infinite variance (`computeElevation`). The `Constant` strategy has no elevation term. It returns its a-priori variance directly (`computeConstant`) and never reaches the guard.

(a) Constant. This strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{180}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). The other strategies scale this value.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{181}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both one, so the shipped model is *symmetric*: code and phase both follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent is $-2p_{\text{type}}$ because the strategy works in variance, not σ: $\left(\sin\theta\right)^{-2p} = \left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` take the code power. That is every row except carrier phase, including Doppler and the fallback paths with an empty `obs_type`.

Both exponents are configurable per observable class. The asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument, so the reasons it is an *option* and not the default are recorded here. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), which amplifies much less at low elevation.

The argument for it has two parts. Phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, and neither follows the $\csc^2\theta$ law that governs code multipath and pseudorange path-length error. The milder amplification also keeps low-elevation phase within the useful range of the standardised-residual cycle-slip test, where a steeper law would deweight it until the test has no effect.

The argument against it is comparability: the code branch and every archived benchmark run at one. The loader therefore ships one (`GnssConfig::getPhaseElevationPower`, whose documentation gives the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [34]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\text{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{182}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). Adding *variances*, as in Equation (182), is the correct way to combine two independent noise sources. The legacy alternative adds standard deviations, $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$. `computeSnr` rejects it, because squaring it produces a cross-term $2ab\,10^{-\mathrm{C/N_0}/10}$ that corresponds to no noise process: it treats the two independent contributions as correlated.

When no `S...` observable is present the model keeps the same functional form. It *synthesises* a density from elevation, so the noise curve stays continuous (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\left(1 - \max(0, \sin\theta)\right) \ \ [\text{dB Hz}], \tag{183}$$

which is a maximum of 50 dB Hz at zenith, falling by 15 dB at the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow.

On code rows, a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR}. Wider-bandwidth signals such as L5/E5a have $\sim\sqrt{10}$ less correlator noise in σ, a factor of $\sim 1/10$ in variance. Phase has no band scale, because PLL noise depends on the carrier-loop bandwidth, not the chip rate.

The Wieser hybrid step then passes the SNR variance through Equation (181). Pure SNR weighting under-weights multipath that is correlated with elevation, because the carrier loop can track cleanly while the range is corrupted. The elevation factor accounts for it.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \left(\sigma_{\text{block}(i),\,\text{type}}\, s_{\text{band}}\, s_{\text{attr}}\right)^2 \left(\sin\theta\right)^{-2p_{\text{type}}}.} \tag{184}$$

Here $\sigma_{\text{block}(i),\text{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m` when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because a RINEX-3 attribute means different things in different constellations: a `C` attribute is the noisy GPS C/A code but the high-precision Galileo E1C pilot. The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase have separate band and attribute tables (`SatBlockNoiseParams::ObservableScale`). Chip-rate calibration governs code, while carrier-tracking calibration governs phase and varies much less between bands. As with the other geometry strategies, the block variance is finally amplified by Equation (181). Phase reaches this path only when `apply_to_phase` is set. Otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies fill diagonal entries only, so the undifferenced $\boldsymbol{R}$ is block-diagonal, as in Equation (9). Off-diagonal correlation appears only after differencing (Section 13.5).

13.2 A-posteriori variance factor

The Kalman-side QC layer of this section and the next three is a diagnostic that is *off by default*. `KalmanFilter::update` calls `computeResidualQc_` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set. The correlation-aware kernel of Section 13.5 also requires `estimation.enable_full_r_kernel`. All three default to false. With them off, the only outlier gate is the fixed $|s| > 6$ path of Section 12.4, the variance factor stays at its initial value of one, and the redundancy numbers, w-tests and minimal detectable biases below are not computed.

After an epoch's update, the post-fit residuals $\hat{\boldsymbol{v}}$ show whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^2}{R_{ii}} \,\Big|_{\boldsymbol{R}\ \text{diag}}, \qquad \text{dof} = \operatorname{tr} \boldsymbol{\Theta},} \tag{185}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (190) below). This is the *effective* redundancy of a recursively filtered epoch, not the $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanResidualQc.cpp`). $\boldsymbol{R}$ is in absolute units, so the a-priori variance of unit weight is $\sigma_0^2 = 1$. A well-scaled model gives $\hat{\sigma}_0^2 \approx 1$. $\hat{\sigma}_0^2 \gg 1$ means the assumed noise is too small (optimistic), and $\hat{\sigma}_0^2 \ll 1$ means it is too large (pessimistic).

The variance factor is used for *reporting only*. It never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$ and leaves the filter state unchanged (`reportStatisticalQc_` in `src/core/filter/KalmanPostFitQc.cpp`).

The same quantity has a batch form in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::buildNeqAccumulator_`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \text{dof} = n_{\text{obs}} - (n_{\text{params}} + n_{\text{local}}), \tag{186}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{187}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution. The redundancy subtracts the Schur-reduced local parameters n_{local} as well as the retained parameters n_{params}. The formal σ of each retained parameter is the square root of the corresponding diagonal of the inverse normal matrix, scaled by $\hat{\sigma}_0^2$. This is the batch counterpart of the recursive dof in Equation (185).

The network estimator uses neither of these as its *operational* variance factor. The undifferenced network estimator does not form Ω/dof. It estimates a robust unit-weight σ as the scaled median of the standardised absolute residuals of its carried-ambiguity pool, $\tilde{v}_i = |\hat{v}_i|\sqrt{w_i}$,

$$\hat{\sigma}_{0,\text{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \text{median}_i\,\tilde{v}_i\big), \tag{188}$$

where the factor 1.4826 converts the median of the absolute values to a Gaussian σ and `network.raim_sigma0_floor` (default 1.0) is a lower bound. The pool holds, for each carried-ambiguity column of a station, the rows whose coefficient on the column exceeds 0.5 and whose kind the row classifier admits: the observation phase rows, the coupling and block carried-prior rows and the SD-EWL constraint rows (`raimPoolAdmitsKind`, `network/assembly/RowClassify.hpp`). `network.raim_pool_data_rows_only` (default off) restricts it to the observation phase rows, and `network.raim_redundancy_normalize` (default off) divides each row by $\sqrt{r_i}$ and drops the rows below the redundancy floor. Unlike the Kalman variance factor of Equation (185), $\hat{\sigma}_{0,\text{net}}$ is *not* used for reporting only. It controls three things:

- a within-epoch RAIM adaptation loop, whenever it exceeds `network.raim_sigma0_trigger` (default 3.0);
- a whole-epoch and a per-constellation product withhold, above `network.raim_withhold_sigma0`;
- the scale of the emitted URA.

Three separate seams implement them. The within-epoch loop and the ceiling are stages 13 and 14 of the epoch's solve (`networkWithinEpochAdapt` and `networkSigma0Ceiling`, `network/qc/RaimWithinEpoch.cpp`; the per-constellation decision is `NetworkEstimator::decidePerConstWithhold`). `networkPostFitRaim` (`network/qc/Raim.cpp`) owns the post-fit pass and the exponentially weighted fold of $\hat{\sigma}_{0,\text{net}}$ into the emitted URA scale. `networkSolPrep` (`network/emit/SolPrep.cpp`) owns the emission-side prune that removes a withheld constellation's satellites. Sections 13.9 and 13.10 develop the adaptation loop and the pooled statistics. The emitted product is not treated here.

13.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{189}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance. `KalmanFilter::update` builds the diagonal $\boldsymbol{Q}_{ii}$ per row as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$. The full matrix is formed only under the off-diagonal kernel of Section 13.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{190}$$

clamped to $[0, 1]$ for numerical safety in both branches of `KalmanFilter::residualQcKernel` (called by `computeResidualQc_`). When $r_i \to 1$ the geometry checks the observation fully, and a blunder appears almost entirely in the residual. When $r_i \to 0$ the observation is *uncheckable*: the estimate absorbs a blunder and the residual shows nothing. The sum dof $= \sum_i r_i = \operatorname{tr} \boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (185).

The network estimator computes Equation (190) in the batch (arrowhead) algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i \, \boldsymbol{a}_i^{\mathrm{T}} \operatorname{Cov}(\boldsymbol{x}) \, \boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\, \boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i} \,]$ and the same $[0, 1]$ clamp. This is algebraically the batch form of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows at or below the redundancy floor 10^{-6} (`kBaardaRedundancyFloor`) are uncheckable and are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (188). This corresponds to the `kUncheckableRedundancy` cutoff of the KF snooping gate below. The network constant is much smaller, because it protects only the $\sqrt{r_i}$ standardisation and not a reset decision.

One case of $r_i = 0$ is exact, and it determines the design of the network's phase-side QC. Suppose each (station, satellite, signal) arc has its own free ambiguity, which is the case whenever `network.phase_leveling` is off. Then every phase row is fitted *exactly* by its own parameter. The post-fit phase residual is identically zero, r_i is exactly zero, and the residuals cannot reveal a phase fault of any size. An undifferenced network with per-arc float ambiguities has *no* phase-side redundancy.

The phase rows gain redundancy, and so the power to detect faults, only when a carried prior constrains the ambiguities. That is why the network's phase RAIM (Section 13.9) requires phase-leveling: without it the test has no power at all. The measure-only instrument `networkPostfitSigma0` (`network/diag/FrameDiag.cpp`) reports the effect. Its pooled phase variance factor is identically zero in that configuration, and the zero is the correct result. Its pools exclude two classes of row for related reasons. Rows that touch a shared spherical-harmonic column (the optional ionosphere field, Section 13.10) are pseudo-observations, not data. The carry-coupling rows describe the carry, not the epoch's measurements.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{191}$$

(the diagonal branch of `KalmanFilter::residualQcKernel`). The divisor is $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not $\sqrt{R_{ii}}$. With $\sigma_0^2 = 1$ known (every $\boldsymbol{R}$ here is absolute), this is the standardised residual of [31], chapter 9; it becomes the studentised residual when an estimated $\hat{\sigma}_0$ replaces σ_0. Under the null hypothesis $w_i \sim \mathcal{N}(0, 1)$. The s used by the fixed gate floors the denominator at R and is therefore conservative. When the observations are correlated, the weight matrix $\boldsymbol{R}^{-1}$ takes the place of $1/R_{ii}$ (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1}\right)_{ii}}}.} \tag{192}$$

This is Baarda's statistic for a single blunder in observation i [1, 38], the most powerful test of that alternative, and the form of the LASER guide's data snooping [32]. In a Kalman update $\boldsymbol{R}^{-1}\hat{\boldsymbol{v}} = \boldsymbol{S}^{-1}\boldsymbol{z}$ and $\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1} = \boldsymbol{S}^{-1}$, with $\boldsymbol{z}$ the innovation and $\boldsymbol{S}$ its covariance, so Equation (192) is also the innovation w-test $(\boldsymbol{S}^{-1}\boldsymbol{z})_i/\sqrt{(\boldsymbol{S}^{-1})_{ii}}$ [36, 39]. It reduces to Equation (191) when $\boldsymbol{R}$ is diagonal, although $\boldsymbol{Q}$ is not diagonal in a Kalman update. Until 2026-10-07 the branch used $\boldsymbol{Q}^{-1}$

in place of $\boldsymbol{R}^{-1}$. That statistic is also standard normal under the null hypothesis, but it tests the direction $\boldsymbol{S}\boldsymbol{R}^{-1}\boldsymbol{e}_i$ rather than observation i: it lost power against a single blunder, and it did not reduce to Equation (191) even for a diagonal $\boldsymbol{R}$. `tests/test_residual_qc_wtest.cpp` checks the routine against a batch Gauss–Markov model with the prior as stochastic constraints.

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{193}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (the diagonal branch of `KalmanFilter::residualQcKernel`). With correlated observations the same definition gives $\mathrm{MDB}_i = \delta_0/\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}$, so that a blunder of size MDB_i in observation i shifts the expectation of Equation (192) by exactly δ_0; it equals Equation (193) when $\boldsymbol{R}$ is diagonal. The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$, for a test power of 80% ($\beta = 0.20$). For the default critical value $k = 3.29$ this gives $\delta_0 \approx 4.13$. The factor $1/\sqrt{r_i}$ expresses the geometry. A poorly checked observation ($r_i \to 0$) has an arbitrarily large MDB, and the code returns $+\infty$ when r_i is at or below the floor 10^{-6} (`kBaardaRedundancyFloor`). A blunder on such a row cannot be detected from the residual. The network estimator computes the same quantity from its own redundancy numbers (`network/qc/Mdb`, `network.mdb`), with $\sigma_i = 1/\sqrt{w_i}$. Both paths take δ_0, the redundancy floor and the $+\infty$ convention from one header, `filter/BaardaReliability.hpp`, so a PPP MDB and a network MDB are commensurable. The network MDB is measure-only: no gate, deweight or withhold reads it. Its loader default is off, and both reference configurations enable it. The network's acting phase-side QC remains the robust $\hat{\sigma}_{0,\mathrm{net}}$ RAIM loop of Equation (188).

13.4 Data-snooping gate override

When `enableDataSnooping()` is set, the QC statistics drive an *alternative* outlier gate, which can select a row that the fixed $|s| > 6$ path missed. Both gates use the same downstream dispatch in `KalmanFilter::validatePostFitResiduals`. Only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$, so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is seeded from the w-test over *checkable* rows only. Rows with redundancy below the `kUncheckableRedundancy` cutoff are skipped. Their residuals are unreliable, and resetting the corresponding ambiguity would discard good information. The same cutoff filters the summary log, so the log and the gate agree. Over the remaining rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{194}$$

and the checkable row with the largest $|w_i|$ becomes the handled index. w_i uses the true residual σ (Equation (191)), not the R-floored s. The gate can therefore catch a blunder on a high-leverage row whose floored s stayed below 6, and in that case it overrides the s choice. The downstream Melbourne–Wübbena tiebreaker still works in the σ domain. When the w-test selects a row that the σ cut missed, no σ candidate exists, and the w-selected index is handled directly. The fixed standardised-residual path is described in Section 12.4.

13.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume that $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which is true for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of Section 5 turns the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\mathrm{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\mathrm{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$. It contains the cross-covariance between observations that share a pivot or reference. The off-diagonal kernel (`estimation.enable_full_r_kernel`, default off) also requires that a dense $\boldsymbol{R}$ was supplied for the epoch, which `KalmanFilter::update` checks with `dense_r_supplied`. Under it the full residual covariance Equation (189) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \mathrm{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{195}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}, \qquad \mathrm{MDB}_i = \frac{\delta_0}{\sqrt{(\boldsymbol{R}^{-1}\boldsymbol{Q}\,\boldsymbol{R}^{-1})_{ii}}}. \tag{196}$$

$\boldsymbol{R}^{-1}$ comes from an $\mathrm{LDL}^{\mathrm{T}}$ solve against the identity. An $\boldsymbol{R}$ that is not positive definite has no weight matrix, and the routine then falls back to the per-row forms. When $\boldsymbol{R}$ is diagonal, $\boldsymbol{\Theta}$, w and MDB equal the per-row quantities of Section 13.3, whatever the off-diagonals of $\boldsymbol{Q}$. The diagonal of the full $\boldsymbol{Q}$ equals the per-row $\boldsymbol{Q}_{ii}$, so the fixed $|s| > 6$ gate is *unchanged behind the flag*; under data snooping (Section 13.4) the gate reads the correlation-aware w_i. The flag only makes the statistics correct on differenced epochs, where the residuals are correlated. Its scope is narrow. The gain and the Joseph-form update already use a dense $\boldsymbol{R}$ whenever one is supplied, with or without the flag. The flag decides only whether the *post-fit residual* covariance is formed densely. Section 5 describes the corresponding correlation on the model side.

13.6 Block Baarda test on a line of sight

One physical fault on one line of sight (multipath, an unflagged slip) enters the code and phase rows of every band at once, and the adjustment spreads it across all of them. The per-row w-test sits below that unit and the station-level median polish above it, so neither names the line of sight. The block test takes the b rows of one (station, satellite) block B and forms Baarda's multi-dimensional statistic for the alternative "an arbitrary bias vector on the block" [32, 38]. With $\tilde{\boldsymbol{e}}_B$ the post-fit residuals of the block and $\boldsymbol{Q}_B$ their $b \times b$ cofactor, off-diagonals kept (`NetworkNormalEquations::blockResidualCofactor`),

$$T_B \;=\; \frac{\tilde{\boldsymbol{e}}_B^{\mathrm{T}}\,\boldsymbol{Q}_B^{-1}\,\tilde{\boldsymbol{e}}_B}{b}, \qquad t_B \;=\; \frac{T_B}{\hat{\sigma}_{0,f}^2}. \tag{197}$$

With a diagonal weight matrix the weight factors of the general form cancel, so for $b = 1$ it is the square of the scalar w-test, and under the null $b\,T_B \sim \sigma_0^2\chi_b^2$, so $E\{t_B\} = 1$ for every block size. $\hat{\sigma}_{0,f}^2$ is a calibration per constellation family: the tail-robust geometric-mean estimator of Section 13.7, fed the χ_1^2-equivalent $u_B = \sqrt{T_B\,b\,e^{\psi(1/2)}/e^{\psi(b/2)}}$ of each block, so that a mix of block sizes does not bias it. The calibration is frozen through the previous epoch before the epoch's blocks feed it, every checkable block feeds it before a threshold is consulted, and it is not clamped. The block survey (`network/qc/BlockBaarda.hpp`, `network.block_baarda`) is measure-only: no weight, state or decision reads it.

Estimable form. On the float vehicle a line of sight's slant ionosphere is a free column that no row outside the block touches, so it absorbs one direction of the block exactly. The cofactor $\boldsymbol{Q}_B$ is then singular however well the data are conditioned, and the whole-block test refuses the block.

The estimable form (`network.block_baarda_estimable`) tests what the data can test. Whiten the cofactor with the row weights, $\tilde{\boldsymbol{Q}} = \boldsymbol{W}^{1/2}\boldsymbol{Q}_B\boldsymbol{W}^{1/2} = \boldsymbol{I} - \boldsymbol{H}_{BB}$, whose eigenpairs $(\lambda_k, \boldsymbol{v}_k)$ have eigenvalues that are redundancy numbers in $[0, 1]$; keep the directions with $\lambda_k \geq \varepsilon$ (`network.block_baarda_estimable_eps`, default 0.05, the row survey's floor) and read the quadratic form on them:

$$\tilde{\boldsymbol{e}}_B^{\mathrm{T}}\boldsymbol{Q}_B^{+}\tilde{\boldsymbol{e}}_B \;=\; \sum_{\lambda_k \geq \varepsilon} \frac{\left(\boldsymbol{v}_k^{\mathrm{T}}\boldsymbol{W}^{1/2}\tilde{\boldsymbol{e}}_B\right)^2}{\lambda_k}, \qquad T_B \;=\; \frac{\tilde{\boldsymbol{e}}_B^{\mathrm{T}}\boldsymbol{Q}_B^{+}\tilde{\boldsymbol{e}}_B}{m}, \quad m = \#\{\lambda_k \geq \varepsilon\}. \tag{198}$$

Under the null this is $\sigma_0^2\chi^2_m$ with the kept count m as its degrees of freedom, which a plain pseudo-inverse lacks, and the scale correction above is taken at m. When nothing is dropped the statistic is the whole-block one. A fault that lies wholly in a dropped direction (an additive slant-ionosphere change is the example) is absorbed by the column that makes the direction null, and no test of the block can see it; this one does not claim to. Both settings are armed in the two reference configurations. The positive control, a 1.5 m code multipath injected on one line of sight, is flagged by the estimable form at 24 times the control's rate and by no other armed check (`docs/refound/BBE_block_baarda_estimable_2026-10-18.md`, section 9).

13.7 Variance components of the carried priors

The network links epochs with carried priors whose variances are set by hand, a floor and a rate per carry family: the phase-leveling ambiguity carry (`pl`), the shared satellite phase-bias carry (`pb`) and the shared satellite code-bias carry (`cb`). A monitor (`NetworkCarryVce`, `network/carry/CarryVce.hpp`; `network.carry_vce`, loader default off, armed on the three families in both reference configurations) checks them against the carry residuals. Each epoch a carried quantity is folded as a pseudo-observation of the state with prior mean $\bar{b}_i$ and prior variance $p_i = \max(\mathrm{var}_{i,\mathrm{prev}} + q\,\Delta t,\ \mathrm{floor})$. After the solve the state has value $\hat{x}_i$ and posterior variance v_i. The carry-row innovation

$$\hat{e}_i \;=\; \bar{b}_i - \hat{x}_i, \qquad r_i \;=\; 1 - w_i v_i \in [0, 1], \quad w_i = 1/p_i, \tag{199}$$

does not depend on how the fold is represented (a diagonal prior or the coupling rows), and r_i is the partial redundancy of the row. The monitor models the carry-row variance as a floor component (cofactor 1, m^2) plus a rate component (cofactor Δt_i, the arc gap, in m^2/s) per (family, constellation), and estimates both by residual-form least-squares variance-component estimation at the weight the solve used [40]:

$$N_{kl} \mathrel{+}= w_i^2\, c_i^{(k)} c_i^{(l)}\, r_i, \qquad l_k \mathrel{+}= w_i^2\, c_i^{(k)}\, \hat{e}_i^2, \qquad \hat{\boldsymbol{\sigma}} = \boldsymbol{N}^{-1}\boldsymbol{l}. \tag{200}$$

The redundancy is taken block-diagonal in the carry rows. The two cofactors are collinear when every arc gap equals the decimation interval, so a weak a-priori regularises $\boldsymbol{N}$ and its condition number is reported. Its primary output is a tail-robust geometric mean of $\hat{e}_i^2$ over a forgetting window (`network.carry_vce_window_epochs`, default 60), corrected by the geometric mean $e^{-\gamma}/2 \approx 0.2807$ of a χ^2_1 variate, and a second channel formed from $\hat{e}_i^2/r_i$. Contaminated carries (a slip or refloat, a datum re-reference, a handover, a RAIM exclusion) are withheld from the accumulators. As a monitor it never feeds back into the solve.

The feedback is `network.carry_vce_apply` (`network/carry/CarryVceApply.hpp`), built and default off, not armed in either reference configuration. It multiplies the fold variance p_i of each carry family and constellation by a factor $\alpha^2 \geq 1$, updated by the variance-factor iteration of the LASER guide (slides 5-93 to 5-95) [32], $\alpha^2_{j+1} = \hat{\sigma}_j^2\,\alpha_j^2$. The group estimate $\hat{\sigma}^2$ divides the weighted residual square sum by the group's redundancy [31]. A carry row folded at $\alpha^2 p_i$ with posterior

variance v_i has the partial redundancy $r_i = 1 - v_i/(\alpha^2 p_i)$ and $E\{\hat{e}_i^2\} = r_i\,\alpha^2 p_i\,\hat{\sigma}^2$. The streaming target is therefore formed row by row, as a geometric mean consistent with the monitor's:

$$\alpha^2 \;\leftarrow\; \frac{\exp\big(\overline{\log(\hat{e}_i^2/r_i)}\big)/c_{\chi_1^2}}{\exp\big(\overline{\log p_i}\big)}, \qquad c_{\chi_1^2} = e^{-\gamma}/2 \approx 0.2807, \tag{201}$$

where the overbar is the monitor's forgetting-weighted mean, p_i is the hand fold variance before the factor, and $c_{\chi_1^2}$ is the geometric mean of a χ_1^2 variate. A row with r_i at or below 10^{-3}, or with $r_i < 0$, carries no information on the factor and is left out and counted. The case $r_i < 0$ arises on the coupled ambiguity carry, whose harvested variance is a conditional variance that can exceed the prior; there r_i is an approximation. The step toward the target is clamp(target/α^2, $[1/s,\, s]$) with the slew limit s (`network.carry_vce_apply_slew`, default 1.05 per epoch), and α^2 is clamped to [1, `network.carry_vce_apply_max`]. The factor is inflate-only, slew-limited, frozen on a contaminated epoch or below a warm-up row count, and valid only from a fresh convergence. Until 2026-10-07 the target omitted the division by r_i. For a scalar carry with true prior variance $a\,p$ and data variance d, $E\{\hat{e}^2\} = r^2(ap + d)$, so that target settled below a, and once d exceeded about $ap/4$ it stayed at $\alpha^2 = 1$: the under-inflation was largest where the carry is stiff against its data.

13.8 Measure-only instruments of the network

Four further instruments read the network's data and priors without acting on them. Each is off in the loader and armed in both reference configurations; none changes a weight, a state or a decision.

- *Prefit survey* (`network/qc/PrefitCensus.hpp`). A raw prefit innovation of 10^5 m is not a fault: a millisecond receiver clock step puts it on every row of a station. The statistic is centred per station and row type, $s_i = |y_i - m_t|\sqrt{w_i}$ with m_t the median of the innovations y_i of type t, a between-satellite single difference free of the receiver clock. It flags magnitude on established cells; a cell born in the epoch is passed to the seed-birth tripwire.

- *Redundancy survey* (`network/qc/RedundancyCensus.hpp`). The row redundancy $r_i = 1 - w_i \boldsymbol{a}_i^{\mathrm{T}} \mathrm{Cov}(\boldsymbol{\theta})\boldsymbol{a}_i$ says how much of a row is cross-checked by the rest of the network. A row with $r_i \to 0$ only feeds a state and its misfit is invisible to every residual test, whatever its size. The alarm is the cross-flag of an uncheckable row ($r_i < \varepsilon$) with a large centred innovation ($s_i > S$). The sweep runs every `network.redundancy_census_stride` epochs (10 in the reference configurations) and hosts the network MDB, the block test and the congruence screen.

- *Prior-residual survey* (`network/qc/PriorResidual.hpp`). The residual of a stochastic constraint in the LASER guide's form (slide 5-74), for every carried stiff prior. For a prior folded into the solve with variance p and posterior variance v, $D\{\hat{e}\} = r\,p = p - v$; for a prior the epoch did not fold, $D\{\hat{e}\} = p + v$. The standardised value is $s = \hat{e}/\sqrt{D\{\hat{e}\}}$ and the empirical one $t = s/\hat{\sigma}_{0,f}$. It is the carry-row counterpart of the per-row w-test of Section 13.3, on the quantities of Section 13.7.

- *Congruence screen* (`network/qc/Congruence.hpp`). Station coordinates are held fixed, so an error in a held coordinate projects coherently into every row of the station and the other tiers are blind to it. The screen estimates the coordinate offset that best explains the station's own post-fit code and phase residuals and tests it against zero,

$$M = \boldsymbol{H}_B^{\mathrm{T}} \boldsymbol{P}_B\, \boldsymbol{Q}_B\, \boldsymbol{P}_B \boldsymbol{H}_B, \qquad \hat{\boldsymbol{\delta}} = M^{+} \boldsymbol{H}_B^{\mathrm{T}} \boldsymbol{P}_B \tilde{\boldsymbol{e}}_B, \qquad T = \frac{\hat{\boldsymbol{\delta}}^{\mathrm{T}} M \hat{\boldsymbol{\delta}}}{3\,\hat{\sigma}_0^2}, \tag{202}$$

with $\boldsymbol{H}_B$ the $b \times 3$ position Jacobian in east, north and up and a coordinate MDB $\delta_0 \hat{\sigma}_0 \sqrt{(M^+)_{kk}}$ per direction. It detects a held-coordinate error and cannot size one.

13.9 Network RAIM: within-epoch adaptation, withhold and post-fit tiers

The network checks its own epoch at three points, in this order: the within-epoch adaptation inside the solve, the $\hat{\sigma}_{0,\text{net}}$ ceiling that can withhold the epoch, and the post-fit tiers after the carry harvest. The doctrine is to adapt and emit within the detected epoch; a withhold is the backstop, and it counts as an adaptation failure unless the epoch is provably unrecoverable.

Within-epoch adaptation. When $\hat{\sigma}_{0,\text{net}}$ exceeds its trigger, the network chooses the *granularity* of its response from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \;\approx\; \mu \;+\; \alpha_r \;+\; \beta_i \;+\; \gamma_{ri}, \tag{203}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `network/qc/RobustStats.hpp`). The polish sweeps medians, not means, so a minority of very large cells (the outliers) does not distort the additive effects. A bad satellite, which raises a whole column, can then be told apart from a bad station, which raises a whole row. Isolated arc slips remain in the interaction term.

Each effect is a candidate culprit. To be eligible it must satisfy effect$/\hat{\sigma}_{0,\text{net}} > k_{\text{RAIM}}$ for the configured gate (`network.raim_k_sigma`, default 6.0). The eligible candidates are ranked with the narrowest response first: on equal score an arc ranks above a satellite, and a satellite above a station (`weChooseCulprit`). The station branch is also reduced by a bias factor (`network.raim_station_drop_bias`, default 1.5, active under `network.raim_surgical_station`, default on). Excluding a station discards good arcs with the bad, so it is chosen only when it dominates by that margin. A consensus rule avoids removing arcs one at a time. When at least `network.raim_min_stations` (default 3) of one satellite's arcs exceed the gate in one iteration, the arc candidate is replaced by the whole satellite. A corrupted *shared* clock or phase bias is a network-wide fault, not a set of independent local ones.

The response distinguishes a bad *measurement* from a stale *parameter*. This distinction is the central design decision. A station culprit is excluded, because its data are bad and exclusion is the right response to bad data. A satellite or arc culprit is almost always a cycle slip. A slipped ambiguity does not need less weight; it needs a new *value*. The epoch is therefore solved again on a rebuilt normal-equation system with the culprit's ambiguities *refloated*. Their measurements keep full weight, so the local block stays positive definite and no singularity is introduced. Their carried prior is withheld, so the new solve can find the new value and absorb the slip locally. The loop repeats until $\hat{\sigma}_{0,\text{net}}$ falls below the trigger or five passes have run. A pass whose re-solve fails is undone to the last landed state; a station the undo cannot re-add is left out of the epoch, and an undo that cannot complete skips the epoch. Only the rare contaminated epochs pay for a second solve.

`network.raim` and `network.raim_within_epoch` both default to false and are armed in the two reference configurations. *Both* tiers also require the phase-leveling carry to be warm: the within-epoch loop inside the solve, and the post-fit pass at its call site. With `network.phase_leveling` off there are no carried ambiguities to refloat. By the redundancy argument of Section 13.3 there is also no phase-side redundancy to test. The network's RAIM then does nothing, as a consequence of the model and not of a policy choice.

The ceiling and the withhold. The capped adaptation may not isolate the fault. With `network.raim_withhold_sigma0` positive (default 0, off; 6.0 in both reference configurations), an epoch whose final $\hat{\sigma}_{0,\mathrm{net}}$ still exceeds the ceiling is withheld: it is not emitted, its carry is not stashed and the epoch is skipped, so the contaminated solution never enters the product store or the carry. The ceiling is a backstop. On the 6 h reference runs it never fired (720 of 720 epochs emitted on each of the real-data and simulated cases; `docs/CONFIG_promotion_ledger.json`, row `network.raim_withhold_sigma0`). Two extensions of the withhold are built and unvalidated: a larger adaptation budget inside a run of consecutive withheld epochs (`network.raim_withhold_adapt_escalate`) and the erase of detector-freed carries at a withhold (`network.raim_withhold_free_erase`).

Post-fit tiers. After the joint solve and the carry harvest, each station is reconstructed once (`networkPostFitRaim`) and its carried-ambiguity residuals are standardised by $\hat{\sigma}_{0,\mathrm{net}}$ of the same pool. Two screens look for slips the detectors missed. The consensus screen refloats a satellite whose standardised residual exceeds k_{RAIM} at `network.raim_min_stations` or more stations network-wide. The attribution screen (`network.raim_attribution`, armed) applies the median polish of Equation (203) and drives the refloat in place of the consensus: a bad satellite (column effect over the bar) is refloated network-wide only when $\hat{\sigma}_{0,\mathrm{net}}$ exceeds the trigger, because on a clean epoch a high column effect is an artifact of a satellite seen by a regional cluster; an isolated arc (interaction over the bar) is refloated at any $\hat{\sigma}_{0,\mathrm{net}}$. A bad station is never refloated wholesale, since a cluster of stations raised by one shared satellite event inflates their row effects and the cascade diverged the solution; it is handled by the within-epoch station exclusion or surfaces as isolated arcs. On the float vehicle the refloat of an arc is a bounded drop of each band's leveling carry, value kept and variance returned to its birth strength.

Refloat maturity gate. An isolated-arc flag meets two gates in this order before it refloats the arc. The maturity gate (`network.raim_refloat_maturity_gate`, unvalidated, off in the reference configurations) skips the refloat until the run is mature: until it has emitted $\max(N_{\mathrm{mat}}, N_{\mathrm{warm}})$ epochs and the $\hat{\sigma}_{0,\mathrm{net}}$ average that scales the emitted URA has received N_{mat} updates, with N_{mat} = `network.raim_refloat_maturity_epochs` (default 0, which leaves the gate inert) and N_{warm} the carry warm-up (`network.phase_leveling_warmup_epochs`). Without `network.emit_ura` the average is never fed and the epoch count alone decides. The persistence gate (`network.raim_arc_persist_epochs` = K, default 1) refloats an arc only on its K-th consecutive flagged epoch. Whole-satellite flags are never gated. The maturity gate changes which information reaches the carried ambiguities, so it is an event at both edges. Each held flag is an arc-epoch on which the refloat did not run, and the *lift*, the first scan with the gate open, is where the test applies to every arc again. Both are counted and written as events of tier `RAIM_MATURITY` (one row per held flag with the arc's identity and one at the lift), with a run-end section and an arming line; the `acted` column of the attribution trace reads a refloat taken, not a flag raised. Graded on 6 h cases, the gate at 120 epochs halves the isolated-arc refloats (107 to 55) and moves no user by more than 0.1 mm; it stays off (`docs/refound/RCG_refloat_cutting_keys_2026-10-29.md`).

Signal-integrity classifier. The second setting that cuts refloats is `network.signal_integrity.census` (unvalidated, off). It classifies each line of sight each epoch (stable, new, re-entry, band regained, degraded), folds the phase rows of a line of sight that has not been vetted at a tenfold standard deviation until it has had `revet_ep` consecutive full-band epochs, and refloats the ambiguities of a line of sight whose reference band returns within `flicker_max_ep` epochs. It detects nothing: across a loss of lock the slip is unbounded and unobservable, so it withholds trust from

the lines of sight no detector can monitor. It cut the isolated-arc refloats from 107 to 78 but the slip-class responses by only 3.3 % (same document), so it stays off.

13.10 Pooled variance factors and the per-constellation carve-out

Equation (188) pools only the carried-ambiguity rows. Two extensions widen or partition that set of rows without changing the estimator. Both are off by default.

Code and iono-constraint pools. Originally no tier screened the code rows or the iono-constraint rows that tie each line of sight to a spherical-harmonic field. Under `network.raim_we_code_iono` the same scaled-median estimator is applied to each of them over its own disjoint set of rows,

$$\hat{\sigma}_{0,\mathrm{code}} = 1.4826\ \mathrm{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\mathrm{iono}} = 1.4826\ \mathrm{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{204}$$

each with its own trigger (`network.raim_we_code_trigger`, default 10; `network.raim_we_iono_trigger`, default 12). Every row belongs to exactly one of the three sets. Neither new pool has the floor of Equation (188), and the phase pool still drives the whole-epoch ceiling. The adaptation gains two responses suited to what these rows can diagnose: a station exclusion for the epoch, and a network-wide *deweighting* of one satellite's code rows. The second is a fallback and not a drop, in line with the project's preference for degrading a measurement over discarding it. The setting is off in the reference configurations: it never tripped on their posture, and its iono pool is empty there, because the tie rows exist only when the optional spherical-harmonic field is estimated. The reference configurations estimate no such field, and the station-relative deweighting of its tie rows was deleted from the code.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` (unvalidated, off) the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{205}$$

over the same rows $\mathcal{P}_c$ that the global statistic pools. The statistic and the ceiling are the same, and only the set of rows is restricted. A constellation is published if and only if $\hat{\sigma}_{0,c}$ is below that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count (`network.per_const_withhold_min_rows`, default 20). Too few rows cannot show that a constellation is clean inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

This is an approximation, in two respects. The residuals in Equation (205) are evaluated at the *joint* solution. A contamination in one constellation perturbs the shared per-station nuisance parameters (receiver clock, tropospheric delay, slant ionosphere) and leaks into the residuals of another. Each row's redundancy is also its share of the joint system's redundancy, not of a separate per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates coupling between constellations. The outcome "every constellation individually clean while the global statistic tripped" is a consequence of that approximation. It is published with a warning and counted. The rank and observability gate keeps whole-epoch semantics. A rank deficiency invalidates the declared datum, so the product would be a meaningless least-norm split and no healthy subset exists.

13.11 Emission-time gates

Everything above tests *residuals* against $\boldsymbol{R}$. The network also prunes the emitted product at emission time (`networkSolPrep`, `network/emit/SolPrep.cpp`). The per-constellation carve-out of

Section 13.10 is applied there, and so are the satellite-level exclusions of the gross line-of-sight screen (Section 24.6). A different kind of gate tests the *solved state*.

Clock-runaway quarantine. Under `network.clock_runaway_check` (loader default off; armed in the two reference configurations) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling (`network.clock_runaway_mag_m`, default 1000 m), or the median |drift| exceeds a rate ceiling (`network.clock_runaway_rate_m_s`, default 10 m/s, inert without `network.clock_model`). Like the carve-out of Section 13.10, this reduces the scope of the product and does not withhold all of it. The code documents one hazard and does not defend against it. The magnitude test is referenced to the global median over *all* clock satellites. If the runaway constellation supplies half or more of them, the comparison inverts: the healthy minority exceeds the ceiling and is quarantined, and the runaway passes. The degenerate single-constellation case is warned at configuration load. The check has not been recorded firing on the reference runs.

Clock quality control. `networkClockQc` (`network/qc/ClockQc.cpp`) is a different tier. It contains the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite sanity monitor for carried clocks. The health gate on the spherical-harmonic field that once sat beside the quarantine was deleted from the code. The field itself remains as a default-off option that the reference configurations do not use.

14 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management

The Kalman filter of Section 12 is a *local* linear estimator. Every update assumes the state is close to the linearisation point. Every dynamic state (ambiguity, per-satellite ionosphere, receiver code bias, satellite bias, or inter-system offset; Section 3) needs a value and a covariance before it can be updated. This document covers the machinery that provides them:

- the weighted-least-squares (WLS) epoch solve, which places the receiver close enough for the linearisation to hold (Section 14.1);
- the first-epoch seeds formed from residual differences, which give each new state a physically reasonable prior (Section 14.2);
- the between-satellite pivot selection and disturbance logic, which keep the single-difference (SD) reference stable across an arc (Section 14.3).

None of this is filtering. It is what makes the filter's local-linear assumptions valid at the first epoch and after every discontinuity. The first-epoch seeds are pure functions of the observation equation (Section 4.4), written in `measurement/Seeds.cpp`. The single-receiver model applies them to its `StateManager` (`measurement/MeasurementModelSeeds.cpp`) and the network estimator to a station's parameter set (`network/assembly/LocalBirth.cpp`).

The lossless covariance re-referencing (the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ S-transform that moves a group onto a new pivot) is *out of scope* here. It is derived in Section 5 and implemented once in `include/filter/CovarianceReReference.hpp`, which both the single-receiver and the network handover paths call. This section notes where the pivot logic calls it.

14.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed

Before the filter can run, the receiver position must be close enough to the truth that the geometry Jacobian Equation (6) is valid to below a decimetre. Broadcast pseudoranges are sufficient for this. The engine runs a GPS-only, code-only iterated Gauss–Newton solve over the core state

$$\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{r}^{\mathrm{T}} & c\,\mathrm{d}t_r & \mathrm{ZWD} \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^5, \tag{206}$$

with $\boldsymbol{r} \in \mathbb{R}^3$ the ECEF receiver position, $c\,\mathrm{d}t_r$ the receiver clock in metres, and ZWD the zenith wet delay (`src/core/math/WlsEstimator.cpp`). At least `kCoreSize` $= 5$ code observations are required before the solve is attempted; the iteration cap is `kMaxIter` $= 5$.

For each code measurement i the prefit residual removes every modelled term and the current state's contribution,

$$\widetilde{z}_i = P_i - \left[\, P_i^{\mathrm{mod}} + (\rho_i - \rho_i^0) + c\,\mathrm{d}t_r + m_w(i)\,\mathrm{ZWD} \,\right], \tag{207}$$

where P_i^{mod} is the modelled pseudorange, $\rho_i - \rho_i^0$ the change in geometric range relative to the range at the iteration's linearisation point, and $m_w(i)$ the wet mapping function (`WlsEstimator::predictObs`, evaluated once per row from the design-row loop of `WlsEstimator::estimate`). The geometric range is *re-evaluated every iteration* about the current position estimate, so each iteration is a Gauss–Newton relinearisation.

The design row for satellite i is the negated line-of-sight stacked with the clock and wet-mapping partials, assembled in the same loop:

$$\boldsymbol{A}_i = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 & m_w(i) \end{bmatrix} = \begin{bmatrix} -e_x & -e_y & -e_z & 1 & m_w(i) \end{bmatrix}, \tag{208}$$

with $\boldsymbol{e}_i$ the receiver-to-satellite unit vector (the same line-of-sight that drives Equation (7)). Stacking all rows into $\boldsymbol{A}$, the prefit residuals into $\widetilde{z}$, and the inverse measurement variances onto the diagonal weight $\boldsymbol{W}$ (clamped from below by `kMinVariance` $= 10^{-6}$ in that same loop), the normal-equations step is the boxed Gauss–Newton update

$$\boxed{\Delta\boldsymbol{\xi} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\widetilde{z},} \tag{209}$$

applied once per Gauss–Newton iteration of `WlsEstimator::estimate`. Iteration terminates on a position increment $\|\Delta\boldsymbol{r}\| <$ `kTol` $= 0.1$ m or at `kMaxIter`. The normal matrix is guarded against rank deficiency: if

$$\det\!\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right) \le 10^{-6} \tag{210}$$

the solve aborts and the epoch is not seeded (the determinant guard immediately before the step). This protects against degenerate single-constellation, nearly coplanar geometry, which would otherwise produce a very large position step.

On convergence the estimator returns both the state $\boldsymbol{\xi} = [\boldsymbol{r}^{\mathrm{T}},\ c\,\mathrm{d}t_r,\ \mathrm{ZWD}]^{\mathrm{T}}$ and its formal covariance, the inverse normal matrix

$$\boldsymbol{P}_{\boldsymbol{\xi}} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1} \tag{211}$$

(the result assembly at the end of `WlsEstimator::estimate`). The filter uses only the *clock* from this result. `Application::handleWlsSeeding` writes $c\,\mathrm{d}t_r$ into the receiver-clock state when one of three triggers fires: the first epoch, a data gap, or a clock *jump* of at least `kClockJumpThreshSec` $= 50\,\mu$s (about 15 km) between the WLS and filter clock estimates. In practice the jump is the most common trigger.

The position, the zenith wet delay, and the covariance (211) are computed but are not used as filter seeds. The position prior is the RINEX-header `approx_position` plus the antenna offset. The WLS position re-seeds the filter only on a kinematic (non-IMU) cold start after a data gap (`Application::handleGap`). The ZWD estimate goes only to a trace-CSV column, and nothing in the filter reads the formal covariance. The whole seeding path is skipped under BSSD, where the filter has no receiver-clock state (`handleWlsSeeding` returns early on `!hasClock()`).

The main use of the clock seed is *re-seeding after a gap*. The receiver clock is the most volatile state across a data gap, because an unsteered oscillator can drift freely while no measurements constrain it. After a gap the WLS clock estimate resets $c\,\mathrm{d}t_r$ before the phase filter resumes. Otherwise the filter would correct a stale clock prior through a series of large innovations.

14.2 First-epoch state seeding by residual differencing

The WLS solve of Section 14.1 seeds the receiver clock and, on a kinematic cold start, the position. It does not touch the *differential* states: inter-system bias, phase clock, uncombined receiver code bias, per-satellite ionosphere, and carrier ambiguity. Each of these is created empty and must be seeded from the measurement residuals. The common idea (`src/core/measurement/Seeds.cpp`) is that each state is what remains in a particular *difference* of residuals after all modelled terms are removed. The seed is that difference, taken as a median over satellites for robustness. The shared prefit residual is

$$\widetilde{z}^{\,i}_{r,s} = \Phi/P^{i}_{r,s} - \big(\text{modelled} + c\,\mathrm{d}t_r + m_w\,\mathrm{ZWD} + \mathrm{ISB} + g_{\mathrm{trop}} + \alpha_s \widehat{I}^{\,i} + \beta_s b_r + b^{i}\big), \tag{212}$$

the observation minus every term the model can already account for, evaluated at the current state (`rawResidual` in `measurement/Seeds.cpp`). The residual is the equation of Section 4.4 built with a mask of the terms known when the state is born, and its terms are subtracted in a fixed order: the modelled value, the receiver clock, the wet troposphere, the inter-system bias, the gradient projection $g_{\mathrm{trop}} = m_g(g_N \cos\mathrm{az} + g_E \sin\mathrm{az})$, the slant ionosphere (with the sign + on a code row and − on a phase row), the receiver code bias and the satellite code bias. The four *differential* seeds (a)–(d) below are differences of (212), the ambiguity seed (e) is the phase residual with its own mask, and a sixth pre-seed (f) anchors the receiver clock itself.

(a) Inter-system bias. Each non-GPS constellation g carries a receiver clock offset relative to the GPS time scale (Section 3). The GPS clock is the datum, and the WLS step has already put it into $c\,\mathrm{d}t_r$. The ISB of g is therefore the median code residual of the constellation minus the median GPS residual:

$$\boxed{\mathrm{ISB}_g = \mathrm{med}_{i\in g}\ \widetilde{z}^{\,i} - \mathrm{med}_{i\in \mathrm{GPS}}\ \widetilde{z}^{\,i}} \tag{213}$$

(`seedIsb`, applied by `MeasurementModel::initializeInterSystemBiases` and by the station birth). Medians are used throughout, so one gross code outlier cannot spoil the seed. A guard requires at least one GPS code observation, the time reference, at the seeding epoch. If GPS is absent the seed reports itself unsettled and *is tried again at the next epoch*. It does not mark the ISB states as initialised against an arbitrary datum. Without the guard, whichever constellation is acquired first would become the implicit zero, and a GPS satellite rising later would then carry the whole offset.

(b) Phase clock. In the uncombined model the phase observations see a receiver clock that differs from the code clock by the receiver phase–code hardware delay (Section 3). Its seed is the median

over satellites carrying *both* a code and a phase observation of the code-minus-phase residual:

$$\delta_\Phi = \mathrm{med}_i \left(\widetilde{z}_P^i - \widetilde{z}_\Phi^i \right) \tag{214}$$

(`seedPhaseClock`, applied by `MeasurementModel::initializePhaseClockOffset`). The covariance of the seed is set from the code noise. It is tighter than the default initial standard deviation, and it does not depend on the number n of differences in the median (the covariance seeding in `initializePhaseClockOffset`):

$$\sigma_{\delta_\Phi} = \max\!\Big(\tfrac{\sigma_P}{\sqrt{n}},\ \sigma_P \Big), \qquad \boldsymbol{P}(\delta_\Phi, \delta_\Phi) = \sigma_{\delta_\Phi}^2, \tag{215}$$

The corresponding row and column of $\boldsymbol{P}$ are set to zero first, so the seeded state starts uncorrelated. Since $\sigma_P/\sqrt{n} \le \sigma_P$ for every $n \ge 1$, the max always selects σ_P, and the seeded variance is σ_P^2 whatever the value of n. The seed is floored at the noise of a single code measurement and does not improve as $1/\sqrt{n}$. The network estimator has no phase clock and does not form this seed.

(c) Uncombined receiver code bias / IFB. In the uncombined PPP-RTK model each non-reference code band carries a receiver differential code bias (DCB) or inter-frequency bias (IFB) state relative to the constellation's BIA clock-reference band (Section 3). Forming the residual difference between band j and the reference band, removing the differential ionospheric contribution (the two bands scale the same slant ionosphere by different α), and dividing by the difference of receiver-bias Jacobian coefficients gives

$$\boxed{\widehat{b}_{r,j} = \frac{\Delta\widetilde{z}_j - \Delta(\alpha_j\, \widehat{I})}{\Delta\beta_j} = \frac{(\widetilde{z}_j - \widetilde{z}_{\mathrm{ref}}) - (\alpha_j - \alpha_{\mathrm{ref}})\, \widehat{I^i}}{\beta_j - \beta_{\mathrm{ref}}}} \tag{216}$$

(`seedCodeBias`), median-aggregated over satellites and subject to the same $|\widehat{b}| > 200\,\mathrm{m}$ reject as the ionosphere seed below; $\widehat{I^i}$ is the current value of the satellite's slant-ionosphere state. After a warm resume of the network estimator, a key that the installed linearisation points cover takes the installed value in place of the median (`StationParams::rx_cb_restore`). The Jacobian coefficient β_s depends on the bias parameterisation (`ReceiverBiasResolver::resolveReceiverBias`): for a pure IFB state it is the unit coefficient $\beta = 1$, whereas for a geometry-free / ionosphere-free DCB state against reference bands of frequencies f_1, f_2 it is

$$\mu_1 = 1, \quad \mu_2 = \frac{f_1^2}{f_2^2}, \quad \Delta\mu = \mu_1 - \mu_2, \qquad \beta_j = \begin{cases} \mu_1/\Delta\mu & j = \text{band 1} \\ \mu_2/\Delta\mu & j = \text{band 2,} \end{cases} \tag{217}$$

guarded by $|\Delta\mu| \ge 10^{-12}$ to avoid dividing by a near-degenerate frequency separation.

For GLONASS a third branch runs before both cases. Under `enable_glonass_ifb` each FDMA band (G1, G2) has one bias state *per FCN slot* k with $\beta = 1$ (`StateKeys::makeGlonassPerSlotIfbKey`). The per-slot carrier $f = f_{\mathrm{base}} + k\,\Delta f$ makes the receiver group delay depend on the slot, which one state per band cannot represent. The central channel $k = 0$ is the datum and has no state. The CDMA bands (G3/G4/G6) use the per-band treatment above. The branch runs before the clock-reference lookup because GLONASS usually has no clock-reference pair in the `.BIA` product, and the routine would otherwise return before reaching any IFB logic. With the flag off, a non-reference GLONASS band has no IFB state (the per-slot and non-reference-band branches of `resolveReceiverBias`).

(d) Per-satellite slant ionosphere. Each tracked satellite's slant ionospheric delay (Section 3) is recovered from the geometry-free combination of two code bands. With ionospheric scale factors $\alpha_s = f_{\text{ref}}^2/f_s^2$ referenced to GPS L1 (Equation (1)), the seed is

$$\boxed{\widehat{I}^i = \frac{(P_1 - P_1^{\text{mod}}) - (P_2 - P_2^{\text{mod}})}{\alpha_1 - \alpha_2}} \tag{218}$$

(`seedIonosphere`), where P_k^{mod} is the modelled pseudorange on band k. Two guards apply. The denominator must satisfy $|\alpha_1 - \alpha_2| \geq 10^{-6}$. The seed is rejected unless it is finite and $|\widehat{I}^i| < 200\,\text{m}$. A slant ionospheric delay above 200 m is not physical. It indicates mis-paired bands or a gross code outlier, and must not be stored in a per-satellite state.

Two BIA-coverage gates also apply, so that the differenced code is consistent in its biases. The first is *presence*: a `.BIA` entry exists on the band, with the zero-valued BeiDou C2I/C6I entries treated as missing. The second is the coverage *application* check (`bias_coverage_requires_apply`). It skips the seed when an entry is loaded but not *applied* to ($P -$ mod), because the raw hardware bias is then still present. The network estimator uses presence only. The application check is needed because the failure it prevents, a zero-valued BeiDou entry that injects 30–60 m, is well below the 200 m reject and would pass it.

(e) Carrier ambiguity. The float ambiguity that subsequently feeds integer resolution (Section 16) is the part of the *phase* residual that remains after every other modelled and estimated term is removed (`seedAmbiguity`, the routine that every phase-seeding call site uses),

$$\widehat{N}_{r,s}^i = \Phi_{r,s}^i - \text{modelled} - \big(c\,\text{d}t_r + \delta_\Phi\big) - m_w\,\text{ZWD} - \text{ISB} - g_{\text{trop}} + \alpha_s\,\widehat{I}^i, \tag{219}$$

This is the phase observation minus its model, the receiver clock *and* the phase clock δ_Φ (seed (b)), the wet delay, the inter-system bias, and the tropospheric gradient, plus the slant ionospheric phase advance $+\alpha_s\,\widehat{I}^i$ (seed (d)). The phase is advanced by the same α_s that delays the code, so the iono term has the opposite sign to (212). The phase clock and the slant iono are removed because they are estimated states in the uncombined model. One term cannot be removed this way: a receiver clock that is still zero on a network cold start. Pre-seed (f) handles that case.

The seeded ambiguity therefore contains the integer cycle count plus the remaining receiver and satellite phase biases. AR recovers the integer part once the satellite phase bias is supplied and the receiver phase bias cancels under between-satellite differencing.

(f) Cold receiver-clock pre-seed (network path). Seeds (a)–(e) assume the receiver clock has already been solved. That holds on the PPP path, where the WLS clock seed of Section 14.1 runs first. The network estimator does not run the WLS seed, so nothing solves the receiver clock before the ambiguity of (e) is seeded. `seedAmbiguity` then subtracts a clock term of zero, and the whole free-running receiver-clock offset goes into every cold ambiguity seed.

The opt-in `seedColdClock` (`network.cold_clock_seed`, loader default off, armed in both reference configurations) fixes this. It writes the median GPS code residual into the clock parameter, the median that ISB seed (a) forms for the time reference. The seed overwrites only a *cold* clock. Any $|x(\text{clock})| >$ `kColdClockEpsilonM` $= 10^{-6}$ m is treated as already solved and is left unchanged. When the epoch has no GPS code observation the seed reports no reference, so it tries again at the next epoch and does *not* mark the clock as seeded. This matches the GPS-reference contract of seed (a). A station's birth applies it once, after the slant ionospheres and before the ambiguities (`seedColdClockOnce`, `network/assembly/LocalBirth.cpp`); the single-receiver model applies it in the same place of its state pre-pass (`MeasurementModel::preseedColdReceiverClock`).

14.3 BSSD pivot management

Between-satellite single differencing (BSSD, Section 5.1) re-expresses each constellation's ambiguities relative to a designated *pivot* satellite p_g. This cancels the receiver phase bias and makes the SD ambiguities integer. The choice of pivot matters. A poor pivot degrades every SD row that references it, and an unstable one forces costly re-referencing handovers. `src/core/filter/PivotManager.cpp` selects and maintains the pivot. The baseline double-difference path uses the similar `src/core/filter/DdPivotManager.cpp`.

14.3.1 Lexicographic candidate ranking

Candidates are ordered *lexicographically*: first by a signal-coverage tier, then by a continuous score within the tier. Coverage comes first because the pivot must track every signal that any non-pivot satellite tracks. A non-pivot signal that the pivot lacks has no reference and is dropped. *Any* full-coverage candidate therefore ranks above *any* partial-coverage candidate, whatever their scores (the lexicographic comparator in `PivotManager::buildEpochMaps`). Candidate a beats b if and only if its phase-signal count is higher, or the counts are equal and its score is higher.

Within a coverage tier the tie-break score combines float-convergence and geometry (`PivotManager::scorePivotCandidate`):

$$\boxed{\text{score}(i) = -\min\!\big(1,\ \sqrt{v_{\max}(i)}\,\big) + \frac{\text{el}(i)}{90} - \tfrac{1}{2}\,\mathbb{1}\!\left[\text{el}(i) < 20^\circ\right],} \tag{220}$$

where $v_{\max}(i)$ is the worst (largest) float-ambiguity variance among the satellite's tracked phase signals and $\text{el}(i)$ is its elevation in degrees.

The first term penalises a pivot that has not converged. On handover the new pivot's variance is added to every SD state in its group. This is the diagonal of the Section 5, $\boldsymbol{P}'(i,i) = \boldsymbol{P}(i,i) + \boldsymbol{P}(p_g,p_g) - 2\boldsymbol{P}(i,p_g)$, implemented in `include/filter/CovarianceReReference.hpp`. A pivot whose own state is still settling therefore passes its uncertainty to the whole constellation. The penalty saturates at -1, equivalent to one metre of 1σ ambiguity uncertainty. A candidate with *no* state yet receives a flat -1 (the negative-`max_amb_variance` sentinel branch of the score). On the first BSSD epoch every candidate in a tier has this sentinel, so the score reduces to elevation and no special first-election branch is needed.

The el/90 term prefers high-elevation pivots. The $-\frac{1}{2}$ penalty below 20° discourages a low pivot, whose tropospheric and multipath residuals exceed the formal stochastic model however well its float state has converged.

Both terms only *rank* candidates that have already passed a hard elevation gate. A satellite below `pivot_min_el` (`getBssdPivotMinElevation`) cannot be elected at all: not in the freshest-peer scan, not in the online pick, and not in the sticky-retention test, all three in `PivotManager::selectPivotCandidates`. The $-\frac{1}{2}$ penalty therefore only breaks ties among the survivors. When *no* satellite passes the mask (a code-only float constellation), the gate is bypassed. The pick reverts to the precomputed best or highest-elevation satellite, so the candidate set is never empty (the fallback from `pickExclusionAwareBest` to the precomputed `best_sat` / `highest_sat`).

Sticky retention is the code's *elevation hysteresis*, and it prevents the pivot changing back and forth between epochs. It uses the same *absolute* gate and does not compare the incumbent with a challenger. The incumbent is kept whenever it still passes `pivot_min_el`, is not excluded, and stays within the toe spread of Section 14.3.2 (the sticky branch of `selectPivotCandidates`). No score is evaluated on that branch, so a better challenger cannot displace a qualifying incumbent. The score ranking runs only after the incumbent has *failed* the gate.

14.3.2 Ephemeris-freshness guardrail

A pivot with a stale broadcast ephemeris puts a step into *every* SD row that references it. Its extrapolated orbit and clock drift away from those of the fresher satellites it is differenced against. This was observed in production: a Galileo pivot held a one-hour-old I/NAV record while the other satellites' records were minutes old, and the whole SD solution was corrupted. The guardrail is gated on the *spread* of ages. A candidate is barred from pivot election when its absolute ephemeris age $|t_{\text{obs}} - t_{\text{oe}}|$ exceeds the constellation's freshest qualified peer by more than

$$\texttt{kPivotToeSpreadMaxS} = 1200\ \text{s} = 20\ \text{min} \tag{221}$$

(the `freshestToeAgeAbs` scan in `PivotManager::selectPivotCandidates`, applied both at the online pick and at the sticky-retention test). Because the gate is on the *spread* and not on an absolute age, the ranking is unchanged when all candidates are about equally fresh. Only a candidate that is much staler than the rest is displaced. The barred satellite can still be used as a non-pivot. The freshest satellite always passes, since its difference from itself is zero, so the candidate set is never empty.

14.3.3 Pivot disturbance detection and handover

In BSSD the pivot's own SD ambiguity is zero with zero variance, because the pivot *is* the reference: $x(p_g) = 0$ and $\boldsymbol{P}(p_g, p_g) = 0$ exactly. Any departure from that state means the filter has perturbed the reference, through process noise or an update, and the SD states are no longer consistent. `isPivotDisturbed` flags this when, for any of the pivot's phase signals,

$$\boxed{\left|x(p_g)\right| > 10^{-6} \quad \text{or} \quad \boldsymbol{P}(p_g, p_g) > 10^{-12}} \tag{222}$$

(`PivotManager::isPivotDisturbed`). The two thresholds are tolerances for numerical zero, not tuned gates. 10^{-6} m on the value and $10^{-12}\,\text{m}^2$ on the variance catch any real leakage onto the pivot and tolerate floating-point round-off in the re-reference arithmetic. The predict step must not create a false disturbance, so the pivot's process noise is suppressed during prediction and $\boldsymbol{P}(p_g, p_g)$ stays at zero (Section 12). Without the suppression, every epoch's $\boldsymbol{Q}$ would trip (222) and force an unnecessary handover.

A handover fires when the incumbent is disturbed or when the ranking elects a different satellite. The outgoing pivot is then classified. In a *clean* handover the old pivot's state is invertible and consistent. It can be re-referenced without loss, and the RTS smoother can later invert it. A *forced* or disturbed handover cannot be inverted cleanly and marks a discontinuity.

In both cases `CovReReference::applyBssd` in `include/filter/CovarianceReReference.hpp` re-references the covariance in place. It subtracts the new pivot's row and column from each non-pivot member of the group, then sets the new pivot's row and column to zero. This is the one shared implementation of the $\boldsymbol{T}\,\boldsymbol{P}\,\boldsymbol{T}^{\mathrm{T}}$ transform Section 5.

The baseline DD path has a different structure. It negates the new-pivot row and column and subtracts from the others, which is a different S-transform, and then renames the affected ambiguity states with `StateManager::renameAmbiguity` (`DdPivotManager::handleHandover`). The two paths share only the positive-definiteness guard applied after the transform (`enforcePsd`). The single-receiver user path runs the transform without the guard. At user scale the groups are small and handovers are rare, so floating-point error in $\boldsymbol{T}\,\boldsymbol{P}\,\boldsymbol{T}^{\mathrm{T}}$ does not accumulate.

Part IV
Cycle-slip detection and repair

15 Cycle-slip detection and repair

The uncombined filter (Section 4) forms no linear combination in its measurement update. The per-signal $P^i_{r,s}$ and $\Phi^i_{r,s}$ enter Equations (4) and (5) directly. Cycle slips are screened by a set of *diagnostic* combinations. In each one the geometric range ρ^i_r, the clocks $c(\mathrm{d}t_r - \mathrm{d}t^i)$ and the troposphere T^i_r cancel. What remains varies between epochs only by measurement noise and by any integer discontinuity of the carrier.

The status of the detectors is as follows.

- Melbourne–Wübbena (`src/core/slip/MwSlipDetector.cpp`, with its bodies split by concern over `MwSlipDetectorEpoch.cpp`, `MwSlipDetectorSample.cpp`, `MwSlipDetectorPairs.cpp`, `MwSlipDetectorStats.cpp`, `MwSlipDetectorQuery.cpp`, `MwSlipDetectorReset.cpp`, `MwSlipDetectorTrace.cpp` and `MwSlipDetectorZd.cpp`) is the primary detector and the only one armed by default.
- GRAPHIC (`src/core/slip/GraphicSlipDetector.cpp`) and the extra-wide-lane detector (`src/core/slip/EwlSlipDetector.cpp`), which needs a third frequency, are compiled and registered. Both are opt-in (Section 15.1), and the reference configurations (`config/golden/golden_field_refound.json` and `golden_sim_refound.json`) enable neither.
- The ionosphere-free phase detector (`src/core/slip/IfPhaseSlipDetector.cpp`) is *compiled and armed on the network path*. It is instantiated per station whenever `network.cycle_slip.if_phase_td` is set (Section 15.2.5), and both reference configurations set it. On the single-receiver user and kinematic path it stays staged (`src/core/slip/IfPhaseStagedUserPath.cpp`), because the position-uncertainty gate it needs is unsafe there.
- The geometry-free phase detector (`src/core/slip/GfSlipDetector.cpp`) is *staged*. It is in the tree but commented out of the explicit CMake source list, unlike the IF detector beside it. Its algebra is recorded here for completeness.

The user and kinematic set is composed at run time from the config block `estimation.cycle_slip`. The network composes its own per-station set from a parallel `network.cycle_slip` block (Section 15.1).

Throughout this section the receiver index r is suppressed (single-receiver PPP / PPP-RTK), and $f_1 > f_2$ denotes the two carriers of a band pair, sorted by frequency so that $f_1 - f_2 > 0$ and $\lambda_1 < \lambda_2$ (`src/core/slip/MwSlipDetectorPairs.cpp`, the band-frequency sort). Carrier phase is read from RINEX in cycles and scaled by $\lambda_s = c/f_s$ before combining, so Φ^i_s below is always in metres, consistent with Equation (5). The operator $\delta\cdot$ denotes a *temporal* integer slip step: for example δN_s is the change in the ambiguity on signal s across the slip epoch. It is distinct from the between-satellite single difference $\Delta\cdot$ of Section 5, which also uses Δ.

15.1 Detector registry and composition

The set of detectors is configurable. Each detector implements the interface `ISlipDetector` (`include/slip/ISlipDetector.hpp`), which has one entry point per epoch

$$\texttt{detect}(\text{SlipEpochContext}) \longrightarrow \{\,\text{SlipEvent}\,\},$$

plus lifecycle hooks (`resetSatellite`, `resetSatelliteSignal`, `enableTracing`, `setCorrections`). A `SlipEvent` is the union of the legacy per-detector result structs. Its `granularity` field distinguishes two kinds of flag. A *per-signal* flag (GRAPHIC, GF, IF) names one carrier. A *band-difference* flag (MW, EWL) reports a wide-lane integer $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$, which alone cannot assign the jump to one signal. Equation (243) resolves those two unknowns downstream. The inputs are bundled in `SlipEpochContext`, so every detector has the same signature. Only the IF detector uses the 3D position sigma and receiver ECEF it carries. On the network path, where that detector is armed, the position gate is always satisfied, because the estimator holds the station positions fixed.

`SlipDetectorRegistry::createEnabled` (`src/core/slip/SlipDetectorRegistry.cpp`) builds the set selected by `estimation.cycle_slip.{mw,graphic,ewl,gf,if}` in a fixed order. MW comes first. It is the primary detector and the only source of the wide-lane integers used by ambiguity resolution (Section 16) and by the BSSD pivot logic (`src/core/filter/PivotManager.cpp`). The auxiliary detectors follow. MW defaults on and the auxiliaries default off, which keeps the historical MW-only behaviour.

A kind that is enabled in config but has no creator in the registry is skipped with a warning, so a config written for a later version still runs. Two kinds are skipped, for different reasons. The source file of `gf` is excluded from the build (Section 15.2.4). The source file of `if` *is* compiled, but the network path instantiates it directly (Section 15.2.5) and not through this registry, so `SlipDetectorRegistry` registers no `IonoFreePhase` creator.

`CycleSlipManager` drives the whole set through one loop. It reaches the MW detector's additional API (WL-lock queries, pivot-handover hints, per-satellite quality scoring) through a typed down-cast (`asMelbourneWubbena`), which keeps the base interface small.

The network does not use this registry for any detector. The `buildStation` lambda of `Application::networkSetup_` (`src/core/network/driver/ApplicationNetworkSetup.cpp`) constructs each station's set directly from a *parallel* config block, `network.cycle_slip.{mw_detect, turbo_edit, if_phase_td}`. These keys are distinct from their `estimation.cycle_slip` namesakes, and all three default off in the loader. Both reference configurations set `mw_detect` and `if_phase_td`; the loader also sets `mw_detect` when the phase-leveling carry is armed with no slip detector. `turbo_edit` stays off in the reference configurations (its setting is KEEP_OFF in `docs/CONFIG_promotion_ledger.json`). The two paths share the detector *classes*. They do not share the composition mechanism, the config block, or the response: the network refloats where the user path repairs (Section 15.3).

Composing by hand has one important consequence. With `network.cycle_slip.mw_detect` on and `network.cycle_slip.turbo_edit` off (and the phase-leveling carry armed, which the MW and TurboEdit detectors require), the station receives an `MwSlipDetector` and a null `TurboEditResolver`. The null pointer is what directs the detection branch away from integer repair and to the refloat path.

`turbo_edit` (Section 15.4) is *not* a detector kind. It is a *coordination mode*. When the MW detector is enabled, it pairs the MW wide-lane integer with the geometry-free phase jump to recover per-signal integers. It is documented separately from the detectors below.

15.2 Test-statistic combinations

15.2.1 Melbourne–Wübbena (wide-lane)

The Melbourne–Wübbena (MW) observable is the wide-lane carrier phase minus the narrow-lane code. Introducing the wide- and narrow-lane wavelengths

$$\lambda_{\mathrm{WL}} := \frac{c}{f_1 - f_2}, \qquad \lambda_{\mathrm{NL}} := \frac{c}{f_1 + f_2}, \tag{223}$$

the engine forms (`MwSlipDetector::computeMwMeters` in `src/core/slip/MwSlipDetectorPairs.cpp`)

$$\boxed{\mathrm{MW}^i \;=\; \underbrace{\frac{f_1\,\Phi_1^i - f_2\,\Phi_2^i}{f_1 - f_2}}_{\text{wide-lane phase}} \;-\; \underbrace{\frac{f_1\,P_1^i + f_2\,P_2^i}{f_1 + f_2}}_{\text{narrow-lane code}}.} \tag{224}$$

The range-additive terms common to Equations (4) and (5), namely geometry, clocks and troposphere, cancel between the two bracketed combinations. The slant ionosphere cancels too. The wide-lane phase has the ionospheric coefficient $-(f_1\mu_1 - f_2\mu_2)/(f_1 - f_2)$ and the narrow-lane code has $+(f_1\mu_1 + f_2\mu_2)/(f_1 + f_2)$. With $\mu_s = f_1^2/f_s^2$ (Equation (1)) the two are equal, so (224) removes the first-order ionosphere exactly. What remains is nearly constant, equal to the wide-lane ambiguity plus hardware bias,

$$\mathrm{MW}^i \;\approx\; \lambda_{\mathrm{WL}}\,N_{\mathrm{WL}}^i \;+\; b_{\mathrm{MW}}^i \;+\; \varepsilon_{\mathrm{MW}}, \qquad N_{\mathrm{WL}}^i = N_1^i - N_2^i, \tag{225}$$

so a wide-lane slip $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ appears as a step of $\lambda_{\mathrm{WL}}\,\delta N_{\mathrm{WL}}$. For GPS L1/L2, $\lambda_{\mathrm{WL}} \approx 0.861\,\mathrm{m}$, and the dominant error is the narrow-lane *code* noise. With code noise $\sigma_P \sim 0.3\,\mathrm{m}$ the MW noise is $\sigma_{\mathrm{MW}} \approx \sigma_P/(\sqrt{2}\,\lambda_{\mathrm{WL}}) \sim 0.25$ wide-lane cycles, so the observable is limited by code noise. The integer-ready (BIA-corrected) variant subtracts the satellite MW bias b_{MW}^i, which has the same algebra as Equation (224) applied to the OSB phase and code biases of Equations (4) and (5).

The series $\mathrm{MW}^i(k)$ is tested against a robust adaptive gate. `estimation.cycle_slip.mw_detect_reference` selects the statistic.

The recommended choice is the first time difference $\mathrm{TD1}(k) = \mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$, gated against the median absolute deviation of its own history. The bundled real-time and replay configs set it explicitly. The second time difference $\mathrm{TD2}(k) = \mathrm{MW}^i(k) - 2\,\mathrm{MW}^i(k-1) + \mathrm{MW}^i(k-2)$ is available (`mw_td_order`). It is insensitive to curvature, at a noise penalty of $\sqrt{6}$ against $\sqrt{2}$ for TD1. `MwSlipDetector::activeStatScale` (`src/core/slip/MwSlipDetectorStats.cpp`) holds the white-noise scaling of each statistic.

TD1 is *not* the loader default. A config that leaves `estimation.cycle_slip.mw_detect_reference` unset uses the legacy Hatch mean, the deviation of the sample from the arc's running mean. It remains the default for backward compatibility and as a case for comparison, alongside the causal local median (`hatch_mean`, `local_median`, `time_difference`). In the Hatch and local-median modes the deviation is a level deviation. In the time-difference modes it is TD1 or TD2 itself, which is immune to a trend in the arc.

The threshold is derived from the data alone, and no σ from the stochastic model enters it. With $\hat{\sigma}$ the MAD-based spread of the active statistic over the arc,

$$\tau \;=\; k_{\mathrm{MW}}\,\max\!\big(\hat{\sigma},\;\sigma_{\min}\big), \tag{226}$$

where k_{MW} is `mw_sigma_multiplier` (default 8) and $\sigma_{\min}$ is the absolute floor `kMinSigmaMeters` ($0.01\,\mathrm{m}$). $\hat{\sigma}$ is used once the arc holds 15 samples (`kMinEpochsForAdaptive`). Before that, the floor is the larger of $\sigma_{\min}$, a young-arc MAD of the samples so far, and an EWMA-smoothed cross-satellite MAD of the zenith-normalised deviations of the other arcs, re-expanded by $1/\sin\theta$ (the elevation clamped at 10° from below). The cross-satellite pool widens in steps when a group is sparse: constellation and band pair, then the constellation, then all satellites except GLONASS. With no pool at any tier, the gate is a gross-slip bound of $\min(\lambda_{\mathrm{WL}}, 1\,\mathrm{m})$. The construction is in `MwSlipDetector::computeAdaptiveThreshold` (`src/core/slip/MwSlipDetectorStats.cpp`) and its empirical constants are not reproduced here.

Wide-lane lock. The detector also supplies the wide-lane integers that ambiguity resolution consumes. An arc keeps the running mean $\bar{m}$ of MW^i and its standard error $\sigma_{\bar{m}}$. With $\bar{n} = \bar{m}/\lambda_{\mathrm{WL}}$ and $f = \bar{n} - \mathrm{round}(\bar{n})$, the arc locks to the integer $\mathrm{round}(\bar{n})$ when it holds at least 10 samples and

$$|f| + 3\,\sigma_{\bar{m}}/\lambda_{\mathrm{WL}} \;<\; \tfrac{1}{2}, \qquad |f| < 0.30, \tag{227}$$

and it is demoted when the running mean drifts more than 0.4 cycle from the locked integer (`kWlLockDriftMaxCyc`). Where a satellite bias product (BIA) is applied, $\bar{m}$ is the bias-corrected statistic. Otherwise, an arc with at least 20 samples, $|f| \le 0.20$ and $\sigma_{\bar{m}}/\lambda_{\mathrm{WL}} \le 0.05$ may commit its residual $f\,\lambda_{\mathrm{WL}}$ as an in-house satellite bias, blended across arcs by an EWMA of weight 0.2 and refused when it disagrees with the stored value by more than 5σ (`MwSlipDetector::tryCommitOrUpdateBias`).

15.2.2 Extra-wide-lane (EWL)

For satellites with three or more frequencies the MW algebra (Equation (224)) is applied to a pair of *non-L1* carriers (`EwlSlipDetector::computeEwlMwMeters`; band digit ≥ 2). The two carriers are close in frequency, so $f_1 - f_2$ is small and the extra-wide-lane wavelength

$$\lambda_{\mathrm{EWL}} \;=\; \frac{c}{f_1 - f_2} \tag{228}$$

is very long: $\lambda_{\mathrm{EWL}} \approx 5.86\,\mathrm{m}$ for GPS L2/L5 and $\approx 9.77\,\mathrm{m}$ for Galileo E5a/E5b. Metre-level code noise is then $\lesssim 0.05$ EWL cycles. The EWL integer $N_{\mathrm{EWL}} = N_2 - N_5$ resolves in one epoch, and the size of a slip δN_{EWL} is determined almost without error. With the L1/L2 wide-lane (Equation (225)) this gives two independent band-difference constraints,

$$\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2, \qquad \delta N_{\mathrm{EWL}} = \delta N_2 - \delta N_5, \tag{229}$$

which leave the per-signal triple $(\delta N_1, \delta N_2, \delta N_5)$ with one degree of freedom [8]. The detector tests the deviation of the EWL statistic from the arc mean against $\max(k_{\mathrm{EWL}}\hat{\sigma},\ \tfrac{1}{2}\lambda_{\mathrm{EWL}})$, with k_{EWL} the `ewl_sigma_multiplier` (default 5) and $\hat{\sigma}$ the MAD-based spread of the between-epoch differences over a 30-epoch window; the floor is half an extra-wide-lane cycle.

15.2.3 GRAPHIC (single-frequency half-sum)

The GRAPHIC combination averages the code and phase on a *single* band (`GraphicSlipDetector::computeGraphic`):

$$\mathrm{GR}_s^i \;=\; \tfrac{1}{2}\big(P_s^i + \Phi_s^i\big), \tag{230}$$

with Φ_s^i in metres. The slant ionosphere enters as $+\mu_s I_{r,1}^i$ on the code and $-\mu_s I_{r,1}^i$ on the phase of the *same* signal (Equations (4) and (5)), so the half-sum cancels it to first order. GRAPHIC is therefore ionosphere-free with one frequency, but it keeps half the code noise. The combination still contains the geometric range, the clocks, the troposphere, and the constant half-ambiguity $\tfrac{1}{2}\lambda_s N_s^i$. The between-epoch first difference

$$\mathrm{dGR}_s^i(k) \;=\; \mathrm{GR}_s^i(k) - \mathrm{GR}_s^i(k-1) \tag{231}$$

removes the constant ambiguity but not the geometry: dGR still holds the range rate times the epoch spacing, hundreds of metres for a moving satellite. Two tests are implemented, selected by `estimation.cycle_slip.graphic_trend_centred` (default off, setting status UNVALIDATED).

The default compares |dGR| with the gate. The gate is a multiple of the MAD sigma of the recent differences, a spread about their median, while |dGR| is measured from zero. For a moving satellite the test therefore fails at the first epoch the window gives a gate (the sixth epoch of an arc) and resets the arc there, again and again. It is kept as the default so that a run that selects the detector keeps its output. The integer estimate is $\delta N = \mathrm{round}\big(2\,\mathrm{dGR}/\lambda_s\big)$.

The trend-centred test uses the change of the difference from one epoch to the next, measured from its median,

$$r_s^i(k) \;=\; \mathrm{dGR}_s^i(k) - \mathrm{dGR}_s^i(k-1) \;-\; m_s^i(k), \tag{232}$$

where $m_s^i(k)$ is the median of that change over the last 30 epochs of the arc. A constant range rate cancels in the change, and the slowly varying range acceleration is the median, so a moving satellite sits at zero and only a jump or noise remains. The gate is a multiple of the MAD sigma of the same window, so the threshold and the tested value share one centre. A slip of δN cycles on signal s produces a jump of $\frac{1}{2}\lambda_s\,\delta N$ in r, and the integer estimate is $\delta N = \mathrm{round}\big(2\,r/\lambda_s\big)$. The first possible detection is one epoch later than in the default, since the second difference exists from the third epoch of an arc. With $\sigma_P \sim 0.3\,\mathrm{m}$ the noise of r is $\sim 0.4\,\mathrm{m}$, so the 6σ gate sits near 2 m, about 20 L1 cycles. The hard floor `kMinThresholdM` $= 0.5\,\mathrm{m}$ (≈ 5 L1 cycles) applies only on very quiet arcs. GRAPHIC is therefore a coarse backstop and not a sub-cycle detector [2, 44].

15.2.4 Geometry-free phase (staged)

The geometry-free (GF) observable is the between-band phase difference

$$\mathrm{GF}^i \;=\; \Phi_1^i - \Phi_2^i \;\approx\; (\mu_2 - \mu_1)\, I_{r,1}^i \;+\; (\lambda_1 N_1^i - \lambda_2 N_2^i), \tag{233}$$

which removes geometry, clocks and troposphere exactly but keeps the *difference* of the two ionospheric delays. $I_{r,1}^i$ varies smoothly, so a second-order time difference

$$\mathrm{TD2}_{\mathrm{GF}}(k) \;=\; \mathrm{GF}^i(k) - 2\,\mathrm{GF}^i(k-1) + \mathrm{GF}^i(k-2) \tag{234}$$

cancels the ionospheric trend up to quadratic order over the sampling interval and leaves noise plus any slip (`GfSlipDetector::computeGfMeters` for Equation (233); the TD2 update is the test statistic for a level jump). The detector needs a high data rate ($\geq 1\,\mathrm{Hz}$). At a coarse rate the remaining ionospheric curvature is too large for (234). The detector is in the tree but excluded from the build.

The TD2 statistic (234) is still used elsewhere. `slip/IonoVariabilityMonitor` forms the same second difference per satellite on the two lowest RINEX bands (`selectPrimaryPair`, `computeGfMeters`), and it *is* compiled. It is a process-noise *monitor*, not a detector. Its per-satellite output is the clamped squared ratio of two MAD-derived values of σ for $\mathrm{TD2}_{\mathrm{GF}}$, one over a recent window and one over a baseline window,

$$m^i \;=\; \mathrm{clamp}\!\left(\left(\hat{\sigma}_{\mathrm{recent}}/\hat{\sigma}_{\mathrm{baseline}}\right)^2,\;\; 1,\;\; m_{\max}\right), \tag{235}$$

with a small floor on $\hat{\sigma}_{\mathrm{baseline}}$. The ratio normalises itself. The slowly varying obliquity and curvature content of Equation (233) is present in *both* windows and cancels. A quiet sky gives $m^i \approx 1$ at any elevation, and the $1/\sin^2\theta$ obliquity scaling already in the ionospheric process noise is not counted twice. The monitor treats a cycle slip as an isolated single-epoch outlier, which the MAD is designed to *reject*. That is the opposite of a detection statistic, so the monitor is not a substitute for the staged detector.

The monitor serves the user path only, and `estimation.adaptive_iono_process_noise.enable` (default off) arms it: m^i scales the static ionospheric spectral density into a per-satellite zenith override, which `StateManager::setIonoProcessNoiseOverrides` consumes. The real-time engine does not use it, and the network reads no monitor output (the network setting that once did was deleted). The window lengths and the clamp $m_{\max}$ are empirical and stay in config.

15.2.5 Ionosphere-free phase (staged on the user path; armed on the network path)

The dual-frequency ionosphere-free (IF) phase combination uses the coefficients

$$c_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \qquad c_2 = \frac{f_2^2}{f_1^2 - f_2^2}, \qquad c_1 - c_2 = 1, \tag{236}$$

giving $\Phi^i_{\mathrm{IF}} = c_1\Phi^i_1 - c_2\Phi^i_2$ and the effective wavelength $\lambda_{\mathrm{IF}} = \left|c_1\lambda_1 - c_2\lambda_2\right|$ (`IfPhaseSlipDetector::computeIfWavelengthMeters`); for GPS L1/L2 $\lambda_{\mathrm{IF}} \approx 0.107\,\mathrm{m}$. The detector does not test the raw combination. It forms the IF observed-minus-computed (OMC) value and differences it twice. The first difference is between satellites, against a high-elevation pivot p_g. It removes the receiver clock, the inter-system offset $c\,\delta t_r^{(g)}$ and the satellite phase clock. The second is between epochs. It removes the slowly varying wet troposphere and the constant ambiguity:

$$\nabla\Delta\Phi^{i,p_g}_{\mathrm{IF}}(k) = \left[\Phi^i_{\mathrm{IF}}(k) - \Phi^{p_g}_{\mathrm{IF}}(k)\right] - \left[\Phi^i_{\mathrm{IF}}(k-1) - \Phi^{p_g}_{\mathrm{IF}}(k-1)\right], \tag{237}$$

which is near zero for a clean arc and jumps by $\sim \delta N\,\lambda_{\mathrm{IF}}$ on a slip. In (237) the symbol Φ_{IF} stands for the observed-minus-computed value, since the model subtraction is what removes the satellite range and clock. The between-satellite step does not cancel the differential geometric error $(\boldsymbol{e}^i - \boldsymbol{e}^{p_g})^{\mathrm{T}}\delta\boldsymbol{r}$, so flagging must be gated by the filter's 3D position sigma. The gate requires a static site with a precise a priori position. The single-receiver kinematic path cannot guarantee that. There the detector is never instantiated, and the user toggle `estimation.enable_if_slip_detector` has no effect. The network estimator holds every station position *fixed*, so the gate is always satisfied, and the detector is armed there.

On the network path (the `buildStation` lambda of `Application::networkSetup_`, gated on `network.cycle_slip.if_phase_td`) an `IfPhaseSlipDetector` is instantiated per *non-pivot* station. The pivot's ambiguities are fixed by the gauge, so on the legacy datum path the pivot has nothing to refloat. Under the prioritised phase datum `network.phase_datum_prioritized`, which the reference configurations use, the detector is instantiated for every station, because the former pivot then has arcs that are not anchored and still need detection. By default the detector forms one signal pair per satellite (the first two distinct phase bands in key order). With `network.cycle_slip.if_phase_td_pairs` set to `all`, which the reference configurations do, it forms every pair of distinct frequencies, each on its own between-epoch gradient chain, pool and reset key, so a slip that leaves the first pair silent is still tested on the others. The detector is built with `setDeferFlagging(true)`. It caches each per-epoch $\nabla\Delta\Phi^{i,p_g}_{\mathrm{IF}}$ of Equation (237) and does *not* apply a threshold locally. The driver (`networkSlipIfTdDriver` in `src/core/network/slip/SlipQc.cpp`) *pools* the deferred jumps across stations by signal pair and removes their median med. The median is the network common mode, a wobble of the frame or datum. It does not depend on elevation, so it is removed in raw metres. Each residual is then normalised to a zenith equivalent,

$$z \;=\; \left(\nabla\Delta\Phi^{i,p_g}_{\mathrm{IF}} - \mathrm{med}\right)\,\sin\theta, \tag{238}$$

and the pooled robust scale $\hat{s} = 1.4826\,\mathrm{median}\,|z|$ is formed (`ifTdPoolStats`). The reference satellite of each constellation is left out of the scale, because its double difference is zero by construction

and would deflate the median. At each satellite's threshold the scale is multiplied by $1/\sin\theta$ again, so *one* scale serves the whole sky. The map is the free function `ifElevMap` ($1/\sin\theta$, clamped at 5° so that it stays bounded near the horizon, which is already elevation-masked). The effective scale $\hat{s}_{\text{eff}}$ is an empirical-Bayes blend of the pooled prior with the station pair's own EWMA scale (`ifTdShrunkScale`). The EWMA carries the weight $w = n/(n+\kappa)$, with n the number of clean samples of the station pair. A quiet reference station and a noisy station with modernised signals therefore each get their own envelope, and the majority does not set one threshold for all. A satellite at elevation θ is flagged when

$$\left|\nabla\Delta\Phi_{\text{IF}}^{i,p_g} - \text{med}\right| \;>\; \max\!\left(\frac{k\,\hat{s}_{\text{eff}}}{\sin\theta},\;\; c_{\text{floor}}\,\lambda_{\text{IF}}\right), \tag{239}$$

where k is a loose MAD multiplier. The floor is proportional to the $(1,1)$ IF jump $\lambda_{\text{IF}} = c/(f_1+f_2) \approx 10.7\,\text{cm}$ and is kept in raw metres, so a simultaneous one-cycle slip is caught at any elevation. Only the clean survivors are folded into the station-pair EWMA, so the scale does not absorb the event it has to catch. The settings k, c_{floor}, κ and the EWMA weight are `network.cycle_slip.if_phase_td_k_mad` (default 8), `if_phase_td_floor_frac` (1.0), `if_phase_td_shrink_kappa` (20) and `if_phase_td_ewma_alpha` (0.15). A fifth setting, `if_phase_td_min_pool`, only counts pools below a size for a log line and enters no decision. They are read once per call inside the driver.

A flag never repairs and never resets filter state in place. It restarts the detector's own arc (arc hygiene). Its two estimator-side consequences are gated separately inside `refloat_pair`. The flagged pair enters the slip funnel of Section 15.3.1 when the double-difference invariant carry is armed. The band ambiguities are marked pending-reseed (the refloat itself) only when the phase-leveling carry is warm. On a cold or absent carry the flag is emitted as diagnostic telemetry and changes nothing, so a clean run log is no evidence that the response acted; the run-end counters are. The loose gate is intentional. The driver is a coarse first pass, and the sharp integer decision is left to a post-estimation phase validation that is not yet built; it is not the discriminating solver of Section 15.4. What the network does with a flag (which bands it frees, how the level is re-established) is the subject of Section 24.

A restart leaves the detector with no double difference for that satellite at the next epoch, so a slip repeated one epoch later passes unseen. With `network.cycle_slip.if_phase_td_reseed_on_flag` (default off, setting status UNVALIDATED) the chain instead keeps the flag epoch's between-satellite gradient as the origin of the next double difference (`IfPhaseSlipDetector::reseedSatellite`). The next double difference then measures only what happened between the two epochs: about zero on an arc that stays clean, and the size of a second step if the slip repeats. A one-epoch spike reads with the opposite sign at the next epoch, so under the re-seed it flags twice where a restart flags once.

The reference satellite. The pool cannot see a slip on the between-satellite reference itself. Such a jump is common to every satellite of that station's chain and is removed with med, and the reference's own double difference is zero. The detector therefore tests the reference separately. A slip on the reference appears as the same step on all n non-reference double differences d_j of the chain, so their median $\text{med}_j\, d_j$ estimates it and their spread $s_d = 1.4826\ \text{median}_j|d_j - \text{med}_j\, d_j|$ measures the noise. The legacy gate fires on $|\text{med}_j\, d_j| > \frac{1}{2}\lambda_{\text{IF}}$ with at least two non-reference members. The empirical gate (`network.cycle_slip.if_phase_td_ref_gate`, default off, set in both reference configurations) keeps that condition as the candidate filter and decides each candidate by three further conditions.

1. Scale: $|\text{med}_j\, d_j| > \max(\phi\,\lambda_{\text{IF}},\; k_r\,\hat{s}_r)$, with $\hat{s}_r$ an empirical-Bayes blend of the chain's EWMA of s_d with the cold prior $\phi\,\lambda_{\text{IF}}/k_r$, $k_r = 6$ and $\phi = 0.5$ (`if_phase_td_ref_k`, `if_phase_td_ref_`

floor_frac). A chain with fewer than 4 non-reference members widens the physical floor by a factor of 3.

2. Coherence: a genuine reference slip is a common step, so $s_d \leq 0.5\,|\mathrm{med}_j\, d_j|$. When the family is incoherent and at least $\max(2, \lceil n/2 \rceil)$ members individually exceed λ_{IF}, the event is a multi-satellite station event and the reference is reset without the median rebase. Any other incoherent family is a drift and does not fire.
3. Persistence: after more than one consecutive firing on the same reference (`if_phase_td_ref_max_consecutive`, default 1) the gate stops firing and forces a re-election that excludes that satellite, which then enters the pool test.

The reset keys are parsed by `parseIfPairResetKey` and the flagged pair is refloated through the same `refloat_pair`. Both bands of the pair are refloated, because the IF combination cannot localise which one slipped. The exception is the common-band scope of Section 15.3, which, when armed, keeps the bands the evidence exonerates. With `if_phase_td_ref_arm_all` the reference test also runs on the additional chains of the `all` pair mode. Two further refinements, an abstention while the empirical scale has fewer than κ samples and an elevation-normalised bar, are UNVALIDATED and off. The gate is pushed only on the network path; the staged user and kinematic path keeps the legacy fixed gate.

15.3 Network slip QC: raw evidence and refloat over repair

The IF-phase driver of Section 15.2.5 is one component of the network's slip layer. The components are free functions in `network/slip/SlipQc` (`src/core/network/slip/SlipQc.cpp`), called in a fixed serial, station-ordered sequence from the per-epoch PASS A slip scan (`Application::networkPassASlipScan_`). The scan is an ordered list of named stages, declared in `network/slip/PassASlipScan` (the detection half) and `network/slip/PassASlipDrains` (the response half), and each stage reads the epoch through one view of references (`PassAView`).

1. The parallel station scan (`passAScanStations`) models each station and runs its MW and IF-phase detectors. It writes only index-addressed per-station slots, so it is safe under OpenMP. Under `mw_detect` it also collects the admitted refloat candidates (`passAMwAdmit`) and the evidence for the common-band scope (`passAMwEvidence`).
2. The serial folds: station presence, drop reasons, the second code and second phase channel bookkeeping.
3. The shared raw observables (`passARawObservables`), below.
4. The MW admission counts, which are formed whether or not any setting acts on them.
5. The common-band verdict (`passACommonBandVerdict`), formed once before any drain.
6. The drains, in order: the integer-repair drain (`networkSlipDrainTORepairs`, applying the integer repairs of the network TurboEdit resolver to the phase-leveling carry), the MW and LLI drain (`networkSlipDrainMwRefloat`), the IF-phase driver, the arc-scoped geometry-free test (Section 15.3.2), the wind-up restart drain, the two gap re-entry drains (per satellite and per signal) and the evaluation-only injected-free drain, each behind its own setting (off in the loader).

The shared evidence is a separate component, `network/slip/RawObservables`.

Shared evidence. Once per epoch per (station, satellite), `networkComputeRawObservables` forms the MW combination of Equation (224) and the GF combination of Equation (233) on the two lowest distinct bands whose phase *and* matching code are both present. It also records the first band's phase-minus-code level and the per-band RINEX LLI byte. These are absolute levels, not differences. `networkQualifyRawObservables` then adjudicates each observable's step against the immediately adjacent processed epoch, that is $\mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$ and $\mathrm{GF}^i(k) - \mathrm{GF}^i(k-1)$. The threshold has the shape of Section 15.2.5: a multiple of an empirical scale, floored from below,

$$g_{\mathrm{MW}} = \max\bigl(k_{\mathrm{mw}}\,\hat{s}_{\mathrm{MW}},\ 0.5\,\lambda_{\mathrm{WL}}\bigr), \qquad g_{\mathrm{GF}} = \max\bigl(k_{\mathrm{gf}}\,\hat{s}_{\mathrm{GF}},\ 0.04\,\mathrm{m}\bigr), \tag{240}$$

with $k_{\mathrm{mw}} = k_{\mathrm{gf}} = 5$. The empirical term $\hat{s}$ is a per-station robust EWMA (weight 0.15) of that station's own clean steps, built from the same $1.4826\,\mathrm{median}\,|\cdot|$ estimator (exposed for test as `networkRobustStepScale`); a station with no folded history takes the scale of the epoch's own steps as its prior. The floor is a physical integer bound kept in raw metres: a fraction of a wide-lane cycle λ_{WL} for MW (default 0.5 cycle) and a few centimetres for GF (default 4 cm). The gate therefore falls back to the physical bound when the empirical term is cold or degenerate, never to a stochastic-model σ. The MW adaptive threshold of Section 15.2.1 follows the same rule. The ordering also matches the IF-TD pool arm: each satellite is thresholded with the pre-update scale, and only the clean survivors are folded into the EWMA, so the scale does not absorb the event it has to catch. A step is measurable only against an adjacent predecessor. Across a gap it cannot be measured, and the code treats that absence of evidence as neither evidence of a slip nor evidence of continuity. The multipliers and floor values live in `SlipQcParams`.

MW admission. A flag of the detect-only MW path becomes a refloat candidate when it passes either of two rules (`networkMwAdmitCandidate`, a pure function). The *integer rule* requires a known non-zero wide-lane integer whose rounding residual is within `network.cycle_slip.turbo_edit_frac_max` (default 0.15 cycle). This rule grades how cleanly the step rounds and says nothing of how far the step stands above the noise, although the detect-only path never applies the integer. The *evidence rule* (`network.cycle_slip.mw_evidence_refloat`, off in the loader, set in both reference configurations) tests the second property directly: the lane has no satellite bias correction, the arc holds at least `mw_evidence_min_arc_epochs` (10) samples, and |deviation|/threshold $\geq$ `mw_evidence_ratio_min` (3.0). Bias-corrected lanes are excluded, because there a re-reference of the detector's own pivot appears as a synchronous step on several satellites. Candidates that fail both rules are counted by cause (loose residual, unknown integer), with the same counts formed when the evidence rule is off, so the effect of arming it is sized beforehand.

Common-band scope. On a triple-frequency arc, a slip on one band moves every MW pair that contains the band, and each firing pair would free both of its bands, so the union is all three. The scope (`network.cycle_slip.common_band_scope`, set in both reference configurations) decides per (station, satellite, epoch) which bands the evidence implicates, by set algebra. An MW pair is *silent* when the detector processed a sample for it, its arc held at least `common_band_min_arc_epochs` (10) samples and it did not flag. A silent pair exonerates its two bands, unless its IF-phase pair flagged this epoch (the equal-count slip leaves MW silent while the IF combination moves λ_{IF} per cycle), unless the IF reference test cannot test it (`common_band_ref_arc`: on the reference satellite the double difference is zero by construction), and unless one of its bands carried a loss-of-lock bit at the previous epoch. A band with a loss-of-lock bit is implicated regardless. With B the tracked bands,

$$\text{implicated} \;=\; B \setminus \bigl\{b:\ \text{no LLI on } b \ \wedge\ \exists \text{ silent, un-vetoed pair } p \ni b\bigr\}. \tag{241}$$

Narrowing needs positive exoneration: with no MW flag, nothing exonerated, or an empty implicated set, the response stays all-band. The verdict is formed once per epoch before the drains and governs each of them, so the per-epoch union of freed bands is exactly the implicated set. The scope only narrows a response; the responses themselves are described in Section 24.

Code-bias jump test. The emitted code biases carry a per-signal discontinuity counter that the user receiver reads. The structural triggers (a clock-pivot change, a re-reference of a signal group, a change of a satellite's band set) detect gauge re-references but not an estimate that wanders under weak separation. With `network.code_bias_disc_jump_test` (off in the loader, set in both reference configurations) the counter also steps when the emitted code bias of a satellite and signal differs from its previous emission by more than the phase-bias threshold of 0.05 m, provided the baseline is fresh. It is a magnitude test on the emitted product and reads no observation; the product and its discontinuity declarations belong to Section 24.

Response doctrine. On the network path a detection refloats: it marks the ambiguity pending-reseed so that PASS B re-estimates it from that epoch's data, and it never applies an integer. This differs from the user-path TurboEdit repair of Section 15.4. The reason is the network's missing satellite phase biases. Without them the network wide-lane integer is not trustworthy, and a wrong integer written into a carried value would bias the gauge, whereas a refloat costs only re-convergence. The one exception is `networkSlipDrainT0Repairs`, active only when the network TurboEdit key of Section 15.4 is armed. It shifts the phase-leveling carry by $\delta N_s\,\lambda_s$ continuously, with the variance unchanged and the covariance preserved. It does so only for an integer that is both confident and of sane magnitude. Both bounds are network settings, separate from the resolver's own constants: a rounding residual (Equation (248)) of at most 0.15 cycle and a magnitude of at most 50 cycles by default. A low-confidence detection is left untouched: it is neither repaired, refloated nor dropped. The float wide-lane's low-confidence repairs are measured to be mostly false positives, and acting on them discards a good carry. The resolver's reset set is not acted on either; without satellite phase biases it flags tens of satellites per epoch. The refinements of the response (the MW re-flag skip, which frees no band twice for one step, the co-resumed share of a gap re-entry, and the re-levelling of freed arcs) are described in Section 24.

Status. Every component is off in the loader, and the reference configurations enable the following. `network.cycle_slip.mw_detect` enables the MW/LLI drain, `network.cycle_slip.mw_lli_corroborate` the LLI qualification and `network.cycle_slip.gap_instrument` the arc-resumption instrument. The raw observables are computed only when a reader asks for them (`SlipQcParams::anyConsumer`). The adjudication above runs only when a decision setting is armed; a run with the raw trace alone computes the levels and adjudicates nothing. Every use of the verdicts in the drains is therefore guarded on whether the evidence exists this epoch. Without it the LLI path stays decisive, because a gate that fired on missing data would do more harm than the chatter it is meant to suppress. The qualification never excludes an LLI assertion. An assertion acts when a measured MW or GF step corroborates it, or when the station's own LLI behaviour is still judged credible by a two-state hysteresis on the EWMA of its LLI-set fraction (`networkLliCredibilityStep`). Only an uncorroborated assertion at a station whose LLI is judged not credible is downgraded. It is then emitted as suppressed and counted, not dropped.

The raw step evidence has a second reader outside this chapter. With `network.carry_invariant_innov_corroborate` armed (off in the loader and in the reference configurations, setting status KEEP_OFF), the innovation gate on the carried double differences does not re-tap a chord

whose arc the raw observables measured as continuous. The gate is fail-safe in one direction only: a measured flat arc suppresses the re-tap, and absence of evidence still re-taps. The slip response is therefore narrowed and never removed. The gate is part of the estimator and is described in Section 22.

15.3.1 The slip funnel

The epoch's *slip funnel* (`NetworkSlipFunnel` in `network/slip/SlipFunnel`) is the set of (station, ambiguity) arcs that the epoch treats as slipped. It is the one list the carried quantities' slip responses read, so a detector adds an arc to it once and every reader sees the same set. It has two members: `carry_slipped_ambs`, the funnel proper, and `carry_slipped_raw`, the subset a raw detector saw step. It is empty at the start of an epoch, filled before the phase datum is elected, read from the datum on, and gone at the epoch's end. A flag that must survive to the next epoch is run state, never a funnel entry.

The producers, in epoch order, each take the funnel as an output.

1. `networkSlipFunnelOpen`: the previous epoch's flags from the innovation gate, consumed one-shot.
2. The PASS A drains of the preceding subsection (MW and LLI, IF-phase, geometry-free arc test, wind-up restart, gap re-entry), into the raw subset.
3. `networkSlipFunnelUnionRaw`: the raw subset joins the funnel.
4. The signal-integrity and measurement-admission refloats, into both sets.

The readers are the phase-anchor election (anchor hygiene, the chain datum, the anchor-slip handover), the gap-resumption count (`networkGapResumeCensus`; a marked arc is never counted as bridged), the erase policy of PASS B, the double-difference carry (a funnel arc leaves the carried set and is re-tapped fresh) and the emission. The funnel serves two design rules. A stiff quantity carried over time must have a slip response, and the funnel is the feed of that response. A mechanism that changes which information reaches which states is an event at both edges, with counters and identities; each funnel entry carries its identity and is the decoupling edge of that arc for the epoch, while the re-coupling edge is the gap-resumption count and the arc ledger.

15.3.2 Arc-scoped geometry-free jump test

Three slip classes pass the MW, IF time-difference and loss-of-lock detectors: an equal count on both bands (the IF step sits on the IF floor and MW does not move), a family of one or two wide-lane cycles whose geometry-free step the free slant ionosphere absorbs, and a slip repeated at the epoch after an IF flag. The raw geometry-free phase step is the one pre-fit witness of all three. `network.cycle_slip.gf_arc_jump` (off in the loader, setting status UNVALIDATED) tests it per (station, satellite) dual-frequency arc (`network/slip/GfArcJump`). With the geometry-free level $\mathrm{GF}(t)$ of (233) on the primary pair, its one-epoch step $d(t) = \mathrm{GF}(t) - \mathrm{GF}(t-1)$ and t_c the arc's last *clean* step,

$$r(t) = d(t) - d(t_c), \qquad \text{flag when} \quad |r(t)| > \max\big(\gamma,\ k_{\mathrm{jump}}\, 1.4826\, \mathrm{MAD}(\text{recent clean } r)\big). \tag{242}$$

The floor γ is `gf_arc_jump_m` (0.04 m, below the 0.0539 m geometry-free step of an equal one-cycle slip on GPS L1/L2), and the scale term is off at the default `gf_arc_jump_k` of 0, the multiplier k_{jump}; it applies once an arc holds `gf_arc_jump_min_count` (10) clean values of the last `gf_arc_jump_window` (20). The reference is the last clean step, not the previous one, so a flagged step never

becomes the reference and a slip repeated at the next epoch still differs from it, while an isolated slip flags once. An arc is consecutive processed epochs on one pair with one step span; a gap, a pair change or a span change restarts it, and testing begins at its third sample. After two consecutive flags the reference is re-seeded from the next step, which bounds the flags that a persistent change of the ionospheric rate can cause. A flag frees the pair's two bands as an IF-phase flag does, subject to the common-band verdict.

15.4 TurboEdit dual-frequency resolution

The TurboEdit resolver is a pure MW+GF joint integer solver (`include/slip/TurboEditResolver.hpp`). Its bodies are split by concern: the per-epoch driver, construction and statistics in `src/core/slip/TurboEditResolver.cpp`, the first pass over the observations (the geometry-free chains and the cross-satellite pools) in `TurboEditChain.cpp`, the adjudication of each row and the reconciliation of the epoch in `TurboEditAdjudication.cpp`, and the gap bridge in `TurboEditBridge.cpp`. It owns the per-arc geometry-free jump history and the integer arithmetic. It returns per-signal integer repairs and a reset-fallback set, and it does not modify filter state. It is wired into the post-processing `CycleSlipManager` and the real-time engine, and is enabled by `estimation.cycle_slip.turbo_edit`.

That key is default off. The bundled real-time and SBF-replay configs turn it on; the internal PPP-RTK AR config leaves it off. The network has a separate key, `network.cycle_slip.turbo_edit`, which builds one resolver per station and feeds the repair drain of Section 15.3. On the network the integer repair is licensed only at about 1 s sampling, where the geometry-free drift between epochs is small; the loader steers 30 s runs to `mw_detect`. With that key off and `network.cycle_slip.mw_detect` on, the same MW detector runs as a pure detector, and the network refloats instead of repairing (Section 15.3). Of the sub-toggles below, the soft-LLI bridge (`turbo_edit_bridge_lli`) is default off and the MW-only code-domain veto (`turbo_edit_mw_only_veto`) is default on.

15.4.1 The 2×2 system

For a dual-frequency phase pair (s_1, s_2) on satellite i, ordered higher frequency first ($f_{s_1} > f_{s_2}$, hence $\lambda_{s_1} < \lambda_{s_2}$), the resolver pairs two scalar slip observables. The first is the MW wide-lane integer of Equation (225). It is taken as input from the MW detector and never recomputed, so the empirically tuned MW path of Section 15.2.1 is the single source of δN_{WL}:

$$\delta N_{\mathrm{WL}} \;=\; \delta N_{s_1} - \delta N_{s_2} \qquad \text{(one equation, two unknowns).} \tag{243}$$

The second is the GF phase jump in metres. Writing the GF level as $\mathrm{GF}(t) = \lambda_{s_1} L_{s_1}(t) - \lambda_{s_2} L_{s_2}(t)$ (carrier in cycles), a simultaneous slip steps its ambiguity content by

$$\delta\mathrm{GF} \;=\; \lambda_{s_1}\,\delta N_{s_1} - \lambda_{s_2}\,\delta N_{s_2}, \tag{244}$$

estimated within the arc by the second-order time difference $\mathrm{TD2}(t) = \mathrm{GF}(t) - 2\,\mathrm{GF}(t-1) + \mathrm{GF}(t-2)$ of Equation (234). TD2 cancels the ionospheric delay up to quadratic order, so at the slip epoch it equals the integer jump plus phase noise. The TD2 chain requires three consecutive epochs; a non-consecutive history re-seeds the chain (Section 15.4.4).

Stacking Equations (243) and (244) gives the linear map from the integer slip pair to the two observed jumps:

$$\boxed{\begin{pmatrix} \delta N_{\mathrm{WL}} \\ \delta\mathrm{GF} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} 1 & -1 \\ \lambda_{s_1} & -\lambda_{s_2} \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} \delta N_{s_1} \\ \delta N_{s_2} \end{pmatrix}.} \tag{245}$$

The determinant $\det \boldsymbol{A} = \lambda_{s_1} - \lambda_{s_2}$ is non-zero for any cross-band pair. It is the centimetre-scale quantity `denom` of Equation (247), $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, and is distinct from the $\sim 0.86\,\mathrm{m}$ wide-lane wavelength. The system is therefore always invertible:

$$\boldsymbol{A}^{-1} = \frac{1}{\lambda_{s_1} - \lambda_{s_2}} \begin{pmatrix} -\lambda_{s_2} & 1 \\ -\lambda_{s_1} & 1 \end{pmatrix}. \tag{246}$$

The implementation does not invert the full matrix. It solves the second row for δN_{s_2} and back-substitutes into the first (`TurboEditAdjudication.cpp`, and `TurboEditBridge.cpp` for the bridged jump):

$$\delta N_{s_2} \;=\; \frac{\delta\mathrm{GF} - \lambda_{s_1}\,\delta N_{\mathrm{WL}}}{\lambda_{s_1} - \lambda_{s_2}}, \qquad \delta N_{s_1} \;=\; \delta N_{s_2} + \delta N_{\mathrm{WL}}, \tag{247}$$

where δGF is the in-arc TD2 or, across a data gap, the bridged jump of Section 15.4.4. The denominator $\lambda_{s_1} - \lambda_{s_2}$ (the variable `denom` in the source) is $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2. The amplification factor on the wide-lane integer is therefore $\lambda_{s_1}/(\lambda_{s_1} - \lambda_{s_2}) \approx -3.53$. Despite that lever, the GF equation must localise δN_{s_2} to well within ± 0.5 cycle; the split-capability gate below enforces this.

15.4.2 Integer rounding and validation

The float δN_{s_2} from Equation (247) is rounded and its residual recorded:

$$d_2 = \mathrm{round}\big(\delta N_{s_2}\big), \quad f = \big|\delta N_{s_2} - d_2\big|, \quad d_1 = d_2 + \delta N_{\mathrm{WL}}. \tag{248}$$

A candidate is accepted as a repair only if three conditions hold. The rounding is clean ($f \le 0.30$, the constant `kMaxFractional`, which leaves a $1 - 0.30 = 0.70$ cycle margin to the nearest wrong integer). The magnitudes are sane ($|d_1|, |d_2| \le$ `kMaxSlipCycles`, which is 1000 cycles). The pair is non-trivial ($d_1 \neq 0 \vee d_2 \neq 0$). An accepted candidate yields `action="repair"`. The pair $(d_1, d_2) = (0, 0)$ resolves as no slip, and the arc continues. Any other outcome resets the satellite.

The joint solve is trusted only when the GF evidence is sharp enough to discriminate the integer lattice. The float δN_{s_2} carries $\sigma_{\mathrm{TD2}}/|\lambda_{s_1} - \lambda_{s_2}|$ cycles of noise, so the solve is permitted only when

$$\sigma_{\mathrm{TD2}} \;\le\; \texttt{turbo_edit_split_max_sigma} \cdot \big|\lambda_{s_1} - \lambda_{s_2}\big| \qquad (\texttt{splitCapable}), \tag{249}$$

Here σ_{TD2} is the arc's own MAD-derived σ when the arc is mature, otherwise the cross-satellite tier value, otherwise NaN. NaN fails the gate, because the absence of an estimate is no evidence of sharpness. The default of `estimation.cycle_slip.turbo_edit_split_max_sigma` is 0.2 cycles. This gate is the only mechanism that makes the resolver rate-adaptive. Millimetre-level noise at 1 s passes easily; the ionospheric residual at a coarse rate (30 s) passes only while the arc is quiet. An MW-flagged pair that fails the gate falls back to a satellite reset. A GF-only exceedance on an arc that fails the gate is ignored, not reset, and the Kalman residual gate remains the backstop. When $\delta N_{\mathrm{WL}} = 0$ (an equal slip $\delta N_{s_1} = \delta N_{s_2}$, to which MW is blind), a GF-only flag fires when |TD2| exceeds the adaptive threshold $\tau = \max(k_\sigma \hat{\sigma}_{\mathrm{TD2}}, 0.04\,\mathrm{m})$, with k_σ the `turbo_edit_sigma_multiplier` (default 6). The floor sits just below $|\lambda_{s_1} - \lambda_{s_2}| \approx 0.054\,\mathrm{m}$, so a ± 1-cycle equal slip remains detectable. The joint solve takes a zero wide-lane jump for a GF-only flag only when the MW detector affirmatively reports the pair clean this epoch. MW silence alone is not taken as a zero wide-lane jump: a locked or immature MW arc can miss a real unequal slip, and forcing zero would alias it into a confident wrong repair. Without that evidence the wide-lane integer is unknown and the satellite is reset. An unstable ionosphere on the constellation (the cross-satellite IVI above 0.06 m) also resets the satellite instead of solving.

15.4.3 MW-only / code-event null line

The asymmetric case is an MW flag with a GF jump that is affirmatively zero. For a fixed $\delta N_{\mathrm{WL}} = k$, every integer pair $(\delta N_{s_1}, \delta N_{s_1} - k)$ predicts, through Equation (244), the GF jump

$$\delta\mathrm{GF} \;=\; \lambda_{s_2} k - (\lambda_{s_2} - \lambda_{s_1})\, \delta N_{s_1}, \tag{250}$$

with its zero crossing at $\delta N_{s_1} = \lambda_{s_2} k/(\lambda_{s_2} - \lambda_{s_1}) \approx 4.53\, k$ for GPS L1/L2 ($\approx 3.95\, k$ for L1/L5). Every small pair with $|\delta N_{s_1}| \le 3$ then predicts $|\delta\mathrm{GF}| \ge 8\,\mathrm{cm}$, orders of magnitude above phase noise. A flat GF therefore leaves only *null-line* pairs of $(4, 3)$ / $(9, 7)$-class magnitude: multi-cycle simultaneous slips on both signals with no loss of lock, which are implausible. The simpler explanation is a narrow-lane code excursion (multipath, NLOS, DLL re-lock), since $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$ and one wide-lane cycle corresponds to $\approx 0.86\,\mathrm{m}$ of narrow-lane code.

The pair is therefore classified as a code-domain artifact, and the caller skips the phase reset, when the GF jump is affirmatively zero within a gate bounded by the lattice discrimination:

$$|\mathrm{TD2}| \le \underbrace{\texttt{kCodeEventZeroSigma} \cdot \hat{\sigma}_{\mathrm{MAD}}}_{\text{zero gate}} \quad\wedge\quad \text{zero gate} \le \underbrace{\texttt{kVetoZeroGateMaxLambdaFrac} \cdot |\lambda_{s_1} - \lambda_{s_2}|}_{\text{discrimination cap}}, \tag{251}$$

with `kCodeEventZeroSigma` $= 3.5$ ($\approx 99.95\%$ two-sided) and `kVetoZeroGateMaxLambdaFrac` $= 0.5$. The cap is the rate-adaptive safeguard. A zero gate wider than half the smallest GF-visible quantum cannot distinguish stillness from a small slip; the veto then stands down and the MW flag governs. The classification has further trust conditions: the veto is enabled, MW flagged, GF did not exceed its threshold, no post-slip cooldown is active, the constellation's ionosphere is stable, neither signal carries an LLI, and the MAD maturity is ≥ 10 samples. The last condition is stricter than the 5 samples detection needs, because affirming stillness preserves state. The implied null-line pair is computed for the trace only and never applied; `did_slip` stays false so the GF chain continues. A real null-line slip is still caught. It moves that signal's Kalman phase innovation by $\ge |4\lambda_{s_1}| \approx 0.76\,\mathrm{m}$ in the same epoch, far beyond any residual gate, so the residual-driven reset corrects it one epoch late. The behaviour is toggled by `estimation.cycle_slip.turbo_edit_mw_only_veto` (default on).

15.4.4 Gap-spanning GF bridge

The TD2 chain needs three consecutive epochs. Any per-signal dropout or receiver gap re-seeds the chain, and the post-gap TD2 is then blind to a re-lock slip whose GF level jump falls between the two chains. When a chain break occurs within `estimation.cycle_slip.turbo_edit_max_gap_s`, the resolver stores the pre-gap GF level $\mathrm{GF}_{\mathrm{ref}}$, its timestamp, the arc's EWMA GF rate $\dot{\mathrm{GF}}$ and the pre-gap spacing dt_0. On return it forms the drift-corrected jump across the gap:

$$\boxed{\delta\mathrm{GF}_{\mathrm{gap}} \;=\; \mathrm{GF}_{\mathrm{now}} - \mathrm{GF}_{\mathrm{ref}} - \dot{\mathrm{GF}}\,\Delta t, \qquad \Delta t = t_{\mathrm{now}} - t_{\mathrm{ref}},} \tag{252}$$

which then takes the place of $\delta\mathrm{GF}$ in Equation (247). Here $\dot{\mathrm{GF}}$ is an EWMA ($\alpha = 0.1$) of the per-epoch GF rate, chosen so that its noise is not amplified when projected over the gap. The uncancelled ionospheric residual is a random walk over the gap. The detection threshold and the code-event zero gate are therefore widened by the factor $w = \sqrt{\Delta t/dt_0}$ (for $\Delta t > dt_0$), and the split-capability σ becomes $\sigma_{\mathrm{bridge}} = \hat{\sigma}_{\mathrm{MAD}}\, w$. The discrimination cap of the zero gate is not widened, because the integer lattice of Equation (245) does not stretch with the gap. The bridge cap defaults to 15 s in code and is raised to 60 s in the bundled real-time and replay configs.

An MW-flagged pair across the gap has three possible outcomes.

1. A *code event* (`action="code_event"`) when $\delta\mathrm{GF}_{\mathrm{gap}}$ is affirmatively zero within the widened gate. This is the null-line logic of Equation (251) applied across the gap, and it preserves phase through a re-acquisition DLL transient.

2. A *repair* or *moot* outcome when the joint solve runs with $\delta\mathrm{GF} = \delta\mathrm{GF}_{\mathrm{gap}}$. Each signal is dispatched by continuity in the observation domain. A signal observed at every epoch through the gap kept its ambiguity, so its nonzero δN is a *repair*. A signal that missed an epoch was pruned and reseeded on return, so its nonzero δN is *moot*: already absorbed, with the reset suppressed.

3. A *reset* when the across-gap σ is too coarse for the split, or when the candidate set holds no integer or more than one and no deferral applies.

The GF equation alone is degenerate along the null-line lattice of Equation (250): $+0.735\,\mathrm{m}$ fits both $(0, -3)$ and $(9, 4)$ at the millimetre level, and only the integer k separates them. The bridged solve therefore requires a unique surviving candidate after three guards. The first is the fractional gate of Equation (248). The second is a GF-separation guard: the candidate's own predicted jump must clear the zero gate, $|\lambda_{s_1} d_1 - \lambda_{s_2} d_2| > \texttt{zero_gate}$, so that a marginal noise exceedance cannot produce a multi-cycle repair. The third is a sibling-pair zero affirmation. Another GF arc of the same satellite that stayed chain-continuous and unflagged through the bridged interval affirms $\delta N = 0$ for both its signals, and any candidate that needs a nonzero integer on an affirmed signal is pruned. The wide-lane integer for the bridged solve is taken in order of trust: the published MW integer k; failing that, at the flag epoch, $\mathrm{round}(f) \pm 3$ around the instantaneous re-acquisition float (biased by the narrow-lane code); and on deferral epochs the settled across-gap MW Hatch mean. When the float wide-lane evidence is unsettled or the lattice is ambiguous (two or more acceptable integers), the solve is deferred instead of guessed, up to `estimation.cycle_slip.turbo_edit_wl_settle_max_s` (default 45 s). The deferral is available only with the soft-LLI bridge (`turbo_edit_bridge_lli`), whose holds keep the parked phase measurements out of the filter meanwhile. A deferral ends in a later repair, a reset or a timeout reset.

15.4.5 Atomic reconciliation

Per-satellite candidates are reconciled atomically. A cross-pair integer disagreement on a shared signal demotes the whole satellite to reset. A satellite that ends in the reset set has its code-event, moot, clean and pending classifications withdrawn, so contradictory evidence always resolves to the conservative outcome. Arc bookkeeping maintains the TD2 precondition of consecutive epochs. A non-consecutive update partially resets the chain but keeps the slowly varying MAD calibration window. A gap beyond the arc's maximum clears the arc. The callers set that maximum to 1.5 times the data interval; the resolver's own default, 120 s, applies only when no caller sets it. An accepted slip re-seeds the chain with a cooldown of 3 epochs that suppresses the TD2 echo. Only clean TD2 samples (not exceeding the threshold, not bridged) feed the MAD window.

15.4.6 Soft-LLI hold

A receiver-declared loss of lock (LLI) normally resets the satellite's phase ambiguities and erases the geometry-free arc. When the gap is short and the data rate is at least 1 Hz on a dual-frequency satellite that is not a BSSD pivot or GLONASS, the soft-LLI bridge (`estimation.cycle_slip.turbo_edit_bridge_lli`, default off; `include/slip/SoftLliBridge.hpp`) holds the ambiguities instead. It keeps a rolling snapshot $(\hat{x}_{\mathrm{snap}}, \sigma^2_{\mathrm{snap}})$ of the converged ambiguity values and variances, taken after each update and only for keys below a variance ceiling. On an LLI it keeps the

geometry-free arc, excludes the held phase measurements from the update across the one-epoch MW confirmation delay, and lets the gap bridge of Section 15.4.4 decide the integer. A unique solve overwrites the ambiguity as

$$\hat{x} \;=\; \hat{x}_{\mathrm{snap}} + \delta N\,\lambda_s, \qquad \sigma^2 \;=\; \max\!\big(\sigma^2_{\mathrm{snap}},\; 10^{-6}\big), \tag{253}$$

with its cross-covariances set to zero, which keeps the converged variance of the ambiguity itself, and releases the hold. Otherwise the ambiguity is reset, and each held signal has its own timeout from its own hold start. The repair assumes that the carrier loop held phase coherence across the LLI. If it re-acquired, δN absorbs the re-acquisition and the restored ambiguity is still the current integer, but a wrong unique solve puts at least $|\delta N\,\lambda_s|$ into the phase innovation when the signal rejoins, and the residual gate resets it one epoch late. Both the post-processing and the real-time paths share this helper.

15.4.7 Non-causal two-sided bridge (staged)

`SlipBridgeKernel` and `JumpGapFitter` (`include/slip/`) recover the integer slip across a flagged boundary from samples on *both* sides, which a causal detector cannot use. They are compiled and tested, no production code calls them, and the four settings that name them are parsed and read nowhere. The fitter fits an independent line $y = a + b\,(t - t_b)$ to each side of the boundary t_b and reports the step of the two extrapolated levels at t_b. Applied to the geometry-free series and the MW series it gives the steps ΔGF and ΔMW, and with the signals sorted high frequency first the attribution is the algebra of Section 15.4.1:

$$\delta N_{\mathrm{WL}} = \mathrm{round}\big(\Delta\mathrm{MW}/\lambda_{\mathrm{WL}}\big), \quad \delta N_2 = \mathrm{round}\left(\frac{\Delta\mathrm{GF} - \lambda_1\,\delta N_{\mathrm{WL}}}{\lambda_1 - \lambda_2}\right), \quad \delta N_1 = \delta N_2 + \delta N_{\mathrm{WL}}. \tag{254}$$

The result is accepted when each rounding residual is at most 0.30 cycle and the integers are not both zero and do not exceed 1000 cycles in magnitude.

15.5 Response on the user path

The user path applies a slip in the filter state through two primitives in `src/core/slip/CycleSlipRepair.cpp`, driven by `CycleSlipManager::detectAndRepair` (`CycleSlipDetection.cpp`, `CycleSlipAdjudication.cpp`, `CycleSlipPivot.cpp`, `CycleSlipNonMwPasses.cpp`).

- *Repair* adds the integer jump to the ambiguity state, $\hat{x} \leftarrow \hat{x} + \delta N\,\lambda_s$, and leaves its covariance unchanged. A deterministic state jump is recorded for the RTS smoother as a control input, so the backward pass does not absorb it as an innovation. A repair is refused for a BSSD pivot, whose single-difference ambiguity is frozen at zero; the pivot case is handled by a variance inflation that makes the pivot manager re-reference the frame.
- *Reset* re-seeds the ambiguity with a large variance. With between-satellite differencing the seed is the single difference of the two absolute ambiguity seeds; when the pivot is not observed the variance is inflated and the re-seed deferred to the next epoch. The smoother segments the satellite at this epoch.

Which signal slipped follows from the cross-consistency of overlapping MW pairs on a satellite with three or more frequencies. A signal that appears in a flagged pair and in no unflagged pair is a culprit. With $\delta N_{\mathrm{WL}} = \delta N_{s_1} - \delta N_{s_2}$, a lone culprit s_1 takes the integer δN_{WL} and a lone culprit s_2 takes $-\delta N_{\mathrm{WL}}$. The integers gathered for one culprit across its pairs must agree; otherwise the signal

is reset. When all pairs are flagged there is no clean reference, the attribution is inconclusive, and the TurboEdit resolution of Section 15.4 or a full reset applies. The auxiliary detectors (GRAPHIC, EWL) trigger resets of the signals they flag after the MW path has handled its satellites.

Part V
Integer ambiguity resolution

16 Integer ambiguity resolution

Integer ambiguity resolution (AR) maps the real-valued, bias-contaminated phase ambiguities of the float solution to the integer carrier-cycle counts they represent. The engine runs MLAMBDA [37] on a between-satellite-differenced ambiguity sub-problem extracted from the filter state and covariance. The operative quantity is the ambiguity differenced against a per-constellation pivot p_g (Section 5). The single-receiver and baseline modes share the integer-estimation machinery, but the condition that makes the differenced ambiguity an integer differs by mode.

The undifferenced ambiguity $N^i_{r,s}$ of the phase observation equation (Equation (5)) carries a receiver phase bias $b_{r,s}$ and a satellite phase bias b^i_s. In single-receiver PPP and PPP-RTK the pivot difference is a *between-satellite single difference* $\Delta\cdot$ (Section 5.1). It cancels the receiver bias $b_{r,s}$, which is common to every tracked satellite, and leaves the satellite bias. The residual $\Delta N^{i,p_g}_{r,s}$ is therefore integer only after b^i_s is removed by a consistent external phase-bias product (BIA or SSR, Section 18). In baseline mode the state holds true double differences $\nabla\Delta\cdot$ (Section 5.3); the additional between-receiver difference also cancels the satellite bias, so $\nabla\Delta N$ is integer without an external product. The equations below keep the resolver's internal label $\nabla\Delta$ for the differenced ambiguity, since the algebra is the same in both cases: it denotes the between-satellite single difference Δ in the single-receiver case and the full double difference only in baseline mode.

The implementation separates a driver from a numerical kernel. The driver, `src/core/filter/AmbiguityResolver.cpp`, assembles the state sub-vector, rescales between metres and cycles (metre↔cycle), looks up the per-pair λ, forms the differences, performs the WL/NL decomposition and applies the returned integer fix. The numerical core lives in the `GnssProcessor::ils` namespace of `src/core/filter/IlsCore.{hpp,cpp}`: the LDL^{T} factorisation (in the factor order of (259)), the $\boldsymbol{Z}$-reduction, the Schnorr–Euchner search, the bootstrap success rate and the full acceptance battery. These are pure free functions over ($\hat{\boldsymbol{N}}$ cycles, $\boldsymbol{Q}_{\hat{N}}$ cycles^2), with no dependence on `StateManager`, `KalmanFilter` or the configuration. The PPP-RTK network (Section 20) calls the same `ils` core under its own gate set, as a read-only search and a measure-only held solve, because the emitted product is float (Section 16.7). The single-receiver and baseline paths below are configured under the JSON path `estimation.ambiguity_resolution.*`.

16.1 The integer least-squares problem

The float solution gives a real-valued ambiguity estimate $\hat{\boldsymbol{N}}$ (in cycles) with covariance $\boldsymbol{Q}_{\hat{N}}$, both extracted from the filter state $\boldsymbol{x}$ and covariance $\boldsymbol{P}$. Integer ambiguity resolution is the weighted closest-lattice-point problem

$$\boxed{\check{\boldsymbol{N}} \;=\; \arg\min_{\boldsymbol{a}\in\mathbb{Z}^n} \big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big)^{\mathrm{T}}\, \boldsymbol{Q}_{\hat{N}}^{-1}\, \big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big),} \tag{255}$$

that is, the minimisation of the squared Mahalanobis distance from $\hat{\boldsymbol{N}}$ to a lattice point $\boldsymbol{a}$

in the metric of $\boldsymbol{Q}_{\hat{N}}^{-1}$. This integer least-squares (ILS) problem is NP-hard. LAMBDA and its MLAMBDA variant make it tractable in three stages, none of which has a tuning parameter: (i) an LDL^{T} factorisation of $\boldsymbol{Q}_{\hat{N}}$, (ii) a decorrelating integer $\boldsymbol{Z}$-transform (lattice reduction), and (iii) a depth-first shrinking-ellipsoid search over the reduced lattice. The hat/check convention of the notation block applies: $\hat{\boldsymbol{N}}$ is the float estimate and $\check{\boldsymbol{N}}$ the integer-fixed solution.

Float extraction and rescaling to cycles. The filter stores per-band ambiguity states in metres ($\lambda_s N$, one per tracked satellite and signal). The resolver therefore forms the between-satellite-differenced float and covariance by explicit differencing against the pivot p_g, then rescales by wavelength. In baseline mode the state already holds the double difference, and the differencing step reduces to a direct selection (Section 16.6). For a differenced pair k with satellite index $i(k)$ and pivot index $p(k)$ (`src/core/filter/AmbiguityResolver.cpp`):

$$y_k^{\nabla\Delta} = \boldsymbol{x}_{i(k)} - \boldsymbol{x}_{p(k)}, \qquad \text{(DD float, m)} \tag{256}$$

$$\boldsymbol{Q}_{kl}^{\nabla\Delta} = \boldsymbol{P}_{i_k i_l} - \boldsymbol{P}_{i_k p_l} - \boldsymbol{P}_{p_k i_l} + \boldsymbol{P}_{p_k p_l}, \qquad \text{(DD covariance, m}^2\text{)} \tag{257}$$

$$\hat{\boldsymbol{N}}_k = y_k^{\nabla\Delta}/\lambda_k, \qquad (\boldsymbol{Q}_{\hat{N}})_{kl} = \boldsymbol{Q}_{kl}^{\nabla\Delta}/(\lambda_k\lambda_l). \qquad \text{(cycle domain)} \tag{258}$$

The rescaling (258) is required. On metre-domain floats MLAMBDA would round toward integer-metre values, which have no carrier-cycle meaning, and the ratio test (Section 16.5) would return ≈ 1 irrespective of float quality. The per-pair wavelength λ_k uses the GLONASS FDMA per-satellite frequency where applicable. $\boldsymbol{Q}^{\nabla\Delta}$ is symmetrised, $\boldsymbol{Q}^{\nabla\Delta} \leftarrow \frac{1}{2}(\boldsymbol{Q}^{\nabla\Delta} + \boldsymbol{Q}^{\nabla\Delta\mathrm{T}})$, before factorisation.

The resolver elects its own DD reference: in each (constellation, signal) group of AR-eligible ambiguities, the highest-elevation member is the reference and every other member is differenced against it. This reference differs from the filter's BSSD pivot. Under BSSD the pivot's ambiguity is frozen at zero variance, which excludes it from the candidate set (Section 16.4); the resolver therefore differences against a second reference, and the resulting quantity is a difference of single differences. The two elections also use different rules. The BSSD pivot is chosen on a hard phase-coverage tier, a convergence/elevation score and an ephemeris-freshness bar (Section 5); the AR reference is chosen on elevation alone (`AmbiguityResolver::buildDdPairs`).

16.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform

Factorisation. MLAMBDA factorises the symmetric positive-definite ambiguity covariance as

$$\boldsymbol{Q}_{\hat{N}} \;=\; \boldsymbol{L}^{\mathrm{T}}\,\boldsymbol{D}\,\boldsymbol{L}, \qquad \boldsymbol{L} \text{ unit lower-triangular}, \;\; \boldsymbol{D} = \mathrm{diag}(d_1,\ldots,d_n), \;\; d_i > 0, \tag{259}$$

eliminated in reverse index order, from the last row up (the Chang–Yang MLAMBDA convention). The transposed unit lower-triangular factor stands on the left, as written in (259); the shorthand "LDL-transpose" used in this section names that factorisation, not the reversed factor order. A non-positive pivot $d_i \le 0$ signals a covariance that is not positive definite and aborts the fix. After reduction, d_i is the *conditional variance* of the i-th decorrelated ambiguity given the later ones; the success-rate gate of Section 16.5 uses it directly.

The $\boldsymbol{Z}$-transform. The central device of LAMBDA [37] is a unimodular integer transform $\boldsymbol{Z} \in \mathbb{Z}^{n\times n}$, $|\det \boldsymbol{Z}| = 1$, that re-parameterises

$$\boldsymbol{z} = \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}, \qquad \boldsymbol{Q}_{\hat{z}} = \boldsymbol{Z}^{\mathrm{T}}\,\boldsymbol{Q}_{\hat{N}}\,\boldsymbol{Z}, \tag{260}$$

so that the transformed ambiguities are nearly decorrelated (small off-diagonal $\boldsymbol{L}$ entries) and their conditional variances are ordered. The decorrelated float vector is $\bar{\boldsymbol{z}} := \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}$, with components $\bar{z}_k$; it is the quantity the search of Section 16.2 rounds, computed as `z_float = Z.transpose()*a` in `ils::mlambda` (`src/core/filter/IlsCore.cpp`). Because $\boldsymbol{Z}$ is unimodular, $(\boldsymbol{Z}^{\mathrm{T}})^{-1}$ is also integer and the integer minimiser maps back exactly, $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$. Decorrelation therefore leaves the solution of (255) unchanged. It makes the search ellipsoid closer to spherical, which reduces the number of enumerated points by orders of magnitude. The reduction interleaves two operations in a sweep from $j = n-2$ downward.

Integer Gauss transformations reduce the sub-diagonal $\boldsymbol{L}$ entries by integer column operations,

$$\mu = \lceil L_{ij} \rfloor, \qquad \boldsymbol{L}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{L}_{(:,i)}, \qquad \boldsymbol{Z}_{(:,j)} \mathrel{-}= \mu\, \boldsymbol{Z}_{(:,i)}, \tag{261}$$

where $\lceil \cdot \rfloor$ is round-to-nearest, keeping every $|L_{ij}| \le \frac{1}{2}$.

Permutations (conditional adjacent swaps) reorder two ambiguities when the swap reduces the later conditional variance. With $\bar{d}_j = d_j + L_{j+1,j}^2\, d_{j+1}$, the swap fires when

$$\bar{d}_j + \varepsilon \;<\; d_{j+1}, \qquad \varepsilon = 10^{-6}, \tag{262}$$

after which d_j, d_{j+1}, the affected rows/columns of $\boldsymbol{L}$ and the columns of $\boldsymbol{Z}$ are updated by the standard rank-one formulas ($\eta = d_j/\bar{d}_j$, $\lambda = d_{j+1}L_{j+1,j}/\bar{d}_j$, $d_j \leftarrow \eta\, d_{j+1}$, $d_{j+1} \leftarrow \bar{d}_j$), and the sweep restarts from $j = n-2$. The guard $\varepsilon = 10^{-6}$ prevents repeated swaps of near-equal variances. It is a numerical safeguard rather than an estimator setting and has no configuration key.

The shrinking-ellipsoid search. The reduced problem is solved by depth-first enumeration of the lattice points inside the ellipsoid $\sum_i (z_i - \hat{z}_i)^2/d_i \le \chi^2$, in Schnorr–Euchner order (nearest integer first, then zig-zagging outward). Recursing from the last level $k = n-1$ down to 0,

$$\hat{z}_k = \bar{z}_k + \sum_{i>k} (z_i - \hat{z}_i)\, L_{ik}, \qquad \text{(conditional centre)} \tag{263}$$

$$z_k = \lceil \hat{z}_k \rfloor, \qquad \mathrm{dist}_k = \mathrm{dist}_{k+1} + \frac{(\hat{z}_k - z_k)^2}{d_k}. \qquad \text{(partial distance)} \tag{264}$$

A complete integer vector that reaches level 0 with cumulative distance below the current bound χ^2 is recorded. Once the candidate list is full, the bound shrinks to the worst retained candidate, so the ellipsoid contracts as better candidates are found. The search (`ils::search` in `src/core/filter/IlsCore.cpp`) retains the $m = 2$ best candidates for the ratio test, sorted by ascending squared distance, under a hard iteration cap (`kMaxIter` = 10 000). Fewer than two candidates is a failure. The outputs are the two smallest objective values $s_1 \le s_2$ of (255) and the best integer vector $\check{\boldsymbol{z}}$, back-transformed via $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$.

16.3 Wide-lane / narrow-lane decomposition (PPP path)

PPP and PPP-RTK AR use a *wide-lane-first* decomposition when Melbourne–Wübbena (MW) resolved wide-lane (WL) integers are available. It halves the search dimension and exploits the long WL wavelength [19]. The driver attempts the WL-constrained narrow-lane (NL) fix before any unconstrained LAMBDA. When WL integers exist and the constrained fix fails, the epoch stays float: the driver does not fall through to an unconstrained search, because that search would include DDs whose WL is not locked and could return integers that contradict the MW wide lane. A wrong-WL fix is worse than no fix, so the enforced rule is "no NL AR for satellites whose WL is float" (`src/core/filter/AmbiguityResolver.cpp`).

WL from Melbourne–Wübbena (integer by contract). The DD wide-lane integer is not searched. It is the difference of the MW detector's single-difference WL integers (`src/core/slip/MwSlipDetectorSample.cpp`; the lock criterion is in Section 15.2.1):

$$\nabla\Delta N_{\mathrm{WL}} = N^{i}_{\mathrm{WL}} - N^{p_g}_{\mathrm{WL}} \in \mathbb{Z}, \qquad \lambda_{\mathrm{WL}} = \frac{c}{f_1 - f_2}. \tag{265}$$

NL search variable. For each dual-frequency DD pair the transform $\boldsymbol{T}$ forms the float narrow-lane ambiguity in NL cycles. The default row is the narrow-lane combination below. It is not ionosphere-free: its slant-ionosphere sensitivity is 0.959 NL cycles per TECU, so the default per-pair cap (271) of 0.15 cycles implicitly requires the slant ionosphere to about 0.156 TECU.

$$\hat{n}_{\mathrm{NL}} = \frac{f_1\,\nabla\Delta N_1 + f_2\,\nabla\Delta N_2}{f_1 + f_2} = \frac{1}{f_1 + f_2}\left(\frac{f_1}{\lambda_1}\,\nabla\Delta_1 + \frac{f_2}{\lambda_2}\,\nabla\Delta_2\right), \qquad \lambda_{\mathrm{NL}} = \frac{c}{f_1 + f_2}, \tag{266}$$

with reduced covariance $\boldsymbol{Q}_{\mathrm{NL}} = \boldsymbol{T}\,\boldsymbol{Q}^{\nabla\Delta}\,\boldsymbol{T}^{\mathrm{T}}$. Only NL-paired DDs enter LAMBDA; unpaired single-signal DDs are excluded so that they cannot distort the ratio test. An optional ionosphere-free search row (configuration key nl_iono_free_row, default off) targets the same L1 integer with zero slant-ionosphere sensitivity. For GPS L1/L2 it multiplies the WL sensitivity by 8.06 (a 1-cycle WL error moves the L1 integer target by 3.53 cycles instead of 0.438) and the searched variance by about 65. The equations of this subsection describe the default narrow-lane row.

Integer-recentring shift. The float $\hat{n}_{\mathrm{NL}}$ is not integer-valued in general. Substituting $N_2 = N_1 - \mathrm{WL}$ gives $\hat{n}_{\mathrm{NL}} = N_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$, which for GPS L1/L2 with $\mathrm{WL} = 1$ lies at $N_1 - 0.4379$. Neither neighbouring integer is then materially closer, and the ratio test settles near 1.6 regardless of float quality. The resolver therefore adds the known-WL offset, which makes the search variable integer in expectation,

$$\tilde{n}_k = \hat{n}_{\mathrm{NL},k} + \frac{f_2}{f_1 + f_2}\,\nabla\Delta N_{\mathrm{WL},k} \;\approx\; N_{1,k} \;\in\; \mathbb{Z} + \text{noise}, \tag{267}$$

runs MLAMBDA on $\tilde{\boldsymbol{n}}$ (a constant per-row shift leaves $\boldsymbol{Q}_{\mathrm{NL}}$ unchanged), then back-shifts $\check{n}_{\mathrm{NL}} = \check{N}_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$.

Back-transform to per-signal DDs. The fixed integers map back to the two per-signal DDs in metres,

$$\nabla\Delta_1^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} + \tfrac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_1, \qquad \nabla\Delta_2^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} - \tfrac{f_1}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_2, \tag{268}$$

and re-enter the filter through the constrained update of Section 16.6.

State-basis variants. When `estimation.state_ambiguity_basis = nl_wl` the state holds NL/WL ambiguities directly, and there are no per-band DD pairs to enumerate. In this basis only the MW-resolved WL integers are applied, as constraint rows; a full NL LAMBDA search with the WL pre-constrained is not implemented. The complementary WL-only mode (`estimation.ambiguity_resolution.wl_only`, in the `per_band` basis) applies the cycle-domain constraint $\nabla\Delta_1/\lambda_1 - \nabla\Delta_2/\lambda_2 = \nabla\Delta N_{\mathrm{WL}}$ as one pseudo-observation per pair, without an NL search. When MW re-locks at a new integer, the covariance of the affected single-difference states is inflated so that the state moves to the new integer within one epoch.

Both WL-constraint paths (this one and the `nl_wl`-basis constraint) take their pseudo-observation noise from a separate setting, $\boldsymbol{R} = \sigma^2_{\text{wl}}\,\boldsymbol{I}$ with $\sigma^2_{\text{wl}} =$ `wl_only_pseudo_var` (10^{-4} cycles^2, in the cycle domain), rather than the metre-domain σ^2_{hold} of the NL/DD feedback in (273). Both paths apply three acceptance gates to the MW integers. The first is the per-arc fractional-residual point test (`wl_max_frac`, default 0.40; on both arcs in the `per_band wl_only` path, on the single arc in the NL/WL basis). The second is a bootstrap success-rate gate (key wl_only_ffr, default on): the product of per-arc rounding success rates, taken over distinct arcs, must reach the threshold min_success_rate of (270), and the weakest arc is dropped until it does. The third is an innovation screen (key wl_only_max_innov_cyc, default 0.5 cycle) that rejects a constraint whose asserted WL integer differs from the predicted DD wide lane by more than that amount. The cross-frequency parity check runs only in the `nl_wl` basis (`applyMwWlConstraintsNlWl_` in `src/core/filter/AmbiguityResolver.cpp`). For a triple-frequency satellite it rejects all three WL DDs when the telescoping canonical cycle counts are inconsistent, $\nabla\Delta_{ab} - \nabla\Delta_{ac} + \nabla\Delta_{bc} \neq 0$. The `per_band wl_only` path has no cross-frequency consistency check.

16.4 Partial / subset ambiguity resolution

Partial AR operates at two levels: a static candidate gate that decides which ambiguities enter the integer problem, and an optional backward-elimination search that looks for a fixable subset when the full set fails validation.

Upstream candidate gating (always on). Subset selection begins when the DD pairs are built (`src/core/filter/AmbiguityResolver.cpp`). A satellite/signal ambiguity enters the integer set only if it passes every filter:

- **Initialised**: the state entry is non-zero (seeded by data), and **non-frozen**: its variance is non-zero, which excludes a BSSD-frozen pivot.
- **Elevation**: $\theta^i_r \geq \theta_{\text{AR}}$ (`elevation_mask`). In the single-receiver path the elevation comes from the epoch's observations; an ambiguity whose satellite has no observation is given elevation 0 and is removed by any positive mask (the consequence for fix-and-hold is stated in Section 16.6).
- **Resolvable**: the ambiguity is not flagged unresolvable for lack of bias or antenna (BIA, ATX) coverage. This applies to the single-receiver path; in baseline DD the analogous filter is the `exclude_keys` set of non-integer-capable DDs from `MeasurementModel`.
- **Constellation**: $g(i) \in$ `active_systems`, when that list is non-empty.
- **Pairable**: the (constellation, signal) group holds ≥ 2 satellites, so a pivot difference exists.
- **Mature** (`min_arc_info`, default 0 = off): both constituent arcs of a candidate (the higher-frequency arcs of the satellite and of the reference) have accumulated at least $\sum_k \sin^2\theta^i_{r,k}$ of elevation-weighted code information since arc birth, in epoch-equivalents at zenith. Maturity is measured in information rather than epochs: under the elevation stochastic model an arc rising off the mask and one at zenith accumulate information at very different rates, so two arcs of equal age are not equally determined. The counter restarts on both ways an arc can end: the key leaving the state vector, and a refloat in place. A refloat keeps the key and would otherwise let the new arc inherit the pre-slip age (`AmbiguityResolver::refreshArcAges_`, keyed on `StateManager::isAmbiguityPendingReseed`). The gate applies only to the WL-constrained NL candidate set of Section 16.3, not to the baseline DD, WL-only or unconstrained full-set paths. It guards against wrong fixes, since one immature candidate can fail the joint ratio test of (269) for the whole set.

It does not speed convergence: it admits a candidate only once the candidate is well determined, which is when the float solution gains least from a fix.

The constellation filter keeps a system in the float solution while excluding it from the candidate set. It serves a constellation whose residual model bias would lower the ratio test.

Dynamic backward-elimination subset (`enable_partial_ar`, default off). With the candidate set fixed, the default is all-or-nothing: the full set is searched and either passes all gates of Section 16.5 or the epoch stays float. When `enable_partial_ar` is set, a failure of the full-set battery starts a greedy backward elimination (`ils::selectFixableSubset` in `src/core/filter/IlsCore.cpp`, called through the driver-side `selectFixableSubset_` configuration adapter) before the epoch falls back to float. A failure of MLAMBDA itself on the full set (a covariance that is not positive definite, or fewer than two candidates) does not start the elimination. Starting from the full candidate set, each iteration re-runs the complete MLAMBDA pipeline of Sections 16.1 to 16.2 on the current subset and re-tests the same acceptance battery: the ratio lower and upper bounds (269), the bootstrap success rate (270), and the per-entry fractional cap (271). On any failure it drops one entry and retries, down to a single ambiguity:

- if the *fractional cap* is the failing gate, the entry with the largest $|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k|$ (nearest a half-cycle boundary) is removed;
- otherwise (including a failure of MLAMBDA on the subset) the entry with the largest formal float variance $(\boldsymbol{Q}_{\hat{N}})_{kk}$ is removed: the least precise ambiguity, whose removal is expected to improve the bootstrap rate and the search conditioning most.

The accepted subset is always re-validated by the full battery on its own covariance sub-block. The procedure can therefore only fix fewer ambiguities than the full set, never accept a set that fails the battery: a poor elimination order costs fixes, not integrity. With the option off, the path is bit-identical to the all-or-nothing behaviour. The same routine serves the per-band SD, baseline DD and WL-constrained NL paths and operates only in the cycle domain, independent of the decorrelation. Each caller maps the surviving indices back to its own state rows and applies them through the constrained update of Section 16.6.

16.5 Validation: ratio test and fixed-failure-rate gate

A fix is committed only if it passes all three of the following gates. The code evaluates them in order (ratio lower bound, ratio upper bound, bootstrap success rate, per-pair cap) and stops at the first failure. The ratio and fixed-failure-rate (FFR) gates bound different things: the ratio bounds the discrimination between the two best candidates, the FFR gate the absolute success probability.

Ratio test (discrimination).

$$\boxed{\text{ratio} = \frac{s_2}{s_1} \;\geq\; \tau_{\text{ratio}}, \qquad s_1 \leq s_2 \text{ the two smallest ILS objectives,}} \tag{269}$$

with s_1, s_2 from the search of Section 16.2 and threshold τ_{ratio} (`ratio_threshold`). An upper-bound rejection, ratio $> \tau_{\text{ratio}}^{\max}$ (`ratio_upper_bound`), catches a degenerate covariance direction: a near-zero s_1, typically left by an earlier wrong-integer hold, produces a very large but meaningless ratio. A non-positive squared distance for either candidate rejects the fix before the ratio is formed.

Fixed-failure-rate gate (bootstrap success rate). The gate uses a lower bound on the ILS success probability that follows from the problem geometry,

$$\boxed{P_{s,\mathrm{boot}} = \prod_{i=1}^{n} \operatorname{erf}\!\left(\frac{1}{2\sqrt{2\,d_i}}\right) \;\geq\; P_s^{\min},} \tag{270}$$

where the d_i are the post-reduction LDL^{T} conditional variances of (259) and $P_s^{\min}$ is `min_success_rate`. Each factor is $\Pr\!\big(|\mathcal{N}(0,d_i)| < \tfrac{1}{2}\big)$, the probability that sequential bootstrapped rounding lands on the correct integer in that dimension. The product is Teunissen's lower bound on the ILS success rate. The acceptance threshold is therefore tied to the problem geometry rather than to a fixed ratio alone.

Per-pair residual backstop. The joint ratio test can pass on a borderline set in which one pair lies near a half-cycle boundary. The fix is therefore rejected if the fractional residual of any single DD exceeds a cap,

$$\max_k \big|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k\big| \;\leq\; \kappa_{\mathrm{frac}} \quad \text{(cycles)}, \tag{271}$$

with $\kappa_{\mathrm{frac}} =$ `nl_max_per_pair_frac`.

Convergence guards. Before any search the resolver skips AR while the float solution is immature: during its first `min_epochs` invocations (a global counter of resolver calls, not a per-candidate age), and while the position standard deviation $\sqrt{\boldsymbol{P}_{00}+\boldsymbol{P}_{11}+\boldsymbol{P}_{22}}$ exceeds `max_position_sigma_m`. A marginal fix accepted before the covariance settles can lock onto wrong integers and corrupt every later epoch.

16.6 Feedback: constrained pseudo-observation update

Accepted integers are fed back through a finite-variance pseudo-observation rather than the classical perfect back-substitution (the $R = 0$ conditioning form)

$$\boldsymbol{x}_{\mathrm{fix}} = \boldsymbol{x} - \boldsymbol{Q}_{\boldsymbol{aa}}\,\boldsymbol{Q}^{\nabla\Delta-1}\big(y^{\nabla\Delta} - \check{\boldsymbol{N}}\big), \qquad \boldsymbol{P}_{\mathrm{fix}} = \boldsymbol{P} - \boldsymbol{Q}_{\boldsymbol{aa}}\,\boldsymbol{Q}^{\nabla\Delta-1}\boldsymbol{Q}_{\boldsymbol{aa}}^{\mathrm{T}}, \tag{272}$$

The update is a *Joseph-form pseudo-observation* (`src/core/filter/AmbiguityResolver.cpp`; kernel `KalmanFilter::josephPseudoUpdate` in `include/filter/KalmanFilter.hpp`). The reason is structural. (272) drives the fixed ambiguity rows of $\boldsymbol{P}$ to ≈ 0 variance, so the next epoch's LAMBDA sees a near-singular $\boldsymbol{Q}^{\nabla\Delta}$ and returns large, meaningless ratios that keep tripping the upper-bound guard of (269). A finite-variance pseudo-observation is intended to keep $\boldsymbol{P}$ well-conditioned, so that later epochs can re-confirm the integer (converging toward it) or detect a slip through its residual.

This intent is not yet validated. Reducing σ^2_{hold} (over the range 1e-3 to 1e-7, in square metres) reduced the fix count and made the kinematic solution diverge, which is the signature of a constraint that drives the covariance singular rather than tightening it. The resolver therefore carries a read-only conditioning probe (`qbCondition` / `rebuildQbCycles` in `src/core/filter/AmbiguityResolver.cpp`) that eigen-decomposes the cycle-domain DD covariance before every search and after every fix, and logs the conditioning on every fix and every rejection. The mechanism described below is the design intent, not a validated result.

For each accepted DD pair k the constraint row, innovation and noise are

$$\boxed{\boldsymbol{H}_{(k,\,i)} = +1, \quad \boldsymbol{H}_{(k,\,p)} = -1, \qquad v_k = \lambda_k\,\check{\boldsymbol{N}}_k - \big(\boldsymbol{x}_i - \boldsymbol{x}_p\big), \qquad \boldsymbol{R} = \sigma^2_{\mathrm{hold}}\,\boldsymbol{I},} \tag{273}$$

followed by $(\boldsymbol{x}_{\text{fix}}, \boldsymbol{P}_{\text{fix}}) = \textsc{JosephUpdate}(\boldsymbol{x}, \boldsymbol{P}, \boldsymbol{H}, \boldsymbol{v}, \boldsymbol{R})$, with $\sigma^2_{\text{hold}} =$ `hold_variance`. The DD/baseline path uses a single-column selector $\boldsymbol{H}_{(k,\,\text{idx})} = +1$, because the state value already is the double difference. The same update applies a partial-AR subset, with fewer rows. The updated state and covariance replace the filter's on every epoch with an accepted fix, in every mode other than 0. The resolver does not distinguish modes 1 (continuous) and 2 (instantaneous); only mode 0 (float) and mode 3 (fix-and-hold) change its behaviour. The $R = 0$ form (272) is not implemented; it appears in the code only in comments, as the rejected alternative.

Fix-and-hold mode (`estimation.ambiguity_resolution.mode` = 3) applies a second pseudo-observation update on each epoch with an accepted fix, holding the differenced ambiguities at their fixed values with the same hold variance. In the baseline path (`AmbiguityResolver::holdAmbiguitiesDd`) the hold covers every DD-keyed ambiguity in the state, including keys excluded from the fix. In the single-receiver path (`AmbiguityResolver::holdAmbiguities`, called by both the post-processing and the real-time engines) the hold covers the single-signal DD ambiguities that the accepted fix constrained (`AmbiguityResolution::fixed_dd_rows`), so the epoch's candidate set, pivots and elevation mask carry into it. A candidate the fix left out (below the mask, outside a partial subset, without a widelane lock) stays float, and a widelane-only fix, which constrains no single-signal DD, holds nothing. The callers install the fixed state before the hold, so the hold's innovation is zero: it tightens the covariance of the fixed pairs a second time and moves no mean. Under the NL/WL state basis the hold is a no-op by design.

16.7 Network integer machinery (all default off)

Status. Network-side ambiguity resolution is not built for the emitted product. The integer search makes no wrong fix at 300 stations (the sticky cycle basis of Section 16.7.1 settles 45 026 and 365 566 integers at 60 and 300 stations on the simulated network, none wrong: `docs/refound/AGTB_arc_graph_continued_2026-10-10.md`). Holding the fixes then failed: a held solve of the shared parameters neither reaches nor helps the user on the reference configuration, and a slip that no detector flags leaves the zero-process-noise float on the wrong integer, which the held solve cannot see (`docs/refound/FCFC_fixed_cycle_ledger_2026-10-11.md`, section 9). Every mechanism below is therefore off in the loader. The search, the ledger and the fold are measure-only instruments, the SD extra-wide-lane rows act on the solve only when armed, and the emitted network product is float. The mathematics is settled, and the instruments are kept to grade the float.

Probe: a log-only search. Under `network.ambiguity_resolution.enable` (default `false`) the seam `network/ar/ProducerAr` (`src/core/network/ar/ProducerAr.cpp`) reconstructs each station's float ambiguities from the reduced solution (`NetworkEstimator::reconstructStation`, in `network/solve/Estimator.hpp`) and forms an independent between-station double-difference basis per (constellation, signal), restricted to GPS and Galileo because GLONASS FDMA leaves a per-slot receiver term that survives the difference. It builds the exact DD covariance from `NetworkEstimator::jointConditionalCovariance`, rescales it to cycles by (258), and calls `ils::ilsFix` (or the decorrelated partial fix below) under a gate set stricter than the user path: bootstrap success rate at least 0.9999 and per-entry residual at most 0.10 cycle, because a wrong network fix would reach every user. The result is logged and discarded: nothing is re-pinned or re-solved, and the run is byte-identical whether the block executes or not. The probe is meaningful because the between-station DD ambiguity

$$\nabla\Delta N^{i,p_g}_{r_1 r_2,s} \;=\; \Delta N^{i,p_g}_{r_1,s} - \Delta N^{i,p_g}_{r_2,s} \tag{274}$$

is invariant under the network's choice of phase-bias S-basis: the satellite bias b_s^i cancels in the between-receiver difference exactly as it does in the baseline DD of Section 5.3. The probe therefore reports the network's intrinsic integer recoverability rather than a property of the datum it holds. Its convergence gauge is the median DD formal standard deviation in cycles. That value must fall well below the half-cycle scale before (255) is reachable, and it is a better progress indicator than the fix count. `ils_max_dim` (default 0, uncapped) restricts the search to the lowest-formal-σ subset in the manner of Section 16.4, because the full search does not return on a full-network star basis at operational station counts.

Decorrelated partial fix. With a free slant ionosphere each uncombined L1 or L2 DD is imprecise (about 0.3 cycle), while combinations of a pair (the wide lane, the ionosphere-free part) are precise. A subset chosen entry by entry (Section 16.4) then finds nothing, and the per-entry residual cap rejects correct fixes. `ils::ilsFixDecorrelatedPartial` (`src/core/filter/IlsCore.cpp`), armed by `network.ambiguity_resolution.decorrelated_partial`, works in the decorrelated space of Section 16.2 instead.

1. With $\boldsymbol{Q}_{\hat{N}} = \boldsymbol{L}^{\mathrm{T}}\boldsymbol{D}\boldsymbol{L}$ and $\boldsymbol{z} = \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}$ the search fixes $\boldsymbol{z}$ from its last component, so the trailing components are fixed first.
2. The largest trailing set whose bootstrap rate (270) is at least $P_s^{\min}$ is searched, on the trailing block of the factors. If its ratio test fails the set shrinks by one component and is searched again.
3. The floats are conditioned on the fixed components $\check{\boldsymbol{z}}_F$,

$$\check{\boldsymbol{a}} = \hat{\boldsymbol{N}} - \boldsymbol{Q}_{aF}\boldsymbol{Q}_{FF}^{-1}\big(\hat{\boldsymbol{z}}_F - \check{\boldsymbol{z}}_F\big), \qquad \boldsymbol{Q}_{\check{a}} = \boldsymbol{Q}_{\hat{N}} - \boldsymbol{Q}_{aF}\boldsymbol{Q}_{FF}^{-1}\boldsymbol{Q}_{Fa}, \tag{275}$$

   with $\boldsymbol{Q}_{aF}$ the covariance of the floats with $\hat{\boldsymbol{z}}_F$.
4. An input entry is reported fixed when its conditional standard deviation is at most `decorrelated_determined_sigma_cyc` (0.05 cycle) and its conditional value lies within the per-entry residual cap of an integer: the fixed combinations then determine that single-signal ambiguity.

A caller forms any combination of the conditioned values (the wide lane and the narrow lane of an L1/L2 pair) from $\check{\boldsymbol{a}}$ and $\boldsymbol{Q}_{\check{a}}$. With `block_search`, the search rows that share one baseline (its L1 and L2 DDs) are fixed as one block on their marginal covariance, each block with its own bootstrap rate and ratio test (`netar::blockDecorrelatedFix`). One weak baseline no longer refuses the whole network, and the cost is the sum of small blocks instead of one large search. The merged result keeps the smallest accepted block ratio and the product of the accepted blocks' bootstrap rates, a bound for the epoch.

16.7.1 The observation graph and its bases

The search needs a set of independent double differences. Their number and algebra are properties of the observation graph, a bipartite graph whose vertices are the stations and the satellites of one constellation and whose *arcs* are the (station, satellite) pairs with a live ambiguity in a band. The cycle space of that graph and its use for the carried double differences are described in Section 22.3; this subsection adds what the integer search uses. One owner, `network/ar/ArcGraph`, serves the chain datum, the invariant carry, the search and the fixed-cycle ledger, as plain value types and free functions with no state.

An arc set is a family: one constellation with one band, or an L1/L2 pair that shares one graph. A band-b lattice has dimension $E - V + C$ over the arcs present in that band, with E arcs, V vertices and C connected components. Three bases for the search exist, in increasing coverage.

- The *star* basis (the default), per (constellation, signal), takes a reference satellite (tracked at the most stations) and a reference station (tracking it and the most satellites), and forms one DD for each other station and satellite that both that station and the reference station track.
- `tree_basis` grows a spanning forest of station baselines per (constellation, signal) by a Prim rule on satellite bitmasks (`growBaselineForest`). The root is the incumbent root or the station with the most arcs. Each outside station joins the in-tree station that shares the most satellites with it, at least two, through its first common satellite as reference, and the growth stops when no station reaches that count. The DD of a satellite j on the baseline (r, p) with reference k is

$$\nabla\Delta N_{rp}^{jk} = (N_{rj} - N_{rk}) - (N_{pj} - N_{pk}). \tag{276}$$

  Only the chords whose satellite the link parent tracks are closed.
- `cycle_basis` (needing `tree_basis`, `decorrelated_partial` and `block_search`) closes every chord of every band. The growth finishes through one common satellite when two are not available, and the reference satellite of a link is the most durable one. It is sticky: the forest is seeded with the last search's links, a station whose link waits for its parent rejoins after that parent, and a chord that was closed by a borrowed DD keeps its partner and reference while they still close it, so the DD identities outlive an epoch as satellites rise and set. A chord that no DD can close is a long fundamental cycle, counted and never searched.

A basis cycle owns one chord that no earlier cycle of its band contains (a *unit-triangular* basis). The cycles are therefore independent, and the integer coordinates of any cycle in the basis follow by back-substitution on the pivots, exactly (`cycleCoordinates`). Between two epochs, `basisChangeMap` gives each new cycle as an integer combination of stored cycles; the matrix is unimodular on an unchanged graph. A new cycle is *fresh* when it touches an arc whose integer may have changed (a slip or a new life of the arc) or leaves the stored span. This is the same exactness that makes the carried double differences safe across a change of tree.

Arc life and the suspect-corner mask. An arc keeps one integer for as long as nothing breaks its phase or resets its float. The life stamp of an arc (`network/ar/ArcLife`) reads the events that end a life from their owners: a birth or a re-entry after a gap in the arc ledger, an erase, a drop, a re-level or a screen flag on the leveling carry. Equal stamps mean a continuous integer. The search reads the stamp once per epoch, and counts the echo that an event leaves one search later as part of the same life.

A slip that no pre-fit detector caught leaves the float variance, the bootstrap rate and the ratio where a clean arc has them, so every gate passes and the search fixes a wrong integer. The slip layer holds evidence about such arcs, from the slip epoch or a few epochs later. The suspect-corner mask (`network.ambiguity_resolution.suspect_epochs`, off at 0; `network/ar/ArSuspect`) keeps per (station, satellite) corner the epoch of the last event of each cause: an MW flag dropped for an unknown integer, an arc geometry-free jump flag (Section 15.3.2), a drop or re-level of the leveling carry, a free-epoch latch, and a RAIM attribution drop. A stamp masks the searches $e, e+1, \ldots, e+W-1$ with W the setting. A DD is masked when any of its four corners is suspect. It stays in the basis and the trace, and the search does not see it. The release is by time only and is counted, not judged.

16.7.2 Ledger of fixed cycles

A fixed DD is an integer combination of band-tagged arcs. `arcgraph::CycleLattice` keeps the fixed cycles in reduced echelon form: row k owns a pivot arc at coefficient $+1$ that no other row

contains. The lattice is then *saturated*: an integer cycle in the rational span of the rows has integer coordinates (its values at the pivots), so the integer it implies is exact. Offering a fixed cycle with its integer reduces it by the pivots and gives one of four verdicts: *added* (a new direction), *confirmed* (in the span, with the implied integer), *contradicted* (in the span, with another integer: a wrong fix or a missed release) or *not primitive* (refused). The ledger (`network/ar/ArLattice`, armed by `trace_lattice` or by the fold at source `lattice`) admits a fix only after it has kept one integer for `fixed_cycles.persist_ep` epochs (10) with no new life on its arcs.

A verdict on a cycle puts its integer in question without saying which of its arcs moved, so the ledger *releases the arcs* of the cycle, eliminating each of them from the lattice. Dropping only the rows behind the cycle is not enough: in reduced echelon form other rows can hold one of its arcs as a non-pivot term, keep that arc's old integer, and imply the released cycle again once unrelated fixes complete the span. Every release has a cause (a vanished arc, a new life, a within-epoch replay or gross-line-of-sight decision that named the line of sight, a contradiction, a failed float test, a refusal of the fold) and an event row.

16.7.3 Fixed cycles folded into a detached held solve

The fixed-cycle fold (`network.ambiguity_resolution.fixed_cycles.fold`, values `off`, `observe`, `emit`; default `off`, setting status KEEP_OFF) folds the integers the search fixed into a second solve of the shared parameters (clocks, phase biases, code biases), made from a copy of the float solve's own reduced normals (`NetworkEstimator::solveHeldDetached`). The estimator, the carried ambiguities, the search and post-fit QC keep the float: with zero-process-noise ambiguities the float at an epoch already holds the whole history, so a hold fed back into the filter would add only cross terms and let the hold confirm itself. Under `emit` the solution preparation and the emitter read the held vector; every other reader keeps the float.

One epoch has five steps (`network/ar/FixedCycles`).

1. *Units.* A unit is one DD identity of a pair family that the search determined in *both* bands: two integers (n_1, n_2). A DD determined in one band only has little test power and is never a unit.
2. *The per-unit test.* With $\boldsymbol{v} = \boldsymbol{y} - \boldsymbol{n}$ in cycles and $\boldsymbol{\Sigma}_{\text{pair}}$ the pair's float covariance from the search's own readback,
$$T = \boldsymbol{v}^{\mathrm{T}} \boldsymbol{\Sigma}_{\text{pair}}^{-1} \boldsymbol{v} \;\sim\; \chi^2_2 \quad \text{under right integers.} \tag{277}$$
A unit above the $1 - \alpha_u$ quantile ($\alpha_u = 10^{-6}$, `unit_alpha`) is not placed, because the overall block test cannot see one wrong pair among the rows of a network-sized block.
3. *Placement.* A unit becomes two soft rows $\boldsymbol{z} = \boldsymbol{C}\boldsymbol{x}_L \sim \mathcal{N}(\hat{\boldsymbol{z}}, \boldsymbol{P}^z)$, one per band, on the live ambiguity columns of its four band-arcs. The target is λN minus the linearisation terms re-based on the epoch's linearisation point, and the hold standard deviation is `sigma_cyc` (0.003 cycle) of the band. One block per connected station component, so no two blocks share a station.
4. *The held solve.* The fold is the Woodbury fold of the carried double differences (Section 22.4) returned as operands instead of written into the kernel. Per block of K rows, with $\boldsymbol{N}_{rr}$, $\boldsymbol{N}_{sr}$ and $\boldsymbol{u}_r$ the station blocks of the reduced normals,
$$\boldsymbol{B} = \sum_r \boldsymbol{C}_r \boldsymbol{N}_{rr}^{-1} \boldsymbol{C}_r^{\mathrm{T}}, \quad \boldsymbol{G} = \sum_r \boldsymbol{N}_{sr} \boldsymbol{N}_{rr}^{-1} \boldsymbol{C}_r^{\mathrm{T}}, \quad \boldsymbol{z}_{\text{free}} = \sum_r \boldsymbol{C}_r \boldsymbol{N}_{rr}^{-1} \boldsymbol{u}_r, \quad \boldsymbol{A} = \boldsymbol{P}^z + \boldsymbol{B}, \tag{278}$$
$$\Delta\boldsymbol{N} = \boldsymbol{G}\boldsymbol{A}^{-1}\boldsymbol{G}^{\mathrm{T}}, \qquad \Delta\boldsymbol{u} = \boldsymbol{G}\boldsymbol{A}^{-1}\big(\boldsymbol{z}_{\text{free}} - \hat{\boldsymbol{z}}\big). \tag{279}$$
The stations of a block are touched by no other block, so the corrections add exactly across blocks. $\boldsymbol{P}^z$ is diagonal, so $\boldsymbol{A}$ is as sparse as the overlap of the rows' stations and is factored by

a sparse LDL^{T} after a Jacobi equilibration. Two screens guard a block: $\boldsymbol{B}$ alone is singular exactly when the rows are linearly dependent, whatever the hold, and then $\boldsymbol{A}$. A refused float reproduction, a non-finite held estimate, or a shift of the shared estimate above `max_shift_m` (0.5 m) on any column leaves the epoch float, counted.

5. *Acceptance.* Each block's own statistic $\chi^2_b = \boldsymbol{v}_b^{\mathrm{T}} \boldsymbol{S}_{bb}^{-1} \boldsymbol{v}_b$, with $\boldsymbol{v}$ the float cycle values minus their targets and $\boldsymbol{S} = \mathrm{Var}(\text{float cycles}) + \boldsymbol{P}^z$, is tested against the $1 - \alpha_g$ quantile of χ^2_K ($\alpha_g = 10^{-4}$, `gate_alpha`). A failing block drops its unit of largest T and refolds, up to `max_iter` (3) passes, and a block that still fails stays float for the epoch.

Under `source:lattice` the units are the L1/L2 pairs the ledger implies, and the epoch's steps are the ledger's own: it opens by releasing what the epoch's evidence ended, re-tests every implied pair against the search's float exactly as in step 2, admits fixes that persisted, optionally stops placing a unit whose pair the search has not determined for `lapse_ep` epochs (default 0, off), and then folds as above. Both tests see a wrong integer only where the float disagrees with it.

SD extra-wide-lane rows and integers: Route B and the A2 datum. The second mechanism (`network/ar/SdEwlAr` and `SdEwlRows`, gated on `network.sd_ewl.enable` for the float rows and on `network.sd_ewl.fix_enable`, default `false`, for the integers) uses the MW statistic on the extra-wide-lane band pair (GPS L2/L5, Galileo E5b/E5a; BeiDou deferred, GLONASS excluded for its FDMA frequencies). For each satellite s against a pivot satellite p at a station the float row is the pseudo-observation

$$z_{SD} = \overline{\mathrm{MW}}(r,s) - \overline{\mathrm{MW}}(r,p) = \lambda_{\mathrm{EWL}}\big[N_{\mathrm{EWL}}(r,s) - N_{\mathrm{EWL}}(r,p)\big] + \big[B^s_{\mathrm{EWL}} - B^p_{\mathrm{EWL}}\big], \tag{280}$$

with coefficients $\pm\lambda_{\mathrm{EWL}}/\lambda_b$ on the metre-valued ambiguity states and, from $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$, $+f_1/(f_1-f_2)$ and $-f_2/(f_1-f_2)$ on the satellite phase biases and $-f_1/(f_1+f_2)$ and $-f_2/(f_1+f_2)$ on the satellite code biases of the two bands, negated for p. The rows of one constellation at a station share the pivot arc's noise, $\boldsymbol{R} = \mathrm{diag}(\mathrm{var}_s) + \mathrm{var}_p \mathbf{1}\mathbf{1}^{\mathrm{T}}$, and are whitened by the Cholesky factor of $\boldsymbol{R}$. They enter the station block before the reduction, so they compose exactly with the post-reduction double-difference fold. The float rows add no rank: the MW row is a linear combination of observables already in the epoch's normal equations, and at a realistic row noise they raised the soft-end eigenvalue of the normal equations by 0.51 per cent, below the bar set for them (`docs/SPEC_sd_ewl_prereduce.md`). The only new information a row can carry is integrality. Rounding the SD statistic directly is circular, because it requires the satellite bias difference $B^s_{\mathrm{EWL}} - B^p_{\mathrm{EWL}}$ that the network estimates. Route B avoids the circularity by differencing the SD MW statistic between stations, which gives the same double difference as (274):

$$\boxed{\big(\mathrm{MW}^i_{r_1} - \mathrm{MW}^{p_g}_{r_1}\big) - \big(\mathrm{MW}^i_{r_2} - \mathrm{MW}^{p_g}_{r_2}\big) \;=\; \lambda_{\mathrm{EWL}}\,\nabla\Delta N^{i,p_g}_{r_1 r_2,\mathrm{EWL}} \;+\; \nabla\Delta\varepsilon^{\mathrm{MW}},} \tag{281}$$

in which the satellite bias pair cancels exactly: the satellite pair is the same on both sides, and each station's receiver term has already cancelled inside its own single difference. What remains is an integer plus averaged MW noise at the extra-wide-lane wavelength. An integer decided this way uses no a-priori satellite bias and cannot be corrupted by a wrong one, which is why Route B is preferred over rounding against a bias product. The cancellation in (281) has a precondition that belongs to the algebra, not to the implementation: b_{EWL} is a per-signal combination, so both stations' arcs must use the same two tracking channels. Otherwise a per-signal bias difference is folded in as though it were an integer. The seam therefore elects a network-common channel pair per satellite by station consensus and admits only stations that hold that pair.

Equation (281) yields a double difference, while the constrained row needs a single-differenced integer per (station, satellite). The A2 datum supplies one. Within each connected component a reference station r_0 is elected and kept sticky, $\Delta n_{r_0}^{i,p_g} := 0$ is defined, and the rounded DD fixes are propagated outward along a spanning tree; the link satellite absorbs the per-station free integer. The asserted quantity is then

$$\Delta n_r^{i,p_g} \;=\; \Delta N_{r,\mathrm{EWL}}^{i,p_g} - \Delta N_{r_0,\mathrm{EWL}}^{i,p_g}, \tag{282}$$

the true SD integer minus a constant per (satellite pair, component), which is the classical wide-lane fractional-cycle-bias convention modulo λ_{EWL} [8, 19]. The constant is absorbed into the estimated bias the network reports. Two consequences of (282) matter for the design. First, the per-station free integer, and with it the spanning tree, cancels at the point of use, so a change of spanning tree between epochs is not a defect. Second, r_0's own integers are absorbed into the constant, so the choice of r_0 is a datum decision rather than a measurement.

Acceptance is the scalar case of the battery in Section 16.5. A rounded DD is admitted on its fractional residual, as in (271), and on the $n = 1$ instance of the bootstrap bound (270), each with its own threshold (network.sd_ewl.fix_max_frac_cyc, default 0.15 cycle, and network.sd_ewl.fix_min_ffr, default 0.999):

$$P_{s,\mathrm{round}} = \mathrm{erf}\left(\frac{1}{2\sqrt{2}\,\sigma}\right), \qquad \sigma \text{ in cycles}, \tag{283}$$

so a single rounding and the n-dimensional search share the same failure-rate semantics.

The mechanism is subject to the design rule that a stiff, time-linked quantity needs a wired event response. The fixed row writes a λ_{EWL}-quantised value into satellite bias states with very small process noise, so an integer that changes between epochs would inject a step that those states take hours to relax out of. The wired response is withhold-on-disagreement. When this epoch's integer differs from the previous one on a continuous arc, the fix for that arc is withheld rather than repaired, because on a disagreement epoch neither value is known to be right. An arc that restarted (slip, gap, re-acquisition) replaces the stored integer without counting as a disagreement, because its integer has legitimately changed. The stored integer is used only to veto, never to supply a value; this keeps the mechanism short of a carried zero-process-noise hold, which is deliberately not built.

Settings. All mechanisms of this subsection are off in the loader, and the reference configurations set none of them. The loader disarms `network.sd_ewl.fix_enable`, with a warning, unless the float row layer (`network.sd_ewl.enable`) is armed. That layer needs `network.phase_leveling` and an MW-class slip detector, because it reads the arc accumulator they build, and its pairs need GPS L5 or Galileo E5b, which the dual-frequency reference configurations do not carry. The fixed-cycle fold's emit mode needs the two slip tests of Section 15.3.2 and Section 15.2.5 (`gf_arc_jump` and `if_phase_td_reseed_on_flag`), refuses a state snapshot, and checks that every station is a simulated one.

16.8 Configuration summary

All tunable thresholds live under the JSON path `estimation.ambiguity_resolution.*`, parsed in `src/core/config/GnssConfigLoad.cpp` with defaults in `include/filter/AmbiguityResolver.hpp`. The estimator theory of Sections 16.1 to 16.6 has no settings, since changing it would change the mathematics rather than a policy. In the mode row of the table, modes 1 and 2 behave identically in the code (Section 16.6). The table omits three WL acceptance keys described in Section 16.3 (wl_only_ffr, default on; wl_only_min_arc_count, default 10; wl_only_max_innov_cyc, default 0.5)

and the search-row option nl_iono_free_row (default off). The network mechanisms of Section 16.7 are configured separately, under `network.ambiguity_resolution.*` (including `fixed_cycles.*`) and `network.sd_ewl.*` (parsed in `src/core/config/GnssConfigLoadNetwork.cpp`); every master switch defaults to off, and the settings are omitted from the table. The verdict for each key is in `docs/CONFIG_promotion_ledger.json`.

Symbol	Config key (`...ambiguity_resolution.`)	Default	Role
	`mode`	`0`	0 float / 1 continuous / 2 instantaneous / 3 fix-and-hold
τ_{ratio}	`ratio_threshold`	`3.0`	ratio-test acceptance (269)
$\tau_{\text{ratio}}^{\max}$	`ratio_upper_bound`	`1e4`	degenerate-covariance reject (269)
$P_s^{\min}$	`min_success_rate`	`0.999`	FFR gate (270)
κ_{frac}	`nl_max_per_pair_frac`	`0.15`	per-pair residual cap (271)
θ_{AR}	`elevation_mask`	`20°`	candidate elevation mask
σ^2_{hold}	`hold_variance`	`0.001` m^2	pseudo-obs $\boldsymbol{R}$ (273)
$n^{\text{ep}}_{\min}$	`min_epochs`	`10`	convergence guard
$\sigma^{\max}_{\text{pos}}$	`max_position_sigma_m`	`0.05` m	convergence guard
$\sum \sin^2\theta_{\min}$	`min_arc_info`	`0.0` (off)	per-candidate arc-maturity gate, WL/NL path only (Section 16.4)
	`wl_only`	`false`	MW-WL-only, no NL search
σ^2_{wl}	`wl_only_pseudo_var`	`1e-4` cyc^2	WL-constraint pseudo-obs $\boldsymbol{R}$ (Section 16.3)
	`wl_max_frac`	`0.40`	per-arc WL fractional-residual gate (Section 16.3)
	`enable_partial_ar`	`false`	backward-elimination subset fix (Section 16.4)
	`active_systems`	`[]` (all)	constellation subset (Section 16.4)
	`state_ambiguity_basis` †	`per_band`	`per_band` vs `nl_wl` dispatch

† Path is `estimation.state_ambiguity_basis`, one level above the `ambiguity_resolution` block.

Part VI
Inertial aiding

17 Inertial aiding

With an IMU configured, a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame replaces the inter-epoch time update of Section 12.1. Two switches control it. `imu.enabled` enables the IMU. `estimation.aiding.imu` selects the IMU source in the `AidingSourceManager`. When `estimation.aiding.imu` is absent it defaults to the value of `imu.enabled`; an explicit value takes precedence (`src/core/config/GnssConfigLoad.cpp`).

The mechanization is implemented in `src/core/filter/ImuIntegrator.cpp`. The per-epoch prediction runs `Application::predictEpoch_` (`src/core/app/PppEpochSteps.cpp`) → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, and `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`) splices the resulting `PredictContribution` into the filter covariance (`KalmanFilter::spliceAndPropagate` in `src/core/filter/KalmanKernels.cpp`). The real-time engine (`RealTimeEngine::processEpoch`) uses the same `predictAided` / `collectPredict` path. Only the sample ingest differs. Post-processing reads a CSV file with `loadData`, which sorts the samples by time. In real time each sensor sample reaches `ImuIntegrator::addSample` through `ImuAidingSource::ingestSample`, which appends

it to the same buffer in arrival order. For samples that arrive in time order the two paths therefore give the same `integrate` results.

The `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the interface through which every non-GNSS modality contributes up to three items: a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows. The IMU is its first client; a zero-velocity update source is the second, and the registry declares no implementation for the non-holonomic, wheel, barometric and vision kinds. `ImuAidingSource` reproduces the arithmetic of the older monolithic `KalmanFilter::predictImu`, so the strapdown equations below apply unchanged. `predictImu` has been removed from the source; `predictAided` replaces it.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix} \delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{284}$$

carries the linearised transition and process noise that propagate the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$ and three gyro-bias states $\boldsymbol{b}_g$ augment the position state (and the three velocity states when `estimation.estimate_velocity` is set). `StateManager` allocates the nine added states as one contiguous block (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 2 (Implemented scope). `ImuIntegrator` fixes the block order of Equation (284) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$). The filter carries no lever-arm state and no sensor scale-factor states, and the GNSS measurement model has no IMU-to-antenna lever-arm term (Section 17.4). These are planned extensions. The equations below omit the corresponding terms.

17.1 Strapdown mechanization

For each IMU sample, the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are corrected with the current filter bias estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{285}$$

and integrated over the sample interval Δt. A sample whose interval is not positive or exceeds 10 s is skipped. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body $\rightarrow$ ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{286}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To limit numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 integrated samples, starting with the first. The velocity and position propagate as

$$\boxed{\begin{aligned} \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, && (287)\\ \boldsymbol{r}_{k+1} &= \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. && (288) \end{aligned}}$$

The specific force is resolved with the attitude already updated by Equation (286), and gravity is evaluated at the position of the start of the step. Equation (287) applies the Coriolis acceleration

$-2\,[\boldsymbol{\omega}_{ie}^{e}]_{\times}\boldsymbol{v}$. The centrifugal acceleration is part of the gravity vector $\boldsymbol{g}^{e}$: $\boldsymbol{g}^{e}$ is a *plumb-bob* gravity (gravitational plus centrifugal). Equation (288) is a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^{e}(\boldsymbol{r}) = -\frac{\mu}{\|\boldsymbol{r}\|^{3}}\,\boldsymbol{r} \;+\; \omega_{e}^{2}\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005\times10^{14}\,\mathrm{m^{3}\,s^{-2}}. \tag{289}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are not modelled. Over one GNSS epoch interval the resulting deterministic acceleration error is small, and the estimated accelerometer bias $\boldsymbol{b}_a$ absorbs it.

17.2 INS error-state dynamics

Linearising Equations (286) to (288) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (284),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \mathbf{0} & \boldsymbol{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -2[\boldsymbol{\omega}_{ie}^{e}]_{\times} & -[\boldsymbol{f}^{e}]_{\times} & \boldsymbol{C}_{b}^{e} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -[\boldsymbol{\omega}_{ie}^{e}]_{\times} & \mathbf{0} & -\boldsymbol{C}_{b}^{e} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}, \qquad \boldsymbol{f}^{e} := \boldsymbol{C}_{b}^{e}\,\boldsymbol{f}_{b}. \tag{290}$$

Both sensor biases are random walks (no deterministic dynamics: the null bottom rows of Equation (290)). Two terms of the standard ECEF error model are omitted by design: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^{e}]_{\times}$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = +\partial\boldsymbol{g}^{e}/\partial\boldsymbol{r}$ (the gradient of the *plumb* gravity of Equation (289), which already subsumes the centrifugal $-[\boldsymbol{\omega}_{ie}^{e}]_{\times}^{2}$; the leading sign follows from differentiating the velocity update Equation (287)). The resulting first-order model is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is also first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_{k} = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{291}$$

The process noise maps the continuous accelerometer and gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\mathrm{ins}}]_{k} = \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\mathrm{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\mathrm{ins}} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \boldsymbol{C}_{b}^{e} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\boldsymbol{C}_{b}^{e} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \boldsymbol{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \mathrm{diag}\big(\sigma_{a}^{2},\sigma_{g}^{2},\sigma_{ba}^{2},\sigma_{bg}^{2}\big)\otimes\boldsymbol{I}_{3}, \tag{292}$$

where the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ are read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}`. The defaults in `include/config/GnssConfigTypes.hpp` are $0.05\,\mathrm{m\,s^{-2}\,Hz^{-0.5}}$, $5\times10^{-4}\,\mathrm{rad\,s^{-1}\,Hz^{-0.5}}$, $1\times10^{-4}\,\mathrm{m\,s^{-3}\,Hz^{-0.5}}$ and $1\times10^{-6}\,\mathrm{rad\,s^{-2}\,Hz^{-0.5}}$, in that order. Over one GNSS epoch the per-sample pairs $([\boldsymbol{\Phi}_{\mathrm{ins}}]_{k}, [\boldsymbol{Q}_{\mathrm{ins}}]_{k})$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_{k}\,\boldsymbol{\Phi}_{\mathrm{ins}}, \qquad \boldsymbol{Q}_{\mathrm{ins}} \leftarrow [\boldsymbol{\Phi}_{\mathrm{ins}}]_{k}\,\boldsymbol{Q}_{\mathrm{ins}}\,[\boldsymbol{\Phi}_{\mathrm{ins}}]_{k}^{\mathrm{T}} + [\boldsymbol{Q}_{\mathrm{ins}}]_{k}. \tag{293}$$

17.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\text{ins}}, \boldsymbol{Q}_{\text{ins}})$ is written over the indices the INS owns in an identity-seeded full-state transition $\boldsymbol{F}$ and in the random-walk process-noise baseline $\boldsymbol{Q}$. The write is an assignment, so the baseline noise on those rows is not counted twice. The baseline process noise is evaluated over the interval the INS integrated. The non-INS states (clock, troposphere, ionosphere, ambiguities, ...) keep their random-walk model from Section 12.1; because the transition is the identity there, their means are not propagated. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\big(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\big).} \tag{294}$$

The mechanised values of Equations (287) to (288) overwrite the nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$. The attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because Equation (286) has already applied the rotation increment to $\boldsymbol{C}_b^e$: after each aided prediction the IMU aiding source writes the propagated attitude into the filter through its attitude setter. The filter holds the single nominal attitude. It is initialised to the identity (there is no initial alignment step), and on the live path only two operations write it: that setter call and the feedback step below. After the next GNSS measurement update, the corrected attitude error feeds back multiplicatively into the DCM and is reset again,

$$\boldsymbol{C}_b^e \leftarrow \big(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\big)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{295}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small angle, so the linearisation in Equation (290) remains valid.

17.4 Coupling modes and the GNSS update

The coupling mode (`imu.coupling_mode`) determines how the GNSS observations correct the inertial errors. In both modes the filter processes the raw GNSS observations; the modes differ in the Doppler Jacobian and in the relinearisation path (see the remark at the end of this section).

Loosely coupled (default). Among the INS states, the GNSS Jacobians of Section 4.3 touch only the position and velocity columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ have no direct observation partials. They are corrected indirectly, through the cross-covariances that Equation (294) builds between the position/velocity block and the attitude/bias block. The feedback of Equation (295) then applies the attitude correction.

Tightly coupled. Tight coupling requires the velocity block, so the loader enables it regardless of `estimation.estimate_velocity`. A `TightCoupled` `coupling_mode` with the flag false sets the flag true and logs `"forcing estimate_velocity=true"` (`GnssConfigLoad.cpp`). The three velocity states, and with them the Doppler rows that carry the Jacobian below, are therefore always present. In this mode the Doppler observation, and only the Doppler observation, gains direct attitude and accelerometer-bias columns. It is the one aiding contribution that adds columns to existing GNSS rows instead of appending rows of its own. Each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which passes the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`), and `MeasurementModel::buildDopplerRow` (`src/core/measurement/MeasurementModelRow.cpp`) writes the columns. With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and

$\boldsymbol{e}_r^i$ the unit line-of-sight of Equation (7), the Doppler measurement function h_{D} contributes

$$\boxed{\frac{\partial h_{\mathrm{D}}}{\partial \delta \boldsymbol{\psi}} = -\left[\boldsymbol{f}^e\right]_\times^{\mathrm{T}} \boldsymbol{e}_r^i, \qquad \frac{\partial h_{\mathrm{D}}}{\partial \boldsymbol{b}_a} = -\left(\boldsymbol{C}_b^e\right)^{\mathrm{T}} \boldsymbol{e}_r^i.} \tag{296}$$

The specific force used in Equation (296) is the static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^{\mathrm{T}}(-\boldsymbol{g}^e) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`), evaluated at the current position estimate rather than from the IMU samples. It is exact only when the true platform acceleration is negligible compared with gravity. In either mode the code and carrier-phase rows carry no attitude or bias partials; only the Doppler rows couple directly to the inertial errors.

Remark 3 (Relinearisation under tight coupling). The tightly coupled path keeps a single *naive* second-pass relinearisation instead of the prior-anchored iterated EKF of Section 12.3. Re-deriving the gain from a fixed prior is inconsistent with the multiplicative attitude feedback of Equation (295), which has already changed $\boldsymbol{C}_b^e$ and zeroed the attitude error between passes. The aiding layer exposes the requirement as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, true for the IMU source under tight coupling), so any later source with nominal-state feedback selects the same path. Direct `tightlyCoupledImu()` checks remain in `src/core/app/PppEpochUpdate.cpp` and `src/core/filter/KalmanFilter.cpp` (the IEKF gating and the prior capture), so the migration to the capability query is incomplete.

Part VII
Real-time SSR corrections

18 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock and hardware biases. They arrive as RTCM 3.x messages (legacy 1057–1068 and 1240–1270) or in the IGS-SSR `4076` repackaging [13, 29]. Each SSR quantity refines one term already present in Equation (4) and Equation (5) of the Foundations document, and none introduces a new state. SSR is what distinguishes the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade, and the satellite biases B_s^i, b_s^i are *known* instead of absorbed into the ambiguity. This document describes the user-side application of the corrections. The same store receives a third-party IGS-RTS stream and the products of the engine's own network estimator (Section 20), so the application mathematics below does not depend on their source.

Decoded records are cached per satellite in `products/SsrCorrectionStore`: a process-wide singleton for the live stream, or a caller-owned instance for the closed-loop user of the network (Section 18.9). The orbit is applied in `model/ModelGeometry`, the clock in `corrections/GnssCorrections` and the biases in `measurement/ObservationEquation`. Each queries the store and falls back to broadcast-only when it holds nothing usable. In post-processing the store is empty, so every equation below reduces to the broadcast-only observation model. The streaming and decoding layer (RTCM frame parsing, the per-satellite mutex-guarded cache, the real-time push loop) is described in Section 19. This document covers only the store-driven correction mathematics, which does not depend on how the records arrived. There is no `enable_ssr` flag: SSR is applied whenever the store holds a usable record.

18.1 Orbit correction: refines ρ_r^i

The orbit message carries a delta in the satellite's radial, along-track and cross-track (RAC) frame, referenced to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}^i_{\mathrm{brdc}}$ and velocity $\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{297}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction relative to the content epoch of the orbit record, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}^i_{\mathrm{brdc}} \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{298}$$

The subtraction follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). If the broadcast velocity or the position-velocity cross product is degenerate, the frame is undefined and the broadcast position is returned unchanged. The corrected $\boldsymbol{r}^i$ enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}^i_{\hat{r}}$ of the Foundations document unchanged. The orbit correction therefore acts on no observation equation directly, only on the geometry (Section 4) that feeds it.

The delta is applied only when its IODE matches that of the broadcast ephemeris from which the baseline orbit was evaluated (Equation (301)). Over a product orbit the gate compares with the IODE of the orbit node nearest in time; SP3 nodes carry none, so the gate passes. On a live ephemeris cutover, the baseline position is re-referenced to the ephemeris *version* against which the SSR delta was issued, so the satellite does not drop to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). The clock is queried against the same broadcast ephemeris, so the broadcast clock's *radial* component cancels consistently against Equation (300). The decoded RAC scale factors are those of RTCM 10403.3: the radial δ_r is a 22-bit two's-complement field at 0.1 mm, along-track and cross-track are at 0.4 mm, and the rates are at 1 and 4 μm/s respectively (`rtcm/RtcmSsrDecoder.cpp`).

The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$. Legacy RTCM SSR corrects to the satellite antenna phase centre (APC); IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (default OFF, byte-identical) the geometry records whether the applied overlay was APC-referenced (`ModelGeometry::ssrOrbitApcApplied`). `PppModel` then omits its own satellite-PCO term on that geometry, so the phase-centre offset already contained in the orbit correction is not counted twice.

18.2 Clock correction: refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in the time since the reference epoch, optionally augmented by an additive high-rate delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{299}$$

converted to seconds (`SsrCorrectionStore::applyClockToBias`) and *added* to the baseline clock (`corrections/GnssCorrections.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{300}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live mode, `products/LiveEphemerisStore`). The term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is added to the polynomial and does not replace it (`corrections/GnssCorrections.cpp`); it is subject to its own freshness test in the store. The C_2 field is a 27-bit two's-complement quantity at $2 \times 10^{-8}\,\mathrm{m/s^2}$. The time base of the polynomial is the record's content epoch (the provider's reference epoch), not its arrival time.

18.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a specific broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\mathrm{IODE}_{\mathrm{brdc}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{SSR}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{brdc}} = \mathrm{IODE}_{\mathrm{SSR}}\big), \tag{301}$$

The gate is permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) takes precedence over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). The two overlays gate on separate anchors: the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream, `clock_iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (301) is a single gate. Only a network clock that stamps its own clock IODE (the `brdc_rts` clock mode) makes them differ. At a correction-set boundary (the provider switches to a new ephemeris reference, hence a new IODE), the gate (301) declines the mismatched record until the matching broadcast ephemeris is available. This avoids a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$.

The two sides of (301) are compared in the IGS-SSR convention. A `4076` message carries an 8-bit IODE field for every constellation. A legacy Galileo message carries the full 10-bit IODnav, and the decoder keeps its low 8 bits (`normalizeSsrIode` in `rtcm/RtcmSsrDecoder.cpp`). The broadcast side converts its record in the same way (`LiveEphemerisStore::ssrIode`): the GPS and QZSS IODE as broadcast, the low 8 bits of the Galileo IODnav, and $\lfloor t_{oe}/720 \rfloor \bmod 240$ for BeiDou, with t_{oe} the seconds of week of the ephemeris reference time in BDT. The BeiDou broadcast AODE is a different quantity and is not used.

Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent. The age is that of the record's receive time relative to the query epoch, and the test is symmetric: a record dated more than the same limit ahead of the observation (a desynchronised live stream) is also rejected. The stale-hold rule extends this binary clock gate to three stages (`SsrCorrectionStore::classifyClockAge`). With `processing.ssr_clock_stale_hold_factor` $f > 1$, a clock aged in ($\texttt{max_age_s}$, $f \cdot \texttt{max_age_s}$] is *StaleHeld*. It is still applied, extrapolated over the full age: the code evaluates the whole polynomial (299), of which $C_0 + C_1\,\delta t$ is the dominant part. The variance of the consuming rows grows by $q\,\delta t^3/3$, the zero-order-hold variance of a clock that wanders with density q (`network.ura_staleness_q2`, which defaults to `network.clock_model_q2`; $1.5 \times 10^{-7}\,\mathrm{m^2/s^3}$ is used when it is zero; `measurement/MeasurementModelVariance.cpp`). A record beyond the hold window is *Rejected*. The default $f = 0$ reduces the rule to the plain fresh-or-absent gate (byte-identical).

Orbit-coherence echo. A network clock in the `brdc_rts` mode is estimated against one orbit update, and the radial error of the broadcast orbit is absorbed one for one into that clock. A user that applies a different orbit update receives a range bias of decimetres to metres that residual tests do

not detect. The network therefore echoes the identity of the orbit it used (its IOD-SSR and content epoch) on each clock record, and the user compares it with the orbit it applies (`SsrCorrectionStore::classifyOrbitEcho`). The verdict is one of: no echo stamped, match, no applied orbit, IOD-SSR mismatch, or content-epoch skew beyond a budget. Every verdict other than a match is counted per satellite. The clock is withheld only when `processing.network_consumer.orbit_clock_max_skew_s` is positive (default 0: count only), so a stream that stamps no echo is unaffected.

18.4 Satellite biases: refine $-B_s^i$ and $-b_s^i$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a known additive correction to the modelled observable, with no state and no Jacobian column (the `SsrBias` term built by `addSsrBias` in `measurement/ObservationEquation.cpp`). The code OSB refines B_s^i in the undifferenced code equation:

$$\boxed{P_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i + \mu_s I_{r,1}^i + B_{r,s} \underbrace{-\,B_s^i}_{\text{SSR OSB}} + \varepsilon^P,} \tag{302}$$

and the phase OSB refines b_s^i in the undifferenced phase equation:

$$\boxed{\Phi_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i - \mu_s I_{r,1}^i + \lambda_s N_{r,s}^i + b_{r,s} \underbrace{-\,b_s^i}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{303}$$

The stored value is the satellite bias as it is present in the observation (the underbraced term), so it is added to the clean model prediction with sign $+1$ (`kSatBiasSign` in `measurement/ObservationEquation.hpp`), the constant that is also the partial of the network estimator's satellite code-bias and phase-bias states. Code and phase share it. The sign relating the value a provider sends to the stored value is fixed in the bias bodies of `rtcm/RtcmSsrDecoder.cpp`. It can be confirmed only empirically against a provider's stream: float ambiguity fractions that cluster near one half instead of zero indicate an inverted constant. The value enters the predicted observable as the `SsrBias` term of the observation equation, summed with the receiver clock, troposphere, ionosphere, estimated code and phase biases and the ambiguity. Biases are queried per *raw* signal only: an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`).

The 5-bit signal identifier of each bias is translated through the SSR table (`RtcmSignalMap::ssrToRinexSuffix`), which differs from the MSM table that numbers the cells of an observation message. GPS L1 C/A is SSR identifier 0 and L2 Z-tracking is 11, against MSM identifiers 2 and 10; only a few identifiers coincide (Galileo E1 C is 2 in both) (`rtcm/RtcmSignalMap.hpp`). A bias whose identifier has no entry for its constellation is dropped by the decoder, and a bias read through the wrong table would be applied to the rows of another signal.

By default the application path has no age gate: the bias is loaded whenever the store carries the exact RINEX signal key (or, for code only and with the optional band fallback `processing.ssr_bias_band_fallback` armed, a same-band sibling key; phase never falls back, to preserve the integer property). A lookup that misses the exact key while a same-band sibling exists is counted per satellite, signal and domain, whether or not the fallback resolves it. The optional staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) measures the age of the bias record against the observation epoch. Its limit is `ssr_bias_stale_hold_factor` × `processing.ssr_bias_max_age_s`; the factor defaults to 2 and the maximum age defaults to the clock maximum age, hence

30 s. Below that limit the last value stays applied at full weight across staleness gaps, so a provider update interval near the maximum age produces no model step; a bias older than the limit is dropped (`measurement/ObservationEquation.cpp`). The derived configuration of the closed-loop user and the standalone `network_consumer` profile arm the gate under bias consumption unless the configuration sets the key explicitly. The consequence of the application path for the state vector, avoiding a redundant estimated bias on the same row, is the subject of the next subsection.

18.5 Satellite-bias state gate and SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see Section 3). The estimated state and a known SSR code OSB both enter the code row with unit coefficient, so allowing both for the same (i, s) is redundant: the stream already fixes the satellite code bias, and a free state collinear with it is nearly unobservable and absorbs noise. State allocation therefore consults both correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates` in `measurement/MeasurementModelStates.cpp`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{304}$$

where hasBIA tests the static `.BIA` product (`biaCoversSignal`, from `PppModel::hasBiasEntry`), hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal` in `measurement/MeasurementModelRow.cpp`) and pivotCovers is the band gate of between-satellite differencing (`signalCoveredByPivot`): only signals that the pivot satellite tracks are observable in a single difference, and a state created for another signal would be seeded against an unconverged state vector, so its creation waits until the signal is usable. On a GLONASS FDMA signal that carries a per-slot inter-frequency-bias state, no sat-bias state is created either, because the two would be collinear. The gate rests on two design points:

- **Presence, not magnitude.** `hasCodeBias` tests whether the key is *present* in the per-signal map, which differs from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum, so the estimator must not re-estimate it. The two queries agree by construction, including the optional same-band fallback, so a covered signal is never both estimated and SSR-corrected on one row. With the staleness gate armed, the presence test uses the same age limit as the application path; otherwise it ignores age.
- **Per-signal, not global.** IGS streams are partial (for example, Galileo E1 covered and E5a/E5b not). The gate keys on the exact (i, s) pair. A signal the stream does not carry keeps its estimated state, its only available code correction, while a covered signal on the same satellite is left to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is always false and the gate reduces to the static-`.BIA` test.

Exemption of the network estimator. When the engine is the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles are reversed: it *estimates* the satellite code and phase biases as states and emits them as a product (Section 20). Importing SSR biases there would count them twice and make those states unobservable. Both the application path (302), (303) (`addSsrBias` adds no term) and the hasSsrCB branch of the state gate (304) (`ssrCoversCodeSignal` returns false) are therefore disabled for the network, whose own admission

owns the satellite code-bias states. The network's emitter (`products/NetworkProductEmitter.cpp`, called from `network/emit/EmitStore.cpp`) pushes the estimated clocks and code and phase biases into a separate store instance, which closes the loop against a user at a test station (Section 20.11).

Integer recoverability via consistent phase biases. Removing b_s^i restores the integer property of $N_{r,s}^i$ stated in the phase observation equation of the Foundations document. With the satellite phase bias known, the remaining non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ alone, which cancels under between-satellite single differencing $\Delta\cdot$ (Section 5). The result is an integer single-difference ambiguity admissible to AR. This defines a *consistent* phase-bias product in operational terms: the provider must generate b_s^i on the same network datum whose integer property the rover then uses. The phase-bias message therefore carries three metadata fields per signal that AR depends on (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*. The message header carries two further one-bit indicators, the dispersive-bias and the Melbourne-Wübbena consistency indicators. They are read once, after the solution identifier and before the satellite count, and copied onto every record of the message (`decodeSsrHeader` in `rtcm/RtcmSsrDecoder.cpp`); they are not repeated in the satellite block. This holds for the legacy messages `1265` to `1270` and for `4076`. A decoder that reads them per satellite shifts the count and every block after the first. The satellite block also holds a yaw angle (9 bits, 180/256 degrees per unit) and a yaw rate (8 bits, 180/8192 degrees per second per unit), which are stored and not used.

18.6 Phase-bias steps and the user's ambiguities

A phase bias that changes between correction sets moves the term that the model adds to every phase row of the satellite. Let $\tilde{b}_s^i$ be the applied bias (zero when none is applied, which includes a bias dropped by the staleness gate) and $A_s^i = \lambda_s N_{r,s}^i$ the float ambiguity state in metres. The predicted phase carries $\tilde{b}_s^i + A_s^i$, both with coefficient $+1$. A change $\Delta = \tilde{b}_k - \tilde{b}_{k-1}$ between consecutive epochs carries no information about the position, and the phase model stays consistent only if the ambiguity absorbs it exactly:

$$A_s^i \;\leftarrow\; A_s^i - \Delta, \qquad \text{covariance unchanged.} \tag{305}$$

A step is signalled in two ways. An incremented *discontinuity counter* (`StateManager::noteSsrDiscontinuity`) declares that the provider's bias jumped, which breaks the integer connection of that arc. Under the staleness gate, a bias that changes between applied and dropped (a freshness flip) steps the model term without any counter. A step can also occur with no declaration. The user reacts in `RealTimeEngine::resetOnPhaseBiasDisc`, in this order.

1. A bias whose store entry carries a nonzero hold generation (the network's phase-bias hold) is claimed first by `reactToPbHold_`. Its answer, including the presence edges of a held bias, is in Section 24.10.
2. With `processing.ssr_phase_bias_step_translate` (default OFF) every step with $|\Delta|$ above `processing.ssr_phase_bias_step_translate_eps_m` (default 5 mm) is translated by (305), declared or not (`PbStepTranslateLedger`). Inside the band the filter absorbs the move as noise and nothing is written.
3. Otherwise, with `processing.ssr_phase_bias_disc_carry` (default OFF), a declared step is translated by (305) (`PbDiscCarryLedger`). A counter change and a freshness flip on one key in one epoch are one event and one translation.

4. Otherwise the ambiguity is reset: the state is re-seeded from code at the configured ambiguity σ (`StateManager::resetAmbiguity`).

Both translations record, for every (satellite, signal), the value the model applied at the previous update, read as the model reads it (age-gated when the staleness gate is armed), so Δ is the step the phase row actually took. The declared-step carry reads an unapplied bias as zero, so a freshness flip is translated by the whole bias. The every-step translation needs an applied value on both sides and sends a declared flip to the reset. Both fall back to the reset, with each reason counted, when the key has no recorded value (a first sighting, or the first event after a restore), when the ambiguity basis is narrow-lane and wide-lane (the shift would need the basis transform), when the user is between-satellite differenced (its states are differences) or when the ambiguity key is not in the state. The every-step translation counts an undeclared step in these classes without resetting. Each event takes exactly one counted outcome, and the run-end counters close. The record of applied values is not part of the filter-state snapshot, so the first declared step per key after a warm restore takes the reset (Section 19.14).

Both settings are off in the reference configurations (`config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`) and carry the ledger verdict KEEP_OFF. The declared-step carry is exact and does not move the emitted product, but failed two of five bars in its re-grade (`docs/refound/CC-arm_pb_disc_carry_rf0hv_2026-09-15.md`). The every-step translation was superseded by the network's phase-bias hold, which changes what the product declares (`docs/refound/PBH_phase_bias_hold_2026-09-19.md`; Section 24.8). The reset is the phase-bias counterpart of the orbit and clock IOD gate (301): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity. A change of the network's solution-generation id, and a re-reference of its code-bias or clock datum, are answered by resets and re-inflations described in Section 19.10.

18.7 GLONASS

GLONASS SSR is decoded and stored in the same way as the CDMA constellations. The legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the same orbit, clock and bias structures (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias equations above apply unchanged; nothing in Equations (298), (300), (302) and (303) is constellation-specific.

The GLONASS-specific difference is the time tag of the legacy messages; the correction mathematics is shared. Legacy GLONASS SSR (`1063` to `1068` and `1266`) carries its content epoch as a 17-bit time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) instead of a 20-bit GPS time of week, and its satellite identifiers have 5 bits. IGS-SSR `4076` uses one layout for every constellation (6-bit satellite identifier, 8-bit IOD, 20-bit GPS-aligned time of week), so the conversion applies to the legacy messages only. The decoder converts the legacy field to the GPS time scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`), resolving the day from the parser's last known GPS stream time. Without that conversion the SSR epoch would be $\{0, 0\}$; the store would then drop every GLONASS record as stale or IOD-mismatched (the `max_age_s` gate that accompanies (301)), and the satellite would fall back to broadcast grade without any error. After conversion, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (298) and (300) as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate estimator-side option (the FDMA per-channel receiver delay does not cancel in SD); see Section 5.

18.8 Ionosphere: $\mu_s I^i_{r,1}$ off the stream, via the slant channel

The SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, but this engine decodes neither. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on an RTCM stream writes the slant-iono state $I^i_{r,1}$. For a user of an RTCM stream, the ionospheric term $\mu_s I^i_{r,1}$ is therefore informed only by (i) the per-satellite iono *state* estimated by the filter and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The store has a dedicated undifferenced slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`) that closes the atmosphere loop outside the RTCM stream. When the network estimates a spherical-harmonic VTEC field, its emit seam (`network/emit/EmitStore`) evaluates the field (Section 9.2.4) at the user's pierce points and pushes a per-LOS slant delay and 1σ. The user (`RealTimeEngine::applyNetworkIonoPrior`, run after the measurement update and before AR) folds each fresh record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`), with innovation $I_i^{\text{net}} - \hat{I}_i$ and $\boldsymbol{H}$ selecting the states. This is the ionosphere-weighted constraint that makes single-epoch AR feasible. The fold is gated by `network.closed_loop.consume_network_iono` (default OFF: no rows are built and the update is byte-identical). It admits a value only when the network refreshed it in the current epoch (`max_age` $= \frac{1}{2}$ the epoch spacing, or 30 s when the spacing is unknown), so a satellite the network did not re-supply floats instead of inheriting a stale delay.

The reference configurations estimate no spherical-harmonic field (the slant ionosphere is free per line of sight), so the supply is gated on a model that is never built and the channel stays empty, although the reference configurations carry the setting as true by lineage. The ledger verdict of the setting is KEEP_OFF (`docs/CONFIG_promotion_ledger.json`; `docs/refound/AB1_sh_consumer_product_2026-09-20.md`). The channel has no record in the product file either, so the offline replay and the file-tail user cannot receive it; the setting is inert there and draws a warning.

Three properties of the channel follow from the field and are not chosen by the user. The supply covers GPS and Galileo only: GLONASS is excluded from the SH field, so a user's GLONASS slant states float. When a second shell is armed, the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the user does not depend on the layer model. The per-LOS σ is the field's representativeness floor (Equation (102)). Under `network.iono_supply_formal_sigma` (default OFF, byte-identical) it is root-sum-squared with the SH coefficients' *formal* σ, propagated through the same weighted basis, and the supply is skipped for a satellite that has no estimated satellite code bias (OSB) in the current network solution.

At the user the rows are not independent. They are folded as a per-constellation common-mode block with the noise structure of Equation (98): rows of one constellation share the variance `common_mode_sigma_m`^{2} in addition to their own diagonal variance. Frozen BSSD pivots, and any slant state with zero variance, are excluded for the reason given there: the common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

A standardised STEC product on the stream remains open: the in-process channel is a closed-loop store hand-off, not an IGS-SSR or RTCM message that a user of a third-party stream can decode.

18.9 The user of the network's corrections

The network's products reach a user engine in three ways: the in-process closed loop, in which the network's emit seam fills an owned store each epoch; the offline replay of a recorded product file (`network.closed_loop.replay_products_path`); and, in a real-time run, a tail of a growing product file (`realtime.network_products_path`). The first two are described in Section 20.11. Each gives the engine an owned `SsrCorrectionStore` (`RealTimeEngine::setSsrStore`) in place of the process-wide one.

The configuration of the user is derived from the configuration of the network by `GnssConfig::deriveClosedLoopConsumerConfig`. The standalone profile `processing.network_consumer` applies the same couplings (`applyNetworkConsumerCouplings`). The derived configuration differs from that of the network in these respects.

- The processing mode is the single-receiver PPP-RTK user. Estimation of satellite clocks, satellite phase biases, satellite code biases and the network ionosphere is switched off, because the corrections supply them.
- Residual satellite code biases are estimated only when the user does not take the network clock (`estimate_sat_biases` is the negation of clock consumption). When no satellite-bias state is estimated, the variance of a code row on a signal that neither a `.BIA` entry nor the stream covers is multiplied by the ordinary user factor 100 (`no_bia_code_downweight`), in place of the weight the network gives it.
- The undifferenced user applies no observation differencing. The phase biases are applied to the raw float (303), and AR forms between-satellite differences at fix time, which cancels the receiver phase bias. AR is direct per-band MLAMBDA without the Melbourne-Wübbena wide lane (`ar_wl_only` false, after which the driver sets `ar_ignore_mw_wl`), and starts only once the position σ is below 0.2 m. The reference configurations run this user float (`network.closed_loop.ar_mode` 0).
- The bias staleness gate is armed under consumption unless the configuration sets the key explicitly.
- Settings that act on the network's own solution (the canonical-signal rule, the extra-band code weight, the second code and phase channels) are pinned to neutral values, so that a case moves the network and leaves its user unchanged.

The user's answers to the network's declarations are the phase-bias reactions of Section 18.6 and the datum-break reactions of Section 19.10.

18.10 Provenance and code pointers

Both transports map onto the same internal correction structures (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) and a separate phase-bias block (1265–1270). IGS-SSR `4076` repackages the same payloads under a message indicator (IM). Its *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$), and its *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA). This subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). A `4076` message uses one set of field widths for every constellation (6-bit satellite identifier, 8-bit IOD, 20-bit GPS-aligned time of week), where the legacy blocks use per-constellation widths. Applying the legacy Galileo or BeiDou width to a `4076` message would over-read the IOD by two bits and shift every later field. RTCM counts SBAS satellites from 1 at PRN 120, and the decoders name

identifier n as S($n+19$), so PRN 120 is S20 as in RINEX and SBF (`rtcm/RtcmSatId.hpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier before the header is parsed, so they return `ok = false` with an empty header. For this reason the ionospheric term receives no SSR contribution from the stream (the `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The correction definitions Equations (298), (300) and (301) are implemented in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, the interface this document describes. The stateful streaming layer (the mutex-guarded per-satellite cache with freshness gating on receive time, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections) is described in Section 19. It is kept separate so that the correction mathematics can be analysed and tested in isolation. The estimator-side warm-up gate `estimation.defer_until_corrections_ready.enabled` (`config/GnssConfigLoad.cpp`; default OFF, real-time only) defers estimation until a quorum of satellites is centimetre-grade from broadcast plus SSR (by default at least 5 in total and 4 per constellation). The filter then seeds code and phase together, instead of cold-starting on metre-grade broadcast clocks and fixing a contaminated ionosphere/bias/ambiguity partition.

The store also carries a per-epoch *heartbeat* of the network (`NetworkProducerStatus`). It lets a user distinguish "the network withheld this epoch" from "the network stopped": a withheld epoch still writes its status line, although it publishes no corrections. The token set of the product file is forward-compatible: an unrecognised token from a newer network maps to `State::Unknown` and is reported, never fatal (`NetworkProducerStatus::parseState` / `stateName`, declared in the store's header). The token `withheld_sh` is retained so that archived product files parse; the spherical-harmonic health gate that wrote it was deleted from the network, and no current product contains it.

Part VIII
Real-time engine

19 Real-time engine

The real-time path reuses the estimator, measurement model and state manager of post-processing and reorganises the control flow around a *streaming* contract. Complete RTCM 3 (or SBF) frames arrive one at a time from a file or a network socket and are decoded into observations, corrections and broadcast ephemeris. A buffered epoch passes through the Section 12 predict/update cycle as soon as its boundary is detected. Post-processing iterates a closed list of epochs in time order with the full orbit and clock product set on disk; the engine sees each epoch once, receives corrections that may lead or lag the observations, and cannot look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. The engine takes its input through the frame feeds `feedRtcmFrame` and `feedSbfFrame`; at stream end `flush` processes whatever epochs are still buffered. This section covers the parts specific to that path. The estimation mathematics, the observation equations and the SSR correction model are documented elsewhere and are only cross-referenced here.

19.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services the sources in turn without blocking on any one of them:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches the complete frames to the engine (`pumpFrames`). The primary drain is capped at 1 MiB per iteration, and the corrections and ephemeris streams are read one 4 KiB chunk per iteration, so a high-rate observation stream cannot starve the sparse SSR and navigation streams. When a live network-product tail is configured, the loop also polls it at about 5 Hz and wakes the obs/SSR hold buffer of Section 19.7 when new products arrive. The loop exits when every source is inactive (file EOF or client stopped), when a stop is requested (Ctrl+C or the console-close handler), or when the `processing.max_duration_s` data-time budget latches. On exit the engine flushes the final buffered epoch before teardown, so no observations or trace rows are lost.

A socket source counts as *active* while its client is running, whether or not it is connected at that moment. A transient drop or the pre-connect window at startup therefore does not end the session; the client reconnects in the background.

19.2 NTRIP and TCP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and appends the received bytes to a mutex-guarded buffer with a hard cap of 4 MiB. When the cap is reached the buffer drops its oldest bytes. Callers read through a blocking `read` or a timeout-bounded `tryRead`; the multi-stream loop uses the latter, so one stream cannot block the others. The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0` and a `User-Agent`), and the mountpoint is the URL path. HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured. An optional NMEA GGA sentence for VRS mountpoints is sent as the NTRIP v2 `Ntrip-GGA` request header on the GET; the request is never turned into a POST.

Transport security. The transport defaults to plain HTTP, which serves plain-HTTP casters such as IGS-IP (`igs-ip.net`) unchanged. A `use_tls` flag selects the `https` URL scheme and sets `CURLUSESSL_ALL`, as required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER`/`VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off, so a plain-HTTP run is unaffected.

Reconnection. The receive loop reconnects automatically. After each transfer ends it classifies and logs the outcome: a transport error, a non-200 HTTP status (with hints for 401, 403 and 404), a connection that delivered zero bytes (an idle mountpoint, or a VRS mountpoint that needs a GGA position), or a clean stream end. It then always waits `reconnect_delay_s` plus a uniform 0–2000 ms jitter before retrying, so a misconfigured mountpoint appears in the log as an error and

several streams dropped by one caster outage do not reconnect in lockstep. An NTRIP v1 caster answers "ICY 200 OK", which libcurl reports as status code 0; the loop treats code 0 as success.

Raw TCP. `TcpClient` (`src/core/realtime/TcpClient.cpp`) serves a receiver that pushes RTCM or SBF over a TCP server socket. It follows the pattern of the NTRIP client: a background thread appends the received bytes to a mutex-guarded buffer with the same 4 MiB cap, and the poll loop reads through `tryRead`. After a closed connection or an error it redials after `reconnect_delay_s` (default 5 s, without jitter). A receiver that stops sending without closing the socket (a hung output task, or a dropped route that leaves no reset) would keep the stream silent for good. With `realtime.tcp_silent_timeout_s` greater than zero, a connection that delivers no byte for that long is closed and redialed through the same path. The corrections and ephemeris streams have the same key in their own blocks (`realtime.corrections`, `realtime.ephemeris`). The default is 0 (off, byte-identical), and a positive value has to exceed the longest interval between messages of the stream, which for an ephemeris-only stream can be a minute or more.

19.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames and so separates transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{306}$$

so the total frame length is $3+L+3$. `nextFrame` scans for the `0xD3` preamble (discarding other bytes), reads the 10-bit payload length (a length above 1023 bytes is rejected as a false preamble), waits until the full $3+L+3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. On a CRC mismatch the reader advances past the candidate preamble byte and resumes scanning, so a corrupt or misframed byte cannot stall it. Only CRC-valid frames are emitted, and the downstream parser (`RtcmParser`) relies on frame integrity. The reader counts extracted frames and discarded bytes for stream-health diagnostics, and stamps each frame with the steady-clock instant at which its bytes were handed over (`lastFrameReceipt`), which the engine uses only for the receive-to-process latency in its backlog warning.

Septentrio SBF. A parallel binary path decodes Septentrio SBF frames into the same observation and ephemeris outputs (`src/core/sbf/`). `SbfStreamReader` performs the `$@` (`0x24 0x40`) sync and the CRC-16/XMODEM check, and `SbfParser` dispatches blocks by ID and decodes the `MeasEpoch` observations. `Meas3Decoder` unpacks the compact, differentially encoded `Meas3Ranges` observations; GLONASS FDMA slots are resolved through `SbfSignalMap`. `SubframeAssembler` rebuilds broadcast ephemeris from raw navigation messages: GPS and QZSS LNAV subframes 1 to 3 with $\mathrm{IODE}_{SF2} = \mathrm{IODE}_{SF3} = \mathrm{IODC} \bmod 256$, Galileo I/NAV word types 1 to 4 with a common IODnav (word type 5 adds the BGDs and the health) and BeiDou D1 subframes 1 to 3; GLONASS raw strings are not assembled. This section covers only the framing and the feed seam (Sections 19.1 and 19.9). The `Meas3Ranges` differential range reconstruction is the direct analogue of (307) and (308) below and the one part of the SBF path that is algebra rather than ICD bookkeeping. It is *not* yet derived in this reference, and no other document derives it: this is a known gap. The remaining SBF detail (`SubframeAssembler` ICD scaling, the `SbfSignalMap` tables, the framing constants) belongs in the Doxygen headers and is out of scope here.

Galileo I/NAV pages. A 128-bit I/NAV word is rebuilt from the 234-bit page held in the eight navigation words of a `GALRawINAV` block. The receiver drops the 6 tail bits of the even half page, so the even half occupies page bits 0 to 113 and the odd half starts at bit 114 (even/odd flag and page type, then its 16 data bits at bits 116 to 131). The word is the 112 data bits of the even half (page bits 2 to 113) followed by those 16 bits. The CRC-24Q over the 196 protected bits (page bits 0 to 195) equals page bits 196 to 219 under this layout, and matches none with the odd data placed at bit 120. The IODnav lives in the even half. An odd half read 6 bits late therefore leaves the record self-consistent while its last 16 bits, which carry low-order bits of $\sqrt{A}$, IDOT, C_{rs}, a_{f1} and a_{f2}, are wrong, and the orbit is off by up to about 300 m, with the SSR correction applied on top of it (`docs/cards/realtime/INAV-ODD-PAGE.md`).

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the first 12 bits of the payload (the frame header carries only the length) and dispatches on it: 1005/1006 station coordinates; 1033 receiver and antenna descriptors and 1230 GLONASS code-phase biases (both logged, neither consumed); 4076 IGS-SSR; legacy SSR (1057–1068, 1240–1270); broadcast ephemeris (1019/1020/1042/1044/1046); and MSM4, MSM5 and MSM7 observations. Any other type is ignored and logged at trace level. The week that promotes a truncated epoch field to a full `GnssTime` comes from the parser's per-stream time tracker rather than from the host clock at each frame (Section 19.6). One parser instance serves the primary, corrections and ephemeris streams, so all three share one GPS timeline.

19.4 MSM observation decode

A single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) decodes every MSM observation message. It is parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Three variants are instantiated: MSM4, MSM5 and MSM7. The variant mechanism can describe MSM6, but no MSM6 entry exists, so MSM6 is not decoded. The header carries a 64-bit satellite mask, a 32-bit signal mask and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and is decoded into parallel vectors before recombination. RTCM counts SBAS satellites from 1 at PRN 120, and the decoders name satellite n as S($n + 19$), so PRN 120 is S20 as in RINEX and SBF (`rtcm/RtcmSatId.hpp`).

Range reconstruction. For each satellite the rough range in milliseconds is the sum of the 8-bit integer part (DF397) and the 10-bit modulo part (2^{-10} ms, DF398). Each cell adds a signed fine term with a variant-specific scale. With c the speed of light, the pseudorange and carrier phase of a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{P}_{\text{fine}}\, \sigma_P\Big), \tag{307}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{\Phi}_{\text{fine}}\, \sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{308}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 (and MSM5) and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). Cells holding the RTCM-defined invalid sentinel are skipped. Carrier phase is emitted in cycles, the RINEX-3 convention the observation model consumes (Section 4); when the wavelength λ cannot be resolved, the phase is stored in metres under the same observation code. The decoder also produces CNR (DF403 in whole dB-Hz for MSM4/5, DF408 in 1/16 dB-Hz for MSM7) and, for MSM5/7, a Doppler value from the rough+fine phase-range-rate (the negated rate over the wavelength). The half-cycle-ambiguity bit (DF420) sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. The 4-bit DF419 "extended satellite information" field determines the framing of the per-satellite block. In MSM5 and MSM7 it is present in the bitstream for every constellation, but only GLONASS assigns it a meaning (the FDMA frequency-channel number). The decoder therefore gates two operations separately:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6): when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

Reading DF419 only for GLONASS (or skipping it in MSM7) under-consumes the per-satellite block by $4\,N_{\text{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The rough range survives but the fine terms are biased, which produces an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination. These observations look plausible and pass an over-read check. Gating on the variant is the fix for the GPS L1/L2 geometry-free sign anomaly, and it matches RTCM 10403.3 and the cssrlib reference. As a second guard, the parser's `warnIfUnderConsumed` warns whenever a decode leaves ≥ 8 payload bits unread, which detects this class of misalignment where the bit reader's over-read flag does not.

19.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, from which the geometry layer resolves satellite orbits when no SP3/CLK file is loaded. The field order differs by message (QZSS, for example, leads with its clock block). GLONASS carries a sign-magnitude PZ-90 state vector, which is integrated separately.

SSR corrections. `RtcmSsrDecoder` decodes both the legacy SSR messages and the IGS-SSR 4076 container and pushes the records into the shared `SsrCorrectionStore` with their SSR header (provider epoch and IOD-SSR), so staleness and IOD diagnostics refer to the provider solution rather than to the time of receipt. The store holds orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records per satellite. The orbit and clock query paths read the store directly, so no per-epoch wiring is needed. The correction mathematics (the radial/along/cross orbit projection, the clock and bias application, the ionosphere model) is documented in Section 18 and is not repeated here. Each stream has an ingest policy (`realtime.corrections.corrections_accept`). The default `all` pushes every record. The policy `orbit_only` pushes orbit and URA records and only the orbit part of a combined record, and drops every record that carries clock or bias content, so that the clock of an RTS stream cannot shadow an injected network clock. Two reactions to the stream are specific to the real-time path:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity. By default the engine resets the affected ambiguity before the same epoch's measurement update (`StateManager::noteSsrDiscontinuity` detects the counter change). Two opt-in settings replace the reset: the declared-step carry `processing.ssr_phase_bias_disc_carry` and the step translation `processing.ssr_phase_bias_step_translate` (which also covers undeclared steps and takes precedence). Both shift the ambiguity by the phase-bias step instead of re-floating it (Section 18.6);

- every SSR push re-attempts a held observation epoch (Section 19.7), because the correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 adds a 16-bit antenna height in the same unit. A reference station that streams its own coordinates thereby supplies an a-priori seed (Section 19.9).

19.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields, which are promoted to a common GPS-time `GnssTime`. Each parser instance keeps a tracker, in continuous GPS seconds, of the last decoded stream epoch. It promotes every mod-week or time-of-day field by *nearest continuation* to that tracker: the week (or day) that minimises $|\Delta t|$, as in RTKLIB's `adjweek` (`RtcmParser::anchorGpsSeconds`). On this path the host clock (`TimeUtils::nowContinuousGpsSeconds`, POSIX UTC plus the leap offset, which `realtime.leap_seconds` overrides) is read only at cold start, on the first time-bearing frame after construction, so a skewed host clock or a race at a week rollover cannot move an established stream to another week. Each successful MSM or SSR epoch advances the tracker (`updateTimeAnchor`). The Keplerian broadcast-nav decoders read the tracker for their week but do not advance it. The anchor below is the tracker's current value. The 1020 GLONASS ephemeris TOE is dated from the same tracker; a 1020 that arrives before any MSM or SSR frame has set it is held and dated by the first such frame (see the GLONASS item below).

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds of week. BeiDou adds the constant 14 s BDT→GPS offset and re-wraps the result into the week.
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week followed by a 27-bit time-of-day in ms. Reading only 27 bits misaligns the satellite mask and corrupts the whole GLONASS decode. `TimeUtils::glonassTodToGpsTime` promotes the ms-of-day: it places the reported time-of-day in the GLONASS = UTC(SU)+3 h ("Moscow") day nearest the stream-time tracker and converts back to GPS time, re-adding the leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{309}$$

  where t^{0}_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\mathrm{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match (± 12 h). The 3-bit day-of-week is read but not used, since the anchor resolves the day. The same conversion promotes any GLONASS SSR epoch against the tracker. It also promotes the 1020 ephemeris `toe` against the same tracker, so a file replay of historical data keeps each GLONASS ephemeris on the day of its observations. A 1020 that arrives before any MSM or SSR frame has set the tracker has no stream epoch to be dated from. It is held (at most 48 records) and dated by the first such frame; an input that carries navigation messages only fills the hold, which is then dated from the host clock.
- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\mathrm{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per satellite. The decoder maps DF419 to the channel as $k = v - 7$ for the valid range and uses it in Equation (308). The channel is also stored on the observation, so the slot map and the MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or holds a reserved value, the channel is marked unknown and the phase is stored in metres; a default of zero would decode to a false $k = -7$.

Each decoded record carries its own content epoch (`epoch_time`). The synchronisation logic of Section 19.7 compares against that epoch, not against the time of arrival.

19.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections rather than with the host clock. In a live session the observation stream commonly runs ahead of a lagging SSR stream. An observation epoch that no satellite's SSR can cover is processed entirely at broadcast grade (and, with `drop_broadcast_grade`, loses every satellite). The opt-in `processing.enable_obs_ssr_sync` hold buffer prevents this. With t_{obs} the observation epoch, the SSR coverage horizon is the freshest stored SSR content epoch (`SsrCorrectionStore::newestSsrEpoch`) plus the maximum SSR age:

$$t_{\mathrm{horizon}} = t_{\mathrm{SSR}}^{\mathrm{newest}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{310}$$

The decision is:

- $t_{\mathrm{obs}} \le t_{\mathrm{horizon}}$: SSR covers the epoch. The epoch is processed normally and any hold is cleared;
- $t_{\mathrm{obs}} > t_{\mathrm{horizon}}$: the epoch is *deferred*. The hold buffer keeps only the latest such epoch (skip-to-latest semantics), and `drainHeldObs` re-attempts it as soon as an SSR push (or a new network-product epoch, through `notifyProductsAdvanced`) advances the horizon.

A liveness backstop keeps a dead SSR feed from starving the filter. If a hold persists past `obs_ssr_sync_max_hold_s` of host-clock time, the engine latches a degraded "outage" state and passes every epoch through (at broadcast grade, or dropped by the existing path) until SSR covers the observations again. The held epoch is drained with skip-to-latest *bypassed*, so the SSR-aligned epoch is not discarded. The final `flush` bypasses both the hold and skip-to-latest, so the last epoch is always processed. The feature is inert when disabled, before the first epoch, and in any run without a live SSR orbit and clock source.

A separate, coarser `enable_obs_skip_to_latest` drops observation epochs whose lag behind the *host-clock* GPS time exceeds `obs_skip_max_lag_s` (default 5 s), so a backlogged stream fast-forwards to the freshest epoch. The skip returns before any continuity state is touched, so the next processed epoch measures its time step from the last accepted one and trips the gap path (Section 19.10) when the skipped span exceeds the gap tolerance.

19.8 Ephemeris management

`LiveEphemerisStore` keeps, per satellite, the current record and one previous record (the current record is demoted to previous when the IODE changes), and evaluates the orbit and clock against the appropriate one. Its policies in real time are the following.

- **Freshest-toe selection.** An incoming record is rejected when its `toe`, expressed in continuous GPS seconds (`toeGpsContinuous`, which reconciles the BDT and GLONASS time scales), is older than the held record's by more than 1 s. A stale or re-ordered broadcast therefore never overwrites a newer one. A record from the IGS stream is also protected from a receiver-decoded record at equal `toe`: the IGS-RTS SSR deltas are computed against the IGS broadcast, and a receiver ephemeris with different fit parameters would make the pairing inconsistent. Receiver nav replaces an IGS-stream record only when its reference epoch is at least 1800 s newer (the GLONASS nominal `toe` step), which indicates that the IGS stream has stalled on that satellite.

- **toe-aware and IOD-aware selection.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`). For an SSR-corrected satellite it first selects the record the SSR delta was computed for (`pickForSsr`, which matches the SSR IODE against the current and the retained previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would drop the satellite to broadcast grade.
- **Fit-window guard.** A satellite whose selected record has $|t - t_{oe}|$ beyond 4 h (GPS, QZSS), 2 h (Galileo, BeiDou) or 0.5 h (GLONASS) is skipped with a warning instead of receiving an extrapolated position. The bound is about the broadcast fit interval with a margin of about two.
- **SSR matching convention.** For the IOD gate of Section 18.3 the store converts the broadcast issue of data (`LiveEphemerisStore::ssrIode`): the IODE for GPS and QZSS, the low 8 bits of the Galileo IODnav, and $\lfloor t_{oe}/720 \rfloor \bmod 240$ for BeiDou.

19.9 A-priori seeding from RTCM 1005/1006

A *live* SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record (a file-fed source does; see the third item below). Without an a-priori position the engine falls back to a coarse Earth-centred seed at the geographic pole. `seedInitialState` calls `PppModel::initializeState` and then applies the fallback, since a state write before that call would be overwritten. The model it calls is built by `initProcessingComponents`, so `seedInitialState` must follow it in the construction order. From there almost every satellite sits below the elevation mask, the first `computeModels` yields no usable satellite, WLS never runs and the filter cannot bootstrap. The engine therefore seeds the a-priori position from the stream itself until the Kalman filter owns the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`);
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed. This makes `receiver.reference_position` optional for any station that streams 1005/1006. An optional blunder gate (`realtime.arp_blunder_gate_m`; 0, the default, disables it) rejects a broadcast ARP that disagrees with an existing a-priori by more than that distance, so a wrong mountpoint or stale 1005 coordinates cannot move the seed;
- from the RINEX header of a *file-fed* source, reduced from the marker to the antenna reference point through `ANTENNA:DELTA H/E/N` (`RinexParser::getArpPositionXyz`, which calls `Coordinates::markerToArp`). The position state, and the ATX PCO/PCV applied to it, refer to the ARP rather than the monument, so seeding on `APPROX POSITION XYZ` unreduced would start the filter one eccentricity away. The reduction is an exact no-op for a zero or absent eccentricity, so a live SBF/RTCM stream is unaffected; it matters only for the file-fed closed-loop user, whose header carries a real eccentricity. The reduction is not applied twice: `PppModel::initializeState` reduces from the *raw* marker, and only while the state position is still zero or non-finite.

A configured `reference_position` always takes precedence. The seed is applied only while `first_epoch_` is set. The startup gate keeps that flag set through warmup, so a 1005/1006 message that arrives a few seconds into the stream (these messages are sent less often than the 1 Hz observations) still seeds before the first estimation epoch. The flag clears in the first epoch that passes the startup gate, after measurement conditioning and before the predict (Section 19.10). From then on the filter owns the position, and a decoded ARP or PVT is only logged for comparison.

19.10 The per-epoch real-time update

`processEpoch` runs the predict/update pipeline of post-processing, with streaming-specific guards in front and the post-processing-only stages removed. It is a thin ordered driver. Each step is a private member function (a *seam*), with one file per concern: admission (`RealTimeEngineAdmit.cpp`), the frame ingest (`RealTimeEngineIngest.cpp`), construction (`RealTimeEngineInit.cpp`), measurement-set preparation (`RealTimeEnginePrepare.cpp`), slip QC (`RealTimeEngineSlipQc.cpp`), SSR datum-break reactions (`RealTimeEngineSsrBreaks.cpp`), the pre-update innovation screen (`RealTimeEngineScreen.cpp`), estimation (`RealTimeEngineEstimate.cpp`), observability (`RealTimeEngineSsrDiag.cpp`) and the filter-state snapshot (`RealTimeEngineStateSnap.cpp`). The ordered driver itself is `RealTimeEngine.cpp`. Seams exchange per-epoch values through `RtEpochContext`, which holds nothing beyond the epoch; a value that must survive to the next epoch is a member of the engine. The four admission gates are the only seams that may end an epoch early. The driver itself also ends an epoch when no observation of an active system remains or when `computeModels` fails; after the startup gate every step runs unconditionally. An optional staged filter-state snapshot (warm resume, Section 19.14) is restored before the gates, so the first time step spans the actual outage. In order:

1. **Lag, sync and duration gates.** The host-clock backlog warning, the obs↔SSR hold of Section 19.7 and the skip-to-latest drop form the first gate (`admitBacklogAndSync`); the `max_duration_s` latch is the second (`gateMaxDuration`). Both run before any continuity state is touched.
2. **Step and gap detection** (the third gate, `stepTimeAndDetectGap`). $\Delta t = t_k - t_{k-1}$, with a week-rollover correction; the first epoch assumes 1 s. A non-advancing $\Delta t \leq 0$ is a duplicate (a second co-timestamped SBF block or a retransmit) and is skipped with the state untouched. Folding it in would double-count the measurements and pass $\Delta t = 0$ to the process-noise build, which throws. A Δt above the gap tolerance flags a gap. The tolerance is `processing.data_gap_tolerance_s` when set. When unset it is 1.5 times the median of the most recent nine spacings of accepted epochs (`EpochCadence`), as in post-processing, but never below 1 s: that is also the value until a first spacing has been measured, and a stream of 1.5 Hz or faster keeps it. A spacing is judged before it joins the window.
3. **Models and startup gate** (the fourth gate, `gateCorrectionsReady`). Observations are filtered to the active systems and run through `computeModels`. The signal roster that the network declares in its product, when the stream carries one, is handed to the model first (`refreshDeclaredRoster_`), because the model decides a constellation's canonical signals on its first epoch; on a stream without a declaration this does nothing. When `estimation.defer_until_corrections_ready.enabled` is set, estimation (no predict, no update) is deferred until enough cm-grade broadcast+SSR satellites are present: 5 in total and 4 per active constellation by default, the per-constellation floor reduced to what the constellation currently fields but never below two. A timeout (`timeout_s`, default 0: none) forces a degraded start. The filter then initialises code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Measurement-set conditioning.** After two observability sweeps (the SSR coverage log and the SSR trace of the user), `dropBroadcastGrade` removes broadcast-grade satellites when the startup gate's `drop_broadcast_grade` is set. In modes with a receiver-clock state, `seedClockFromWls` replaces the filter clock with a WLS clock estimate when the two differ by at least 5×10^{-5} s (`kJumpThresh`, 0.05 ms, about 15 km), after a gap, or on the first epoch. Under BSSD (no receiver-clock state) `regaugeBssd` subtracts the median of the epoch's code innovations, one

receiver clock common to all satellites, from every code and phase observable. This common-mode shift leaves the single differences exactly unchanged and keeps the per-satellite bias states at DCB scale instead of absorbing the full unsteered receiver clock. The clock seed and the BSSD re-gauge are mutually exclusive: the first requires a receiver-clock state, the second its absence. The first-epoch flag (`first_epoch_`) clears at the end of this step.

5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$, the aiding path shared with post-processing; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of Section 12.
6. **Cycle-slip detection and repair** (`detectAndRepairSlips`). The MW detector, the optional TurboEdit MW+GF joint repair and the soft-LLI gap bridge run before the update, with the detectors and repair machinery of Section 15. They are enabled under `estimation.cycle_slip`. TurboEdit is for high-rate data only; its GF-arc gap tolerance is 1.5 times the configured stream rate (1 Hz if none is set), because a live stream has no RINEX interval header.
7. **SSR datum-break reactions.** These react to the network moving its datum, or to the broadcast baseline moving under the corrections, and do nothing without SSR content. The order is fixed. A change of the SSR solution-generation id (`reactToGenerationChange`) resets every ambiguity and re-inflates the receiver code-bias states; it runs first because it subsumes the per-signal reactions. `logProducerStatus` logs a status transition of the network. `resetOnPhaseBiasDisc` claims held phase biases first and then resets (or, with the carry or translation settings, shifts) the ambiguity of a signal whose phase bias shows a discontinuity or a freshness flip, as described in Section 18.6. `reinflateOnCodeBiasDisc` re-inflates the receiver code-bias states of a constellation on a code-bias datum re-reference, and `reinflateOnClockDisc` re-inflates the affected inter-system bias states on a clock datum re-reference. Then the opt-in declared-level-step reaction `reactToDeclaredLevelSteps_` (`processing.ssr_level_step_inflate`, default off) compares each satellite's network-declared clock and code-bias levels with their previous values. When the resulting step in a code signal's level exceeds a 0.1 mm dead band, it inflates the covariance along the direction that moves the modelled code but not the modelled phase (the satellite's slant ionosphere together with its ambiguities, each ambiguity weighted by the squared ratio of the GPS L1 frequency to its signal frequency). The added standard deviation is the step size times a configured scale. A satellite without an ionosphere state or an ambiguity is refused. The reaction failed its bars and is off in the reference configurations (`config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`; ledger verdict KEEP_OFF). Last, the opt-in broadcast-baseline jump response `reactToEphemerisJumps_` (Section 19.13).
8. **Measurement update** (`runMeasurementUpdate`). With `network.closed_loop.innovation_screen` the first pass is built by the innovation screen of Section 19.11, with the rows that it holds out excluded. The outlier and forced-pivot-handover rebuild loop of the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear the MW/TurboEdit arc history of KF-reset signals, and remove one outlier per pass. Phase signals held by the soft-LLI bridge are excluded from the update.
9. **Pseudo-observation constraints.** After the update and before AR, so the reduced covariance reaches the resolver. An optional soft a-priori ZWD pseudo-observation (`applyZwdConstraint`, `estimation.zwd_constraint`) anchors the BSSD [up, ZWD] near-null subspace. When the engine consumes a network ionosphere product, `applyNetworkIonoPrior` constrains each satellite's ionosphere state with a slant-ionosphere prior from the same epoch, with an error block common to each constellation.
10. **Ambiguity resolution** (`resolveAmbiguities`). After a successful float update, MLAMBDA

AR runs on the committed state with the configuration surface of post-processing and is fed the MW-resolved wide-lane integers (Section 16). The closed-loop network-bias validation suppresses the wide-lane integers, so AR runs on the float state alone.

11. **State extraction and trace** (`writeEpochTraces`). The posterior is synced back to the PPP state and the trace rows are written. The optional ambiguity-level check (Section 19.12), the per-satellite readout of the user (`network.trace_consumer_sat`, measure-only) and the periodic filter-state snapshot (Section 19.14) follow.

Post-processing stages skipped in real time. Non-causal and batch stages do not fit a streaming pipeline without look-ahead and are absent:

- the backward RTS Section 12 is disabled under `AppMode::RealTime`, because smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source replaces them (a real-time *file* replay without corrections still loads them). Orbit and clock are gated separately, so a configuration with precise orbits and network clocks loads the SP3 file and skips the CLK file;
- the stages that need a GIM have no source in the stream: higher-order ionosphere, GIM ionosphere weighting and adaptive ionospheric process noise are skipped with a warning when enabled.

19.11 Pre-update innovation screen

The measurement update spreads one gross row over every state the row touches, and the post-fit residuals are computed after the state has absorbed the error. The screen tests the innovations against the predicted state instead, while that state is still unspoiled. The statistic, the sequential removal, the episodes and the counters are in `realtime/InnovationScreen` (pure arithmetic); the engine side is `RealTimeEngineScreen.cpp`. The screen is enabled by `network.closed_loop.innovation_screen` (derived into `processing.innovation_screen`; default OFF and byte-identical). It runs inside `runMeasurementUpdate`, after the predict, the slip detection and the datum-break reactions, and before the first update pass.

Statistic. Let $\boldsymbol{H}$ be the design matrix of the first pass, $\boldsymbol{P}^{-}$ the predicted covariance, $\boldsymbol{R}$ the measurement noise and $\boldsymbol{\nu}$ the innovation vector. Only undifferenced single-satellite code and phase rows are eligible (`processing.innovation_screen_rows`: `all`, `phase` or `code`). Differenced rows, aiding rows and combinations are marginalised out of the covariance, so a fault in one of them cannot make an eligible row look wrong. The innovation covariance of the m eligible rows is

$$\boldsymbol{S} = \boldsymbol{H}\,\boldsymbol{P}^{-}\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R} + \mathrm{diag}(f_i^2), \tag{311}$$

with f_i a model-error floor per row class (`processing.innovation_screen_phase_floor_m`, default 0.3 m, and `processing.innovation_screen_code_floor_m`, default 1.0 m). The formal noise is millimetres on phase while the innovations of a healthy user spread over decimetres, so without the floor the statistic flags ordinary rows. The robust inflations that the filter applies inside the update are not in $\boldsymbol{S}$. Row i is tested against what all other rows imply for it,

$$w_i = \frac{(\boldsymbol{S}^{-1}\boldsymbol{\nu})_i}{\sqrt{(\boldsymbol{S}^{-1})_{ii}}}, \qquad \sigma_i = \frac{1}{\sqrt{(\boldsymbol{S}^{-1})_{ii}}}. \tag{312}$$

The ratio $(\boldsymbol{S}^{-1}\boldsymbol{\nu})_i/(\boldsymbol{S}^{-1})_{ii}$ is the innovation of row i minus its prediction from the other rows, and σ_i is the standard deviation of that difference. Without a fault w_i is standard normal, and a gross error d on row i alone shifts it by d/σ_i. The marginal form $\nu_i/\sqrt{S_{ii}}$ is not used. A receiver clock with random-walk noise and a free position put tens of metres into every S_{ii}, which hides any fault below that size, while the conditional form lets the other rows fix the common clock, geometry and troposphere first. A row whose own state is loose (a new ambiguity) has a large σ_i and cannot trip, since it carries no prior to contradict.

Sequential removal and abstention. The eligible row with the largest $|w_i|$ is flagged when it exceeds k (`processing.innovation_screen_k`, default 6). With $\boldsymbol{T} = \boldsymbol{S}^{-1}$, $\boldsymbol{g} = \boldsymbol{S}^{-1}\boldsymbol{\nu}$ and j the flagged row, the removal updates both by the rank-one downdate

$$T_{ab} \leftarrow T_{ab} - \frac{T_{aj}\,T_{jb}}{T_{jj}}, \qquad g_a \leftarrow g_a - \frac{T_{aj}\,g_j}{T_{jj}}, \tag{313}$$

and the test repeats until no remaining row exceeds k, so that two gross rows cannot hide each other in the one-pass statistic. A screen that would exclude most of an epoch has decided that the prior is wrong, so it abstains: nothing is excluded, the epoch goes to the update and the post-fit tests as it would without the screen, and the epoch is counted with its reason. It abstains when more than `processing.innovation_screen_max_fraction` (default 0.25) of the epoch's rows would be flagged, when fewer than six rows would remain, or when $\boldsymbol{S}$ is not positive definite.

Exclusion as an event at both edges. In the mode `act` (`processing.innovation_screen_mode`) a flagged row is held out of the update through the exclusion keys that the measurement model reads when it builds the pass; in the mode `observe` everything is computed and counted and nothing is written. The decoupling edge is the event *Exclude*, with the identity of the row, w_i, σ_i, the innovation and the number of consecutive flagged epochs. The re-coupling edge is a re-validation: every epoch tests every row again, including the rows excluded at the epoch before. An exclusion episode ends in exactly one way.

- *Readmit*: the key passes the test again and its row enters the update.
- *Refloat*: a phase key has been flagged on `processing.innovation_screen_refloat_ep` (default 3) consecutive epochs. A persistent step cannot heal while its ambiguity is held out of the update, so the ambiguity is re-floated through the first-sighting reset (`StateManager::resetAmbiguity` at the configured ambiguity σ, with the Melbourne-Wübbena and TurboEdit arcs of the signal forgotten) and the row enters the update in the same epoch. A code row has no state to heal and stays excluded while it fails.
- *Lost*: the key leaves the epoch's row set while excluded.
- *Released*: the screen abstained, and every open exclusion ends with its row in the update.

At run end the counters close: episodes opened equal readmitted plus re-floated plus lost plus released plus still open, and every screened epoch with eligible rows is exactly one of clean, flagged or abstained. The counters also give the epochs by their largest $|w_i|$ in eight buckets (below 2, then edges at 3, 4, 5, 6, 10 and 100), which shows the false-flag rate of a healthy run before the screen is allowed to act.

The screen is built and not armed (ledger verdict UNVALIDATED). On the real-data reference configuration it flagged no row. At the 16 further test stations it flagged only one Galileo E5a code row that the update already leaves out, and a 92 m step on one phase row moved the position by at most 1.5 cm, so no benefit was shown (`docs/refound/NIS_consumer_innovation_screen_2026-10-17.md`, section 9).

19.12 Ambiguity-level check

A carried or translated ambiguity (Section 18.6) is continuous, and nothing establishes that it is correct. The check re-validates the float ambiguities of a dual-frequency satellite against the level of its own code (`AmbLevelCheck`, `RealTimeEngine::checkAmbiguityLevel_`; `processing.amb_level_check`, derived from `network.closed_loop.amb_level_check`; default OFF). It runs after the update, the constraints and AR. It reads the committed pre-fit innovations of the two lowest bands of a satellite, $i_{\phi 1}, i_{\phi 2}$ on phase and i_{c1}, i_{c2} on code, in which the satellite clock, code bias and phase bias are already applied. Two combinations measure the inconsistency between the code level and the float ambiguities:

$$m_{\mathrm{WL}} = \frac{1}{\lambda_{\mathrm{WL}}}\left[\frac{f_1 i_{\phi 1} - f_2 i_{\phi 2}}{f_1 - f_2} - \frac{f_1 i_{c1} + f_2 i_{c2}}{f_1 + f_2}\right] \quad \text{(cycles)}, \tag{314}$$

$$m_{\mathrm{IF}} = \frac{f_1^2(i_{\phi 1} - i_{c1}) - f_2^2(i_{\phi 2} - i_{c2})}{f_1^2 - f_2^2} \quad \text{(m)}. \tag{315}$$

Both are free of geometry, troposphere and first-order ionosphere: the coefficients of each combination sum to one on the geometry, and the ionosphere cancels between the phase and code parts. A phase-only receiver clock and errors of the corrections do not cancel, so a trip identifies an ambiguity error only together with other evidence.

Each epoch adds one sample per satellite to a window of `processing.amb_level_check_window_ep` samples (default 20). When the window is full, the check trips if

$$|\bar{m}_{\mathrm{WL}}| > k\,\sigma_{\mathrm{WL}} \quad \text{or} \quad |\bar{m}_{\mathrm{IF}}| > k\,\sigma_{\mathrm{IF}}, \qquad \sigma = \max\!\big(s/\sqrt{n},\ \text{floor}\big), \tag{316}$$

with $k =$ `processing.amb_level_check_k` (default 4), $s/\sqrt{n}$ the standard error of the window mean, and floors of 0.02 cycles (WL) and 0.05 m (IF). The WL test runs first. The σ is a floored estimated standard error and not a calibrated ambiguity uncertainty: for independent zero-mean Gaussian samples and inactive floors the ratio follows Student's t with $n-1$ degrees of freedom, and temporal correlation and the changing state need separate calibration. The window restarts whenever the ambiguity of either band is born, reset, reseeded or translated (the generation tokens of `StateManager`), or when a state is missing or at the reset variance, so an epoch that holds a translation cannot trip. A trip empties the window. In the mode `act` (`processing.amb_level_check_mode`) both ambiguities of the satellite are re-floated through the reset path of the datum-break reactions; the mode `observe` only counts. Every full-window evaluation is exactly one of no trip, WL trip or IF trip. A between-satellite differenced user contributes no rows, and the check has no effect there. The setting is off in the reference configurations (ledger verdict KEEP_OFF): under the network's phase-bias hold the translations restart its window.

19.13 Broadcast-baseline jump response

The range model of the user starts from a broadcast record (orbit and clock) and adds the SSR correction. When the record that a satellite uses is replaced (a new issue of data, a same-IOD overwrite, or a switch between the IGS broadcast stream and the receiver's own navigation message), the modelled range moves by

$$\Delta\rho = \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{p}_{\mathrm{new}} - \boldsymbol{p}_{\mathrm{old}}) - c\,(\mathrm{d}t_{\mathrm{new}} - \mathrm{d}t_{\mathrm{old}}), \tag{317}$$

with both records evaluated at the same epoch and $\boldsymbol{u}$ the unit line of sight from the receiver to the satellite. Consecutive records of a healthy satellite agree to well under a metre. A large $\Delta\rho$

means that one record was wrong and that the ionosphere, satellite-bias and ambiguity states of the satellite absorbed the error of the old one; a Galileo record assembled at the wrong page offset (Section 19.3) is an example.

`EphemerisJumpLedger` remembers the record that each satellite used last and reports a change of its orbit or clock content with $\Delta\rho$. Only satellites whose orbit came from the broadcast store in this epoch are tested. A change is exactly one of undecided (the old record is beyond its fit window), below the threshold, or a jump. The response `reactToEphemerisJumps_` acts on a jump above `processing.ephemeris_jump_threshold_m` (default 10 m) when `processing.ephemeris_jump_reseed` is set (default OFF: measured and logged, nothing written). The state values are kept and their covariance is broken: the variances of the ionosphere of the satellite (and of its rate) and of every satellite-bias state are raised to at least $\Delta\rho^2$, and every ambiguity of the satellite is reset at a standard deviation of at least $|\Delta\rho|$. For the between-satellite pivot of a constellation, every other satellite of that constellation that holds a state is re-seeded instead, because each single-differenced state carries the range of the pivot. A configuration with precise orbits and live clocks (`realtime.use_precise_orbits`) is not covered, because the orbit of its satellites is not from the broadcast store. The response is not armed: over a 6 h real replay it saw 377 record changes, the largest 0.83 m, and wrote nothing, and it is not yet graded on a real jump (`docs/cards/realtime/EPH-JUMP-RESEED.md`).

19.14 Filter-state snapshot and warm resume

The engine can write its filter memory periodically and resume from it after a restart, so that the first hours of convergence are not repeated (`realtime.state_snapshot_path`; empty, the default, switches the whole seam off and is byte-identical). The file (`realtime/RtStateSnapshot`) is line-tagged text with round-trip-exact doubles, strict parsing, a record count at the end and an atomic write through a temporary file. It holds the state vector, the full covariance, the kind and key of every dynamic state, the solution-generation id and epoch counter, the discontinuity-counter memory of the state manager and the applied maps of the staleness gate. The last three are the continuity baselines of the corrections: they keep the datum-break reactions armed against everything the network did during the outage. The file is rewritten at the tail of an epoch every `realtime.state_snapshot_interval_s` of data time (default 10 s). A hash over the facts that fix what a state means (systems, signals, frequency mode, wind-up convention, correction routing, state shape and AR settings) guards the resume.

The resume is a single step at the first processed epoch, before the admission gates, because the hash depends on setters called after construction and the age needs a real epoch time. The snapshot is discarded, with a logged reason and a cold start, in these cases: the age $t_k - t_{\mathrm{snap}}$ is not positive (resuming at or before the snapshot epoch would double-count its measurements); the age exceeds `realtime.state_snapshot_horizon_s` (default 300 s, since every restored arc would carry unbounded exposure to slips in the gap); the hash differs; or the layout preconditions fail (a manager that is not core-only, a different core size, a duplicate key). Two scopes restore (`realtime.state_snapshot_restore_scope`). The scope `core` (default) restores the core block (position, clock, tropospheric delay, inter-system biases) and the baselines, and the dynamic states are re-created as on a cold start. The scope `full` replays the exact dynamic layout and restores the state and covariance wholesale; it resolves the kind of every dynamic row before it restores the first one, so a kind unknown to this build (a newer writer) is refused while the manager is still untouched and the cold start follows. In both scopes the previous-epoch time becomes the snapshot epoch, so the first predict inflates over the true outage and the first spacing is flagged as a data gap.

With `realtime.state_snapshot_seam_refloat` (default ON) the resume also answers an

unsignalled transient of a network that itself resumed warm. The variance of the tropospheric delay is re-inflated to its initial value with zeroed cross-covariances and the value kept, so that the transient can move into the delay as on a cold start, and the variances of the inter-system biases are re-inflated with the values kept. In the `full` scope every restored ambiguity is also marked for re-seeding. The snapshot does not hold the applied-value map of the phase-bias carry and translation (Section 18.6). The setting is UNVALIDATED in the ledger: it was built for warm-resumed closed-loop runs, and no benefit was graded on a continuous product stream (`docs/cards/consumer/CC-declared-step-carry.md`).

19.15 Assessing a real-time run

A streaming run has no answer key, unlike a simulated one. The self-referenced statistic of the ordinary state viewer (scatter about the run's own median) cannot see an error *common* to every epoch, and a correction-driven pipeline is most likely to make that kind of error. A static receiver on a surveyed mark supplies the missing datum. Let $\boldsymbol{r}_{\mathrm{ref}}$ be the surveyed antenna position (`receiver.reference_position_ecef`) with geodetic latitude and longitude (φ, λ), and $\hat{\boldsymbol{r}}_k$ the filter's posterior position at epoch k. The error is the ECEF difference rotated into the local horizon frame at the *reference* point,

$$\begin{bmatrix} e_k \\ n_k \\ u_k \end{bmatrix} = \boldsymbol{R}(\varphi, \lambda)\left(\hat{\boldsymbol{r}}_k - \boldsymbol{r}_{\mathrm{ref}}\right), \qquad \boldsymbol{R} = \begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\varphi\cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi \\ \cos\varphi\cos\lambda & \cos\varphi\sin\lambda & \sin\varphi \end{bmatrix}, \tag{318}$$

with $\varepsilon_k^{\mathrm{3D}} = \|(e_k, n_k, u_k)\|$ and $\varepsilon_k^{\mathrm{H}} = \|(e_k, n_k)\|$. The difference is taken in ECEF before the rotation, so the rotation origin cancels and (318) is exact rather than a local-tangent approximation.

Convergence must be suffix-stable. The first epoch whose error falls below a threshold measures the noise of a solution, not the solution. A float PPP-RTK trajectory dips under any loose bound long before it *holds* it, and a run that later diverges scores the same as one that does not. The convergence epoch for threshold θ is therefore defined as

$$k^\star(\theta) = \min\Big\{\, k : \max_{j \geq k} \ \varepsilon_j^{\mathrm{3D}} \leq \theta \,\Big\}, \tag{319}$$

the first epoch from which the error stays within θ for the *entire remainder* of the run ($k^\star = \varnothing$ when no such epoch exists, the correct verdict for a run that never settles). The suffix maximum is computed in one reverse cumulative pass. The suffix function counts a non-finite error as a violation, but the scorecard first drops every trace row whose position is non-finite, so an epoch without a position is skipped rather than breaking the hold. The metric is reported at the four thresholds $\theta \in \{1.0, 0.5, 0.2, 0.1\}$ m in place of a single number, which keeps it informative for runs of very different quality. The steady-state statistics are taken over $k \geq k^\star$ of the coarsest threshold achieved, so the convergence transient is excluded from the accuracy claim by construction; when no threshold is held, they fall back to the second half of the run and the summary flags the fallback. The window is stated alongside the number it produced.

Covariance credibility. Grading the solution against the surveyed point also tests whether the filter's own uncertainty is credible. With $\boldsymbol{P}_k^{xx}$ the position block of the posterior covariance, the rotation-invariant scalar

$$\sigma_k^{\mathrm{3D}} = \sqrt{\operatorname{tr} \boldsymbol{P}_k^{xx}} = \sqrt{\sigma_x^2 + \sigma_y^2 + \sigma_z^2} \tag{320}$$

is compared directly with ε_k^{3D}. The trace is invariant under the rotation (318), so no per-axis horizon σ has to be constructed from a trace that carries only the diagonal. A float solution whose error persistently exceeds $3\sigma^{3D}$ is *overconfident* as well as inaccurate. This is the usual signature of unmodelled correction error absorbed by ambiguity and bias states whose formal variances do not reflect it. The simulator dashboard applies the same covariance-optimism diagnostic against its answer key; here the surveyed mark takes the place of the truth trajectory.

What the reference does not certify. The surveyed point grades *position* only. A static run cannot separate a receiver-clock error from a common-mode range error, and under BSSD (Section 19.10) there is no receiver-clock state to grade. The per-satellite bias states are gauge-fixed, so their absolute values cannot be checked against an external reference either. A position that grades well therefore certifies the geometry and the correction stack *jointly*, not any single component. The real-time scorecard viewer (`pyviz/gnssviz/viewers/rt_scorecard.py`) of the report built by `pyviz/run_report.py --run-config` implements (318)–(320). An explicit `--ref-ecef` overrides the configured reference position.

Part IX
Network mode

20 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`), which *produces* the satellite-side corrections that the single-receiver user modes consume. The other modes in this reference either difference away the satellite clock and hardware biases (Section 5), take them as known inputs from precise orbit, clock and bias products, or import them as known SSR quantities (Section 18). The network estimator inverts that relationship. It estimates the satellite clock and the satellite *phase* bias as explicit states, and the satellite *code* bias as well when `network.estimate_sat_code_bias` is set (armed by both reference configurations; the loader default is off), jointly across a network of stations with known coordinates, and emits them in IGS-SSR convention for a downstream user. The loop is closed by design: the single-receiver PPP-RTK engine documented elsewhere is the product test, run on a *test station* held out of the network solve.

Scope and status. The numerical core consists of the arrowhead normal-equation kernel (`network/solve/NormalEquations`) and the semantic estimator that drives it (`network/solve/Estimator`). It is validated on a synthetic noise-free network, where it recovers the truth up to the declared S-transform (`tests/network/test_network_estimator.cpp`), and on real and simulated networks through the two reference configurations (`config/golden/golden_field_refound.json`: 58 stations of real data, day 129 of 2024; `config/golden/golden_sim_refound.json`: 60 stations of a simulated data set). Both are GPS L1/L2 and Galileo E1/E5a, dual frequency, with a free per-line-of-sight slant ionosphere (no global ionosphere map, no spherical-harmonic constraint), the satellite geometry-free code bias pinned at the broadcast group delay, the prioritized phase datum with a held basis, the float ambiguity vehicle, and the armed quality control: post-fit network RAIM with its attribution pass and within-epoch tier (Section 13.9), the gross line-of-sight screen, the measurement admission screen and the clock runaway check. The network owns its stations' parameter sets (Section 20.2) and builds every row through the one observation equation that also serves the single-receiver filter and the first-epoch seeds (Section 4.4). The pipeline around

the core is complete: the `PPP_RTK_Network` enum with the forced uncombined configuration of the PPP-RTK family, the explicit satellite phase-bias state (Section 20.3), the mode dispatch in `Application::run`, the setup calls and epoch source of `Application::runNetwork`, and one epoch as `Application::networkProcessEpoch_`, which calls a fixed, ordered sequence of single-concern seams (Section 20.1). Every step derived in this chapter runs inside one of those seams.

This chapter treats the single-epoch solve and its datum. The between-epoch carry has three layers; each is developed in its own document.

- The *shared border* (satellite clocks, phase biases, code biases) is always carried as a filtered solve. In the reference configurations the carried posterior is one dense information block $(\boldsymbol{P}_{\text{prev}} + \boldsymbol{Q})^{-1}$ (`network.shared_carry_full_cov`, `networkInjectD2CarryPrior` in `network/state/BorderPrior`). With that setting off, each key is carried as a diagonal prior of variance $\text{var} + q\,\Delta t$. A satellite clock takes the increment of the temporal clock anchor instead when `network.clock_temporal_anchor` is on (both reference configurations), and a phase or code bias variance is floored when a floor is set. A clock-pivot handover drops the carry of the constellation whose datum changed (`network.scoped_handover_response`), unless `network.handover_stransform` re-expresses it in the incoming datum (Section 20.5); both settings are on in the reference configurations.
- The *station-local columns* are carried as whitened coupling rows of each station's design, not as independent priors (Section 22). In the reference configurations the carried families are the float ambiguities (the float ambiguity vehicle, `network.ambiguity_vehicle` set to `float_kf`; the loader default is `level`) and the receiver code biases (`network.carry_rx_code_bias_in_block`). The vehicle, its exact coupling time update (`network.carry_coupling_exact_update`) and the temporal clock anchor are developed in Section 23.
- The *invariant DD carry* (`network.carry_invariant`, default off, off in the reference configurations) carries the double-differenced cycle basis instead of the raw ambiguities (Section 22.3).

The event responses and the holds on the emitted product are in Section 24. The single-difference extra-widelane constraint `network.sd_ewl`, a PASS B row set that is not a layer of the carry, is default off.

20.1 Architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection maps each piece of it to the code that executes it, which is spread over several translation units.

Decomposition. The network driver began as a single function of more than 14,000 lines. At that size it could not be reasoned about as a whole, exercised below the granularity of a full run, or reviewed: a search for the datum build could not bound where it was *not*. It was decomposed into single-concern *seams*, first by verbatim relocation gated on byte-identical output against the pre-cut binary (per-increment records: `docs/network_decomposition_plan.md`), then into one source folder per concern under `src/core/network/` and `include/network/`: `driver`, `state`, `assembly`, `carry`, `solve`, `qc`, `slip`, `ar`, `emit`, `diag` and `runend`. The driver (`network/driver/ApplicationNetwork.cpp`) holds the setup calls and the epoch source. One epoch is the spine (`network/driver/ApplicationNetworkEpoch.cpp`). The placement rules for new network code (the anti-regrowth decision tree in `CLAUDE.md`) exist to keep both short.

Four stages. A network run has four stages: three translation units and the seam sequence between them.

- **Setup** (`network/driver/ApplicationNetworkSetup`). `networkValidateInputs_` rejects a station with no observation `rinex_path` before anything is allocated. `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and, when the closed loop is armed, the test-station engine (Section 20.11). Its output is one `NetworkSetupResult`, filled in place and neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header (`network/driver/SetupResult.hpp`) is normative. Further setup builders, one per concern and called in a fixed order from `runNetwork`, configure the engines the run owns.
- **Assembly** (`network/driver/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*. It runs each present station's geometry models and pre-allocates the shared satellite states. On the legacy single-pivot datum (`network.phase_datum_prioritized` off, the loader default) it takes the datum (pivot) station *first*, so that the phase-bias columns admitted to the shared block are those backed by a pivot-receiver ambiguity pin, and the local half of the datum (Section 20.4) therefore constrains the column layout as well as the solve. Under the prioritized datum that the reference configurations arm, the pivot is not taken first and the shared set is the union elected by the phase-anchor epoch (Section 20.6). PASS B is *assembly*. Each station's undifferenced design block is computed in parallel over the stations (`network.parallel_stations`; `networkAssemblyProduce`), with no receiver-side differencing, and then *split* (`networkSplitStation`): the satellite clock and satellite-bias columns scatter into the shared border $\boldsymbol{A}_{s,r}$, and the remainder, less the columns held at their seed and the structurally unobservable ones, becomes the private block $\boldsymbol{A}_{r,r}$ of Equation (321). The three receiver-position columns are dropped, because network station coordinates are held fixed; the split throws if one reaches $\boldsymbol{A}_{r,r}$. The blocks are then folded into the estimator serially in station order (`networkAssemblyConsume`). Equation (329) is formed from these two pieces.
- **The epoch seams**: the ordered sequence tabulated below, which is the body of the loop.
- **Teardown** (`network/runend/Teardown`). The run-end readout is one report per subsystem, each a row of `kNetworkRunEndReports` (`network/runend/RunEnd`). `networkTeardown` walks that table and returns the process exit code; the driver ends with a `return` of that call. The service counters (`NetworkServiceCounters`, `network/state/RunContext`) and the summary writers (`NetworkRunTraces`, `network/diag/RunTraces`) are held outside the teardown, which reads them.

The seam contract. A seam is a free function (or one of five `Application` members) over the run, `NetworkRun&`, and the epoch, `const NetworkEpoch&`, with any outputs as further parameters. The run (`network/driver/Run.hpp`) holds the references to the context, the configuration, the logger, the trace bundle and the setup, and owns the estimation engines, the emit engines, the timing record and the closed-loop tally. The epoch (`network/driver/Epoch.hpp`) holds the values the stages hand each other, grouped by the stage that writes them; it is built empty with the epoch and destroyed at its end, so nothing in it reaches the next epoch. A seam *owns no state*: every carried object is a reference into the run. It opens with one binding per name it uses, reads through the epoch, and writes only through its output parameters. A block that used to `continue` the epoch loop returns an `EpochAction` of `Proceed`, `SkipEpoch` or `StopRun` (`network/driver/RunTypes`) instead; the per-skip bookkeeping stays inside the seam. A block whose locals are read further down the loop returns a result struct (`NetworkSolveGlueResult`, `network/solve/SolveGlue`) that the spine

stores in a group of the epoch (`ep.solve`); later seams read the values there. The placement of a new epoch-level value, event or observer is fixed by the same decision tree: a value the next epoch reads is run state, never an epoch member.

The run context. Carried state, configuration snapshots and counters live in `NetworkRunContext` (`network/state/RunContext`). It holds a `const NetworkKnobs` snapshot of the configuration, the cross-epoch `NetworkCarryState`, the entity tables (`ctx.entities`, `network/state/Entities`) and `NetworkServiceCounters`. The entity tables key the cross-epoch state by typed identity (station, constellation, line of sight, satellite and signal, satellite, arc; `common/GnssIdentity`), and each field has a declared persistence row (`kStationFields` and its siblings). A cross-epoch value outside the entities has a row in `net::kRunFields` (`network/state/RunFields`); an undeclared member is rejected by the verification script `scripts/nom_n4_verify.py`. Persistence follows the *three-place rule*: a carry field that is a snapshot member appears in the warm-resume fingerprint, in the restore and in the snapshot write, in `network/state/SnapshotFingerprint`, `network/state/WarmResume` and `network/state/StateSnapshotWrite` respectively. A field present in only two of the three yields a warm resume that disagrees with the run that wrote it, with no error raised.

The ordered epoch seams. The table below lists the per-epoch call sequence in spine order (`networkProcessEpoch_`, `network/driver/ApplicationNetworkEpoch`). The spine's section timers bracket groups of the calls. Rows marked *default off* are inert unless armed. The measure-only instruments (the tallies, the frame diagnostics, the first-epoch tables) are not spine calls. Each is a row of the table `kNetworkEpochObservers` (`network/driver/EpochObservers`, 22 rows in six slots), and the spine walks a slot with `networkObserveEpoch`; the row after the table summarises them.

seam call (spine order)	concern (translation unit)
`networkFastForward`, `networkInjectEpoch`, `networkCanonicalPairAdmission`, `networkSecondBandAdmission`	start-offset skip; simulated-fault injection (default off); record admission upstream of every model and detector: the canonical-pair rule and the second-band rule (both default off; Section 20.10) (`network/driver/EpochAdmission`, `network/carry/Inject`, `network/assembly/CanonicalPairAdmission`, `SecondBandAdmission`)
`networkBuildSignalRoster`, `networkAdmitEpoch`	signal roster and arc ledger, advanced on every synced epoch; the epoch's clears and broadcast feeds, the max-duration stop, the decimation skip (`network/assembly/SignalRoster`, `network/driver/EpochAdmission`)
`networkSlipFunnelOpen`, `networkPassAHead_`, `networkPassASlipScan_`, `networkSlipFunnelUnionRaw`	PASS A: the slip funnel opens with the previous epoch's deferred flags; per-station geometry and shared-state pre-allocation, on the legacy datum the datum station first; the slip scans that fill the funnel (`network/slip/SlipFunnel`, `network/driver/ApplicationNetworkAssembly`, `network/slip/PassASlipScan`, `SlipQc`)
`networkAssemblyPrefilter_`	raw slip observables; cross-station observability prune (`network/driver/ApplicationNetworkAssembly`)
`networkSignalIntegrity`, `networkMeasurementAdmission`	per-line-of-sight band-state classifier (default off); product-referenced corrupt-channel screen, the one admission test a differential screen cannot see (armed by `network.adm_screen`) (`network/assembly/SignalIntegrity`, `MeasurementAdmission`)
`networkPhaseAnchorEpoch`, `networkReceiverPhaseBias-Allocation`	election of the *local* half of the phase datum: the candidate tally and coverage gate, then the armed realisation (`network/solve/PhaseAnchorEpoch`, `PhaseDatum`, `AnchorHygiene`, `ChainDatum`); the receiver phase-bias allocation is inert unless the chain datum is armed

seam call (spine order)	concern (translation unit)
networkAssemblyRegisterColumns_	clear the layout, register this epoch's shared columns, open the arrowhead system (network/driver/ApplicationNetworkAssembly)
networkSdEwlAr, networkOraclePinEpoch, networkAmbFloatDropEpoch	single-difference extra-widelane decision (default off); simulation-only ambiguity pin; the float vehicle's bounded drop (network/ar/SdEwlAr, network/slip/OraclePin, network/ar/AmbFloat)
networkClockLevelRestoreRaise, networkHandover-StransformBeforeRows, networkCarryDecide, networkCouplingExactPrepare	the first two default off; the fold decision for every carried border entry, made once per epoch and read by both the coupling rows and the border fold; the inputs of the exact coupling time update (network/state/CarryDisposition, network/state/BorderPrior)
networkPassB_	PASS B produce and consume: the split design blocks handed to addStation (network/driver/ApplicationNetworkAssembly, network/assembly/StationBuild, network/carry/Coupling)
networkGapResumeCensus, networkCarryDriverBeginEpoch, networkBuildCarryDdPriors	arc-resumption tally (measure-only); the invariant DD carry's predict and prior blocks (default off) (network/slip/GapResumeCensus, network/carry/DdPriorBuild)
networkSolveGlue	the solve as a fixed list of 14 stages (below) (network/solve/SolveGlue, calling Sbasis, CodeBiasDatum, ClockDatum, RankGate, SbasisCompletion, network/qc/RaimWithinEpoch)
networkInstallCarry, networkProducerAr	DD posterior read-back into the invariant carry (default off); network-side integer decision, read-only and log-only unless its fixed-cycle fold is set to emit (default off) (network/carry/InstallCarry, network/ar/ProducerAr)
networkGrossLosScreen (decision half), networkFixedCycles, networkSolPrep	gross line-of-sight decision, so a declaration reaches the epoch it declares (network.gross_los_emit_exclude); fixed-cycle fold (default off); map to the solution epoch and per-satellite discontinuity signalling (network/qc/GrossLos, network/ar/FixedCycles, network/emit/SolPrep)
networkPbHoldCloneUpdate, networkEmitStore	smoothed-hold clock clones (default off); emit re-base, phase-bias hold, store push, product write, test-station step (network/emit/PbHoldClone, EmitStore, PbHold)
networkIonoDiag, networkDdFrac, networkShDevVtec, networkShHealth, networkLeveling	diagnostic sweeps, and the harvest of the station-local carries after the solve (networkHarvestLocalCarries, network/carry/LocalCarryHarvest) (network/diag/IonoDiag, DdFrac, ShDevVtec, ShHealth, Leveling)
networkPostFitRaim, networkGrossLosScreen (response half)	post-fit network RAIM with its attribution pass; the gross line-of-sight response (network/qc/Raim, GrossLos)
networkClockQc	post-solve satellite-clock quality control and the retro-flag feed (network/qc/ClockQc)
networkSignalRosterAudit, networkBlockBaardaPredict, networkCongruencePredict	roster disagreement pass; the predict halves of the block-Baarda and station-congruence tallies (network/assembly/SignalRoster, network/qc/BlockBaarda, Congruence)
networkStashStatus, networkPbHoldCloneHarvest, networkClockLevelMonitor, networkBufferedTraceDrains	carry re-stash and the epoch-status record; the clones' re-harvest and the level monitor (both default off); the buffered trace rows (network/diag/StashStatus, network/emit/PbHoldClone, network/solve/ClockLevel)
networkWriteStateSnapshot	warm-resume state snapshot (network/state/StateSnapshotWrite)

seam call (spine order)	concern (translation unit)
`networkObserveEpoch`, six slots	the 22 measure-only instruments, by slot: admitted (satellite-identity tally), before columns (level-authority tally), after clock QC (phase-visible monitor), roster tail (ledger emit, second-phase and all-channel censuses, second-constellation census, the *parameters as armed* and *contributions as armed* tables), frame diagnostics (post-fit variance factor and residual traces, reduced-normal spectrum, station-block audit, prefit census, redundancy census, block-Baarda and congruence drains, minimal-detectable-bias flush, datum-graph census, reduced-RHS attribution and its flight recorder), stash tail (carry-state trace drain, entity reach sampler) (`network/driver/EpochObservers`)

The solve. `networkSolveGlue` (`network/solve/SolveGlue.hpp`) calls 14 stages in a fixed order; the estimator accumulates its folds in that order, so the order is part of the numerical result. In order: (1) `networkCensusSigma0`, the prefit variance tally; (2) `networkCodeBiasDatum`, the satellite code-bias datum rows (Section 20.7); (3) `networkClockDatum`, the clock datum and pivot election (Section 20.5); (4) `networkPreGatePriors`; (5) `networkEstimabilityDump`; (6) `networkKaulaMargin`; (7) `networkSbasisCompletion`; (8) `networkRankGate` (Section 20.8); (9) `networkClockModelWarm`; (10) `networkPostGatePriors` (the ephemeris re-level, the handover S-transform, the border carry prior, the clock-oscillator prior and the cold clock anchor); (11) `networkPrimarySolve`; (12) `networkClockAnchorGuardEpoch`; (13) `networkWithinEpochAdapt`, the within-epoch RAIM tier; (14) `networkSigma0Ceiling`. Four stages can end the epoch, five ways; each way is a row of `kSolveSkips`. Everything the rank gate must see is folded before stage 8.

Reading key. The derivations that follow map to places in that sequence. The rank deficiency and its minimum-constraint datum (Section 20.4 to Section 20.8) are built and gated inside `networkSolveGlue`. The arrowhead Schur reduction (Section 20.9) runs in the kernel that the same seam invokes, on normals accumulated station by station during PASS B. The emitted product and the test station (Section 20.11) are handled in `networkEmitStore`. The rest of the table is assembly, datum election, ambiguity resolution and solution preparation, with quality control, diagnostics and carry bookkeeping, around those places.

20.2 Joint multi-station observation model

The network estimator uses the same *undifferenced, uncombined* observations as single-receiver PPP (Section 4), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The unknowns partition into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are identical for every station, since a satellite's clock and hardware delays do not depend on the observer. Orbits are held fixed to a precise product; orbit determination is outside the scope of the network estimator. A fourth family, the coefficients of a global spherical-harmonic VTEC expansion (`network.estimate_iono_sh`), is kept in the code and is off in both reference configurations: the ionosphere is the free per-line-of-sight slant delay below (`docs/refound/AB1_sh_consumer_product_2026-09-20.md`).
- **Private (station-side).** For each station r the parameter set of Table 6. Station coordinates are known and held fixed, so position is not a free parameter.

Table 6: Parameter families of a network station. The kind names are those of `NetworkParamKind` (`common/ParamRef`).

family	kind	content
receiver clock and inter-system biases	`rx_clock`	$c\,\mathrm{d}t_r$ and one $c\,\delta t_r^{(g)}$ per active constellation g other than the time reference (GPS)
wet troposphere	`zwd`	ZWD_r (no gradient columns)
slant ionosphere	`slant_iono`	one $I_{r,1}^i$ per tracked satellite, scaled to signal s by μ_s (+ on code, − on phase)
ambiguities	`ambiguity`	one float ambiguity per satellite and phase signal, $\lambda_s N_{r,s}^i$ in metres
receiver code biases	`rx_code_bias`	differential bias of the constellation's clock-reference pair, inter-frequency bias of each further band, inter-signal correction of a second code
receiver phase biases	`rx_phase_bias`	$\delta_{r,s}$ per (system, signal); only under the chain datum (`network.phase_datum_eq34`, default off)
phase colour	`phase_colour`	per-satellite phase nuisance, Gauss–Markov (`network.phase_colour_state`, default off)
position	`rx_position`	three columns, held at the roster coordinate

The parameter set. A station's model is a list of columns, held in one `StationParams` (`network/state/StationParams`). Each column has a typed identity (`ParamRef`, `common/ParamRef`), a linearization point and the disposition the split gave it (Section 3.5 describes the layout from the state-vector side). The set also names the shared border parameters the station's rows reference and the shared parameters a gate withheld from it. Table 6 lists the families.

Dispositions and the *parameters as armed* table. Every column of every station, and every shared satellite parameter a gate withheld from a station, is an instance with a declared *disposition* on each epoch (`ParamDisposition`, `network/assembly/ParamInventory`): *estimated in the shared border*, *estimated in the local block*, *held at its seed* (in the station's model at a fixed value, outside the solve), *absorbed* (a shared satellite phase or code bias a gate withheld from the station, which another column of the station takes up), *pinned* (the station coordinates), *dropped with a stated reason* and *unobserved* (in the layout, but its design column is zero this epoch). The run prints the counts per parameter class and disposition as the *parameters as armed* table once after the first emitted epoch and again at the run end, with a warning for a class governed by a rule whose premise does not hold, and the run manifest carries the same record. A gate that withholds a shared parameter has a row in a reason table (`AbsorbedReason`): the GLONASS code-only scope, the coverage allowlist of the phase-bias columns, the legacy pivot allowlist, the code-bias admission tally, the anchored-union coverage gate and the anchor-hygiene block. On both reference configurations the border count plus the absorbed count of a satellite phase-bias class equals the ambiguity count of the same signal, because the screens that remove an observation remove its ambiguity with it. A second table, *model as armed*, is built from the parameter registry (`NetworkParamRegistry`, `network/assembly/ParamRegistry`), which declares once per parameter kind whether it is estimated and carried, its process-noise rule and rate, its variance floors, its gap horizon, the route it takes into the next epoch's solve (dense border, diagonal border, local block, conditional row, diagonal station prior) and the responses it has to slips, gaps, handovers, decoupling and re-admission.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{321}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. There is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. The reduction of Section 20.9 rests on that sparsity.

One equation. A row's predicted observable is the term list of `observationEquation` (`measurement/ObservationEquation.hpp`; Section 4.4). The network reads it through `NetView`, which answers for each parameter with the slot of the station's set and its linearization point (`networkStationRows`, `network/assembly/StationRows`). A shared satellite parameter answers with its border column and the value zero: it is absolute in the network solve and is never written back to a station. On the network's rows the satellite clock enters with the partial -1 (`kSatClockPartial`) and each satellite bias, code or phase, with $+1$ (`kSatBiasSign`); the same two constants multiply the known terms of a user of the corrections. A network station always has one ambiguity per band, and the network takes no SSR bias term, because it estimates the biases themselves. The estimated bias parameters therefore carry the sign of the term as it appears in the observation, $-B_s^i$ and $-b_s^i$ of Equation (4) and Equation (5).

No receiver-side differencing. Unlike the PPP-RTK user mode, the network estimator applies *no* between-satellite or between-receiver differencing. Its rows are the undifferenced equation of `networkStationRows`, and the differencing strategy of the mode is `DifferencingMode::None` (`differencingModeFor` in `measurement/MeasurementBuilder.cpp`), overriding any `between_satellite_sd` toggle in a network configuration. Differencing would cancel the satellite-side terms the network *estimates*. The network carries them as explicit columns instead and resolves the resulting rank deficiency with a minimum-constraint datum (Section 20.4).

20.3 The explicit satellite phase-bias state

The main contrast with the user side is the treatment of the satellite phase bias b_s^i (written $\bar{b}_s^i$ once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (Section 5) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The network estimator *generates* that known product, so it carries the satellite phase bias as an estimated state. The shared border gains a per-(satellite, signal) phase-bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{registered by } \texttt{NetworkEstimator::registerPhaseBias}(i,s), \quad \text{layout key } \texttt{"}i\texttt{|PB|}s\texttt{"}, \tag{322}$$

disjoint from the satellite *code*-bias columns (layout key `"`i`|CB|`s`"`) and the clock column (`"`i`|CLK"`). A station's parameter set names the parameter by the typed identity of kind `sat_phase_bias`; the spelling `"`i`|`s`|PB"` of `filter/StateKeySchema` is the single-receiver filter's key for the same family (Section 3). Its process noise is a small random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default $1 \times 10^{-12}\,\mathrm{m^2\,s^{-1}}$, two orders of magnitude below the `sat_code_bias` default of $1 \times 10^{-10}\,\mathrm{m^2\,s^{-1}}$). The rate is the carry rate of the border, applied as $\mathrm{var} + q\,\Delta t$ in the carried prior. Satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column insensitive to real slow drift (the registry kind `sat_phase_bias`, `network/assembly/ParamRegistry`).

Remark 4. The estimated $\bar{b}_s^i$ is not the raw satellite phase hardware delay, which is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [24]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. Because of this lumping the phase-bias product must be *consistent* (Section 18): the datum of the network and the integer recovery of the user must use the same S-system.

20.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (321) is rank deficient. The linear dependencies that the user side removes by differencing appear here as exact *gauge freedoms* of the joint normal matrix. No observation distinguishes a common shift of all satellite clocks of a constellation from the opposite shift of the receivers' clocks (the receiver clock for the time-reference constellation, the inter-system bias for each other one); none fixes the split of an absolute phase bias between the satellite column and the per-station ambiguity datum; none fixes the common level of the code biases between the satellite and receiver sides, or their split between the clock, the ionosphere and the geometry-free combination. These are the S-system ambiguities of [36]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space. The gauges are, in the order the datum treats them: one clock gauge per constellation; one phase-bias against ambiguity gauge per (satellite, signal); and the code-bias gauges of Section 20.7.

Exact gauge, soft elimination. The datum is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a structural elimination of columns. Each shared gauge that is a pin is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution moves to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `applyDatumPins` in `network/solve/Estimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam). The estimator's own reduced system stays unmodified, so per-station back-substitution (Section 20.9) stays valid. The pinned diagonal is set to the reduced normals' own scale (the largest diagonal magnitude, at least one) rather than to unity. An *under*-specified datum then still appears as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude. The gauges that are rows (code-bias gauges, the sum-to-zero and broadcast-anchored clock datums, the S-basis completion) are folded as tight constraint rows before the rank gate; their weights are listed in the *contributions as armed* table (`network/assembly/Contributions`).

In S-transform language, let $\boldsymbol{x}$ collect the rank-deficient undifferenced parameters and $\boldsymbol{S}$ be the S-basis selection. The emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$. Choosing the datum is choosing $\boldsymbol{S}$, and the pins and rows above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The datum is declared in the product (`SbasisDeclaration`, Section 20.11) and has two halves that live in two places:

- **Shared-block gauges** touch the shared satellite columns: the clock datum, one per constellation, and the code-bias gauges. The clock pins go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** live on one station's private block: the phase-bias gauge is realised by tight prior rows on the ambiguities of the elected anchor station, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9). The *caller* folds them into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$ at

Table 7: Datum realisations. The reference configurations are `config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`.

gauge	reference configurations	other realisations (default status)
clock, per constellation	hard pin of one satellite clock, the pivot; sticky incumbent (`network.sticky_pivot`); Section 20.5	sum-to-zero over the constellation (`network.clock_datum_sum_to_zero`); per-satellite soft anchor to the a priori clock (`network.clock_datum_broadcast_anchored`); declared pivot (`network.clock_datum_satellite`); coverage election (`network.pivot_election`)
phase bias against ambiguity, per (satellite, signal)	prioritized anchor election, held basis (`network.phase_datum_prioritized`, `network.phase_datum_held_basis`); Section 20.6	one pivot receiver (the loader default); single-receiver chain (`network.phase_datum_eq34`, off)
satellite code biases	ionosphere-free pin, geometry-free pin and sum-to-zero rows, time-linked, geometry-free pin at the broadcast group delay (`network.sat_gf_datum`); Galileo family difference; Section 20.7	geometry-free pin at zero or none; Kaula prior (`network.cb_kaula_prior`, off)
receiver code biases	loose zero-mean prior on each station's differential bias (`network.rx_code_bias_datum`)	none

information 10^{10} (row kind `ambiguity_pin`), not through the shared `Datum`. Station priors also make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 20.9).

Selectable datum realisations. Each gauge admits alternatives, selected by configuration and recorded in the `SbasisDeclaration`. All produce the same estimable $\boldsymbol{S}\boldsymbol{x}$ and differ in how they distribute the gauge. Table 7 lists what the reference configurations arm and what the loader offers instead.

The removed ensemble realisation of the phase gauge (a sum-to-zero over the receivers) remains in the S-basis declaration so that a stored product naming it can be read, and its Schur-projected fold remains a kernel capability (Section 22.2); no configuration the loader accepts arms it.

20.5 The clock datum and the pivot election

The pivot pin. Adding a constant to every satellite clock of a constellation and absorbing it in the receivers' clock and inter-system bias leaves every row unchanged. In the default pin mode (`SbasisDeclaration::IsbDatum::PerConstellationSatClockPin`) the gauge is fixed by one hard pin per constellation: the clock correction of one satellite, the *pivot*, is held at 0, that is at the a priori product clock, and every other clock of the constellation is estimated against it. No system is anchored to a receiver clock; each station's clock and inter-system bias stay free local states. After the local ambiguity pins are eliminated, the reduced shared normals then have exactly one null direction per constellation, which is the declared S-basis dimension that the rank gate checks (Section 20.8). The two alternatives are constraint-realised and need no pivot: the sum-to-zero mode replaces the pin by the tight row $\sum_i \Delta\delta t^i = 0$ over the constellation's registered clocks, and the broadcast-anchored mode anchors each clock to its own a priori value by a soft pseudo-observation of declared 1σ. In both the declared dimension is 0. The reference configurations keep the pin. `network.clock_datum_station` names the reference receiver of the declaration; the pin is on a satellite clock, never on that receiver's clock.

Choosing the pivot. The pivot is only a gauge, but a satellite that few stations track, that the broadcast flags unhealthy, or that enters the Earth's shadow soon, holds the whole constellation's clock level on a weak and short-lived tie. The choice per constellation, in priority order (`NetworkSbasis::electClockPivot`):

1. the declared pin `network.clock_datum_satellite`, kept even when unhealthy;
2. the sticky incumbent (`network.sticky_pivot`, armed in the reference configurations), while it stays registered and, under `network.sat_health`, healthy: a new pivot is chosen only when the incumbent is absent, and the old one returning does not flip the datum back;
3. the pick of an earlier election that no emitted product has adopted yet;
4. a new pick: the first registered satellite in lexical order or, with `network.pivot_election` set to `coverage`, the satellite the coverage ranking below names.

Election by predicted coverage. The election (`network/solve/PivotElection`) applies at a constellation's first epoch and at a forced handover only (the incumbent left the registered set, or the broadcast flags it unhealthy). It reads no observation: it uses the orbit product, the roster coordinates and the Sun. For a candidate satellite i with an orbit and the samples $t_k = t_0 + k\,\Delta$, $k = 0, \ldots, K$, with $K = \lfloor H/\Delta \rfloor$, horizon H (`network.pivot_election_horizon_s`, default 7200 s) and step Δ (`network.pivot_election_step_s`, default 300 s), the score is

$$n_i \;=\; \Big|\Big\{(k,r) \;:\; r \in \mathcal{R},\; 0 \le k \le K,\; \theta_r^i(t_k) \ge \theta_{\min}\Big\}\Big|, \tag{323}$$

with $\theta_{\min}$ the run's elevation mask (`processing.elevation_mask_deg`), and the umbra time of i is the first sample inside the cylindrical Earth shadow (radius scaled by `GnssConfig::getEclipseShadowScale`). Candidates that stay out of the umbra for the whole horizon come first, and the largest n_i wins. When every candidate enters the umbra, the one that enters it last wins, then the larger n_i. A tie goes to the lexically first satellite, and a candidate without an orbit is never elected. The ranking is a pure function of integer counts, so the preview before PASS B and the solve glue, which both elect, always agree. The setting is default off, never armed, and inert under the two constraint-realised clock datums (`docs/refound/PIV_pivot_election_coverage_2026-10-17.md`).

Handover. When the pin moves from satellite p to q inside a constellation, the estimable clock solution of the whole constellation is re-referenced. The pin swap is a rank-one S-transform along the constellation's structural clock null direction v_c:

$$c_0 = \frac{x_q^{\mathrm{pre}}}{v_c[q]}, \qquad x_i \leftarrow x_i - v_c[i]\,c_0, \qquad \mathrm{var}_i \leftarrow \mathrm{var}_i + v_c[i]^2\,\mathrm{var}_q, \tag{324}$$

where x_q^{pre} is the carried level of the incoming pivot in the outgoing frame (`network/solve/HandoverStransform`). Only the constellation's clock columns have a nonzero component of v_c, and every between-satellite difference is unchanged to machine precision. With `network.handover_stransform` the carried clocks are re-expressed this way and folded; without it the carry of the changed constellation is skipped for one epoch and re-levels from the data. The code-bias and phase-bias carries are released to fold unchanged, because their gauges are realised independently of the clock pivot. The variance update drops the cross term, which is conservative for the positively correlated clock common mode; the full $\boldsymbol{SPS}^{\mathrm{T}}$ form is a separate default-off setting (`network.handover_stransform_dense`). The time-linked clock anchor and the clock step test that bind a satellite's carried clock across epochs are in Section 23.

20.6 The phase datum

The satellite phase bias $\bar{b}^i_s$ is collinear with the column of ambiguities $\{N^i_{r,s}\}_r$ (null vector $\delta\bar{b}^i_s = -\delta N^i_{r,s}$ for any per-satellite shift). The datum fixes this gauge once per (satellite, signal) key by pinning the ambiguity of one station. The emitted phase bias therefore carries that station's ambiguity lump (Section 20.3).

Legacy single pivot. By default the datum receiver is one pivot receiver, whose ambiguities are all pinned (at information 10^{10}), and the shared phase-bias set is restricted to the keys that pivot tracks. A satellite low or unseen at the pivot then has no clean gauge.

Prioritized anchors. With `network.phase_datum_prioritized` (armed in the reference configurations) the shared set is the *union* of the keys that at least `network.phase_datum_min_stations` stations track (the coverage gate; a key below the bar is pruned at every station and, with `network.phase_datum_gate_drop_erase`, its carried ambiguities are erased at the drop edge). For each key one tracking station is the *anchor*: its ambiguity is pinned and the other stations' are free. This is the same one-pin-per-gauge constraint realised at a different station per key (`networkBuildPhaseDatumCandidates` and `networkElectPhaseAnchors`, `network/solve/PhaseDatum`). The election keeps the sticky incumbent while it is present, still tracks the key and is not deposed by a screen flag on its own column (`network.phase_datum_sticky`, default on). Otherwise, under the held basis, the candidates whose arc is old enough (`network.phase_anchor_min_age_ep`) and has been free of screen flags for `network.phase_anchor_flag_quiet_ep` epochs are ranked by descending elevation, and the highest one whose carried level passes the handover ledger test is elected. When none passes, the highest eligible one is elected and pinned at 0, a cold handover, and when none is eligible the coverage-ranked sweeps over the trackers elect (coverage is counted above `network.phase_datum_pref_elev_deg`, default 15°). A handover pins the incoming anchor at its own carried level (`network.phase_datum_handover_continuous`), so the family's frame does not step. The ledger test refuses a successor with no ledger entry, a non-finite value, or a carried variance that is not finite, not positive or above `network.phase_datum_handover_max_var_m2`; a cold handover steps the emitted phase bias, and it is declared to the user (`network.phase_datum_held_mark_cold`). The *held basis* (`network.phase_datum_held_basis`) freezes the pin value of a retained incumbent for its whole tenure, so the datum is constant between declared handovers (Section 23). The election runs once per epoch after the admission screens, whose withholds prune the states an election could otherwise pin (`network/solve/PhaseAnchorEpoch`).

Chain datum. With `network.phase_datum_eq34` (off) the gauge is fixed by one datum-receiver phase-bias pin plus a spanning tree of ambiguity pins over the bipartite receiver–satellite graph per (constellation, signal) connected component, with per-(receiver, system, signal) phase-bias split states $\delta_{r,s}$ that carry each receiver's fractional bias (`network/solve/ChainDatum`). It is the substrate of the invariant DD carry (Section 22.3).

20.7 The satellite code-bias datum

With `network.estimate_sat_code_bias` the network estimates a code bias B^i_s per satellite and code signal, and the external `.BIA` product is not used. The code biases are collinear with the clock, the slant ionosphere and the receiver code biases. The datum rows are built by `NetworkSbasis::buildCodeBiasDatumRows`, folded by `networkCodeBiasDatum` (stage 2 of the solve) as tight rows

of information 10^3, before the rank gate. With no ionosphere constraint (`network.w6_iono_constraint none`, armed in the reference configurations) these rows are the whole code-bias datum.

Per-satellite pins. For a satellite with code signals s_1, s_2 (the two lowest distinct-frequency signals of its registered set, chosen deterministically), two rows are built. The ionosphere-free pin ties the clock-common gauge,

$$a_1\, B^i_{s_1} + a_2\, B^i_{s_2} = 0, \qquad a_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \quad a_2 = -\frac{f_2^2}{f_1^2 - f_2^2}, \tag{325}$$

so the ionosphere-free lump lives in the estimable satellite clock. The geometry-free pin fixes the split that the slant ionosphere would otherwise absorb,

$$\frac{B^i_{s_2} - B^i_{s_1}}{\mu_{s_2} - \mu_{s_1}} \;=\; d^i_{\mathrm{GF}}, \qquad \mu_s = \frac{f_1^2}{f_s^2}, \tag{326}$$

with d^i_{GF} a datum quantity: any value is legal and the estimable parameters absorb the difference. `network.sat_gf_datum` selects it: `none`, `zero`, or `broadcast_tgd` (pinned in both reference configurations), where $d^i_{\mathrm{GF}} = c\,T_{\mathrm{GD}}$ for GPS $(L1, L2)$ and $c\,\mathrm{BGD}(E1, E5a)$ for Galileo $(E1, E5a)$. The identity is exact: in the model's sign convention the broadcast biases are $cT_{\mathrm{GD}}\,\mu_s$, whose ionosphere-free combination is identically zero and whose geometry-free combination is cT_{GD}, so the two pins do not conflict. The target is used only when the pinned pair is the pair the group delay describes; any other pair falls back to zero and is counted. Extra signals of a satellite are pinned through their geometry-free combination against the band-1 reference at the same target, and a channel whose every column the datum pins has its sum-to-zero row suppressed.

Level of each channel. The common level of a channel (g, s) between the satellite and the receiver sides is fixed by a sum-to-zero row over the satellites that carry it. On the receiver side the same freedom is fixed by a loose zero-mean prior $\delta_r \sim N(0, \sigma^2)$ on each station's differential bias (`network.rx_code_bias_datum`, armed; σ = `network.rx_code_bias_datum_sigma_m`, default 10 m). The prior adds $1/\sigma^2$ along the common mode, independent of the station count, and leaves the between-station differences to the data.

Time linking. With `network.code_datum_time_linked` (armed) the rows pin the *change* of the datum instead of its level. The right-hand side of the ionosphere-free row becomes the satellite's previous solved combination $a_1\hat{B}_{s_1,\mathrm{prev}} + a_2\hat{B}_{s_2,\mathrm{prev}}$, but only when the pair is unchanged and both signals have previous values; otherwise the row re-seeds at 0 and the satellite is marked. The geometry-free rows carry their previous combination (`network.sat_gf_datum_time_linked`), with the absolute target only at a first sighting. The sum-to-zero row spans the channel's *veterans* (satellites with a previous solved value) with right-hand side $\sum_{\mathrm{vet}} \hat{B}^i_{s,\mathrm{prev}}$, so a rise or set does not move the target. A channel with no veteran re-seeds at $\sum = 0$, which is the one path on which the emitted level re-references; it is counted and declared.

Family difference. Galileo band-1 and band-5 codes come in two families by tracking attribute: the combined I+Q correlator output (attribute `X`, the *secondary* family) and the discrete components (the pilot, the *primary* family; `channelFamilyOf`). Their biases are distinct hardware paths with their own bias. A time-linked sum-to-zero row per family would leave the two families' levels independent, so that the combined and pilot clocks differ by a datum accident. With `network.`

`code_datum_family_difference` (armed with `network.code_datum_time_linked`) the sum-to-zero row of a secondary channel X is replaced by a row over the satellites $\mathcal{C}$ that carry both X and its primary partner P of the same band,

$$\sum_{i\in\mathcal{C}} \left(B^i_X - B^i_P\right) \;=\; \sum_{i\in\mathcal{C}_{\mathrm{vet}}} \left(\hat{B}^i_X - \hat{B}^i_P\right)_{\mathrm{prev}}, \tag{327}$$

time-linked like the sum-to-zero row. The partner is the same-band primary channel with the most satellites in common. The sum-to-zero row is kept where no partner or common satellite exists, where every spanned secondary column is pinned by another row, and for one epoch when the family row has no veteran but the sum-to-zero row has. At the first sighting of a secondary or partner column, with `network.code_family_birth_prior_sigma_m` (armed at 0.10 m), one soft row $B^i_X - B^i_P = \mu$ is folded, with μ the family's veteran mean difference and 0 without a veteran. The option `network.code_datum_pair_continuity`, which holds the ionosphere-free pin pair through a gap, is off: it preserved the two families' birth realisations and worsened pilot-family users (`docs/refound/GFC_galileo_family_clock_datum_2026-10-10.md`, `docs/refound/GAD_galileo_adoption_2026-10-13.md`).

20.8 The rank gate and the S-basis completion

A null-space rank gate eigen-decomposes the pre-constraint reduced normals and asserts that the count of small eigenvalues equals the *declared* S-basis dimension. It is implemented as `NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the constrained solve, against the declared dimension of the clock datum's plan. An eigenvalue is small when it is below $\mathrm{tol}\cdot\max(\lambda_{\max}, 1)$, with tol = `network.rank_gate_tol` (loader default 10^{-9}, 10^{-11} in the reference configurations). The call sits in stage 8 of the solve (`networkRankGate`, `network/solve/RankGate`). The gate complements the positive-definiteness guard. The guard catches the exact-deficiency (under-specified datum) case on the pinned pivot; the rank gate checks the reduced deficiency against the declared gauge dimension in both directions and records whether a mismatch was an under- or an over-specified datum, which need opposite fixes.

Two mechanisms keep a sound epoch from being withheld by a weak column. *S-basis completion* (`network.sbasis_complete`, armed; stage 7, `networkSbasisCompletion`) constrains each separated extra near-null direction of the reduced system along its own eigenvector at its carried level, with a tight row of information 10^3, instead of withholding. It is capped per epoch (`network.sbasis_complete_max`, default 6), refuses when the weight is less than two decades above the cutoff, and skips a direction not separated by a decade from the declared-gauge cluster; both edges are events. Under the prioritized phase datum the gate then runs a *drop-and-heal* pass that soft-anchors the near-null phase-bias columns toward 0 at $\sigma = 1\,\mathrm{m}$ (`healNearNullPhaseBias`, `network/solve/PhaseDatum`) and re-gates once. On the legacy pivot datum there is nothing to heal and the epoch is withheld immediately.

20.9 Per-station Schur reduction (arrowhead normals)

Stacking Equation (321) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system. The station–station off-diagonal blocks

are exactly zero, because stations couple only through $\boldsymbol{x}_s$:

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{328}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{A}_{r,r}, \tag{329}$$

and $\boldsymbol{u}_s^r = \boldsymbol{A}_{s,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$, $\boldsymbol{u}_r^r = \boldsymbol{A}_{r,r}^{\mathrm{T}} \boldsymbol{W}_r \boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `network/solve/NormalEquations.hpp`). The station's diagonal prior rows can be passed to `addStation` so that the positive-definiteness test judges the block the solve uses (`network.station_prior_before_pd`, armed): a station whose observation-only block is rank deficient in the direction its own datum row gauges is then kept.

Lossless marginalisation. The arrowhead system is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^r$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added, and `reduce` and `reconstructStation` reuse the factor. The elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r \in \mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}} \right) + \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{330}$$

$$\tilde{\boldsymbol{u}}_{s} = \sum_{r \in \mathcal{R}} \left(\boldsymbol{u}_{s}^{r} - \boldsymbol{N}_{sr}^{r} \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \boldsymbol{u}_{r}^{r} \right) + \boldsymbol{u}_{s}^{\mathrm{prior}}, \tag{331}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation. Solving Equations (330) and (331) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = \left(\boldsymbol{N}_{rr}^{r}\right)^{-1} \left(\boldsymbol{u}_r^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\, \boldsymbol{x}_s \right), \tag{332}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost is linear in the station count and the per-station eliminations run in parallel (`reconstructStation`, `network/solve/NormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (330) (soft physical priors, the datum rows and the carried border prior) are added before the reduction through `addSharedInfo` and pass through the Schur complement unchanged.

Exact sparse station normals. A station's design rows touch a handful of the n_s shared columns (its satellites' clocks and biases) and of its own locals. The dense products of Equation (329) spend almost all of their work on zeros. With `network.sparse_station_normals` (default on, set in both reference configurations; `false` gives the dense products) the blocks are formed from the nonzero entries only (`NetworkNormalEquations::formStationNormalsSparse`). The result is not an approximation. Each element is summed in the order Eigen's blocked product sums it: the terms in ascending row order from +0, except that an element of a 4-column block in the 2-row tile of its row panel keeps two accumulators (even and odd row index) that are added before the

remaining rows. A skipped term is an exact zero, so the blocks equal the dense products bit for bit. The function reports that equality is not guaranteed, and the caller then forms the products densely, when the build contracts multiply and add, when Eigen runs more than one thread (the driver pins it to one for the epoch loop), when a result is a vector or the shapes are so small that Eigen evaluates the product coefficient by coefficient, when the row count exceeds the panel depth, and when an input, weight or weighted entry is not finite. The shared block $\boldsymbol{N}_{ss}^{r}$ is accumulated on the square of the columns any row touches and added to the border entry by entry, which replaces an $n_s \times n_s$ product and symmetry check per station and epoch. The equality is gated against Eigen's own product in `tests/network/test_neq_sparse_station_normals.cpp` and was checked on whole runs (`docs/refound/NETCOST_gps_galileo_cost_attribution_2026-10-18.md`).

The datum frame, and the covariance of the locals. The covariance of the shared estimate is the inverse of the datum-constrained reduced normals, $\boldsymbol{\Sigma}_{ss} = \tilde{\boldsymbol{N}}_{ss}^{-1}$ of the constrained system, recovered from the same $\mathrm{LDL}^{\mathrm{T}}$ factor as the solve (`solveSharedConstrained`). A datum-pinned column and a folded ensemble gauge column carry only an artificial variance, so the consumers of the covariance treat them as exact. The *datum frame* $\boldsymbol{\Sigma}_{ss}^{\mathrm{df}}$ is $\boldsymbol{\Sigma}_{ss}$ with the rows and columns of those columns zeroed. A station's locals then have the covariance

$$\mathrm{Cov}(\boldsymbol{x}_r) \;=\; (\boldsymbol{N}_{rr}^{r})^{-1} \;+\; \boldsymbol{M}_r\, \boldsymbol{\Sigma}_{ss}^{\mathrm{df}}\, \boldsymbol{M}_r^{\mathrm{T}}, \qquad \boldsymbol{M}_r = (\boldsymbol{N}_{rr}^{r})^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}}, \tag{333}$$

which every reader of the datum-frame covariance uses: the conditional variance and regression of each carried column (the carry harvest), the row redundancy of the RAIM and Baarda tests, the redundancy and block tallies, and the covariance of a carried double difference (Section 22.4). The frame is a function of three inputs only (the last solve's $\boldsymbol{\Sigma}_{ss}$, the pinned columns of the datum and the folded gauges), so the solve builds it once, right after $\boldsymbol{\Sigma}_{ss}$, and stores it with the list of pinned columns. A reader whose datum pins the same columns in the same order reads the stored matrix; any other datum builds a local frame by the same function. The stored frame is invalidated at the start of the next solve, by a reset and by a change of the gauges, and only the non-const solve writes it, so the parallel station loops read immutable data. This replaced one copy of the n_s^2 matrix per call, and it changes no output bit.

The *border-known variance* is the case $\boldsymbol{\Sigma}_{ss}^{\mathrm{df}} = \mathbf{0}$ of Equation (333): $\mathrm{diag}\big((\boldsymbol{N}_{rr}^{r})^{-1}\big)$, the variance of a local column with the whole shared border treated as known. The anchor-handover test uses it, because the uncertainty of the shared phase-bias column moves the pin target and the realised frame together and so is common mode in a handover step. It is computed by `stationConditionalVarianceDiag`, one local solve $n_c = (\boldsymbol{N}_{rr}^{r})^{-1} e_c$ per column, without forming or multiplying the all-zero $n_s \times n_s$ matrix. The result is bit-identical to the product form: with $m_c = \boldsymbol{N}_{sr} n_c$ finite, $m_c^{\mathrm{T}} \mathbf{0}\, m_c = \pm 0$ leaves a nonzero variance unchanged, and the two cases where the sum could differ (a zero variance, a non-finite m_c) take the product itself (`docs/refound/CVF_conditional_variance_frame_2026-10-29.md`).

Remark 5. This arrowhead topology with exact reduction mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver. In the specialisation, the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. A two-stage covariance-form alternative was rejected because the normal-equation Schur form is exact and cheaper here.

20.10 Constellation scope

The network estimator supports GPS, Galileo and BeiDou on the same footing, selected by `processing.active_systems`. GPS is the time reference: it carries no inter-system bias, and

every other active constellation adds one per station. The reference configurations are GPS and Galileo.

Galileo. The reference configurations estimate E1 and E5a for Galileo beside L1 and L2 for GPS (`processing.frequency_mode dual`), with both code families of each Galileo band on the signal list (`processing.signal_priorities.Galileo`: `C1C`, `C1X`, `L1C`, `L1X`, `C5Q`, `C5X`, `L5Q`, `L5X`). The package that makes this posture work includes the code-bias family difference (Section 20.7), the receiver cross-system antenna fallback (`model_corrections.enable_receiver_cross_system_atx_fallback`, which substitutes the nearest GPS band when the receiver antenna entry lacks a Galileo frequency) and, on the user side, `estimation.clock_disc_isb_reinflate_std` of 10 m: a network clock-datum re-reference steps one constellation's satellite clocks by a common constant, which the receiver clock absorbs for the reference constellation but which lands on the near-frozen inter-system bias of any other, so the user re-inflates that bias on a consumed clock-datum discontinuity (`docs/refound/GAD_galileo_adoption_2026-10-13.md`).

Free slant ionosphere. The ionosphere of the network posture is the free per-line-of-sight slant delay $I^i_{r,1}$ of each station, re-estimated every epoch, with no external global map and no spherical-harmonic constraint (`estimation.ionosphere_weighting.enable` off, `network.w6_iono_constraint none`). The datum of the code biases is therefore the pins above and nothing else (Section 20.7).

BeiDou and the second-band rule. BeiDou runs under the same rules (code-bias datum, phase datum, clock pivot) and is not armed in the reference configurations. On B1I and B3I it runs healthy on the 58-station real data set: every epoch is emitted and none is withheld. A station that tracks a non-reference constellation on one band only leaves three directions of its local block collinear: the constellation's inter-system bias, the slant ionosphere of its lines of sight and their ambiguities. One band gives a code row and a phase row per satellite against two free unknowns, so a common shift of the bias is absorbed by the unknowns of every satellite. The positive-definiteness guard of `addStation` then rejects the whole station at every epoch, and its other constellations leave the solve with it. The second-band rule (`network.second_band_admission`, default off) withholds the records of a constellation at a station unless the station holds it on two modelled bands. A band is modelled for a satellite when the epoch carries an admitted code signal and an admitted phase signal on it. Under `network.second_band_scope station` (default) the constellation is admitted at the station while at least `network.second_band_min_sats` of its satellites are on two modelled bands there, and under `satellite` each satellite is tested alone. The unit that fails is withheld whole, upstream of the roster, the models and the detectors, so it is absent as if the RINEX record were missing and its return is read as a re-entry. Each edge is an event (`network/assembly/SecondBandAdmission`). The time-reference constellation is outside the rule. With BeiDou on B1I and B3I the rule reproduces the subtraction of the single-band stations; the setting is unvalidated and not armed (`docs/refound/BDS_beidou_third_constellation_2026-10-18.md`, `docs/refound/BDS305_beidou_305_stations_6h_2026-10-29.md`).

Third band: the receiver band-5 code delay. When the satellite code biases are estimated, the receiver inter-frequency bias columns of the bands outside a constellation's clock-reference pair would absorb the code information that separates that band's satellite clock from its phase bias. The declared rule (`RxIfbRule`, `network/assembly/ParamInventory`) therefore leaves them out of the station's local block, in the station's model at the value their first-epoch seed gave them

(disposition *held at seed*). The rule rests on a premise, that an ionosphere weighting leaves each station's ionosphere level free to fold the band's receiver delay; without that weighting nothing takes the delay up, and the *parameters as armed* table prints a warning. With `network.estimate_rx_extra_band_code_delay` (default off, unvalidated) the columns are estimated in the local block and carried by the receiver code-bias carry. The setting acts only where a constellation tracks a band outside a two-band pair under the run's frequency mode, so it is inert on the dual-frequency reference configurations (`docs/refound/L5B_band5_receiver_delay_2026-10-10.md`).

GLONASS. GLONASS phase biases are excluded. The reason is structural: GLONASS is FDMA, so each satellite transmits on a slightly different frequency, and the receiver inter-channel (hardware) bias does *not* cancel across satellites as it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 20.3 and 20.4) assumes a per-signal bias common to a constellation's satellites. The per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/attic/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is available behind `network.estimate_glonass` (`enableGlonassNetwork`, default off, kept off). It admits GLONASS to the solve with a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is unaffected. The mode is mutually exclusive with the satellite code-bias slice: GLONASS admission skips the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error. This mode is distinct from the float-only GLONASS support that the user engine carries for positioning. With the code-only mode off, the network estimator generates no GLONASS bias products.

Broadcast health. Broadcast satellite health enters the network path through `network.sat_health` (`off`, `pivot` or `exclude`, default `off`; unvalidated, not armed). Under `exclude`, setup removes a satellite's observations from the synchronised epochs at which the broadcast flags it; under `pivot`, only the clock-pivot election reads the health series (Section 20.5).

20.11 The emitted product and the test station

The emitted product. The network emits one `NetworkSolutionEpoch` per content epoch: the per-satellite clock C_0 and the per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). The product also declares its datum (the S record, `SbasisDeclaration`): the reference receiver, the clock pivot of each constellation, the convention of the phase-bias gauge, the code-bias datum, the sign of the biases, and the wind-up model revision (`windup_id`, `kWindUpYawModelId`). A user checks the declaration before it claims integer recovery (`NetworkProductReader::checkContract`), so a product built under a different wind-up convention is refused. Network and user share one wind-up convention, the accumulated wind-up of the `nominal-wu93` frame, and a mismatch is absorbed by the ambiguities and is invisible to the post-fit variance factor.

A user applies the product as known corrections: the satellite clock correction with the factor -1 and each signal bias with the factor $+1$, the constants the network's own rows use (Section 4.4), restoring the integer ambiguity property as described in Section 18.

The test station. The validation follows from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but takes the network's own bias and clock product. A station held *out* of the network solve, with a known coordinate, and positioned as a user against the produced corrections is a closed-loop product test of the whole chain. With integer recovery on, its fixed ambiguities show whether the datum of the network and the integer recovery of the user are mutually consistent; with it off (the reference configurations set `network.closed_loop.ar_mode` to 0) its position error is the grade. The test station runs through the standard PPP path and is not a separate estimator; only the network estimator takes the joint multi-station pipeline of this section. The reference configurations hold out WTZZ00DEU (real data) and SZZ000SIM (simulated data).

Two forms of the test. The *in-process* form (`network.closed_loop.enable`) builds the user engine from a derived configuration during setup (`GnssConfig::deriveClosedLoopConsumerConfig`) and gives it an *owned* SSR correction store instead of the process-wide one, so the network's products cannot leak into a store that another component reads. The emit seam `networkEmitStore` steps the user after that epoch's products have been pushed, which closes the loop *within* the epoch. The user runs a between-satellite single-difference model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`, which the reference configurations arm. The derived configuration is a full copy of the network's and leaves `estimation.accumulate_phase_windup` unchanged, so the in-process user runs the network's wind-up mode; the reference configurations arm accumulation on both beds. The *offline replay* form (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before input validation and setup. It fills the same owned store from an already-written product file through the same reader path, so the recorded stream's discontinuity bookkeeping is reconstructed exactly, and a position-domain case costs minutes instead of a full run. The replay derives its user through the same call from the replay configuration, and the product's S-basis contract records only the wind-up model revision, so a replay matches the network's mode only when its configuration does. The setting `network.closed_loop.consume_network_iono` would supply the user a slant ionosphere from a spherical-harmonic model; the reference configurations build none, so it supplies nothing.

Holds on the product. The reference configurations also arm two emit-side holds and the user's answer to them, treated in Section 24. The phase-bias generation hold (`network.emit_pb_hold`) freezes each emitted phase bias at the first value of its generation and declares every new generation. The phase-clock hold (`network.emit_pb_hold_phase_clock`), a mode of the same hold, keeps the satellite's phase-visible clock over the generation instead. Both run inside `networkEmitStore` (`network/emit/PbHold`) and leave the solve unchanged. Under `network.closed_loop.pb_hold_presence_flip` the derived user (`processing.ssr_pb_hold_presence_flip`) answers a held phase bias that appears on, or leaves, a phase row it already uses as it answers an unheld one.

21 Worked example: estimability in a two-receiver network

This section works the rank deficiency and minimum-constraint datum of Section 20.4 through numerically on the smallest network that shows the structure: two receivers $\mathcal{R} = \{A, B\}$ and four satellites $i \in \{1, 2, 3, 4\}$. Everything not needed for the estimability argument is removed. Positions are known, so the geometric range drops out; a single frequency is used; the troposphere and ionosphere are absent; the data are noise-free. Every number below is therefore exact, and the gauge

freedoms are visible by inspection. The subject of the example is the null space of the system, not its size.

21.1 Code only: clocks and one gauge freedom

With geometry removed, the single-frequency pseudorange of Section 4 collapses to a difference of two clock lumps,

$$P_r^i \;=\; a_r \;-\; g^i, \qquad a_r := c\,\mathrm{d}t_r + B_r, \quad g^i := c\,\mathrm{d}t^i + B^i, \tag{334}$$

where a_r is the receiver clock and g^i the satellite clock. On a single frequency the receiver and satellite *code biases* B_r, B^i are inseparable from the respective clocks, so each "clock" here is a clock + code-bias lump. With one band, the IF-vs-DCB split of Section 20.3 does not arise. The system has $|\mathcal{R}| + |\{i\}| = 2 + 4 = 6$ unknowns and $2 \times 4 = 8$ observations.

The ground truth (metres) is

$$a_A = 5, \quad a_B = 3, \qquad g^1 = 2, \quad g^2 = -1, \quad g^3 = 4, \quad g^4 = 0.5,$$

and Equation (334) maps it to the eight pseudoranges

P_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	3	6	1	4.5
$r=B$	1	4	−1	2.5

The deficiency from two directions. *(i) Invariance.* Adding a constant κ to all six unknowns, $a_r \mapsto a_r + \kappa$ and $g^i \mapsto g^i + \kappa$, leaves every difference $a_r - g^i$, and therefore every observation, unchanged. The truth above and the shifted set $\{15, 13, 12, 9, 14, 10.5\}$ fit the data identically. The data determine clock differences only, never absolute clocks.

(ii) Singular normals. With the unknowns ordered as $\boldsymbol{x} = [\,a_A, a_B, g^1, g^2, g^3, g^4\,]^{\mathrm{T}}$ and unit weights, the normal matrix $\boldsymbol{N} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ is

$$\boldsymbol{N} = \begin{bmatrix} 4 & 0 & -1 & -1 & -1 & -1 \\ 0 & 4 & -1 & -1 & -1 & -1 \\ -1 & -1 & 2 & 0 & 0 & 0 \\ -1 & -1 & 0 & 2 & 0 & 0 \\ -1 & -1 & 0 & 0 & 2 & 0 \\ -1 & -1 & 0 & 0 & 0 & 2 \end{bmatrix}, \qquad \boldsymbol{N}\,\mathbf{1} = \mathbf{0}, \tag{335}$$

a 6×6 matrix of rank 5. The all-ones vector $\mathbf{1} = [1, 1, 1, 1, 1, 1]^{\mathrm{T}}$ is an exact null vector (every row sums to zero), and it is the common-shift direction of view (i). The system $\boldsymbol{N}\boldsymbol{x} = \boldsymbol{u}$ has no unique solution without one added constraint, a *datum*.

Two datum choices, same fit. Pinning one clock removes the gauge. Two natural choices give different parameter values, and both fit the eight observations exactly:

datum (S-basis)	a_A	a_B	g^1	g^2	g^3	g^4
$a_A \equiv 0$ (reference receiver)	0	−2	−3	−6	−1	−4.5
$g^1 \equiv 0$ (reference satellite)	3	1	0	−3	2	−1.5

Under $a_A \equiv 0$, every satellite-clock estimate comes out as $\hat{g}^i = g^i - a_A^{\text{true}}$: the reference receiver's clock is absorbed into all of them. The estimated "satellite clock" is the physical clock shifted by the chosen datum. This offset is the *lumping*, and the user must account for it. For this reason the network declares its datum in the emitted product (Section 20.11).

What is invariant. The estimable quantities (combinations annihilated by the null vector $\mathbf{1}$, i.e. differences) are identical in both bases:

estimable quantity	$a_A \equiv 0$	$g^1 \equiv 0$	(truth)
$a_A - a_B$	2	2	2
$g^2 - g^1$	-3	-3	-3
$g^3 - g^1$	2	2	2
$a_B - g^4$	2.5	2.5	$(= P_B^4)$

The absolute values are a convention; the differences are physical. This is the S-system of [36] in six parameters.

Remark 6 (Why the network cannot difference). The double difference of this code-only model is $\nabla\Delta P_{AB}^{i,1} = (P_A^1 - P_A^i) - (P_B^1 - P_B^i)$. The between-satellite step gives $g^i - g^1$ for both receivers, so the between-receiver step yields exactly $\mathbf{0}$. Double differencing eliminates the entire model: DD removes clocks, and here the model contains nothing else (Section 5). A DD user estimates nothing. The network estimator instead keeps the clocks and pins one datum. This degeneracy is, in miniature, the reason a network estimator exists.

21.2 Adding carrier phase: the satellite phase bias needs the network

The single-frequency carrier phase, with the same clock lumps, is

$$\Phi_r^i \;=\; a_r - g^i \;+\; b^i \;+\; N_r^i, \tag{336}$$

where b^i is the satellite phase bias, the quantity the network emits, and N_r^i is the float ambiguity (in metres). The receiver phase bias and the receiver code-minus-phase offset are constant per receiver and are absorbed into N_r^i. The satellite bias b^i stays explicit because the network emits it. It enters with the partial $+1$, as the network's rows carry a satellite bias (Section 4.4), so it has the sign of the term as it appears in the observation, that is $-b_s^i$ of Equation (5). The truth is extended with

$$b^1 = 0.10, \qquad b^2 = -0.20, \qquad b^3 = 0.05, \qquad b^4 = 0.15,$$
$$N_A^i : 10,\ -5,\ 20,\ 8, \qquad N_B^i : 12,\ -3,\ 18,\ 9,$$

giving the eight carrier-phase observations

Φ_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r = A$	13.10	0.80	21.05	12.65
$r = B$	13.10	0.80	17.05	11.65

Counting: one receiver vs. two. The code fixes the clocks, so per satellite the phase contributes $b^i + N_r^i$. With one receiver this is a single equation $b^i + N_A^i$ in two unknowns. The satellite phase

bias is then inseparable from that receiver's ambiguity, and the equation carries no redundancy. With two receivers there are two equations,

$$b^i + N_A^i, \qquad b^i + N_B^i, \tag{337}$$

in three unknowns (b^i, N_A^i, N_B^i): rank 2, one deficiency per satellite. The estimable two-dimensional space contains the *between-receiver single-difference ambiguity* $\nabla N_{AB}^i = N_A^i - N_B^i$. This quantity does not exist with one receiver, and it is the integer a user resolves (Section 5). The second receiver gives b^i an identity shared across stations.

Datum and solution. The joint code + phase system has a 1-dimensional clock gauge (as before) plus one phase-bias↔ambiguity gauge per satellite: $b^i \mapsto b^i + c_i$, $N_r^i \mapsto N_r^i - c_i$ for all r. The minimum-constraint datum used here has two parts. The first is the clock pin $a_A \equiv 0$. It is S-transform-equivalent to the reference-*satellite* clock pin $g^1 \equiv 0$ that the estimator's S-basis declares (Section 20.4): in its default pin mode the shared-block gauge pins one satellite clock per constellation, not a receiver clock (the optional `network.clock_datum_sum_to_zero` and `network.clock_datum_broadcast_anchored` modes replace the pin). The second part is the set of reference-receiver ambiguity pins $N_A^i \equiv 0$ for every i: the local datum on the pivot receiver, folded into station A's rows. The solution is

	$i=1$	$i=2$	$i=3$	$i=4$
emitted phase bias $\bar{b}^i = b^i + N_A^i$	10.10	−5.20	20.05	8.15
$\hat{N}_A^i$ (datum)	0	0	0	0
$\hat{N}_B^i = N_B^i - N_A^i$	2	2	−2	1

with the clocks unchanged from Section 21.1. The fit checks directly, e.g. $\hat{\Phi}_B^3 = a_B - g^3 + \bar{b}^3 + \hat{N}_B^3 = -2 - (-1) + 20.05 + (-2) = 17.05$, matching the observation.

Reading the result. The solution shows two properties.

- *The lump dominates the emitted bias.* The physical satellite phase bias of satellite 1 is $b^1 = 0.10$ m, while the emitted estimable bias is $\bar{b}^1 = 10.10$ m: it has absorbed receiver A's ambiguity (10 m) for that satellite. This is Odijk's $\bar{b}^i = b^i +$ (pivot-receiver ambiguity) (Section 20.3, Eq. 4/9) in numbers. A user recovers integers only by removing the same lumped quantity, so the datum of the network and the integer recovery of the user must belong to one S-system.
- *The network makes the integer estimable.* The estimable $\hat{N}_B^i = N_B^i - N_A^i = \nabla N_{AB}^i$ is the between-receiver single-difference ambiguity, the quantity an RTK user fixes to an integer. It is absent from a single-receiver problem; the second station introduces it. The satellite phase bias becomes a definite, transferable number at the cost of pinning one reference ambiguity, and it carries that pin.

Up to the geometry and the second frequency it omits, this four-satellite, two-receiver instance has the structure of the synthetic network on which the estimator core is tested. The `RecoversSyntheticTruthUpToSTransform` test (`tests/network/test_network_estimator.cpp`) builds rows of this shape (four receivers and five satellites, from a seeded random truth), with the code row $a_r - g^i$ and the phase row $a_r - g^i + b^i + N_r^i$ of this section. It pins the reference-satellite clock datum $g^1 \equiv 0$ (S-transform-equivalent to the $a_A \equiv 0$ used above) through `NetworkEstimator::solve`, and pins the pivot receiver's ambiguities with tight-prior rows of information 10^{10} in that

station's block. It asserts recovery of the satellite clocks $g^i - g^1$, the lumped phase bias $b^i + N_A^i$, the receiver clocks $a_r - g^1$ and the ambiguities $N_r^i - N_A^i$ to 10^{-6} m: the lumped $\bar{b}^i$ and the single-difference ambiguities above, not the unobservable physical b^i. Two further tests of the file vary the datum on the same network: a second, independent gauge pinned beside the clock pin, and the anchor-free sum-to-zero phase gauge, whose result is a pure S-transform of the pin datum (the between-receiver differences equal the truth).

22 Between-epoch carry: arrowhead admissibility and the DD cycle basis

The Network-mode document establishes the arrowhead normal-equation topology and its exact Schur reduction (Section 20.9), and the worked estimability example establishes the minimum-constraint datum (Section 21). This document covers the step between them: *how information from outside the current epoch enters an arrowhead system.*

The structure of the problem sets the requirements. A network estimator that solves each epoch independently discards what it has learned about the carrier-phase ambiguities, the only constant quantities in the problem. An estimator that carries them without care destroys either the sparsity that keeps the solve affordable or the datum invariance that gives the product its meaning. Section 22.1 lists what is carried at all and the route each family takes. The design of the fold then has three parts, developed in order:

1. An arrowhead system admits exactly *three* kinds of information fold. The columns the new information touches determine which one applies; its physical meaning does not (Section 22.2).
2. The carried quantity must therefore be gauge-free. The natural gauge-free basis of the ambiguity block is the *cycle space* of the receiver–satellite observation graph: the double differences indexed by the spanning tree's *chords* (Section 22.3).
3. Carrying that basis costs one bounded Woodbury correction per connected component, with closed forms for the posterior and for an exact innovation test (Section 22.4).

Section 22.5 then carries a two-receiver, two-satellite network, the smallest network whose observation graph has a cycle, through every step with exact arithmetic.

The reference configurations (`config/golden/golden_field_refound.json` and `config/golden/golden_sim_refound.json`) do not arm the invariant DD carry developed here (`network.carry_invariant` is false in both); they carry the float ambiguity vehicle instead (Section 23). The door analysis of Section 22.2 still governs where that vehicle's folds enter. The shared-border carry takes door 1, and the vehicle's per-station block, with its regression on the shared border, enters before the reduction as whitened rows of that station's own design (`CouplingRowBuilder::buildLocalBlock`, `network/carry/Coupling`), so it touches only that station's blocks and is bound by door 2's ordering constraint. No fold of the reference configurations takes door 3.

Remark 7 (Theoretical footing). Time-linking a *selected* set of linear functions $\boldsymbol{z} = \boldsymbol{F}^{\mathrm{T}}\boldsymbol{x}$ while re-estimating the remainder fresh each epoch is the generalised Kalman filter of Teunissen, Khodabandeh & Psychas (*J. Geod.* 95:108, 2021) and the generalised least-squares filter of Hou & Zhang (*J. Geod.* 99:3, 2025). Both leave open the construction of the transition $\boldsymbol{\Phi}^z$ of the linked functions across a *changing* linked set. Choosing $\boldsymbol{z}$ as the cycle basis closes that gap: every basis change the graph can force is then a unimodular integer matrix (Section 22.3). The spanning-tree S-basis itself is due to Jonge [14], who selects it once and never re-selects it. The construction here extends it to a basis that changes from epoch to epoch.

22.1 The carried families and how they are folded

Two places hold information across epochs: the shared border and each station's local columns. Both fold before the solve of the next epoch and neither adds a station–station coupling.

The shared border. After every kept epoch the posterior value of each border key (satellite clock, satellite phase bias, satellite code bias) and the covariance of the border are stashed (`networkStashCarry`, `network/state/BorderStash`). At the next epoch the fold (`networkInjectD2CarryPrior`, `network/state/BorderPrior`) applies the time update in information form, as a border prior of door 1. The diagonal route gives each key the variance $\mathrm{var} + q\,\Delta t$, with q the rate of the registry kind (`network/assembly/ParamRegistry`). The dense route (`network.shared_carry_full_cov`, on in the reference configurations) folds one information block

$$\boldsymbol{\Lambda} \;=\; (\boldsymbol{P}_{\mathrm{prev}} + \boldsymbol{Q})^{-1}\,, \tag{338}$$

the information-form Kalman time update on the border, where $\boldsymbol{Q}$ is diagonal and $\boldsymbol{P}_{\mathrm{prev}}$ is the stashed covariance in the datum frame of Section 20.9. The dense block keeps the cross-covariances between clock and phase bias that the diagonal route discards. A key absent from the epoch's layout is *marginalised* (its rows and columns of $\boldsymbol{P}_{\mathrm{prev}}$ are not selected) and a key with no matrix row folds on the diagonal. The fold of each key is decided once per epoch (`networkCarryDecide`, `network/state/CarryDisposition`) and read by the border fold and the coupling rows alike. The decision is one of folded, anchored or loose (the temporal clock anchor), reinitialised (the clock monitor's flag), seeded (a parked entry whose column returned inside `network.readmit_seed_horizon_s`, folded at q over the gap since its park), parked, re-referenced (a phase-bias frame the datum moved), retired (the datum of its constellation changed), screened (a non-positive variance), carried by the two-state clock oscillator model (`network.clock_model`, off) or folded by another prior (the spherical-harmonic one, off).

The station-local columns. A local column carried as an independent diagonal prior would count the information it shares with the border twice. The posterior of the joint $(\boldsymbol{x}_s, r)$ factorises into the marginal of $\boldsymbol{x}_s$, which the dense border block carries, and the distribution of r conditional on $\boldsymbol{x}_s$; only the conditional is carried for r. After the solve, the harvest (`networkHarvestLocalCarries`, `network/carry/LocalCarryHarvest`) reads for each carried column c of a station: its value $\hat{r}_c$; its border-known conditional variance $v_c = (\boldsymbol{N}_{rr}^{-1})_{cc}$, which does not depend on the datum (Section 20.9); and its regression on the border, $\boldsymbol{m}_c = \boldsymbol{N}_{sr}\,\boldsymbol{N}_{rr}^{-1}\boldsymbol{e}_c$, restricted to the free (non-datum) shared columns and kept only where $|m_{cj}|$ exceeds a relative threshold (10^{-8}, `kCouplingSparsifyRelThresh`). Both come from one local solve with the cached factor of the station's block. The next epoch carries c as one pseudo-observation row of that station's own design,

$$r_c + \sum_j m_{cj}\,s_j \;=\; \hat{r}_c + \sum_{j\in\mathrm{present}} m_{cj}\,\hat{s}_j, \qquad \sigma^2 \;=\; v_c + q\,\Delta t + \sum_{j\in\mathrm{departed}} m_{cj}^2\,P_{jj}, \tag{339}$$

written in the correction coordinates of the rows (coefficient $+1$ on the local column, m_{cj} on the border). A border term whose column left the layout enters through its marginal variance, the third term of σ^2. The variance is floored (`network.phase_leveling_var_floor_m2` for the ambiguities) and scaled by the carry variance factor of the variance-component monitor. The row is built in PASS B by `CouplingRowBuilder` (`network/carry/Coupling`), before the reduction, so it is a door 2 fold, and combined with the dense border block it reproduces the batch solve for a constant parameter when the carried columns are conditionally independent given the border, that is when $\boldsymbol{N}_{rr}$ is

Table 8: Carried station-local families. The kind is the registry kind (`network/assembly/ParamRegistry`); the file holds the family's PASS B step and its harvest.

family (file)	process noise	gap horizon	route
ambiguity (`LevelingCarry`)	random walk, `network.phase_leveling_q_amb` (default 0, a random constant)	`network.phase_leveling_max_gap_s` (default 120 s, 900 s in the reference configurations)	local block with coupling rows (float vehicle)
receiver code bias (`RxCodeBiasCarry`)	random walk, `network.rx_cb_carry_q` (default $1 \times 10^{-9}\,\mathrm{m^2\,s^{-1}}$)	900 s (fixed, `kRxCbMaxGapS`)	local block (`network.carry_rx_code_bias_in_block`); else diagonal station prior
wet delay (`ZwdCarry`)	random walk, `network.zwd_carry_q`	the ambiguity horizon	local block (`network.carry_zwd_in_block`); else conditional row or diagonal prior
slant ionosphere (`SlantIonoCarry`)	random walk per epoch, `network.carry_iono_in_block_q_m2` (default $2.5 \times 10^{-3}\,\mathrm{m^2}$ at L1)	`network.carry_iono_in_block_max_gap_s` (0 takes the ambiguity horizon)	local block
receiver clock and inter-system bias (`RxClockCarry`)	random walk per epoch, `network.carry_rx_clock_q_m2` (default $0.09\,\mathrm{m^2}$), with a drift state for the clock	`network.carry_rx_clock_max_gap_s`	local block
phase colour (`PhaseColourCarry`)	Gauss–Markov, stationary variance σ^2_{zen} at the zenith with an elevation law, τ 300 s (`network.phase_colour_*`)	900 s (`network.phase_colour_max_gap_s`)	local block

diagonal (`DenseCarryRecursiveEqualsBatch` and `ConditionalCarryRecursiveEqualsBatch` in `tests/network/test_network_estimator.cpp`; the diagonal carry does not). The time update that shrinks each present column, its gauge projection and the exact update are in Section 23. The float vehicle folds the carried columns of a station as one jointly whitened block, which keeps the cross-covariances between them. Each family has a PASS B step that installs its rows and a harvest in the same file; Table 8 lists them.

In the reference configurations the ambiguity and the receiver code-bias families are carried; the wet delay, the slant ionosphere, the receiver clock and the phase colour are re-estimated at every epoch (their settings are off). The stationary standard deviation of a phase-colour state at elevation e is $\sigma_{\mathrm{zen}} / \sin\big(\max(e, e_{\mathrm{floor}})\big)$.

The registry also declares, per family, the events it responds to: a slip, a gap, a datum handover, a decoupling and a re-admission (`ParamEvent`). A carried quantity with a stiff or zero rate needs such a response before it may be armed, because a stiff prior carried past a jump moves the whole solution (Section 22.5); the responses are developed in Section 24. The remaining carry, the invariant DD carry (`network.carry_invariant`), is the only one that is not a conditional row: it folds a prior on the double-differenced cycle basis after the reduction, door 3, and is developed in Sections 22.3 and 22.4.

22.2 Three doors into an arrowhead system

The epoch's system has the structure of Section 20.9: a shared satellite border $\boldsymbol{x}_s$, one private block $\boldsymbol{x}_r$ per station, and zero station–station coupling. Consider a Gaussian prior $\boldsymbol{z} \sim N(\hat{\boldsymbol{z}}, \boldsymbol{P}^z)$ on a linear function $\boldsymbol{z} = \boldsymbol{C}\boldsymbol{x}$ of the epoch's parameters. In information form it is the increment $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^z)^{-1}\boldsymbol{C}$ to the normal matrix and $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^z)^{-1}\hat{\boldsymbol{z}}$ to the right-hand side. The block in which that

increment lands determines how it can be folded.

Door 1: the shared border. If $\boldsymbol{C}$ touches only shared columns, the increment lands entirely in $\boldsymbol{N}_{ss}$. Such a fold is *reduction-preserving*:

> **Lemma.** For any symmetric $\Delta\boldsymbol{N}$ and any $\Delta\boldsymbol{u}$ on the shared border,
>
> $$\widetilde{(\boldsymbol{N}_{ss} + \Delta\boldsymbol{N})} = \tilde{\boldsymbol{N}}_{ss} + \Delta\boldsymbol{N}, \qquad \widetilde{(\boldsymbol{u}_s + \Delta\boldsymbol{u})} = \tilde{\boldsymbol{u}}_s + \Delta\boldsymbol{u}.$$
>
> *Proof.* The Schur subtrahend $\sum_r \boldsymbol{N}_{sr}^r (\boldsymbol{N}_{rr}^r)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}}$ is a function of $\boldsymbol{N}_{sr}$ and $\boldsymbol{N}_{rr}$ only; $\boldsymbol{N}_{ss}$ enters Equation (330) additively. □

A border prior may therefore be folded *before or after* the reduction with identical results. The between-epoch carry of the satellite clock and biases costs no second Schur pass even when it is applied late, for example between the null-space rank gate and the solve. `addSharedInfo` (`network/solve/NormalEquations`) uses this: when a reduction is already cached, it folds the delta in place and leaves the cache current. The dense form of the same carry (`addSharedPriorDense`, `network/solve/Estimator`) takes a *covariance* rather than an information matrix. A carried key absent from the current epoch's layout must be *marginalised*, a row and column deletion on $\boldsymbol{P}^z$; deleting the same rows of $(\boldsymbol{P}^z)^{-1}$ would instead *condition* on the departed key and bias every surviving parameter.

Door 2: one station's own columns, before the reduction. If $\boldsymbol{C}$ touches only station r's local columns, the increment lands in $\boldsymbol{N}_{rr}^r$. For a diagonal prior on columns $c \in \mathcal{C}$ with information weights $\Lambda_c \geq 0$ and information right-hand sides η_c,

$$\boldsymbol{N}_{rr}^r \mathrel{+}= \sum_{c\in\mathcal{C}} \Lambda_c \, \boldsymbol{e}_c \boldsymbol{e}_c^{\mathrm{T}}, \qquad \boldsymbol{u}_r^r \mathrel{+}= \sum_{c\in\mathcal{C}} \eta_c \, \boldsymbol{e}_c, \tag{340}$$

which the implementation applies as a sequence of rank-one updates of the *cached* $\mathrm{LDL}^{\mathrm{T}}$ factor (`addStationPrior`). The raw $\boldsymbol{N}_{rr}$ is never stored or re-factorised, and a positive Λ_c keeps the factor positive-definite by construction (a zero weight skips the factor update and still adds its right-hand-side increment). Unlike door 1, this fold is *not* reduction-preserving: it changes $\boldsymbol{N}_{rr}$, hence the Schur subtrahend, hence $\tilde{\boldsymbol{N}}_{ss}$. It must be applied *before* the reduction; there is no post-reduce fast path. It is cheap for two reasons: it is orthogonal to every other station (the arrowhead has no station–station block to disturb), and it conditions only $\boldsymbol{x}_r$.

The two doors have opposite ordering constraints: door 1 may be late, door 2 must be early. The asymmetry follows directly from Equation (330).

Door 3: local columns across several stations. If $\boldsymbol{C}$ touches local columns of two or more stations, the information increment $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^z)^{-1}\boldsymbol{C}$ has a nonzero (r, r') block. It is representable neither as a border increment nor as a per-station one. Folding it into the unreduced system *destroys the arrowhead*: it fills in the station–station zeros that make the marginalisation exact and cheap.

This is the case that matters in practice. The only ambiguity combinations worth carrying are between-station ones, because a single station's ambiguity is not estimable on its own (Section 21.2). Such a prior is folded *after* the reduction, by Woodbury through the block-diagonal local normals, using the per-station factors the reduction has already computed and left intact. Section 22.4 gives the algebra; the rest of this subsection records the two hazards it creates.

Remark 8 (Post-reduce folds do not compose). Two door-3 folds applied to the same reduced system are each derived against the *pristine* $\boldsymbol{N}_{rr}$. Composing them over coupled local columns is therefore inexact: neither fold sees the other's augmentation of $\boldsymbol{N}_{rr}$, so the composition drops the cross term $\boldsymbol{C}_L \boldsymbol{N}_{rr}^{-1} \boldsymbol{C}_{L,\text{other}}^{\mathrm{T}}$ of the joint middle inverse without any signal. The implementation enforces the exclusion: the sum-to-zero ensemble phase gauge (`foldReducedSumZeroGauges`) and the carried DD priors (`foldReducedDdPriors`) are mutually exclusive, and `NetworkEstimator::solve` throws if both are set. No configuration the loader accepts arms the ensemble gauge any more; the kernel keeps the fold and the exclusion is tested (`DdPriorAndSumZeroGaugeAreMutuallyExclusive`). Under the invariant carry the phase datum is realised through door 2, by the pre-reduce tree pins. These compose *exactly* with a post-reduce door 3 fold, because the fold then reads an already-pinned factor.

The same truncation applies between DD blocks. `foldReducedDdPriors` folds one block per connected component of one (constellation, signal) group, each against the unmodified per-station factors, so the result is exact only when no station is touched by two blocks. A station tracking two signals sits in two blocks, and the fold then makes the block-Jacobi approximation of dropping the cross terms between them.

Remark 9 (A door-3 fold cannot repair a rank deficiency). Let $\delta\boldsymbol{x}_s$ span a null direction of $\tilde{\boldsymbol{N}}_{ss}$, with the matching local response $\delta\boldsymbol{x}_r = -(\boldsymbol{N}_{rr}^{r})^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}} \delta\boldsymbol{x}_s$. If the carried function is estimable, $\boldsymbol{z} = \boldsymbol{C}_L \boldsymbol{x}_L$ is invariant along that direction, so $\boldsymbol{C}_L\, \delta\boldsymbol{x}_L = -\boldsymbol{G}^{\mathrm{T}} \delta\boldsymbol{x}_s = 0$ with $\boldsymbol{G}$ as defined in Equation (343). Every column of $\boldsymbol{G}$ is therefore orthogonal to the null space, the rank-K update $\boldsymbol{G}\boldsymbol{M}\boldsymbol{G}^{\mathrm{T}}$ adds nothing along it, and the declared S-basis dimension the rank gate tests is unchanged by the carry. A carry that changed it would be carrying a gauge rather than information.

22.3 The observation graph and its cycle space

Door 3 admits only gauge-free functions, and the carried basis is chosen accordingly. The graph structure determines that choice; no numerical criterion enters.

The graph. For a constellation g and a signal s, let $G_{g,s} = (V, \mathcal{E})$ be the graph whose vertices $V = \mathcal{R} \cup \mathcal{S}_g$ are the receivers and the satellites, and whose *arcs* are the receiver–satellite pairs carrying an ambiguity this epoch: $(r, i) \in \mathcal{E}$ iff station r tracks satellite i on signal s. There is at most one arc per pair, so an unordered vertex pair is a unique arc identity (`ArcKey`, `network/ar/ArcGraph`). The graph is *bipartite* by construction: no arc joins two receivers or two satellites.

Trees and chords. A connected component with n_c receivers, m_c satellites and $|\mathcal{E}_c|$ arcs has a spanning tree of $n_c + m_c - 1$ edges. The remaining

$$K_c \;=\; |\mathcal{E}_c| \;-\; (n_c + m_c - 1) \tag{341}$$

arcs are the *chords*. Adding a chord to the tree closes exactly one cycle, its *fundamental cycle*: the chord plus the unique tree path joining its endpoints. The K_c fundamental cycles are a basis of the component's cycle space. Equation (341) is the first Betti number of the component. It is the exact number of independent double differences the component supports, and it is a property of the *graph*, independent of the estimator.

The carried functions. The graph is bipartite, so every cycle alternates receiver–satellite–receiver–satellite, has even length, and admits an alternating signing of its arcs. For chord c with

fundamental cycle $\mathcal{C}_c$, define

$$z_c \;=\; \sum_{(r,i)\,\in\,\mathcal{C}_c} \sigma_{(r,i)}\, N^i_{r,s}, \qquad \sigma_{(r,i)} = \pm 1 \text{ alternating}, \tag{342}$$

where the sign is $+1$ when the arc is traversed from its lower vertex id to its higher one, and receiver ids lie below satellite ids, so the sign alternates around the cycle (`network/ar/ArcGraph`). For the shortest cycle (length four) z_c is a plain double difference $N^{i_1}_{r_1} - N^{i_1}_{r_2} + N^{i_2}_{r_2} - N^{i_2}_{r_1}$, and for a longer fundamental cycle is a higher-order alternating difference on six or more columns.

> **Proposition (gauge-freeness).** z_c is invariant under both ambiguity gauges of the network model: the receiver shift $N^i_r \mapsto N^i_r - d_r$ for all i, and the satellite shift $N^i_r \mapsto N^i_r - c^i$ for all r.
>
> *Proof.* A cycle visits each of its vertices exactly once, and each visited vertex is incident to exactly *two* arcs of the cycle, which by the alternating signing carry opposite signs. A shift attached to that vertex (d_r at a receiver, c^i at a satellite) therefore contributes $(+1)\delta + (-1)\delta = 0$ to Equation (342). Summing over vertices gives invariance. □

Hence z_c touches no shared column: the receiver and satellite phase biases cancel analytically rather than numerically. Three consequences follow, and together they make the cycle basis the carried object:

- **Estimable in every S-system.** z_c is invariant under the datum, so it may be compared across epochs whose datum differs and graded without an S-transform (Section 20.4).
- **Integer, and constant between slips.** z_c is an integer combination of integer ambiguities, and its process noise is $q_z = 0$ in the absence of a cycle slip. It is the *only* part of the ambiguity block for which a zero-q time link is physically correct.
- **Basis changes are unimodular.** Two spanning trees of the same arc set give cycle bases related by an integer matrix $\boldsymbol{T}$ with $\det \boldsymbol{T} = \pm 1$, so $\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$, $\boldsymbol{P}^{z\prime} = \boldsymbol{T}\boldsymbol{P}^z\boldsymbol{T}^{\mathrm{T}}$ is an *exact* change of basis with no information loss. Basis churn defeats any carry of the raw ambiguities, whose individual values depend on the datum; in the cycle basis it has no effect by construction.

Graph events are exact operations. Every way the graph can change between epochs maps to an exact operation on the carried $(\hat{\boldsymbol{z}}, \boldsymbol{P}^z)$. The z-store (`ZComponent`, `network/ar/ZStore`) implements each as a primitive and serves as the test oracle of the driver; the producing code does not build it.

Graph event	Operation on the store
Chord dies (arc lost)	Drop that row and column of $\boldsymbol{P}^z$, pure marginalisation
Chord born (arc closes a new cycle)	Append an entry initialised by tap-off $\hat{z}_{\mathrm{new}} = \boldsymbol{f}^{\mathrm{T}}\hat{\boldsymbol{x}}$ with its conditional variance and cross-covariances
Leaf growth (arc brings in a new vertex)	Extends the tree, closes no cycle, touches no z
Tree edge dies	Pivot a covering chord into the tree ($\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$, $\boldsymbol{T}$ unimodular), then kill the arc as a chord. An uncovered tree edge is a cut edge: its death splits the component, which merely re-partitions existing blocks
Components merge	Union the two blocks block-diagonally (they were independent), promote one bridge arc to the tree, tap off any surplus bridges

The network estimator does not apply these as a sequence of events. The per-epoch driver (`network/carry/InvariantCarryDriver`) receives the current epoch's tree/chord partition from the chain-datum election, takes the trees and fundamental cycles from `network/ar/ArcGraph` (`arcgraph::singleTreeOf`, `arcgraph::fundamentalCycle`) and computes the *composed* transform directly. A new chord's fundamental cycle lies in the previous epoch's cycle space if and only if every arc of that cycle was present last epoch; its carried mean is then $\hat{z}_c^{\text{new}} = \sum_j w_j \, \hat{z}_j^{\text{old}}$, with w_j the incidence of old chord j's arc in the new cycle. Assembling $\boldsymbol{W}$ over the contributing components gives $\hat{\boldsymbol{z}}^{\text{prior}} = \boldsymbol{W}\hat{\boldsymbol{z}}^{\text{old}}$ and $\boldsymbol{P}^{z,\text{prior}} = \boldsymbol{W}\boldsymbol{P}^{z}\boldsymbol{W}^{\mathrm{T}}$. A dead chord's column of $\boldsymbol{W}$ is zero (marginalisation). Merges and splits follow from the contributor gather with no special case. `networkBuildCarryDdPriors` (`network/carry/DdPriorBuild`) resolves each arc to an estimator (station, local ambiguity column), marks the tree edges as the pinned gauge and the others as chords, and excludes GLONASS, whose FDMA signals have no common-wavelength integer double difference. `networkInstallCarry` installs the posterior after the accepted solve, so a withheld or contaminated epoch never enters the carry.

A chord whose cycle uses a newborn arc is *fresh*. The driver gives it a finite diffuse prior (variance `fresh_prior_var`, default 1.0e6 in state units squared) rather than the exact zero-information tap-off of the table. Its posterior is therefore the data-only tap-off up to a term of the order of the ratio of the data variance to `fresh_prior_var`. An arc flagged as slipped this epoch counts as absent from the previous arc set, so every chord whose cycle uses it is also born fresh. The production caller passes only arcs that resolve to an estimator column; a carried chord whose cycle has an unresolvable arc this epoch is dropped from the carry and re-tapped when the arc returns.

Remark 10 (Why the tree must avoid slipped arcs). A slip on a *chord* costs exactly one carried DD. A slip on a *tree edge* corrupts every fundamental cycle routed through it. With `network.phase_datum_eq34_slip_aware_election` armed (default off), the election therefore keeps slipped arcs out of the tree wherever the graph allows it. The argument is structural and independent of any detector threshold; the setting measured net-harmful on a 63-station case and is off (`docs/SPEC_d5_churn_limit_cycle.md`).

22.4 Folding the carried double differences

Let one connected component carry K double differences with prior mean $\hat{\boldsymbol{z}}$ and covariance $\boldsymbol{P}^{z}$, expressed as $\boldsymbol{z} = \boldsymbol{C}_L \boldsymbol{x}_L$ where $\boldsymbol{C}_L$ is the $K \times n_L$ signed incidence matrix of Equation (342) over the epoch's local ambiguity columns, and $\boldsymbol{C}_{L,r}$ is its restriction to station r. This is a door-3 fold. Woodbury through the block-diagonal local normals gives, per component,

$$\boldsymbol{B} = \sum_r \boldsymbol{C}_{L,r}\boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}, \quad \boldsymbol{G} = \sum_r \boldsymbol{N}_{sr}\boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}, \quad \boldsymbol{z}_{\text{free}} = \sum_r \boldsymbol{C}_{L,r}\boldsymbol{N}_{rr}^{-1}\boldsymbol{u}_r, \quad \boldsymbol{M} = (\boldsymbol{P}^{z} + \boldsymbol{B})^{-1}, \tag{343}$$

and the reduced system takes the rank-K correction

$$\boxed{\begin{aligned} \tilde{\boldsymbol{N}}_{ss} &\mathrel{+}= \boldsymbol{G}\,\boldsymbol{M}\,\boldsymbol{G}^{\mathrm{T}}, && (344)\\ \tilde{\boldsymbol{u}}_s &\mathrel{+}= \boldsymbol{G}\,\boldsymbol{M}\,(\boldsymbol{z}_{\text{free}} - \hat{\boldsymbol{z}}). && (345) \end{aligned}}$$

Every ingredient of Equation (343) comes from the *cached* per-station $\mathrm{LDL}^{\mathrm{T}}$ factors. No re-reduction is needed and the factors are left unmodified. The only new inverse is the $K \times K$ middle matrix $\boldsymbol{M}$, which is per component and bounded in size. Back-substitution then returns the *constrained* locals,

$$\boldsymbol{x}_r \mathrel{-}= \boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}\,\boldsymbol{M}\,\big(\boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s - \hat{\boldsymbol{z}}\big), \tag{346}$$

so the ambiguities read by the quality-control layer and by the next epoch's tap-off are in the same frame as the fold.

Posterior. The value the store carries forward is the prior shrunk towards the data-implied DD,

$$\hat{\boldsymbol{z}}^{+} = \hat{\boldsymbol{z}} + \boldsymbol{P}^{z}\,\boldsymbol{M}\,\boldsymbol{v}, \qquad \boldsymbol{v} = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_{s} - \hat{\boldsymbol{z}}, \tag{347}$$

with covariance, in the datum-as-known shared frame $\boldsymbol{\Sigma}_{ss}$ (the datum frame of Section 20.9: the solve's shared covariance with every datum-pinned and gauge column treated as exact),

$$\boldsymbol{P}^{z+} = \underbrace{\left(\boldsymbol{P}^{z} - \boldsymbol{P}^{z}\boldsymbol{M}\boldsymbol{P}^{z}\right)}_{\text{conditional on } \boldsymbol{x}_{s}} + \left(\boldsymbol{P}^{z}\boldsymbol{M}\boldsymbol{G}^{\mathrm{T}}\right)\boldsymbol{\Sigma}_{ss}\left(\boldsymbol{G}\boldsymbol{M}\boldsymbol{P}^{z}\right). \tag{348}$$

The first term is the posterior given the border; the second restores the coupling through it. $\boldsymbol{\Sigma}_{ss}$ is datum-constrained, but $\boldsymbol{P}^{z+}$ is datum-invariant because $\boldsymbol{z}$ is estimable. `ddPriorPosteriorMean` and `ddPriorPosteriorCovariance` evaluate the two expressions per block from the cached fold factors. The driver builds the datum frame once per install (`NetworkEstimator::datumFrameSharedCov`) and passes it to every block.

Innovation. The same factors give an exact test of the carry against the epoch's data (`ddPriorInnovation`). Under the null hypothesis that the carried DD is still valid,

$$\boldsymbol{v} = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_{s} - \hat{\boldsymbol{z}}, \qquad \mathrm{Cov}(\boldsymbol{v}) = \boldsymbol{M}^{-1} - \boldsymbol{G}^{\mathrm{T}}\,\boldsymbol{\Sigma}_{ss}\,\boldsymbol{G}, \tag{349}$$

and $|v_i|/\sqrt{\mathrm{Cov}(\boldsymbol{v})_{ii}}$ is the standardised statistic that flags a slipped carried DD for re-tapping. With `network.carry_invariant_innov_gate` armed (default off), the driver flags a chord when the statistic exceeds `network.carry_invariant_innov_k` (default 4.0) times a scale factor: 1 by default, or the LS-VCE estimate of the empirical innovation scale when `network.carry_invariant_innov_calibrate` is armed. The *minus* sign in Equation (349) is exact rather than a conservative approximation. $\boldsymbol{x}_s$ is itself influenced by the prior through Equation (345), and subtracting $\boldsymbol{G}^{\mathrm{T}}\boldsymbol{\Sigma}_{ss}\boldsymbol{G}$ corrects for testing the carry against a solution the carry helped produce. Consistency with Equations (347) and (348) is direct: $\boldsymbol{P}^{z}\boldsymbol{M}\,\mathrm{Cov}(\boldsymbol{v})\,\boldsymbol{M}\boldsymbol{P}^{z} = \boldsymbol{P}^{z} - \boldsymbol{P}^{z+}$.

The hard-constraint limit. Setting $K = 1$, $\hat{z} = 0$ and $\boldsymbol{P}^{z} \to \boldsymbol{0}$ turns Equations (344) and (345) into the rank-one fold of an exact linear constraint $\sum_r z_r = 0$ on local columns, with $\boldsymbol{M} \to \boldsymbol{B}^{-1}$. That is the anchor-free sum-to-zero phase-bias gauge (`foldReducedSumZeroGauges`). The DD prior is its soft generalisation: the two are the same object, which is why they cannot be composed.

Positive-definiteness, and why it is counted. $\boldsymbol{P}^{z} + \boldsymbol{B}$ is positive-definite in exact arithmetic, but it can fail to be numerically. A stiff carry ($\boldsymbol{P}^{z}$ near zero) added to an ill-conditioned $\boldsymbol{B}$ subtracts operands of similar magnitude to reach a much smaller result. When the screen fails, the fold is applied *inertly*, $\boldsymbol{M} := \boldsymbol{0}$: the reduced system is untouched and the posterior equals the prior. The block keeps its slot, so the per-block readouts stay index-aligned with the carry's block list.

The inert branch raises no error, and that makes it hazardous. The component's carried sub-state stops taking data, while every downstream reading of it (its formal σ, the innovation test, any ambiguity-resolution probe) continues to report on a quantity that is no longer updated. A carry can therefore diffuse for hundreds of epochs behind clean logs. The implementation treats the PD screen as *always-on telemetry*: per-fold counts of degenerate blocks and their dimension, the worst relative pivot, and the count of repairs are exposed unconditionally (`DdFoldStats`, `network/`

`solve/NormalEquations`). An optional PD-safe path (`network.carry_invariant_pd_safe`, default off) replaces the absolute screen with an equilibrated factorisation and a scale-relative pivot test, evaluates the posterior in a product form that avoids the subtraction, and applies a counted eigenvalue-clamp repair to round-off-level failures. The project rule is that run-end counters are the telemetry of an armed path (`CLAUDE.md`, the pre-flight invariants for estimation changes): an inert path needs a counter, because the absence of an error is no evidence that the mechanism ran.

22.5 Worked example: the smallest network with a cycle

This subsection carries a complete network through every step above with exact arithmetic. The network is the smallest one whose observation graph has a cycle: two receivers, two satellites, one signal. As in Section 21, everything inessential is removed: known coordinates (so the geometric range drops out), one frequency, no atmosphere, noise-free data and unit weights, so every number below is exact. Ambiguities are in cycles.

Model and truth. With geometry and the receiver/satellite clocks removed by the code observations, the carrier phase reduces to

$$\Phi_r^i \;=\; \beta_r \;+\; b^i \;+\; N_r^i, \qquad r \in \{A, B\}, \quad i \in \{1, 2\}, \tag{350}$$

where β_r is the receiver phase bias, b^i the satellite phase bias (the quantity the network emits), and $N_r^i \in \mathbb{Z}$ the ambiguity. The truth is

$$\beta_A = 0.30, \quad \beta_B = -0.20, \qquad b^1 = 0.10, \quad b^2 = -0.15,$$

$$N_A^1 = 3, \quad N_A^2 = -1, \quad N_B^1 = 5, \quad N_B^2 = 2,$$

which through Equation (350) gives the four observations

Φ_r^i	$i = 1$	$i = 2$
$r = A$	3.40	−0.85
$r = B$	4.90	1.65

The graph, its tree, and its one chord. The observation graph has $n_c = 2$ receivers, $m_c = 2$ satellites and $|\mathcal{E}_c| = 4$ arcs, so by Equation (341) its spanning tree has $2 + 2 - 1 = 3$ edges and it has exactly $K_c = 1$ chord (Figure 4). With the tree $\{(A, 1), (A, 2), (B, 2)\}$, the chord is $(B, 1)$, and its fundamental cycle $B \to 1 \to A \to 2 \to B$ gives the single carried function

$$z \;=\; N_B^1 - N_A^1 + N_A^2 - N_B^2. \tag{351}$$

Two exact checks of the Proposition follow. From the truth, $z = 5 - 3 + (-1) - 2 = -1$, an integer as required. From the *observations* alone, with nothing estimated,

$$(\Phi_B^1 - \Phi_B^2) - (\Phi_A^1 - \Phi_A^2) = 3.25 - 4.25 = -1,$$

because β_A, β_B, b^1 and b^2 all cancel in the double difference. The carry transports this single number -1; every other parameter in this subsection depends on the datum.

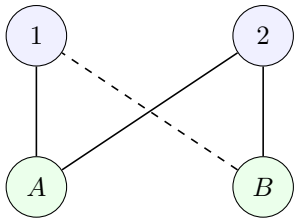


Figure 4: The smallest observation graph with a cycle: two receivers, two satellites, four arcs. Three arcs form a spanning tree; the fourth is a chord, and its fundamental cycle is the one estimable double difference Equation (351).

The tree pins are the datum; the chord is left free. Model Equation (350) has $2+2+4=8$ unknowns and 4 observations, hence a four-dimensional null space (choose $\delta\beta_A, \delta\beta_B, \delta b^1, \delta b^2$ freely; the δN_r^i follow). Pinning the three *tree* ambiguities $N_A^1 = N_A^2 = N_B^2 = 0$ removes three of those four dimensions, leaving the single direction $(\beta_A, \beta_B, b^1, b^2) \mapsto (\beta_A + t,\ \beta_B + t,\ b^1 - t,\ b^2 - t)$, the one remaining receiver-versus-satellite gauge (the network's clock gauge in miniature). One *shared* pin, $b^1 \equiv 0$, removes it.

These four arcs contain the whole datum architecture. The tree edges are pinned through door 2 (locally, before the reduction), the last gauge is fixed by the shared minimum-constraint datum, and the chord is left free: it is the only ambiguity that carries information rather than convention. The pins are taken here as *exact* to keep the arithmetic simple. The implementation realises them as tight finite priors folded by `addStationPrior`, so the pinned columns remain present in $\boldsymbol{N}_{rr}$. The exact-pin idealisation has one visible consequence: with the tree ambiguities identically zero, $\boldsymbol{C}_L$ in Equation (351) collapses onto the chord's own column. With finite pins it retains all four entries, and the fold algebra below has the same form.

Arrowhead blocks. The shared block is ordered $\boldsymbol{x}_s = [\,b^1, b^2\,]^{\mathrm{T}}$ and the free locals $\boldsymbol{x}_A = [\,\beta_A\,]$, $\boldsymbol{x}_B = [\,\beta_B,\ N_B^1\,]^{\mathrm{T}}$. With unit weights the blocks of Equation (329) are

$$\boldsymbol{N}_{ss}^A = \boldsymbol{I}_2, \quad \boldsymbol{N}_{sr}^A = \begin{bmatrix}1\\1\end{bmatrix}, \quad \boldsymbol{N}_{rr}^A = \begin{bmatrix}2\end{bmatrix}, \quad \boldsymbol{u}_s^A = \begin{bmatrix}3.40\\-0.85\end{bmatrix}, \quad \boldsymbol{u}_r^A = \begin{bmatrix}2.55\end{bmatrix},$$

$$\boldsymbol{N}_{ss}^B = \boldsymbol{I}_2, \quad \boldsymbol{N}_{sr}^B = \begin{bmatrix}1 & 1\\1 & 0\end{bmatrix}, \quad \boldsymbol{N}_{rr}^B = \begin{bmatrix}2 & 1\\1 & 1\end{bmatrix}, \quad \boldsymbol{u}_s^B = \begin{bmatrix}4.90\\1.65\end{bmatrix}, \quad \boldsymbol{u}_r^B = \begin{bmatrix}6.55\\4.90\end{bmatrix},$$

with $(\boldsymbol{N}_{rr}^B)^{-1} = \left[\begin{smallmatrix}1 & -1\\-1 & 2\end{smallmatrix}\right]$. There is no A–B block: the stations share only b^1 and b^2.

Schur reduction. Eliminating each station's locals through Equation (330) gives

$$\boldsymbol{N}_{ss}^A - \boldsymbol{N}_{sr}^A(\boldsymbol{N}_{rr}^A)^{-1}\boldsymbol{N}_{sr}^{A\mathrm{T}} = \begin{bmatrix}0.5 & -0.5\\-0.5 & 0.5\end{bmatrix}, \qquad \boldsymbol{u}_s^A - \boldsymbol{N}_{sr}^A(\boldsymbol{N}_{rr}^A)^{-1}\boldsymbol{u}_r^A = \begin{bmatrix}2.125\\-2.125\end{bmatrix},$$

while station B contributes exactly zero to both: it has two observations and two free locals, so eliminating the locals absorbs its rows entirely. Summing,

$$\tilde{\boldsymbol{N}}_{ss} = \begin{bmatrix}0.5 & -0.5\\-0.5 & 0.5\end{bmatrix}, \qquad \tilde{\boldsymbol{u}}_s = \begin{bmatrix}2.125\\-2.125\end{bmatrix}, \qquad \mathrm{eig}(\tilde{\boldsymbol{N}}_{ss}) = \{0,\ 1\}. \tag{352}$$

The reduced system has rank 1 and one zero eigenvalue, with eigenvector $[1, 1]^{\mathrm{T}}$: the single gauge predicted above. A null-space rank gate that declares dimension 1 for this configuration passes; one that declares 0 or 2 throws (Section 20.4).

Datum, solve, back-substitute. Applying the shared pin $b^1 \equiv 0$ to Equation (352) and solving gives $\hat{b}^2 = -4.25$, and back-substituting through Equation (332),

$$\hat{\beta}_A = 3.40, \qquad \hat{\beta}_B = 5.90, \qquad \hat{N}_B^1 = -1.00.$$

The fit is exact (all four residuals vanish), and the result agrees to machine precision with a dense stacked solve of the same system: the lossless-marginalisation claim of Section 20.9 in numbers.

These estimates do not resemble the truth. The estimated satellite phase bias $\hat{b}^2 = -4.25$ differs from $b^2 = -0.15$, and $\hat{N}_B^1 = -1$ differs from $N_B^1 = 5$; each has absorbed the datum. The chord's double difference in this frame is

$$\hat{z} = \hat{N}_B^1 - \hat{N}_A^1 + \hat{N}_A^2 - \hat{N}_B^2 = -1 - 0 + 0 - 0 = -1,$$

identical to the truth and to the value read directly off the observations. Datum invariance here is an identity, not an asymptotic property.

Folding a carried DD. Suppose the previous epoch carried this chord. With $\boldsymbol{C}_L = [\,0 \;\; 1\,]$ on station B (one row, the exact-pin collapse noted above), Equation (343) gives

$$\boldsymbol{B} = 2, \qquad \boldsymbol{G} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad \boldsymbol{z}_{\text{free}} = 3.25,$$

and the identity $z = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s = 3.25 - 4.25 = -1$ recovers the DD: $\boldsymbol{z}_{\text{free}}$ is a data-only quantity in the correction frame, not the DD itself. Also $\boldsymbol{G}^{\mathrm{T}}[1,1]^{\mathrm{T}} = 0$: the fold direction is orthogonal to the gauge, as the second Remark of Section 22.2 requires, so no choice of prior can make this carry fix the datum.

A consistent carry is a no-op. With $\hat{z} = -1$ and a rigid prior $\boldsymbol{P}^z = 0$, Equations (344) and (345) reproduce $\hat{b}^2 = -4.25$, $\hat{\beta}_A = 3.40$, $\hat{\beta}_B = 5.90$, $\hat{N}_B^1 = -1$ and zero residuals. The innovation is $v = 3.25 - 4.25 - (-1) = 0$. Carrying information the data already implies changes nothing.

A stale carry distorts the whole solution. Suppose instead that the chord slipped by one cycle undetected, so $\hat{z} = 0$ is folded rigidly. Re-solving gives

	$\hat{b}^2$	$\hat{\beta}_A$	$\hat{\beta}_B$
consistent carry ($\hat{z} = -1$)	-4.25	3.40	5.90
stale carry ($\hat{z} = 0$)	-3.75	3.15	5.15

and post-fit residuals of $+0.25$, -0.25, -0.25, $+0.25$ cycles on rows $(A,1)$, $(A,2)$, $(B,1)$, $(B,2)$ respectively. The one-cycle error in a stiff carried quantity does not stay in the ambiguity that slipped. It moves the *emitted satellite phase bias* by half a cycle and spreads the remainder evenly over every observation in the network. This is the general mechanism by which a stale zero-q carry damages a solution: the network cannot reject information asserted with infinite confidence, so it redistributes the contradiction into whatever states can absorb it. It is the reason for the project rule that any zero-q or stiff time link requires a wired, tested slip, jump or event response before it may be armed (`CLAUDE.md`, the pre-flight invariants for estimation changes). The same rule applies to the edges of the carry: every mechanism that changes which information reaches which states is an event at both edges, with counters and identities. The driver treats a slipped or unresolvable arc as a decoupling event (the chord is dropped or born fresh, and counted), and a returning arc re-couples only through a fresh tap-off.

The innovation test, and its limits. From Equation (349), with $\boldsymbol{\Sigma}_{ss} = \mathrm{diag}(0,\, 1)$ (the pinned column treated as exact),

$$v = 3.25 - 3.75 - 0 = -0.5, \qquad \mathrm{Cov}(v) = \boldsymbol{M}^{-1} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{\Sigma}_{ss}\boldsymbol{G} = 2 - 1 = 1, \qquad \frac{|v|}{\sqrt{\mathrm{Cov}(v)}} = 0.5.$$

The test detects the slip at only 0.5σ, far below any usable flag threshold (the default `network.carry_invariant_innov_k` is 4.0). The residuals show why: with four observations and almost no redundancy, the parameters absorbed half the error, so only half of it reached the innovation. The statistic is exact rather than conservative; its *power* is a property of the network's redundancy, and a two-by-two network has almost none. Any estimation-side slip test is diagnostic of the carry only in proportion to the part of the error the network cannot absorb.

A tree-edge death: the unimodular pivot. The two-by-two network is too small to show a pivot: with one chord, every tree edge whose death matters is a cut edge. A third satellite gives $n_c = 2$, $m_c = 3$, $|\mathcal{E}_c| = 6$, so the tree has four edges and there are $K_c = 2$ chords. The tree $\{(A,1),(A,2),(A,3),(B,1)\}$ with chords $(B,2)$ and $(B,3)$ gives

$$z_1 = N_B^2 - N_A^2 + N_A^1 - N_B^1, \qquad z_2 = N_B^3 - N_A^3 + N_A^1 - N_B^1.$$

Let the tree edge $(A,1)$ die. Both chords cover it, so one covering chord, $(B,2)$, is promoted into the tree and the dead arc is killed as a chord. Against the new tree $\{(A,2),(A,3),(B,1),(B,2)\}$ the cycle basis is

$$z_1' = N_A^1 - N_B^1 + N_B^2 - N_A^2 = z_1, \qquad z_2' = N_B^3 - N_A^3 + N_A^2 - N_B^2 = z_2 - z_1,$$

that is $\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$ with

$$\boldsymbol{T} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \qquad \det \boldsymbol{T} = 1.$$

$\boldsymbol{T}$ is integer with unit determinant, so $\boldsymbol{P}^{z\prime} = \boldsymbol{T}\boldsymbol{P}^{z}\boldsymbol{T}^{\mathrm{T}}$ is an exact change of basis: the carried covariance is transformed without approximation. Killing the dead arc's row of $\boldsymbol{z}'$ is then a marginalisation, and the surviving z_2' continues with no loss. A carry of the raw ambiguities would require an S-transformation of a datum-dependent state for the same graph event; the cycle basis removes that requirement.

22.6 Where each object lives

Object	Symbol	Home
Arrowhead accumulation, Schur reduction, back-substitution	`addStation`, `reduce`, `reconstructStation`	`network/solve/NormalEquations`
Door 1: shared border prior	`addSharedInfo`, `addSharedPriorDense`	`network/solve/NormalEquations`, `network/solve/Estimator`
Border carry: stash, decision, fold	`networkStashCarry`, `networkCarryDecide`, `networkInjectD2CarryPrior`	`network/state/BorderStash`, `CarryDisposition`, `BorderPrior`
Door 2: pre-reduce local prior and coupling row	`addStationPrior`, `CouplingRowBuilder`	`network/solve/NormalEquations`, `network/carry/Coupling`
Harvest of the station-local carries	`networkHarvestLocalCarries`, `networkHarvestColumn`	`network/carry/LocalCarryHarvest` and the family files
Door 3: post-reduce Woodbury folds	`foldReducedSumZeroGauges`, `foldReducedDdPriors`	`network/solve/NormalEquations`
Cycle algebra (trees, fundamental cycles, chord restriction)	`ArcKey`, `arcgraph::singleTreeOf`, `arcgraph::fundamentalCycle`	`network/ar/ArcGraph`
Lifecycle primitives (oracle)	`ZComponent`	`network/ar/ZStore`
Per-epoch reconcile $\boldsymbol{W}$, posterior install, innovation gate	`buildDdPriors`, `installPosterior`	`network/carry/InvariantCarryDriver`, `DdPriorBuild`, `InstallCarry`
Posterior and innovation readouts	`ddPriorPosteriorMean`, `ddPriorPosteriorCovariance`, `ddPriorInnovation`	`network/solve/NormalEquations`
PD screen telemetry	`DdFoldStats`, `ddPriorBlockDegenerate`	`network/solve/NormalEquations`
Datum and rank gate	`NetworkSbasis`, `assertNullSpaceRank`	`network/solve/Sbasis`

23 The float ambiguity vehicle

The reference configurations (`config/golden/golden_field_refound.json` for real data, `config/golden/golden_sim_refound.json` for the simulated network) carry the carrier-phase ambiguities between epochs as float states of each station's local block. The invariant double-difference carry of Section 22 is off (`network.carry_invariant` false), and the loader refuses the float vehicle on a configuration that enables it. The invariant carry transports only the gauge-free chords and folds them through door 3 after the reduction (Section 22.2). The float vehicle transports every station's own ambiguity columns, conditional on the border, as rows of that station's block before the reduction. Gauge invariance then comes from each row's full border regression (Section 23.2) and from the held phase-datum basis (Section 23.5). The ambiguity process noise is zero: the parameter registry (`network/assembly/ParamRegistry`) takes it from `network.phase_leveling_q_amb` (class `leveling_ambiguity` of `network.process_noise`), loader default 0, kept by the reference configurations. The loader requires `network.phase_leveling` and `network.carry_conditional_coupling` (which itself requires `network.shared_carry_full_cov`) and forces the retired station-block switch `network.phase_leveling_local_block` off, because the vehicle supplies the joint block structurally.

23.1 The armed posture

Table 9 lists the settings both reference configurations arm that change the estimate or the emitted product and belong to the ambiguity carry (the `network.` prefix omitted).

Built and not armed. `network.amb_float_birth_var_m2` is absent, so its default 0 disables the birth floor of Equation (359); the drop-only floor `network.amb_float_drop_var_m2` is 0; `network.amb_float_declared_handover`, `network.carry_rebase_complete`, `network.carry_coupling_exact_cross`, `network.handover_stransform_before_rows`, `network.handover_stransform_dense`, `network.carry_coupling_clock_clone`, `network.carry_coupling_shared_history` and `network.ctu_switch_on_resume` are off. The clock clone and the shared history carry the satellite-clock uncertainty shared between the rows of one satellite; the self-contained clock level they were built for fails against an external clock product over several days (the two keys' rows in `docs/CONFIG_promotion_ledger.json`), so both stay off. The retired keys `network.phase_leveling_los_scale`, `network.carry_coupling_joint_whiten` and `network.amb_float_q_m2s` no longer exist.

23.2 The carried local column

The entry. `LocalCarry` (`network/driver/RunTypes`) is keyed by (station, satellite, signal) in the arc table (`net::ArcState`). After the accepted solve the harvest (`networkHarvestLocalCarries`, ambiguity rows in `networkHarvestAmbiguityStation` of `network/carry/LevelingCarry`) writes, for carried column c of station r (ambiguity in metres),

$$\hat{a}_c = x_c^{\mathrm{lin}} + \widehat{\delta x}_c, \qquad v_c = \left(\boldsymbol{N}_{rr}^{-1}\right)_{cc}, \qquad \boldsymbol{m}_c = \boldsymbol{N}_{sr}\,\boldsymbol{N}_{rr}^{-1}\boldsymbol{e}_c, \qquad C_{cc'} = \left(\boldsymbol{N}_{rr}^{-1}\right)_{cc'}: \tag{353}$$

`value_abs`, the pure conditional variance `var`, the border regression `coupling` (one term per border column j above `kCouplingSparsifyRelThresh` of the row maximum, with the solved value $\hat{s}_j$ and the datum-frame variance p_j, 0 when pinned) and the cross-covariance block `blk`, all from one local solve $\boldsymbol{N}_{rr}^{-1}\boldsymbol{e}_c$ (`stationRegression`). The entry also keeps `var_first` (the first-stash v_c), `var_birth`, the birth stamps and `relevel_s`, the time of the last re-level. The anchored column of each family is excluded at inject and harvest: its value is the datum.

Why the regression row. With $\boldsymbol{u}_r = \boldsymbol{x}_r + (\boldsymbol{N}_{rr}^r)^{-1}\boldsymbol{N}_{sr}^{r\mathrm{T}}\boldsymbol{x}_s$, completing the square in each station's block of Equation (328) splits the posterior quadratic form exactly:

$$(\boldsymbol{x}-\hat{\boldsymbol{x}})^{\mathrm{T}}\boldsymbol{N}(\boldsymbol{x}-\hat{\boldsymbol{x}}) = (\boldsymbol{x}_s-\hat{\boldsymbol{x}}_s)^{\mathrm{T}}\tilde{\boldsymbol{N}}_{ss}(\boldsymbol{x}_s-\hat{\boldsymbol{x}}_s) + \sum_r (\boldsymbol{u}_r-\hat{\boldsymbol{u}}_r)^{\mathrm{T}}\boldsymbol{N}_{rr}^r(\boldsymbol{u}_r-\hat{\boldsymbol{u}}_r). \tag{354}$$

Door 1 carries the border term. The vehicle carries the second: component c of $\boldsymbol{u}_r$ is $x_c + \boldsymbol{m}_c^{\mathrm{T}}\boldsymbol{x}_s$, with conditional covariance $(\boldsymbol{N}_{rr}^r)^{-1}$, and becomes the pseudo-observation

$$\delta x_c + \boldsymbol{m}_c^{\mathrm{T}}\boldsymbol{x}_s = \left(\hat{a}_c - x_c^{\mathrm{lin}}\right) + \boldsymbol{m}_c^{\mathrm{T}}\hat{\boldsymbol{s}} + \epsilon_c, \qquad \epsilon_c \sim N(0,\sigma_c^2), \tag{355}$$

with $\sigma_c^2 = v_c$ before the time update (Section 23.6). A row stating $\hat{a}_c$ at v_c with no border coefficients would state the border along $\boldsymbol{m}_c$ a second time, on top of door 1.

Linearisation invariance. Replacing x_c^{lin} by $x_c^{\mathrm{lin}} + \delta$ maps $\delta x_c \mapsto \delta x_c - \delta$ and the right-hand side of Equation (355) by $-\delta$. Storing the absolute $\hat{a}_c$ makes a shift of the linearisation point an exact solve no-op.

setting = reference value (loader default)	mechanism	where
`ambiguity_vehicle` = `float_kf` (`level`)	joint local block; bounded drop	Sections 23.3 and 23.4
`phase_leveling` = true (false)	the station-local carry the vehicle folds	Section 23.2
`carry_conditional_coupling`, `carry_coupling_time_update`, `carry_coupling_gauge_project`, `carry_coupling_reinit_sources` = true (false)	border regression rows; scalar κ fallback; gauge restore; re-realised columns routed as departed	Sections 23.2 and 23.6
`shared_carry_full_cov` = true (false)	dense border carry $\boldsymbol{P}$	Section 23.6
`station_prior_before_pd` = true (false)	receiver DCB datum rows inside the tested block	Section 23.2
`phase_datum_prioritized`, `phase_datum_held_basis` = true (false)	per-(satellite, signal) anchor, elected and held	Section 23.5
`phase_datum_handover_continuous` = true (false); `phase_datum_handover_max_var_m2` = 0.01 (1.0)	successor pinned at its carried level	Section 23.5
`phase_anchor_carry_reref`, `phase_datum_held_cold_release`, `phase_datum_held_mark_cold` = true (false)	cold handover erased and marginalised, declared	Section 23.5
`phase_datum_gate_drop_erase` = true (false); `phase_datum_min_stations` = 2 (1)	erase at the coverage gate's drop edge	Section 23.4
`handover_stransform` = true (false)	clock-pivot S-transform of the border carry	Section 23.5
`rank_gate_tol` = 10^{-11} (10^{-9}); `sbasis_complete` = true (false)	gate and heal cutoff; completion of realised near-null directions	Section 23.5
`carry_coupling_exact_update`, `carry_rx_code_bias_in_block`, `amb_float_drop_decouple_always` = true (false)	exact time update; receiver code bias in the block; drop always decouples	Sections 23.4 and 23.6
`clock_temporal_anchor` = true (false); `clock_anchor_q` = 2.9×10^{-8} (10^{-4}); `clock_anchor_var_floor_m2` = 10^{-5} (0); `clock_anchor_guard_space` = `phase` (`clock`)	binding satellite-clock carry; guard on the phase-visible clock	Section 23.6
Event-response and emitted-product settings, documented in Section 24:		
`cycle_slip.relevel_free_epoch`; `cycle_slip.mw_reflag_skip`; `cycle_slip.windup_restart_refloat`; `estimation.accumulate_phase_windup`	free-epoch latch; one slip frees an arc once; wind-up restart response	
`readmit_seed_horizon_s` = 3900 (0); `readmit_seed_mirrors`; `cycle_slip.gap_reentry_relevel`; `cycle_slip.gap_reentry_per_signal`; `cycle_slip.tracker_modeled_first`	re-admission seeds; per-row and per-signal gap re-entry	
`gross_los_screen`; `gross_los_cold_bound_m` = 12 (0)	acting code-row tier per line of sight	
`emit_pb_hold`; `emit_pb_hold_phase_clock`; `closed_loop.pb_hold_presence_flip`	generation hold; level step moved to the held bias; presence edges at the user	
`clock_disc_jump_test`; `emit_clock_frame_hold`; `emit_clock_disc`; `emit_ura`	clock step test; held clock frame; clock discontinuity counter; quality channel	

Table 9: Settings of the float ambiguity vehicle armed by both reference configurations. Values in parentheses are loader defaults.

Gauge invariance. Every data row of station r satisfies $\boldsymbol{A}_{s,r}\boldsymbol{g}_s + \boldsymbol{A}_{r,r}\boldsymbol{g}_r = \boldsymbol{0}$ along each structural gauge direction, hence $\boldsymbol{N}_{sr}^{r\mathrm{T}}\boldsymbol{g}_s + \boldsymbol{N}_{rr}^{r}\boldsymbol{g}_r = \boldsymbol{0}$ and

$$\boldsymbol{m}_c^{\mathrm{T}}\boldsymbol{g}_s = -\,g_r(c)\,: \tag{356}$$

the row of Equation (355) is blind to every gauge, provided the datum-pinned terms are kept. Without them every row observes the clock gauge and the null-space gate withholds.

Which door. The row's increment lands in station r's own $\boldsymbol{N}_{ss}^{r}$, $\boldsymbol{N}_{sr}^{r}$, $\boldsymbol{N}_{rr}^{r}$, so the arrowhead is preserved. It is an observation of that station: `appendCouplingRows` (`network/carry/Coupling`) appends it to $\boldsymbol{A}_{s,r}$, $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}_r$, $\boldsymbol{y}_r$, and it enters through `addStation`. Like door 2 it precedes the reduction; unlike door 2 it also adds to the station's border blocks. The phase-datum pins are $\boldsymbol{A}_r$ rows of weight 10^{10} with right-hand side $t - x_c^{\mathrm{lin}}$. No carried prior folds before `network.phase_leveling_warmup_epochs` emitted epochs (default 6). The rows are built in the parallel station produce of PASS B as the steps `kStationBuildSteps` and `kStationCarrySteps` of `network/assembly/StationBuild`.

Station prior before the PD test. `addStation` rejects a station whose $\boldsymbol{N}_{rr}$ factor fails the positive-definiteness pivot test. With `network.station_prior_before_pd` the receiver DCB datum rows ($\Lambda = 1/\sigma^2$, $\sigma =$ `network.rx_code_bias_datum_sigma_m`, $\eta = -\Lambda\, x_c^{\mathrm{lin}}$) are folded into a copy of $\boldsymbol{N}_{rr}$, $\boldsymbol{u}_r$ before the test (`StationPrior`) and withheld from the later `addStationPrior` call; without the setting, stations were rejected for that missing row. On the reference configurations every carried ambiguity row and in-block receiver code-bias row is also a row of $\boldsymbol{A}_{r,r}$, so the test judges the block the solve uses.

23.3 The joint local block

Under `float_kf`, `NetworkKnobs::pl_local_block` follows the vehicle and `CouplingRowBuilder::buildLocalBlock` folds each station's carried columns jointly (given `carry_conditional_coupling`). For the columns C that pass the inject predicates (present, not gauge-pinned, not pending a reseed, finite x_c^{lin}, a live entry), with the row ingredients $(\boldsymbol{m}_c, y_c, \sigma_c^2)$ of Section 23.6,

$$(\boldsymbol{\Sigma}_C)_{ii} = \sigma_{c_i}^2, \qquad (\boldsymbol{\Sigma}_C)_{ij} = \begin{cases} C_{c_i c_j} & \text{same harvest block,} \\ 0 & \text{otherwise (counted),} \end{cases} \qquad \boldsymbol{\Sigma}_C = \boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}, \tag{357}$$

and the station receives the unit-weight rows $\boldsymbol{L}^{-1}[\,\boldsymbol{I}_C \mid \boldsymbol{M}_C \mid \boldsymbol{y}_C\,]$ (`whitenLocalBlock`). Each block of $\boldsymbol{\Sigma}_C$ is a principal submatrix of one posterior, so $\boldsymbol{\Sigma}_C$ is positive-definite by construction; after one ridge retry a failed factor reverts the station to per-column rows, counted. A column refloated within the epoch has its row and column deleted before the factor: an exact marginalisation. The receiver code bias joins the same block (Section 23.6, paragraph on the receiver code bias in the block).

$\boldsymbol{\Sigma}_C$ is the joint conditional covariance of $\boldsymbol{u}_C$ with every non-carried local integrated out. Independent rows fold $\mathrm{diag}(\boldsymbol{\Sigma}_C)^{-1}$. For n columns of variance σ^2 and pairwise correlation ρ, along $\boldsymbol{v} = \boldsymbol{1}/\sqrt{n}$,

$$\boldsymbol{v}^{\mathrm{T}}\boldsymbol{\Sigma}_C^{-1}\boldsymbol{v} = \frac{1}{\sigma^2\,[1 + (n-1)\rho]}, \qquad \boldsymbol{v}^{\mathrm{T}}\mathrm{diag}(\boldsymbol{\Sigma}_C)^{-1}\boldsymbol{v} = \frac{1}{\sigma^2}, \tag{358}$$

an over-count of $1 + (n-1)\rho$ along the directions the non-carried locals share: the slant ionosphere of a line of sight's two bands, and the receiver clock, zenith wet delay and receiver DCB of every column. The block is exact in every local-local direction given the border.

23.4 Birth prior and the bounded drop

Birth. A new arc folds no carried prior in its first epoch; a prior from the same epoch's data would count it twice. The first harvest records $\sigma^2_{\text{first}} = v_c$ and sets, once,

$$\sigma^2_{\text{birth}} = \max\{\sigma^2_{\text{first}},\ f(e)\}, \qquad f(e) = \frac{\sigma^2_{\text{zen}}}{\left[\sin \max(e,\ 5^\circ)\right]^{2p}} \tag{359}$$

(`ambFloatBirthFloor`), with σ^2_{zen} = `network.amb_float_birth_var_m2` and p = `filter.code_elevation_power`, the code weights' mapping, because the floor states what a code-determined level is worth. An unknown elevation reads as zenith; $\sigma^2_{\text{zen}} \le 0$ gives $f \equiv 0$. On the reference configurations $\sigma^2_{\text{birth}} = \sigma^2_{\text{first}}$.

The drop. A slip lowers confidence in the level and keeps the arc's identity. `ambFloatDrop` (through `ambFloatDropFloored`) sets

$$\sigma^2_{\text{tgt}} = \max\{\sigma^2_{\text{birth}},\ \sigma^2_{\text{first}},\ f_b(e),\ f_d(e)\}, \qquad \sigma^2_c < \sigma^2_{\text{tgt}} \ \Rightarrow\ \sigma^2_c \leftarrow \sigma^2_{\text{tgt}},\ \ \boldsymbol{m}_c \leftarrow \varnothing,\ \ C_{c\cdot} \leftarrow \varnothing, \tag{360}$$

with f_b, f_d the floor at the birth floor and at `network.amb_float_drop_var_m2`. The age clock restarts; $\hat{a}_c$, `var_first`, `var_birth`, the birth stamps, `last_seen_s` and `n_folds` are kept. A drop only loosens: for $\sigma^2_c \ge \sigma^2_{\text{tgt}}$ it is idempotent, and `network.amb_float_drop_decouple_always` makes that branch clear the regression and block row too. Without it an arc slipping one epoch after birth kept its pre-slip regression and cross terms, and under the exact update its stale value went into the satellite clocks. On the reference configurations $\sigma^2_{\text{tgt}} = \sigma^2_{\text{first}}$; the prior still folds in the same epoch.

Causes. `networkAmbFloatApplyDrops` (`networkAmbFloatDropEpoch` in the epoch spine) runs between the PASS-A slip scans and PASS B, so the inject folds at birth strength where the leveling vehicle skips: `relevel_free` (the latched free set of `network.cycle_slip.relevel_free_epoch`: slip detectors, gap re-entry and wind-up restart) and `stale_gap` (gap above `network.phase_leveling_max_gap_s`, reference value 900 s, default 120 s). `networkPostFitRaim` drops on `t4_attrib` and `t3_consensus`, where the leveling vehicle erases. Under the held basis each latched arc is stamped in the station row's `amb_flag_ep`. The causes are the constants of `AmbDropCause`. An optional maturity gate on the isolated-arc refloats of post-fit RAIM (`network.raim_refloat_maturity_gate`, off) holds a refloat on an arc younger than its horizon; the gate and its telemetry belong to the quality-control chapter (Section 13.9), and the vehicle sees it only as the absence of that drop.

The erases that remain. The post-harvest prune erases an arc unseen past `network.phase_leveling_max_gap_s`. On the reference configurations three more responses erase carried ambiguities: the gross line-of-sight screen (`network.gross_los_bound` off; Section 24.6), the anchor re-reference (Section 23.5), and the coverage gate's drop edge, where the survivor's carried ambiguity would absorb the removed bias.

Both edges. A mechanism that changes which information reaches which states is an event at both edges. The decoupling edge is bounded: a free epoch opens a direction the border re-balances into, mainly the satellite clock, since the anchor pin holds the phase bias; the drop removes information down to a stated level. The re-coupling edge re-validates: the arc re-enters as an uncoupled row at σ^2_{tgt}, its cross terms read as pair misses until the next harvest, and it re-earns its variance from the data. Drops are counted by cause and traced in the trace file `network_amb_state.csv`.

23.5 The held S-basis

The family gauge. Each (satellite i, signal s) family carries the gauge

$$G(c): \qquad a^i_{r,s} \mapsto a^i_{r,s} - c \;\; \forall r, \qquad b^i_s \mapsto b^i_s + c, \tag{361}$$

with b^i_s the satellite phase bias. The prioritized datum (`networkElectPhaseAnchors` in `network/solve/PhaseDatum`, called by `networkPhaseAnchorEpoch`) pins $a^i_{r^\star,s} = t$ at one anchor $r^\star$. Two realisations differ by an exact S-transform [36],

$$\boldsymbol{x}(r', t') = G(c)\, \boldsymbol{x}(r^\star, t), \qquad c = \hat{a}^i_{r',s}(r^\star, t) - t', \tag{362}$$

and every carried reference of the family (each $\hat{a}$, the carried phase bias, each $\hat{s}_j$ on its column) lives in the realisation it was harvested in.

Held basis. Under `network.phase_datum_held_basis` a retained incumbent re-pins at the value frozen at its election (`PhaseBasisEntry::value`): $c = 0$ within a tenure. A datum re-derived after a drop or a flag would apply $G(c)$, $c \neq 0$, a phase-bias step the user re-converges through. The basis ambiguity is a held constant outside the carry; every other ambiguity is a live state. A screen flag on the incumbent's own column within `network.phase_anchor_flag_quiet_ep` epochs (default 5) deposes it at the election. A new election ranks by elevation the candidates aged at least `network.phase_anchor_min_age_ep` epochs (default 10), unflagged and with a usable ledger entry; failing that, by elevation alone (a cold pin), then coverage. A slip response that frees a family's anchor arc bans that incumbent for the epoch (`network.cycle_slip.anchor_slip_handover`), so the election hands the family to a successor pinned at its own carried level.

Handover. `AnchorHandoverLedger` holds each station's previous-epoch $\hat{a}$ (written ℓ) and conditional variance; an entry is usable when $0 < \text{var} \leq$ `network.phase_datum_handover_max_var_m2`. A continuous handover pins at $t' = \ell_{r'}$: c is the successor's one-epoch revision, bounded by the ceiling, and the family does not step. A cold handover pins at $t' = 0$: $c = \ell_{r'} =: \Delta$. The reference configurations answer a cold handover, a first sighting or a bridge exit by erasing the family's carried ambiguities and marginalising its phase-bias key out of the border carry for one epoch (`network.phase_anchor_carry_reref`, `network.phase_datum_held_cold_release`); with `network.phase_datum_held_mark_cold` the emitted product declares a handover if and only if it is cold. The exact alternative (`declaredHandoverShiftCarry`, behind `network.amb_float_declared_handover`, off) applies $G(\Delta)$ to the carry,

$$\hat{a}^i_{r,s} \leftarrow \hat{a}^i_{r,s} - \Delta, \qquad \hat{b}^i_s \leftarrow \hat{b}^i_s + \Delta, \qquad \hat{s}_j \leftarrow \hat{s}_j + \Delta \;\; (j = b^i_s), \tag{363}$$

variances untouched; moving one reference alone injects the frame step as row bias. The setting `network.carry_rebase_complete` (off) extends the shift to the sites the mechanism table lists as owed.

Clock pivot. At a clock-pivot swap $p \to q$, `network.handover_stransform` (`network/solve/HandoverStransform`) re-expresses the border carry along the structural clock null direction $\boldsymbol{v}_c$,

$$c_0 = x_q / v_c[q], \qquad x_i \leftarrow x_i - v_c[i]\, c_0, \qquad \text{var}_i \leftarrow \text{var}_i + v_c[i]^2\, \text{var}_q, \tag{364}$$

cross term dropped, and releases that constellation's one-epoch carry skip (it needs `network.scoped_handover_response`, armed).

Rank gate and heal. `NetworkSbasis::assertNullSpaceRank` requires

$$n_{\text{null}} = \#\big\{k : \lambda_k(\tilde{\boldsymbol{N}}_{ss}) < \tau \, \max(\lambda_{\max}, 1)\big\}, \qquad \tau = \texttt{network.rank_gate_tol}, \tag{365}$$

to equal the declared S-basis dimension (solve stage 8, `networkRankGate`). On failure, `healNearNullPhaseBias` soft-anchors toward 0 at $\sigma = 1$ m every phase-bias column with absolute component above 0.3 in a direction below the same cutoff, through `addSharedPrior` (door 1, no second reduction), and re-gates once; a second failure withholds. A held basis whose anchor station is rejected leaves its column near-null, and the heal makes that a bounded one-epoch degradation. Gate, heal and estimability dump read one τ. With the exact update the level keeps a code-scale variance in the first minutes, and at $\tau = 10^{-9}$ weak but real phase-bias common modes were counted as null; the reference configurations set 10^{-11} and also arm `network.sbasis_complete`, which constrains a realised near-null direction beyond the declared dimension along its own eigenvector (solve stage 7) instead of withholding.

23.6 The coupling time update

A row harvested at epoch $k-1$ states $l = r + \boldsymbol{m}^{\mathrm{T}}\boldsymbol{s} + e$, $e \sim N(0, v)$, given that epoch's border $\boldsymbol{s}$. The border moves, $\boldsymbol{s}^{+} = \boldsymbol{s} + \boldsymbol{w}$, $\boldsymbol{w} \sim N(\boldsymbol{0}, \boldsymbol{Q})$, and the border fold re-injects it at $N(\hat{\boldsymbol{s}}, \boldsymbol{P} + \boldsymbol{Q})$, with $\boldsymbol{P}$ the stashed dense posterior and $\boldsymbol{Q}$ the fold's own inflation. The row must be re-expressed on $\boldsymbol{s}^{+}$ with the same $\boldsymbol{Q}$.

The scalar update. `scalarRowIngredients` shrinks each present term with $q_j \Delta t > 0$, $p_j > 0$:

$$\kappa_j = \frac{p_j}{p_j + q_j \Delta t}, \qquad m_j \leftarrow \kappa_j m_j, \qquad \sigma^2 \mathrel{+}= m_j^2 \, \kappa_j \, q_j \Delta t; \tag{366}$$

a departed or re-realised term adds $m_j^2 p_j$ and is deleted, and `tuGaugeRestore` spreads each (constellation, kind) deficit over the family's present terms so Equation (356) holds. q_j is the registry's border q (`couplingQ`); for a clock, `filter.process_noise.sat_clock_q`, reference value $1 \times 10^{-3}\,\mathrm{m^2\,s^{-1}}$. The fold anchors the same clock at `network.clock_anchor_q` with floor `network.clock_anchor_var_floor_m2` (reference value $1 \times 10^{-5}\,\mathrm{m^2}$). At the floor and $\Delta t = 30$ s, $\kappa_{\text{clk}} \approx 3 \times 10^{-4}$ while $\kappa_{\text{pb}} \approx 1$: the row loses its own clock coefficient.

The level lock. In the single-station, two-band model (phase $\phi_j = \rho + T + dt_r - c + p_j + N_j - \alpha_j I$, code $P_j = \rho + T + dt_r - c + b_j + \alpha_j I$, $\alpha_1 = 1$), the code rows give $\partial I / \partial c = \kappa' = \sum_j w_j \alpha_j / \sum_j w_j \alpha_j^2$, and

$$\text{exact row: } N_j - (1 + \alpha_j \kappa')\, c + p_j = r_j, \qquad \text{shrunk row: } N_j + p_j = r_j. \tag{367}$$

With the phase row, the exact row constrains $\alpha_j(I - \kappa' c)$ and leaves c to the code. The shrunk row gives $c + \alpha_j I = r_j - \phi_j + \rho + T + dt_r$ for $j = 1, 2$, which fixes the satellite clock level at phase precision at every station and epoch. The level is estimable from code only; stated at phase precision, it freezes (`docs/refound/CTU_coupling_time_update_2026-09-28.md`).

The exact update. `network.carry_coupling_exact_update` (`network/carry/CouplingExact`) splits a row's stashed terms into S, the columns the fold carries with a finite inflation $\boldsymbol{Q}_S$, and D, the rest (departed, re-initialised, re-referenced, skipped). Over $K = S \cup D$,

$$\begin{aligned} \boldsymbol{m}_{\text{new}} &= (\boldsymbol{P}_{SS} + \boldsymbol{Q}_S)^{-1} \boldsymbol{P}_{S,K}\, \boldsymbol{m}_K, \qquad \operatorname{Var} e' = v + \boldsymbol{m}_K^{\mathrm{T}} \boldsymbol{B}\, \boldsymbol{m}_K, \\ \boldsymbol{B} &= \boldsymbol{P}_{KK} - \boldsymbol{P}_{K,S} (\boldsymbol{P}_{SS} + \boldsymbol{Q}_S)^{-1} \boldsymbol{P}_{S,K}. \end{aligned} \tag{368}$$

With $\boldsymbol{F}_S = (\boldsymbol{P}_{SS} + \boldsymbol{Q}_S)^{-1}$, $\boldsymbol{g} = \boldsymbol{Q}_S \boldsymbol{m}_S$, $\boldsymbol{u} = \boldsymbol{P}_{S,D} \boldsymbol{m}_D$, $\boldsymbol{z} = \boldsymbol{F}_S \boldsymbol{u}$ and $\boldsymbol{F}_S \boldsymbol{P}_{SS} = \boldsymbol{I} - \boldsymbol{F}_S \boldsymbol{Q}_S$, `couplingExactRow` evaluates both halves free of cancellation:

$$\boldsymbol{m}_{\text{new}} = \boldsymbol{m}_S - \boldsymbol{F}_S \boldsymbol{g} + \boldsymbol{z}, \qquad \boldsymbol{m}_K^{\mathrm{T}} \boldsymbol{B}\, \boldsymbol{m}_K = \boldsymbol{m}_{\text{new}}^{\mathrm{T}} \boldsymbol{g} + \boldsymbol{g}^{\mathrm{T}} \boldsymbol{z} + \big(\boldsymbol{m}_D^{\mathrm{T}} \boldsymbol{P}_{DD} \boldsymbol{m}_D - \boldsymbol{u}^{\mathrm{T}} \boldsymbol{z}\big), \tag{369}$$

the variance clamped at 0 (counted). With D empty, $\boldsymbol{Q}_S = \boldsymbol{0}$ returns the harvested row and a diagonal $\boldsymbol{P}$ returns Equation (366). The difference is in D: a decoupled or departed clock correlated with its phase biases along the level gives $m_{\text{new},p} \approx m_p + (P_{pc}/P_{pp})\, m_c$, so the row keeps its co-movement with the level.

The assembled row. `exactRowIngredients` emits $\boldsymbol{m}_{\text{new}}$ on S with right-hand-side terms $m_{\text{new},j} \hat{s}_j$, $\hat{s}_j$ the mean the fold centres its prior on. A datum-pinned term is kept as harvested; a present term with no dense row takes Equation (366) at the fold's inflation; a departed one adds $m_j^2 p_j$; SH-VTEC terms keep the scalar update. Per constellation the clock coefficients the update removes (harvested minus emitted) go to the stash's pinned pivot clock, so the row keeps its identity along the declared clock gauge. The variance

$$\sigma_c^2 = v_c + \textstyle\sum_{\text{dep}} m_j^2 p_j + \sum_{\text{scalar}} m_j^2 \kappa_j q_j \Delta t + \boldsymbol{m}_K^{\mathrm{T}} \boldsymbol{B}\, \boldsymbol{m}_K \tag{370}$$

is the diagonal of $\boldsymbol{\Sigma}_C$ in Equation (357). Only rows harvested at the stash of $\boldsymbol{P}$ take this route; others, and epochs whose $\boldsymbol{P}_{SS} + \boldsymbol{Q}_S$ does not factor, take the scalar update, counted. $\boldsymbol{Q}_S$ is the fold's own decision, made once before PASS B (`networkCarryDecide` in `network/state/CarryDisposition`, `networkCouplingExactPrepare` in `network/state/BorderPrior`: $q =$ `var_eff` $- P_{pp}$), so rows and fold read one $\boldsymbol{Q}$. The covariance $\boldsymbol{m}_i^{\mathrm{T}} \boldsymbol{B}\, \boldsymbol{m}_k$ between rows is dropped; the within-station half (`network.carry_coupling_exact_cross`) is built and off.

Information loss. Under the scalar update with the loose clock rate, a young arc's ambiguity information is discarded at every inject; under the exact update the steady loss is the anchored clock's walk. The per-arc accounting of the variance budget is an open item (`docs/refound/AIL_ambiguity_information_loss_2026-10-01.md`).

Receiver code bias in the block. The diagonal receiver code-bias carry folded each column at its conditional variance with no border coefficients, the double count that Equation (355) avoids; the border information along each satellite's level roughly doubled every epoch. `network.carry_rx_code_bias_in_block` harvests these columns with their regression and cross covariances into the block of Equation (357), with no diagonal fallback.

Temporal clock anchor. The guarded anchor, `network.clock_temporal_anchor` (`network/solve/ClockAnchor`), folds each satellite clock of the border carry at

$$v_{\text{eff}} = \begin{cases} \max\{v + q_a\, \Delta t_i,\ v_{\text{floor}}\} & \text{anchored}, \\ v + q_c\, \Delta t & \text{loose}, \end{cases} \qquad \Delta t_i = t - t_{\text{last}}(i), \tag{371}$$

$q_a =$ `network.clock_anchor_q`, $v_{\text{floor}} =$ `network.clock_anchor_var_floor_m2`, $q_c =$ `sat_clock_q`; an anchored carry older than `network.clock_anchor_max_gap_s` (default 120 s) decouples. Without the anchor, at the reference q_c the carried prior spread $\sqrt{q_c \Delta t}$ exceeds the data's own per-epoch spread, the prior does not bind, and the emitted clock is re-drawn from each epoch's data as a random walk. Once the anchor binds, the emitted walk is $\sqrt{q_a \Delta t}$, because for the scalar carry

$\mathrm{Var}(\hat{x}-\mu) = P^- - P^+$ and $\mathrm{Var}(\hat{x}_k - \hat{x}_{k-1}) \to q_a \Delta t$. The guard standardises the common-removed residual by $\sqrt{\max(P^- - P^+, \sigma_f^2)}$, $\sigma_f =$ `network.clock_anchor_sigma_floor_m`. A trip at `network.clock_anchor_k` (default 6), or above `network.clock_anchor_abs_m` (default 2 m), sets the satellite loose, withdraws its prior, re-solves before emission and marks the emitted product; the re-solve needs a carry redundancy of at least 0.05. The guard runs as solve stage 12, between the primary solve and the within-epoch RAIM tier, so that a prior fault is refloated before the tier can attribute it to data. Re-coupling needs `network.clock_anchor_recouple_epochs` (default 4) in-band epochs. Both edges are events: the decoupling reasons are counted separately (statistic, backstop, stale carry, datum change, refloat by another tier), and the re-coupling edge is a counted event that the emitted product does not mark, because re-engaging a prior centred on the last posterior steps no emitted value.

Guard space. With the exact update the level carries a code-scale variance while the phase-visible combinations are phase-precise, and a clock-space statistic trips on legitimate level convergence. Under `network.clock_anchor_guard_space` = `phase` the trip reads

$$d = c - \textstyle\sum_j w_j\, p_j, \qquad \boldsymbol{w} = \arg\min \|\boldsymbol{w}\| \text{ s.t. } \sum_j w_j = 1,\ \sum_j w_j \lambda_j^2 = 0, \tag{372}$$

one phase bias per band, constellation median removed, $\sqrt{S}$ floored at `network.clock_anchor_phase_sigma_floor_m` (default 0.02 m). The level (a common shift of c and every p_j) and the ionosphere direction (p_j by $\alpha_j \delta$) leave d unchanged; GPS L1/L2 gives $\boldsymbol{w} = (2.5457, -1.5457)$. A satellite without a two-band phase statistic in an epoch is tested on the backstop alone. The backstop stays on the clock.

Neighbouring clock mechanisms. Four mechanisms act on the same clock columns and are described where their mathematics lives. The cold receiver-clock pre-seed (`network.cold_clock_seed`, armed) removes the free-running receiver-clock offset from cold ambiguity seeds (Section 14.2). The clock runaway check (`network.clock_runaway_check`, armed) drops from the emission a constellation whose median clock differs from the global median by more than `network.clock_runaway_mag_m` (1000 m); its drift ceiling needs the two-state clock model, which is off (Section 13.11). The clock step test, the held clock frame and the clock discontinuity counter act on the emitted clocks (Section 24.7). The cold-start a-priori clock anchor (solve stage 10, `networkFoldColdClockAnchor`) pulls each clock toward the a-priori product clock with $\sigma =$ `network.apriori_clock_anchor_sigma_m`, or `filter.initial_std.sat_clock` when unset; it binds only below 100 m, and the reference value of 1000 m leaves it off.

Fresh convergence only. A snapshot written without the exact update, the in-block receiver code bias, the drop decouple or the phase-space guard is not resumed into a run that arms them unless `network.ctu_switch_on_resume` (default off). That seam failed its gate: the restored ambiguities, and the slant ionosphere they tie, had absorbed the frozen level error and hold it; the ambiguity-side response is not built (`docs/refound/CTU2_level_recursion_followup_2026-09-29.md`).

23.7 Status

The float vehicle with the held basis, the station prior before the PD test, the exact update with its companions and the level-aware hold are armed on both reference configurations. Their record is in `docs/refound/RF-1_decision_2026-09-14.md`, `RF-2_held_basis_2026-09-15.md`, `GRB_golden_rebase_2026-09-22.md`, `CTU_coupling_time_update_2026-09-28.md` and `EXL_exact_update_lah_`

promotion_2026-10-01.md. The registry and the per-epoch fold decision are consolidations that change no output (PRM_parameter_registry_2026-09-29.md).

The baseline after the latest arming (docs/refound/BOF_built_off_follow_through_2026-10-28.md), over a 6 h run on the reference configurations: the test station WTZZ00DEU on real data, which the network does not use, has a median 3-D position error per hour of 0.059, 0.012, 0.023, 0.012, 0.017 and 0.012 m; on the simulated network the test station SZZ000SIM, graded from the run start, has 0.137, 0.072, 0.031, 0.024, 0.026 and 0.018 m. The first hour is the convergence hour and the settled claims come from hours 2 to 6.

Open: the birth floor, set by no reference configuration; the switch-on seam's ambiguity-side response and the guard's answer to a phase-bias event; the information budget; the residual non-finite-input station rejections; and the station level under three frequency bands, for which the anchor's level directions held in the phase datum are derived offline and not built (docs/refound/R1D_station_level_datum_2026-10-23.md, ALD_anchor_level_datum_2026-10-23.md). The register of open items is docs/status/OPEN_ITEMS.md.

24 Event responses and the emitted product

This section covers two duties of the network estimator of Section 20: the response to events on carried state (a slip, a tracking gap, a satellite's absence and return, a restart of the wind-up model, a gross code fault on one line of sight), and the emitted product (which clock and phase-bias values reach the user, how a change is declared, how the user answers). The carried local column, the bounded drop operator, the held S-basis and the coupling time update belong to Section 23. Settings are quoted with their loader defaults; the reference configurations are config/golden/golden_field_refound.json (real data) and golden_sim_refound.json (simulated network), which agree on every setting named here.

24.1 The event doctrine

A carried quantity enters the next epoch as a prior. In the information form of Section 22, a carried value $\hat{x}$ with variance P folds as the pseudo-observation

$$x = \hat{x} + e, \qquad e \sim \mathcal{N}\big(0,\ P + q\,\Delta t\big), \tag{373}$$

with q the process-noise rate of its kind. If the quantity moved by δ after $\hat{x}$ was formed, the pseudo-observation is biased by δ at weight $(P + q\,\Delta t)^{-1}$. For a time-linked quantity $q \approx 0$ and P is small, so the states sharing its rows (ionosphere, clocks, troposphere) absorb the bias; a wired response must change $(\hat{x}, P)$ before the fold. The project rule is that a mechanism changing which information flows into which states is an event at both edges. The decoupling edge bounds what the decoupled states can accumulate; the re-coupling edge re-validates or re-seeds what it reconnects. Each edge writes an event row and a counter, so an inert response reads as zero.

Vocabulary. The parameter registry (network/assembly/ParamRegistry, network/assembly/Responses) names five events (ParamEvent: slip, gap, handover, decouple, readmit) and seventeen policies (ResponsePolicy). Those used here: *Drop* returns the variance to birth strength, clears the border regression and the block row, and keeps the value (Section 23); *EraseAndRebirth* erases the entry for the next harvest to re-create; *Park*, *SeedAtVarPlusQGap* and *Discard* keep an entry past its column's departure, seed the returned column, or drop the entry unfolded; *Hold* freezes and re-applies an entry. Each response site is one of 68 rows of ResponseSite, a (kind, event, policy,

code site) tuple with an arming predicate (`responseSiteArmed`) and a counter; a site that responds where it is not declared armed is a run-end declaration error. Marks that only re-seed a linearisation point are excluded, among them the pending-reseed mark, which PASS B's measurement pre-pass consumes (a solve no-op on the float vehicle). The acting ambiguity response is the free-epoch latch (`network.cycle_slip.relevel_free_epoch`), which ends in a Drop. A flagged satellite clock whose reinitialisation sigma is not positive is left out of the stash entirely (a hard refloat); the sites `ClkHardRefloat`, `CbHardRefloat` and `PbHardRefloat` count each entry left out, and are inert on the reference configurations, where the clock monitor does not run.

Armed set. The reference configurations arm the MW/LLI and IF triple-difference slip drains (`network.cycle_slip.mw_detect`, `if_phase_td`; Section 15.3) with three refinements: `mw_evidence_refloat` admits an MW flag on its own strength (a lane without bias correction, an arc of at least `mw_evidence_min_arc_epochs` epochs, ratio to the threshold above `mw_evidence_ratio_min`); `common_band_scope` frees only the bands the evidence implicates; `anchor_slip_handover` bans a freed phase anchor's family for that epoch, so the election picks a successor pinned at its own carried value. They also arm the responses of Section 24.2 to Section 24.10. Each arming setting is off in the loader.

24.2 One slip frees an arc once: the MW re-flag skip

An MW flag can re-detect a step the network has already answered. A locked arc absorbs a step and flags it samples later (drift demotion, then the unlocked flag), and the first sample after an LLI epoch flags the step that the LLI response freed at that epoch. The IF time-difference response and the gap re-entry response leave the same situation. Freeing the band a second time discards only the re-convergence since its re-level, and opens the decoupling edge twice for one event.

The MW detector reports the epoch t_s at which each flagged step entered the data ($t_s = 0$ when unknown): the flag's own epoch for a first detection, an earlier epoch otherwise. A band whose last re-level (`LocalCarry::relevel_s`, r_b) is not earlier than the step holds no pre-step level. With t the current epoch and $\tau = 0.5$ s (`kMwReflagEpochTolS`, half the shortest sampling interval),

$$\text{covered}(b) \;=\; \big(0 < t_s < t - \tau\big) \;\wedge\; \big(r_b > 0\big) \;\wedge\; \big(r_b \geq t_s - \tau\big). \tag{374}$$

Under `network.cycle_slip.mw_reflag_skip` the MW drain does not free a covered band; the candidate's other bands are freed as before, and the integer still reaches a waiting bridge pend, because the rule is tested after the delta-bridge intercept. A step of the current epoch is a first detection and is never covered. The slip-QC counters tally every such candidate and band whether or not the setting is armed (`mw_reflag_*`), and an armed skip writes an `mw_wl_reflag` event row with action `skip` naming the bands skipped and freed. The setting needs `network.cycle_slip.mw_detect` (the loader disarms it with a warning otherwise), is a forward-only response policy exempt from the snapshot fingerprint, and is armed on both reference configurations (`docs/refound/MRF_mw_reflag_skip_2026-10-28.md`).

24.3 Gap re-entry, keyed per signal

An arc that returns within T_{pl} (`network.phase_leveling_max_gap_s`, loader default 120 s, reference value 900 s) has its carried entry re-injected at the pre-gap value. No time-difference detector covers the return epoch: MW needs an adjacent predecessor on the same pair, the IF triple difference three consecutive epochs. The gap re-entry drains (`networkSlipDrainGapReentry` in `network/slip/SlipQc`) run serially in the PASS-A slip band, after the wind-up restart drain and before the inject, so an arc is freed in the epoch it returns.

Row-level verdict. `networkComputeRawObservables` builds one raw row per (station, satellite) on the two lowest distinct bands s_1, s_2 with phase and matching code. A row absent for at least one processed epoch and present now is a `GapResumption`, with verdict

$$\text{slipped} = \underbrace{\big(|\Delta\text{MW}| > g_{\text{MW}} \vee |\Delta\text{GF}| > g_{\text{GF}}\big)}_{\text{shifted}} \vee \big(\kappa_{\text{lli}} \wedge (\text{LLI}_{s_1} \vee \text{LLI}_{s_2})\big) \vee \big(\kappa_{\text{unm}} \wedge \neg\,\text{measurable}\big), \quad (375)$$

with $\Delta\text{MW}, \Delta\text{GF}$ the raw level changes across the break, g the station's empirical gates, κ_{lli} = `gap_reentry_relevel_lli` (default true) and κ_{unm} = `gap_reentry_relevel_unmeasurable` (default false: missing evidence is not evidence of a slip). A slipped row frees both bands.

The target is the carry. A returning arc has no live ambiguity column at drain time: it was pruned with the gap, and PASS B's pre-pass re-creates it after the drains. The response synthesises the inject's key (`StateKeys::makeAmbiguityKey`) and asks the carry whether anything needs protection: a leveling-carry entry (r, i, s) with

$$0 \le t - t_{\text{seen}} \le T_{\text{pl}}, \quad (376)$$

exactly what the inject would fold. A protectable arc enters the free-epoch latch (`relevel.free_ambs`, cause `kCauseGap`); a live column also takes the pending-reseed mark. `network.cycle_slip.relevel_free_epoch` is the latch: it carries the freed key past the pre-pass to the inject, where the arc folds at birth strength after the bounded drop of Section 23. An arc without a protectable entry is counted `no_carry`. The instrument `networkGapResumeCensus` (`network/slip/GapResumeCensus`) is the re-coupling count: it counts, for the arcs that returned after a break, those that re-entered on state that bridged the break, split by whether the raw level shifted, and those that re-entered as a fresh float. A funnel arc is never counted as bridged.

Per-signal key. The raw row is rebuilt every epoch from whichever phases have a matching code. A receiver that loses lock on its primary pair while another signal tracks through (band 5, or a fallback signal) keeps its row, so the row-level tier sees no gap and the returning phases would fold across the break. `network.cycle_slip.gap_reentry_per_signal` keys the re-entry on (station, satellite, signal). For a signal s last seen at epoch k_s, the level test differences raw phases (metres) against a partner p present at k and k_s:

$$d_s = \big[\Phi_s - \Phi_p\big](k) - \big[\Phi_s - \Phi_p\big](k_s) \approx \lambda_s\,\delta N_s - \lambda_p\,\delta N_p - (\mu_s - \mu_p)\,\delta I, \quad (377)$$

up to noise, multipath and the wind-up change. A partner that tracked through the gap is preferred: then $\delta N_p = 0$ and d_s isolates the candidate's step plus the ionospheric change over the gap. With g_{GF} the station's GF gate, which does not grow with the gap,

$$\text{slipped}_s = \big(|d_s| > g_{\text{GF}}\big) \vee \big(\kappa_{\text{lli}} \wedge \text{LLI}_s\big) \vee \big(\kappa'_{\text{unm}} \wedge \neg\,\text{measurable}_s\big), \quad (378)$$

with LLI_s the returning signal's own flag and κ'_{unm} = `gap_reentry_per_signal_unmeasurable` (default true: the carry of a satellite that stayed present is certain to fold). Only s is freed; the other bands tracked through. Both tiers share one responder (`gapReentryRespondArc`); the per-signal tier skips a satellite the row-level tier reported.

Co-resumed share. The level test of Equation (377) cannot see a step common to the candidate and its partner, which a channel-delay change at re-acquisition is. The signals of one (station, satellite) that return at the same epoch are one re-acquisition of that line of sight. With `network.cycle_slip.gap_reentry_coresumed_share` (default off, not armed on the reference configurations) the first co-resumed signal with a verdict of its own, shifted, LLI or unmeasurable, is the trigger, and every other co-resumed signal of the line of sight is freed with the trigger's cause (event cause `shared`, scope `los_shared`). A signal that tracked through the gap has no resumption and is never freed by the share, and an arc already in the free set of the epoch is not counted twice. The setting needs the per-signal tier and the row-level response; armed without them it is inert and the loader warns.

Tracker naming. The row-level tier named each band lexically: where the canonical band-1 phase is L1W the tracker named L1C, the key missed the carry, and the stale ambiguity folded. `network.cycle_slip.tracker_modeled_first` names each band by the station's canonical phase latch (`TrackerLatchLookup`) when that signal has raw phase and code this epoch, so the tracker key equals the carried key; `gap_tracker_key_mismatch` is then zero by construction.

24.4 Re-admission after a long absence

A satellite in eclipse, or below every mask, leaves the shared layout, and its clock and code-bias columns depart; on return they re-realise from code. `network.readmit_seed_horizon_s` $= T_{\mathrm{rd}}$ (loader default 0, off; reference value 3900 s) bridges the absence.

At departure the stash (`network/state/BorderStash`) parks each D2 clock entry whose column left the layout: it keeps the entry with its departure time t_{park}, unfolded. Code-bias entries park too under `network.readmit_seed_code_bias` (default true). Phase-bias entries do not: their frame is re-realised at the cold re-pin on return, and a stiff seed would conflict with that pin. When the column returns with $0 < t - t_{\mathrm{park}} \le T_{\mathrm{rd}}$, the fold decision (`carryFoldDecide` in `network/state/CarryDisposition`) is *Seeded*,

$$P_{\mathrm{seed}} = P + q\,(t - t_{\mathrm{park}}), \tag{379}$$

with q the registry rate of its kind. A clock under the temporal anchor (`network.clock_temporal_anchor`, armed) folds through the anchor with the staleness bound raised to $\max(\texttt{clock_anchor_max_gap_s}, T_{\mathrm{rd}})$ (`clockAnchorFoldVariance`), so it folds anchored over its true gap instead of re-drawing from code. The anchor's guard is the re-coupling check: a trip within its verify window (`kReadmitVerifyWindowS`, 300 s) after a seed is recorded as the data rejecting the pre-gap realisation (`readmit_seed_moved`) and decouples the clock. A stash-side check compares each seed with the solved value and counts a standardised residual above `kReadmitSeedK` (6).

A parked entry is dropped past the horizon (`ClkParkExpire`, `CbParkExpire`), when its column returns without consuming it (`ClkParkDiscard`), and at a clock-datum change of its constellation that the handover S-transform did not re-express (`ClkParkDiscardDatum`). The returning satellite is then a different arc and registers cold. The clock-only variant (`readmit_seed_code_bias` false) was falsified on the triple-frequency simulated network: the clock tails returned. The code-bias seeds fix the split between clock and code bias at the return; without them the cold code-bias realisation relocates into the clock (`docs/cards/network-vehicle/READMIT-EDGE.md`).

Seed mirrors. The epoch-tail snapshots that pair each folded border prior with its re-stashed solved value skip every parked entry, although the fold applies a seed at the variance P_{seed} of Equation (379). With `network.readmit_seed_mirrors` (armed on both reference configurations) a

seed that the fold applied is a row of the carry variance-component monitor (code-bias family) and of the prior-residual diagnostic (`network/qc/PriorResidual`; code-bias and clock families), with the seed gap as its interval and the variance the fold used. The mirrors read the seed; they change no fold input unless the monitor's variance factor is applied (`network.carry_vce_apply`, off). The setting is inert without the horizon and without either reader, and the loader warns in both cases.

24.5 Carrier-phase wind-up: accumulation and its restart

The wind-up angle $\theta^{\mathrm{raw}} \in (-\pi, \pi]$ is an atan2 (Section 11.3), applied as $\lambda_s\theta/2\pi$ on every phase signal of the satellite. In *fractional* mode (`estimation.accumulate_phase_windup` false, the loader default) $\theta = \theta^{\mathrm{raw}}$, and each crossing of $\pm\pi$ steps the applied correction by one cycle on every signal inside a continuous arc. The reference configurations accumulate: `GnssCorrections::computePhaseWindUp` keeps (θ_{k-1}, t_{k-1}) per satellite and forms

$$\theta_k = \begin{cases} \theta_{k-1} + \mathrm{w}\left(\theta_k^{\mathrm{raw}} - \theta_{k-1}\right), & 0 < t_k - t_{k-1} < T_{\mathrm{wu}}, \\ \theta_k^{\mathrm{raw}} \quad \text{(restart)}, & \text{otherwise}, \end{cases} \qquad \mathrm{w}(x) = x - 2\pi\,\mathrm{round}(x/2\pi), \tag{380}$$

with T_{wu} = `estimation.phase_windup_max_gap_s` (loader default 300 s; not set by the reference configurations). A repeated call within one epoch ($t_k = t_{k-1}$) reuses the accumulated θ_k and is not a restart. A restart is recorded in the ledger `wuRestarts`. The state is keyed by satellite; each station owns its correction object, so the key is injective. The optional gap bridge (`estimation.phase_windup_gap_bridge`, default off, not armed) replaces the restart for gaps between T_{wu} and `estimation.phase_windup_bridge_max_gap_s` by a walk of the angle through virtual epochs; the walk and its refusals are described with the observable in Section 11.3, and a refused bridge falls back to the restart answered below.

The restart is an integer, band-common step. By (380), $\theta_k - \theta_k^{\mathrm{raw}} \in 2\pi\mathbb{Z}$ at every epoch, so the continuation a restart replaces equals $\theta_k^{\mathrm{raw}} + 2\pi n$ for an integer n. The restart moves the modelled phase of every signal by $-n\lambda_s$: the same integer number of cycles on every signal, which the float ambiguities see as a slip $\delta N_s = n$ for all s. The raw observations did not change, so no raw-observable detector can see it. In residual space

$$\frac{f_1 n\lambda_1 - f_2 n\lambda_2}{f_1 - f_2} = n\,\frac{c - c}{f_1 - f_2} = 0, \qquad c_1 n\lambda_1 - c_2 n\lambda_2 = n\,\frac{c}{f_1 + f_2} = n\,\lambda_{\mathrm{NL}}, \tag{381}$$

with c_1, c_2 the coefficients of Equation (236): the step is MW-blind and appears in the ionosphere-free combination as $n\lambda_{\mathrm{NL}} = n\lambda_{\mathrm{IF}}$. It also happens across a gap of at least T_{wu}, where no time difference has an adjacent predecessor. The leveling carry survives gaps up to T_{pl}, and $T_{\mathrm{wu}} < T_{\mathrm{pl}}$ on the reference configurations; a return after a gap in $[T_{\mathrm{wu}}, T_{\mathrm{pl}}]$ therefore steps the model while the carried ambiguity re-injects at its pre-gap value, the situation that every stiff time-linked quantity must have a wired response for.

The restart response. `network.cycle_slip.windup_restart_refloat` (disarmed by the loader without accumulation or phase leveling) drains each present station's ledger in the PASS-A slip band (`networkSlipDrainWindupRestart`). A record not stamped with the current epoch is refused as stale; a gap beyond T_{pl} is `beyond_carry`, since the carry was pruned with it. Otherwise `networkWindupRestartTargets` resolves the stepped arcs: the satellite's live ambiguity columns at the station if any exist, else its leveling-carry entries satisfying (376) that the arc ledger (`network/state/ArcLedger`) does not record as dead. Each target enters the free-epoch latch

(cause `kCauseWindup`; the pending-reseed mark only on a live column), every target band is freed because the accumulator's verdict needs no raw corroboration, and the satellite's open bridge pends are voided, since a pend captured before the step would re-inject a value in the pre-restart frame. The drain must walk the carry and not only the live ambiguity map: a restarted arc's column was pruned with the gap, so a walk of live columns alone reaches no in-horizon restart, the inject folds the carry across the step, and the counter reports an answer that did not happen.

One mode for the network and the user. The measured gain of accumulation is in the user's own rows; the emitted product alone is neutral (`docs/refound/WUP_windup_accumulated_promotion_2026-10-01.md`). The S line of the emitted product declares both the wind-up construction (`windup_id` = `nominal-wu93`) and the mode (`wumode=`: `off`, `fractional` or `accumulated`, from `windUpModeOf`); Section 24.9 describes the check. The in-process user inherits the network's mode through the derived configuration; a replay or stand-alone user holds the rule by configuration.

24.6 The gross line-of-sight screen

Post-fit RAIM (Section 13.9) admits a row only if it carries an ambiguity coefficient above 0.5. A code row carries none, so a gross code error on one line of sight never enters $\hat{\sigma}_0$, a cell maximum or the median polish. `network.gross_los_screen` (`network/qc/GrossLos`) is the acting tier for code rows.

Statistic. Per station the seam forms $\boldsymbol{v} = \boldsymbol{y} - \boldsymbol{A}_s\hat{\boldsymbol{x}}_s - \boldsymbol{A}_r\hat{\boldsymbol{x}}_r$ with $\hat{\boldsymbol{x}}_r$ from Equation (332). A code row touches no SH, ambiguity or `PB:` column, touches a `CB:` or `CLK:` column and names a satellite; coupling rows are skipped. The statistic

$$r_j = |v_j|\,\sqrt{w_j}\,f_j \tag{382}$$

uses the row's own weight w_j and undoes the deweight factor f_j the assembly applied (1 if none), so both bars read the data at nominal weight. The row's own σ is used because the pooled $\hat{\sigma}_0$ inflates while a station degrades, exactly when the screen must stay sensitive.

Declaration and release. For one (station, satellite) with n code rows, n_{over} of them above k = `network.gross_los_k` (default 8), the epoch's evidence clears the bar when

$$n_{\text{over}} \geq m_{\text{eff}}, \qquad m_{\text{eff}} = \begin{cases} \max\big(1, \min(m, n)\big) & \texttt{available}, \\ \max(1, m) & \texttt{fixed}, \end{cases} \tag{383}$$

with m = `network.gross_los_min_rows` (loader default 2) and `network.gross_los_min_rows_rule` (loader default `fixed`, reference value `available`). A gross line of sight is wrong on every code band at once; one heavy-tailed row is one band's multipath. Under `fixed` a line of sight with one code row can never be declared; `available` reaches it and still demands two rows where two exist. Declaring takes `gross_los_epochs` (default 2) consecutive epochs of evidence (absence resets the streak) and passes two counted guards: the maturity horizon (`gross_los_warmup_epochs`, default 120 emitted epochs) and the whole-station stand-down (`gross_los_max_station_frac`, default 0.5: a station-wide fault is not a line-of-sight fault). Release takes every code row under `gross_los_clear_sigma` (3) for `gross_los_clear_epochs` (2) epochs and is never guarded. A declaration unseen beyond `gross_los_max_gap_s` (900 s) expires with its own event: the next sighting is a new arc.

The cold bound. The maturity horizon holds every declaration above for the first `gross_los_warmup_epochs` emitted epochs, because a cold post-fit code residual measures the unconverged station block. A faulty receiver channel inside that window is a different case: its carried ambiguity makes the line of sight redundant, and the fault then enters the satellite clock at metres per epoch, which the dense border carry keeps. With `network.gross_los_cold_bound_m` $= b$ (loader default 0, off; reference value 12 m) a line of sight whose largest post-fit code residual $|v|$ exceeds b inside the horizon is declared at once: one epoch, no row count, no persistence. The same whole-station stand-down applies (more than `gross_los_max_station_frac` of a station's lines of sight over the bound declares none). While a cold declaration stands, every epoch on which $|v| > b$ again erases the carried ambiguity again, because a free ambiguity absorbs its phase row exactly whatever the number of bands; at the release and at an expiry the ambiguity is erased once more, so the line of sight reconnects on a fresh ambiguity and not on one harvested while the fault was in its rows. The cold path's edges are counted apart from the sigma-bar path's.

Response. From the next assembly the declared code rows fold at $\sigma \times$`gross_los_deweight` (default 100; `networkGrossLosPublishScales`); the row keeps its sign and still anchors the float if the phase side disappears. At the declaration the line of sight's leveling-carry entries (`AmbGrossErase`, with a pending-reseed mark) and slant-ionosphere entries (`IonoGrossErase`) are erased. Both sites are declared EraseAndRebirth on a decouple event: this response erases where the slip responses drop. Phase rows keep their weight; they have their own tiers. The bounded variant `gross_los_bound` (freeze instead of erase) was falsified and stays off (`docs/refound/RF-3_code_row_qc_2026-09-22.md`).

Two halves and the emission exclusion. The erase must follow the carry harvest (`networkLeveling`), or the harvest re-stashes the erased entries; the decision needs only the solved epoch. With `network.gross_los_emit_exclude` the seam is split (`GrossLosPhase`): the Decide half runs immediately before `networkSolPrep`, the Act half after `networkPostFitRaim`. Unsplit, a declaration reaches the emitted product one epoch late. The emitted clock and code bias are joint-solve states, and removing a station's contribution would take a second solve, so the exclusion acts on the records. For each satellite i screened at n_i stations, e_i of them with a current declaration on it,

$$n_i - e_i < N_{\min} \;\Rightarrow\; \text{withhold } i; \qquad \text{otherwise emit } i \text{ with a discontinuity mark,} \tag{384}$$

with $N_{\min} =$ `network.gross_los_sat_min_stations` (loader default 3). The withhold is per satellite and the epoch still emits. The mark joins the retro queue, whose drain this setting arms, and lands with full channel scope (clock, code bias, phase bias) in the first emission containing the satellite, normally the declaring epoch.

24.7 The emitted clock

The emitted satellite clock c^i is the joint-solve column of Section 20, carried between epochs through the guarded temporal anchor of Section 23. Four output-only steps act on it between the solve and the product; none changes the solve.

Held frame (`network.emit_clock_frame_hold`). The common mode of the satellite clocks of one constellation is the clock-ensemble gauge: a user's receiver clock absorbs it identically on code and phase. The solve re-realises it freely, so the raw emitted clock moves by a common amount

from epoch to epoch that carries no physical content (`docs/SPEC_clock_emit_rebase.md`). For constellation k let $c^i_{\text{held}}(t^-)$ be the previous emitted value and A_k the frame offset. The median innovation of the fresh, unforced keys (at least `min_common` = 4 of them),

$$\mu_k(t) = \operatorname*{med}_i \Big[\big(c^i(t) - A_k\big) - c^i_{\text{held}}(t^-) \Big], \qquad A_k \leftarrow A_k + \mu_k(t), \qquad c^i_{\text{held}}(t) = c^i(t) - A_k, \tag{385}$$

is accumulated, and every satellite, a new sighting included, is emitted in the held frame. One subtrahend serves the constellation, so between-satellite differences are unchanged. When $|A_k|$ exceeds `network.clock_frame_leash_m` (default 50 m) the constellation re-anchors to the solve frame, and the re-anchor is a datum mark so the re-coupling edge is declared.

Clock step test (`network.clock_disc_jump_test`). After the frame hold, a satellite whose common-removed step

$$d^i(t) = c^i_{\text{held}}(t) - c^i_{\text{held}}(t^-) \tag{386}$$

(with the hold off, the raw step less its median μ_k) exceeds `network.clock_disc_jump_m` (default 1.5 m, a flat threshold set well above the per-epoch walk step) is reported for a retro-bump: an `R` record names the satellite, the satellite's discontinuity counters step, and the user resets that satellite's ambiguities at the event epoch. Keys already marked in the epoch (a pending retro-bump, a datum-changed constellation) are excluded from the vote and from the test, so a marked event neither drags the frame nor is marked twice. A satellite returning after more than `max_gap_s` is a first sighting and is not tested unless the gap-aware extension (`network.clock_disc_gap_jump_m`, default 0, off) tests it against its retained pre-gap baseline. The clock counter of the emitter has no magnitude test of its own; the step test lives at the emission-preparation tier and drives the forced path, which a replay reproduces from the serialised `R` records. A clock step that reaches the product without being declared is a contract violation, so the test must be armed (`docs/cards/clock/CS-clock-step-signalling.md`).

Clock discontinuity counter (`network.emit_clock_disc`). With the setting on, a clock-pivot handover bumps the clock counters of the changed constellations only and serialises that constellation set on the epoch record; with it off the bump is global. The setting is part of the contract (Section 24.9): a replay emitter built without it misses the scoped declarations.

Quality channel (`network.emit_ura`). Each emitted clock with a non-pinned column carries

$$\text{URA}^i = \max\{1,\ \bar{\sigma}_0\}\ \sqrt{\boldsymbol{P}_{cc}}, \tag{387}$$

with $\boldsymbol{P}_{cc}$ the posterior variance of the satellite's shared clock column (a pinned clock has an artificial variance and no URA) and $\bar{\sigma}_0$ the exponentially weighted variance factor of post-fit RAIM (`network.ura_ewma_alpha`, default 0.1; one epoch behind, because the post-fit tier runs after the emission). A satellite refloated within the inflation horizon is multiplied by `network.ura_refloat_inflate` (default 1, no inflation). The channel needs `network.raim`; without it the URA is the formal clock sigma alone and the loader warns.

Neighbouring checks. The clock runaway check (`network.clock_runaway_check`, armed) drops from the emission a constellation whose median clock differs from the global median by more than `network.clock_runaway_mag_m` (1000 m) before the product is written; the drift ceiling needs the two-state clock model, which is off (Section 13.11). The anchor's decoupling and re-coupling edges and its marks are described with the anchor itself; a mark names the satellite and steps its counters, so the user re-floats exactly that satellite.

24.8 The emitted phase bias

For satellite i, signal s and emission epoch t, let $c^i(t)$ be the emitted clock (C_0, metres) and $p^i_s(t)$ the estimator's phase bias in the emit frame (the `bias_m` `networkPbHold` receives). The product carries $C^i = c^i$ and P^i_s; the user applies $-C$ on code and $-C + P$ on phase (Equation (303)).

The generation hold. Under `network.emit_pb_hold` each key (i, s) carries a generation $g \geq 1$, started at t_0 when the key is first emitted or was last emitted more than T_{h} ago (`network.emit_pb_hold_max_gap_s`, loader default 300 s; not set by the reference configurations). Within a generation

$$P^i_s(t) = p^i_s(t_0). \tag{388}$$

The generation is declared by a `gen=`g token on every P record, by `SsrPhaseBiasEntry::hold_generation` in the in-process store, and by `pbhold=` on the S line (1 plain, 2 phase-clock mode, 3 smoothed). The solve and the C and B records are byte-identical with the setting on. The premise, graded against truth (`docs/refound/PBH_phase_bias_hold_2026-09-19.md`): the phase-bias motion within an arc is error that adds to the clock error. Like the `iode=` cutover, the contract states that the value has not changed and names the epoch at which that statement restarts; handover steps of the estimate stay out of the emitted product.

Ordering contract. `networkPbHold` is the last writer of `bias_m` in `networkEmitStore`: after the carry-invariant emission re-base (gated on `network.carry_invariant`, inert on the reference configurations) and after every discontinuity-marking site; before the emitter's value-jump test, which then screens the held value; before the file write, which reads the same solution as the in-process store.

Both edges. Decoupling: each held emission records $|p^i_s(t) - P^i_s(t)|$ into a run-end distribution. Re-coupling: each generation start is a counted `PB_HOLD` `gen_start` event (cause `first` or `gap`). A mark naming a satellite's clock and phase bias together states that both moved; the plain hold drops the phase-bias half and records the de-commoned clock step it left out (the phase-clock mode below transfers it). The de-commoned step of a satellite is its emitted clock step less the median step of its own constellation, $d^i = [c^i(t) - c^i(t^-)] - M_{k(i)}$, so a common mode that differs between constellations is not charged to either.

The phase-clock mode. Under `network.emit_pb_hold_phase_clock` (armed) the common clock motion of constellation k accumulates over consecutive emissions $t^- < t$,

$$M_k(t) = M_k(t^-) + \operatorname*{med}_{j \in \mathcal{S}_k(t)} \left[c^j(t) - c^j(t^-)\right], \tag{389}$$

over the satellites $\mathcal{S}_k(t)$ of k with a finite clock at both emissions (an even count takes the mean of the two middle values; `pbHoldCommonClockStep`). With $\tilde{c}^i(t) = c^i(t) - M_{k(i)}(t)$ the held emission is

$$\boxed{P^i_s(t) = p^i_s(t_0) + \left[\tilde{c}^i(t) - \tilde{c}^i(t_0)\right].} \tag{390}$$

An emission without the satellite's clock reuses the last transfer; a generation started without one is held plainly. Subtracting from $C^i = c^i$ gives the phase-side identity

$$C^i(t) - P^i_s(t) = c^i(t) - p^i_s(t_0) - c^i(t) + M_k(t) + c^i(t_0) - M_k(t_0) = \left[c^i - p^i_s\right](t_0) + M_k(t) - M_k(t_0). \tag{391}$$

The constant goes into the user's float ambiguity. The common mode is part of every c^j of the constellation, so it enters code and phase rows alike and the receiver clock and inter-system bias absorb it. Satellite-specific clock motion reaches the code rows only. In the family $(C-P)(t) = (c-p)(t_0) + \Phi(t)$ the phase-clock mode is $\Phi = M_k(t) - M_k(t_0)$; the plain hold is $\Phi = c(t) - c(t_0)$, every clock move on the phase side; no hold passes the estimator's own phase-visible change, wander included.

Why the mode exists. Under the exact coupling time update armed on the reference configurations (`network.carry_coupling_exact_update`; Section 23) the satellite's level converges: the estimator moves c^i and every p_s^i by the same amount, invisible to phase. The plain hold freezes P while C follows, turning the convergence into a phase-side drift the user's float ambiguities cannot follow. Under the phase-clock mode the move shifts $\tilde{c}^i$, hence P, by the same amount (unless common to the constellation): $C - P$ stays and the level reaches the code side only (`docs/refound/LAH_level_aware_hold_2026-09-29.md`, `EXL_exact_update_lah_promotion_2026-10-01.md`).

Limitation. The mode is interim. Within a generation the phase-visible clock motion the user sees is the a priori clock product's alone, since the emitted clock refines the a priori clock (Section 20.11) and the hold passes none of the network's satellite-specific motion to the phase side. That is valid with a precise a priori clock (`network.apriori_source sp3`, the default). The loader warns under `brdc_rts` and refuses the mode under `sp3_brdc_clock`. The smoothed member (`network.emit_pb_hold_smoothed`, `pbhold=3`), which carries a clock clone, is built and not armed; it excludes the phase-clock mode and needs the exact update and the temporal anchor.

Remark 11. A datum move between two generation starts, such as a phase-anchor handover with its S-transform, moves the estimator's phase biases and not the held values (counted, measure-only: `prm_pbhold_unmoved_at_shift`). A float user absorbs each held constant in its own ambiguity. An integer-resolving user needs every phase bias of a constellation in one S-basis (Section 20.3; [25]), which a held product does not guarantee after a datum move. The reference configurations run the test station's user float (`closed_loop.ar_mode` 0).

24.9 The product contract

The emitted product is a line-based file read by the replay driver and the real-time user, and pushed record by record into an in-process store for the test station. Each epoch is an `E` line (week, time of week, IOD, provider, solution identifier, the datum-change flag) followed by its `C` (clock, with the counters and the URA), `B` (code bias) and `P` (phase bias with the signal-integer flags, the counter and the `gen=` token) records; `R` records list the retro-bumped satellites, `I` the solution-generation identifier, and `H` (`network.emit_epoch_status_record`, armed) an epoch-status heartbeat that also stands alone for a withheld epoch. Optional records and tokens appear only when their setting is on, so a run with them off writes the base format.

The S line. The header and any periodic re-broadcast (`network.contract_rebroadcast_s`) carry a machine-readable contract (`SbasisContract`). It declares the reference receiver, the clock pivot of each constellation, the ambiguity datum, the inter-system datum, the code-bias datum and the bias sign; the wind-up construction and mode (`wumode=`); the identity of the a priori orbit and clock product and of the frame, coordinate epoch, antenna file and orbit reference point; the code-bias pin pair of each constellation and the value of the geometry-free datum (`sat_gf_datum=`); and the parameters with which a replay emitter reproduces the discontinuity counters: the phase-bias discontinuity threshold (flat in metres, or a fraction of the wavelength through `discfrac=`,

set by network.disc_threshold_frac_cycles, reference value 0.25), the clock-counter channel (emit_clock_disc), the staleness horizon, the code-bias jump test (cbjump=), the clock set-change trigger (clkset=) and the mark scope (discscope=). The token pbhold= (hold, phase-clock mode, smoothed mode) states how the phase-bias channel is read, together with the gen= token on every P record, and gid= names the run that wrote the product.

The check. NetworkProductReader::checkContract compares a contract read from a file with the contract a user expects and lists every mismatch. The comparison covers the reference receiver, the bias sign, the wind-up construction, the ambiguity, inter-system and code-bias datums, the code-bias emission flag, the pivot of each constellation the user declares, the pin pairs of each declared constellation, the seven replay-emitter parameters, the three hold flags, the a priori product, and the identity fields and generation identifier the user declares. Two rules matter for interpretation. The wind-up mode is compared only when both sides declare one, because a product written before the declaration cannot be verified and keeps replaying; a mismatch means a different carrier-phase frame, by whole cycles per satellite and signal, which a float ambiguity absorbs without any residual, so the check is the only place the difference is visible. The hold flags are compared in both directions, because a gen= payload under a contract that declares no hold (or the reverse) means the contract and the records disagree. A product without an S line is refused by the replay unless network.replay_allow_undeclared_wire is set, since the replay emitter would otherwise be built with defaults and miss every clock declaration the network made.

Self-check. At the end of a run the writer is closed, the product is read back, and the parsed contract is checked against the one written, together with the realised pivot map and, when re-broadcast is armed, every re-broadcast receipt. The outcome is written to network_contract_check.csv; a run that emitted nothing records a failure row instead of leaving the file absent, so a missing file means only that the check never ran.

24.10 The user's answer

The in-process user of the test station (undifferenced in the reference configurations) reacts in realtime/RealTimeEngineSsrBreaks, in a fixed order. First, reactToGenerationChange reads the product's solution-generation identifier, which is distinct from the hold generation; a change (a restart of the network, a global re-realisation) resets every network-corrected ambiguity and re-inflates the receiver code-bias states. It runs before the per-signal reactions because a restart re-seeds the discontinuity counters. The order of the phase-bias reactions that follow, with the translation alternatives, is also stated in Section 18.6; this section gives the answer to the hold and to its presence edges.

Hold generation. resetOnPhaseBiasDisc dispatches reactToPbHold_ before any other phase-bias reaction, keyed on the data: a key whose store entry has a nonzero hold_generation is claimed. Its discontinuity counter is consumed, so the per-key memory advances, and ignored, since under a hold a counter change carries no value step. PbHoldLedger::react decides: *FirstSight* adopts the generation, *Held* keeps it, *Refloat* answers a change of a previously seen generation with StateManager::resetAmbiguity at the configured ambiguity σ, the call a discontinuity reset uses. First sightings, held cells and refloats sum to the visited cells.

Presence edges. Under the bias staleness gate (`processing.enable_ssr_bias_max_age`, armed on the derived user of the closed loop) the user tracks

$$a_s(t) = \left[\text{bias present} \wedge \text{age} \leq \texttt{ssr_bias_max_age_s} \cdot \texttt{ssr_bias_stale_hold_factor}\right] \tag{392}$$

(hold factor default 2) and answers an edge $a_s(t) \neq a_s(t^-)$, the freshness flip, with an ambiguity reset. A held product has two such edges with no generation change: a held bias appearing on a phase row already in use without it, and one leaving such a row; either steps the applied correction by the whole bias. Without the setting below, a claimed key skipped every downstream reaction, the freshness flip included, and its memory was not written while held, so both edges went unanswered (a first sighting lacks prior state only on a new row). The update's post-fit peel then attributed the residual to other satellites, reset their ambiguities and could exclude the faulty satellite's code rows; one real-data epoch moved the position by about 490 m (`docs/refound/PBF_pb_hold_presence_edges_2026-09-28.md`). With `processing.ssr_pb_hold_presence_flip`, derived from `network.closed_loop.pb_hold_presence_flip`, a claimed key skips only the counter reaction, which the hold owns, and its freshness edge runs as on an unheld key: on the reference configurations a reset, since the step translation (`closed_loop.pb_step_translate`) and the level check (`closed_loop.amb_level_check`, harmful under the hold) are off. When the network declares a signal roster and the user latches on it (`network.closed_loop.roster_latch`), a flip on a signal for which the network declares no phase-bias family is attributed to an absent family and not to staleness, and the ambiguity is left alone.

Translation instead of reset. Two optional reactions replace the reset by an exact translation of the float ambiguity by the step of the applied phase bias: the declared-step carry (`processing.ssr_phase_bias_disc_carry`) and the step translation (`closed_loop.pb_step_translate`, which also answers undeclared steps). Both are off on the reference configurations. Their algebra, their order against the hold and the reset, and their fallbacks are in Section 18.6; the derived configuration of the in-process user is in Section 18.9.

24.11 Status

Armed on both reference configurations through the promotion ledger (`docs/CONFIG_promotion_ledger.json`): gap re-entry per row (`docs/cards/slip/CARD-G-GAP-REENTRY-RELEVEL.md`), per signal (`docs/refound/GRS_gap_reentry_per_signal_2026-09-20.md`) and with the tracker's band naming (`GRK_gap_reentry_key_mismatch_2026-09-22.md`); the MW re-flag skip; the re-admission horizon and its mirrors (`docs/cards/network-vehicle/READMIT-EDGE.md`); accumulated wind-up with its restart response; the gross line-of-sight screen with its cold bound; the generation hold in its phase-clock mode; the held clock frame, the clock step test, the clock discontinuity counter and the quality channel; the user's presence edges. The baseline with these settings (`docs/refound/BOF_built_off_follow_through_2026-10-28.md`), over a 6 h run: the test station WTZZ00DEU on real data, median 3-D position error per hour 0.059, 0.012, 0.023, 0.012, 0.017 and 0.012 m; the test station SZZ000SIM on the simulated network, graded from the run start, 0.137, 0.072, 0.031, 0.024, 0.026 and 0.018 m. Not armed: the co-resumed share, which answers only the third-frequency case it was built for (`docs/CONFIG_promotion_ledger.json`); the wind-up gap bridge; the smoothed hold; the user's translation reactions.

Open: the hold in the warm-resume snapshot and the cause of the phase-bias wander; the held emission under a phase-bias shift; the regressions of the phase-clock hold at stations tracking the C1W signal and its need for a precise a priori clock, which the self-contained clock level was to remove and did not; a dedicated emitted-product writer for the gross screen; the failed real-data

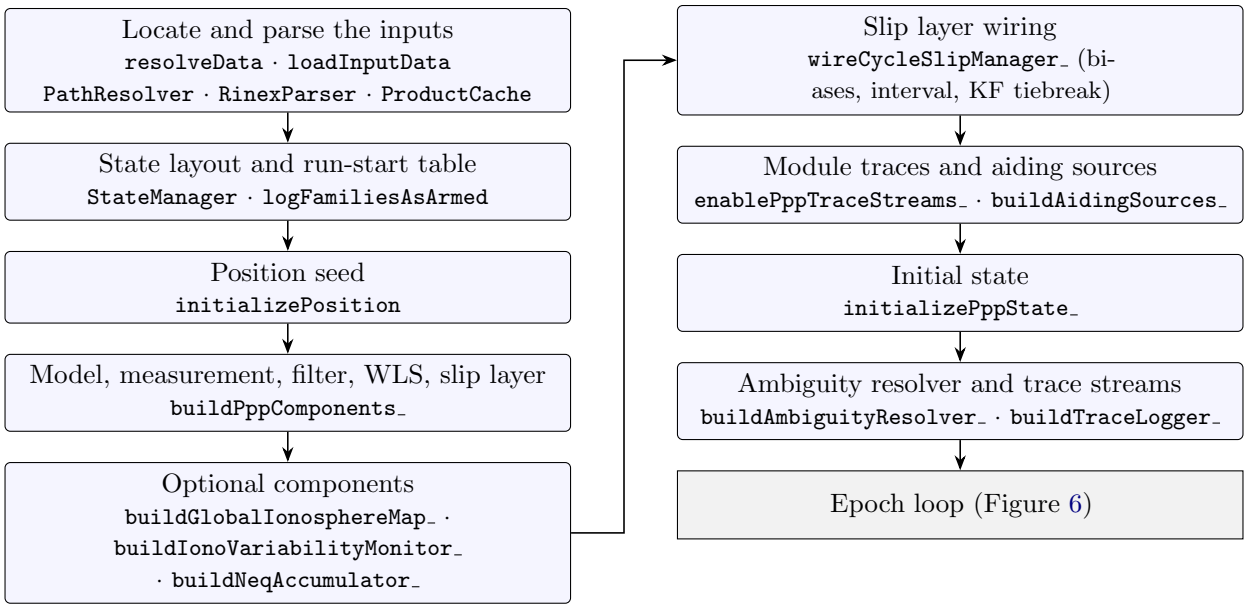


Figure 5: Preparation of a PPP or PPP-RTK run in `Application::runPpp`. The steps after `initializePosition` are private members defined in `app/PppSetup.cpp`, called in this order. `StateManager` declares the state families once and prints them as the run-start table *state families as armed*; the optional components are built only when their settings are on.

triple-frequency bar of the per-signal tier; and the peel's masking of a bad phase row. The register is `docs/status/OPEN_ITEMS.md`.

Part X
Reference

25 Pipeline and architecture diagrams

This section shows the control and data flow behind the algebra of the other chapters. For the single-receiver path it gives the preparation of a run (Figure 5), the per-epoch loop (Figure 6) and the shared geometry kernel behind the three differencing regimes (Figure 7). For the real-time path it gives the order of the seams of one epoch, with the pre-update innovation screen (Figure 8), the SSR correction pipeline (Figure 9) and the streaming front end that feeds it (Figure 10). For the network estimator it gives the joint estimator (Figure 11), the structure of a run (Figure 12), the epoch before and after the solve (Figures 13 and 14), the fourteen stages of the solve (Figure 15), the emission (Figure 16) and the harvest of the carried state (Figure 17). Each box names the function or class that implements the step, so the diagrams also serve as a map into `src/`. The diagrams are maintained by hand and simplify the code: they show the main sequence, not every gate or diagnostic.

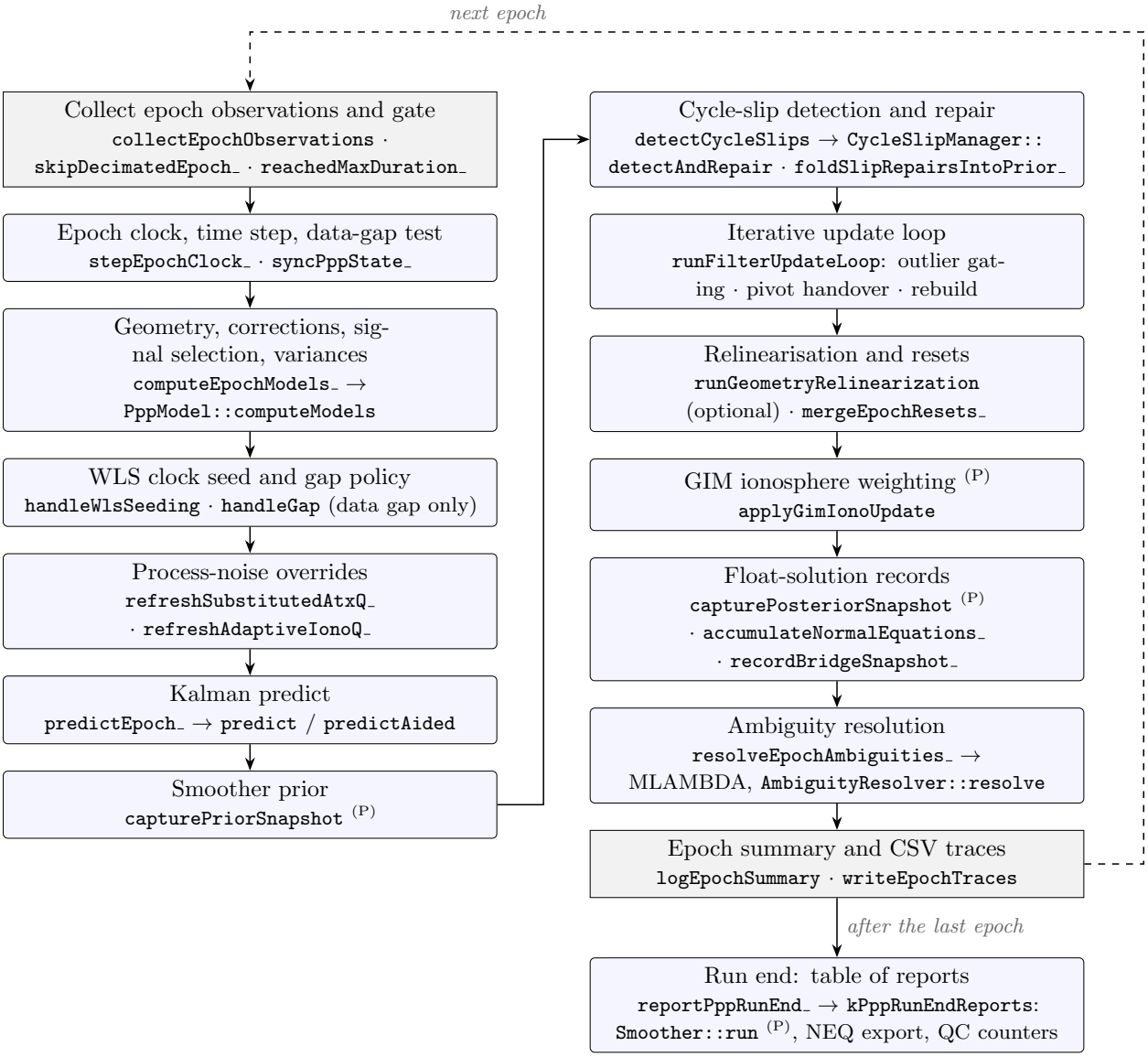


Figure 6: Per-epoch loop of `Application::runPpp`. Solid arrows give the sequence within an epoch. The first column runs from the observations to the prediction; the second column runs from the slip layer to the output, after which the loop returns to the first box for the next epoch. Steps tagged (P) run in post-processing only; the real-time path (`RealTimeEngine`, Figure 8) has no GIM source and no smoother. The smoother runs once, after the forward pass, as one report of the run-end table, and only when `processing.run_smoother` is set. The values the stages hand to one another are the members of one `PppEpochWork`, built empty at the top of each epoch.

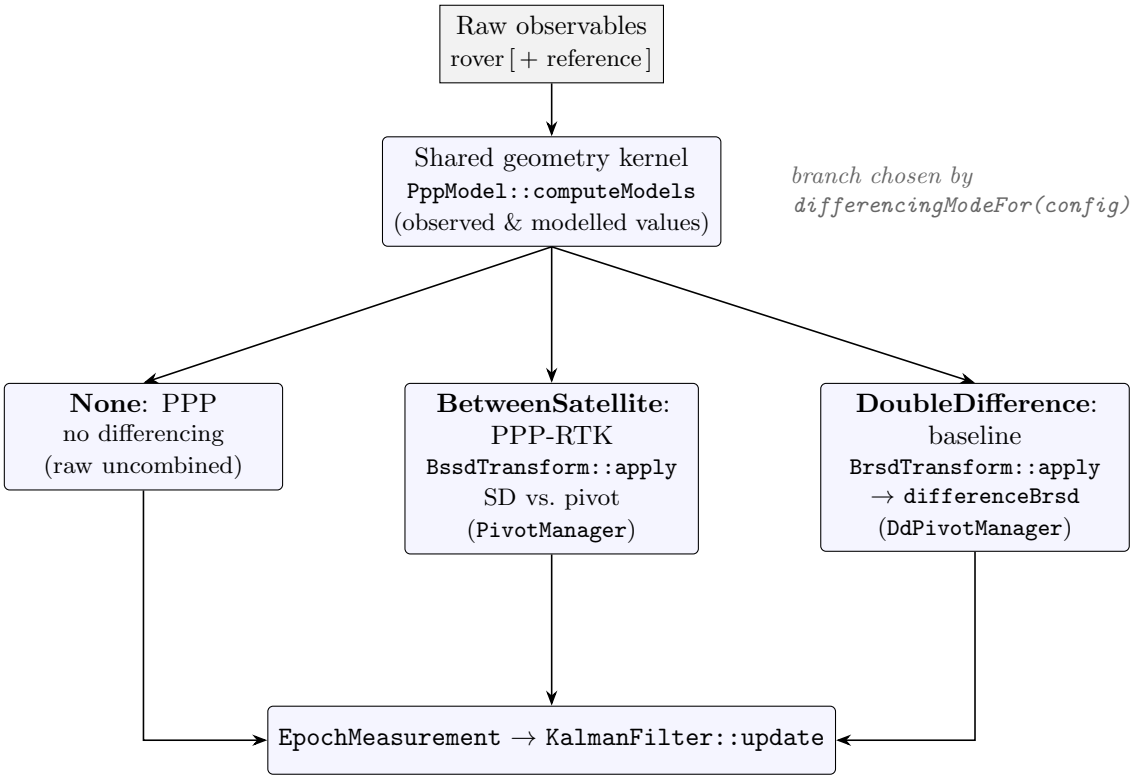


Figure 7: The three differencing regimes share one geometry kernel. The undifferenced observed and modelled values are computed once. The regime returned by `differencingModeFor` (selected once, when `MeasurementBuilderFactory` builds the measurement builder) then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline). All three feed the same Kalman update.

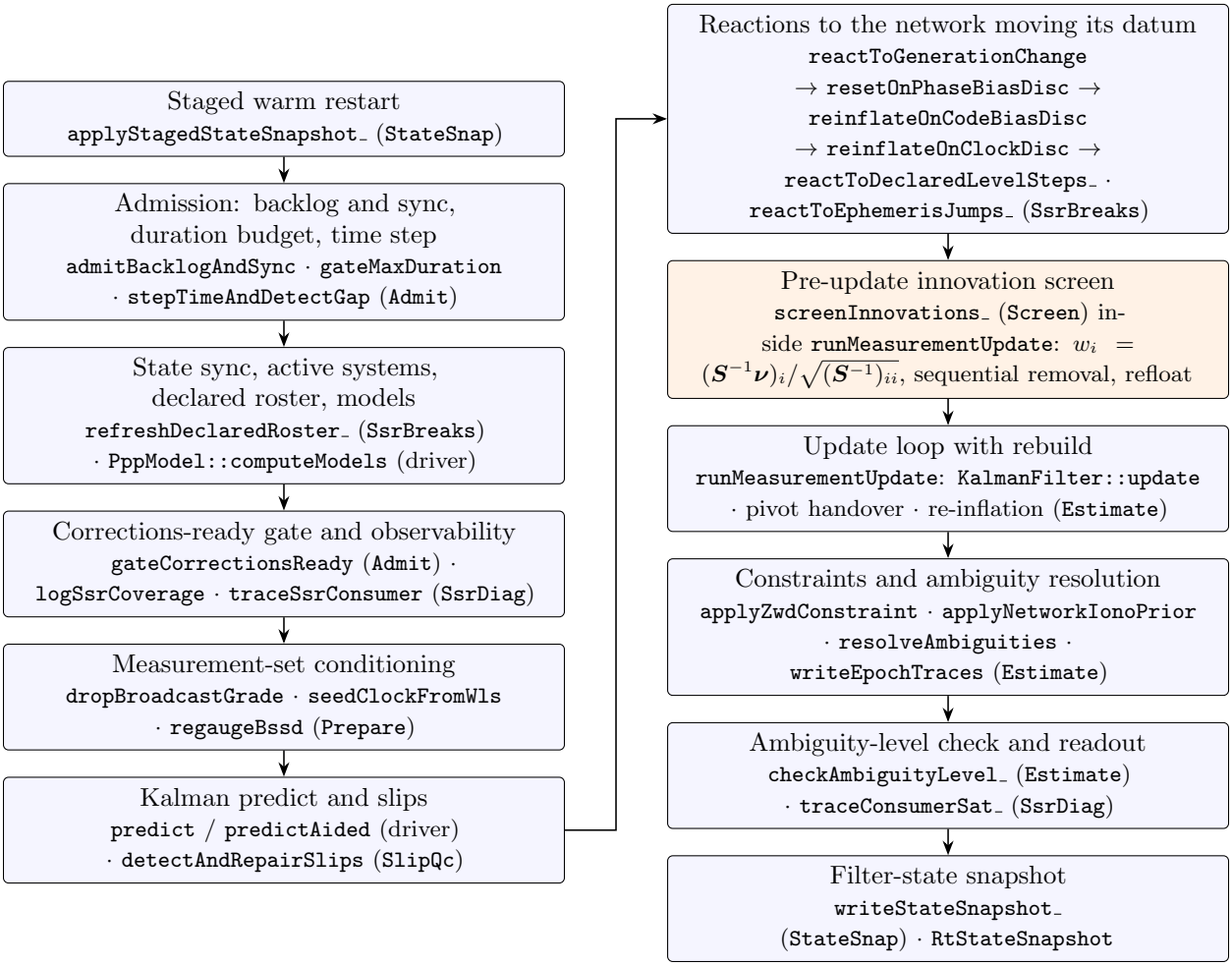


Figure 8: Order of the seams of `RealTimeEngine::processEpoch`. The names in parentheses are the files `RealTimeEngine<name>.cpp` that hold each seam. Only the admission gates, the corrections-ready gate and the empty-epoch checks of the driver can end an epoch; everything after them is straight-line. The reaction to the generation change comes first because it subsumes the finer reactions that follow. The shaded box is the user's pre-update innovation screen (Section 19.11): it tests every row against the predicted state before the first update pass can spread a gross row over the states, and builds that first pass itself with the flagged rows held out. The other seams with a section of their own are the staged warm restart and the filter-state snapshot (first and last box; Section 19.14), the broadcast-baseline jump response (`reactToEphemerisJumps_`, in the reaction box; Section 19.13) and the ambiguity-level check (`checkAmbiguityLevel_`; Section 19.12).

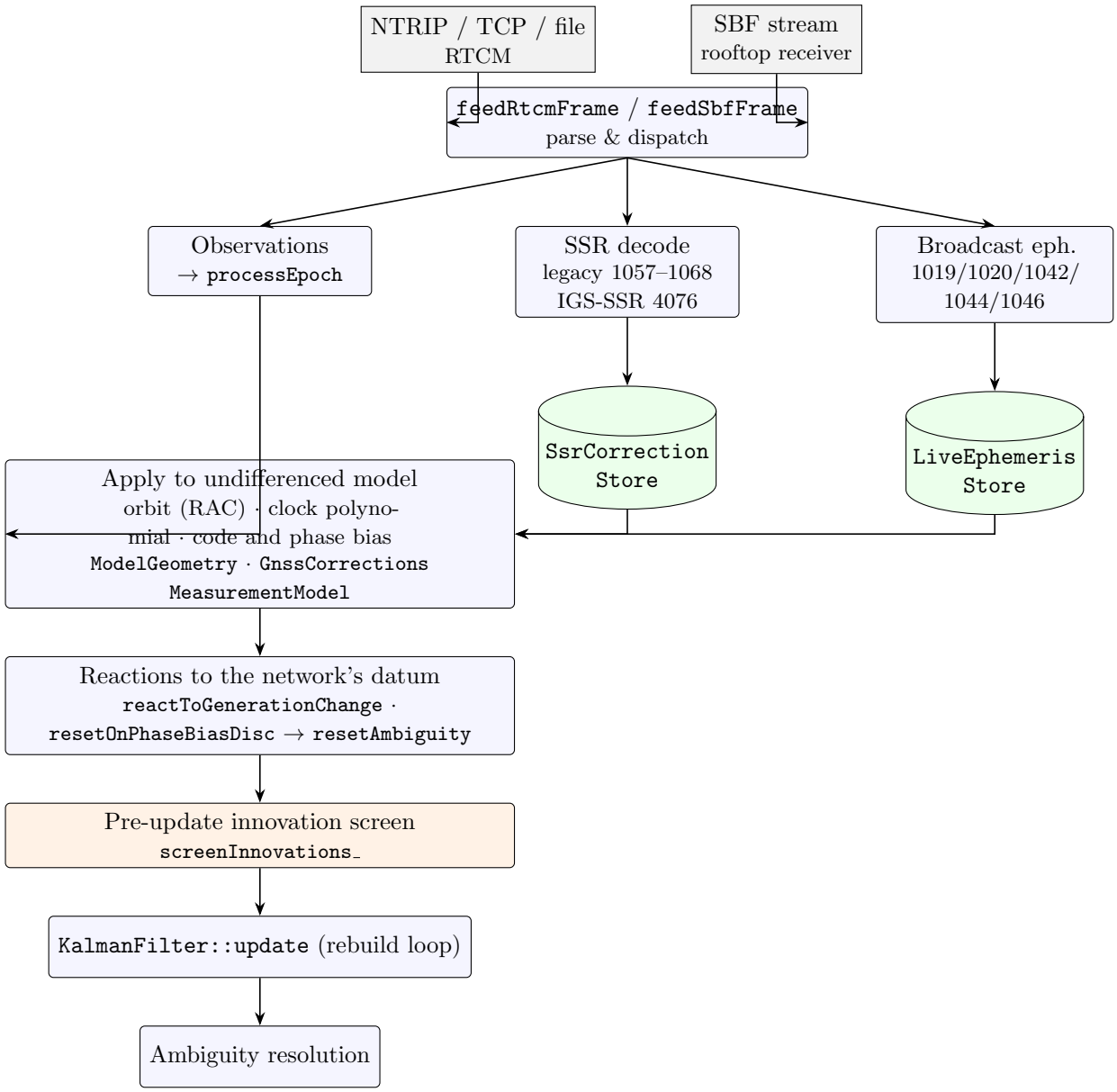


Figure 9: Real-time SSR correction pipeline. The decoder splits the byte stream three ways: observations go to `processEpoch`, SSR corrections to `SsrCorrectionStore`, and broadcast ephemerides to `LiveEphemerisStore`. The legacy SSR decoder accepts message types 1057–1068 and 1240–1270; the diagram lists the first range only. The undifferenced observation model then applies the precise orbit (RAC frame), the clock polynomial and the code and phase biases. After the slip check and before the Kalman update, `reactToGenerationChange` resets every network-corrected ambiguity when the product's solution generation changes, and `resetOnPhaseBiasDisc` answers a change of a phase-bias discontinuity counter (by default with an ambiguity reset), a hold generation, or a freshness flip of the applied bias; `reinflateOnCodeBiasDisc` and `reinflateOnClockDisc` follow for the code-bias and clock counters (Section 24.10; Section 18.6). The innovation screen runs inside the update step ahead of the first pass. See `RealTimeEngine::processEpoch`.

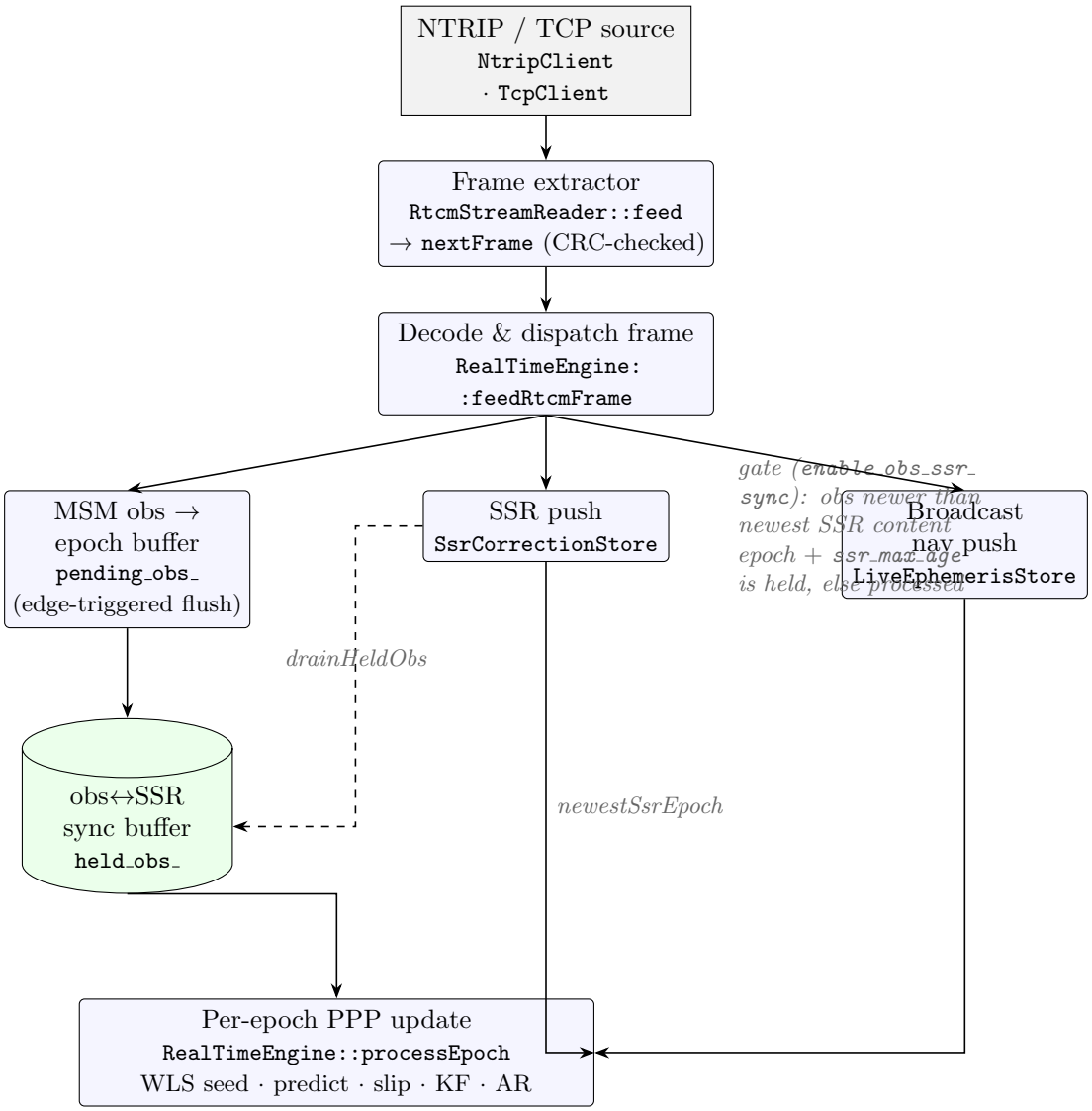


Figure 10: Real-time streaming front-end ahead of the SSR correction stage of Figure 9. `RtcmStreamReader` reassembles the raw bytes from `NtripClient` or `TcpClient` into CRC-validated RTCM3 frames, and `feedRtcmFrame` decodes and dispatches each frame. MSM observations accumulate in `pending_obs_`, which is submitted to `processEpoch` when a frame with a new epoch time arrives (the edge trigger). SSR corrections and broadcast ephemerides go to their stores. With `enable_obs_ssr_sync` set, the first admission gate of `processEpoch` (`admitBacklogAndSync`) holds an observation epoch that is newer than the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`). The held epoch is kept in `held_obs_` (only the latest one) and resubmitted by `drainHeldObs` after each SSR push. An epoch inside the horizon is processed at once. A hold longer than `obs_ssr_sync_max_hold_s` (default 30 s) latches a degraded outage state in which epochs pass through without SSR until the stream covers them again. The gate applies only with a live SSR orbit and clock source and never before the first epoch. See Section 19.

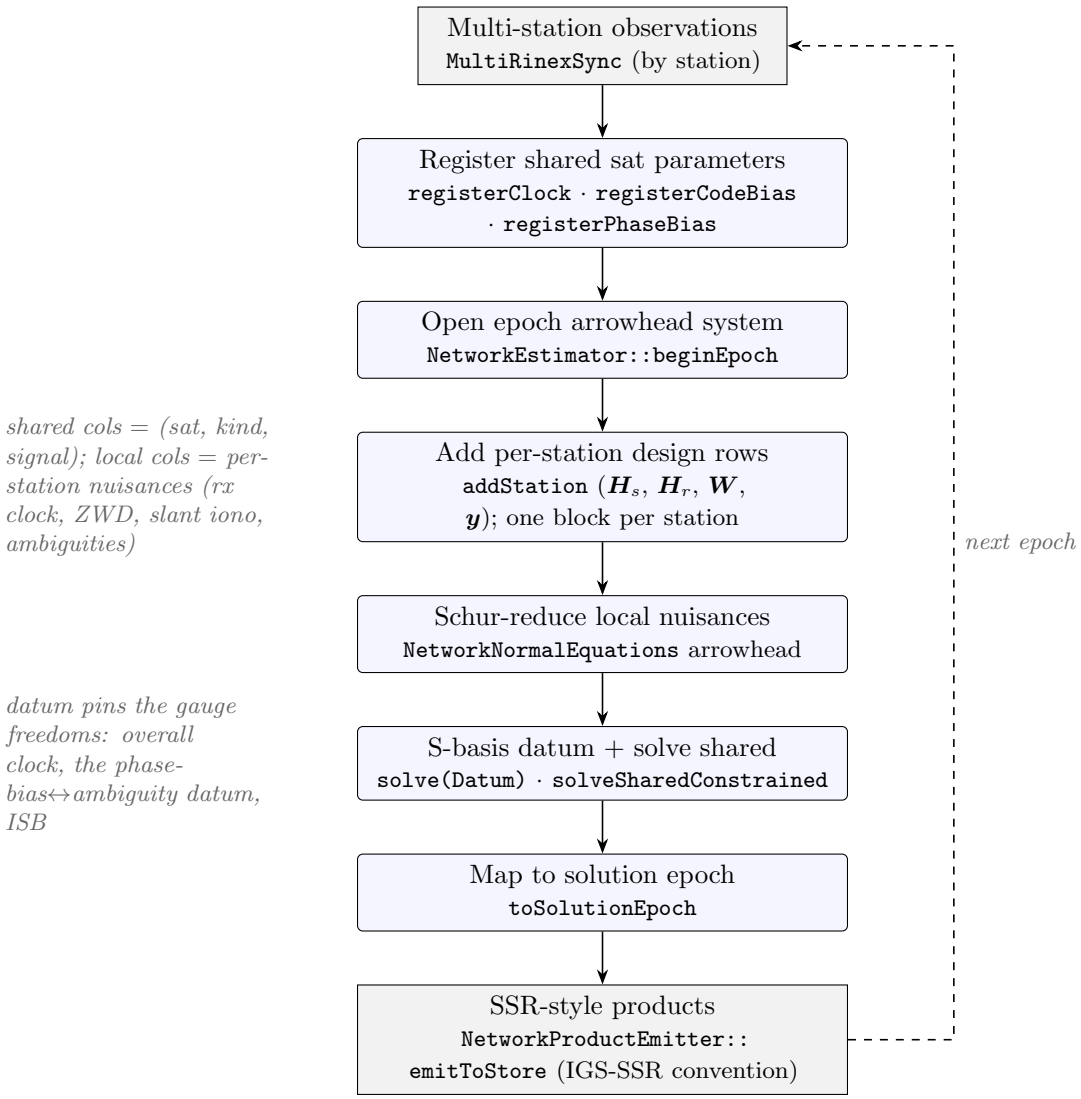


Figure 11: Joint estimator of the PPP-RTK network (`NetworkEstimator`, `network/solve/Estimator`). The shared satellite-side columns, one per (satellite, parameter kind, signal) for the estimable clock, code bias and phase bias, are registered afresh every epoch (`reset`, `registerClock`, `registerCodeBias`, `registerPhaseBias`; `registerVtecCoeff` serves the spherical-harmonic ionosphere, which the reference configurations do not use); `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` adds one pre-formed block of local-versus-shared design rows per station, with no receiver-side differencing. Schur reduction removes the local nuisances; the reduced shared system is rank-deficient by the gauge freedoms. `solve` fixes them with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the estimate to a record that `NetworkProductEmitter` writes in the IGS-SSR convention. The driver only orchestrates: Figure 13 shows where the columns are registered and the rows added, Figure 15 the stages around the datum and the solve, Figure 16 the emission. With `network.closed_loop.enable` set, the emit step also drives a test station's PPP-RTK user on the new products: undifferenced in the reference configurations (`closed_loop.consumer_undifferenced`), between-satellite differenced otherwise. See Section 20 and Section 20.1.

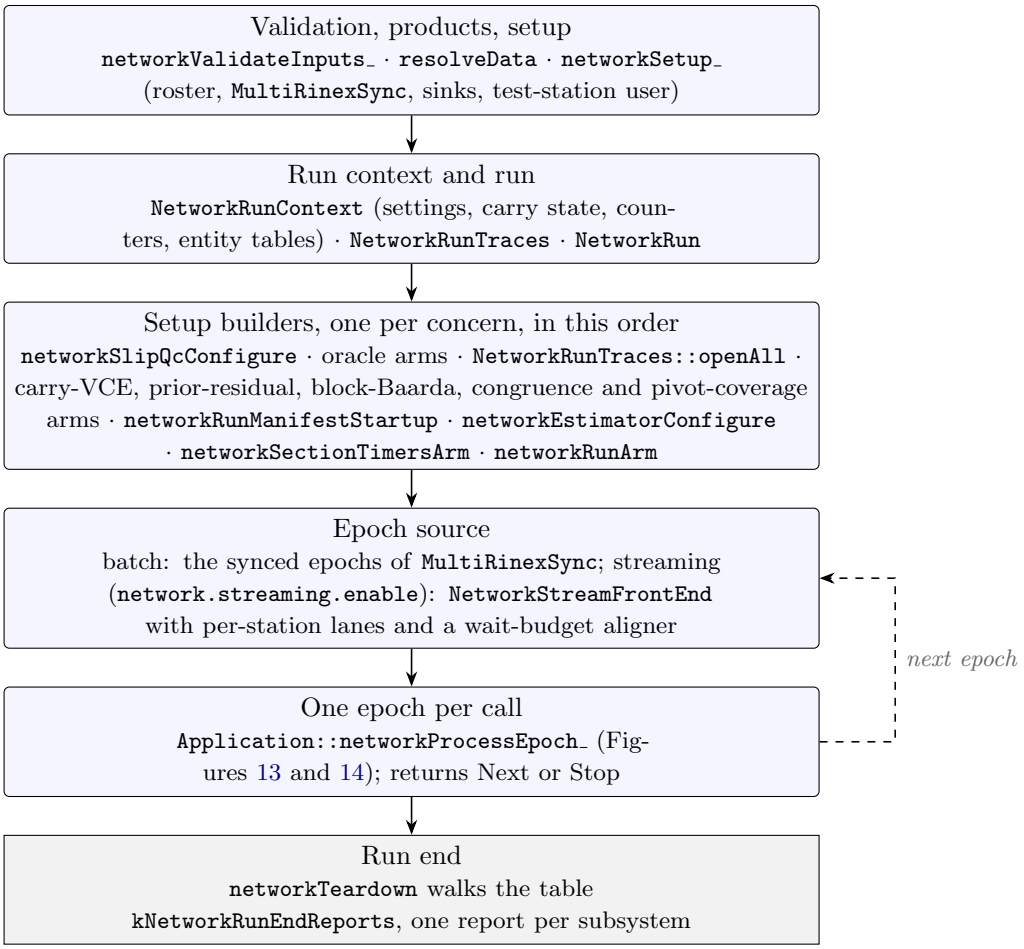


Figure 12: Structure of `Application::runNetwork`, the thin orchestrator of a network run (`network/driver/ApplicationNetwork`). Setup, the builders and the run end are lists of calls; the epoch loop has one body, whichever epoch source feeds it. A closed-loop replay of an existing product file replaces the whole run (`runConsumerReplay`) before validation.

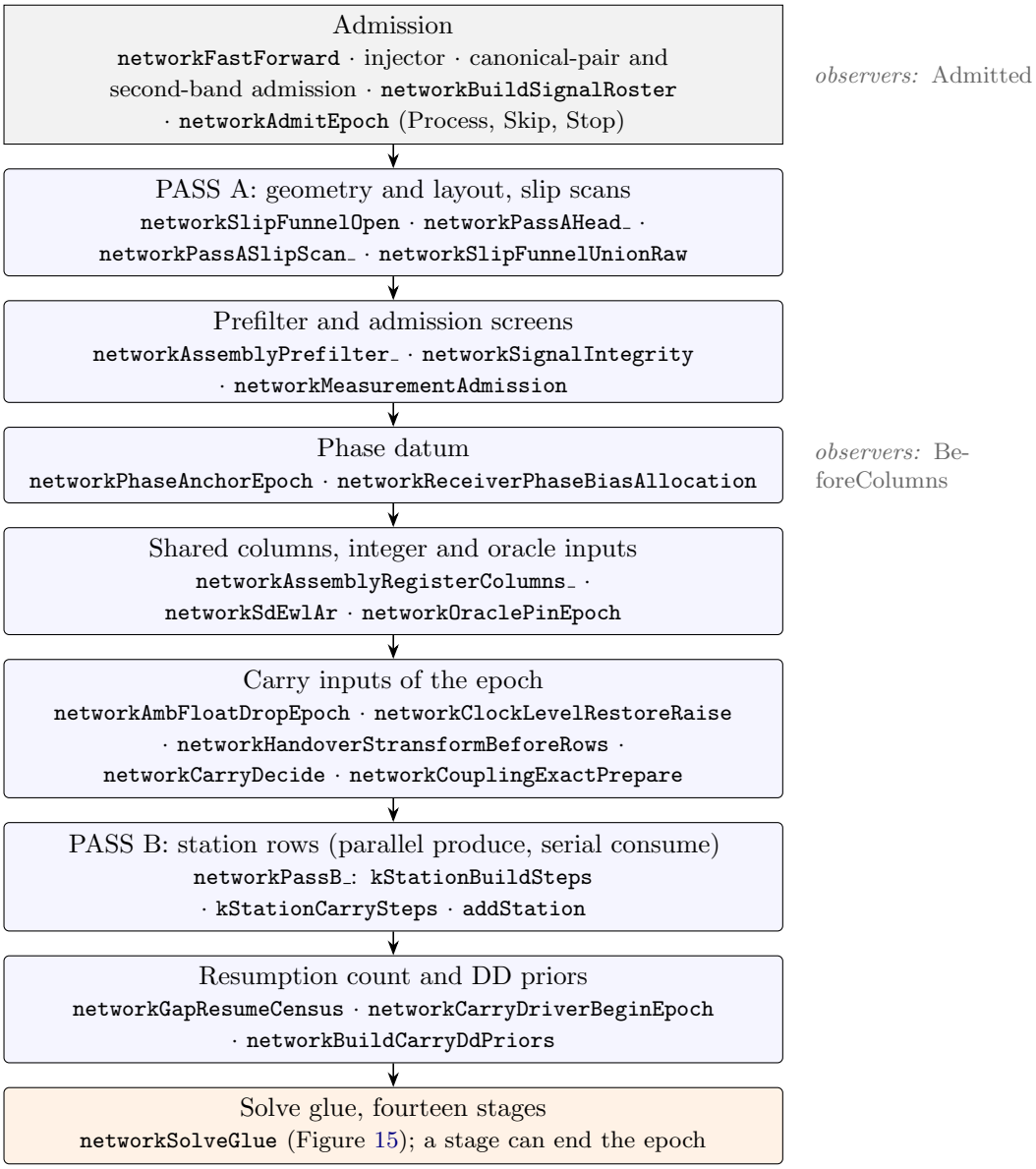


Figure 13: The epoch of `Application::networkProcessEpoch_` up to the solve. Each box is a seam of the form (`NetworkRun&`, `NetworkEpoch`) that owns no state; the epoch's values are grouped by the stage that writes them. The notes mark the two observer slots of this half. An observer is a measure-only instrument: it writes only its own counters and traces, and the table `kNetworkEpochObservers` (`network/driver/EpochObservers`) lists every observer with its slot and the reason for its place. Figure 14 continues the sequence.

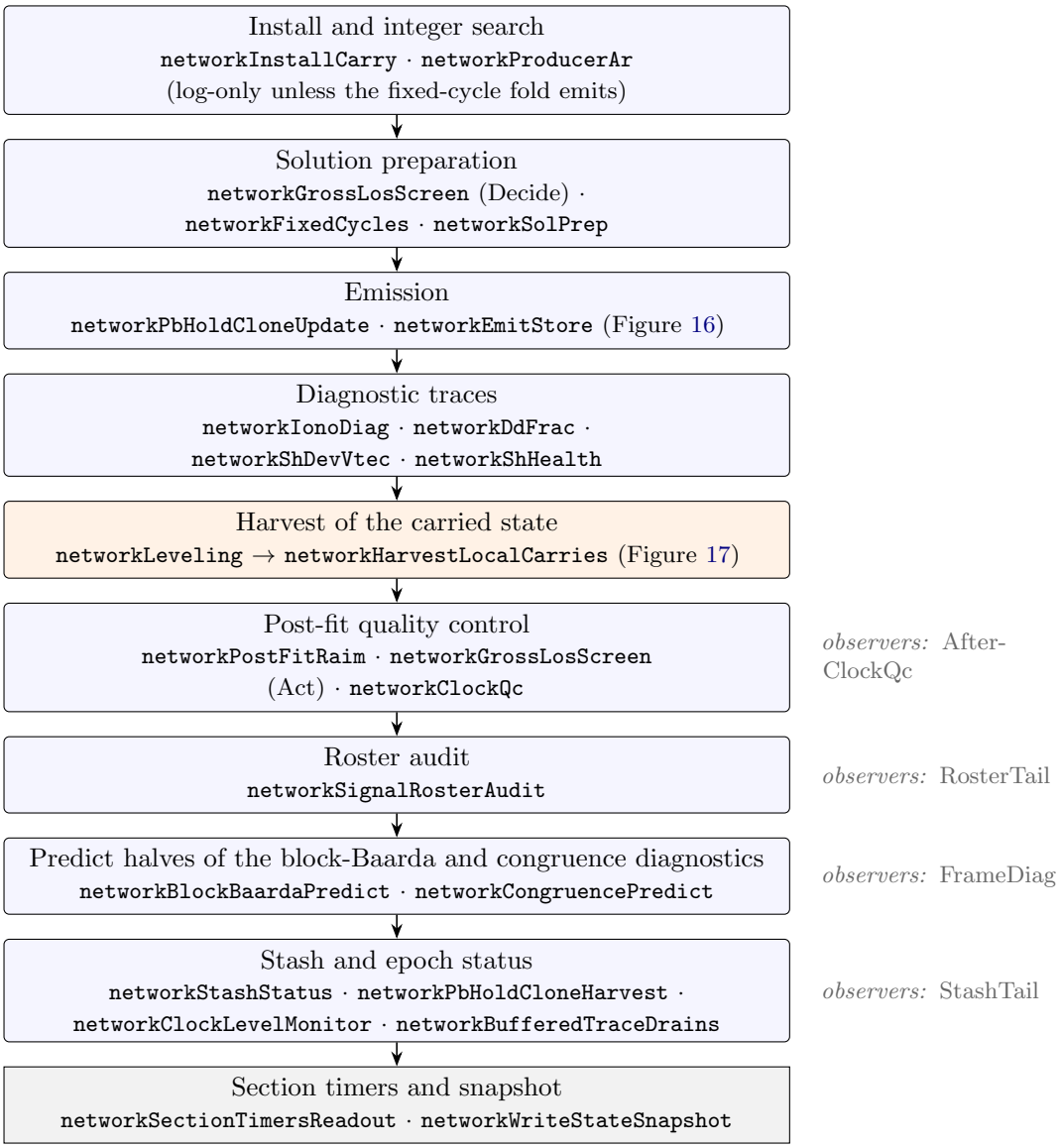


Figure 14: The epoch after the solve. The emission precedes the harvest, and the post-fit tiers follow the harvest because an erase of a carried entry is the refloat: it must act on the entry the harvest wrote. The notes name the four remaining observer slots, so the six slots of the observer table are, in spine order, *Admitted*, *BeforeColumns*, *AfterClockQc*, *RosterTail*, *FrameDiag* and *StashTail*. After the solve the only epoch values a seam writes are the hand-off to the post-fit tiers, the prepared solution and the epoch's events.

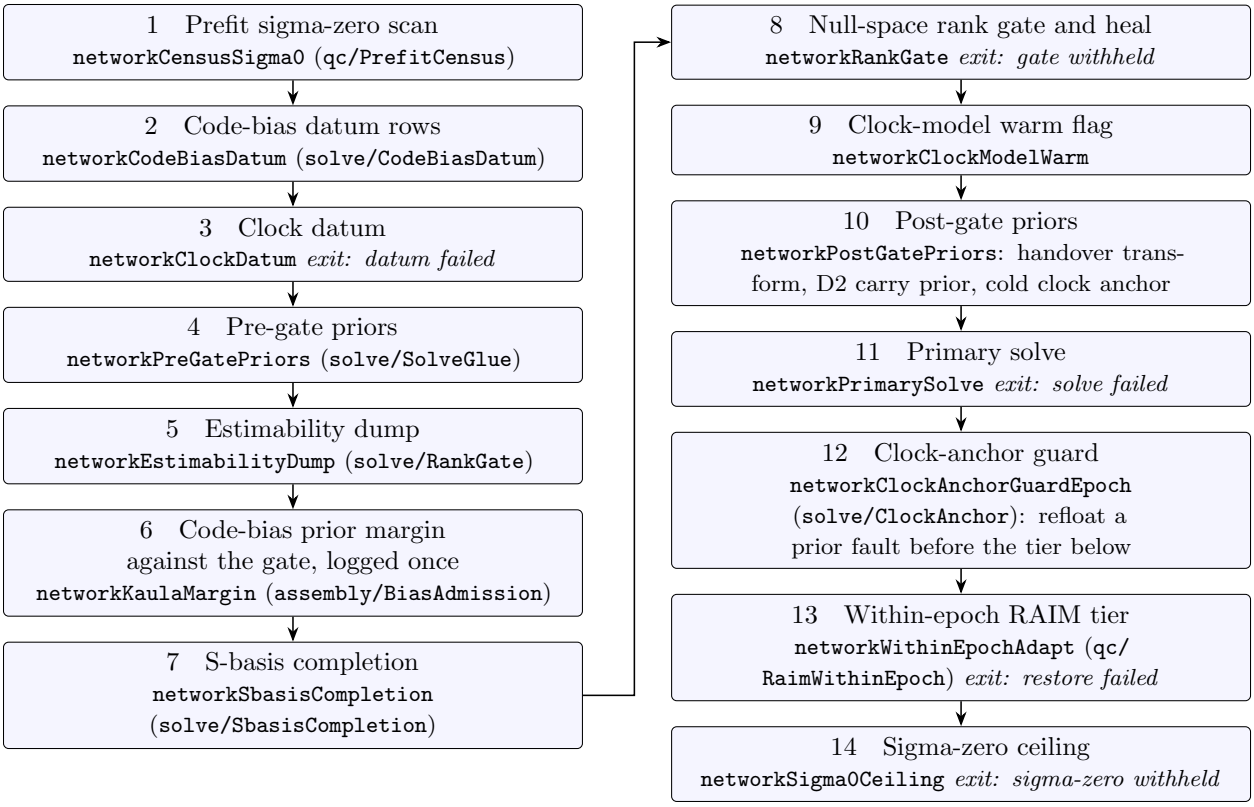


Figure 15: The fourteen stages of `networkSolveGlue` (`network/solve/SolveGlue`), in the order the estimator folds them: the order is load-bearing because the estimator accumulates the folds in this order and each stage reads what an earlier one wrote. Everything the rank gate must see is folded before stage 8; the gate, the dump and the heal read one tolerance. Stage 12 follows the solve and precedes stage 13, so that a prior fault is refloated before the tier can attribute it to data. Four stages can end the epoch, five ways, each a row of `kSolveSkips`. A new fold goes into stage 4 or stage 10 at its place in the order; a new way to end the epoch is a new row of the table and never an inline return. Unlabelled files are in `solve/`.

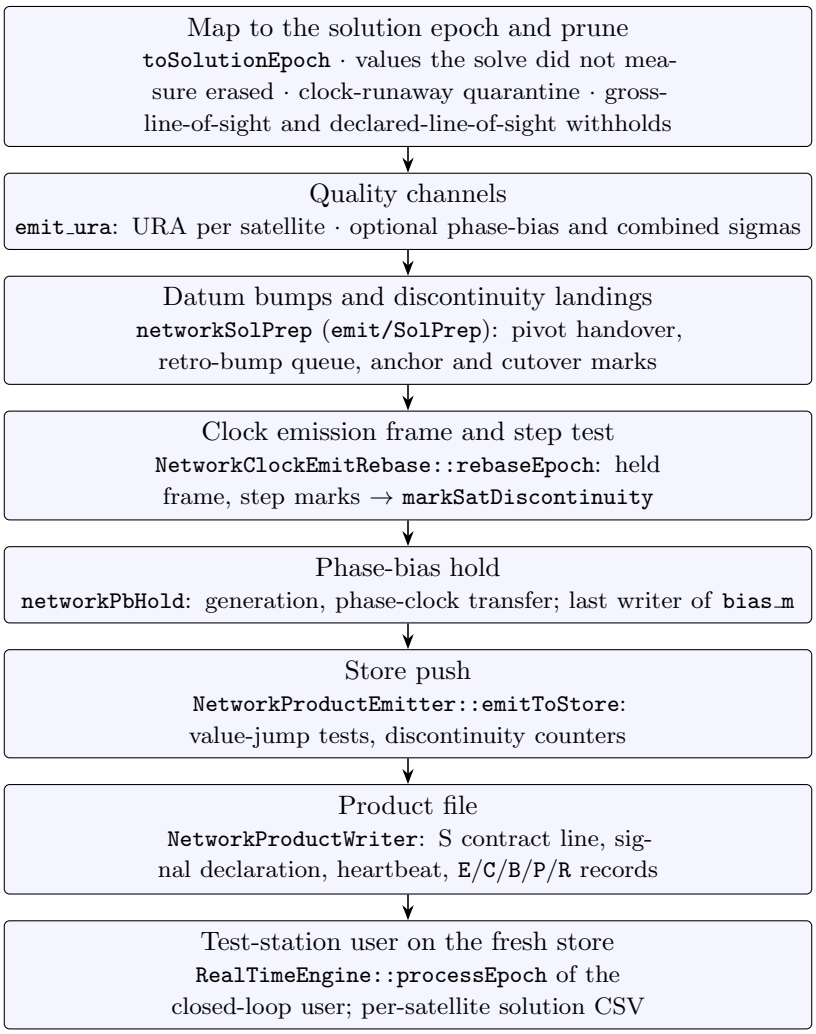


Figure 16: Emission, from the solved shared estimate to the product and the test station. The first three boxes are `networkSolPrep`, the last five `networkEmitStore` (the emit-layer frame re-base of the invariant carry is omitted, inactive in the reference configurations). Every step that changes an emitted value acts before `emitToStore` and before the file write, so the in-process store and a replay of the file carry the same values and reproduce the same discontinuity counters from the serialised records. The solve is unchanged by any step of this figure (Section 24).

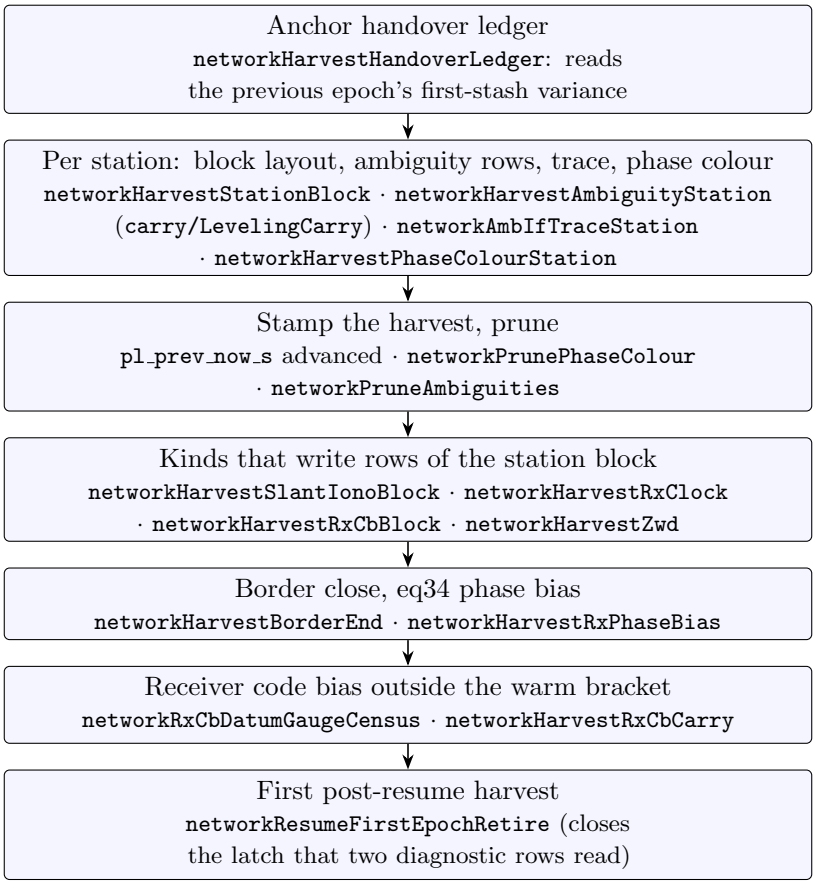


Figure 17: The post-solve harvest of the station-local carries, `networkHarvestLocalCarries` (`network/carry/LocalCarryHarvest`), called by `networkLeveling`. The orchestrator is a list of calls, one per carried kind, each in the file of its carry; the boxes from the second to the fifth run only with the carry warm. The order is fixed. The handover ledger reads the previous epoch's variance before the ambiguity stash rewrites it. Per station the layout, the ambiguity rows, the ionosphere-free trace and the phase colour come first, and the shared-history compression is sensitive to that order. The harvest clock advances after every station and before the prunes. Every writer of a row of the station block runs before the border is closed. The receiver code-bias carry runs outside the warm bracket because it must work on the invariant-carry lineage without phase leveling. RAIM and the gross line-of-sight screen run after the harvest because they set new marks. A carried column's regression goes through `networkHarvestColumn` (Section 23.2).

A Generated reference tables

The tables in this appendix are generated from the source tree by `scripts/gen_report_tables.py`. They are not maintained by hand: a renamed flag, a new cycle-slip detector, or a changed state-key delimiter appears here on the next run of the generator. *Known gap:* the feature-flag table de-duplicates on the C++ member name. A section master switch named `enable` in more than one config sub-struct therefore appears once, and several `network.*` master switches are absent. The table is indicative. The loaders `GnssConfigLoad.cpp` and `GnssConfigLoadNetwork.cpp` (`src/core/config/`) are authoritative, and the verdict on every key the network loader reads (armed, recommended, kept off, unvalidated) is recorded in `docs/CONFIG_promotion_ledger.json`. The fragments under `generated/` are git-ignored and rebuilt on demand (`build.ps1 -Regen` / `build.sh --regen`). `gen_report_tables.py --check` exits non-zero when a fragment changed; it rewrites the fragments before comparing, so after a failed check the regenerated tables are already in place.

A.1 Processing modes

Table 10: `ProcessingMode` – the estimation algorithm selector.

Value	Meaning
`PPP`	Precise Point Positioning (single receiver, external products).
`PPP_RTK`	PPP with RTK enhancements (single receiver, uncombined).
`BaselineSingle`	Double-differenced single baseline: one rover + one reference station.
`BaselineMulti`	Double-differenced multi-receiver network (star topology: N rovers against one reference).
`PPP_RTK_Network`	Minimum-constraint S-system network producer: joint multi-station estimator emitting satellite clock + code/phase bias (+ atmosphere) under a declared S-basis.
`PPP_RTK_User`	PPP-RTK consumer of network PPP_RTK_Network products (post-processing closed-loop counterpart): uncombined PPP-RTK + network SSR bias/atmosphere consume + S-basis cross-check.

A.2 Execution mode and data source

Table 11: `AppMode` – top-level post-processing vs. real-time dispatch.

Value	Meaning
`PostProcessing`	Batch processing from RINEX / product files.
`RealTime`	Streaming / live processing from RTCM data.

Table 12: `DataSource` – bundled fixtures vs. auto-download.

Value	Meaning
`Internal`	Use files under internal_data_root only.
`Production`	Auto-download from CDDIS into data_root.

A.3 Differencing regime

Table 13: `DifferencingMode` – the measurement differencing regime.

Value	Meaning
`None`	Raw observables (PPP / PPP-RTK without SD).
`BetweenSatellite`	Per-constellation single difference vs. pivot (PPP-RTK SD).
`DoubleDifference`	Between-receiver + between-satellite (baseline processing).

A.4 Config-to-differencing mapping

The differencing regime is resolved once per run by the factory below (verbatim from `MeasurementBuilder.cpp`):

```
DifferencingMode differencingModeFor(const GnssConfig& config) {
    // The network producer runs on undifferenced, uncombined rows and estimates the
    // satellite-side bias lumps explicitly, so it never applies receiver-side BSSD,
    // even when a between_satellite_sd toggle is present in its config.
    if (config.getProcessingMode() == ProcessingMode::PPP_RTK_Network)
                                        return DifferencingMode::None;
    if (config.isBaselineMode())        return DifferencingMode::DoubleDifference;
    if (config.enableBetweenSatelliteSd()) return DifferencingMode::BetweenSatellite;
    return DifferencingMode::None;
}
```

A.5 Feature toggles

These are the loop- and state-vector-level switches.

Table 14: Top-level boolean feature toggles, parsed from the `jval<bool>` calls in `GnssConfigLoad.cpp` and `GnssConfigLoadNetwork.cpp`. The legacy alias is the pre-refactor flat key still accepted as a fallback.

JSON path	Default	Legacy alias
`estimation.accumulate_phase_windup`	off	`accumulate_phase_windup`
`estimation.adaptive_iono_process_noise.trace`	off	`adaptive_iono_q_trace`
`estimation.aiding.baro`	off	`aiding_baro`

JSON path	Default	Legacy alias
estimation.aiding.nhc	off	aiding_nhc
estimation.aiding.vision	off	aiding_vision
estimation.aiding.wheel	off	aiding_wheel
estimation.aiding.zupt	off	aiding_zupt
estimation.ambiguity_resolution.enable_partial_ar	off	ar_enable_partial_ar
estimation.ambiguity_resolution.nl_iono_free_row	off	ar_nl_iono_free_row
estimation.ambiguity_resolution.wl_only	off	ar_wl_only
estimation.ambiguity_resolution.wl_only_ffr	on	ar_wl_only_ffr
estimation.between_satellite_sd.enable_glonass	off	enable_glonass_bssd
estimation.between_satellite_sd.enabled	off	enable_between_satellite_sd
estimation.cycle_slip.ewl	off	cycle_slip_ewl
estimation.cycle_slip.gf	off	cycle_slip_gf
estimation.cycle_slip.graphic	off	cycle_slip_graphic
estimation.cycle_slip.graphic_trend_centred	off	cycle_slip_graphic_trend_centred
estimation.cycle_slip.mw	on	cycle_slip_mw
estimation.cycle_slip.turbo_edit	off	cycle_slip_turbo_edit
estimation.cycle_slip.turbo_edit_bridge_lli	off	cycle_slip_turbo_edit_bridge_lli
estimation.cycle_slip.turbo_edit_mw_only_veto	on	cycle_slip_turbo_edit_mw_only_veto
estimation.defer_until_corrections_ready.drop_broadcast_grade	off	defer_until_corrections_ready_drop_broadcast_grade
estimation.defer_until_corrections_ready.require_all_above_mask	off	defer_until_corrections_ready_require_all_above_mask
estimation.enable_cio_frame	off	enable_cio_frame
estimation.enable_data_snooping	off	enable_data_snooping
estimation.enable_full_r_kernel	off	enable_full_r_kernel
estimation.enable_huber_innovation	off	enable_huber_innovation
estimation.enable_huber_r_inflation	on	enable_huber_r_inflation
estimation.enable_if_slip_detector	off	enable_if_slip_detector
estimation.enable_iono_rate	off	enable_iono_rate
estimation.enable_subcycle_slip_suppression	on	enable_subcycle_slip_suppression
estimation.enable_sun_frame_correction	off	enable_sun_frame_correction
estimation.enable_variance_factor	off	enable_variance_factor
estimation.estimate_ifb	off	estimate_ifb
estimation.estimate_phase_clock	off	estimate_phase_clock

JSON path	Default	Legacy alias
estimation.estimate_sat_biases	off	estimate_sat_biases
estimation.estimate_sat_clocks	off	estimate_sat_clocks
estimation.estimate_sat_phase_biases	off	estimate_sat_phase_biases
estimation.estimate_tropo_gradients	off	estimate_tropo_gradients
estimation.estimate_velocity	off	estimate_velocity
estimation.global_vtec_sh.rep_error_d5_keep	off	global_vtec_sh_rep_error_d5_keep
estimation.global_vtec_sh.rep_error_demean	off	global_vtec_sh_rep_error_demean
estimation.global_vtec_sh.sun_fixed	on	global_vtec_sh_sun_fixed
estimation.glonass_ifb.enable	off	enable_glonass_ifb
estimation.lock_signals_to_clock_ref	off	lock_signals_to_clock_ref
estimation.phase_windup_gap_bridge	off	phase_windup_gap_bridge
estimation.promote_sat_code_bias	off	promote_sat_code_bias
estimation.require_bia_for_signals	off	require_bia_for_signals
estimation.residual_history.enabled	off	residual_history_enabled
estimation.trace_windup_census	off	trace_windup_census
network.adm_revet	off	adm_revet
network.adm_revet_census	on	adm_revet_census
network.adm_revet_hold	off	adm_revet_hold
network.adm_screen	off	adm_screen
network.adm_trace	off	adm_trace
network.amb_float_declared_handover	off	amb_float_declared_handover
network.amb_float_drop_decouple_always	off	amb_float_drop_decouple_always
network.ambiguity_resolution.block_search	off	network_ar_block_search
network.ambiguity_resolution.cycle_basis	off	network_ar_cycle_basis
network.ambiguity_resolution.cycle_basis_sticky	on	network_ar_cycle_basis_sticky
network.ambiguity_resolution.decorrelated_partial	off	network_ar_decorrelated_partial
network.ambiguity_resolution.enable_partial_ar	on	network_ar_enable_partial_ar
network.ambiguity_resolution.trace_lattice	off	network_ar_trace_lattice
network.ambiguity_resolution.tree_basis	off	network_ar_tree_basis
network.block_baarda	off	block_baarda
network.block_baarda_corroborate	off	block_baarda_corroborate
network.block_baarda_estimable	off	block_baarda_estimable
network.canonical_pair_admission	off	canonical_pair_admission
network.carry_aware_covariance	off	network_carry_aware_covariance

JSON path	Default	Legacy alias
network.carry_conditional_coupling	off	carry_conditional_coupling
network.carry_coupling_bias_clone	off	carry_coupling_bias_clone
network.carry_coupling_clock_clone	off	carry_coupling_clock_clone
network.carry_coupling_departed_clone	off	carry_coupling_departed_clone
network.carry_coupling_exact_cross	off	carry_coupling_exact_cross
network.carry_coupling_exact_update	off	carry_coupling_exact_update
network.carry_coupling_gauge_project	off	carry_coupling_gauge_project
network.carry_coupling_reinit_sources	off	carry_coupling_reinit_sources
network.carry_coupling_shared_history	off	carry_coupling_shared_history
network.carry_coupling_time_update	off	carry_coupling_time_update
network.carry_invariant	off	network_carry_invariant
network.carry_invariant_emit_rebase	on	network_carry_invariant_emit_rebase
network.carry_invariant_innov_calibrate	off	network_carry_invariant_innov_calibrate
network.carry_invariant_innov_carried_only	off	network_carry_invariant_innov_carried_only
network.carry_invariant_innov_corroborate	off	network_carry_invariant_innov_corroborate
network.carry_invariant_innov_gate	off	network_carry_invariant_innov_gate
network.carry_invariant_innov_retap_only	off	network_carry_invariant_innov_retap_only
network.carry_invariant_pd_safe	off	network_carry_invariant_pd_safe
network.carry_iono_in_block	off	carry_iono_in_block
network.carry_rebase_complete	off	network_carry_rebase_complete
network.carry_rx_clock_in_block	off	carry_rx_clock_in_block
network.carry_rx_clock_isb	on	carry_rx_clock_isb
network.carry_rx_code_bias	off	carry_rx_code_bias
network.carry_rx_code_bias_in_block	off	carry_rx_code_bias_in_block
network.carry_vce	off	carry_vce
network.carry_vce_apply	off	carry_vce_apply
network.carry_zwd	off	carry_zwd
network.carry_zwd_in_block	off	carry_zwd_in_block
network.cb_admission	off	cb_admission
network.cb_excursion_reinit	off	cb_excursion_reinit
network.cb_excursion_reinit_all_sats	off	cb_excursion_reinit_all_sats
network.cb_kaula_prior	off	network_cb_kaula_prior

JSON path	Default	Legacy alias
network.clock_anchor_datum_keep	off	network_clock_anchor_datum_keep
network.clock_anchor_gauge_free	off	network_clock_anchor_gauge_free
network.clock_anchor_mark	on	network_clock_anchor_mark
network.clock_anchor_resolve	on	network_clock_anchor_resolve
network.clock_datum_broadcast_anchored	off	network_clock_datum_broadcast_anchored
network.clock_datum_sum_to_zero	off	network_clock_datum_sum_to_zero
network.clock_disc_jump_test	off	network_clock_disc_jump_test
network.clock_disc_on_set_change	off	network_clock_disc_on_set_change
network.clock_level_bound_raise	off	network_clock_level_bound_raise
network.clock_level_monitor	off	network_clock_level_monitor
network.clock_level_restore_raise	off	clock_level_restore_raise
network.clock_model	off	network_clock_model
network.clock_monitor	off	network_clock_monitor
network.clock_monitor_couple_raim	off	network_clock_monitor_couple_raim
network.clock_pivot_frame_q	off	clock_pivot_frame_q
network.clock_runaway_check	off	network_clock_runaway_check
network.clock_temporal_anchor	off	network_clock_temporal_anchor
network.code_bias_disc_jump_test	off	network_code_bias_disc_jump_test
network.code_datum_family_difference	off	network_code_datum_family_difference
network.code_datum_pair_continuity	off	network_code_datum_pair_continuity
network.code_datum_time_linked	off	network_code_datum_time_linked
network.cold_clock_seed	off	cold_clock_seed
network.congruence_screen	off	congruence_screen
network.consume_phase_bias_sigma	off	network_consume_phase_bias_sigma
network.consume_ura	off	network_consume_ura
network.ctu_switch_on_resume	off	ctu_switch_on_resume
network.cycle_slip.anchor_slip_handover	off	network_cycle_slip_anchor_slip_handover

JSON path	Default	Legacy alias
network.cycle_slip.common_band_ref_arc	off	network_cycle_slip_common_band_ref_arc
network.cycle_slip.common_band_scope	off	network_cycle_slip_common_band_scope
network.cycle_slip.datum_station_detect	off	network_cycle_slip_datum_station_detect
network.cycle_slip.gap_instrument	off	network_cycle_slip_gap_instrument
network.cycle_slip.gap_reentry_coresumed_share	off	network_cycle_slip_gap_reentry_coresumed_share
network.cycle_slip.gap_reentry_per_signal	off	network_cycle_slip_gap_reentry_per_signal
network.cycle_slip.gap_reentry_per_signal_unmeasurable	on	network_cycle_slip_gap_reentry_per_signal_unmeasurable
network.cycle_slip.gap_reentry_relevel	off	network_cycle_slip_gap_reentry_relevel
network.cycle_slip.gap_reentry_relevel_lli	on	network_cycle_slip_gap_reentry_relevel_lli
network.cycle_slip.gap_reentry_relevel_unmeasurable	off	network_cycle_slip_gap_reentry_relevel_unmeasurable
network.cycle_slip.gf_arc_jump	off	network_cycle_slip_gf_arc_jump
network.cycle_slip.if_phase_td	off	network_cycle_slip_if_phase_td
network.cycle_slip.if_phase_td_ref_arm_all	off	network_cycle_slip_if_phase_td_ref_arm_all
network.cycle_slip.if_phase_td_ref_elev_norm	off	network_cycle_slip_if_phase_td_ref_elev_norm
network.cycle_slip.if_phase_td_ref_gate	off	network_cycle_slip_if_phase_td_ref_gate
network.cycle_slip.if_phase_td_ref_warmup_abstain	off	network_cycle_slip_if_phase_td_ref_warmup_abstain
network.cycle_slip.if_phase_td_reseed_on_flag	off	network_cycle_slip_if_phase_td_reseed_on_flag
network.cycle_slip.mw_detect	off	network_cycle_slip_mw_detect
network.cycle_slip.mw_evidence_refloat	off	network_cycle_slip_mw_evidence_refloat
network.cycle_slip.mw_lli_corroborate	off	network_cycle_slip_mw_lli_corroborate
network.cycle_slip.mw_reflag_skip	off	network_cycle_slip_mw_reflag_skip

JSON path	Default	Legacy alias
network.cycle_slip.oracle_relevel	off	network_cycle_slip_oracle_relevel
network.cycle_slip.relevel_bridge_gf	off	network_cycle_slip_relevel_bridge_gf
network.cycle_slip.relevel_delta_bridge	off	network_cycle_slip_relevel_delta_bridge
network.cycle_slip.relevel_free_epoch	off	network_cycle_slip_relevel_free_epoch
network.cycle_slip.tracker_modeled_first	off	network_cycle_slip_tracker_modeled_first
network.cycle_slip.turbo_edit	off	network_cycle_slip_turbo_edit
network.cycle_slip.windup_restart_refloat	off	network_cycle_slip_windup_restart_refloat
network.datum_graph_census	off	datum_graph_census
network.emit_clock_disc	off	network_emit_clock_disc
network.emit_clock_frame_hold	off	network_emit_clock_frame_hold
network.emit_combined_sigma	off	network_emit_combined_sigma
network.emit_epoch_status_record	off	network_emit_epoch_status_record
network.emit_generation_id	off	network_emit_generation_id
network.emit_generation_time	off	network_emit_generation_time
network.emit_pb_hold	off	emit_pb_hold
network.emit_pb_hold_phase_clock	off	emit_pb_hold_phase_clock
network.emit_pb_hold_smoothed	off	emit_pb_hold_smoothed
network.emit_phase_bias_sigma	off	network_emit_phase_bias_sigma
network.emit_ura	off	network_emit_ura
network.estimate_glonass	off	estimate_glonass
network.estimate_glonass	off	estimate_glonass
network.estimate_iono_sh	off	estimate_iono_sh
network.estimate_rx_extra_band_code_delay	off	estimate_rx_extra_band_code_delay
network.estimate_sat_code_bias	off	estimate_sat_code_bias
network.exclude_unmodeled_antenna_stations	off	network_exclude_unmodeled_antenna_stations
network.gross_los_bound	off	gross_los_bound
network.gross_los_emit_exclude	off	gross_los_emit_exclude
network.gross_los_sat_standdown	off	gross_los_sat_standdown
network.gross_los_sat_wire_withhold	on	gross_los_sat_wire_withhold
network.gross_los_screen	off	gross_los_screen
network.gross_los_station_tier	off	gross_los_station_tier

JSON path	Default	Legacy alias
network.h_sigma0_prefit_census	off	network_h_sigma0_prefit_census
network.handover_stransform	off	network_handover_stransform
network.handover_stransform_before_rows	off	network_handover_stransform_before_rows
network.handover_stransform_dense	off	network_handover_stransform_dense
network.iono_supply_formal_sigma	off	iono_supply_formal_sigma
network.ledger_anchor	off	network_ledger_anchor
network.ledger_emit	off	network_ledger_emit
network.ledger_reentry	off	network_ledger_reentry
network.ledger_screens	off	network_ledger_screens
network.level_authority_csgf	off	network_level_authority_csgf
network.level_authority_los_dump	off	network_level_authority_los_dump
network.mark_refsat_reset_disc	off	network_mark_refsat_reset_disc
network.mdb	off	mdb
network.n3c_band_signal	off	network_n3c_band_signal
network.n3c_bia_pin_pair	off	network_n3c_bia_pin_pair
network.n3c_canonical_pair_traits	off	network_n3c_canonical_pair_traits
network.n3c_channel_family	off	network_n3c_channel_family
network.n3c_ledger_lifecycle	off	network_n3c_ledger_lifecycle
network.n3c_wire_per_signal	off	network_n3c_wire_per_signal
network.parallel_carry	off	parallel_carry
network.parallel_seam	off	parallel_seam
network.parallel_stations	on	parallel_stations
network.per_const_withhold	off	network_per_const_withhold
network.phase_anchor_carry_reref	off	network_phase_anchor_carry_reref
network.phase_colour_state	off	phase_colour_state
network.phase_datum_candidacy_satellite	on	network_phase_datum_candidacy_satellite
network.phase_datum_candidacy_usable	off	network_phase_datum_candidacy_usable
network.phase_datum_eq34	off	network_phase_datum_eq34
network.phase_datum_eq34_carry_shift	off	network_phase_datum_eq34_carry_shift
network.phase_datum_eq34_extend_only	off	network_phase_datum_eq34_extend_only

JSON path	Default	Legacy alias
network.phase_datum_eq34_longevity_election	off	network_phase_datum_eq34_longevity_election
network.phase_datum_eq34_pin_round	on	network_phase_datum_eq34_pin_round
network.phase_datum_eq34_seed_round	off	network_phase_datum_eq34_seed_round
network.phase_datum_eq34_seed_round_rehost_frame	off	network_phase_datum_eq34_seed_round_rehost_frame
network.phase_datum_eq34_slip_aware_election	off	network_phase_datum_eq34_slip_aware_election
network.phase_datum_gate_drop_erase	off	network_phase_datum_gate_drop_erase
network.phase_datum_handover_continuous	off	network_phase_datum_handover_continuous
network.phase_datum_held_basis	off	network_phase_datum_held_basis
network.phase_datum_held_cold_release	off	network_phase_datum_held_cold_release
network.phase_datum_held_mark_cold	off	network_phase_datum_held_mark_cold
network.phase_datum_prioritized	off	network_phase_datum_prioritized
network.phase_datum_sticky	on	network_phase_datum_sticky
network.phase_datum_time_linked	off	network_phase_datum_time_linked
network.phase_leveling	off	phase_leveling
network.phase_leveling_local_block	off	phase_leveling_local_block
network.phase_visible_monitor	off	phase_visible_monitor
network.pivot_vanish_return_signal	off	network_pivot_vanish_return_signal
network.prior_residual_census	off	prior_residual_census
network.prior_residual_corroborate	off	prior_residual_corroborate
network.prior_residual_global_sigma0	off	prior_residual_global_sigma0
network.product_flush_per_epoch	off	product_flush_per_epoch
network.raim	off	network_raim
network.raim_attribution	off	network_raim_attribution
network.raim_carry_invariant	off	network_raim_carry_invariant
network.raim_pool_data_rows_only	off	network_raim_pool_data_rows_only
network.raim_rate_hygiene	off	network_raim_rate_hygiene
network.raim_redundancy_normalize	off	network_raim_redundancy_normalize
network.raim_refloat_maturity_gate	off	network_raim_refloat_maturity_gate

JSON path	Default	Legacy alias
network.raim_station_factor	off	network_raim_station_factor
network.raim_surgical_station	on	network_raim_surgical_station
network.raim_we_code_iono	off	network_raim_we_code_iono
network.raim_withhold_free_erase	off	network_raim_withhold_free_erase
network.raim_within_epoch	off	network_raim_within_epoch
network.readmit_seed_code_bias	on	readmit_seed_code_bias
network.readmit_seed_mirrors	off	readmit_seed_mirrors
network.replay_allow_undeclared_wire	off	replay_allow_undeclared_wire
network.require_signal_integer_for_ar	off	network_require_signal_integer_for_ar
network.resume_border_restore	off	resume_border_restore
network.resume_coupling_restore	off	resume_coupling_restore
network.resume_datum_restore	off	resume_datum_restore
network.resume_emit_restore	off	resume_emit_restore
network.resume_frame_restore	off	resume_frame_restore
network.resume_pb_seam_mark	off	resume_pb_seam_mark
network.resume_rxcb_lin_restore	off	resume_rxcb_lin_restore
network.resume_signal_canon_restore	off	resume_signal_canon_restore
network.resume_station_restore	off	resume_station_restore
network.resume_station_restore_rxcb	on	resume_station_restore_rxcb
network.rhs_attrib_on_trip	off	rhs_attrib_on_trip
network.rhs_attrib_on_trip	off	rhs_attrib_on_trip
network.rhs_attrib_on_trip_prev	on	rhs_attrib_on_trip_prev
network.roster_tracker	off	network_roster_tracker
network.rx_code_bias_datum	off	rx_code_bias_datum
network.sat_gf_datum_time_linked	off	sat_gf_datum_time_linked
network.sat_gf_extra_band_free	off	sat_gf_extra_band_free
network.sat_isc_state	off	sat_isc_state
network.sbasis_complete	off	network_sbasis_complete
network.scoped_handover_response	off	network_scoped_handover_response
network.sd_ewl.fix_enable	off	network_sd_ewl_fix_enable
network.second_band_admission	off	second_band_admission
network.second_code_channel	off	second_code_channel
network.second_phase_channel	off	second_phase_channel
network.serialize_disc_counters	off	network_serialize_disc_counters
network.service_mode	off	network_service_mode
network.shared_carry_full_cov	off	shared_carry_full_cov

JSON path	Default	Legacy alias
network.signal_integrity.census	off	network_signal_integrity_census
network.signal_integrity.refloat_on_regain	on	network_signal_integrity_refloat_on_regain
network.signal_integrity.resume_warmup_vet	on	network_signal_integrity_resume_warmup_vet
network.signal_integrity.withhold_instead	off	network_signal_integrity_withhold_instead
network.signal_roster	off	network_signal_roster
network.sparse_station_normals	on	sparse_station_normals
network.state_snapshot_carry	off	state_snapshot_carry
network.state_snapshot_carry_diagonal	off	state_snapshot_carry_diagonal
network.state_snapshot_carry_lin	off	state_snapshot_carry_lin
network.state_snapshot_coupling_capture	off	state_snapshot_coupling_capture
network.state_snapshot_datum_capture	off	state_snapshot_datum_capture
network.state_snapshot_emit_capture	off	state_snapshot_emit_capture
network.state_snapshot_frame_capture	off	state_snapshot_frame_capture
network.state_snapshot_keep_history	off	state_snapshot_keep_history
network.state_snapshot_require	off	state_snapshot_require
network.state_snapshot_rxcb_lin_capture	off	state_snapshot_rxcb_lin_capture
network.state_snapshot_signal_capture	off	state_snapshot_signal_capture
network.state_snapshot_station_capture	off	state_snapshot_station_capture
network.station_marker_to_arp	off	network_station_marker_to_arp
network.station_pd_pin_aware	off	network_station_pd_pin_aware
network.station_prior_before_pd	off	station_prior_before_pd
network.sticky_pivot	off	network_sticky_pivot
network.streaming.msm_lli_from_locktime	off	network_streaming_msm_lli_from_locktime
network.streaming.require_datum_station	on	network_streaming_require_datum_station
network.trace_amb_if	off	trace_amb_if
network.trace_amb_state	off	trace_amb_state
network.trace_anchor_info	off	trace_anchor_info
network.trace_clock_full	off	trace_clock_full
network.trace_consumer_sat	off	trace_consumer_sat

JSON path	Default	Legacy alias
network.trace_dd_fractional	off	trace_dd_fractional
network.trace_epoch_status	off	trace_epoch_status
network.trace_estimability	off	trace_estimability
network.trace_estimability_deficit_only	off	trace_estimability_deficit_only
network.trace_events	off	trace_events
network.trace_flag_rates	off	trace_flag_rates
network.trace_ionosphere	off	trace_ionosphere
network.trace_phase_anchor	off	trace_phase_anchor
network.trace_phase_colour	off	trace_phase_colour
network.trace_postfit_keyed	off	trace_postfit_keyed
network.trace_postfit_rows	off	trace_postfit_rows
network.trace_postfit_sigma0	off	trace_postfit_sigma0
network.trace_prefit_census	off	trace_prefit_census
network.trace_raim_redundancy	off	trace_raim_redundancy
network.trace_rx_states	off	trace_rx_states
network.trace_sat_bias_states	off	trace_sat_bias_states
network.trace_section_timers	off	trace_section_timers
network.trace_sh_spectrum	off	trace_sh_spectrum
network.trace_slip_events	off	trace_slip_events
network.trace_slip_raw	off	trace_slip_raw
network.trace_ssr_consumer	off	trace_ssr_consumer
network.trace_station_block	off	trace_station_block
network.trace_station_pd_null	off	trace_station_pd_null
network.trace_station_residuals	off	trace_station_residuals
network.trace_stations	on	trace_stations
network.trace_topology	off	trace_topology
network.trace_zwd	off	trace_zwd
network.ura_retro_bump	off	network_ura_retro_bump
network.use_sif_coordinates	off	use_sif_coordinates
network.write_run_manifest	off	write_run_manifest
network.zwd_seam_reseed	off	zwd_seam_reseed
processing.amb_level_check	off	amb_level_check
processing.ar_ignore_mw_wl	off	ar_ignore_mw_wl
processing.auto_report	off	auto_report
processing.canonical_signal_code_follows_phase	on	canonical_signal_code_follows_phase
processing.enable_neq_export	off	enable_neq_export
processing.enable_noncausal_slip_bridge	off	enable_noncausal_slip_bridge
processing.enable_noon_turn_attitude	off	enable_noon_turn_attitude
processing.enable_obs_skip_to_latest	off	enable_obs_skip_to_latest
processing.enable_obs_ssr_sync	off	enable_obs_ssr_sync
processing.enable_orbit_normal_attitude	off	enable_orbit_normal_attitude
processing.enable_ssr_bias_max_age	off	enable_ssr_bias_max_age

JSON path	Default	Legacy alias
processing.enable_station_info	off	enable_station_info
processing.enable_trig_sp3_interp	off	enable_trig_sp3_interp
processing.ephemeris_jump_reseed	off	ephemeris_jump_reseed
processing.exclude_eclipsing_satellites	off	exclude_eclipsing_satellites
processing.inhouse_bias_estimation	off	inhouse_bias_estimation
processing.innovation_screen	off	innovation_screen
processing.network_consumer.bias	on	network_consumer_bias
processing.network_consumer.clock	on	network_consumer_clock
processing.network_consumer.enable	off	network_consumer_enable
processing.network_consumer.undifferenced	on	network_consumer_undifferenced
processing.relinearize_geometry	off	relinearize_geometry
processing.report_async	off	report_async
processing.rinex_skip_zero_values	off	rinex_skip_zero_values
processing.run_smoother	off	run_smoother
processing.ssr_bias_band_fallback	off	ssr_bias_band_fallback
processing.ssr_level_step_inflate	off	ssr_level_step_inflate
processing.ssr_pb_hold_presence_flip	off	ssr_pb_hold_presence_flip
processing.ssr_phase_bias_disc_carry	off	ssr_phase_bias_disc_carry
processing.ssr_phase_bias_step_translate	off	ssr_phase_bias_step_translate
processing.uncombined_enabled	off	uncombined_enabled
receiver.enable_antenna_orientation	off	enable_antenna_orientation
receiver.enable_antenna_substitution	off	enable_antenna_substitution
receiver.enable_itrf_velocity	off	enable_itrf_velocity

A.6 Model-correction switches

Table 15: Physical model on/off switches (ModelCorrections). These gate the correction terms applied to the undifferenced observation model.

Field	Default	Effect
enable_troposphere	on	Saastamoinen ZHD + GMF dry mapping (slant hydrostatic delay).

Field	Default	Effect
enable_klobuchar_ionosphere	off	Klobuchar ionospheric delay model (single-frequency fallback only). IF combinations and DD short-baselines cancel ionosphere geometrically; PPP-RTK uncombined estimates per-satellite slant iono states. In all three of those configs the Klobuchar model is a no-op, but constructing the Ionosphere instance still pays for an extra Model::Geometry load and the Klobuchar coefficient parse. Default off; intended only for single-frequency PPP runs that need an a-priori ionosphere.
enable_higher_order_ionosphere	off	Apply second-order ($B \cdot \cos(\theta) \cdot STEC/f^3$) and third-order ($\eta \cdot STEC^2/f^4$) ionospheric delays as a per-signal observation-level correction, evaluated at the GIM-derived IPP using a centered tilted dipole geomagnetic field. Requires a loaded IONEX GIM as the absolute-STEC source; the engine auto-loads the GIM when this flag is on even if estimation.ionosphere_weighting.enabled is false. No-op when no GIM resolves (warning logged at startup). Magnitudes: I_2 ~1–2 cm at L1 during high TEC, I_3 ~mm; the IF combination only partially cancels these, so the largest visible benefit is in standard PPP (IF). For PPP-RTK uncombined and short-baseline DD the higher-order term mostly cancels in the SD / DD differencing, so this flag is closer to a consistency knob than a bias remover. Default off.
enable_relativistic_correction	on	Periodic relativistic clock correction (-2 r·v / c^2).
enable_shapiro_delay	on	Shapiro (gravitational path) delay.
enable_solid_earth_tide	on	IERS 2010 solid Earth tide displacement.
enable_ocean_tide_loading	on	Ocean tidal loading displacement (BLQ).
otl_from_sif	off	Take the ocean tidal loading coefficients from the Station Information File registry (three components, FES2014b) without arming processing.enable_station_info, which would also replace the a-priori position and the ocean pole tide source. The registry is loaded from files.station_info_path(s) or ¡data¿/static/sif/. A station the registry does not resolve falls back to BLQ, then EOT20 (radial only), then zero, exactly as before. The SIF east and north columns are read as the BLQ west and south rows (the M-PAGES reading, checked against HARDISP), not as true east and north. Only used when enable_ocean_tide_loading is on. Default off keeps runs byte-identical.

Field	Default	Effect
otl_nodal_corrections	off	Apply the 18.6-year nodal amplitude factors f and phase corrections u to the 11 loading constituents, and take the Greenwich argument from UTC instead of GPS time (a 0.15 degree shift on M2). Sub-millimetre to about 1 mm at ocean sites; the IERS HARDISP reference gets the same effect from its admittance side lines. Only used when enable_ocean_tide_loading is on. Default off keeps runs byte-identical.
otl_remove_diurnal_translation	off	Remove the net translation (a uniform ECEF displacement) of the SIF diurnal loading field, K1 O1 P1 Q1, from the coefficients of each station served by otl_from_sif. The SIF loading is relative to the centre of mass; on consumer replays of CODE-based network products the translation is not seen in the data (north-component scale -0.11, 95 % interval -0.75 to +0.27, against 1.19 for the rest of the model, 13 sites in 11 six-hour windows), and it carries 5.7 of the 8.0 mm of north loading at WTZZ00DEU in the 01 to 06 UT window of 2024-129. The table holds for the IGS20 SIF registry (scripts/otl_compare/net_translation.py). Only used with otl_from_sif and enable_ocean_tide_loading. Default off keeps runs byte-identical.
enable_pole_tide	on	Pole tide displacement (from ERP): IERS 2010 Sec. 7.1.4 rotational deformation.
enable_ocean_pole_tide	off	Ocean pole tide load displacement (IERS 2010 Sec. 7.1.5 / Desai 2002), added on top of enable_pole_tide. Requires the opole load grid (files.opt_grid_path or ¡data¿/static/opole/). ˜0.1-2 mm; default off keeps runs byte-identical.
enable_atmospheric_loading	off	Atmospheric pressure loading (S1/S2 thermal-tide site displacement). Requires per-station S1/S2 load coefficients (files.atm_tide_grid_path or ¡data¿/static/apl/). Sun-synchronous, so it does not average out in static daily PPP. Default off keeps runs byte-identical.
enable_satellite_pco	on	Satellite antenna phase centre offset.
enable_satellite_pcv	on	Satellite antenna phase centre variation.
enable_receiver_pco	on	Receiver antenna phase centre offset.
enable_receiver_pcv	on	Receiver antenna phase centre variation.
enable_receiver_cross_system_atx_fallback	off	Receiver only: when the receiver ATX entry lacks a non-GPS frequency, substitute the nearest GPS band (E1-¿G01, E5a-¿G05, B1C-¿G01, B2a-¿G05, ...). Valid because the receiver phase pattern is frequency-dependent on shared hardware. Never applied to the satellite antenna (E01 and G01 are different spacecraft). Opt-in.

Field	Default	Effect
enable_atx_frequency_fallback	on	When the ATX entry does not have PCO/PCV for the requested frequency (common for L5 / E5* / B2a on older satellites or receiver antennas), look up the nearest available frequency on the same antenna and use its values instead of skipping the correction without notice. Cross-frequency PCO drift on a single physical antenna is typically ¡1 cm, better than the few-cm vertical residual that comes from skipping a frequency entirely. false restores the legacy "skip if missing" behaviour for A/B comparison.
require_atx_coverage	off	When true, drop observations whose (sat, signal) lacks complete ATX coverage on both the satellite and receiver entries, i.e., satellite PCO+PCV and receiver PCO+PCV must all resolve to non-empty entries for the row's exact frequency, without enable_atx_frequency_fallback substituting a neighbouring frequency. Default false (legacy behaviour: observations are kept and any missing PCO/PCV is zeroed without notice). True suits a run with 'frequency_mode: "all"' (legacy alias 'signal_mode') in which the ATX file serves as the authoritative whitelist, dropping rows that would otherwise be processed without proper antenna calibration.
require_atx_coverage_for_ar	off	When true, AR (WL constraints, LAMBDA NL search) skips (sat, signal) pairs lacking strict ATX coverage even though the underlying observations are still used in the float Kalman update. Softer counterpart to require_atx_coverage: keeps "all"-signal float estimation while restricting integer fixing to the ATX-curated subset. Default false (no AR-side gating).
restrict_ar_for_substituted_atx	off	When true, AR additionally skips (sat, signal) pairs whose ATX correction was computed via the frequency-fallback substitution chain (rather than an exact entry). Targets specifically the cases where the antenna model is approximate; signals lacking ATX entirely (no substitution applied) are still allowed. Usable alongside or instead of require_atx_coverage_for_ar, depending on which kind of imperfection is to be gated. Default false.
loosen_amb_q_for_substituted_atx	off	When true, ambiguity states whose underlying (sat, signal) had their ATX correction frequency-fallback-substituted use a looser process noise (ambiguity_q_substituted_atx) instead of the standard process_noise.ambiguity. Compensates for the small but non-zero modeling error introduced by the substituted PCV/PCO. Default false.

Field	Default	Effect
apply_phase_shifts	off	When true, RINEX phase observations have the receiver-applied "SYS / PHASE SHIFT" values subtracted at parse time, recovering the unshifted phase convention. Default false because most modern BIA products (CODE, CAS, GFZ) are calibrated against the receiver-shifted phases as recorded in RINEX, so leaving the shifts in place is the standard PPP convention. True only when the bias product is explicitly calibrated for the unshifted convention, or when running without external BIA and using in-house bias estimation that benefits from a uniform shift-free input.
enable_phase_windup	on	Carrier-phase wind-up correction.
enable_code_phase_bias	on	Code/phase bias corrections from BIA product files.
bias_coverage_requires_apply	off	When true, the "already corrected?" gates in
refine_emission_time	on	When true, Geometry::computeSatelliteGeometry runs a

A.7 Logging and trace channels

Table 16: Logging and CSV-trace channels (LoggingConfig). Each toggles one spdlog file logger or one per-epoch CSV trace.

Field	Default	Channel
enable_main_log	on	logs/main_log.log
enable_gnss_config	on	logs/gnss_config.log
enable_kalman_filter	on	logs/kalman_filter.log
enable_smoother	on	logs/smoother.log
enable_state_manager	on	logs/state_manager.log
enable_ppp_model	on	logs/ppp_model.log
enable_measurement_model	on	logs/measurement_model.log
enable_mw_slip_detector	on	logs/mw_slip_detector.log
enable_if_slip_detector	on	logs/if_slip_detector.log
enable_gf_slip_detector	on	logs/gf_slip_detector.log
enable_turbo_edit_resolver	on	logs/turbo_edit_resolver.log
enable_cycle_slip_manager	on	logs/cycle_slip_manager.log
enable_ambiguity_resolver	on	logs/ambiguity_resolver.log
enable_corrections	on	logs/corrections.log

Field	Default	Channel
enable_geometry	on	logs/geometry.log
enable_troposphere	on	logs/troposphere.log
enable_ionosphere	on	logs/ionosphere.log
enable_antenna	on	logs/antenna.log
enable_antenna_model	on	logs/antenna_model.log
enable_ocean_tide	on	logs/ocean_tide.log
enable_planetary_model	on	logs/planetary_model.log
enable_stochastic_model	on	logs/stochastic_model.log
enable_rinex_parser	on	logs/rinex_parser.log
enable_product_parser	on	logs/product_parser.log
enable_product_cache	on	logs/product_cache.log
enable_resolver	on	logs/resolver.log
enable_state_initializer	on	logs/state_initializer.log
enable_bssd_pivot_manager	on	logs/bssd_pivot_manager.log
enable_bssd_transform	on	logs/bssd_transform.log
enable_ppp_utils	on	logs/ppp_utils.log
enable_downloader	on	logs/downloader.log
enable_ntrip	on	logs/ntrip.log
enable_rt_engine	on	logs/rt_engine.log
enable_imu	on	logs/imu.log
enable_state_trace	on	logs/state_trace.csv
enable_ambiguity_trace	on	logs/ambiguity_trace.csv
enable_ionosphere_trace	on	logs/ionosphere_trace.csv
enable_sat_bias_trace	on	logs/sat_bias_trace.csv
enable_code_bias_trace	on	logs/code_bias_trace.csv (receiver DCB/IFB)
enable_sat_status_trace	on	logs/satellite_status_trace.csv
enable_dop_trace	on	logs/dop_trace.csv
enable_residual_trace	on	logs/measurement_residuals.csv
enable_wls_trace	on	logs/wls_trace.csv
enable_smoother_trace	on	logs/smoother_trace.csv
enable_mw_slip_trace	on	logs/mwwl_trace.csv
enable_if_slip_trace	on	logs/if_slip_trace.csv
enable_gf_slip_trace	on	logs/gf_slip_trace.csv
enable_turbo_edit_trace	on	logs/turbo_edit_trace.csv
enable_windup_trace	off	logs/windup_trace.csv (phase wind-up diagnostic; off by default)
kalman_filter_concise_log	off	

A.8 Cycle-slip detector registry

Table 17: Cycle-slip detectors known to `SlipDetectorRegistry`. *Active* detectors have a creator in the factory table and run when their flag is set; *staged* detectors are declared in the `SlipDetectorKind` enum but excluded from the build (the registry warns and skips them if requested). The $k\sigma$ column is the MAD gate multiplier from `CycleSlipConfig`.

Detector	Config flag	Default	$k\sigma$	Status	Combination
`MelbourneWubbena`	`estimation.cycle_slip.mw`	on	8.0	**active**	Wide-lane MW combination (primary; the registry always has a creator for it).
`Graphic`	`estimation.cycle_slip.graphic`	off	6.0	**active**	GRAPHIC (code+phase average) per-signal detector.
`ExtraWideLane`	`estimation.cycle_slip.ewl`	off	5.0	**active**	Extra-wide-lane MW for satellites with 3 or more frequencies.
`GeometryFree`	`estimation.cycle_slip.gf`	off	–	*staged*	2nd-order time-differenced geometry-free (staged: not compiled, no registry creator).
`IonoFreePhase`	`estimation.cycle_slip.if`	off	–	*staged*	Between-satellite, between-epoch IF phase OMC; no registry creator (network.cycle_slip.if_phase_td).

A.9 State-vector key schema

Table 18: Dynamic state-vector key formats, reconstructed from the `make*Key` bodies in `StateKeySchema.hpp`. Angle-bracketed fields are runtime values; literal characters are the delimiter / infix / prefix / suffix constants.

State	Key format	Builder
Ambiguity	`<satellite>_<signal>`	`makeAmbiguityKey`

State	Key format	Builder
NL Ambiguity	`<satellite>_NL_<band>`	`makeNlAmbiguityKey`
WL Ambiguity	`<satellite>_WL_<band_a>_<band_b>`	`makeWlAmbiguityKey`
DD Ambiguity	`DD|<cons>|<sat_i>|<code>|<pivot>`	`makeDdAmbiguityKey`
Ionosphere	`<satellite>`	`makeIonosphereKey`
IFB	`<system>_IFB<band>`	`makeIfbKey`
DCB	`<system>_DCB`	`makeDcbKey`
Glonass Per Slot IFB	`<system>_IFB<band>_k`	`makeGlonassPerSlotIfbKey`
Receiver Phase Bias	`<system>_RPB_<signal>`	`makeReceiverPhaseBiasKey`
Receiver Isc	`<system>_ISC_<signal>`	`makeReceiverIscKey`
Satellite Bias	`<satellite>|<signal>`	`makeSatBiasKey`
Satellite Phase Bias	`<satellite>|<signal>|PB`	`makeSatPhaseBiasKey`
Satellite Clock	`<satellite>|SC`	`makeSatClockKey`
System Time Offset	`<system>`	`makeSystemTimeOffsetKey`
Pivot Seed	`<cons>|<band>`	`makePivotSeedKey`
Vtec Sh	`|||<trig>`	`makeVtecShKey`

B Supplement: linear algebra in GNSS

This supplement connects four topics of an advanced linear algebra course to computations that recur in GNSS positioning: vector and matrix norms, Hermitian matrices (real symmetric, in estimation), the singular value decomposition (SVD), and the Moore–Penrose pseudoinverse. Thirteen numerical examples carry the argument. All of them use one simulated epoch (Section B.1), and every number is printed by `scripts/linalg_supplement_examples.py` (Section B.7). Where the engine's own version of a result is derived elsewhere in this reference, the text links to it.

B.1 The model used throughout

Linearised GNSS observation equations have the form

$$\boldsymbol{y} = \boldsymbol{A}\,\boldsymbol{x} + \boldsymbol{\varepsilon}, \qquad \mathsf{E}\{\boldsymbol{\varepsilon}\} = \boldsymbol{0}, \qquad \mathsf{D}\{\boldsymbol{\varepsilon}\} = \boldsymbol{R}, \tag{393}$$

with m observations, n parameters and observation covariance $\boldsymbol{R}$. For code positioning the design matrix is the geometry matrix $\boldsymbol{G}$, with one row per satellite,

$$\boldsymbol{g}_i^{\mathrm{T}} = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 \end{bmatrix}, \tag{394}$$

where $\boldsymbol{e}_i$ is the unit line of sight to satellite i, expressed here in local east, north and up components, and the last column belongs to the receiver clock $c\,\mathrm{d}t_r$, kept in metres (Equation (6)). One epoch of code and phase on a single frequency adds one float ambiguity N_i, in cycles, per satellite:

$$P_i = \boldsymbol{g}_i^{\mathrm{T}}\boldsymbol{x} + \varepsilon_i^{P}, \qquad \Phi_i = \boldsymbol{g}_i^{\mathrm{T}}\boldsymbol{x} + \lambda_{\mathrm{L1}} N_i + \varepsilon_i^{\Phi}, \qquad i = 1, \ldots, m, \tag{395}$$

with P_i and Φ_i the prefit residuals in metres and $\boldsymbol{x}$ the corrections to east, north, up and the clock. Weighted least squares with $\boldsymbol{W} = \boldsymbol{R}^{-1}$ gives the best linear unbiased estimator and its covariance,

$$\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}, \qquad \boldsymbol{P}_{\hat{x}} = \boldsymbol{N}^{-1}, \qquad \boldsymbol{N} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}, \tag{396}$$

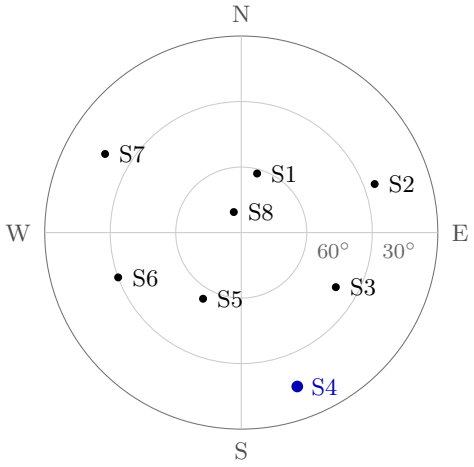


Figure 18: Sky plot of the eight satellites used in every example: azimuth clockwise from north, zenith at the centre, horizon at the rim. S4, at 15° elevation, carries the multipath error of Example 1. No satellite is below the horizon, which is the geometric reason for Example 8.

where $\boldsymbol{N}$ is the normal matrix. Figure 18 shows the simulated epoch, and the table below lists its settings.

Setting	Value
Signal	GPS L1, $\lambda_{\mathrm{L1}} = 0.1903$ m
Azimuth / elevation	S1 15°/62°, S2 70°/25°, S3 120°/40°, S4 160°/15°, S5 210°/55°, S6 250°/30°, S7 300°/18°, S8 340°/80°
Code σ_P	0.3 m (geodetic receiver); 3 m (low-cost receiver, Example 11)
Phase σ_Φ	3 mm (geodetic receiver); 2 mm (low-cost receiver)

B.2 Norms

Four norms and their unit balls.

$$\|\boldsymbol{v}\|_1 = \sum_i |v_i|, \qquad \|\boldsymbol{v}\|_2 = \Big(\sum_i v_i^2\Big)^{1/2}, \qquad \|\boldsymbol{v}\|_\infty = \max_i |v_i|, \qquad \|\boldsymbol{v}\|_{\boldsymbol{W}} = \big(\boldsymbol{v}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{v}\big)^{1/2}, \tag{397}$$

where $\boldsymbol{W}$ is symmetric positive definite. The first three treat every component alike. The fourth builds its metric from a matrix, and in estimation that matrix is an inverse covariance (Figure 19).

Least squares picks its norm from the covariance. Weighted least squares minimises a weighted L2 norm of the residuals,

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}} \|\boldsymbol{y} - \boldsymbol{A}\boldsymbol{x}\|^2_{\boldsymbol{R}^{-1}}, \tag{398}$$

so choosing the weight matrix is choosing the norm. With the Cholesky factor $\boldsymbol{R} = \boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$, the whitened quantities $\bar{\boldsymbol{y}} = \boldsymbol{L}^{-1}\boldsymbol{y}$ and $\bar{\boldsymbol{A}} = \boldsymbol{L}^{-1}\boldsymbol{A}$ have unit covariance, and Equation (398) becomes ordinary least squares, $\min\|\bar{\boldsymbol{y}} - \bar{\boldsymbol{A}}\boldsymbol{x}\|_2^2$. The same squared norm returns throughout GNSS quality control:

- the overall model test behind RAIM, $T = \hat{\boldsymbol{\varepsilon}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{\varepsilon}} \sim \chi^2(m-n)$ when the model is correct (Section 13.2);

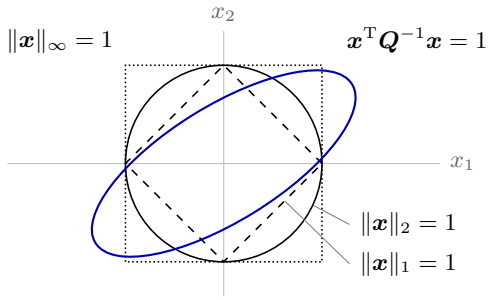


Figure 19: Unit balls of the four norms of Equation (397) in the plane: the L1 diamond (dashed), the L2 circle (solid), the L∞ square (dotted) and, in blue, the weighted norm with $\boldsymbol{W} = \boldsymbol{Q}^{-1}$ for $\boldsymbol{Q} = \left[\begin{smallmatrix} 1.8 & 0.9 \\ 0.9 & 0.9 \end{smallmatrix}\right]$. Its unit ball is an ellipse with axes along the eigenvectors of $\boldsymbol{Q}$ and semi-axes $\sqrt{\lambda_i(\boldsymbol{Q})}$; whitening by the Cholesky factor of $\boldsymbol{Q}$ maps it onto the circle.

- the Kalman innovation test, $\boldsymbol{\nu}^{\mathrm{T}}\boldsymbol{S}^{-1}\boldsymbol{\nu} \sim \chi^2(m_k)$ (Section 12.4);
- integer least squares, $\check{\boldsymbol{N}} = \arg\min_{\boldsymbol{a}\in\mathbb{Z}^n} \|\hat{\boldsymbol{N}} - \boldsymbol{a}\|^2_{\boldsymbol{Q}^{-1}_{\hat{N}}}$ (Equation (255)), whose ratio test compares the two smallest values of this norm.

The residuals are a projection of the data,

$$\hat{\boldsymbol{\varepsilon}} = (\boldsymbol{I} - \boldsymbol{\Pi}_A)\,\boldsymbol{y}, \qquad \boldsymbol{\Pi}_A = \boldsymbol{A}\boldsymbol{N}^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}. \tag{399}$$

For uncorrelated observations the diagonal entries r_i of $\boldsymbol{I} - \boldsymbol{\Pi}_A$ are the redundancy numbers of Equation (190). They sum to $m - n$, and r_i is the share of an error in observation i that stays visible in its own residual.

L1: bounding the influence of one observation. The L2 norm squares residuals, so a single large error dominates the fit. Least absolute deviations, $\min\|\boldsymbol{y} - \boldsymbol{A}\boldsymbol{x}\|_1$, grow only linearly with the residual. Robust GNSS estimators sit between the two and run as iteratively reweighted least squares. The Huber function is quadratic near zero and linear in the tails,

$$\rho(v) = \begin{cases} \tfrac{1}{2}v^2, & |v| \le k, \\ k|v| - \tfrac{1}{2}k^2, & |v| > k, \end{cases} \qquad w(v) = \min\Big(1, \frac{k}{|v|}\Big), \tag{400}$$

with v the standardised residual and w the weight of the next iteration. The IGG-III and Danish schemes go further and drive the weight of large residuals toward zero.

Example 1 (a multipath error on a low satellite). S4 (15° elevation) delivers a non-line-of-sight signal with a +20 m range error; the other code errors are Gaussian with $\sigma = 0.3$ m. Over 4000 trials:

Estimator	RMS 3-D position error
Least squares, no fault	0.52 m
Least squares, with the fault	12.89 m
Huber, $k = 1.5$	1.15 m
L1 (least absolute deviations)	0.99 m
Least squares, S4 excluded	0.61 m

Least squares hides the fault. The redundancy number of S4 is $r_4 = 0.354$, so only 7.1 m of the 20 m error stays in its own residual; the rest moves the solution by $(-1.9,\ 7.7,\ 10.2)$ m in east, north and up and by 8.0 m in the clock. The low satellites have the smallest redundancy numbers (S7 at 18°: 0.258; S2 and S4: 0.354; S3 at 40°: 0.673). This fault is large enough for Baarda's w-test [1] to flag: the expected statistic of Equation (191) is $20\sqrt{r_4}/0.3 = 39.7$ against a critical value of 3.29. The minimal detectable bias of Equation (193) for S4 is, however, 2.1 m: a smaller multipath error passes the test and still biases the position. The robust estimators cut the error by a factor of 11 to 13 with no explicit detection step, and excluding S4 after detection returns 0.61 m.

L∞: rounding, integrity and convergence. Rounding each float ambiguity to its nearest integer treats the coordinates separately. It returns the integer vector nearest in the unweighted L2 norm, and its pull-in region, the set of float values that round to $\boldsymbol{a}$, is a unit cube, which is a ball of the L∞ norm:

$$\mathcal{R}_{\boldsymbol{a}} = \left\{\boldsymbol{x} : \|\boldsymbol{x} - \boldsymbol{a}\|_\infty \le \tfrac{1}{2}\right\}. \tag{401}$$

Integer least squares measures distance in the $\boldsymbol{Q}_{\hat{N}}^{-1}$ norm instead. Its pull-in regions are Voronoi cells, hexagons in two dimensions:

$$\mathcal{S}_{\boldsymbol{a}} = \left\{\boldsymbol{x} : \|\boldsymbol{x} - \boldsymbol{a}\|_{\boldsymbol{Q}_{\hat{N}}^{-1}} \le \|\boldsymbol{x} - \boldsymbol{u}\|_{\boldsymbol{Q}_{\hat{N}}^{-1}} \text{ for all } \boldsymbol{u} \in \mathbb{Z}^n\right\}. \tag{402}$$

The L∞ norm has two other everyday roles. Integrity monitoring bounds the largest position error over the fault modes it considers, and iterative solvers stop when $\|\Delta\boldsymbol{x}\|_\infty$ falls below a tolerance.

Example 2 (rounding versus integer least squares). Two float ambiguities with standard deviations of 0.30 and 0.26 cycles and a correlation of 0.96. Success rates from 10^6 Monte Carlo samples:

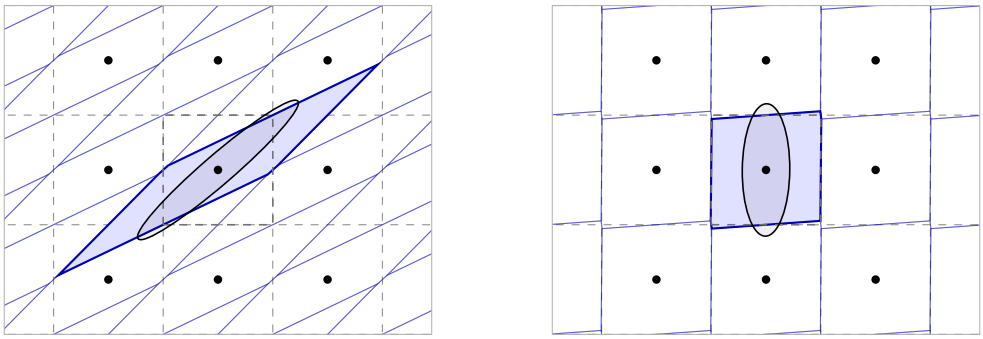
Covariance	Correlation	κ	Rounding	Integer LS
$\boldsymbol{Q}_{\hat{N}}$	0.96	50.1	0.900	0.957
$\boldsymbol{Q}_{\hat{z}} = \boldsymbol{Z}\boldsymbol{Q}_{\hat{N}}\boldsymbol{Z}^{\mathrm{T}}$	0.03	7.8	0.957	0.957

The transformation $\boldsymbol{Z} = \left[\begin{smallmatrix} 1 & -1 \\ 1 & -2 \end{smallmatrix}\right]$ has integer entries and determinant -1, so it maps integers to integers and preserves $\det \boldsymbol{Q}_{\hat{N}}$ (4.77×10^{-4} cycles^4 before and after). Integer least squares does not change under it; rounding improves until it matches (Figure 20). Rounding also cannot use the correlation: at correlations of 0.98 and 0.99 it stays at 0.903 and 0.904, while integer least squares rises to 0.973 and 0.989. The decorrelation of Section 16.2 makes the search space nearly spherical so that the integer search is fast [37]; it does not change the answer.

Matrix norms and the condition number.

$$\|\boldsymbol{A}\|_2 = \sigma_{\max}(\boldsymbol{A}), \qquad \|\boldsymbol{A}\|_F = \Big(\sum_{i,j} a_{ij}^2\Big)^{1/2} = \Big(\sum_i \sigma_i^2\Big)^{1/2}, \qquad \kappa_2(\boldsymbol{A}) = \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{403}$$

The condition number bounds how far relative errors in the data can grow in the solution. In floating point a solve loses roughly $\log_{10}\kappa$ of the 16 significant digits that double precision carries, and forming normal equations squares it: $\kappa_2(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}) = \kappa_2(\boldsymbol{A})^2$ [10].



(a) correlation 0.96 (b) after $\boldsymbol{Z}$, correlation 0.03

Figure 20: Pull-in regions for the two covariances of Example 2, drawn to scale in cycles; dots are integer vectors and the correct one is at the centre. Dashed grey: rounding cells, the unit squares of Equation (401). Blue: integer least-squares cells of Equation (402), the central one shaded. Grey ellipse: the 95 % region of the float solution. In (a) the float solution reaches well outside the central square but only to the edge of the central hexagon. In (b) the two kinds of cell nearly coincide, which is why rounding matches integer least squares after decorrelation.

Example 3 (dilution of precision is a matrix norm). With unit weights, $\boldsymbol{P}_{\hat{x}} = (\boldsymbol{G}^{\mathrm{T}}\boldsymbol{G})^{-1} = \boldsymbol{V}\boldsymbol{\Sigma}^{-2}\boldsymbol{V}^{\mathrm{T}}$, so

$$\mathrm{GDOP} = \sqrt{\mathrm{tr}\,(\boldsymbol{G}^{\mathrm{T}}\boldsymbol{G})^{-1}} = \Big(\sum_i \sigma_i^{-2}\Big)^{1/2} = \|\boldsymbol{G}^{+}\|_F. \tag{404}$$

For Figure 18 the singular values of $\boldsymbol{G}$ are 3.331, 1.668, 1.331 and 0.590, and both sides equal 1.9705. The spectral norm $\|\boldsymbol{G}^{+}\|_2 = 1/\sigma_{\min} = 1.694$ is the largest gain from a unit-norm vector of range errors to position and clock. GDOP averages over all directions; the spectral norm is the worst case. The partial measures are HDOP 0.983, VDOP 1.423 and TDOP 0.944.

Example 4 (a determinant does not measure conditioning). Scaling an $n \times n$ matrix by c gives $\det(c\boldsymbol{N}) = c^n \det \boldsymbol{N}$ but leaves $\kappa(c\boldsymbol{N}) = \kappa(\boldsymbol{N})$. For the code-only normal matrix with five parameters (east, north, up, clock, zenith wet delay, the state of Section 14.1) and weights $1/\sigma^2$:

σ	$\det \boldsymbol{N}$	$\kappa(\boldsymbol{N})$
0.3 m	2.8×10^{6}	1.93×10^{3}
1 m	16.4	1.93×10^{3}
3 m	2.8×10^{-4}	1.93×10^{3}
5 m	1.7×10^{-6}	1.93×10^{3}

A guard that rejects the solution when the determinant falls below a fixed threshold, such as Equation (210), responds to the variance units and not to the geometry: the same satellites pass easily at 0.3 m and approach the threshold as the variances grow. A scale-invariant rank test compares each singular value with the largest (for a symmetric positive semidefinite matrix, the eigenvalues) and counts those below $\tau\,\sigma_{\max}$. The table also prices the zenith delay: $\kappa(\boldsymbol{G}^{\mathrm{T}}\boldsymbol{G})$ is 31.8 for position and clock alone and 1933 once the delay joins them, because its mapping function (roughly $1/\sin e$) is one more function of elevation that the up and clock columns nearly reproduce.

B.3 Hermitian matrices

A matrix is Hermitian when it equals its conjugate transpose, $\boldsymbol{A} = \boldsymbol{A}^{\mathsf{H}}$. For real matrices that is symmetry, and every covariance, weight, normal and Kalman covariance matrix in GNSS estimation is real symmetric. The spectral theorem gives

$$\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\mathrm{T}}, \qquad \boldsymbol{V}^{\mathrm{T}}\boldsymbol{V} = \boldsymbol{I}, \qquad \lambda_i \in \mathbb{R}, \tag{405}$$

and a covariance matrix is positive semidefinite, $\lambda_i \geq 0$. The eigenvalues bound the variance in any direction $\boldsymbol{d}$ (the Rayleigh quotient):

$$\lambda_{\min} \leq \frac{\boldsymbol{d}^{\mathrm{T}}\boldsymbol{Q}\,\boldsymbol{d}}{\boldsymbol{d}^{\mathrm{T}}\boldsymbol{d}} \leq \lambda_{\max}. \tag{406}$$

Complex Hermitian matrices appear one level down, in the receiver's signal processing (Example 7).

Example 5 (an error ellipse is an eigendecomposition). Code-only solution for Figure 18 with $\sigma = 0.3$ m. The horizontal block of $\boldsymbol{P}_{\hat{x}} = \sigma^2(\boldsymbol{G}^{\mathrm{T}}\boldsymbol{G})^{-1}$ is

$$\boldsymbol{Q}_{EN} = \begin{bmatrix} 0.0337 & 0.0053 \\ 0.0053 & 0.0533 \end{bmatrix} \ \mathrm{m}^2.$$

Its eigenvalues, 0.0547 and 0.0324 m^2, give 1σ semi-axes of 0.234 and 0.180 m. A 95 % region in two dimensions scales them by $\sqrt{\chi^2_2(0.95)} = 2.448$, to 0.572 by 0.440 m, with the major axis along the first eigenvector at azimuth 14.1°. The vertical standard deviation is larger, 0.427 m, for the reason Example 8 gives.

Cholesky: whitening, simulation, decorrelation. A symmetric positive definite matrix factors as $\boldsymbol{Q} = \boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$ with $\boldsymbol{L}$ lower triangular. The factor is used for

- solving the normal equations, at half the cost of an LU factorisation and without pivoting;
- whitening correlated observations: $\boldsymbol{L}^{-1}\boldsymbol{y}$ has identity covariance (the network estimator whitens its correlated ionosphere-constraint and extra-wide-lane row blocks this way, in `networkWriteWhitenedRows`, `src/core/network/assembly/StationBuild.cpp`);
- simulating correlated noise: $\boldsymbol{\varepsilon} = \boldsymbol{L}\boldsymbol{z}$ with $\boldsymbol{z} \sim N(\boldsymbol{0}, \boldsymbol{I})$;
- in the form $\boldsymbol{Q}_{\hat{N}} = \boldsymbol{L}^{\mathrm{T}}\boldsymbol{D}\boldsymbol{L}$, the LAMBDA decorrelation and search (Equation (259)).

Example 6 (double differences share a reference satellite). With undifferenced variance σ^2 on every phase observation, n double differences (Section 5.3) against one reference satellite have

$$\boldsymbol{Q}_{\nabla\Delta} = 2\sigma^2\big(\boldsymbol{I}_n + \boldsymbol{1}\boldsymbol{1}^{\mathrm{T}}\big), \qquad \boldsymbol{Q}_{\nabla\Delta}^{-1} = \frac{1}{2\sigma^2}\Big(\boldsymbol{I}_n - \frac{\boldsymbol{1}\boldsymbol{1}^{\mathrm{T}}}{n+1}\Big), \tag{407}$$

the inverse following from the Sherman–Morrison formula. The eigenvalues are $2\sigma^2$ with multiplicity $n-1$ and $2\sigma^2(n+1)$ along $\boldsymbol{1}$. For eight satellites ($n = 7$) and $\sigma = 3$ mm they are 18 mm^2 six times and 144 mm^2 once. The reference satellite's error enters every double difference, so the common-mode direction carries eight times the variance of any other. Treating the double differences as independent, each with its true variance $4\sigma^2$, overweights that common mode by a factor of 4 and underweights every other direction by a factor of 2. The inverse above, or whitening by the Cholesky factor, weights them correctly.

Example 7 (antenna arrays: a complex Hermitian covariance). An array of M elements sampling complex baseband (I/Q) snapshots $\boldsymbol{s}(t) \in \mathbb{C}^M$ has the spatial covariance

$$\boldsymbol{C}_s = \mathsf{E}\{\boldsymbol{s}\,\boldsymbol{s}^{\mathsf{H}}\} = \sum_k p_k\,\boldsymbol{\alpha}(\theta_k)\,\boldsymbol{\alpha}(\theta_k)^{\mathsf{H}} + \sigma_n^2 \boldsymbol{I}, \tag{408}$$

which is Hermitian and positive definite; $\boldsymbol{\alpha}(\theta)$ is the steering vector of a source at angle θ and p_k its power. GNSS signals arrive 20 to 30 dB below the thermal noise, so the eigenvectors with large eigenvalues belong to interference. Projecting onto their orthogonal complement, $\boldsymbol{\Pi}^{\perp} = \boldsymbol{I} - \boldsymbol{U}_J \boldsymbol{U}_J^{\mathsf{H}}$, removes the interference without knowing where the satellites are [15]. With four elements at half-wavelength spacing, a jammer at 40° with a jammer-to-noise ratio of 40 dB, a satellite at −15° with a signal-to-noise ratio of −20 dB, and 4000 snapshots:

- the sample covariance has eigenvalues 0.98, 1.00, 1.03 and 39 648 (noise power 1). One stands out, near M times the jammer power; the satellite does not show in the spectrum at all;
- the projection attenuates the jammer direction by 77.9 dB and costs the satellite direction 0.09 dB.

Array weights	Output signal-to-jammer-plus-noise ratio
Single element	−60.0 dB
Beam steered at the satellite	−43.3 dB
Projection, then beam	−14.1 dB
MVDR, $\boldsymbol{w} \propto \boldsymbol{C}_s^{-1}\boldsymbol{\alpha}(\theta_{\text{sat}})$	−14.1 dB

The last two recover the full array gain of a clean sky, $-20\ \text{dB} + 10\log_{10} 4 = -14.0$ dB. Correlation with the spreading code then lifts the signal above the noise as usual.

Keeping a Kalman covariance symmetric and positive definite. Rounding error erodes symmetry and positive definiteness in the short-form update $(\boldsymbol{I}-\boldsymbol{K}\boldsymbol{H})\boldsymbol{P}$, most of all when states with millimetre-level uncertainty (converged ambiguities) share the matrix with states whose uncertainty is kilometres (a freshly initialised clock). The engine uses the Joseph form and symmetrises the result (Equation (169)). Square-root and UD filters go further: they carry a factor $\boldsymbol{S}$ with $\boldsymbol{P} = \boldsymbol{S}\boldsymbol{S}^{\mathrm{T}}$ and $\kappa(\boldsymbol{S}) = \sqrt{\kappa(\boldsymbol{P})}$ [4]. These, and not the pseudoinverse, are the tools for a filter whose code and phase precisions differ by three orders of magnitude.

B.4 The singular value decomposition

Every real $m \times n$ matrix factors as

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}} = \sum_{i=1}^{r} \sigma_i\,\boldsymbol{u}_i \boldsymbol{v}_i^{\mathrm{T}}, \qquad \sigma_1 \ge \sigma_2 \ge \cdots \ge \sigma_r > 0, \tag{409}$$

with orthonormal $\boldsymbol{u}_i$ in observation space and $\boldsymbol{v}_i$ in parameter space. A parameter change along $\boldsymbol{v}_i$ moves the observations by σ_i along $\boldsymbol{u}_i$. The σ_i^2 are the eigenvalues of $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$, and for a symmetric positive semidefinite matrix the eigenvalues and singular values coincide. The numerical rank is the number of σ_i above a tolerance $\tau\sigma_1$. The $\boldsymbol{v}_i$ with $\sigma_i \approx 0$ span the null space: parameter combinations the data cannot see. In a network that estimates every receiver and satellite clock, adding one constant to all of them leaves every observation unchanged, so a zero singular value is structural there (Example 13; Section 21). The network estimator counts such directions from the eigenvalues of its reduced normals before every solve (Section 20.4).

Example 8 (the weakest direction of the geometry). The right singular vector for the smallest singular value of $\boldsymbol{G}$ (0.590) is

$$\boldsymbol{v}_4 = (-0.03,\ -0.14,\ 0.83,\ 0.54) \quad \text{in (east, north, up, clock).}$$

Raising the antenna by 0.83 m while advancing the clock by 0.54 m changes the eight ranges by 0.32 m at most. The clock column is all ones and the up column is $-\sin e_i$, negative for every satellite above the horizon, so the two columns are close to parallel: the up and clock estimates correlate at +0.93, and VDOP (1.42) exceeds HDOP (0.98). Satellites below the horizon would break the near-parallelism, which is why ground-based GNSS measures height worse than horizontal position.

Example 9 (attitude from antenna baselines). Three antennas on a vehicle give two baselines with known body-frame vectors $\boldsymbol{b}_k$ (1.5 m forward and 1.0 m to the side) and RTK-measured local-frame vectors $\boldsymbol{r}_k$. The attitude is the rotation that minimises $\sum_k \|\boldsymbol{r}_k - \boldsymbol{C}\boldsymbol{b}_k\|^2$ (Wahba's problem [41]). With $\boldsymbol{B} = \sum_k \boldsymbol{r}_k \boldsymbol{b}_k^{\mathrm{T}} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$ [20],

$$\hat{\boldsymbol{C}} = \boldsymbol{U}\ \mathrm{diag}\big(1,\ 1,\ \det(\boldsymbol{U}\boldsymbol{V}^{\mathrm{T}})\big)\ \boldsymbol{V}^{\mathrm{T}}. \tag{410}$$

An unconstrained 3×3 fit would have nine unknowns and six equations; the rotation constraint makes the problem well posed, and the SVD solves it in closed form. With 3 mm noise on each baseline component over 5000 trials, the RMS errors are 0.095° in yaw, 0.116° in pitch and 0.171° in roll. Each error follows its baseline: roll depends on the 1.0 m side baseline (3 mm over 1.0 m is 0.17°) and pitch on the 1.5 m forward one (0.11°). $\boldsymbol{B}$ has rank 2 (singular values 2.25, 0.99 and 0), and the determinant term fixes the third axis.

The same step keeps an inertial attitude matrix orthonormal. Replacing a drifted attitude matrix $\boldsymbol{C}$ (the $\boldsymbol{C}_b^e$ of the inertial path) by $\boldsymbol{U}\boldsymbol{V}^{\mathrm{T}}$ from its SVD gives the nearest orthogonal matrix in the Frobenius norm: in the example $\|\boldsymbol{C}^{\mathrm{T}}\boldsymbol{C} - \boldsymbol{I}\|_F$ drops from 5.2×10^{-3} to 9×10^{-16}. The engine applies it during strapdown integration and after every attitude feedback (Section 17.1).

Example 10 (common-mode error in a regional network). Daily coordinate residuals of the stations in a regional network share a common-mode signal from reference-frame, large-scale atmospheric and loading errors. With the residual matrix $\boldsymbol{X}$ (days × stations, column means removed), the leading term $\sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^{\mathrm{T}}$ of its SVD estimates that signal: $\boldsymbol{u}_1$ carries its time series and $\boldsymbol{v}_1$ each station's response [7]. The example is synthetic: up component, 12 stations, 730 days, a time-correlated common signal of 2.5 mm scaled by a station response between 0.8 and 1.2, and 2.0 mm of white noise per station.

- The first component carries 66.1 % of the variance; the next two, 3.9 % and 3.6 %.
- Removing it lowers the mean station RMS from 3.27 mm to 1.91 mm, the white-noise level.
- The recovered series correlates 0.977 with the true common mode, and $\boldsymbol{v}_1$ correlates 0.962 with the true station responses.

Ill-posed inversions: truncated SVD and Tikhonov. Ionospheric and water-vapour tomography lead to systems whose singular values decay toward zero with no clear gap. The least-squares solution

$$\hat{\boldsymbol{x}} = \sum_i \frac{\boldsymbol{u}_i^{\mathrm{T}}\boldsymbol{y}}{\sigma_i}\,\boldsymbol{v}_i \tag{411}$$

then amplifies noise through the small σ_i. Truncated SVD keeps the first k terms; Tikhonov regularisation replaces $1/\sigma_i$ by $\sigma_i/(\sigma_i^2 + \alpha^2)$. Both accept bias to reduce variance, with k or α chosen from the data (the L-curve, generalised cross-validation). That is a deliberate choice for an ill-posed problem, and a different thing from what a default pseudoinverse tolerance does to a well-posed one (Example 11). The engine's SP3 orbit fit solves a well-posed least-squares problem through Equation (411) with a relative cutoff (Section 8.2).

B.5 The pseudoinverse

The Moore–Penrose pseudoinverse is $\boldsymbol{A}^+ = \boldsymbol{V}\boldsymbol{\Sigma}^+\boldsymbol{U}^{\mathrm{T}}$, where $\boldsymbol{\Sigma}^+$ inverts the nonzero singular values and keeps the zeros. It is the unique matrix that satisfies the four Penrose conditions

$$\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{A} = \boldsymbol{A}, \qquad \boldsymbol{A}^+\boldsymbol{A}\boldsymbol{A}^+ = \boldsymbol{A}^+, \qquad (\boldsymbol{A}\boldsymbol{A}^+)^{\mathrm{T}} = \boldsymbol{A}\boldsymbol{A}^+, \qquad (\boldsymbol{A}^+\boldsymbol{A})^{\mathrm{T}} = \boldsymbol{A}^+\boldsymbol{A}. \tag{412}$$

Three cases matter in GNSS.

- *Full column rank*, as in ordinary positioning: $\boldsymbol{A}^+ = (\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathrm{T}}$, so the pseudoinverse is least squares computed through the SVD. With weights, $\hat{\boldsymbol{x}} = (\boldsymbol{W}^{1/2}\boldsymbol{A})^+\boldsymbol{W}^{1/2}\boldsymbol{y}$; applying $\boldsymbol{A}^+$ to $\boldsymbol{y}$ directly drops the weights.
- *Rank deficient*, as with a datum defect: $\boldsymbol{A}^+\boldsymbol{y}$ is the least-squares solution of minimum norm. It is orthogonal to the null space, which means it chooses a datum without saying so (Example 13; [36]).
- *Numerically rank deficient*: singular values below $\tau\sigma_1$ count as zero. Deliberate truncation regularises an ill-posed problem (Section B.4); a default tolerance applied to a badly scaled but well-posed problem discards real information (Example 11).

In the engine the pseudoinverse appears in two full-rank roles: the SP3 orbit fit above, and the fallback of the Kalman gain solve when the $\mathrm{LDL}^{\mathrm{T}}$ factorisation of $\boldsymbol{S}$ fails (Section 12.2).

Example 11 (metre-level code, millimetre-level phase). One epoch of code and phase, Equation (395), with float ambiguities estimated as corrections to a priori values and a position offset of (3, −2, 5) m. The phase-to-code weight ratio is $(\sigma_P/\sigma_\Phi)^2$: 10^4 for the geodetic receiver and 2.25×10^6 for the low-cost one. Each solver column gives the position error against a stable reference, SVD least squares on the whitened system in double precision.

			double precision		single precision	
Case	$\kappa(\boldsymbol{W}^{1/2}\boldsymbol{A})$	$\kappa(\boldsymbol{N})$	Cholesky on $\boldsymbol{N}$	$\boldsymbol{N}^+$	normal eqs.	QR on $\boldsymbol{W}^{1/2}\boldsymbol{A}$
Geodetic	1.8×10^3	3.4×10^6	< 1 µm	< 1 µm	8.3 cm	7 µm
Low-cost	2.8×10^4	7.6×10^8	< 1 µm	< 1 µm	**42 m**	3 µm
Clock in seconds	4.6×10^{11}	2.1×10^{23}	< 1 µm	**5.1 m**	6.9 cm	2 µm

- *Not near-singular.* Double precision carries about 16 digits, and $\kappa(\boldsymbol{N})$ between 10^6 and 10^9 leaves 7 to 10 of them. The weight matrix is diagonal, so its inverse is exact however wide its range.
- *The weight range costs digits through the normal equations*, because $\kappa(\boldsymbol{N}) = \kappa(\boldsymbol{W}^{1/2}\boldsymbol{A})^2$. In single precision that is the difference between 42 m and 3 µm in the low-cost case. QR on the whitened system never forms $\boldsymbol{N}$.

- *Units, not information.* A clock parameter in seconds multiplies its column by $c = 2.998 \times 10^8$ m/s and drives $\kappa(\boldsymbol{N})$ to 2.1×10^{23}. Cholesky is insensitive to this kind of diagonal scaling and still returns the position within 1 µm. The pseudoinverse of $\boldsymbol{N}$ with numpy's default relative tolerance ($10^{-15}\sigma_1$) sets 11 of its 12 singular values to zero and returns the a priori position unchanged: a 5.1 m error with no warning.

The remedies are consistent units (the clock in metres, unknowns as corrections to a priori values), whitening followed by QR or by Cholesky in double precision, and square-root forms in filters. The pseudoinverse is not a remedy for ill-conditioning.

Example 12 (what phase adds to a single epoch). Order the parameters of Equation (395) as the position and clock $\boldsymbol{x}$ followed by the ambiguities $N_1, \ldots, N_m$, with $\boldsymbol{W}_P = \sigma_P^{-2}\boldsymbol{I}$ and $\boldsymbol{W}_\Phi = \sigma_\Phi^{-2}\boldsymbol{I}$. The normal matrix is

$$\boldsymbol{N} = \begin{bmatrix} \boldsymbol{N}_{xx} & \boldsymbol{N}_{xa} \\ \boldsymbol{N}_{ax} & \boldsymbol{N}_{aa} \end{bmatrix} = \begin{bmatrix} \boldsymbol{G}^{\mathrm{T}}(\boldsymbol{W}_P + \boldsymbol{W}_\Phi)\,\boldsymbol{G} & \lambda_{\mathrm{L1}}\,\boldsymbol{G}^{\mathrm{T}}\boldsymbol{W}_\Phi \\ \lambda_{\mathrm{L1}}\,\boldsymbol{W}_\Phi\boldsymbol{G} & \lambda_{\mathrm{L1}}^2\,\boldsymbol{W}_\Phi \end{bmatrix}. \tag{413}$$

Eliminating the ambiguities (the Schur complement) leaves

$$\boldsymbol{N}_{xx} - \boldsymbol{N}_{xa}\boldsymbol{N}_{aa}^{-1}\boldsymbol{N}_{ax} = \boldsymbol{G}^{\mathrm{T}}\boldsymbol{W}_P\,\boldsymbol{G}, \tag{414}$$

the code-only normal matrix. With float ambiguities, one epoch of phase adds nothing to the position: in Example 11 the code-plus-phase and code-only positions agree to 3×10^{-13} m. This is the single-receiver case of the statement in Section 13.3 that per-arc float ambiguities leave no phase-side redundancy. Phase precision reaches the position only once the ambiguities are fixed to integers or tied together over time.

The weight range reappears in the float ambiguities:

$$\boldsymbol{Q}_{\hat{N}} = \frac{1}{\lambda_{\mathrm{L1}}^2}\Big(\sigma_\Phi^2\,\boldsymbol{I}_m + \sigma_P^2\,\boldsymbol{\Pi}_G\Big), \qquad \boldsymbol{\Pi}_G = \boldsymbol{G}(\boldsymbol{G}^{\mathrm{T}}\boldsymbol{G})^{-1}\boldsymbol{G}^{\mathrm{T}}. \tag{415}$$

$\boldsymbol{\Pi}_G$ is an orthogonal projector of rank 4, so the eigenvalues of $\boldsymbol{Q}_{\hat{N}}$ are $(\sigma_P^2 + \sigma_\Phi^2)/\lambda_{\mathrm{L1}}^2$ four times and $\sigma_\Phi^2/\lambda_{\mathrm{L1}}^2$ the remaining $m - 4$ times, and

$$\boxed{\kappa(\boldsymbol{Q}_{\hat{N}}) = 1 + \frac{\sigma_P^2}{\sigma_\Phi^2}.} \tag{416}$$

Computed: eigenvalues of 2.486 and 2.49×10^{-4} cycles^2 and $\kappa = 1.0001 \times 10^4$ in the geodetic case, and $\kappa = 2.25 \times 10^6$ in the low-cost case, both matching Equation (416). The ambiguity search space is an ellipsoid 100 times longer than it is wide in the geodetic case and 1500 times in the low-cost case. That elongation is why LAMBDA decorrelates before it searches (Example 2).

Example 13 (a network with a datum defect). Three receivers track eight satellites, and the only parameters are clocks: $y_{ri} = \mathrm{d}t_r - \mathrm{d}t^i + \varepsilon_{ri}$ in metres, with $\sigma = 1$ cm. Adding the same constant to every clock leaves every observation unchanged, so $\boldsymbol{A}\mathbf{1} = \mathbf{0}$: rank 10 for 11 parameters, with a smallest singular value of 3×10^{-16}. This is the null vector of Section 21, which compares two pinned datums. Here the pseudoinverse is the second solution:

- *Pseudoinverse*: minimum norm, hence $\mathbf{1}^{\mathrm{T}}\hat{\boldsymbol{x}}^{+} = 0$; the datum is that the eleven clocks sum to zero.

- *Explicit S-basis*: the first receiver's clock is fixed to zero (the pivot) and the remaining full-rank system is solved.

Quantity	Pseudoinverse	Pivot
Satellite clock differences $\mathrm{d}t^i - \mathrm{d}t^1$ (estimable)	agree to 2×10^{-10} m	
Receiver clocks (not estimable)	23.6, −128.1, 51.7 km	0, −151.7, 28.1 km
Trace of the covariance (unit variance)	2.674	3.833
Change in the receiver clocks when the +468 µs satellite sets	+13 534.8 m (45.15 µs), all three	0, 0.8, −7.7 mm

The pseudoinverse has the smaller trace, the minimum-trace property of free-network adjustment, but it does not remove the defect. It picks a datum, and that datum moves whenever the parameter set changes. For a departing parameter x_k the jump is $(x_k - \bar{x})/(n-1)$, with $\bar{x}$ the mean of the n parameters before it left: here $(468.0 - 16.5)/10 = 45.15$ µs, as computed. GNSS networks and PPP-RTK therefore choose the S-basis explicitly: a pivot receiver, a pivot satellite, or a zero-mean condition with rules for set changes [25]. Each estimable combination then has a stated meaning (Section 20.4).

Choosing a solver.

Situation	Tool
Full rank, well scaled	Cholesky on $\boldsymbol{N}$ in double precision, or QR on $\boldsymbol{W}^{1/2}\boldsymbol{A}$
Wide weight range, single precision, long filters	Whitening and QR; a square-root or UD filter for sequential updates
Mixed units (seconds, absolute coordinates)	Rescaling: the clock in metres, unknowns as corrections to a priori values
Datum defect	An explicit S-basis; the pseudoinverse only when estimable functions are all that matter
Ill-posed inversion (tomography)	Truncated SVD or Tikhonov with a data-driven parameter
Rank diagnosis	Singular values or eigenvalues relative to the largest, never the determinant

B.6 Exercises

1. Show that GDOP $= \|\boldsymbol{G}^{+}\|_F$ and $\|\boldsymbol{G}^{+}\|_2 = 1/\sigma_{\min}$ for a geometry matrix of full column rank (Example 3). Which of the two answers a worst-case question?
2. Form the partitioned normal matrix Equation (413), eliminate the ambiguities, and show that the reduced normal matrix equals $\boldsymbol{G}^{\mathrm{T}}\boldsymbol{W}_P\boldsymbol{G}$ (Example 12). Explain what changes once the ambiguities are fixed to integers.
3. Derive Equation (415) and the eigenvalues of $\boldsymbol{Q}_{\hat{N}}$, and show Equation (416).
4. For $\boldsymbol{Q}_{\nabla\Delta}$ of Equation (407), find the eigenvalues and eigenvectors and verify the Sherman–Morrison inverse (Example 6). Whiten simulated double differences with the Cholesky factor and check that their sample covariance is close to the identity.
5. Show that the pseudoinverse solution of Example 13 satisfies $\mathbf{1}^{\mathrm{T}}\hat{\boldsymbol{x}}^{+} = 0$, and derive the jump $(x_k - \bar{x})/(n-1)$ when parameter k leaves.

6. In the third case of Example 11 the clock column is scaled by c. Show that column scaling changes κ but not the least-squares position, then explain why the pseudoinverse of $\boldsymbol{N}$ failed while Cholesky did not. (The pseudoinverse tolerance is relative to σ_1.)

7. The WLS seed rejects a solution through the determinant guard of Equation (210) (`WlsEstimator::estimate`). Use Example 4 to explain what this test measures, and replace it with a scale-invariant test.

8. Show that maximising $\operatorname{tr}(\boldsymbol{C}^{\mathrm{T}}\boldsymbol{B})$ over rotations gives Equation (410), and explain why two baselines suffice although $\boldsymbol{B}$ has rank 2 (Example 9).

9. Move the multipath error of Example 1 from S4 to S7. Predict the change from the redundancy numbers, then run the script to check.

B.7 Reproducing the numbers and further reading

Every number in Examples 1 to 13 is printed by

```
python scripts/linalg_supplement_examples.py
```

which needs only numpy and runs in under a minute. Each example has its own random seed, so the output is reproducible and one example can be changed without disturbing the others.

The same operations appear in the engine as follows.

Operation	Symbol (file)
Cholesky whitening of correlated row blocks into unit-information rows	`networkWriteWhitenedRows` (`src/core/network/assembly/StationBuild.cpp`), called by the ionosphere-constraint rows (`src/core/network/assembly/IonoConstraintRows.cpp`); the single-difference extra-wide-lane rows whiten their own groups (`src/core/network/ar/SdEwlRows.cpp`)
Condition number of the WLS normal matrix (logged through an SVD) and the determinant guard	`WlsEstimator::estimate` (`src/core/math/WlsEstimator.cpp`)
$\mathrm{LDL}^{\mathrm{T}}$ solve of $\boldsymbol{S}$ with a pseudoinverse fallback; Joseph update; SVD re-orthonormalisation after attitude feedback	`KalmanFilter::update` (`src/core/filter/KalmanFilter.cpp`)
SVD re-orthonormalisation during strapdown integration	`ImuIntegrator::integrate` (`src/core/filter/ImuIntegrator.cpp`)
SP3 orbit fit by truncated-SVD least squares	`Interpolate::trigSvdInterp` (`src/core/math/Interpolate.cpp`)
Null-space (datum) count from eigenvalues	`NetworkSbasis::assertNullSpaceRank` (`src/core/network/solve/Sbasis.cpp`)
$\mathrm{LDL}^{\mathrm{T}}$ decorrelation and integer search (MLAMBDA)	`src/core/filter/IlsCore.cpp`, `AmbiguityResolver` (`src/core/filter/AmbiguityResolver.cpp`)

For further reading, Strang [33] covers norms, the SVD and the pseudoinverse at the level of the course; Higham [10] covers conditioning, normal equations against QR, and scaling; Teunissen [38] treats weighted least squares, redundancy and S-bases in geodetic notation; and Odijk et al. [25] apply S-system theory to undifferenced PPP-RTK networks.

References

[1] W. Baarda. *A Testing Procedure for Use in Geodetic Networks*. Vol. 2. Publications on Geodesy, New Series 5. Delft, The Netherlands: Netherlands Geodetic Commission, 1968.

[2] Simon Banville and Richard B. Langley. "Mitigating the impact of ionospheric cycle slips in GNSS observations". In: *Journal of Geodesy* 87.2 (2013), pp. 179–193. DOI: 10.1007/s00190-012-0604-1.

[3] Jean-Paul Berrut and Lloyd N. Trefethen. "Barycentric Lagrange Interpolation". In: *SIAM Review* 46.3 (2004), pp. 501–517. DOI: 10.1137/S0036144502417715.

[4] Gerald J. Bierman. *Factorization Methods for Discrete Sequential Estimation*. New York, NY, USA: Academic Press, 1977.

[5] Guang Chen and Thomas A. Herring. "Effects of atmospheric azimuthal asymmetry on the analysis of space geodetic data". In: *Journal of Geophysical Research: Solid Earth* 102.B9 (1997), pp. 20489–20502. DOI: 10.1029/97JB01739.

[6] Shailen D. Desai. "Observing the pole tide with satellite altimetry". In: *Journal of Geophysical Research: Oceans* 107.C11 (2002), p. 3186. DOI: 10.1029/2001JC001224.

[7] D. Dong et al. "Spatiotemporal filtering using principal component analysis and Karhunen-Loeve expansion approaches for regional GPS network analysis". In: *Journal of Geophysical Research: Solid Earth* 111.B3 (2006), B03405.

[8] Jianghui Geng and Yehuda Bock. "Triple-frequency GPS precise point positioning with rapid ambiguity resolution". In: *Journal of Geodesy* 87.5 (2013), pp. 449–460. DOI: 10.1007/s00190-013-0619-2.

[9] *GLONASS Interface Control Document: Navigational Radiosignal in Bands L1, L2 (Edition 5.1)*. Russian Institute of Space Device Engineering. Moscow, 2008.

[10] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2nd ed. Philadelphia, PA, USA: SIAM, 2002. DOI: 10.1137/1.9780898718027.

[11] M. Mainul Hoque and Norbert Jakowski. "Estimate of higher order ionospheric errors in GNSS positioning". In: *GPS Solutions* 12.2 (2008), pp. 87–97. DOI: 10.1007/s10291-007-0069-5.

[12] IAU SOFA Board. *SOFA Tools for Earth Attitude*. Software version 18; http://www.iausofa.org/. International Astronomical Union, Standards of Fundamental Astronomy (SOFA). 2021.

[13] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[14] P. J. de Jonge. "A processing strategy for the application of the GPS in networks". PhD thesis. Delft, The Netherlands: Delft University of Technology, 1998.

[15] Elliott D. Kaplan and Christopher J. Hegarty, eds. *Understanding GPS/GNSS: Principles and Applications*. 3rd ed. Boston, MA, USA: Artech House, 2017.

[16] J. A. Klobuchar. "Ionospheric time-delay algorithm for single-frequency GPS users". In: *IEEE Transactions on Aerospace and Electronic Systems* AES-23.3 (1987), pp. 325–331. DOI: 10.1109/TAES.1987.310829.

[17] Jan Kouba. "A simplified yaw-attitude model for eclipsing GPS satellites". In: *GPS Solutions* 13.1 (2009), pp. 1–12. DOI: 10.1007/s10291-008-0092-1.

[18] Daniel Landskron and Johannes Boehm. “VMF3/GPT3: refined discrete and empirical troposphere mapping functions”. In: *Journal of Geodesy* 92.4 (2018), pp. 349–360. DOI: 10.1007/s00190-017-1066-2.

[19] D. Laurichesse et al. “Integer ambiguity resolution on undifferenced GPS phase measurements and its application to PPP and satellite precise orbit determination”. In: *Navigation* 56.2 (2009), pp. 135–149. DOI: 10.1002/j.2161-4296.2009.tb01750.x.

[20] F. Landis Markley. “Attitude determination using vector observations and the singular value decomposition”. In: *Journal of the Astronautical Sciences* 36.3 (1988), pp. 245–258.

[21] Jean Meeus. *Astronomical Algorithms*. Richmond, VA, USA: Willmann-Bell, 1991. ISBN: 0-943396-35-2.

[22] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Berlin, Heidelberg: Springer, 2000. DOI: 10.1007/978-3-642-58351-3.

[23] A. E. Niell. “Global mapping functions for the atmosphere delay at radio wavelengths”. In: *Journal of Geophysical Research: Solid Earth* 101.B2 (1996), pp. 3227–3246. DOI: 10.1029/95JB03048.

[24] D. Odijk. “Fast precise GPS positioning in the presence of ionospheric delays”. In: *Publications on Geodesy* 52 (2003).

[25] D. Odijk et al. “On the estimability of parameters in undifferenced, uncombined GNSS network and PPP-RTK user models by means of S-system theory”. In: *Journal of Geodesy* 90.1 (2016), pp. 15–44.

[26] Gérard Petit and Brian Luzum. *IERS Conventions (2010)*. IERS Technical Note 36. International Earth Rotation and Reference Systems Service (IERS). Frankfurt am Main: Verlag des Bundesamts für Kartographie und Geodäsie, 2010. ISBN: 3-89888-989-6.

[27] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd. Cambridge: Cambridge University Press, 2007.

[28] H. E. Rauch, F. Tung, and C. T. Striebel. “Maximum likelihood estimates of linear dynamic systems”. In: *AIAA Journal* 3.8 (1965), pp. 1445–1450. DOI: 10.2514/3.3166.

[29] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.

[30] J. Saastamoinen. “Atmospheric correction for the troposphere and stratosphere in radio ranging of satellites”. In: *The Use of Artificial Satellites for Geodesy*. Ed. by S. W. Henriksen, A. Mancini, and B. H. Chovitz. Vol. 15. Geophysical Monograph Series. Washington, D.C.: American Geophysical Union, 1972, pp. 247–251. DOI: 10.1029/GM015p0247.

[31] K. Snow. *Adjustment Computations*. Lecture notes based on courses GS 650, GS 651 and GS 762 taught by B. Schaffrin at The Ohio State University. Jan. 2026.

[32] K. Snow, F. Yang, and A. Whitacre. *A Guide to LASER: LSA Libraries and Programs, Version 2.7*. Polaris Geospatial Services. Columbus, Ohio, Aug. 2024.

[33] Gilbert Strang. *Linear Algebra and Learning from Data*. Wellesley, MA, USA: Wellesley-Cambridge Press, 2019.

[34] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[35] Tomoji Takasu and Akio Yasuda. “Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB”. In: *International Symposium on GPS/GNSS*. Jeju, Korea, 2009, pp. 4–6.

[36] P. J. G. Teunissen. “Generalized inverses, adjustment, the datum problem and S-transformations”. In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[37] P. J. G. Teunissen. “The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation”. In: *Journal of Geodesy* 70.1–2 (1995), pp. 65–82. DOI: 10.1007/BF00863419.

[38] P. J. G. Teunissen. *Adjustment Theory: An Introduction*. Series on Mathematical Geodesy and Positioning. Delft, The Netherlands: Delft University Press, 2000.

[39] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.

[40] Peter J. G. Teunissen and A. R. Amiri-Simkooei. “Least-squares variance component estimation”. In: *Journal of Geodesy* 82.2 (2008), pp. 65–82. DOI: 10.1007/s00190-007-0157-x.

[41] Grace Wahba. “A least squares estimate of satellite attitude”. In: *SIAM Review* 7.3 (1965), p. 409.

[42] J. T. Wu et al. “Effects of antenna orientation on GPS carrier phase”. In: *Manuscripta Geodaetica* 18 (1993), pp. 91–98.

[43] G. Wübbena. “Software developments for geodetic positioning with GPS using TI 4100 code and carrier measurements”. In: *Proceedings of the First International Symposium on Precise Positioning with the Global Positioning System*. Rockville, MD, USA, 1985, pp. 403–412.

[44] Thomas P. Yunck. “Coping with the atmosphere and ionosphere in precise satellite and ground positioning”. In: *Environmental Effects on Spacecraft Positioning and Trajectories*. Geophysical Monograph Series 73 (1993), pp. 1–16. DOI: 10.1029/GM073p0001.